



Hewlett Packard
Enterprise

Open System Services Programmer's Guide

Notices

The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty. Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Confidential computer software. Valid license from Hewlett Packard Enterprise required for possession, use, or copying. Consistent with FAR 12.211 and 12.212, Commercial Computer Software, Computer Software Documentation, and Technical Data for Commercial Items are licensed to the U.S. Government under vendor's standard commercial license.

Links to third-party websites take you outside the Hewlett Packard Enterprise website. Hewlett Packard Enterprise has no control over and is not responsible for information outside the Hewlett Packard Enterprise website.

Acknowledgments

Microsoft® and Windows® are either registered trademarks or trademarks of Microsoft Corporation in the United States and/or other countries.

Intel®, Itanium®, Optane™, Pentium®, Xeon®, Intel Inside®, and the Intel Inside logo are trademarks of Intel Corporation or its subsidiaries.

Adobe® and Acrobat® are trademarks of Adobe Systems Incorporated.

UNIX® is a registered trademark of The Open Group.

Java is a registered trademark of Oracle and/or its affiliates.

OpenJDK Trademark Notice

<https://openjdk.org/legal/openjdk-trademark-notice.html>

Warranty

OSF MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THE OSF MATERIAL PROVIDED HEREIN, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

OSF shall not be liable for errors contained herein or for incidental consequential damages in connection with the furnishing, performance, or use of this material.

© 1990, 1991, 1992, 1993 Open Software Foundation, Inc. This documentation and the software to which it relates are derived in part from materials supplied by the following:

© 1987, 1988, 1989 Carnegie-Mellon University. © 1989, 1990, 1991 Digital Equipment Corporation. © 1985, 1988, 1989, 1990 Encore Computer Corporation. © 1988 Free Software Foundation, Inc. © 1987, 1988, 1989, 1990, 1991 Hewlett-Packard Enterprise Company. © 1985, 1987, 1988, 1989, 1990, 1991, 1992 International Business Machines Corporation. © 1988, 1989 Massachusetts Institute of Technology. © 1988, 1989, 1990 Mentat Inc. © 1988 Microsoft Corporation. © 1987, 1988, 1989, 1990, 1991, 1992 SecureWare, Inc. © 1990, 1991 Siemens Nixdorf Informations system AG. © 1986, 1989, 1996, 1997 Sun Microsystems, Inc. © 1989, 1990, 1991 Transarc Corporation.

This software and documentation are based in part on the Fourth Berkeley Software Distribution under license from The Regents of the University of California. OSF acknowledges the following individuals and institutions for their role in its development: Kenneth C.R.C. Arnold, Gregory S. Couch, Conrad C. Huang, Ed James, Symmetric Computer Systems, Robert Elz. © 1980, 1981, 1982, 1983, 1985, 1986, 1987, 1988, 1989 Regents of the University of California.



Contents

About This Document.....	12
About TNS, TNS/R Native, TNS/E Native, and TNS/X Native Processes.....	12
About the Examples.....	12
About OSS Utilities.....	13
Supported Release Version Updates (RVUs).....	13
Intended Audience.....	13
New and changed information.....	13
Related Information.....	14
Publishing History.....	17
 Overview of the OSS Programming Environment.....	 18
The OSS API.....	19
Standards Conformance.....	19
Extensions Based on Standards or Proposed Standards.....	20
Two File Systems.....	20
The OSS File System.....	20
The Guardian File System.....	22
Two Process Types.....	24
Process Identification.....	25
Process Groups.....	26
Process Ancestors.....	27
Process Attributes.....	27
Process Environment.....	27
Process Pairs.....	28
Native and TNS Processes.....	28
Selecting Source and Target Execution Environments.....	28
The Typical Case.....	28
Special Cases of Cross Compilation.....	28
Calling Functions and Procedures in OSS Programs.....	29
OSS Functions.....	29
Guardian Procedures.....	29
Guardian C Functions.....	32
Refer to Bits From Left to Right.....	33
Using the Development Tools.....	33
Using Tools in Both Environments.....	33
OSS Tools.....	34
Guardian Tools.....	41
Using the NonStop Development Environment for Eclipse.....	43
Using the Enterprise Toolkit - NonStop Edition.....	44
Using OpenSSL.....	44
 Managing Files.....	 46
The OSS and Guardian File Systems.....	46
Physical and Logical Names.....	47
Filename and Pathname Mappings.....	47
Obtaining Corresponding Guardian and OSS Filenames.....	49

Using Guardian Procedures.....	50
OSS File-Access Permissions.....	55
Safeguard Protection and Disk Volumes.....	56
OSS ACLs.....	56
Accessible Files.....	57
Using the ANSISTREAMS Pragma (Guardian C Program).....	57
Common and Unique Characteristics.....	58
Features Common to the UNIX and OSS File Systems.....	58
Features Unique to the OSS File System.....	58
Accessing Files From the OSS API.....	58
Using Functions With Environment-Specific Parameters.....	60
Accessing Files on SMF Logical Disk Volumes.....	61
Accessing Files From the Guardian API.....	61
Guardian Procedure Extensions for OSS.....	61
Using Extended Guardian Procedures.....	65
Accessing OSS Files Larger Than 2 GB.....	68
Small Files and Large Files.....	68
Large File Safe APIs.....	69
Large File Aware APIs.....	70
Interoperability.....	71
Obtaining OSS I/O size limits.....	73
Accessing OSS SEEP-Protected Files.....	75
Common Completion.....	76
Performance Considerations.....	79
File Caching.....	79
OSS Directory Traversal.....	81
Performance and Reliability Tradeoffs.....	82
OSS File-System Functions.....	83
Guardian File-System Procedures.....	100

Managing Processes..... 107

Common and Unique Characteristics.....	107
Features Common to UNIX and OSS Process Management.....	108
Features Unique to OSS Process Management.....	115
Process Attributes.....	116
OSS Attributes in Guardian Processes.....	116
Guardian Attributes in OSS Processes.....	116
Comparing OSS and Guardian Process Attributes.....	116
Process-Management Interoperability.....	117
Signals Interoperability.....	121
Creating and Controlling Processes.....	123
Using the OSS API.....	123
Using the Guardian API.....	127
Querying the Process Environment.....	137
Using the OSS API.....	137
Using the Guardian API.....	138
Modifying the Process Environment.....	142
Using the OSS API.....	142
Using the Guardian API.....	143
Performance Considerations.....	144
Using Dynamic Servers Spread Across Available Processors.....	145
Using Static Servers Spread Across Available Processors.....	149
OSS Process-Management Functions.....	157
Guardian Process-Management Procedures.....	166

OSS Process Pairs.....	169
OSS Process Pairs Overview.....	169
OSS Process Pair Considerations.....	169
OSS Process Pair Creation.....	169
OSS Process Attributes.....	170
Process Status Display.....	170
Known Restrictions.....	171
Other Considerations.....	171
File Number Conflicts.....	171
Managing Memory.....	172
Flat, Selectable, and Extended Segments.....	172
Flat Segments.....	172
Selectable Segments.....	173
Native Processes.....	173
TNS Processes.....	173
User-Created Segments.....	173
OSS Shared Memory Segments.....	173
Guardian Memory Segments.....	174
Memory-Management Interoperability.....	174
Pool Functions.....	175
Managing Time.....	176
Common and Unique Characteristics.....	176
Features Common to UNIX and OSS Time Management.....	176
Features Unique to OSS Time Management.....	177
Time-Management Interoperability.....	177
Querying the System Clock.....	178
Querying Process Times.....	178
Limiting Processor Time Use.....	178
OSS Time-Management Functions.....	179
Guardian Time-Management Procedures.....	182
Interprocess Communication.....	185
Available Methods.....	185
Common and Unique Characteristics.....	187
Features Common to UNIX and OSS Interprocess Communication.....	187
Features Unique to OSS Interprocess Communication.....	188
Interprocess-Communication Interoperability.....	188
Using Guardian AF_INET and AF_INET6 Sockets, and OSS AF_INET and AF_INET6 Sockets	189
Using AF_UNIX Sockets	198
Using FIFOs.....	201
Using Shared Memory.....	201
Using the \$RECEIVE File.....	202
Performance Considerations.....	207
OSS Interprocess-Communication Functions.....	208
Guardian Interprocess-Communication Procedures.....	218
Managing I/O.....	223
Common and Unique Characteristics.....	223
Features Common to UNIX and OSS Terminal I/O Management.....	223



Features Unique to OSS Terminal I/O Management.....	223
I/O Interoperability.....	224
Tape I/O.....	224
Tape I/O With pax	224
Tape I/O With the Guardian API.....	225
Terminal I/O.....	226
Accessing Terminals From the OSS Environment.....	226
Blocking and Nonblocking Opens.....	226
Blocking and Nonblocking Terminal I/O.....	226
Terminal Parameters.....	227
Control Characters.....	227
Accessing Terminals From the Guardian Environment.....	228
Using OSSTTY to Redirect Input and Output to Guardian Objects.....	230
Starting OSSTTY.....	231
Examples of OSSTTY Usage.....	233
Printer I/O.....	242
Printing With the OSS API.....	242
Printing OSS Files With the Guardian API.....	244
OSS I/O Management Functions.....	248
Guardian I/O Management Procedures.....	250

Using Logging Mechanisms.....252

Common and Unique Characteristics.....	252
Features Common to UNIX and OSS Logging.....	252
Features Unique to OSS Logging.....	253
Logging Interoperability.....	253
How to Log Information With the OSS API.....	253
Specifying a Collector	255
Specifying the Severity Level of Logged Events.....	255
Initializing Logging Parameters.....	256
Logging a Message.....	258
Closing a Collector	259
Logging Parameters and EMS Tokens.....	259
OSS Logging-Mechanism Functions.....	260

Using Subsystems From OSS.....262

Communications Subsystems.....	262
Subsystems Supporting the Subsystem Programmatic Interface (SPI).....	262
NonStop SQL/MP.....	263
NonStop SQL/MX.....	263
NonStop TS/MP.....	264
NonStop TMF.....	264
NonStop Tuxedo System.....	264
NonStop Distributed Computing Environment (DCE).....	265
Simple Network Management Protocol (SNMP).....	265

Managing OSS Security.....266

Common and Unique Characteristics.....	266
Functions Available for Security Management.....	267
Security Auditing of OSS files.....	271
How to Identify an Audited file in Audit Records.....	271
What Operations and Objects Are Audited.....	271



Considerations for File Auditing.....	275
Example of Managing file Ownership.....	276
Using OSS Access Control Lists (ACLs).....	279
Definitions.....	279
Access Control List Entries.....	280
ACL Notation.....	281
Types of ACL Entries.....	281
Optional ACL Entries.....	282
ACL Uniqueness.....	283
ACL Inheritance.....	283
Access Check Algorithm.....	286
ACL Operations Supported.....	287
ACL Interaction with <code>stat ()</code>	287
ACL Interaction with <code>chmod ()</code>	287
ACL Interaction with <code>chown ()</code>	287
OSS Network File System (NFS) and ACLs.....	287
Header file.....	288
Example Program For ACLs.....	288
Restricted-Access Filesets and File Privileges.....	291
Restricted-Access Filesets.....	291
File Privilege File Attribute.....	292
OSS SEEP Programming.....	294
OSS SEEP Management.....	294
OSS SEEP Consultation.....	295
OSS SEEP System and Library Calls.....	297
Final Result of the Operation.....	298
OSS SEEP Interprocess-Communication Messages.....	300
OSS SEEP Design.....	306
OSS SEEP-Related EMS Events.....	307

64-Bit Support in OSS and Guardian..... 308

Features of 64-Bit Support.....	308
64-Bit OSS Processes.....	309
Process Type Attribute.....	310
New <code>PROCESS_GETINFOLIST_</code> Attributes.....	310
New <code>PROCESSOR_GETINFOLIST_</code> Attribute.....	312
64-Bit Heap and 32-Bit Heap for 64-Bit OSS Processes.....	312
ILP32 and LP64 Data Models.....	313
Sharing Memory Between 32-Bit and 64-Bit Processes.....	314
C/C++ Language Features Supporting 64-Bit Addressing.....	315
32-Bit and 64-Bit Applications Constraints.....	319
64-Bit Support for OSS and Guardian APIs.....	319
Changed OSS Functions.....	319
32-Bit and 64-Bit Public DLLs.....	320
Neutral Public DLLs.....	321
SQL/MX.....	321
New OSS Function in <code>YPUTDLL</code>	321
New Guardian Procedures and OSS Functions.....	321
Guardian Procedures Callable from 64-Bit OSS Processes.....	323
OSS Functions Callable from 32-bit Processes Using 64-bit Pointers.....	328
32-Bit and 64-Bit Object Files.....	328
Linker Object Files.....	329
Data Models for Object Files.....	329
Mixed Data-Model Programming.....	330

New Pointer Modifiers.....	331
New pragma Directives.....	331
Mixed Data-Model Prototypes in cextdecs.h.....	331
Mixed Data-Model Programming Example.....	332
New C/C++ builtins.....	333
New OSS String Manipulation Functions.....	333
File and Socket I/O APIs 32-Bit Support for 64-Bit Segments.....	334
64-Bit Data Types and Data Structures Changes.....	335
64-Bit Data Types Changes.....	335
64-Bit Data Structures Changes and Additions.....	337
New 64-Bit Data Structures for Process Creation.....	338
Converting 32-Bit Applications to 64-Bit Applications.....	338
Step 1: Identify Programs that Need to be Compiled in the LP64 Data Model.....	339
Step 2: Identify Non-Portable Constructs.....	339
Step 3: Using the C/C++ Compiler to Identify Needed Source Code Changes.....	339
Step 4: Compile in the LP64 Data Model.....	342
64-Bit TNS/E and TNS/X Debugger Support.....	342
Tool Control Language (TCL) Support.....	343
Snapshot Support.....	343
eInspect Support.....	344
eGarth Support.....	344
Visual Inspect Support.....	344

Using the Standard POSIX Threads Library..... 345

Standard POSIX Threads.....	345
Compiling and Linking Considerations.....	346
Error Reporting Changed From Previous Standard.....	347
Attributes Defined by Threads Macros.....	347
Signal Handling.....	348
Thread-Aware Signal Handling.....	350
Threads Portability.....	358
Numerical Limits in Standard POSIX Threads.....	359
Keeping Application Design Simple.....	360
Threading Considerations.....	360
Sharing Process Resources.....	360
Using Standard Libraries.....	360
Addressing the Thread Stack.....	361
Making Process-Blocking Calls.....	361
Using Nowait Input or Output.....	362
Yielding Control.....	362
Using Signals.....	362
Spawning a New Process.....	363
Isolating Faults.....	363
Single Versus Multiple Application Threads.....	363
Programs With a Single Application Thread.....	364
Programs With Multiple Independent Application Threads.....	364
Programs With Multiple Interdependent Application Threads.....	365
Jacket Routines.....	365
SERVERCLASS_SEND and SERVERCLASS_SENDL Jacket Routines.....	365
SERVERCLASS_DIALOG Jacket Routines.....	366
TMF Transaction Jacket Routines.....	366
Reference Pages for Thread Functions.....	367
Threaded Application Examples.....	367
Thread-Aware and Nonblocking OSS Functions.....	372

Regular Files.....	373
Non-Regular Files.....	374
Equivalent System, Thread-Aware, and Nonblocking Thread-Aware Functions.....	375
Preprocessor Directives Automatically Map Standard System Calls to Thread-Aware or Nonblocking Thread-Aware System Calls.....	378
Using Thread-Aware Functions.....	381
Reentrant OSS Functions.....	401
Example Program Using Reentrant Functions.....	402
Thread-Aware Toolkit Functions and Types.....	404
Using Thread-Aware Toolkit Functions.....	405
Caveats to Using the Thread-Aware Toolkit	408
Thread-Aware \$RECEIVE I/O Functions.....	408
Using Thread-Aware \$RECEIVE Functions.....	409
Caveats to Using Thread-Aware \$RECEIVE.....	411
SQL/MX Considerations.....	411
Thread-Aware Enscribe Functions.....	411
Using Thread-Aware Enscribe Functions.....	413
OSS File Descriptor and Timer Callbacks.....	414
Using OSS and Timer Callbacks.....	415
Caveat to Using Callbacks.....	415

Using the POSIX User Thread (PUT) Model Library 416

Compiling and Linking Considerations.....	416
Header Files.....	416
Linking to the POSIX User Thread Model Library.....	416
Defines.....	416
Environment Variables.....	417
Standards Conformance.....	417
Error Reporting Changed From Draft 4 Standard.....	417
Exceptions and Extensions to the Standard Document.....	418
Attributes Defined by Threads Macros.....	418
Process Resources.....	419
Numerical Limits in the POSIX User Thread Model Library.....	420
Protected Stack Limits.....	420
Limits Imposed by the Standard.....	421
Signals and Signal Handling.....	423
Types of Signals.....	423
Using Signals.....	425
Thread-Aware Signal Handling.....	427
Thread-Aware and Nonblocking OSS Functions.....	433
Regular Files.....	434
Non-Regular Files.....	434
Equivalent System, Thread-Aware, and Nonblocking Thread-Aware Functions.....	435
The Set of Thread-Aware I/O Functions Must Be Used Together.....	437
Using Thread-Aware Functions.....	438
Thread-Aware User Events.....	445
User Event API usage.....	445
Threads Portability.....	448
Thread Safety.....	448
Functions That Are Not Thread Safe.....	449
Cancellation Points.....	450
Potential Cancellation Points.....	451
Reentrant OSS Functions.....	453
Example Program Using Reentrant Functions.....	454

Thread-Aware Toolkit Functions and Types.....	455
Using Thread-Aware Toolkit Functions.....	457
Caveats to Using the Thread-Aware Toolkit	459
Thread-Aware \$RECEIVE I/O Functions.....	459
Using Thread-Aware \$RECEIVE Functions.....	460
Caveats to Using Thread-Aware \$RECEIVE.....	462
Jacket Routines (Nonblocking Versions of SERVERCLASS_ and TMF System Calls).....	462
SERVERCLASS_SEND and SERVERCLASS_SENDL Jacket Routines.....	462
SERVERCLASS_DIALOG Jacket Routines.....	463
TMF Transaction Jacket Routines.....	463
Thread-Aware Enscribe Functions.....	464
Using Thread-Aware Enscribe Functions.....	466
OSS File Descriptor and Timer Callbacks.....	468
Using OSS and Timer Callbacks.....	469
Caveat to Using Callbacks.....	469
Converting Applications that use the SPT Library.....	470
Build Scripts.....	470
Header Files.....	470
Standard and Nonstandard Pthread Types.....	471
Global Variables.....	472
Concurrency Level Defaults.....	472
Differences in Functions and Function Behavior.....	472
Threaded Application Programming.....	475
Keeping Application Design Simple.....	475
Threading Considerations.....	475
Single Versus Multiple Application Threads.....	477
Threaded Application Examples.....	479
64-Bit Considerations for the PUT Model Library.....	484
Thread Local Storage Usage.....	484

Websites..... 485

Support and other resources..... 486

Accessing Hewlett Packard Enterprise Support.....	486
Accessing updates.....	486
Remote support.....	487
Customer self repair.....	487
Warranty information.....	487
Regulatory information.....	487
Documentation feedback.....	488

Documented OSS Functions..... 489

API Interoperability in TNS Processes..... 553

Calling Functions and Macros.....	553
Environment of the Module.....	553
Environment of the Process.....	553
Function and Macro Behavior.....	554
Using the API Interoperability Table.....	554
G and O Notation.....	555
Specified By.....	555

Defining Parameters and Objects.....	555
Usage Notes.....	556
Examples.....	557
API Interoperability in Native Processes.....	589
Comparing API Interoperability in TNS and Native Processes.....	589
Calling Functions and Macros.....	589
Environment of the Module.....	590
Environment of the Process.....	590
Function and Macro Behavior.....	590
Internationalization Functions.....	591
Using the API Interoperability Tables.....	591
G and O Notation.....	591
Specified By.....	591
Defining Parameters and Objects.....	592
Usage Notes.....	592
Examples.....	596
Glossary.....	704

About This Document

The *Open System Services Programmer's Guide* describes how to write software applications for the OSS environment. Emphasis is on the C and C++ programming languages. This guide includes information about:

- How the OSS environment differs from the standard UNIX programming environment
- When you need to use the Guardian application program interface (API) in OSS C programs
- Which tasks can be performed only from the Guardian or OSS API
- Performance considerations
- 64-bit support, beginning with the H06.24 and J06.13 RVUs

The basics of how to program in a UNIX environment are well documented in commercial texts and are not an objective of this guide. Please refer to the following books for information about programming in a UNIX environment.

- *Advanced Programming in the UNIX Environment*, W. Richard Stevens, Addison-Wesley, 1992.
- *Multithreaded Programming with Pthreads*, Bill Lewis and Daniel J. Berg, Sun Microsystems Press, 1998.
- *POSIX Programmer's Guide*, Donald Lewine, O'Reilly and Associates, Inc., 1991.
- *Practical UNIX & Internet Security*, Simon Garfinkel and Gene Spafford, O'Reilly and Associates, Inc., 1996.
- *Pthreads Programming*, Bradford Nichols, Dick Buttlar, and Jacqueline Proulx Farrell, O'Reilly and Associates, Inc., 1996.
- *UNIX Network Programming*, 2nd Edition, W. Richard Stevens, Prentice-Hall, 1997.

About TNS, TNS/R Native, TNS/E Native, and TNS/X Native Processes

A TNS process is initiated by executing a TNS interpreted or accelerated object file; TNS object code executes on TNS, TNS/R, TNS/E, and TNS/X systems. A TNS/R native process is initiated by executing a TNS/R native object file; TNS/R native object code executes only on TNS/R systems. A TNS/E native process is initiated by executing a TNS/E native object file; TNS/E native object code executes only on TNS/E systems. A TNS/X native process is initiated by executing a TNS/X native object file; TNS/X native object code executes only on TNS/X systems.

In this guide, the term “native process” refers generically to TNS/R, TNS/E, and TNS/X native processes. When it is necessary to distinguish between platforms, the terms “TNS/R native process”, “TNS/E native process”, and “TNS/X native process” are used. In most cases, the behavior of TNS and native processes is the same. When it is not, the guide provides considerations or notes about how TNS process behavior differs from that of native processes.

About the Examples

This guide contains programming examples in C to illustrate how programming in the OSS environment differs from programming in a standard UNIX environment.

The examples are designed to focus on a particular function or programming construct. They are intentionally short and to-the-point to avoid distracting from the example's primary focus.

The effort to keep the examples concise means, however, that the programming examples are not models of how an experienced programmer would verify user input. Verifying user input can require a lot of code, and this guide assumes you are an experienced programmer who knows how to edit user input.



About OSS Utilities

Hewlett Packard Enterprise is not responsible for the proper functioning of unsupported utilities or facilities and will not respond to product reports about them. Such utilities and facilities include those in the OSS /bin/unsupported directory. Use these utilities and facilities at your own risk.

Supported Release Version Updates (RVUs)

This manual supports L15.02 and later L-series RVUs, J06.03 and later J-series RVUs, H06.03 and later H-series RVUs, and G06.15 and later G-series RVUs until otherwise indicated in a replacement publication.

Intended Audience

This guide is intended for intermediate and advanced C application programmers who write programs for the OSS environment. Experience with programming in a UNIX environment is assumed.

New and changed information

Changes to the 10-180013-Q422 manual

Updated the following sections:

- **Memory-Management Interoperability**
- **Available Methods**
- **OSS Interprocess-Communication Functions**

Added a new topic

SQL/MX Considerations

Changes to the 862337-008a manual

Added information about **OpenSSL**.

Changes to the 862337-008 manual

- Updated the section **Documented OSS Functions**.
- Updated the section **Thread Local Storage Usage**.

Changes to the 862337-007 manual

Updated the section **SERVERCLASS_DIALOG Jacket Routines**.

Changes to the 862337-006 manual

- Added J06.23 applicability content for the section **Obtaining OSS I/O size limits**.
- Following functions are also updated with the J06.23 applicability content:

```
faccessat() fchdir() fchmodat() fchownat() fdopendir() fstatat()
fstatat64() linkat() mkdirat() mkfifoat() mknodat() openat()
```



```
openat64()  pread()  pread64_()  pwrite()  pwrite64_()  readlinkat()
renameat()  symlinkat()
```

For the preceded functions the following sections were updated with the applicability content:

- **Table 37: OSS Function Calls Audited When Used With Audited Filesets**
- **Table 70: System and Equivalent Thread-Aware Functions**
- **Documented OSS Functions**
- Modified the declaration of environ from `extern char *environ[]` to `extern char **environ` in the sections **Using Dynamic Servers Spread Across Available Processors** and **Using Static Servers Spread Across Available Processors**.

Changes to the 862337-005 manual

This edition of the manual is updated with the following:

- Added a new section **Obtaining OSS I/O size limits**.
- Updated `PUT_PROTECTED_STACK_DISABLE` description in the section **Environment Variables**.
- Added a new consideration related to `PUT_TMF_JOIN` in the section **TMF Transaction Jacket Routines**.

Related Information

NOTE: The OSS and Guardian documents are available from the NonStop Technical Library (NTL). The NTL is located at <http://www.hpe.com/info/nonstop-docs> (external view).

The following table shows the programming guides, reference manuals, and user guides that contain information related to the information in this guide.



Table 1: Related Manuals

Programming Guides	C/C++ Programmer's Guide	OSS Programmer's Guide	Software International-ization Guide
	Guardian Programmer's Guide	Availability Guide for Application Design	OSS Porting Guide
Reference Manuals	OSS Library Calls Reference Manual	OSS System Calls Reference Manual	OSS Shell and Utilities Reference Manual
	Guardian Procedure Calls Reference Manual	Guardian Native C Library Calls Reference Manual	Guardian TNS C Library Calls Reference Manual
User Guides	Guardian User's Guide	OSS User's Guide	

VST002 VSD

The following are related programming guides:

- *Availability Guide for Application Design*. This manual introduces the various products and concepts involved in designing and writing applications with increased availability. It describes alternative approaches to application design

and the benefits and compromises of different approaches, and it includes check lists on how to go about developing more available applications.

- *C/C++ Programmer's Guide*. This guide describes the implementation of the C and C++ programming languages on NonStop systems. It discusses the C language syntax, the C compiler, and the C program execution environment. This guide is intended for system and application programmers writing programs for the Guardian and OSS environments.
- *Guardian Programmer's Guide*. This guide describes how to access the Guardian API using Guardian procedure calls. It is written for application and system programmers.
- *Open System Services Porting Guide*. This guide describes conversion techniques for moving existing C-language applications into the OSS environment. It discusses both C-language and operating system considerations. This guide is intended for application programmers, with some information for selected operating system users.
- *Software Internationalization Guide*. This guide describes how to use the OSS internationalization features. It is written for application and system programmers.

The following are related reference manuals:

- *Guardian Procedure Calls Reference Manual*. This manual describes the syntax for most Guardian procedure calls. It is intended for system and application programmers.
- *Guardian Native C Library Calls Reference Manual*. This manual describes syntax and semantics of the native C run-time library calls and header files for the Guardian environment. This manual is intended for system and application programmers.
- *Guardian TNS C Library Calls Reference Manual*. This manual describes syntax and semantics of the TNS C run-time library calls and header files for the Guardian environment. This manual is intended for system and application programmers.
- . This manual describes syntax and semantics of the C run-time library calls and header files for the OSS environment. This manual is intended for system and *Open System Services Library Calls Reference Manual* application programmers.
- *Open System Services Shell and Utilities Reference Manual*. This manual presents rules and syntax for each command that the user enters interactively to access the OSS command interpreter (called the *shell*), and utilities and other functions that perform general-purpose and program-development operations. It is written for all audiences: system administrators, managers, and operators; end users; and application and system programmers.
- *Open System Services System Calls Reference Manual*. This manual describes all OSS system calls (file system and kernel functions), their syntax, required external data structures, resulting operations, and source file (header) locations for all literals and symbolic definitions. It is written for application and system programmers.

The following are related user guides:

- *Guardian User's Guide*. This guide provides basic information about the programs and utilities that are used most often in the Guardian environment by general system and application users. It also provides more detailed procedures for system operations, management, and maintenance tasks that are useful to both beginning and experienced users of NonStop systems.
- *Open System Services User's Guide*. This guide provides basic orientation, tutorial, and task information for using the OSS shell and utilities. It is written for all audiences: system administrators, managers, and operators; end users; and application and system programmers.



Publishing History

Part Number	Published
10-180013-Q422	September 2022
862337-008a	December 2020
862337-008	October 2020
862337-007	May 2020
862337-006	May 2019



Overview of the OSS Programming Environment

OSS is an alternative to the Guardian interface through which users and programs can interact with the NonStop operating system. OSS interfaces and functions greatly resemble those of UNIX systems in general, because of their common relationship to the POSIX standards. OSS adds to the UNIX API by offering access to Guardian fundamentals and interoperability with the Guardian environment by offering extensions to the standard API, additional OSS function calls, and access to Guardian procedure calls.

OSS provides:

- A basis for middleware products. OSS supports a variety of middleware software, such as the Tuxedo, Distributed Object Manager/MP (DOM), and Server for Java products, that might also be used in an application.
- The Perl, PHP, and Python scripting languages.

¹

- An open interface to the operating system for supporting portable applications.

This guide focuses on the latter function: writing applications in C for the OSS target execution environment without the use of middleware. For your convenience, **Using Subsystems From OSS** contains a brief overview of some of the middleware available for NonStop systems. For more information on such middleware, refer to the manual set for the specific software product in which you are interested.

The OSS programming environment allows you to access Guardian objects and subsystems as well as OSS objects. OSS accomplishes this interoperability between environments using standard library and system function calls, and Hewlett Packard Enterprise extensions to these calls and to Guardian procedures.

This section discusses the following topics:

Topic	Content
<u>The OSS API</u>	Relationship to the Guardian API and the NonStop operating system, and conformance to industry standards.
<u>Two File Systems</u>	Basic differences between the OSS and Guardian file systems, including directory structures, file access, and file types.
<u>Two Process Types</u>	Basic differences between the OSS and Guardian processes, including process identifiers, groups, ancestors, attributes, and pairs.
<u>Selecting Source and Target Execution Environments</u>	Typical source and target execution environments and exceptions to the typical case.
<u>Calling Functions and Procedures in OSS Programs</u>	How to call OSS functions and Guardian functions and procedures and when to use them.

Table Continued

¹ For detailed information about these scripting languages in the OSS Script Languages (T1203) product, see the *Script Languages User Guide*.



Topic	Content
<u>Using the Development Tools</u>	Available editors, development utilities, compilers, linkers, optimizers, debuggers, performance analyzers, and porting kit.
<u>Using the Enterprise Toolkit - NonStop Edition</u>	An integrated development environment that enables you to build applications on a PC.

The OSS API

An application program interface (API) is a set of functions or procedure calls that permits user programs to communicate with the operating system. The OSS API coexists on NonStop systems with the traditional Guardian interfaces, as shown in the following figure.

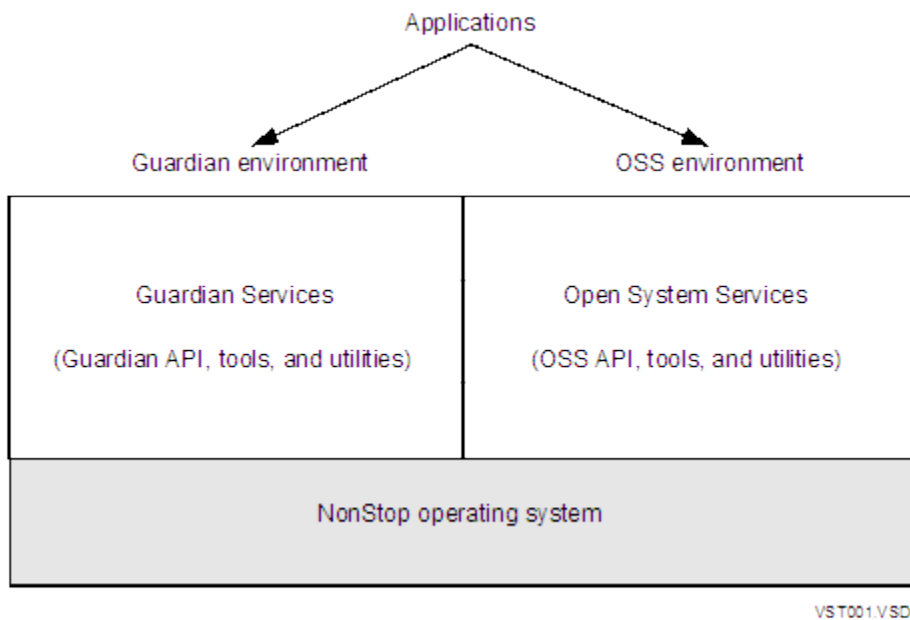


Figure 1: The Operating Environment for NonStop Systems

An application program can be compiled to run in either the OSS or Guardian environment and can use services from each environment, including the API, tools, and utilities.

Standards Conformance

The OSS API conforms to the POSIX.1 and POSIX.2 standards, and partially conforms to the XPG4 specifications. These standards and specifications are described in the following documents:

- *Information Technology – Portable Operating System Interface (POSIX) – Part 1: System Application Program Interface (API) [C Language]*, ISO/IEC 9945-1 (IEEE Std 1003.1, SH13680), First Edition 1990.
- *Information Technology – Portable Operating System Interface (POSIX) – Part 2: Shell and Utilities*, ISO/IEC 9945-2 (IEEE Std 1003.2a), First edition 1992.
- *X/Open CAE Specifications – System Interfaces and Headers, Issue 4.*
- *X/Open CAE Specifications – System Interfaces and Headers, Issue 4, Version 2.*

Conformance to POSIX.1

You might need to write applications that strictly conform to a certain standard or specification. A strictly-conforming POSIX.1 application cannot use any of the following:

- Any function unique to a NonStop system, such as `tdm_fork()`, that is an extension to the standard.
- The Guardian environment. This is not discussed by the standard.
- A Guardian object. This is not discussed by the standard.
- A remotely mounted fileset. This is not discussed by the standard.
- A symbolic link. This is not discussed by the current version of the standard.
- A POSIX.2 function, such as those provided in the `regex.h` and `glob.h` header files.
- An XPG4 function, such as those provided in the `fnmatch.h`, `nl_types.h`, and `wchar.h` header files.
- An XPG4 function that is an extension to standard header files, such as `getw()` in `stdio.h`.
- Any function related to OSS access control lists (ACLs)
- Any function designed to interact with OSS files that are larger than approximately 2 gigabytes. These functions, often called 64-bit APIs or 64-bit functions, typically have the number 64 in the name (for example the `open64()` function).
- Functions specified only in the ISO/IEC 9899:1999 standard, which is also referred to as the C99 standard.

Extensions Based on Standards or Proposed Standards

The OSS API also includes functions that are Hewlett Packard Enterprise extensions to the XPG4 specifications, but that are also based on other approved or proposed standards:

- Functions designed to interact with OSS files larger than approximately 2 gigabytes in addition to smaller OSS files. These functions, often called 64-bit APIs or 64-bit functions, typically have the number 64 in the name (for example the `open64()` function). These 64-bit APIs are based on the *Adding Support for Arbitrary File Sizes to the Single Unix Specification*, X/Open Large File Summit (LFS) submission, version 1.5 (March 20, 1996), and are supported on systems running J-series RVUs, H06.06 or later H-series RVUs, and G06.29 or later G-series RVUs.
- Functions that support OSS access control lists (ACLs). These functions are based on the HP-UX implementation and *Information Technology – Portable Operating System Interface (POSIX) – Part 1: System Application Program Interface (API) Amendment #: Protection, Audit, and Control Interfaces [C Language]* (IEEE Draft P1003.1e), October 1997.
- Several functions that support wide-character formats. These functions conform to the ISO/IEC 9899:1999 standard.

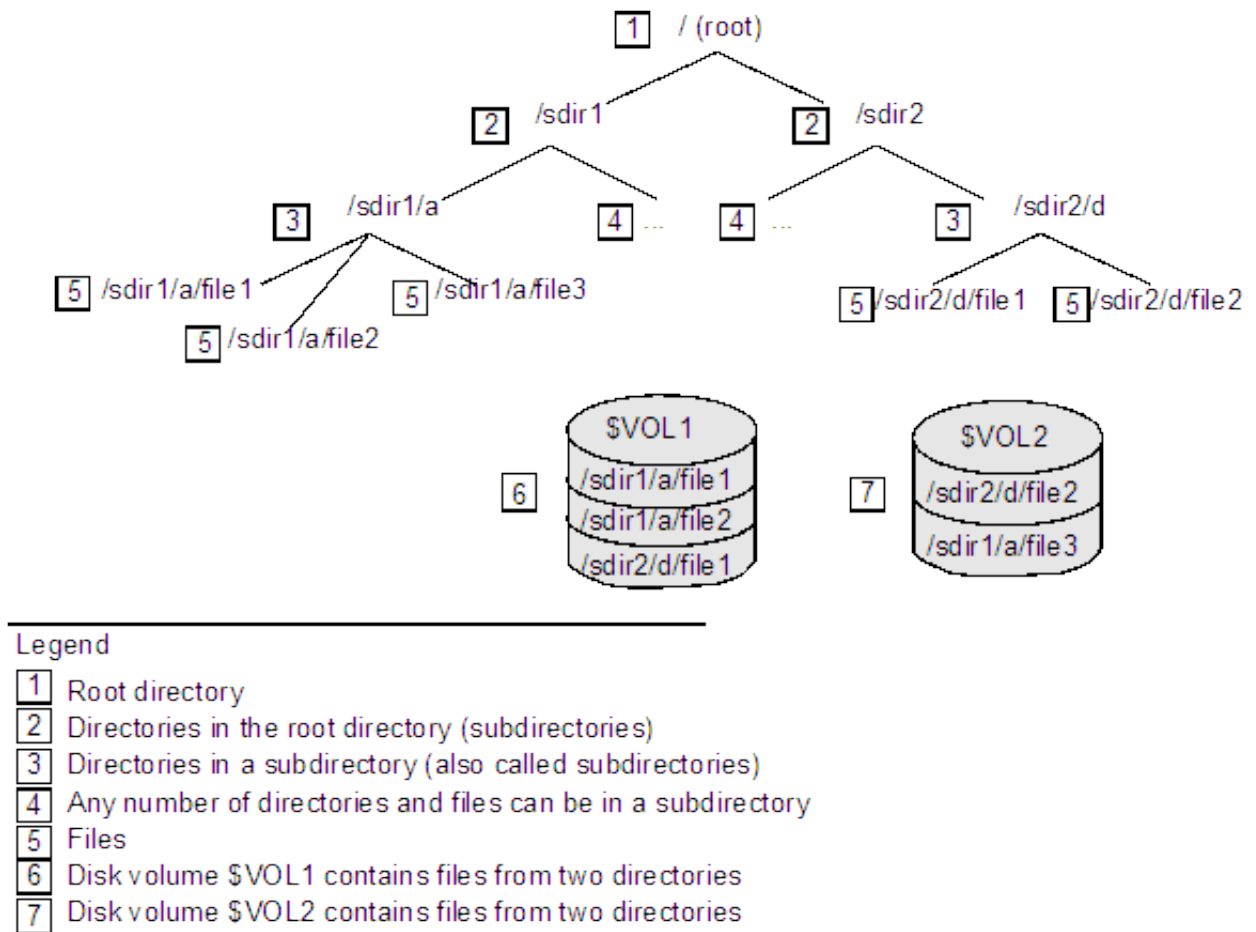
Two File Systems

The OSS environment enables you to access two file systems: the OSS file system and the Guardian file system. The files in these file systems can exist as local files or remote files. A local file resides on the local node. A remote file resides on a remote node.

The OSS File System

The OSS file system is a multilevel file system consisting of directories and subdirectories stemming from a “root” directory denoted by `/`. A directory can contain files and more directories. A directory that is in another directory is called a subdirectory. There are no theoretical limits to the number of subdirectories and files in a directory. The following figure shows the structure of an OSS file system and that there is no direct relationship between OSS filenames and disks.





VST003.VSD

Figure 2: OSS Filenames and Disks

Open System Services manages the directory tree using filesets. A fileset is a directory tree that can be mounted or unmounted. Most literature about UNIX refers to filesets as file systems. In the OSS and Guardian environments, the term file system also refers to the software that manages the file system.

OSS files and directories are designated by pathnames. A pathname is the string of characters that uniquely identifies a file within its file system. A pathname can be either relative or absolute. An absolute pathname starts with a slash (/) and begins at the root directory. A relative pathname does not start with a slash and begins with the current working directory. The `PATH_MAX` symbolic constant in the `limits.h` header file defines how long OSS pathnames can be.

A directory or fileset can occupy many disk volumes. Pathnames are logical names. They have no relationship to disk volumes, as shown in the figure. A file cannot occupy more than one disk volume. Refer to the `pathname(5)` reference page either online or in the *Open System Services System Calls Reference Manual*.

File Types

Open System Services supports these file types:

- Regular files (disk files)
- Directories
- FIFOs/pipes
- Sockets

- Symbolic links
- Character special files (Only /dev/tty terminal character files and /dev/null)

All OSS regular (data) files are odd-unstructured files. OSS regular files can be accessed using Guardian functions and procedures as well as OSS functions.

File Access

Access to an OSS file or directory is determined by a set of permission bits associated with it. The file owner, groups the owner belongs to, and others each have read, write, or execute permission for a file.

For filesets that support OSS ACLs, access to an OSS file or directory is determined by the ACL. For more information about OSS ACLs, see [Using OSS Access Control Lists \(ACLs\)](#).

Access to Files on Remote NonStop Nodes

Any file visible to the OSS filesystem on a different node in an Expand network can be accessed by preceding the file name of the name of a special directory. This special directory, which exists just under the root directory, is called /E. The OSS pathname begins with /E, followed by the node name and the absolute pathname of the file on the remote node. The directory is treated as any another directory, and it includes all other nodes in the Expand network that support Open System Services. (Guardian files are also accessible on those other nodes). Here is an example, of a pathname for an OSS file named thatfile on a remote node named \COMM:

/E/comm/thatdir/thatfile

The pathnames of files on the local node also exist in /E as a special case. The directory /E/localnode/ is a symbolic link to the local root directory /.

The Guardian File System

The Guardian file system consists of a limited number of levels. A complete (fully qualified) Guardian file name has four components: a node name, volume name, subvolume name, and file identifier. A node name begins with a backslash (\), and a volume name begins with a dollar sign (\$). Components are separated by a period (.). The following figure shows the components of the fully qualified file name \mynode.\$myvol.Mysubvol.myfile.

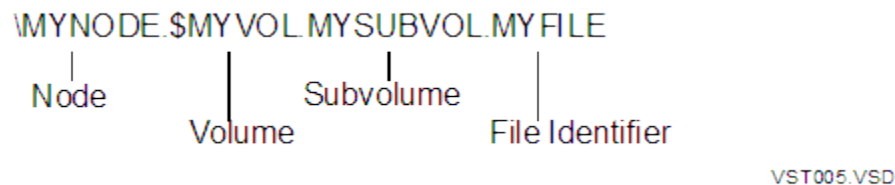


Figure 3: Components of Guardian File Names

Guardian file names visible to an OSS program are physical names: they refer to physical objects. A node name refers to a physical node, and a volume name refers to a physical disk volume. A node cannot contain nodes, a volume cannot contain volumes, and a subvolume cannot contain subvolumes. The relationship between Guardian file names and physical devices is shown in the following figure.

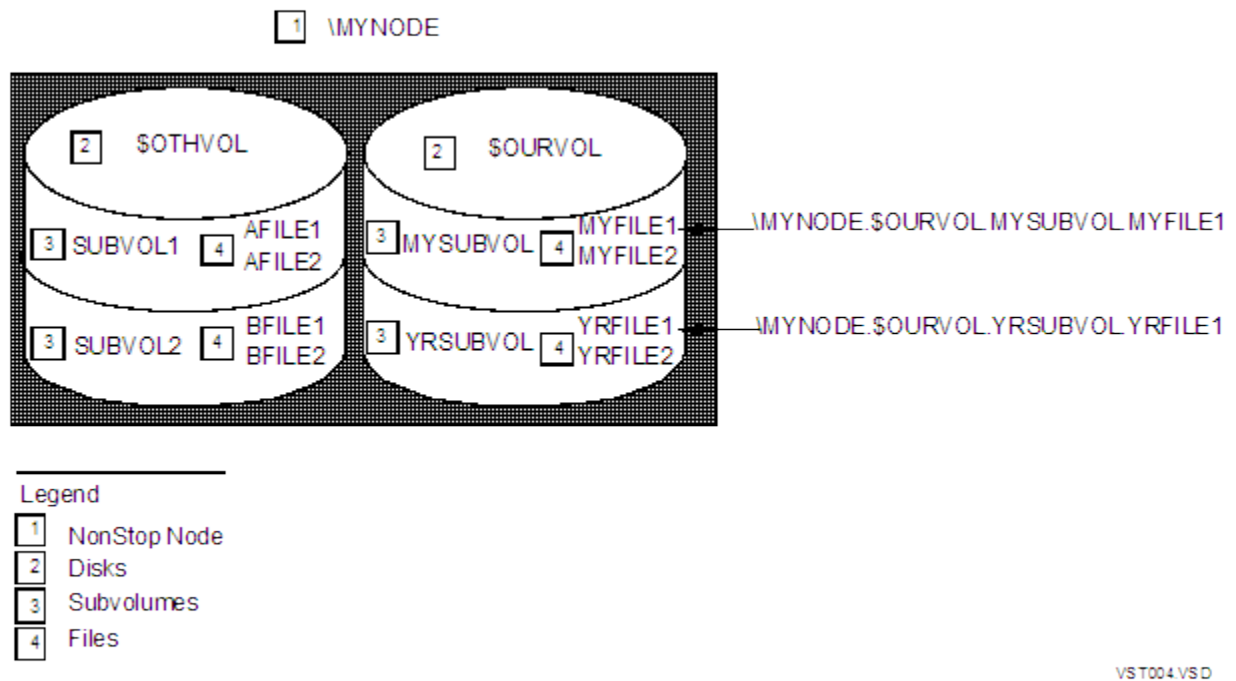


Figure 4: Guardian File Names and Disks

Guardian files on the local node appear as OSS files in the /G directory of the OSS file system. The local node name is omitted.

Guardian files on remote NonStop nodes also appear as OSS files in the /E directory of the OSS file system. The pathnames for such files begin with /E/nodename and continue with the /G directory on the indicated node. For example:

/E/node2/G/volume/subvol/fileid

The following figure shows the Guardian file /G/volume1/subvol1/myfile in the OSS file system.

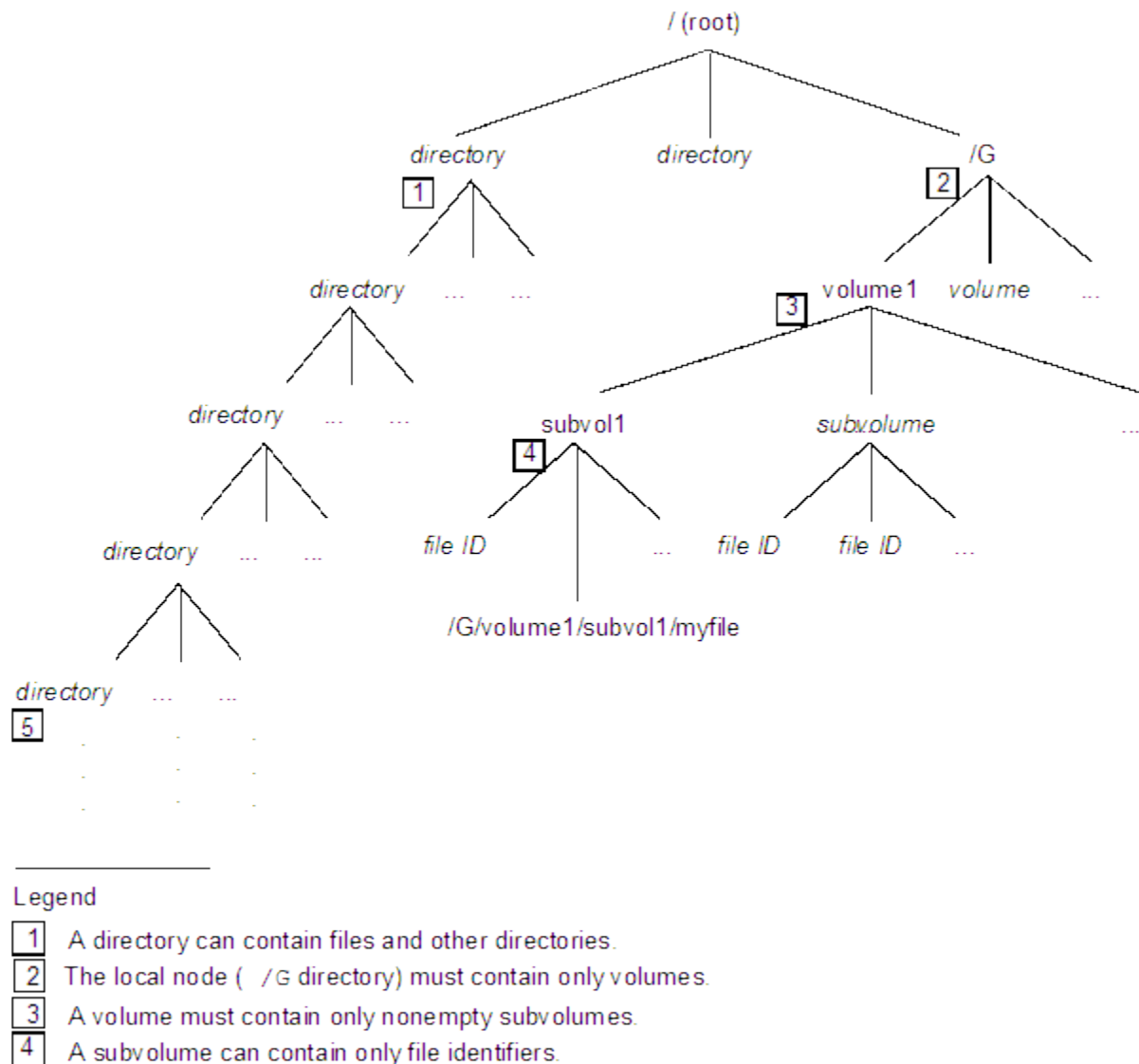


Figure 5: Guardian Files in the OSS File System

Using OSS functions, you can read and write only odd-unstructured Guardian files. EDIT files are unstructured files, and you can access them only for reading. OSS functions cannot access structured Guardian files (SQL, entry-sequenced, key-sequenced, and relative). You can access structured Guardian files using Guardian procedure calls in an OSS program.

Guardian file access is determined by the RWEp character string. This is a four-character string; the positions determine who can Read, Write, Execute, or Purge a file (hence RWEp). For further information about Guardian file security, refer to the *Guardian User's Guide*.

You can create and manage both OSS and Guardian processes in the OSS environment. OSS and Guardian processes share some process identifiers and attributes. OSS processes can belong to process groups, while Guardian processes

cannot. Guardian processes can belong to process pairs, but OSS process pairs are supported only on J06.14 and later J-series RVUs and H06.25 and later H-series RVUs. (For details of OSS process pairs, see **OSS Process Pairs**.)

Creator processes and the process run-time environment are defined differently for the two types of processes. In the OSS environment, methods for creating a new process are:

- `fork()` creates a new OSS process, running the same program image as the calling process, in the same CPU. `tdm_fork()` is a variant of `fork()` that permits access to additional process attributes.
- `exec...()` associates a new program image with the current OSS process. Functions in the `exec...()` family of functions are: `execl()`, `execle()`, `execlp()`, `execv()`, `execve()`, `execvp()`, `tdm_execve()`, `tdm_execvep()`, and `fexecve()`.
- `fork() + exec()` is the traditional way to create a new OSS process running a different program image.
- `tdm_spawn()` is an alternative approach that effectively performs both the `fork()` and the `exec()`. (That is, creates a new process and associates it with a new program image.)
- `PROCESS_SPAWN[64]_` is another alternative; it is a Guardian procedure that creates an OSS process with Guardian ancestry.

Process Identification

The following table shows the ways in which OSS processes and Guardian processes can be identified. Processes always have “required” identifiers, never have identifiers that are “not available,” and might have those classified as “optional.” You can specify optional identifiers when you call process-creation procedures or functions.

Table 2: OSS and Guardian Process Identifiers

Process Identifier	OSS Process	Guardian Process
OSS Process ID	Required	Not available
Process file name	Required	Required
Process handle	Required	Required
Process name	Optional	Optional

The following list defines the OSS and Guardian process identifiers:

OSS Process ID

Uniquely identifies an OSS process within a node.

Process file name

Uniquely identifies an OSS or Guardian process within a NonStop network, in printable string format.

Process handle

Uniquely identifies an OSS or Guardian process within a NonStop network, in internal format.

Process name

Uniquely identifies a named OSS or Guardian process within a NonStop system.

Though every process has a process file name and a process handle, their explicit use in APIs is not always necessary. The process handle is typically used with Guardian procedures that query or modify process state; the process file name is



typically used to open a process for inter-process communication via the Guardian file system. Process file name, process handle, and process name are Guardian concepts and seldom used for OSS processes.

Process Groups

An OSS process can belong to a process group, which is identified by a group ID. Processes in the same group can be treated collectively; for example, all processes in a group that are active at the same time can receive the same job control signals. Therefore, when you enter a command from the OSS shell to suspend a job, all processes in the group associated with that job can receive the signal to stop. (Signals can be sent to an individual process or to an entire process group.)

A Guardian process cannot belong to a process group or session, which is a collection of one or more process groups.

Using Some OSS Group Information Functions provides code to demonstrate the use of some of the OSS group information functions including `getgrent()`, `endgrent()`, `setgrent()`, `getgrgid()`, and `getgrnam()`.

Using Some OSS Group Information Functions

```
#include <limits.h>
#include <ctype.h>
#include <sys/types.h>
#include <grp.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

struct group *gnam = NULL;
gid_t grouplist[NGROUPS_MAX];
int gcnt;
char groupname[32];

int main(int argc, char *argv[])
{
    int i;
    char **ptr;
    /*
    Get the group number or name from the command line.
    If no argument is provided, use the group number for the user.
    */
    if(argc > 1) {
        if(isdigit(argv[1][0]))          /* check if numeric value */
            gnam = getgrgid(atoi(argv[1]));
        else                             /* else assume group name */
            gnam = getgrnam(argv[1]);
    } else
        gnam = getgrgid(getgid());
    if(gnam == NULL) {
        fprintf(stderr, "Can't get group entry for %s\n", argv[1]);
        exit(1);
    }

    /* Print group name entry information. */
    printf("Group name entry -> = %s\n", gnam->gr_name);
    printf("Group number entry -> = %d\n", gnam->gr_gid);

    /* Print names of other members of this group using getgrent(). */
    strcpy(groupname, gnam->gr_name);    /* save name of group name */
    setgrent();                          /* reset to beginning */
}
```



```

while((gnam = getgrent()) != NULL) {
    if(strcmp(gnam->gr_name, groupname) == 0)
        break;
}
endgrent();          /* close the file */
ptr = gnam->gr_mem;
printf("All members of group (by name) are:\n");
i = 0;
while(*ptr)
    printf("%d: %s\n", i++, *ptr++);

/* Print supplementary group IDs for user. */
if((gcnt = getgroups(NGROUPS_MAX, grouplist)) < 0) {
    fprintf(stderr, "Can't get list of group membership\n");
    exit(1);
}
printf("\nNumber of supplementary group IDs for user = %d\n", gcnt);
for(i = 0; i < gcnt; i++)
    printf("\tGroup ID #d:\t%d\n", i, grouplist[i]);
return(0);
}

```

Process Ancestors

The creator process of an OSS process is the “parent” process of the newly created “child” process. The child process inherits many OSS and Guardian attributes of the parent process.

The creator process of a Guardian process is the “MOM” process of the newly created process if the new process is unnamed. If the new process is named, the creator process is its “ancestor.” A Guardian process inherits some attributes from its ancestor or MOM if the attributes are not specified in the call to the `PROCESS_LAUNCH_` procedure. For more information about relationships between Guardian processes, see the *Guardian Programmer's Guide*.

Process Attributes

All processes, OSS and Guardian, have Guardian attributes and some OSS attributes. OSS processes have additional OSS-specific attributes, such as an OSS process ID. Open System Services includes standard UNIX functions such as the `fork()` function and the exec set of functions as well as Hewlett Packard Enterprise extension functions, which are `tdm_fork()`, `tdm_execve()`, `tdm_execvpe()`, `tdm_spawn()`, and `tdm_spawnp()`. The Hewlett Packard Enterprise extension functions allow you to set attributes when a process is created that you cannot set with standard UNIX functions.

For example, when you create a new OSS process image with the `execve()` function, the new process retains many attributes of the calling process, and you cannot overwrite them in the call to `execve()`. The `tdm_execve()` function allows you to either retain the caller's attributes or specify other values. For example, you can change the attribute specifying the processor in which a process executes.

Beginning with the H06.24 and J06.13 RVUs, you can determine if a process is a 32-bit or 64-bit OSS process and if a system (local or remote) can run 64-bit OSS processes. See **Process Type Attribute** for details.

Process Environment

The run-time environment of an OSS process is defined using environment variables. A Guardian process uses PARAMs, ASSIGNs, and DEFINEs to define its environment, but a Guardian process also has environment variables if it is created using Guardian C functions.



Process Pairs

A Guardian process can belong to a process pair, which consists of a primary process that performs the application and a secondary (backup) process in another processor that is ready to take over if the primary process fails. Process pairs provide one way in which you can create fault-tolerant applications. You implement process pairs using Guardian checkpointing procedures (passive checkpointing) or by sending explicit messages from the primary to the backup process (active checkpointing).

Prior to the J06.14 and H06.25 RVUs, an OSS process cannot belong to a process pair.

Beginning with the J06.14 and H06.25 RVUs, OSS process pairs are supported. For details of OSS process pairs, see **OSS Process Pairs**.

Native and TNS Processes

A TNS/R native process is initiated by executing a TNS/R native object file; TNS/R native object code executes only on TNS/R systems. A TNS/E native process is initiated by executing a TNS/E native object file; TNS/E native object code executes only on TNS/E systems. A TNS/X native process is initiated by executing a TNS/X native object file; TNS/X native object code executes only on TNS/X systems. A TNS process is initiated by executing a TNS interpreted or accelerated object file; TNS object code executes on TNS, TNS/R, TNS/E, and TNS/X systems.

On TNS/R systems, OSS and Guardian processes can be either TNS/R native or TNS processes. On TNS/E systems, Guardian processes can be either TNS/E native or TNS processes, but OSS processes can only be TNS/E native processes; the TNS/E OSS environment does not support TNS processes. On TNS/X systems, Guardian processes can be either TNS/X native or TNS processes, but OSS processes can only be TNS/X native processes; the TNS/X OSS environment does not support TNS processes.

Selecting Source and Target Execution Environments

Application program development takes place in the source environment and includes writing, compiling, and debugging code. Once developed, the application program runs in the target execution environment. The source and target execution environments are usually the same (OSS or Guardian environment), but there are some special cases in which they are different.

The Typical Case

In most cases, you will use a combination of OSS and UNIX workstation tools to develop source code and produce applications that run in the OSS target execution environment from the OSS shell.

Special Cases of Cross Compilation

There are two special cases of cross compilation in which you might want to use different source and target environments.

- OSS source environment, Guardian target execution environment. You might want to produce applications to run in the Guardian environment from a Tandem Advanced Command Language (TACL) prompt but take advantage of the OSS development tools to do some initial development (editing, some compiling and debugging). In the OSS environment, you can use filenames containing more than eight characters and a deep, tree-structured directory system, as compared with an eight-character limit for Guardian file identifiers and a shallow file system.
- Guardian source environment, OSS target execution environment. This is the situation of porting existing Guardian applications to run in the OSS environment from the OSS shell.

For more information about cross compilation, see the *Open System Services Porting Guide*.



Calling Functions and Procedures in OSS Programs

In general, you should use OSS function calls because doing so makes applications more portable. However, there are cases in which you will want or need to call Guardian system procedures and Guardian C functions.

The following subsections describe when to use which category of functions or procedures and provide examples of calling Guardian procedures directly from an OSS program.

OSS Functions

Use OSS functions wherever possible, even to manipulate Guardian objects. For example, if you want to open a Guardian file that is odd-unstructured, use the OSS `open()` function. Call OSS functions as you would any standard C function.

OSS functions are defined in library header files in the `/usr/include` directory. For detailed information about OSS functions, see the following manuals:

- Library functions (those in logical section 3 of the reference pages) are described in the *Open System Services Library Calls Reference Manual*.
- System functions (those in logical section 2 of the reference pages) are described in the *Open System Services System Calls Reference Manual*.
- The Hewlett Packard Enterprise C language for NonStop systems, run-time library, and compiler information are described in the *C/C++ Programmer's Guide*.

Guardian Procedures

You can (and in some cases must) use Guardian procedures when you cannot accomplish a task with OSS functions. For example, to get the file code of a Guardian file, you must use a Guardian file system procedure, such as `FILE_GETINFO_` or `FILE_GETINFOBYNAME_`. You also might want to use Guardian procedures in cases where the Guardian procedure provides more information. For example, `FILE_GETINFOLIST_` provides much more information than the `stat()` function about Guardian or OSS files.

Many of the Guardian procedures can be called directly from an OSS C program by including the `cextdecs.h` header file in the program. The `cextdecs.h` file contains function prototype declarations for the Guardian procedures you can call directly from your C program. You must use the `EXTENSIONS` pragma with this file and should compile using the `-Wextensions` flag.

For information about Guardian procedures, see the *Guardian Procedure Calls Reference Manual*, the *Guardian Programmer's Guide*, and the *Open System Services Porting Guide*. Some procedures in the *Guardian Procedure Calls Reference Manual* are described as “superseded” by other procedures. For example, the `OPEN` procedure is superseded by the `FILE_OPEN_` procedure. Superseded procedures do not take full advantage of more current features and you should not use them. They are provided only for backward compatibility.

A call to a Guardian procedure usually either returns a return value or sets a condition code. Most of the Guardian procedures you need to use fall into one of these two categories, and you can call these procedures directly from an OSS program. Those procedures that return both a return value and a condition code cannot be called directly from an OSS (or Guardian C) program.

Procedures That Return a Return Value

Calling a Guardian Procedure That Returns a Return Value shows an OSS program that calls the Guardian procedure `FILE_GETINFOBYNAME_` to get information about a Guardian file, whose filename is provided as an input parameter to the procedure. The procedure return value is assigned to `retcode`.

Calling a Guardian Procedure That Returns a Return Value

```
#include <stdio.h>
#include <stdlib.h>
```



```

#include <string.h>
#include <cextdecs.h(FILE_GETINFOBYNAME_)>    /* Note 1 */

char *filename;
short typeinfo[5];

int main(int argc, char *argv[]) {
    short retcode, physreclen;    /* Note 2 */

    /* Get Guardian filename and pass it to the Guardian procedure. */
    if(argc > 1)
        filename = argv[1];
    else
        filename = "$system.system.stdioh";
    retcode = FILE_GETINFOBYNAME_(filename, /* Guardian filename */
        (short) strlen(filename), /* filename length */
        typeinfo, /* return array of file information */
        &physreclen, /* returned physical record length */
        , /* desired options, if any Note 3 */
        , /* tag or timeout value, if any */
    );

    /* Check for valid filename. */
    if (retcode != 0) {
        fprintf(stderr, "retcode = %d\n", retcode);
        fprintf(stderr, "Non-existent file or filename in bad format\n");
        exit(1);
    }

    /* Print information for a disk file. */
    printf("Filename = %s\n", filename);
    printf("device type = %d\n", typeinfo[0]);
    printf("device subtype = %d\n", typeinfo[1]);
    printf("physical record length = %d\n", physreclen);
    if(typeinfo[0] == 3) { /* it's a disk file */
        printf("object type = %d\n", typeinfo[2]);
        printf("file type = %d\n", typeinfo[3]);
        printf("file code = %d\n", typeinfo[4]);
    }
    return(0);
}

```

Note 1

The `cextdecs.h` library header file contains function prototype declarations for Guardian procedures you can call directly from a C program. It is important to specify which Guardian procedures the program will call; otherwise, all function prototype declarations in `cextdecs.h` are included in your program. In this program, only the function prototype declaration for the `FILE_GETINFOBYNAME_` procedure is included.

To use `cextdecs.h` with a native program, you must specify the `EXTENSIONS` pragma, because the procedure declarations in `cextdecs.h` use `_tal` syntax instead of the pragma function syntax.

Note 2

The integer into which a value is returned, `retcode`, is declared as a short integer. This is because the integer data type in Guardian procedures represents a 16-bit word, while the integer data type in OSS C programs represents a 32-bit word.

Note 3

You must insert a placeholder comma (,) if you omit optional parameters, unless you omit them from the end of the list. Omitted optional parameters contain default values, if they are defined.

Procedures That Return a Condition Code

Calling a Guardian Procedure That Returns a Condition Code calls the Guardian procedures `FILE_OPEN_` and `READX` to open and read data from a Guardian file, which is provided as an input parameter to the procedure. `FILE_OPEN_` returns a return value in `retcode`, and `READX` returns a condition code in `CC`.

Calling a Guardian Procedure That Returns a Condition Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <tal.h>      /* Note 1 */
#include <cextdecs.h(FILE_OPEN_, READX)>

_cc_status CC;      /* Note 2 */
short filenum, bytesread, retcode;
char *filename;
char buffer[1024];

int main(int argc, char *argv[]) {
    int i;

    /* Get Guardian filename and open the file. */
    if (argc > 1)
        filename = argv[1];
    else
        filename = "$system.system.stdioh";
    retcode = FILE_OPEN_(filename, /* name of Guardian file*/
                        (short)strlen(filename), /* filename length */
                        &filenum, /* file number returned */
                        1, /* open for read */
                        , /* exclusion */
                        , /* nowait depth */
                        , /* sync or receive depth */
                        );

    /* Check if open was successful. */
    if (retcode != 0) {
        fprintf(stderr, "retcode = %d\n", retcode);
        fprintf(stderr, "Can't open %s\n", filename);
        exit(1);
    }

    /* Read from the file and check for valid condition code. */
    CC = READX( filenum, /* file number */
                buffer, /* buffer address */
                1024, /* size of buffer */
                &bytesread, /* number of bytes read */
                );
    if(_status_ne(CC)) { /* if condition code indicates an error */
        fprintf(stderr, "Read error, bytes read = %d\n", bytesread);
        exit(1);
    }
}
```

```

    }

/* Print number of bytes read and contents of first ten bytes.*/
printf("Bytes read = %d\n", bytesread);
printf("First 10 bytes read (in octal): ");
for (i = 0; i < 10; i++)
    printf("%o ", buffer[i]);
printf("\n");
return(0);
}

```

Note 1

The `tal.h` library header file must be included when you call Guardian procedures that return condition codes.

Note 2

The `_catalysts` type-specifier indicates that a procedure does not return a value but does set a condition code.

Procedures That Return Both a Return Value and a Condition Code

Procedures that return both a return value and a condition code cannot be called directly from an OSS program, so you must write a “jacket” procedure in pTAL that can be called from a C program. This jacket procedure can contain only the code required to call the desired Guardian procedure, and the jacket procedure must be callable from an OSS program. Most Guardian procedures, but not all, can be called from an OSS program.

You will probably not need to call procedures that return both a return value and a condition code, but if you do, see the *C/C++ Programmer's Guide* for details.

Guardian C Functions

The native C run-time library supports both the OSS and Guardian environments, which means that there is a single library for functions that are part of both APIs. Most differences that exist in behavior of the functions in the two APIs for TNS programs no longer exist for native programs. For information about interoperability between the Guardian and OSS APIs in native processes, see **Table 79: API Interoperability in Native Processes (Functions a through m)**.

There are a few functions that are specified by Hewlett Packard Enterprise and can be called only from a Guardian module, which means you must call them from a separate module compiled for the Guardian environment. For information on these functions, see the *Open System Services Library Calls Reference Manual* or the *Guardian Native C Library Calls Reference Manual*.

Considerations for TNS Programs

To call Guardian C functions from an OSS TNS program in which the `main()` module is compiled to run in the OSS environment, you must write the function calls in a separate module compiled for the Guardian environment and link (bind) that module with the OSS program. The Guardian C Run-Time Library contains the functions in the ISO/ANSI C standard with Hewlett Packard Enterprise extensions. In many cases, C functions of the same name in the OSS and Guardian APIs behave identically, but in some cases they do not. Also note that not all functions in each API can be called from each environment. Before writing mixed-module programs, review **Table 79: API Interoperability in Native Processes (Functions a through m)**, which describes the interoperability between the Guardian and OSS APIs in TNS processes.

Using Guardian C functions is an alternative to using Guardian procedures when you cannot accomplish a task with OSS functions. Because of the mixed-module programming and API interoperability considerations, it is easier to use Guardian procedures. However, if you are more familiar with the C language than with Guardian procedures, you might want to use Guardian C functions. For example, if you want to open a Guardian file and then just read and write to it, this can be done equally well using Guardian procedures or Guardian C functions.

However, you need to look at the whole set of required operations to determine whether you can implement them all using Guardian C functions or that there would be a mix of Guardian procedures and Guardian C functions. If there is a mix, it would be easier to use all Guardian procedures. For example, if your goal is to open a Guardian file and later use



FILE_GETINFOLISTBYNAME_ to get information about it, you could open the file with the Guardian C `open ()` function. Then you have to use another Guardian C function to translate the file descriptor returned by `open ()` to a filename, which is required by FILE_GETINFOLISTBYNAME_.

Compiling and Linking Separate Guardian Modules

To call a Guardian module from an OSS program, you must compile and link that module with the OSS program in which the `main ()` module is compiled for the OSS environment. The separate Guardian module must be compiled with the `SYSTYPE` pragma set to `GUARDIAN`. This gives you the Guardian, not the OSS, API. The default `SYSTYPE` pragma for programs compiled with the `c89` or `c99` utility is `SYSTYPE OSS`.

To compile a module with the `SYSTYPE` pragma set to `GUARDIAN`, specify the `-Wsystype` flag of the `c89` or `c99` utility.

The `c89` or `c99` utility is used to compile and link OSS programs. **Specifying the systype Option to the c89 Utility** compiles but does not link the source file `gprog.c` with a `SYSTYPE` of `GUARDIAN`.

Specifying the systype Option to the c89 Utility

```
$ c89 -c -Wsystype=guardian gprog.c
```

The result of **Specifying the systype Option to the c89 Utility** is an object file named `gprog.o`. **Linking Guardian Modules to OSS Programs** compiles the main OSS program and links it with the Guardian C module object file into an executable program file named `outfile`.

Linking Guardian Modules to OSS Programs

```
$ c89 -o outfile gprog.o ossprog.c
```

The operations shown in **Specifying the systype Option to the c89 Utility** and **Linking Guardian Modules to OSS Programs** cannot be combined into one `c89` command. You must use two `c89` commands, because you cannot compile two modules with different `systype` values in the same `c89` command.

For some basic examples of how to use the `c89` and `c99` utilities, see **Using the Development Tools** and for more detail, refer to *C/C++ Programmer's Guide* and to the `c89 (1)` or the `c99 (1)` utility reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.

Refer to Bits From Left to Right

When using OSS functions, you set the values of parameters, options, flags, and fields instead of referring to values by their bit numbers within fields. When using Guardian procedures, sometimes you must refer to a particular bit in a field by its number. For example, to open an OSS file using its pathname with a Guardian procedure, you set bit 10 of the `options` parameter of the `FILE_OPEN_` procedure to 1.

Remember that NonStop software refers to bits from left to right, which means that bit 0 is the leftmost bit in a field.

Using the Development Tools

This subsection describes the programming development tools available to you from the OSS and Guardian environments, including editors, development utilities, compilers, linkers, optimizers, debuggers, and performance analyzers.

Using Tools in Both Environments

You can edit, compile, link, and optimize programs to run in either the OSS or Guardian target execution environment from either development environment. While it is easier in most cases to use OSS tools to develop applications to run in the OSS target execution environment, there are some cases in which you will want to use Guardian tools. Also, there are some tools that can run only from the Guardian environment, such as the TEDIT editor or the Measure performance analyzer.



Multiple Windows

Maintaining two windows open to the same NonStop system, one with a TACL prompt and one with an OSS shell prompt, lets you move easily between the Guardian and OSS environments. For example, you can write, compile, and test programs with OSS tools in the OSS environment and move to the TACL prompt to debug programs and analyze performance. For information on running Open System Services, see the *Open System Services User's Guide*.

gtac1 Utility

Using the `gtac1` utility is another way to mix development environments. `gtac1` runs a process in the Guardian environment from the OSS environment and lets you use Guardian tools from the OSS shell. You can use `gtac1` to start an interactive TACL process, execute a single Guardian environment command, or run a Guardian environment program. **Using `gtac1` to Run a TACL Command** uses `gtac1` to run the TACL STATUS command.

Using `gtac1` to Run a TACL Command

```
$ gtac1 -c 'status *, user'
```

You can also pipe (send) the output of a Guardian command or program to an OSS utility. **Using `gtac1` to Pipe TACL Command Output to an OSS Utility** pipes the output of the TACL FILES command to the OSS `grep` utility.

Using `gtac1` to Pipe TACL Command Output to an OSS Utility

```
$ gtac1 -c 'files' | grep -c 'tdm'
```

When using `gtac1`, keep in mind that block-mode applications, such as TEDIT and Pathway, require 6530 terminal emulation. 6530 terminal emulation is available only in the Guardian environment, which means that block mode applications are not accessible through `gtac1`.

For more information about `gtac1`, refer to the *Open System Services User's Guide* and to the `gtac1 (1)` utility reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.

OSSTTY

OSSTTY provides a way for OSS processes to interact with Guardian processes using the OSS standard files. Using OSSTTY, an OSS process can redirect its standard files (`stdin`, `stdout`, `stderr`) to a Guardian process or to a Guardian EDIT file. For example, the following command uses OSSTTY to redirect the `stdin` of the OSS child process to `$vhs`, and the `stdout` to `$zhome`:

```
TACL> osh/ in $vhs, out $zhome/ -osstty
```

For more information on using OSSTTY, see **Managing I/O**. A complete description of the `osstty` command options is given in the *Open System Services Management and Operations Guide*.

OSS Tools

The following tools are available to you from the OSS environment at the OSS shell prompt. If you provide your own tools, be aware that OSS environment tools run faster as compiled programs than as OSS shell scripts.

Editing Tools

Hewlett Packard Enterprise provides the following text-editing tools in the OSS environment:



- `ed` and GNU `ed`³ are line editors. For detailed information on `ed`, refer to the *Open System Services User's Guide* and to the `ed(1)` reference page either online or in the *Open System Services Shell and Utilities Reference Manual*. For more information on GNU `ed`, see <http://www.gnu.org/software/ed/>.
- `gawk`³, is used to modify text files and extract data based on patterns. For more information, see <http://www.gnu.org/software/gawk/>.
- `sed` and GNU `sed` are stream editors. For detailed information on `sed`, refer to the `sed(1)` reference page either online or in the *Open System Services Shell and Utilities Reference Manual*. Starting from J06.20 and L16.05 RVUs, GNU `sed`³ is supported. For more information on GNU `sed`, see <http://www.gnu.org/software/sed/>.
- `vi`, a screen-oriented line editor. For detailed information, refer to the *Open System Services User's Guide* and to the `vi(1)` reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.
- `vim`, the Vi IMproved command-line editor, beginning with the H06.25 and J06.14 RVUs. For detailed `vim` information, see the `vimvolume(1)` reference page.



TIP: The *OSS Core Utilities User Commands* appendix in the *Open System Services User's Guide* provides details for viewing the `vim volume(1)` reference page.

³ Available on L16.05 and later L-series RVUs, and J06.20 and later J-series RVUs.

C Development Utilities

OSS provides a variety of tools to assist you in developing C applications. The following are some of the more useful programming tools:

- `ar`, which creates and maintains groups of one or more named files as a single library file written in an archive format. If the archive contains object files, the archive can be used as an object file library by the Binder. For detailed information, refer to the `ar(1)` utility reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.
- `ncompress`³, which reduces the size of the named files using adaptive Lempel-Ziv coding. It is one of the fastest programs to compress data. For more information, see <http://ncompress.sourceforge.net/>.
- `cpio`³, which copies files into and out of a `cpio` or `tar` archive. For more information, see <http://www.gnu.org/software/cpio/cpio.html>.
- GNU `Diffutils`², which is a package of several programs related to finding differences between files. The GNU `Diffutils` provides the GNU `cmp`, `diff`, `diff3`, and `sdiff` commands. For detailed GNU `Diffutils` information, see the corresponding OSS Core Utilities reference page, such as `cmpvolume(1)`.

- GNU `Findutils`

²

, which is a package of basic file searching utilities. The GNU `Findutils` provides the GNU

`find`

,

`locate`

,

`updatedb`

, and

² Available beginning with the J06.17 and H06.28 RVUs and included in the OSS Core Utilities (T1202) product.



xargs

commands. For detailed GNU Findutils information, see the corresponding OSS Core Utilities reference page, such as *findvolume(1)*

.

- `grep`

, which is a command-line utility for searching plain-text data sets for lines matching a regular expression. Additionally, GNU

`grep`²

is included in the OSS Core Utilities (T1202) product. For detailed information about the GNU

`grep`

utility, see the OSS Core Utilities

grepvolume(1)

reference page.

- `help2man`³, which automatically generates manual pages from program output. For more information, see <http://www.gnu.org/software/help2man/>.
- `lex`, which allows you to create structured input analyzers. You provide regular expressions that specify what you want to find in the input stream and `lex` generates a C language lexical analyzer from these rules that provides tokens for your application. The `lex` utility is often used in conjunction with `yacc`, which parses the stream of tokens that the lexical analyzer produces. For detailed information, refer to the *Open System Services User's Guide* and to the `lex(1)` utility reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.

`flex`³, which is an alternative for `lex` is also supported. It is an alternative for `lex`. For more information on `flex`, see <http://flex.sourceforge.net/>.
- `make`, which maintains program dependencies. The `make` utility is useful in combining your program modules into a coherent application and enabling you to maintain individual modules. For detailed information, refer to the *Open System Services User's Guide* and to the `make(1)` utility reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.

Additionally, the GNU `make`² utility is included in the OSS Core Utilities (T1202) product. For detailed information about the GNU `make` utility, see the OSS Core Utilities *makevolume(1)* reference page.
- `nm`, which lists symbolic information appearing in an object file, executable file, or object-file library. For detailed information, refer to the `nm(1)` utility reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.
- `patch`², which takes a patch file containing a difference listing produced by the `diff` program and applies those differences to one or more original files, producing patched versions. For detailed information about the GNU `patch` utility, see the OSS Core Utilities *patchvolume(1)* reference page.
- `strip`, which removes any information not required for executing files. Do not use `strip` before using the symbolic debuggers, because `strip` removes the symbol region required for symbolic debugging. For detailed information, refer to the `strip(1)` utility reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.
- `yacc`, which enables you to generate parsing programs. This utility is useful if, for example, your application has an input language. You supply a grammar for the language, and `yacc` produces a C program that parses the input stream. The `yacc` utility is often used in conjunction with `lex`, from which it receives tokens. For detailed information, refer to the *Open System Services User's Guide* and to the `yacc(1)` utility reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.



You can also use `bison`³ to generate a wide range of parsing programs. For more information, see <http://www.gnu.org/software/bison/>.

² Available on J06.16 and later J-series RVUs, H06.27 and later H-series RVUs, and included in the OSS Core Utilities (T1202) product.

³ Available on L16.05 and later L-series RVUs, and J06.20 and later J-series RVUs.



TIP: The *OSS Core Utilities User Commands* appendix in the *Open System Services User's Guide* provides details for viewing the OSS Core Utilities reference pages.

Compilation Tools

The `c89` and `c99` utilities control the C compilation system in the OSS environment. There are native versions of `c89` and `c99` for TNS/E and TNS/X systems, a native version of `c89` for TNS/R systems, and a TNS version of `c89` for TNS/R systems. (A TNS version of `c89` is not supported on TNS/E or TNS/X systems). The native versions are the default versions. The `c99` utility is supported for TNS/E targets on TNS/E systems only. For TNS/X targets, `c99` utility is supported on TNS/X systems only.

Native `c89` and native `c99` provide an interface to the components of the C compilation system, including the C and C++ language preprocessors, C and C++ compilers, linkers (`nld`, `ld`, `elld`, and `xld` utilities), optimizer (built into the compilers), and the SQL compilers. The `noft` (on TNS/R systems), `enoft` (on TNS/E systems), and `xnoft` (on TNS/X systems) utilities allow you to examine various attributes of native object files.

TNS `c89` provides an interface to the components of the C compilation system, including the C and C++ language preprocessor, C and C++ compilers, linkers (Binder), Accelerator (on TNS/R systems), and the SQL/MP compiler.

The following table shows some sample `c89` commands. For each sample command, the table shows the name of the output file produced and what operations are performed.

Table 3: Sample c89 Commands

Command	Output File	Operations Performed
<code>c89 test1.c</code>	<code>a.out</code>	- TNS/R target: Compiles, links (using the default linker <code>nld</code>), produces non-PIC executable program file. - TNS/E target: Compiles, links (using the <code>elld</code> linker), produces PIC (position-independent code) executable program file. - TNS/X target: Compiles, links (using the <code>xld</code> linker), produces PIC (position-independent code) executable program file.
<code>c89 test1.c -Wcall_shared</code>	<code>a.out</code>	Compiles, links (using the <code>ld</code> linker for TNS/R target, the <code>elld</code> linker for TNS/E target, or the <code>xld</code> linker for TNS/X target), produces position-independent code (PIC) executable file.
<code>c89 -g test1.c</code>	<code>a.out</code>	Compiles, links, generates symbolic information for debugging (used by the Inspect debugger for a TNS/R target, or the Native Inspect debugger for a TNS/E or TNS/X target), produces executable program file.

Table Continued

Command	Output File	Operations Performed
<code>c89 -c test1.c</code>	<code>test1.o</code>	Compiles, does not link, produces object file.
<code>c89 -o testout test1.c</code>	<code>testout</code>	Compiles, links, produces executable program file.
<code>c89 -o testout test1.o test2.c</code>	<code>testout</code>	Compiles <code>test2.c</code> , links it with <code>test1.o</code> , produces executable program file.
<code>c89 -O testout</code>	<code>testout</code>	Optimizes an executable program file.
<code>c89 -Wlp64 cmain64.c</code>	<code>a.out</code>	Beginning with H06.24 and J06.13 RVUs: Compiles with the LP64 data model, links, produces 64-bit OSS executable program file. This command is also available on L15.02 and later RVUs.

For more information, see:

Topic	Manual
<code>c89</code>	The <i>C/C++ Programmer's Guide</i> and the <code>c89 (1)</code> utility reference page either online or in the <i>Open System Services Shell and Utilities Reference Manual</i>
<code>c99</code>	The <i>C/C++ Programmer's Guide</i> and the <code>c99 (1)</code> utility reference page either online or in the <i>Open System Services Shell and Utilities Reference Manual</i>
<code>nld</code>	The <i>nld Manual</i> and the <code>nld (1)</code> reference page either online or in the <i>Open System Services Shell and Utilities Reference Manual</i>
<code>ld</code>	The <i>ld Manual</i> and the <code>ld (1)</code> reference page either online or in the <i>Open System Services Shell and Utilities Reference Manual</i>
<code>noft</code>	The <i>noft Manual</i> and the <code>noft (1)</code> reference page either online or in the <i>Open System Services Shell and Utilities Reference Manual</i>
<code>eldxld</code>	The <i>eld and xld Manual</i> and the <code>eld (1)</code> reference page either online or in the <i>Open System Services Shell and Utilities Reference Manual</i>
<code>enoft</code>	The <i>enoft Manual</i> and the <code>enoft (1)</code> reference page either online or in the <i>Open System Services Shell and Utilities Reference Manual</i>
<code>xnoft</code>	The <i>xnoft Manual</i>
Binder	<i>Binder Manual</i>
Accelerator	<i>Accelerator Manual</i>

Table Continued

Topic	Manual
Object Code Accelerator	<i>Object Code Accelerator (OCA) Manual</i>
Object Code Accelerator for TNS/X	<i>Object Code Accelerator for TNS/X (OCAX) Manual</i>

Compiler Output - Two Object File Types

The TNS/R native environment supports two object file types: position-independent code (PIC) object files and non-PIC object files. The TNS/E and TNS/X native environments support only PIC object files. PIC enables you to create dynamic-link libraries (DLLs) and to create programs that use DLLs. A DLL is a type of library that can be loaded dynamically by an executing process.

To create a PIC linkfile, an option flag is required on the TNS/R `c89` command. On TNS/E and TNS/X systems, PIC is the only object file type supported, and no option flag is required on the TNS/E or TNS/X `c89` commands. You then use the `ld` utility (for a TNS/R target), the `eld` utility (for a TNS/E target), or the `xld` utility (for a TNS/X target) to link the PIC object file into a DLL or PIC program file. Refer to the *ld Manual* or the *eld and xld Manual* for more information about PIC files and the `ld`, `eld`, or `xld` utility.

PIC files are not supported in the TNS environment.

Compilation Considerations for TNS Programs

The `ld`, `nld`, `eld`, `xld`, `noft`, `enoft`, and `xnoft` utilities are not available to TNS programs.

The Accelerator program optimizes TNS programs for TNS/R systems. (OSS does not support the TNS compilation tools on TNS/E and TNS/X systems.) You can include the `-O` flag on the `c89` command line with your first compilation, but this is not a good idea because the optimization step is time-consuming, increases your compilation time, and loses some debugging information. A better method is to compile and debug your program completely and then optimize the executable program file.

Command Line Debuggers

Two command-line symbolic debuggers are provided in the OSS and Guardian environments: Inspect and Native Inspect. The G-series Inspect debugger supports debugging of TNS and non-position-independent code (non-PIC) TNS/R object files in the OSS and Guardian environments on TNS/R systems. (To debug a PIC object file on a TNS/R system, you must use either Visual Inspect or the low-level Debug facility.) The H-series Inspect debugger supports debugging of TNS processes in the Guardian environment on TNS/E systems; it does not support debugging of TNS/E and TNS/X native processes. The Native Inspect debugger is supported on TNS/E and TNS/X systems, and supports debugging of native processes in the OSS and Guardian environments.

The Native Inspect debugger is conceptually similar to the Inspect debuggers and has similar functionality, but has different syntax and commands. The Native Inspect syntax is based on `gdb`, a debugger that is widely used throughout the industry and is familiar to many application developers.

Both Inspect and Native Inspect are interactive symbolic debuggers. Both provide source-level, or symbolic, access to a program by allowing you, for example, to refer to a location by the logical name you have given it in the source code or by the source code line number. The Inspect and Native Inspect debuggers also provide machine-level access to a program by allowing you to refer to an item by its absolute or relative address, display and modify machine registers, and display machine code. However, if you optimize a program, you might need to use low-level debugging capabilities. On TNS/E and TNS/X systems, Native Inspect provides commands for low-level debugging; on TNS/R systems, you can use the Debug facility to look at optimized code.

You can start the Inspect or Native Inspect debugger by itself and then run programs within it, or you can start both the Inspect or Native Inspect debugger and your program at the same time. **Starting the Inspect or Native Inspect Debugger by Itself** starts the Inspect debugger (on a TNS/R system) or Native Inspect debugger (on a TNS/E or TNS/X system) by itself, from both the TACL prompt (two examples) and the OSS shell.



Starting the Inspect or Native Inspect Debugger by Itself

```
>RUN INSPECT
```

or

```
$ gtacl -p inspect
```

Starting the Inspect or Native Inspect Debugger and a Program at the Same Time starts the Inspect (on a TNS/R system) or Native Inspect debugger ((on a TNS/E or TNS/X system)) and a program to be debugged at the same time, both from the TACL prompt and the OSS shell.

Starting the Inspect or Native Inspect Debugger and a Program at the Same Time

```
> RUND gprog
```

```
$ run -debug -inspect=on ossprog
```

For more information on the G-series Inspect debugger, see the *Inspect Manual*. If you are familiar with `dbx`, a debugger commonly found on UNIX systems, see the *Inspect Manual* for a table contrasting `dbx` and Inspect commands. For more information on the Native Inspect debugger, see the *Native Inspect Manual*.

Beginning with the H06.24 and J06.13 RVUs, you have the ability to debug 64-bit OSS programs. See [64-Bit TNS/E and TNS/X Debugger Support](#) for details.

PC-Based Debuggers

NonStop Development Environment for Eclipse debugging

NSDEE Core with Debugging also includes a separately installed integrated debugger that you can use to debug TNS/E or TNS/X processes. See [Using the NonStop Development Environment for Eclipse](#).

Beginning with the H06.24 and J06.13 RVUs, you have the ability to debug 64-bit OSS programs. See [64-Bit TNS/E and TNS/X Debugger Support](#) for details.

Visual Inspect

Visual Inspect is a PC-based interactive debugging tool that provides a graphical user interface (GUI) in which you control the debug session and execute debugging commands through pull-down menus and dialog boxes. Using Visual Inspect, you control the execution and debugging of your program from a PC while the program runs on a NonStop server.

Visual Inspect supports high-level symbolic debugging of native and TNS processes (interpreted and accelerated) in the Guardian and OSS environments through a PC-based graphical user interface. Visual Inspect can also be used for debugging snapshot files.

Visual Inspect is supported on both G-series and H-series systems. G-series and H-series Visual Inspect are identical, except that the H-series version has been enhanced with machine-level debugging capabilities; you can use H-series Visual Inspect for low-level debugging tasks on the PC that must be done on the NonStop server using Debug or Inspect on G-series systems.

NOTE:

Visual Inspect debugger is not supported on L-series systems (TNS/X).

Visual Inspect is the preferred debugger in the native environments. It offers a simpler and more intuitive user interface and more capabilities compared to the command-line debuggers, and is designed to work in complex distributed environments. For example, working from a PC, you can use Visual Inspect to debug multiple processes residing on the same or on different nodes in a network. The processes can be TNS, native TNS/R, or native TNS/E, in the Guardian or OSS environment.

You can start Visual Inspect by entering the RUNV command, either at the TACL prompt or from the OSS shell. The following example starts execution of a program under control of Visual Inspect on the client PC `mypc`:



Starting the Visual Inspect Debugger

```
> RUNV WSADDR=MYPC gprog  
$ runv -wsaddr=mypc ossprog
```

You can also start Visual Inspect by first establishing a client connection to Visual Inspect on a PC, then entering a `run` command with `-inspect=on`. For example, assuming a Visual Inspect client connection exists, the following command starts a program under control of Visual Inspect:

```
$ run -debug -inspect=on ossprog
```

Visual Inspect can also be started from within the Enterprise Toolkit - NonStop Edition (ETK) PC-based program development environment (see [Using the Enterprise Toolkit - NonStop Edition](#) for more information).

NOTE:

Beginning with the H06.24 and J06.13 RVUs, 64-bit OSS programs are supported. The Native Inspect debugger starts when specifying a 64-bit OSS program as the `ossprog` parameter for either the `run -debug` or `runv` command, because Visual Inspect does not currently support 64-bit programs.

See the *Visual Inspect online help* for more information on Visual Inspect.

Porting Kit FLOSS

Beginning with the H06.15 and J06.04 RVUs, Freeware Look for OSS (FLOSS) is supported. FLOSS is a set of command-line utilities and libraries that aid in porting open source software to OSS. FLOSS is required only if you are rebuilding any of the open source packages that are posted in the [ITUGLIB](#) website or wish to use it to assist in porting your own software package. FLOSS is not required if the pre-built software packages are installed and used. For detailed information, refer to the *FLOSS on NonStop User Manual*.

Guardian Tools

The following tools are available to you from the Guardian environment at the TACL prompt.

Editor

TEDIT is a full-screen text editor. For information about TEDIT features and commands, see the *PS Text Edit and PS Text Format User's Guide*.

File-Type Conversion Tools

TEDIT creates type 101 files. Files in the `/G` directory created with an OSS editor such as `vi` have a type of 180. If you have a type 180 file you want to edit with TEDIT, use the conversion tool CTOEDIT to convert the type 180 file to a type 101 file, as shown in [Convert Type 180 File to Type 101 File](#).

Convert Type 180 File to Type 101 File

```
> CTOEDIT ossfile, editfile
```

To convert a type 101 file to a type 180 file, use the EDITTOC conversion tool, as shown in [Convert Type 101 File to Type 180 File](#).



Convert Type 101 File to Type 180 File

```
> EDITTOC editfile, ossfile
```

Note that files of type 180 do not have a concept of line numbers. Converting from file type 101 to 180 and back to 101 results in the loss of line number information.

Compilation Tools

There are two sets of native compilation tools and one set of TNS compilation tools:

- The TNS/R native compilation tools are the the TNS/R native C and C++ compilers, the `ld` and `nld` utilities (linkers), and the `noft` utility for examining attributes of TNS/R native object files. Control over code optimization is built into the compilers
- The TNS/E native compilation tools are the TNS/E native C and C++ compilers, the `eld` utility (linker), and the `enoft` utility for examining attributes of TNS/E native object files. Control over code optimization is built into the compilers.
- The TNS/X native compilation tools are the TNS/X native C and C++ compilers, the `xld` utility (linker), and the `xnoft` utility for examining attributes of TNS/X native object files. Control over code optimization is built into the compilers.
- The TNS compilation tools are the TNS C compiler, Binder (linker), and Accelerator (optimizer on systems running G-series RVUs), Object Code Accelerator (optimizer on systems running H-series RVUs), or Object Code Accelerator for TNS/X (optimizer on systems running L-series RVUs). On systems running G-series RVUs, the TNS compilation tools are supported in the OSS and Guardian environments. On systems running H-series, J-series, and L-series RVUs, the TNS compilation tools are supported only in the Guardian environment.

For more information about the Guardian compilation tools, see the following manuals:

- *C/C++ Programmer's Guide*
- *ld Manual*
- *nld Manual*
- *eld and xld Manual*
- *noft Manual*
- *enoft Manual*
- *xnoft Manual*
- *Binder Manual*
- *Accelerator Manual*
- *Object Code Accelerator (OCA) Manual*
- *Object Code Accelerator for TNS/X (OCAX) Manual*

Debuggers

See the discussion about the Inspect, Native Inspect, NonStop Development Environment for Eclipse (NSDEE), and Visual Inspect products in [OSS Tools](#).



Measure Performance Analyzer

The Measure product is a data collection and measurement tool that provides a wide range of performance statistics on system resources; Measure interfaces can be used from the Guardian environment to gather information on OSS objects. Using the Measure product, you can gather information from applications, systems, and network components, then use this data to perform sizing evaluations for applications, balance and tune your system, detect bottlenecks, and balance workloads. For example, you can use the Measure product to see how processor-bound a program is so you can evaluate whether it is worthwhile to optimize the program.

Measure provides both a command interface and a programmatic interface. The command interface is a set of commands you can enter at the terminal or input from a file. The programmatic interface is a set of procedures you can call from an application program. Using the command or programmatic interface, you can configure a measurement, start the measurement, stop the measurement, and create a formatted report.

Measure provides flexibility in the reporting of performance data, allowing you to manipulate the content and format of output displays. You can use a standard format that can be modified. You can also display data in plot and bar-chart formats, and you can create custom reports using Enform or the HPE NonStop SQL/MP command interpreter and report writer.

The Measure product has been enhanced to include features specifically designed to support the OSS environment. These features are summarized in the following subsections. For more information about the Measure product, see the *Measure User's Guide*.

OSS Pathnames in Measure Commands

You can specify OSS pathnames in Measure commands and procedure calls. Measure also supports the display of OSS pathnames in output reports.

OSS Opens in Measure

The FILE entity now encompasses the OSS file open types, including OSS Regular Files, OSS Pipes, OSS FIFO, and OSS Sockets. This enhances the ability of Measure to analyze and improve interprocess data flow for OSS applications.

OSS Entities in Measure

Measure provides instrumentation of a number of OSS components whose capacity or operation influence overall system performance. These include the OSS CPU counters and OSS Name Server entities. The OSS CPU counters entity provides information about the OSS elements that operate in each processor of a system. These elements include the POSIX Extended Segment, the OSS File System Cache, the OSS File Manager, the OSS Pipe Pool, and the OSS Pipe Server. The OSS Name Server entity provides information about the operation and performance of OSS Name Server processes.

Other Measure Enhancements for OSS

Other Measure features have been enhanced for OSS support, including:

- **PROCESS entity.** This entity provides information about one or more processes on a local system. For OSS instrumentation, the PROCESS entity record is extended to include aggregate counters for three OSS file types: TTY, `/dev/null`, and interactions with the OSS Name Server for directory information.
- **DISKFILE Entity.** This entity measures all I/O operations performed by all opener processes on a specified disk file. For OSS instrumentation, the DISKFILE entity record is extended by four counters. They report on cache callback management in the OSS File System.

Using the NonStop Development Environment for Eclipse

The NonStop Development Environment for Eclipse (NSDEE) is a Windows-hosted integrated development environment for NonStop applications. NSDEE supports building NonStop applications locally using Windows-hosted cross-compilers



and tools, or remotely using compilers and tools on a NonStop server. NSDEE also provides facilities for transferring source files and binaries to and from NonStop systems as well as facilities for editing remote source files locally.

NSDEE Core with Debugging also includes a separately installed integrated debugger that you can use to debug TNS/E or TNS/X processes.

The documentation for NSDEE is integrated with the software. After you install the software, you can access the documentation from the Eclipse Workbench help menu:

Procedure

1. To access the Workbench help menu, select **Help > Help Contents**.
2. From the list of manuals, select the *NonStop Development Environment for Eclipse User Guide*.
3. If you have installed NSDEE Core with Debugging, you can also select the *NonStop Development Environment for Eclipse Debugging Supplement* from the list of manuals.

Using the Enterprise Toolkit - NonStop Edition

The Enterprise Toolkit - NonStop Edition (ETK) is an integrated development environment that enables you to build NonStop applications on a PC. Hewlett Packard Enterprise has integrated its native C and C++ PC cross compiler and related tools into a development environment that can produce code for both Guardian and OSS environments.

ETK is an extension package to Microsoft Visual Studio .NET that provides a full range of application development functions. ETK has commands and options that enable you to compile and link your applications on a PC and then run them on a NonStop system. After compiling and linking, you can send your object file directly to the NonStop system either automatically with project deployment or manually with the Transfer Tool. You perform all functions using a familiar graphical user interface.

ETK provides a choice of compilers, editors, and debuggers, including the Visual Inspect debugger. Using ETK, you can create and maintain archive files, perform source control, and create projects that contain embedded SQL/MP or SQL/MX source code.

On G-series systems, many of these development functions can also be performed using the HPE Tandem Development Suite (TDS), but ETK provides certain advantages and is the recommended development environment. TDS is not supported on system running H-series RVUs.

ETK is delivered on a separate compact disc and is not available on the Hewlett Packard Enterprise site update tape (SUT).

For more information on using the ETK and native C and C++ cross compilers on a PC, refer to the online help for the ETK. For more information on using TDS, refer to the TDS online help and to the *C/C++ Programmer's Guide*.

NOTE:

ETK is not supported on L-series RVUs.

Using OpenSSL

Starting with the L20.05 RVU, the OpenSSL toolkit is ported to the NonStop OSS platform. The version corresponds to OpenSSL version 1.1.1*. The libraries are released as both archive libraries and DLLs.

For more information on Open SSL, see the following resources:



- <https://www.openssl.org>
- <https://www.openssl.org/docs/man1.1.1/man3>
- *Open System Services Management and Operations Guide*



Managing Files

You access OSS files in the OSS environment as you would access UNIX files. The OSS environment also enables you to access files in the Guardian environment, which means that you can create applications for NonStop systems that use data in Guardian files and that take advantage of OSS portability and other OSS features.

In the OSS environment, sockets, pipes, FIFOs, and terminals are also files. These files are discussed elsewhere in this guide. With respect to the OSS environment, this section discusses only directories and regular files.

This section discusses the following topics:

Topic	Content
<u>The OSS and Guardian File Systems</u>	Differences between the two file systems, including filename formats and mapping, and Guardian security and OSS file-access permissions.
<u>Common and Unique Characteristics</u>	Summary of the differences between the OSS and UNIX file systems.
<u>Accessing Files From the OSS API</u>	Discussion and examples of how to access files from the OSS API.
<u>Accessing Files From the Guardian API</u>	Discussion and examples of how to access files from the Guardian API.
<u>Accessing OSS Files Larger Than 2 GB</u>	Discussion of support for OSS files larger than approximately 2 GB, including accessing such files from OSS APIs and Guardian APIs, and interoperability issues.
<u>Common Completion</u>	An enhancement to the Enscribe file system and the OSS file system.
<u>Performance Considerations</u>	Discussion of how to get optimum performance for your application.
<u>OSS File-System Functions</u>	List of each OSS file-system function with notes about its OSS implementation and use with Guardian files.
<u>Guardian File-System Procedures</u>	List of each Guardian file-system procedure that affects or is affected by OSS files with notes about its use with OSS files.

The OSS and Guardian File Systems

This subsection describes differences between the Guardian and OSS file systems and the implications of the differences for OSS programming.



Physical and Logical Names

Visible Guardian filenames are physical: they include the names of physical storage devices. A fully qualified filename includes the name of a physical system (node) and the name of a physical disk volume.

OSS pathnames are logical names: they do not include the names of storage devices. An OSS pathname describes a path through the OSS directory tree to a file. Every OSS pathname (the character string that identifies a file within its file system) for a disk file has a physical name in the Guardian namespace, but that physical (Guardian) name cannot normally be used to access files. To access OSS files, you must use OSS pathnames. For further information about OSS files and pathnames, refer to the *Open System Services User's Guide*.

The OSS file system theoretically allows an unlimited degree of nesting of directories within a directory. You can have subdirectories in a directory, subsubdirectories in a subdirectory, and so on. The Guardian environment allows only three levels in a node: volume, subvolume, and file ID. You cannot have nesting; therefore, you cannot have a volume that is within another volume.

Filename and Pathname Mappings

The /G directory contains Guardian files, but they are referred to by OSS pathnames. Guardian filenames are much more restricted than OSS pathnames and cannot always be used interchangeably. For example, the OSS filenames `properties.c` and `properties.d` are both translated into the Guardian file ID `PROPERTI`, because Guardian file IDs must be no longer than eight characters, whereas OSS filenames can be much longer. This subsection describes how OSS pathnames and Guardian filenames are mapped to each other, so that you can construct pathnames and filenames whose information does not get lost in the translation process.

Translating OSS Pathnames to Guardian Filenames

OSS functions use /G directory pathnames when they access Guardian files. /G directory pathnames for Guardian files on the local node begin with the root directory; Guardian files on another NonStop server node can be found in the /E directory under the node name and that node's corresponding /G directory.

The following steps describe how the local node pathname:

```
/G/Data.Volume/src.11.v3.4.8/properties.c
```

is translated into a Guardian filename.

Procedure

1. The leading /G is replaced by the local node name with a backslash, a slash, and a dollar sign (\$). The intermediate result is:

```
\NODE/$Data.Volume/src.11.v3.4.8/properties.c
```

2. The portable pathname characters period (.), comma (,), hyphen (-), and underscore (_) are deleted. The intermediate result is:

```
\NODE/$DataVolume/src11v348/propertiesc
```

3. Name components, which are between the slashes, are truncated to eight characters. The intermediate result is:

```
\NODE/$DataVol/src11v348/properti
```

4. Slashes are replaced by periods. The final result is:

```
\NODE.$DataVol.src11v348.properti
```

The following steps describe how the remote node pathname:

```
/E/NODE1/G/Data.Volume/src.11.v3.4.8/properties.c
```

is translated into a Guardian filename.



1. The leading /E is replaced by a backslash. The intermediate result is:
`\NODE1/G/Data.Volume/src.ll.v3.4.8/properties.c`
2. The /G/ is replaced by a slash and a dollar sign (\$). The intermediate result is:
`\NODE1/$Data.Volume/src.ll.v3.4.8/properties.c`
3. The portable pathname characters period (.), comma (,), hyphen (-), and underscore (_) are deleted. The intermediate result is:
`\NODE1/$DataVolume/src11v348/propertiesc`
4. Name components, which are between the slashes, are truncated to eight characters. The intermediate result is:
`\NODE1/$DataVol/src11v348/properti`
5. Slashes are replaced by periods. The final result is:
`\NODE1.$DataVol.src11v348.properti`

The pathname /G/volname/subvol/fileid of a permanent Guardian disk file is translated to the filename \NODE.\$VOLNAME.SUBVOL.FILEID.

The pathname /G/volname/#num of a temporary Guardian disk file is translated to \NODE.\$VOLNAME.#num.

With relative pathnames, the name component . (dot) is deleted. The pathname ./subvol/fileid is treated as /G/volname/subvol/fileid.

If a relative pathname contains the name component . . (dot-dot), the preceding name component is deleted. If the deleted component was the first component of the Guardian filename, normal root resolution continues at the root of the root fileset. For example, the pathname /G/vol1/./vol2/subvol/fileid is treated as /G/vol2/subvol/fileid, and ./vol1/./././dir is treated as /dir.

If the name resulting from the translation is not a valid Guardian filename, the translation operation terminates with the error ENOENT, except when the creation of such a name is attempted, which fails with the error EINVAL.

The following examples result in invalid filenames:

/G/data/a/b/c (too many levels)
 /G/oss/src:v3.4/logger.c (the colon :) is an illegal character)

The following table shows pathnames that can be translated to valid Guardian filenames and the corresponding Guardian filename.

Table 4: OSS Pathname to Guardian Filename Translation Examples

OSS Pathname on Node Named NODE	Guardian Filename
/G/Data.Volume/src.ll.v3.4.8/properties.c	\NODE.\$DATAVOL.SRC11V34. PROPERTI
/G/data/mysubvol/file	\NODE.\$DATA.MYSUBVOL.FILE
/G/disk	\NODE.\$DISK
/G/ztn1/#pty7	\NODE.\$ztn1.#pty7



Only filenames on the local node can appear in the /G directory, so only local names are translated. OSS files are stored with Guardian filenames in subvolumes whose names begin with ZYQ. The OSS absolute pathname that corresponds to a Guardian filename has the following form:

```
/G[/volume[/subvolume[/fileid]]]
```

The number of components of the translated pathname depends on the number of components in the filename. The components of a pathname are derived from a filename as follows:

1. The filename is converted to lower case.
2. *volume* is derived by replacing the dollar sign (\$) in the filename with /G/.
3. A period (.) in the filename is replaced with a slash (/).

The following table shows pathnames that can be translated to valid OSS filenames and the corresponding OSS filename.

Table 5: Guardian Filename to OSS Pathname Translation Examples

Guardian Filename	OSS Pathname
\$VOL	/G/vol
\$vol	/G/vol
\$VOL.SUBVOL	/G/vol/subvol
\$p.#1234	/G/p/#1234
\$p.#PTY12	/G/p/#pty12
\$pubs.henry.panaceas	/G/pubs/henry/panaceas

Obtaining Corresponding Guardian and OSS Filenames

You can obtain the Guardian filename that corresponds to an OSS pathname and vice-versa with OSS shell utilities, the Guardian File Utility Program (FUP), or through programmatic interfaces.

Using OSS Shell Utilities

You can obtain the Guardian equivalent of an OSS pathname with the OSS shell utility `gname`, and you can obtain the OSS equivalent of a Guardian filename with the OSS shell utility `pname`.

To display the fully qualified Guardian filename that corresponds to an OSS pathname, enter the following from the OSS shell:

```
gname [-s] pathname
```

where the `-s` flag displays only the Guardian filename and *pathname* is an OSS pathname. **Using `gname` to Obtain a Guardian Filename** shows examples of using `gname`.

Using `gname` to Obtain a Guardian Filename

```
$ gname test2  
gname: test2 --> \NODE.$VOL.ZYQ00000.Z0000KHP
```



```
$ gname -s test
\NODE.$VOL.ZYQ00000.Z0000DV3

$ gname /G/zznt/#pty00bk
gname: /G/zznt/#pty00bk --> \NODE.$ZZNT.#PTY00BK
```

To display the OSS pathname that corresponds to a Guardian filename, enter the following from the OSS shell:

```
pname [-s] filename
```

where the `-s` flag displays only the OSS pathname and *filename* is a Guardian fully qualified or system-qualified filename.

Using pname to Obtain an OSS Pathname shows examples of using `pname`.

NOTE:

As shown in **Using pname to Obtain an OSS Pathname**, you must put a backslash (\) before the slash (/), the backslash (\), and dollar sign (\$) characters, or else the shell will interpret them with their special shell meanings rather than as plain characters. You can also put the filename inside single quotation marks.

Using pname to Obtain an OSS Pathname

```
$ pname \$VOL.ZYQ00000.Z0000DV3
pname: $VOL.ZYQ00000.Z0000DV3 --> /home/henrysp/test

$ pname \\node.\$VOL.ZYQ00000.Z0000DV3
pname: \node.$VOL.ZYQ00000.Z0000DV3 --> /home/henrysp/test

$ pname -s \$VOL.ZYQ00000.Z0000DV3
/home/henrysp/test

$ pname -s '$VOL.ZYQ00000.Z0000DV3'
/home/henrysp/test
```

File Utility Program (FUP)

The Guardian FUP INFO command with the DETAIL option displays the OSS pathname for a file next to the PATH: heading, as illustrated in **FUP INFO DETAIL Display With an OSS File**.

FUP INFO DETAIL Display With an OSS File

```
> fup info \NODE.$ROOT.ZYQ00000.Z0000066, detail

$ROOT.ZYQ00000.Z0000066          12 Apr 2000, 13:01
  OSS
  PATH: /bin/ls
  OWNER -1
  SECURITY: -r-xr-xr-x
  CREATION DATE:  7 Dec 1998, 10:12
  ACCESS TIME:   11 Apr 2000, 10:55
  EOF: 89232
```

In the figure, the OWNER field contains -1 because the owner is the super ID. For other owners, the field contains the structured form (*group_number*, *member_number*) of the NonStop operating system user ID. The structured form can be converted to the scalar form used as the OSS user ID (UID) using the following formula:

$$\text{UID} = \text{member_number} + (256 * \text{group_number}).$$

Using Guardian Procedures

You can obtain the Guardian filename that corresponds to an OSS pathname with the Guardian procedure `PATHNAME_TO_FILENAME_`, and you can obtain the OSS pathname that corresponds to a Guardian filename with the Guardian procedure `FILENAME_TO_PATHNAME_`.



Using the Guardian PATHNAME_TO_FILENAME Procedure shows how to use the PATHNAME_TO_FILENAME_ procedure. The program takes as its input an OSS pathname and prints the corresponding Guardian filename.

Using the Guardian PATHNAME_TO_FILENAME_ Procedure

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <cextdecs.h(PATHNAME_TO_FILENAME_)>

char filename[64];
short filelen;
short status;

int main(int argc, char *argv[]) {
    short ret;

    /* Make sure the correct number of arguments are entered.
       If not, print usage message and exit. */
    if(argc < 2) {
        fprintf(stderr,"Usage: pthtofil pathname\n");
        exit (1);
    }
    /* Invoke the Guardian procedure call.
       Note the special format for options. */
    ret = PATHNAME_TO_FILENAME_(argv[1], /* OSS pathname */
                               filename, /* Guardian filename buffer */
                               64, /* size of filename buffer */
                               &filelen, /* length of filename */
                               &status ); /* if = 1, Guardian file (/G)
                                             if = 0, OSS file */

    /* If the call fails, print a message and exit. */
    if (ret != 0) {
        fprintf(stderr,"Bad path name\n");
        exit(1);
    }
    filename[filelen] = '\000'; /* make NULL-terminated string */

    /* Determine the file type and print the file. */
    if(status&1)
        printf("Guardian (/G) file = %s\n", filename);
    else
        printf("OSS file = %s\n", filename);
    return(0);
}
```

Using the Guardian FILENAME_TO_PATHNAME Procedure shows how to use the FILENAME_TO_PATHNAME_ procedure. The program takes as its input a Guardian filename and prints out the corresponding OSS pathname. This program example demonstrates the use of /E.

Using the Guardian FILENAME_TO_PATHNAME_ Procedure

```
Guardian FILENAME_TO_PATHNAME procedure */

#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
```



```

#include <string.h>
#include <tal.h>
#include <cextdecs.h(FILENAME_TO_PATHNAME_)>

char pathname[PATH_MAX];
short pathlen;
short index = 0;

int main(int argc, char *argv[]) {
    short ret;

    /* Make sure the correct number of arguments are entered.
       If not, print a usage message and exit. */
    if(argc < 2) {
        fprintf(stderr,"Usage: filtopth filename\n");
        exit (1);
    }

    /* Invoke Guardian procedure call. Note the special format for options */
    while(index != -1) { /* print all possible pathnames */
        ret = FILENAME_TO_PATHNAME_(argv[1], /* Guardian filename */
                                   (short)strlen(argv[1]), /* length of filename */
                                   pathname, /* buffer for OSS pathname */
                                   PATH_MAX, /* length of buffer */
                                   &pathlen, /* length of pathname */
                                   6, /* absolute pathname wrt. system root */
                                   /* includes system name in the form
                                      /E/system/pathname */
                                   &index /* index of link to file */
                                   );
        /* If the call fails, print a message and exit. */
        if (ret != 0) {
            fprintf(stderr,"Bad file name\n");
            exit(1);
        }
        /* Print pathname */
        printf("Pathname = %s\n", pathname);
    }
    return(0);
}

```

Using FILE_COMPLETE_SET_, FILE_COMPLETE_, and FILE_COMPLETE_GETINFO_ shows the use of FILE_COMPLETE_SET, FILE_COMPLETE_, and FILE_COMPLETE_GETINFO_ to wait for the completion of I/O on Guardian and OSS files. Once enabled for completion, files from the specified set can be completed multiple times. Guardian files are opened for NOWAIT I/O, and OSS files can be nonblocking or blocking.

Using FILE_COMPLETE_SET_, FILE_COMPLETE_, and FILE_COMPLETE_GETINFO_

```

/* Guardian FILE_COMPLETE_* procedures */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <tdmext.h>
#include <unistd.h>
#include <fcntl.h>
#include <zsysc>.h>
#include <tal.h>
#include <cextdecs.hh(FILE_OPEN_,READX,FILE_COMPLETE_,AWAITIOX,\

```

```
FILE_COMPLETE_SET_,FILE_COMPLETE_GETINFO_)>
```

```
_cc_status CC;
short filenum, fnum, bytesread;
int fd,nb;
short retcode, status;
char *pathname;      /* OSS path name */
char *filename;      /* Guardian file name */
char bufoss[1025];   /* allow for NULL character at end of OSS buffer */
char bufgdn[1025];   /* allow for NULL character at end of Guardian buffer */
zsys_ddl_complete_element_def comp_elem[2]; /* completion elements */
short err_elem;      /* error return for FILE_COMPLETE_SET_ */
short num_elem;      /* Total number of Guardian and OSS files */
zsys_ddl_completion_info_def comp_info;    /* completion information */
zsys_ddl_complete_element_def info_list[7];
                                   /* array of information elements */

char buffer[1026]; /* allow for NULL termination of string in buffer */
long request_tag;
int i;

int main(int argc, char *argv[]) {
/* Check for the proper number of arguments;
   if no path/file names are supplied, use the default names. */
   if (argc > 1)
       pathname = argv[1];
   else
       pathname = "/usr/include/stdio.h";
   if(argc > 2)
       filename = argv[2];
   else
       filename = "$system.system.stdioh";

/* Open the OSS file in non-blocking and read-only mode */
   fd = open(pathname, O_NONBLOCK|O_RDONLY);
/* If the open is unsuccessful, print a message and exit. */
   if (fd < 0) {
       fprintf(stderr, "Cannot open %s\n", pathname);
       exit(1);
   }

/* Open the Guardian file in nowait mode */
   retcode = FILE_OPEN_(filename,          /* name of Guardian file */
                        (short) strlen(filename), /* filename length */
                        &filenum,          /* file number returned */
                        1,                  /* open for read */
                        ,                   /* shared exclusion */
                        1,                  /* number of concurrent I/O
                                           operations permitted */
                        ,                   /* sync or receive depth */
                        ,                   /* Guardian filename */
                        );
/* If the open is unsuccessful, print a message and exit. */
   if (retcode != 0) {
       fprintf(stderr, "Cannot open %s\n", filename);
       fprintf(stderr, "retcode = %d\n", retcode);
       exit(1);
   }
}
```



```

    }

/* Set the completion elements for the OSS file and the Guardian file */
comp_elem[0].z_fnum_fd = fd;          /* OSS file */
comp_elem[0].u_z_options.z_options.z_filetype = 1;
comp_elem[0].u_z_options.z_options.z_read_ready = 1;
comp_elem[0].u_z_options.z_options.z_exception = 1;
comp_elem[1].z_fnum_fd = filename;    /* Guardian file */
comp_elem[1].u_z_options.z_options.z_filetype = 0;
status = FILE_COMPLETE_SET((short *) comp_elem, 2, &err_elem);
if(status) {
    if(err_elem)
        fprintf(stderr, "Error %d in FILE_COMPLETE_SET for OSS file = \
        %s\n", err_elem, pathname);
    else
        fprintf(stderr, "Error %d in FILE_COMPLETE_SET for Guardian file = \
        %s\n", err_elem, filename);
    exit(1);
}

/* Start non-blocking I/O on OSS file */
nb = read(fd, bufoss, 1024);
if(nb < 0) {
    fprintf(stderr, "OSS file read error %d\n", nb);
    exit(1);
}

/* Start nowait read on the Guardian file using READX.
Note that a request tag is not required when only one
outstanding read is issued. It is shown here for
demonstration purposes only. */
request_tag = 123;          /* form unique request tag */
CC = READX( filename,      /* file number */
            bufgdn,        /* buffer address */
            1024,          /* size of buffer */
            ,              /* number of bytes read, not
                             used for nowait I/O */
            request_tag     /* unique request tag */
            );

/* If the condition code returned is not the one for success,
print a message and exit. */
if(_status_ne(CC)) {        /* check the success condition code */
    fprintf(stderr, "Read failed on Guardian file\n");
    exit(1);
}

/* Get info. about the set of files enabled for common completion */
status = FILE_COMPLETE_GETINFO_(
    (short *) info_list,    /* array of information elements */
    2,                     /* only two requested */
    &num_elem               /* actual number of files */
);
if(status) {
    fprintf(stderr, "File Complete Getinfo error %d\n", status);
    exit(1);
}

/* The program can execute code while the read operations are being
completed by the system. */

```

```

for (i = 0; i < 100000; i++) ;
printf("Now issuing FILE_COMPLETE_ to check for I/O completion.\n");

status = FILE_COMPLETE_(
    (short *) &comp_info      /* completion information */
    ,                          /* time limit (* 0.01 sec.) */
    ,                          /* array of complete elements */
    ,                          /* number of complete elements */
    ,                          /* error index */
    );
if(status) {
    exit(1);
}

/* AWAITIOX completes the READX operation. */
CC = AWAITIOX(&filenum ,      /* file number */
    ,
    &bytesread,               /* number of bytes read */
    &request_tag              /* request tag */
    );

/* If the condition code returned is not the one for success,
print a message and exit. */
if(_status_ne(CC)) { /* check the success condition code */
    fprintf(stderr, "Awaitiox failed.\n");
    exit(1);
}
printf("Bytes read = %d\n", bytesread);

/* Print out the first ten bytes in octal plus "string". */
printf("First 10 bytes read (in octal): ");
for (i = 0; i < 10; i++)
    printf("%o ", bufgdn[i]);
printf("\n");
bufgdn[bytesread] = '\0'; /* make sure string is NULL-terminated */
printf("\n%s\n", bufgdn);
return(0);
}

```

OSS File-Access Permissions

You obtain file-access permissions and other OSS file information with the `stat()`, `lstat()`, and `fstat()` functions. Mappings of Guardian security to OSS file-access permissions are summarized in the following table. Refer to the `stat(2)` reference page either online or in the *Open System Services System Calls Reference Manual* for details about OSS file access permissions and how they and Guardian security attributes and settings are mapped to each other.

Table 6: Mappings of Guardian Security to OSS File-Access Permissions

Guardian Security Attributes and Settings	OSS File-Access Permission
Network (N)	Other
Any User (A)	Other

Table Continued



Guardian Security Attributes and Settings	OSS File-Access Permission
Community or Group (C or G)	Group
User (U or O)	Owner
Super ID (-)	None (-)
Read	Read
Write	Write
Execute	Execute
Purge	(Ignored)

Mappings of OSS file-access permissions to Guardian security are based on user information and are not predictable based only on file permission information. OSS file-access permissions do not reflect:

- Safeguard access control lists (ACLs). To determine whether there is an Safeguard ACL on a Guardian file, use the Guardian `FILE_GETINFOLISTBYNAME_` procedure.
- OSS ACLs . To determine whether there is an OSS ACL on an OSS file you can use these functions:
 - The OSS `stat()`, `fstat()`, and `lstat()` system calls return an ACL -PRESENT flag that specifies if the file or directory has an optional OSS ACL entry.
 - Guardian `FILE_GETINFOLIST_` and `FILE_GETINFOLISTBYNAME_` procedures return an ACL -PRESENT flag that specifies if the file or directory has a optional OSS ACL entry.
- Information about the file privilege file attribute (H06.22 and later H-series RVUs and J06.11 and later J-series RVUs only). To list the file privileges for a file, use the `getfilepriv` command. Also, the OSS `stat()`, `fstat()`, and `lstat()` system calls return a `stat` or `stat64` structure that includes the `st_fileprivs` member, which indicates the file privileges for the file. The Guardian `FILE_GETINFOLIST_` and `FILE_GETINFOLISTBYNAME_` procedures do not indicate if a file has file privileges.

Safeguard Protection and Disk Volumes

You cannot use Safeguard access control lists (ACLs) to protect individual OSS files. However, you can protect the disk volume in which OSS files reside by restricting the ability of users to create files in that volume.

If an Safeguard ACL restricts create access within a disk volume where OSS files reside, you cannot create files in that volume unless you are one of the users listed in the Safeguard ACL. If you are not listed in the Safeguard ACL, you can still open, read, and write to any existing file in that volume.

The Safeguard product is a NonStop subsystem. For information about programmatic access to this subsystem, see the *Safeguard Management Programming Manual* and the *SPI Programming Manual*.

OSS ACLs

OSS ACLs are supported for directories, regular files, first-in, first-out (FIFO) files, and bound AF_UNIX sockets in Version 3 catalog filesets on systems running J-series RVUs, H06.08 and later H-series RVUs, and G06.29 and later G-series RVUs.



ACLs offer a greater degree of selectivity than permission bits. ACLs allow a process whose effective user ID matches the file owner, super ID, or a member of the Safeguard SECURITY-OSS-ADMINISTRATOR security group to permit or deny access to a file to a list of specific users and groups.

ACLs are supported as a superset of the UNIX operating system discretionary access control (DAC) mechanism for files, but not for other objects such as interprocess communication (IPC) objects.

All OSS system calls that include pathnames are subject to the ACLs on any directory or file in the path.

For details about OSS ACLs, see [Using OSS Access Control Lists \(ACLs\)](#).

OSS ACLs and NFS Clients

OSS ACLs are not supported by the OSS Network File System (NFS) for J06.08 and earlier J-series RVUs, H06.19 and earlier H-series RVUs, or G-series RVUs. Any attempt by NFS clients to access OSS objects protected by OSS ACLs that contain optional ACL entries is denied.

OSS ACLs are supported by the OSS NFS for J06.09 and later J-series RVUs and H06.20 and later H-series RVUs as follows:

- Access by the OSS Network File System (NFS) to OSS objects protected by OSS ACLs that contain optional ACL entries can be allowed, depending upon the NFSPERMMAP attribute value for the fileset that contains the object.
- The NFSPERMMAP attribute value selects the algorithm used to map the OSS ACL permissions for the object to the standard permissions (rwxrwxrwx) expected for the object by NFS V2 clients.
- The default value for the NFSPERMMAP attribute, DISABLED, specifies that any attempt by NFS clients to access OSS objects protected by OSS ACLs that contain optional ACL entries is denied. This behavior matches the behavior for J06.08 and earlier J-series RVUs, H06.19 and earlier H-series RVUs, and G-series RVUs.

For a detailed description of OSS ACLs, including examples, see the `acl (5)` reference page either online or in the Miscellaneous Topics section of the *Open System Services System Calls Reference Manual*. For information about the NFSPERMMAP attribute, see the *Open System Services Management and Operations Guide*. For more information about OSS NFS file system security, see the *Overview of NFS for Open System Services* and the *Open System Services NFS Management and Operations Guide*.

Accessible Files

All OSS source and text files are odd-unstructured files. Files in the `/G` directory that are used as OSS source or text files are Guardian file type 180. Files in the OSS environment are Guardian file type 100 regardless of content. These files permit reading and writing of both even and odd byte counts and positioning to both even and odd byte addresses. Unstructured files do not have a built-in record structure.

Only odd-unstructured files can be read and written in the OSS environment. EDIT files (file type 101) are unstructured but not odd-unstructured, and they can be opened only for reading in the OSS environment. All other Guardian files are inaccessible from the OSS file system but can be accessed through Guardian file-system procedures.

On H-series RVUs prior to H06.06 and on G-series RVUs prior to G06.29, OSS files had a size limit of approximately 2 GB (GB), and an underlying Guardian file of Format 1 (the file format was normally not visible to customer applications). For J-series RVUs, H06.06 and later RVUs, and G06.29 and later G-series RVUs, OSS files larger than approximately 2 GB are supported, and can have an underlying Guardian file of either Format 1 or Format 2 (see [Accessing OSS Files Larger Than 2 GB](#)).

Using the ANSISTREAMS Pragma (Guardian C Program)

The ANSISTREAMS pragma in a Guardian C program causes your application's standard input (`stdin`), standard output (`stdout`), and standard error (`stderr`) files to be opened as odd-unstructured disk files with no line length limit. Without this pragma these files are opened as EDIT files with a line length limit of 239 characters. The ANSISTREAMS pragma enables your programs to comply with the ISO/ANSI C standard, which requires that the C compiler support lines containing at least 254 characters including the terminating newline character.



If the `SYSTYPE OSS` pragma is set (the default setting for the `c89` and `c99` utilities), you do not need to specify the `ANSISTREAMS` pragma because it is set automatically.

For further information about the `ANSISTREAMS` pragma, refer to the *C/C++ Programmer's Guide*.

Common and Unique Characteristics

The OSS file system looks like a typical UNIX file system, but there are a few important differences.

Features Common to the UNIX and OSS File Systems

The OSS file system conforms to the POSIX.1 specification. There is no major difference between the OSS file system and most UNIX file systems. There are some differences in the particulars of some functions. These differences are summarized for each function in **[OSS File-System Functions](#)** and explained in detail in the reference pages, which can be found online and in the *Open System Services System Calls Reference Manual* and the *Open System Services Library Calls Reference Manual*.

Features Unique to the OSS File System

The OSS file system has the following unique features:

- Administration through the Guardian environment and nonstandard information displays. For example, ownership is displayed in the *groupname.username* format rather than the standard *username* format. For additional information, refer to the *Open System Services User's Guide*.
- The `/G` directory, which is the OSS window into the Guardian file system on your local NonStop node. All Guardian disk files on a node have a name in the `/G` directory.
- The `/E` directory, which is the OSS window into file systems on remote NonStop nodes.
- OSS file caching.
- Common Completion.

Accessing Files From the OSS API

In the OSS environment, you access OSS files in the same way that you would in any POSIX-compliant UNIX programming environment. OSS functions are tailored to the OSS environment and work more efficiently than Guardian procedures on OSS files.

You can access up to four types of Guardian files on physical disk volumes with OSS function calls:

- Odd-unstructured Format 1 files (file codes 0, 100, and 180). These files permit reading and writing of both even and odd byte counts and positioning to both even and odd byte addresses. Unstructured files do not have a built-in record structure.
- EDIT files (file type 101). These files are unstructured. You can access them only for reading.
- Telnet processes.
- For J-series RVUs, H06.06 and later RVUs, and G06.29 and later G-series RVUs, odd-unstructured Format 2 files (file codes 0, 100, and 180). These files permit reading and writing of both even and odd byte counts and positioning to both even and odd byte addresses. Unstructured files do not have a built-in record structure. For detailed information about accessing these files, see **[Accessing OSS Files Larger Than 2 GB](#)**.

You cannot access:



- Files on logical disk volumes administered through the Storage Management Foundation (SMF) on systems that are running either J-series RVUs prior to J06.15 or H-series RVUs prior to H06.26. (For details of accessing files on SMF logical disk volumes on systems with later J-series or H-series RVUs, see [Accessing Files on SMF Logical Disk Volumes](#)).
- A volume, a subvolume, or a process other than a TTY simulation process (`/G/vol`, `/G/vol/subvol`, or `/G/process`, respectively)
- A file within a subvolume with a reserved name beginning with ZYQ (for example: `/G/vol12/zyq00004/z000002x`)

Odd-unstructured files and EDIT files are opened as OSS regular files.

If you want your application to modify odd-unstructured Guardian files, or if your application uses data stored in odd-unstructured Guardian files or EDIT files, you can use the interoperability capabilities of the OSS environment and access these files with OSS functions. If you need to modify EDIT files or open structured files, you can use Guardian procedures as described in [Accessing Files From the Guardian API](#).

NOTE: Optical disks are inaccessible directories in `/G`. This means that OSS functions cannot open, create, or read files in directories under `/G` when the files reside on optical disks.

As an alternative to using Guardian procedures when you cannot accomplish a task with OSS functions, you might be able to accomplish the task with the Guardian C functions. Write a separately compiled module using Guardian C functions and link (bind) this module into the OSS program. Whether you should do this depends on whether you can accomplish the required set of operations with Guardian C functions. Refer to [Overview of the OSS Programming Environment](#) for more information about calling Guardian C functions from G-series TNS OSS programs.

[Reading a Guardian File With an OSS Function Call](#) shows how to read a Guardian file with OSS function calls; in this case, `open()` and `read()`. The Guardian file is accessed through the `/G` directory.

Reading a Guardian File With an OSS Function Call

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

char buffer[1026]; /* allow for NULL termination of string in buffer */
char *filename;    /* filename pointer */
int fd;            /* file descriptor */
int nbytes;        /* number of bytes read */

int main(int argc, char *argv[]) {

    /* Check for the proper number of arguments; if no filename
       is supplied, open the default file. */
    if(argc > 1)
        filename = argv[1];
    else
        filename = "/G/system/system/stdioh"; /* this file is opened if
                                                no name is supplied */
    fd = open(filename, O_RDONLY); /* open Guardian file for read */

    /* Check for the validity of the file descriptor; if it is not valid,
       print a message and exit. */
    if(fd < 0) {
        fprintf(stderr, "Can't open %s\n", filename);
    }
}
```

```

        exit(1);
    }

/* If the read is unsuccessful, print an error message and exit. */
nbytes = read(fd, buffer, 1024);
if (nbytes < 0) {
    perror("File read error: ");
    exit(1);
}

/* Print the number of bytes read and the first part of the file. */
printf("Number of bytes read = %d\n", nbytes);
buffer[nbytes] = 0; /* make string NULL-terminated */
printf("%s\n", buffer);
return(0);
}

```

Using Functions With Environment-Specific Parameters

There are several file system functions that allow you to operate on a file in one environment from a process running in the other environment. These functions are `freopen()`, `remove()`, `rename()`, `tmpfile()`, and `tmpnam()`. Each of these functions has environment-specific variants with suffixes to indicate which type of file you want to operate on. For example, `fopen_guardian()` opens a Guardian file from an OSS process and `fopen_oss()` opens an OSS file from a Guardian native process.

NOTE:

If a TNS module calls any of the environment-specific variants, the feature test macro `_INTEROPERABLE` should be specified when the module is compiled. Specifying `_INTEROPERABLE` allows subsequent I/O function calls to know which type of I/O stream they are operating on.

Using `fopen_guardian()` in an OSS Module opens a Guardian file using `fopen_guardian()` in an OSS module and opens an OSS file using `fopen()`.

Using `fopen_guardian()` in an OSS Module

```

/* Verify current process is an OSS process */
#ifdef _OSS_TARGET
#error "Wrong systype"
#endif

#include <stdio.h>
int main (int argc, char *argv[]) {
    FILE *G_File;
    FILE *P_File;
    int c;

/* Open the Guardian file, note that since this is an OSS
   process, need to explicitly call a version of fopen() that
   opens a Guardian file. */
    G_File = fopen_guardian ("system.system.stdioh", "r");

/* Because this is an OSS process, we get the OSS fopen() by
   default. */
    P_File = fopen("mystdio.h", "w");

/* Note that the rest of the I/O routines do not require

```



```

environment-specific parameters. They can operate on OSS or
Guardian files. */
while(!feof(G_File)) {
    c = getc(G_File); /* read from Guardian file */
    putc(c, P_File); /* write to OSS file */
}
fclose(G_File);
fclose(P_File);
return(0);
}

```

Accessing Files on SMF Logical Disk Volumes

Beginning with the J06.15 and H06.26 RVUs, new Storage Management Foundation (SMF) features are:

- All OSS file-system functions that currently operate on unstructured Guardian files by name operate the same on unstructured Guardian files on an SMF virtual disk. The OSS functions are: `access()`, `chdir()`, `chown()`, `creat()`, `creat64()`, `lchown()`, `lstat()`, `lstat64()`, `open()`, `open64()`, `rename()`, `stat()`, `stat64()`, `statvfs()`, `statvfs64()`, and `utime()`.
- All OSS file-system functions that currently operate on file descriptors for an open of a Guardian file operate the same on file descriptors for an open of a Guardian file on an SMF virtual disk. The OSS functions are: `close()`, `dup()`, `dup2()`, `fchown()`, `fcntl()`, `fstat()`, `fstat64()`, `fstatvfs()`, `fstatvfs64()`, `fsync()`, `ftruncate()`, `ftruncate64()`, `lseek()`, `lseek64()`, `read()`, `read64_()`, `write()`, and `write64()`.
- Read access to Guardian edit files on an SMF virtual disk is supported.
- Creation of an OSS process from an OSS program file residing on a SMF virtual disk is supported.

For more details about SMF features beginning with the J06.15 and H06.26 RVUs, see the *HPE NonStopStorage Management Foundation User's Guide*.

Accessing Files From the Guardian API

Guardian files can be managed from the Guardian API as described in the *Guardian Programmer's Guide* and the *Guardian Procedure Calls Reference Manual*.

OSS files can also be managed from the Guardian API. You would use Guardian procedures to access OSS files when you need to perform operations that use the `nowait` feature of the Guardian environment.

Nowait I/O is the ability of an application process to continue executing in parallel with read and write operations; the application process runs concurrently with the I/O operation. The OSS environment uses waited I/O, where an application process is suspended during read and write operations. The process does not resume execution until the I/O operations are complete. Nowait I/O does not work on files that have been opened with the OSS `open()` function.

When you use a Guardian I/O procedure in `nowait` mode, you set the maximum number of concurrent I/O operations that you want to allow and then complete the I/O operations by calling the `AWAITIO[X]` procedure later in your program, as shown in **Reading an OSS File in Nowait Mode With Guardian Procedures**. For details on using `nowait` I/O with Guardian procedures, refer to the *Guardian Programmer's Guide*.

Guardian Procedure Extensions for OSS

Hewlett Packard Enterprise provides extensions to Guardian procedures so that you can use them with OSS files. The extensions are summarized in **Table 9: Guardian File-System Procedures**.

The following procedures have OSS parameters and OSS attributes:



- FILE_GETINFO_
- FILE_GETINFOBYNAME_

The following procedures have OSS attributes:

- FILE_GETINFOLIST_
- FILE_GETINFOLISTBYNAME_
- FILE_OPEN_
- FILE_OPEN_CHKPT_
- SETMODE

Some Guardian procedures fail when they reference an OSS file; these are noted in **Table 9: Guardian File-System Procedures**.

For information on accessing OSS files larger than approximately 2 GB using Guardian APIs and for information about compatibility issues for such files, see **Accessing OSS Files Larger Than 2 GB**. Additional considerations or restrictions on Guardian procedures used on OSS files are summarized in **Table 9: Guardian File-System Procedures**.

The following example shows the use of the Guardian FILE_OPEN_ and READX procedures to read data from an OSS file in nowait mode. The AWAITIOX procedure is used to wait for I/O completion on the read operation.

Reading an OSS File in Nowait Mode With Guardian Procedures

```
/* Reading an OSS file in Nowait Mode with Guardian Procedures */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <tal.h>
#include <cextdecs.h(FILE_OPEN_, READX, AWAITIOX)>

_cc_status CC;
short filenum, bytesread, retcode;
char *pathname;
char buffer[1025]; /* allow for NULL character at end of buffer */
long request_tag;
int i;

int main(int argc, char *argv[]) {
/* Check for the proper number of arguments;
   if no pathname is supplied, use the default name. */
   if (argc > 1)
       pathname = argv[1];
   else
       pathname = "/usr/include/stdio.h";

/* Invoke the Guardian procedure. Note the value of "options" field. */
   retcode = FILE_OPEN_(pathname, /* name of OSS file */
                        , /* pathname length not needed */
                        &filenum, /* file number returned */
                        1, /* open for read */
                        , /* shared exclusion */

```

```

        1,          /* number of concurrent nowait I/O
                     operations permitted */
        ,          /* sync or receive depth */
        040,       /* <10>=1 indicates OSS pathname */
    );

/* If the open is unsuccessful, print a message and exit. */
if (retcode != 0) {
    fprintf(stderr, "retcode = %d\n", retcode);
    fprintf(stderr, "Cannot open %s\n", pathname);
    exit(1);
}

/* Read the file using the Guardian READX procedure.
   Note that a request tag is not required when only one
   outstanding read is issued. It is shown here for
   demonstration purposes only. */
request_tag = 123;          /* form unique request tag */
CC = READX( filenum,       /* file number */
            buffer,        /* buffer address */
            1024,          /* size of buffer */
            ,              /* number of bytes read, not
                           used for nowait I/O */
            request_tag    /* unique request tag */
            );

/* If the condition code returned is not the one for success,
   print a message and exit. */
if(_status_ne(CC)) {       /* check the success condition code */
    printf(stderr, "Read failed\n");
    exit(1);
}

/* The program can execute code while the read operation is being
   completed by the system. */
for (i = 0; i < 100000; i++) ;
printf("Now issuing AWAITIOX to wait for I/O completion.\n");

/* AWAITIOX completes the READX operation. */
CC = AWAITIOX(&filenum,    /* file number */
              ,
              &bytesread,   /* number of bytes read */
              &request_tag, /* request tag */
              );

/* If the condition code returned is not the one for success,
   print a message and exit. */
if(_status_ne(CC)) {       /* check the success condition code */
    fprintf(stderr, "Awaitiox failed.\n");
    exit(1);
}
printf("Bytes read = %d\n", bytesread);

/* Print out the first ten bytes in octal plus "string". */
printf("First 10 bytes read (in octal): ");
for (i = 0; i < 10; i++)
    printf("%o ", buffer[i]);

```

```

    printf("\n");
    buffer[bytesread] = 0; /* make string NULL-terminated */
    printf("\n%s\n", buffer);
    return(0);
}

```

Reading an OSS File With Guardian Procedures shows the use of the Guardian FILE_OPEN_ and READX procedures to read data from an OSS file whose name is provided as input to the program. This program waits for I/O operations to terminate.

Reading an OSS File With Guardian Procedures

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <tal.h>
#include <cextdecs.h(FILE_OPEN_, READX)>

_cc_status CC;
short filenum, bytesread, retcode;
char *pathname;
char buffer[1025]; /* allow for NULL-terminated string */

int main(int argc, char *argv[]) {
    int i;

    /* If no pathname is supplied, use the default name. */
    if (argc > 1)
        pathname = argv[1];
    else
        pathname = "/usr/include/stdio.h";

    /* Invoke the Guardian procedure. Note the special value of options. */
    retcode = FILE_OPEN_(pathname, /* name of OSS file */
        , /* length of pathname not needed */
        &filenum, /* file number returned */
        1, /* open for read */
        , /* shared exclusion */
        , /* nowait depth */
        , /* sync or receive depth */
        040, /* <10>=1 indicates OSS pathname */
    );

    /* If the open fails, print a message and exit. */
    if (retcode != 0) {
        fprintf(stderr, "retcode = %d\n", retcode);
        fprintf(stderr, "Cannot open %s\n", pathname);
        exit(1);
    }

    /* Read the file with the Guardian READX procedure. */
    CC = READX( filenum, /* file number */
        buffer, /* buffer address */
        1024, /* size of buffer */
        &bytesread, /* number of bytes read */
    );
}

```




```

/* If the condition code returned is not the one for success,
print a message and exit. */
if(_status_ne(CC)) {
    fprintf(stderr, "Read error, bytes read = %d\n", bytesread);
    exit(1);
}
printf("Bytes read = %d\n", bytesread);

/* Print out the first ten bytes in octal plus "string". */
printf("First 10 bytes read (in octal): ");
for (i = 0; i < 10; i++)
    printf("%o ", buffer[i]);
printf("\n");
buffer[bytesread] = '\000'; /* make string NULL-terminated */
printf("\n%s\n", buffer);
return(0);
}

```

Using Extended Guardian Procedures

Using a Guardian Procedure to Get OSS File Information by Name shows the use of the extended Guardian procedure `FILE_GETINFOBYNAME_` to get information about an OSS file whose pathname is provided as input to the program.

Using a Guardian Procedure to Get OSS File Information by Name

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <cextdecs.h(PATHNAME_TO_FILENAME_,FILE_GETINFOBYNAME_)>

char *pathname;
char filename[64];
short filelen;
short status;
short typeinfo[5];
short flags;

int main(int argc, char *argv[]) {
    short retcode, physreclen;

    /* If no pathname is supplied, get information
    about the default file. */
    if(argc > 1)
        pathname = argv[1];
    else
        pathname = "/usr/include/stdio.h";

    /* Get the Guardian filename equivalent first. */
    retcode = PATHNAME_TO_FILENAME_(pathname, /* OSS pathname */
        filename, /* Guardian filename buffer */
        64, /* Size of filename buffer */
        &filelen, /* length of filename */
        &status); /* if == 1, Guardian file (/G)
        if == 0, OSS file */

    /* If the procedure is unsuccessful, print a message and exit.*/
    if (retcode != 0) {

```



```

        fprintf(stderr, "Bad pathname, return code = %d\n", retcode);
        exit(1);
    }

    /* Invoke the Guardian procedure call. */
    retcode = FILE_GETINFOBYNAME_(filename, /* Guardian filename */
        filelen, /* filename length */
        typeinfo, /* return array of file information */
        &physreclen, /* returned physical record length */
        , /* desired options, if any */
        , /* tag or timeout value, if any */
        &flags); /* new OSS flags word */
    if (retcode != 0) {
        fprintf(stderr, "retcode = %d\n", retcode);
        fprintf(stderr, "Non-existent file or pathname in bad format\n");
        exit(1);
    }

    /* Print the information about file, indicating whether
    it is an OSS file or a Guardian file. */
    if(flags&1)
        printf("OSS File, ");
    else
        printf("Guardian File, ");
    printf("Pathname = %s\n", pathname);
    printf("device type = %d\n", typeinfo[0]);
    printf("device subtype = %d\n", typeinfo[1]);
    printf("physical record length = %d\n", physreclen);

    /* If the device type is a disk file,
    print the disk file information. */
    if(typeinfo[0] == 3) {
        printf("object type = %d\n", typeinfo[2]);
        printf("file type = %d\n", typeinfo[3]);
        printf("file code = %d\n", typeinfo[4]);
    }
    return(0);
}

```

Using a Guardian Procedure to Get OSS File Information by Number shows the use of the Guardian FILE_OPEN_ and FILE_GETINFO_ procedures to get information about an OSS file. The file is opened, and the file number is passed to the FILE_GETINFO_ procedure.

Using a Guardian Procedure to Get OSS File Information by Number

```

/* Using a Guardian Procedure to Get OSS File info by Number*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <cextdecs.h(FILE_OPEN_, FILE_GETINFO_)>

short typeinfo[5];
char filename[64];
char *pathname;

int main(int argc, char *argv[]) {

```

```

    short filenum, filelen, retcode, lasterror;

/* If no pathname is supplied, open the default file. */
if(argc > 1)
    pathname = argv[1];
else
    pathname = "/usr/include/stdio.h";
    retcode = FILE_OPEN_(pathname,          /* OSS pathname */
                        ,                  /* pathname length not required */
                        &filenum,          /* returned file number */
                        1,                  /* read-only access */
                        ,                  /* shared exclusion */
                        ,                  /* nowait depth */
                        ,                  /* sync or receive depth */
                        040,                /* <10>=1, set to OSS pathname */
                        );

/* If the FILE_OPEN_ procedure is unsuccessful,
print a message and exit. */
if (retcode != 0) {
    fprintf(stderr, "retcode = %d, filenum = %d\n",
            retcode, filenum);
    fprintf(stderr, "Cannot open %s\n", pathname);
    exit(1);
}

/* Get information about the file by file number. */
retcode = FILE_GETINFO_(filenum, /* file number */
                        &lasterror, /* error from last operation */
                        filename, /* Guardian filename returned */
                        64, /* length of filename buffer */
                        &filelen, /* number of bytes in filename */
                        typeinfo ); /* return array of file information */

/* If the procedure is unsuccessful, print a message and exit.*/
if (retcode != 0) {
    fprintf(stderr, "retcode = %d, filenum = %d\n",
            retcode, filenum);
    fprintf(stderr, "Non-existent file or filename in bad format\n");
    exit(1);
}

/* Print the file status information. */
printf("Pathname = %s\n", pathname);
filename[filelen] = '\000'; /* make string NULL-terminated */
printf("Filename = %s\n", filename);
printf("device subtype = %d\n", typeinfo[1]);
printf("device type = %d\n", typeinfo[0]);

/* If the device type is a disk file,
print the disk file information. */
if(typeinfo[0] == 3) {
    printf("object type = %d\n", typeinfo[2]);
    printf("file type = %d\n", typeinfo[3]);
    printf("file code = %d\n", typeinfo[4]);
}

```

```
    return(0);  
}
```

Accessing OSS Files Larger Than 2 GB

Small Files and Large Files

For RVUs earlier than H06.06 and G06.29, OSS files always had an underlying Guardian file format of Format 1 and were limited in size to approximately 2 GB (GB). This size limitation was a direct result of using 32-bit data types for file sizes and offsets. For J-series RVUs, H06.06 and later RVUs, and G06.29 and later G-series RVUs, 64-bit data types and offsets can be used, and OSS files are either small files or large files.

Characteristics of Small Files

Small files:

- Have an underlying Guardian file format of either Format 1 or Format 2. This underlying file format is normally not visible to customer applications.
- Have a size limit of approximately 2 GB when opened or created using a 32-bit API. For information about the exact size limits, see the *Open System Services Management and Operations Guide*.
- Can be accessed using the existing 32-bit application programmatic interfaces (APIs) such as `creat()` and `open()` in addition to the 64-bit APIs such as `creat64()` and `open64()`. If the file is accessed using a 64-bit API:
 - The file is automatically converted to use an underlying Guardian file format of Format 2.
 - The file no longer has a size limit of approximately 2 GB. It can grow to the size limit for large files.
- Can be moved, copied, or restored to systems running RVUs that do not support OSS large files. In such cases, if the file has an underlying Guardian file format of Format 2 but a size of less than approximately 2 GB:
 - The file is automatically converted to use an underlying Guardian format of Format 1.
 - The file becomes subject to the file size limit of approximately 2 GB.

Characteristics of Large Files

Large files:

- Are larger than approximately 2 GB.
- Have a size limit of approximately 1 terabyte, constrained by the space available on the disk volume. For information about the exact size limits, see the *Open System Services Management and Operations Guide*.

NOTE: For J-series RVUs, H06.06 and later H-series RVUs, and G06.29 and later G-series RVUs, a `pax` archive file is limited to 8 GB.

- Have an underlying Guardian file format of Format 2. This underlying file format is normally not visible to customer applications when OSS APIs are used, but might affect program execution when Guardian APIs are used. For more information, see **Compatibility Issue For Guardian Files Created by OSS APIs**
- Can be accessed using 64-bit APIs such as `creat64()` and `open64()` only:



- An application can explicitly call the 64-bit version of a function when you use the `#define _LARGEFILE64_SOURCE 1` feature test macro or an equivalent compiler command option to compile the application.
- An application call to `interface()` is automatically mapped to the `interface64()` function when you use the `#define _FILE_OFFSET_BITS 64` feature test macro or an equivalent compiler command option to compile the application
- Cannot be restored to systems running RVUs that do not support OSS files larger than approximately 2 GB.
- If OSS APIs are used, large files cannot be accessed from systems running RVUs that do not support OSS files larger than approximately 2 GB. However, Guardian Enscribe APIs with 64-bit elections can access these files.

Large File Safe (LFS) Applications and Large File Aware (LFA) Applications

An application that is large OSS file safe (LFS) causes no data loss or corruption when it encounters an OSS file larger than approximately 2 GB. It can not process OSS files larger than approximately 2 GB, but it has the appropriate logic to handle errors or warnings detected during file manipulation operations and fail gracefully.

An application that is large OSS file aware (LFA) can process OSS files larger than approximately 2 GB with the same functionality as processing OSS files smaller than approximately 2 GB. Such an application can handle large files as input and generate large files as output. Applications that are large file aware can also process small files.

Large File Safe APIs

For J-series RVUs, H06.06 and later H-series RVUs, and G06.29 and later G-series RVUS, existing I/O APIs are modified to fail safely on OSS files larger than approximately 2 GB. These APIs return an error (EOVERFLOW or EFBIG) when an action cannot be performed on OSS files larger than approximately 2 GB or an attribute cannot be represented for such a file.

The existing 32-bit I/O APIs modified to be large file safe include:

```
creat()
open()
ftruncate()
lseek()
fstat()
lstat()
stat()
fstatvfs()
statvfs()
ftw()
nftw()
```

Other existing I/O APIs modified to be large file safe include:

```
read()
readv()
write()
writev()
readdir()
readdir_dt()
fcntl()
```

The existing Native C run-time library APIs modified to be large file safe are:

```
fclose()
fflush()
```



```

fgetc()
fgetpos()
fgets()
fgetwc()
fgetws()
fopen_oss()
fprintf()
fputc()
fputs()
fputwc()
fputws()
fread()
freopen_oss()
fwrite()
fscanf()
fseek()
fsetpos()
ftell()
getc()
getchar()
gets()
getw()
getwc()
getws()
getwchar()
printf()
putc()
putchar()
puts()
putw()
putws()
putwchar()
scanf()
vfprintf()
vprintf()

```

In addition, Hewlett Packard Enterprise supports two new 32-bit LFS APIs, `fseeko()` and `ftello()`, because `fseek()` and `ftell()` use data types that cannot be extended to support large files.

Functions added in J-series RVUs or in RVUs later than H06.06 or G06.29 are either large file aware or large file safe.

Large File Aware APIs

For J-series RVUs, H06.06 and later H-series RVUs, and G06.29 and later G-series RVUs, new 64-bit APIs are available. These large file aware APIs support both small files and large files. Typically, the 64-bit APIs include the number 64 in the name of the API.

Beginning with the H06.24 and J06.13 RVUs, 64-bit OSS programs (compiled with the LP64 data model) are large file aware by default and the 32-bit programmatic interfaces are mapped to their 64-bit counterparts.

An application can explicitly call a 64-bit function when you use the `#define _LARGEFILE64_SOURCE 1` feature test macro or an equivalent compiler command option to compile the application.



An application call to *interface* () is automatically mapped to the *interface64* () function when you use the `#define _FILE_OFFSET_BITS 64` feature test macro or an equivalent compiler command option to compile the application.

If a small file is accessed using a 64-bit API:

- The file is automatically converted to use an underlying Guardian file format of Format 2.
- The file no longer has a size limit of approximately 2 GB. It can grow to the size limit for large files.

The LFA OSS APIs are:

```
creat64 ()
open64 ()
ftruncate64 ()
lseek64 ()
fcntl ()
fstat64 ()
lstat64 ()
stat64 ()
fstatvfs64 ()
statvfs64 ()
readdir64 ()
readdir64_dt ()
ftw64 ()
nftw64 ()
glob ()
```

The LFA native C run-time library APIs are:

```
__ns_backup_fopen64 ()
__ns_fopen64_special ()
fgetpos64 ()
fopen64 ()
fopen64_guardian ()
fopen64_oss ()
fopen64_std_file ()
freopen64 ()
freopen64_guardian ()
freopen64_oss ()
fseeko64 ()
fsetpos64 ()
ftello64 ()
scandir64 ()
tmpfile64 ()
tmpfile64_guardian ()
tmpfile64_oss ()
```

For H06.23 and earlier H-series RVUs or J06.12 and earlier J-series RVUs, the Native C++ run-time libraries do not have new 64-bit specific APIs in the C++ library products. However, the logic implementing IOStream classes is enhanced to be large OSS file safe for programs built with the regular compilation environment or large OSS file aware for programs built with the large file compilation environment (that is, macro `_FILE_OFFSET_BITS` is defined to have value 64.).

Interoperability



Opening OSS Files

Applications running on systems that do not include support for OSS large files can use either OSS or Enscribe APIs to open any OSS file smaller than approximately 2 GB, even if that file is on a system that includes support for OSS large files.

Hewlett Packard Enterprise does not support OSS opens of OSS files larger than approximately 2 GB on systems that do not include support for OSS large files. However, because Enscribe has long understood Guardian Format 2 files, a program running on a system that does not support large files can use Enscribe with 64-bit elections to open an OSS file that resides on a system that supports large files. If the file is smaller than approximately 2 GB, the file size limit for that open is approximately 2 GB. If the file is larger than approximately 2 GB, the file size limit for that open is the size of the volume. The actual size limits depend on the API used and how the file was created. For detailed information about size limits when opening files, see the *Open System Services Management and Operations Guide*.

Opening Guardian C Files

For J-series RVUs, H06.06 and later H-series RVUs, and G06.29 and later G-series RVUs, OSS can use the `creat64()` and `open64()` functions to create Guardian files in the `/G` name space. The resulting files, regardless of size, are Guardian Format 2 files with file code 180. If a program running on any RVU attempts to open this type of file using `FILE_OPEN_` without 64-bit elections, the open succeeds if the file is smaller than approximately 4 GB and fails if the file is larger than 4 GB.

If the open call includes 64-bit elections and is issued on J-series RVUs, H06.06 or later H-series RVUs, or G06.29 or a later G-series RVUs, the open succeeds regardless of file size.

If an application running on an RVU earlier than G06.29 or H06.06 invokes `FILE_OPEN_` with 64-bit elections, the open succeeds. However, if the file size is smaller than 4 GB, the file size limit for that open is 4 GB. If the file size is larger than 4 GB, the file size limit for that open is the limit defined at file-creation time.

For detailed information about size limits when opening files, see the *Open System Services Management and Operations Guide*.

Compatibility Issue For Guardian Files Created by OSS APIs

For RVUs before G06.29 and H06.06, OSS files always had an underlying Guardian file format of Format 1. For J-series RVUs, H06.06 and later H-series RVUs, and G06.29 and later G-series RVUs, OSS files can have an underlying Guardian file format of Format 1 or Format 2. For applications using OSS APIs to access OSS files, the underlying Guardian file format is not visible and there are no known compatibility issues. For applications using Enscribe APIs to access Guardian files created by OSS APIs, there is a known compatibility issue.

Before to the G06.29 and H06.06 RVUs, Guardian files created by OSS APIs were always Format 1 with a maximum size of 175 megabytes. For J-series RVUs, H06.06 and later H-series RVUs, and G06.29 and later G-series RVUs, Guardian files created by the `open()` and `creat()` APIs are Guardian Format 1 with a maximum size of approximately 2 GB, and Guardian files created by the `open64()` and `creat64()` APIs are Guardian Format 2 with a maximum size of approximately 26 GB.

The standard OSS utilities, such as `cp`, `pax` and `pinstall`, now use the `open64()` and `creat64()` functions and create a Guardian Format 2 file when given a `/G` output file name. For example:

```
cp myfile /G/disk/subvol/myfile
```

Guardian applications that use file attributes such as file format and maximum size might fail when unexpected values are returned (such as Format 2 if only Format 1 is expected).

Guardian applications that use file data and ignore file attributes are unaffected if all of these conditions are true:

- The application is running on J-series RVUs, H06.06 or later H-series RVUs, or G06.29 or later G-series RVUs.
- The Format 2 file is unstructured, code 180 with an EOF less than the Format 1 limit of approximately 4 GB.



However, FILE_OPEN_ fails with error 580 if any of these conditions are true:

- The RVU is earlier than G06.29 or H06.06
- The file code is not 180
- The EOF is greater than the Format 1 limit

Recovery choices include:

- Change the application to accept Guardian Format 2 files.
- Protect the application by converting the file to Guardian Format 1 using the following steps:

```
>FUP CREATE TEMP, FORMAT 1, TYPE U, ODDUNSTR, CODE 180, &  
EXT (28,1400), MAXTENTS 749  
  
>FUP COPY myfile,TEMP  
  
>FUP PURGE myfile  
  
>FUP RENAME TEMP,myfile
```

- Protect the application by creating the file as a Guardian Format 1 file and then appending data to the file. For example, replace the command:

```
cp myfile /G/disk/subvol/myfile
```

With the commands:

```
gtac1 -c 'FUP CREATE $disk.subvol.myfile, FORMAT 1, TYPE U, ODDUNSTR, CODE 180, EXT (28,1400),  
MAXTENTS 749  
  
cat myfile >> /G/disk/subvol/myfile
```

Obtaining OSS I/O size limits

Starting with the L19.03/J06.23 RVU, applications can use the `pathconf(3)` and `fpathconf(3)` APIs to obtain the maximum OSS I/O size allowed on various OSS files by using the `_PC_REC_MAX_XFER_SIZE` variable name.

fpathconf(3)

Supports the `_PC_REC_MAX_XFER_SIZE` variable name for opens of OSS regular files (in OSS filesets and in the `/G` namespace), FIFOs, pipes, and OSS sockets.

pathconf(3)

Supports the `_PC_REC_MAX_XFER_SIZE` variable name for OSS regular files (in OSS filesets and in the `/G` namespace), FIFOs, and bound `AF_UNIX` sockets.

For more information about the OSS I/O size limits, see *OSS I/O Size Limits* section of the *Environment Limits* appendix of the *Open System Services Management and Operations Guide*.



NOTE: To take advantage of the larger I/O transfer sizes, the Expand configurations between the local system and any remote systems must be configured using the `MAXIOSIZE_2MB` I/O attribute. If the attribute is not set, attempts to transfer a larger regular file I/O requests over the Expand connections will fail.

Following is an example of a function using `fpathconf(3)` to acquire the maximum OSS I/O size allowed on an OSS open. An application may incorporate a function similar to the following function to obtain the maximum I/O size allowed on an OSS open for most OSS file types on any RVU:

```
int FILE_IOSIZE_LIMIT_GET( int fdnum )
{
    /*
     * On L19.03/J06.23 and later L-series/J-series RVUs, the I/O size limit for an
     * OSS regular file I/O size can vary from 52KB to 868KB, depending on the system
     * type and RVU of the application's system, the system type and RVU of the
     * target file's system, and that the configuration of the Expand connections
     * between the systems are configured to handle > 60 KB I/O transfer sizes
     * (if applicable).
     *
     * Modify as needed for your application.
     */
    int value;
    struct stat buf;

#ifdef _PC_REC_MAX_XFER_SIZE
#define _PC_REC_MAX_XFER_SIZE 10
#endif

    if ( ( fstat( fdnum, &buf ) ) != -1 )
    {
        /* fdnum is valid and buf contains valid information */
        if ( S_ISREG(buf.st_mode) != 0 )
        {
            if ( ( value = fpathconf( fdnum, _PC_REC_MAX_XFER_SIZE ) ) < 0 )
            {
                /* if fpathconf(fd, _PC_REC_MAX_XFER_SIZE) returns an error,
                 * _PC_REC_MAX_XFER_SIZE is not supported on the current RVU;
                 * assume the legacy disk I/O size limit of 52KB.
                 */
                return SSIZE_MAX;
            }
            else
            {
                /* _PC_REC_MAX_XFER_SIZE is supported, so use its return value */
                return value;
            }
        }
        else
        {
            /* return the typical I/O size supported by terminal servers for
             * terminals, /dev/tty, /dev/null.
             * Actual terminal I/O limits will vary.
             */
            return 32768;
        }
    }
    else
    {
        /* Not a regular file or char special file, so assume 52 KB */
        return SSIZE_MAX;
    }
}
```

```

    }
}
else
{
    /* errno should be set, so return -1 */
    return -1;
}
}

```

Accessing OSS SEEP-Protected Files

Since the J06.03, H06.08, and G06.29 RVUs, Safeguard provides the feature that allows a partner or customer Security Event-Exit Process (SEEP) to participate in security policy enforcements of Guardian objects (files, subvolumes, and volumes). This security feature was not supported for OSS objects.

Beginning with the J06.15 and H06.26 RVUs, a partner or customer OSS SEEP is supported and can participate in access-control decisions for OSS objects. This capability increases the acceptability of OSS-based applications to security auditors and improves OSS security manageability.

The OSS name server consults the OSS SEEP for access authorization during operations on files when both of the following conditions are true:

- The file resides in a Version 3 catalog fileset that is OSS SEEP-protected. The fileset's SEEPPROTECTED attribute must be set to ON.
- The corresponding OSS name server is OSS SEEP-enabled. The OSS name server's SEEPENABLED attribute must be set to ON and the OSS SEEP must be running.

The final outcome of the operation depends on:

- OSS SEEP's authorization ruling
- POSIX access control lists (ACLs) and standard OSS permissions evaluations result
- Directory-search authorization errors
- Requestor's SOA privileges

The OSS SEEP is not consulted for:

- Search authorization on the directories in the path
- /E and /G directories

NFS operations are not supported on filesets that are OSS SEEP-protected:

- An attempted NFS mount of a directory that belongs to an OSS SEEP-protected fileset is denied with an EPERM error.
- An attempted NFS operation on an OSS SEEP-protected fileset is denied with an EPERM error.

For information about OSS SEEP configuration and management, fileset and OSS name server attributes, audited SCF operations, and EMS events, see the *Open System Services Management and Operations Guide*.

For information about OSS SEEP management, consultation, system and library calls, IPC messages, programming design, and EMS events, see **OSS SEEP Programming**.



Common Completion

Common Completion is an enhancement to the Enscribe file system and the Open System Services file system that allows completion of a Guardian file or returns ready information for OSS (socket, OSSTTY, and disk) files for read, write, or exception descriptors. Common Completion is a combination of an enhanced version of the `AWAITIO[X]` procedure (for Guardian files) and the equivalent of the `select()` function (for OSS files).

For OSS files, `select()` is used to obtain ready information for OSS socket, OSSTTY, and disk descriptors. Completion for an OSS file means checking for readiness: that is, either data can be sent or received, or an exception has occurred.

The `select()` function checks the status of objects identified by bit masks called file descriptor sets. Each file descriptor set consists of an array of bits whose relative position and value represent file descriptors and the status of their corresponding objects. An object is an open file descriptor for an OSSTTY, a socket, an OSS regular file, a pipe, or a FIFO. There is a file descriptor set for reading, for writing, and for pending exceptions. OSS and Guardian processes can communicate within the same processor and between processors and between nodes using FIFOs. However, they cannot communicate using pipes. Pipes are used to communicate between processes and their processes

NOTE: Although an OSS socket descriptor is ready, a subsequent I/O operation might not finish successfully because sockets can be shared by multiple processes. If process A receives notice that the descriptor is ready for I/O, process B might alter that status before process A can perform an I/O operation on it.

For Guardian files, `AWAITIO[X]` is used to complete nowaited Guardian I/O operation. Completion for Guardian files implies that an I/O operation was initiated in a previous call to a read or write operation. This I/O operation is completed in `AWAITIO[X]` and data is returned to the application.

See the *Open System Services System Calls Reference Manual* and the *Guardian Procedure Calls Reference Manual* for more information.

Using `select()` on FIFOs demonstrates the use of `select()` on FIFOs.

Using `select()` on FIFOs

```
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/time.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <fcntl.h>

char *fifo[3] = { /* up to 3 named pipes */
    "fifo-1",
    "fifo-2",
    "fifo-3"
};

char *ofile[3] = { /* up to 3 output files */
    "ofile-1",
    "ofile-2",
    "ofile-3"
};

fd_set m_rfds;          /* master set of fd_set values */
fd_set s_rfds;          /* fd_set values used by select */
int maxfdp1;            /* Max. file descriptor number plus 1 */
int rd[3];              /* read/write file descriptors */
int fd[3];              /* fifo file descriptors */
int nrd[3];             /* number of bytes read */
```



```

int nread;           /* number of bytes read in one read */
int totread;         /* running subtotal number of bytes read */
int sumread;         /* total number of bytes read */
char buf[3][512];    /* data buffers */

main(int argc, char ** argv)
{
    int i;            /* counter */
    int cnt;          /* open fifo counter */
    int nfifo;        /* number of named pipes created */
    int p;            /* child process number */
    nfifo = argc - 1;
    if(nfifo < 1 || nfifo > 3) {
        fprintf(stderr, "Specify the names of 1 to 3 files to read.\n");
        exit(EXIT_FAILURE);
    }
    for(i = 0; i < nfifo; i++) {          /* create named pipes */
        unlink(fifo[i]);
        if(mkfifo(fifo[i], 0644) < 0) {   /* read/write permissions */
            fprintf(stderr, "Can't create fifo %s\n", fifo[i]);
            exit(EXIT_FAILURE);
        }
    }
    /* Create child process to read files whose names are passed as arguments
       to the program, and write the contents to the named pipes (fifos).*/

    if((p = fork()) < 0) {
        fprintf(stderr, "Can't create new child process.\n");
        exit(EXIT_FAILURE);
    } else if(p) {          /* Execute code for child process */
        printf("Parent process started.\n");
        for(i = 0; i < nfifo; i++)        /* open fifos for write */
            if((fd[i] = open(fifo[i], O_WRONLY)) < 0) {
                fprintf(stderr, "Can't open %s\n", fifo[i]);
                exit(EXIT_FAILURE);
            }
        for(i = 0; i < nfifo; i++)        /* open files for read */
            if((rd[i] = open(argv[i+1], O_RDONLY)) < 0) {
                fprintf(stderr, "Can't open %s\n", argv[i+1]);
                exit(EXIT_FAILURE);
            }
        sumread = 0;
        do {
            totread = 0;
            for(i = 0; i < nfifo; i++) {
                if(nrd[i] < 0)             /* don't read if end-of-file */
                    continue;
                if((nrd[i] = read(rd[i], buf[i], 512)) > 0) {
                    write(fd[i], buf[i], nrd[i]);
                    totread += nrd[i];
                } else
                    nrd[i] = -1;          /* set end-of-file marker */
            }
            sumread += totread;
        } while(totread);
        printf("Total bytes read from files = %d\n", sumread);
        return(EXIT_SUCCESS);           /* write of all fifos done */
    }
}

```

```

    }
/* Execute parent program to read the named pipes
   using the select() function and non-blocking I/O
*/
printf("Parent process code started.\n");
maxfdpl = 3;          /* Set highest descriptor number */
FD_ZERO(&m_rfds);
for(i = 0; i < nfifo; i++)          /* create output files */
    if((rd[i] = open(ofile[i], O_WRONLY | O_CREAT, 0644)) < 0) {
        fprintf(stderr, "Can't create %s\n", ofile[i]);
        exit(EXIT_FAILURE);
    }
for(i = 0; i < nfifo; i++) {
    if((fd[i] = open(fifo[i], O_RDONLY)) < 0) {
        fprintf(stderr, "Can't open fifo %s.\n", fifo[i]);
        exit(EXIT_FAILURE);
    }
    if(fcntl(fd[i], F_SETFL, O_NONBLOCK) < 0) {
        fprintf(stderr, "fcntl() error.\n");
        exit(EXIT_FAILURE);
    }
    if(fd[i] > maxfdpl)
        maxfdpl = fd[i];
    FD_SET(fd[i], &m_rfds);
}
maxfdpl++;          /* one more than max. file descriptor number */
sumread = 0;        /* initialize number of bytes read */
cnt = nfifo;        /* number of fifos still open */
while(cnt) {
    printf("select starting\n");
    s_rfds = m_rfds;
    if(select(maxfdpl, &s_rfds, NULL, NULL, NULL) <= 0) {
        fprintf(stderr, "select() error %d.\n", errno);
        exit(EXIT_FAILURE);
    }
    for(i = 0; i < nfifo; i++) {
        nread = 0;
        if(FD_ISSET(fd[i], &s_rfds)) {
            if((nread = read( fd[i], buf[i], 512)) < 0) {
                fprintf(stderr, "read fifo error %d\n", errno);
                exit(EXIT_FAILURE);
            }
            if(nread > 0) {          /* some data read */
                write(rd[i], buf[i], nread);
                sumread += nread;
            } else {                /* end of file */
                FD_CLR(fd[i], &m_rfds); /* stop reading from this file */
                close(fd[i]);
                --cnt;              /* number of fifos still open */
            }
        }
    }
}
/* end of for loop */
/* end of while loop */

printf("Total bytes read from fifo(s) = %d\n", sumread);

```

```

        return(EXIT_SUCCESS);
    }
                                     /* end of main */

```

Performance Considerations

The following actions improve performance when you use OSS file system functions:

NOTE: Some of these actions affect the configuration of the fileset or of the disk volume and are not controlled by the application.

- Minimize the number of open operations such as `open()` and `opendir()`.
 - Minimize the use of pipes and FIFOs between processors.
 - Minimize the use of `AF_UNIX` sockets between processes.
 - Minimize the use of shared opens (more than one process sharing an open file descriptor) between processors.
 - Minimize the use of operations that alter directory contents, such as `creat()`, `link()`, `unlink()`, and `rename()`.
 - Use file caching, if appropriate (see [File Caching](#)).
 - Use free-space table caching. For more information, see the *SCF Reference Manual for the Storage Subsystem*.
 - Use `O_SYNC` when reliability, not performance, is most important.
 - Prevent `stat()` calls by returning the file type information as part of directory entry (see [OSS Directory Traversal](#)).
-

NOTE: Hewlett Packard Enterprise recommends that the `S_NONSTOP` extension not be used in new applications. The `S_NONSTOP` extension is ignored on nodes running J-series RVUs or an RVU more recent than G06.26 or H06.03; the standard `O_SYNC` feature used with the `FTIOMODE` attribute provides better data integrity and improved performance.

Increase the size of the disk cache, if appropriate. If disk cache is too small, disk processes might perform unnecessary physical disk I/Os. If disk cache is too large, it can induce unnecessary memory swap operations and cache faults. (Swaps are also disk I/Os and therefore time consuming.) Typically, the default values for disk cache is small, so performance can be slow. There are a number of factors involved in determining the best disk cache size for your system. For more information, see the *SCF Reference Manual for the Storage Subsystem* and the *Measure User's Guide*.

File Caching

By default, the OSS environment provides a file cache for regular files in each processor that does input or output with a disk volume that contains OSS files. Hewlett Packard Enterprise strongly recommends that you leave OSS file caching enabled. This cache is necessary for the fault tolerant behavior controlled by the fileset `FTIOMODE` or `NORMALIOMODE` attributes.

If file caching is enabled, the OSS file system determines whether a file should be cached.

If OSS file caching is in effect, the OSS file system reads large blocks of data into its own processor cache buffer and then resolves user application read requests from that cache buffer. If OSS file caching is not in effect, the OSS file system does not perform any local caching of data and all requests are passed to the disk process.

If the OSS file system determines that a file should be cached and all opens of the file are for read access, then the file is cached on all processors that have opens of the file. The opens must be for read access only.

The caching state of a file is dynamic and can change as opens, closes, and other events occur on a file.



You should decide whether to use OSS file caching when you first configure a fileset, because changing the use of OSS file caching requires you to stop the fileset.

Although the storage subsystem can be used to enable or disable file caching for a volume, Hewlett Packard Enterprise recommends that you not modify OSS file caching at the disk level. The FTIOMODE and NORMALIOMODE fileset attributes provide better control over fault-tolerance and performance for file input or output.

FTIOMODE, NORMALIOMODE, Performance, and Fault Tolerance

The value used for the FTIOMODE and NORMALIOMODE options affect application performance and fault tolerance. Fault tolerance for files opened without using the O_SYNC bit is controlled by the NORMALIOMODE option setting; fault tolerance for files opened using the O_SYNC bit is controlled by the FTIOMODE option setting.

NOTE: Hewlett Packard Enterprise recommends that the S_NONSTOP extension not be used in new applications. The S_NONSTOP extension is ignored on nodes running J-series RVUs or an RVU more recent than G06.26 or H06.03; the standard O_SYNC feature used with the FTIOMODE attribute provides better data integrity and improved performance.

The attribute settings for NORMALIOMODE are, from highest to lowest fault tolerance and from lowest to highest performance:

- UNBUFFEREDCP
- DP2BUFFEREDCP
- OSSBUFFEREDCP
- DP2BUFFERED (Same as legacy OSS file caching turned off [OSSCACHING OFF])
- OSSBUFFERED (Same as legacy OSS file caching turned on [OSSCACHING ON])

If NORMALIOMODE is not specified, the default behavior is OSSBUFFEREDCP.

The FTIOMODE settings are a subset of the NORMALIOMODE settings. The setting used for the FTIOMODE attribute of a specific fileset must have a fault-tolerance level at least as high as that of the NORMALIOMODE attribute setting for the fileset. The attribute settings for FTIOMODE are, from highest to lowest fault tolerance and from lowest to highest performance:

- UNBUFFEREDCP
- DP2BUFFEREDCP
- OSSBUFFEREDCP

If FTIOMODE is not specified, the default behavior is UNBUFFEREDCP.

File caching and buffering affect the data integrity of a file and file transfer speed as described in the following table.

Table 7: I/O Buffering, Fault Tolerance, and Performance

NORMALIOMODE or FTIOMODE Attribute Value	Behavior	Fault Tolerance and Performance
UNBUFFEREDCP	Uses unbuffered input/output with checkpointing.	Provides maximum fault tolerance but with reduced performance.
DP2BUFFEREDCP	Uses disk-process-buffered input/output with checkpointing. DP2 buffers file data and checkpoints the file state to its backup process to ensure recovery from single failures.	Provides fault tolerance for single failures, with better performance than UNBUFFEREDCP.

Table Continued



NORMALIOMODE or FTIOMODE Attribute Value	Behavior	Fault Tolerance and Performance
OSSBUFFEREDCP	Uses OSS-buffered input/output with checkpointing. OSS filesystem processes and DP2 share responsibility for buffering file data; OSS provides the buffering whenever possible. DP2 buffers file data and checkpoints the file state to its backup process to ensure recovery from single failures.	Provides fault tolerance for single failures, with better performance than DP2BUFFEREDCP.
DP2BUFFERED	Uses disk-process-buffered input/output without checkpointing.	Provides no fault tolerance, but better performance than DPBUFFEREDCP.
OSSBUFFERED	Uses OSS-buffered input/output without checkpointing.	Provides no fault tolerance, but better performance than OSSBUFFEREDCP.

OSS Directory Traversal

Directory traversal requires an application to read directory contents, looking for target files and recursing through sub-directory structures as necessary. The OSS API that returns directory contents is the `readdir()` API. On NonStop, this requires a call to one of the `stat()` family of APIs to get the file type information from the directory structure they traverse. In T9228, new versions of `readdir*()` APIs are introduced. The new APIs return the file type information as part of the directory structure avoiding the use of `stat()` calls. The new APIs are:

- `readdir_dt()`
- `readdir_r_dt()`
- `readdir64_dt()`
- `readdir64_r_dt()`

For more information on the new `readdir*()` calls, see the corresponding reference pages.

Beginning with J06.20 and L16.05 RVUs, you can invoke the new APIs directly for faster directory traversal. However, if you still want to use the existing versions of the `readdir*()` calls, you can efficiently traverse directory structures using the new compile option `_D_TYPE_DIRENT_` in your code to get directory type information as part of the `readdir()` return information. By doing this, you can eliminate the need to call one of `stat()` family of APIs.

The following example shows how to search a directory tree for the current working directory:

Searching the directory tree using `readdir_dt()`

```
void op_dir(char * sfile) {

if (!getcwd(cwdir, MAX_DIR_PATH+1)) {
    perror("getcwd:");
    return;
}
if ((dir_ptr=opendir(".")) == NULL) {
    printf("Error trying to open file \n" );
}
else {
```

```

while ((dp = readdir_dt(dir_ptr)) != NULL) {
    if (!strcmp(dp->d_name, ".") || !strcmp(dp->d_name, ".."))
        continue;
    printf("%s/%s\n", cwdir, dp->d_name);
    if (dp->d_type == DT_DIR) {
        chdir(dp->d_name);
        op_dir(sfile);
    }
    else if (dp->d_type == DT_UNKNOWN) {
        if (stat(dp->d_name, &ft_details) != -1)
            if (S_ISDIR(ft_details.st_mode)) {
                chdir(dp->d_name);
                op_dir(sfile);
            }
    }
} /*while*/
closedir(dir_ptr);
chdir("..");
} /*else*/
} /* op_dir */

```

Performance and Reliability Tradeoffs

As table **Table 7: I/O Buffering, Fault Tolerance, and Performance** shows, you must choose between performance and reliability in OSS file I/O. The higher the reliability, the slower the performance.

For example, if you cannot afford to lose any file I/O operations, you should set the FTIOMODE and NORMALIOMODE attribute to UNBUFFEREDCP. If you have set this attribute and the primary processor fails, the disk process in the backup processor gets the information it needs to pick up where the disk process in the primary processor left off. This information includes any file locks and the read/write position in the file. Using UNBUFFEREDCP means that you do not lose any file I/O operations.

If DP2BUFFERED or OSSBUFFERED is used for NORMALIOMODE, the file is opened without using the O_SYNC bit, and there is a processor failure:

- A checkpoint does not occur between the disk process in the failed processor and the disk process in the backup processor.
- The file descriptor becomes invalid, which means you lose the read/write position in the file. This happens whether file caching is on or off.

You can reset the read/write position in the file by closing the file with the invalid file descriptor, reopening it, and resuming I/O operations. However, if file caching was off, you lose the file operation in process when the processor failed. If file caching was on, you lose all file operations in the cache when the processor failed.

OSS I/O performance is affected by the following:

- Whether free-space table caching is enabled or disabled for the disk process
- Whether OSS file caching is enabled or disabled
- Using the fast-create option when creating a fileset
- Having a volume list for filesets in the superblock
- Setting the trackall flag in the /etc/profile file to create tracked aliases
- Leaving too many unused processes running



OSS File-System Functions

Table **Table 8: OSS File-System Functions** displays information about each OSS function that you can use to manage files. The columns of the table contain the following:

OSS Function

The name of the function and a brief description.

OSS Notes

Notes about the OSS implementation of the function. The note “Can return extended errors” means that the function can return errors that are Hewlett Packard Enterprise extensions to the XPG4 specification. Refer to the function’s reference page for further information.

Beginning with the J06.15 and H06.26 RVUs, a partner or customer OSS Security Event-Exit Process (SEEP) is supported and can participate in access-control decisions for OSS objects. (For OSS SEEP details, see **Accessing OSS SEEP-Protected Files**.) The note “OSS SEEP consultation” indicates that the OSS name server consults the OSS SEEP when this function is called for operations on files in OSS SEEP-protected filesets and the OSS SEEP is running.

Guardian Notes

Notes about using the function when it is called from a Guardian process.

The note “FDM set” (for file, directory, and memory) means that the function is one of a set of functions that have the following effects when the first of them is called from the Guardian environment:

- Two Guardian file-system numbers (not necessarily the next two available) are allocated for the root directory and the current working directory. These file numbers cannot be closed by calling the Guardian FILE_CLOSE_ procedure.
- The current working directory is assigned from the VOLUME attribute of the Guardian environment =_DEFAULTS DEFINE.
- The use of static memory by the process increases slightly.
- The functions in the FDM set are `access()`, `chdir()`, `chmod()`, `chown()`, `chroot()`, `creat()`, `creat64()`, `faccessat()`, `fchdir`, `fchmod()`, `fchmodat()`, `fchown()`, `fchownat()`, `fdopendir()`, `fstatat()`, `fstatat64()`, `fstatvfs()`, `fstatvfs64()`, `ftok()`, `ftw()`, `ftw64()`, `getcwd()`, `glob()`, `lchmod()`, `lchown()`, `link()`, `linkat()`, `lstat()`, `lstat64()`, `mkdir()`, `mkdirat()`, `mkfifo()`, `mkfifoat()`, `mknod()`, `mknodat()`, `nftw()`, `nftw64()`, `open()`, `openat()`, `open64()`, `openat64()`, `opendir()`, `pathconf()`, `pipe()`, `pread()`, `pread64_()`, `pwrite()`, `pwrite64_()`, `readlink()`, `readlinkat()`, `rename()`, `renameat()`, `rename_oss()`, `rmdir()`, `select()`, `socket()`, `socketpair()`, `stat()`, `stat64()`, `statvfs()`, `statvfs64()`, `symlink()`, `symlinkat()`, `unlink()`, and `utime()`.

The information in this table is only a summary of the information in the reference page for a function. For details, refer to the reference pages either online or in the *Open System Services System Calls Reference Manual* and the *Open System Services Library Calls Reference Manual*.

NOTE: To determine which RVUs support an OSS function, see **Documented OSS Functions**.



Table 8: OSS File-System Functions

OSS Function	OSS Notes	Guardian Notes
<code>access()</code> <code>faccessat()</code> Determines the accessibility of a file.	Can return extended errors. The <code>faccessat()</code> function is similar to the <code>access()</code> function except that it also supports <i>fd-relative</i> pathnames. OSS SEEP consultation.	FDM set. Not all the Guardian information can be summarized here; for example, there are several security restrictions. For Guardian information, refer to the reference pages.
<code>chdir()</code> <code>fchdir()</code> Changes the current working directory.	Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> . The <code>fchdir()</code> function is similar to the <code>chdir()</code> function except that the directory is specified by a file descriptor rather than a pathname. OSS SEEP consultation.	FDM set. You can't use <code>chdir()</code> on Guardian process names or subvolumes with a reserved name.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>chmod()</code> <code>fchmod()</code> <code>lchmod()</code> <code>fchmodat()</code> Changes file-access permissions.	Process must have the appropriate privilege. For files that are not in restricted-access filesets, the process must have either the same effective user ID as the owner of the file or the super ID. For files in restricted-access filesets, see <u>Restricted-Access Filesets and File Privileges</u>. Call to <code>chmod()</code> has no effect on descriptor of open file, but new openers of the file are authenticated with the new permissions specified in the call. The <code>fchmod()</code> function is similar to the <code>chmod()</code> function except that it operates on a file descriptor instead of a pathname. The <code>lchmod()</code> function is similar to the <code>chmod()</code> function except when the final component of the path parameter refers to a symbolic link, the <code>lchmod()</code> function changes access permissions for the symbolic link instead of the file to which it refers. The <code>fchmodat()</code> function is similar to the <code>chmod()</code> function except that it also supports <i>fd-relative</i> pathnames. The mode parameter uses symbols. Refer to the reference page. Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code>. OSS SEEP consultation.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>chown()</code> <code>fchown()</code> <code>lchown()</code> <code>fchownat()</code> Changes the owner and group IDs of a file.	The <code>fchown()</code> function is similar to the <code>chown()</code> function except that it operates on a file descriptor instead of a pathname. The <code>lchown()</code> function is similar to the <code>chown()</code> function except when the final component of the path parameter refers to a symbolic link, the <code>lchown()</code> function changes access permissions for the symbolic link instead of the file to which it refers. The <code>fchownat()</code> function is similar to the <code>chown()</code> function except that it also supports <i>fd-relative</i> pathnames. Process must have the appropriate privilege. For files that are not in restricted-access filesets, the process must have either the same effective user ID as the owner of the file or the super ID. For files in restricted-access filesets, see <u>Restricted-Access Filesets and File Privileges</u>. Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code>. OSS SEEP consultation.	FDM set. Can be used only on files in the <code>/G</code> fileset. The <code>_POSIX_CHOWN_RESTRICTED</code> constant is ignored for files in <code>/G</code>. For details on disk file security, refer to the reference pages.
<code>chroot()</code> Changes the effective root directory.	Can return extended errors. OSS SEEP consultation.	FDM set. When called with a <code>path</code> parameter that points to a Guardian subvolume with a reserved name, it fails.
<code>close()</code> Closes the file associated with a file descriptor.	Does not return <code>EINTR</code>. Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code>.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>closedir()</code> Closes a directory stream. Refer to <code>opendir()</code> .	Can return extended errors. If the <code>dir_pointer</code> parameter does not refer to a currently opened directory stream when it is passed to the <code>closedir()</code> function, the following can occur: - The caller might receive a <code>SIGSEGV</code> or <code>SIGILL</code> signal. - User memory might be corrupted.	If the <code>dir_pointer</code> parameter does not refer to a currently opened directory stream when it is passed to the <code>closedir()</code> function, the following can occur: - The caller might cause a trap condition. - User memory might be corrupted.
<code>confstr()</code> Determines the current value of a specified system variable defined by a string value.		
<code>creat()</code> <code>creat64()</code> Creates a file. Refer to <code>open()</code> .	Can return Guardian file-system error numbers in <code>errno</code> . <code>creat64()</code> can access both small files and large files. OSS SEEP consultation.	FDM set. Refer to <code>open()</code> .
<code>ctermid()</code> Generates the pathname for a controlling terminal.	If the parameter is a null pointer, the string is stored in an internal static area and the address is returned. The next call to this function with a null pointer overwrites the contents of the internal static area. Can return extended errors.	For Guardian processes, it returns a null pointer and <code>errno</code> is set to <code>ENOTOSS</code> .
<code>dup()</code> Duplicates an open file descriptor.	Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> .	
<code>dup2()</code> Duplicates and controls an open file descriptor.	Does not return <code>EINTR</code> . Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> .	
<code>fclose()</code> Closes a stream.	Can return extended errors. Can return <code>ENXIO</code> , whose value is an optional extension defined in the XPG4 specification but is not defined in the POSIX standards.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>fcntl()</code> Controls open file descriptors.	Does not return <code>EDEADLK</code> . Advisory record locking is supported only for regular files; attempts on other files give <code>EINVAL</code> . <code>l_len</code> parameter can be negative. Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> .	Access to the file through a Guardian procedure call cannot be blocked by an OSS lock. Guardian use of OSS sockets is not supported.
<code>fopen()</code> <code>fopen64()</code> Opens a file and associates a stream with it.	Can return Guardian file-system error numbers in <code>errno</code> . <code>fopen64()</code> can access both small files and large files OSS SEEP consultation.	
<code>fopen_guardian()</code> <code>fopen64_guardian()</code> Opens a stream (Guardian <code>fopen()</code> function).	Can return Guardian file-system error numbers in <code>errno</code> . <code>fopen64_guardian()</code> can access both small files and large files.	Distinguishes between text and binary files. Explicit calls are made only when the unique behavior is desired. Otherwise use <code>fopen()</code> .
<code>fopen_oss()</code> <code>fopen64_oss()</code> Opens a stream (OSS <code>fopen()</code> function).	Does not distinguish between text and binary files. Explicit calls are made only when the unique behavior is desired. Otherwise use <code>fopen()</code> . Can return Guardian file-system error numbers in <code>errno</code> . <code>fopen64_oss()</code> can access both small files and large files. OSS SEEP consultation.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
fpathconf() Refer to <code>pathconf()</code> .	Can return extended errors.	FDM set. A call to <code>pathconf()</code> for a regular file or terminal character file in <code>/G</code> succeeds for any of the following constants: <pre> _PC_CHOWN_RESTRICTED _PC_LINK_MAX _PC_NAME_MAX _PC_PATH_MAX _PC_PIPE_BUF </pre> If the file named by the <i>path</i> parameter is an empty Guardian subvolume, <code>errno</code> is not set to <code>ENOENT</code> .
fread() Performs input operations.	Can return Guardian file-system error numbers in <code>errno</code> .	
fstat() fstat64() Refer to <code>stat()</code> .	Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> . fstat64() can access both small files and large files.	
fstatvfs() fstatvfs64() Gets fileset information for an open file.	Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> . fstatvfs64() can access both small files and large files.	FDM set.
fsync() Writes modified data and file attributes to permanent storage.	Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> .	Time values are not saved for file types other than regular files in <code>/G</code> .
ftruncate() ftruncate64() Changes file length.	Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> . ftruncate64() can access both small files and large files.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>ftw()</code> <code>ftw64()</code> Traverses a file tree.	Does not return <code>EINVAL</code> . Can return extended errors. Recursive access from <code>/</code> can cause unexpected results. <code>ftw64()</code> can access both small files and large files.	FDM set. All files in <code>/G</code> that are visible to <code>opendir()</code> can be traversed.
<code>fwrite()</code> Writes to an output stream.	Can return Guardian file-system error numbers in <code>errno</code> .	
<code>getcwd()</code> Gets the pathname of the current directory.	If <i>buffer</i> is a null pointer, <code>getcwd()</code> uses <code>malloc()</code> to allocate a buffer. In that case, the caller releases the buffer with <code>free()</code> . Can return extended errors.	FDM set. When a Guardian process makes its first OSS call with a pathname, the process's current working directory becomes the <code>/G</code> version of the default subvolume as contained in the <code>VOLUME</code> attribute of the Guardian <code>=_DEFAULTS DEFINE</code> . Changes to the <code>=_DEFAULTS DEFINE</code> after this point do not affect the current working directory, and changes to the current working directory of the process do not affect the <code>=_DEFAULTS DEFINE</code> .
<code>globfree()</code> Frees all memory associated with a previous call to <code>pglob()</code> .	Can be called only by native processes.	
<code>ioctl()</code> Controls device files.	Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> .	A Guardian process cannot receive the <code>SIGWINCH</code> signal. The <i>files</i> parameter can specify a terminal file in <code>/G</code> .
<code>link()</code> <code>linkat()</code> Creates an additional directory entry for an existing file on the current file system.	Linked pathnames must reside on the same fileset. The <code>linkat()</code> function is similar to the <code>link()</code> function except that it also supports <i>fd-relative</i> pathnames. Not supported for directories. Can return extended errors. OSS SEEP consultation.	FDM set. Cannot create a link to a Guardian file.

Table Continued

OSS Function	OSS Notes	Guardian Notes
<code>lseek()</code> <code>lseek64()</code> Sets the file offset for read or write operations.	If called for nonseekable device, <code>errno</code> is set to <code>EINVAL</code>, except for pipes and FIFOs, in which case <code>errno</code> is set to <code>EISPIPE</code>. Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code>. <code>lseek64()</code> can access both small files and large files.	
<code>lstat()</code> <code>lstat64()</code> Provides information about a symbolic link or any file.	For files other than regular disk files, <code>st_size</code> is set to 0 (zero), and for directories, to 4096. The following bits are ORed into the <code>st_mode</code> field of the <code>stat</code> structure: <code>S_IRWXU</code> <code>S_IRWXG</code> <code>S_IRWXO</code> <code>S_IFMT</code> <code>S_ISVTX</code> <code>S_NONSTOP</code> <code>S_ISGID</code> <code>S_ISUID</code> Can return extended errors. <code>lstat64()</code> can access both small files and large files.	FDM set. There are security and permission mapping considerations. Refer to the reference page for detailed information about using <code>stat()</code> on Guardian files.

Table Continued



OSS Function	OSS Notes	Guardian Notes
mkdir() mkdirat() Creates a directory.	Can not create <code>/lost+found</code> or <code>/dev</code> in the root directory of a fileset or <code>/dev/tty</code> or <code>/dev/null</code>. If there are bits in the mode parameter other than permission bits <code>S_ISVTX</code> and <code>S_IFDIR</code>, <code>errno</code> is set to <code>EINVAL</code>, or the bits are undefined. The group ID of the new directory is set to the group ID of the parent directory if <code>S_ISGID</code> is set there, otherwise, the group ID is set to the effective group ID of the caller. The <code>mkdirat()</code> function is similar to the <code>mkdir()</code> function except that it also supports <i>fd-relative</i> pathnames. Can return extended errors. OSS SEEP consultation.	Succeeds only when creating an empty subvolume exactly three directories under root (<code>/G/vol/subvol</code>). The resulting directory is owned by the super ID, and all permission bits are set (<code>rw-rw-rwx</code>). The group ID is set to 255.
mknod() mknodat() Creates a file or assigns a pathname to a character special file.	The file type <code>S_IFBLK</code> is not a valid file type. The file attribute <code>S_NONSTOP</code> is supported. The <code>mknodat()</code> function is similar to the <code>mknod()</code> function except that it also supports <i>fd-relative</i> pathnames. Can return extended errors. OSS SEEP consultation.	FDM set.
mkstemp() Makes a unique filename.	Can be called by native processes.	Can be called by native processes.
nftw() nftw64() Traverses a file tree.	Can return extended errors. Recursive access from <code>/</code> can cause unexpected results. Can return Guardian file-system error numbers in <code>errno</code>. <code>nftw64()</code> can access both small files and large files.	FDM set. Guardian files that are visible to the <code>opendir()</code> function can be traversed. Security controls on Guardian object do not affect the outcome of a call to <code>nftw()</code>.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>open()</code> <code>open64()</code> <code>openat()</code> Opens a file for reading or writing or creates a file in the OSS environment.	Many flags are implementation defined. Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code>. <code>open64()</code> can access both small files and large files. The <code>openat()</code> function is similar to the <code>open()</code> function except that it also supports <i>fd-relative</i> pathnames. OSS SEEP consultation.	FDM set. There is too much Guardian information to summarize here. For Guardian information, refer to the reference pages and to <u>Accessing OSS Files Larger Than 2 GB</u>.
<code>opendir()</code> <code>fdopendir()</code> Performs operations on directories.	Can return extended errors. There is too much OSS information about the Hewlett Packard Enterprise implementation of this function to summarize here. For OSS information, refer to the reference pages. The <code>fdopendir()</code> function is similar to the <code>opendir()</code> function except that directory is specified by a file descriptor rather than a pathname. OSS SEEP consultation.	FDM set. There is too much Guardian information about using this call on <code>/G</code> and its contents to summarize here. For detailed Guardian information, refer to the reference pages.
<code>pathconf()</code> Retrieves file implementation characteristics.	Can return extended errors.	FDM set. A call to <code>pathconf()</code> for a regular file or terminal character file in <code>/G</code> succeeds for any of the following constants: <pre> _PC_CHOWN_RESTRICTED _PC_LINK_MAX _PC_NAME_MAX _PC_PATH_MAX _PC_PIPE_BUF </pre> If the file named by the <i>path</i> parameter is an empty Guardian subvolume, <code>errno</code> is not set to <code>ENOENT</code>.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>pipe()</code> Creates an interprocess-communication channel.	Can return Guardian file-system error numbers in <code>errno</code> .	
<code>pread()</code> <code>pread64_()</code> Read from a file	The <code>pread*()</code> functions are like the <code>read*()</code> functions except that they read from the given position in the file without changing the current file offset and are only supported on regular files. If <code>nbytes > SSIZE_MAX</code>, <code>pread()</code> returns -1 and <code>errno</code> is set to EINVAL. <code>errno</code> can be set to EIO if a physical I/O error occurs. Can return extended errors.	
<code>pwrite()</code> <code>pwrite64_()</code> Writes to a file.	The <code>pwrite*()</code> functions are like the <code>write*()</code> functions except that they write to the given position in the file without changing the current file offset and are only supported on regular files. If <code>nbytes > SSIZE_MAX</code>, <code>pwrite()</code> returns -1 and <code>errno</code> is set to EINVAL. <code>errno</code> can be set to EIO if a physical I/O error occurs. Can return extended errors.	
<code>read()</code> Reads from a file.	The value of the file pointer returned for a device that is incapable of seeking is always 0. The return of EOF from a device special file has no effect on subsequent <code>read()</code> calls. If <code>nbytes > SSIZE_MAX</code>, -1 is returned and <code>errno</code> is set to EINVAL. Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code>.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>read64_()</code> Reads from a file.	Available beginning with the H06.24 and J06.13 RVUs. Can be called with either a 32-bit or 64-bit pointer argument. The value of the file pointer returned for a device that is incapable of seeking is always 0. The return of EOF from a device special file has no effect on subsequent <code>read()</code> calls. If <code>nbytes > SSIZE_MAX</code>, -1 is returned and <code>errno</code> is set to <code>EINVAL</code>. Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code>.	
<code>readdir()</code> <code>readdir_dt()</code> <code>readdir64()</code> <code>readdir64_dt()</code> Refer to <code>opendir()</code> .	<code>readdir()</code> can return extended errors. If, after a call to <code>fork()</code> or <code>tdm_fork()</code>, both the parent and the child processes make calls to <code>readdir()</code> to access the same OSS directory, the results of the calls are undefined. <code>readdir64()</code> can access both small files and large files. <code>readdir_dt()</code> and <code>readdir64_dt()</code> are functionally similar to the <code>readdir()</code> and <code>readdir64()</code>, but return the new dirent structure that includes file type information. If the file type is not available, the application returns <code>DT_UNKNOWN</code> in the dirent structure.	
<code>readlink()</code> Reads the value of a symbolic link.	Can return extended errors.	FDM set. This function cannot be used on an object in <code>/G</code> because symbolic links cannot be created in <code>/G</code>.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>readv()</code> Reads from a file into scattered buffers.	Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> .	
<code>remove()</code> Removes a file.	If <code>remove()</code> is passed the name of an open file, that file is no longer accessible by that name. Existing opens remain in effect. When there are no more outstanding opens, the physical file is removed. OSS SEEP consultation.	
<code>rename()</code> or <code>rename_oss()</code> <code>renameat()</code> Renames a directory or file within a fileset from the OSS environment.	OSS SEEP consultation. The <code>renameat()</code> function is similar to the <code>rename_oss()</code> function except that it also supports <i>fd-relative</i> pathnames.	FDM set. In /G, this function can be used on files but cannot be used on directories. The new pathname must correspond to a Guardian permanent disk filename on the same volume, and the caller must have Guardian write access to the file.
<code>rename_guardian()</code> Renames a file in the Guardian file system.	Can return Guardian file-system error numbers in <code>errno</code> .	Can be used only on Guardian files in /G, and the caller must have Guardian write access to the file.
<code>rewinddir()</code> Repositions a directory stream to its beginning. Refer to <code>opendir()</code> .	If, after a call to <code>fork()</code> or <code>tdm_fork()</code> , both the parent and the child processes make calls to <code>rewinddir()</code> to access the same OSS directory, the results of the calls are undefined.	
<code>rmdir()</code> Removes a directory.	Fails on <code>lost+found</code> in root directory of a fileset and on /dev. <code>rmdir()</code> can be used to remove the current working directory of a process. This call sets <code>errno</code> to <code>EBUSY</code> when the directory is a mount point. Can return extended errors. OSS SEEP consultation.	FDM set. Fails on disk volumes or processes in /G and <code>errno</code> is set to <code>EBUSY</code> .

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>scandir()</code> <code>scandir64()</code> Scans a directory and builds an array of pointers to copies of selected directory entries.	<code>scandir64()</code> can access both small files and large files.	
<code>seekdir()</code> Sets the position in a directory stream. Refer to <code>opendir()</code> .	Can return extended errors. If, after a call to <code>fork()</code> or <code>tdm_fork()</code> , both the parent and the child processes make calls to <code>seekdir()</code> to access the same OSS directory, the results of the calls are undefined.	
<code>select()</code> Selects among file descriptors for synchronous I/O multiplexing.	Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> .	FDM set.
<code>socket()</code> Creates an endpoint for communications.	Can return extended errors.	FDM set.
<code>socketpair()</code> Creates an endpoint for communications.	Can return extended errors.	FDM set.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>stat()</code> <code>stat64()</code> <code>fstatat()</code> <code>fstatat64()</code> Provides information about a <code>file.stat()</code> function.	For files other than regular disk files, <code>st_size</code> is set to 0 (zero), and for directories, to 4096. The <code>fstat*()</code> functions are similar to the <code>stat*()</code> functions except that it also supports <i>fd-relative</i> pathnames. The following bits are ORed into the <code>st_mode</code> field of the <code>stat</code> structure: <code>S_IRWXU</code> <code>S_IRWXG</code> <code>S_IRWXO</code> <code>S_IFMT</code> <code>S_ISVTX</code> <code>S_NONSTOP</code> <code>S_ISGID</code> <code>S_ISUID</code> Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> . <code>stat64()</code> can access both small files and large files.	FDM set. There are security and permission mapping considerations. Refer to the reference page for detailed information about using <code>stat()</code> on Guardian files.
<code>statvfs()</code> <code>statvfs64()</code> Gets fileset information using a <code>pathname</code> .	<code>statvfs64()</code> can access both small files and large files.	FDM set.
<code>symlink()</code> <code>symlinkat()</code> Makes a symbolic link to a file.	The <code>symlinkat()</code> function is similar to the <code>symlink()</code> function except that it also supports <i>fd-relative</i> pathnames. Can return extended errors. OSS SEEP consultation.	FDM set. This function cannot be used on an object in <code>/G</code> because symbolic links cannot be created in <code>/G</code> .
<code>telldir()</code> Returns the current location of a directory stream.	Can return extended errors.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>ulimit()</code>	Can return extended errors.	
Sets and gets file size limits.	The error <code>EINVAL</code> is returned when the second parameter is too large.	
<code>umask()</code>		
Sets and gets the value of the file mode creation mask.		
<code>unlink()</code>	The calling process requires both execute (search) and write access permission for the directory of the file being unlinked.	FDM set.
<code>unlinkat()</code>	The <code>unlinkat()</code> function is similar to the <code>unlink()</code> function except that it also supports <i>fd-relative</i> pathnames.	
Removes a directory entry.	<code>unlink()</code> is not supported for directories.	
	Can return extended errors.	
	OSS SEEP consultation.	
<code>utime()</code>	Can return extended errors.	FDM set.
Sets the file access and modification times.	OSS SEEP consultation.	The file access time is not updated for I/O operations performed on a file that is opened in the Guardian environment.
<code>write()</code>	Calls to <code>write()</code> with <i>nbytes</i> =0 are supported for all files.	Attempting to write to a Guardian file that is locked with Guardian locks returns -1 and sets <code>errno</code> to <code>EGUARDIANLOCKED</code> .
Writes to a file.	If <i>nbytes</i> > <code>SSIZE_MAX</code> , <code>write()</code> returns -1 and <code>errno</code> is set to <code>EINVAL</code> .	
	<code>errno</code> can be set to <code>EIO</code> if a physical I/O error occurs.	
	Can return extended errors.	
	Can return Guardian file-system error numbers in <code>errno</code> .	

Table Continued



OSS Function	OSS Notes	Guardian Notes
write64_() Writes to a file.	Available beginning with the H06.24 and J06.13 RVUs. Can be called with either a 32-bit or 64-bit pointer argument. Calls to <code>write64_()</code> with <code>nbytes=0</code> are supported for all files. If <code>nbytes > SSIZE_MAX</code> , <code>write64_()</code> returns -1 and <code>errno</code> is set to <code>EINVAL</code> . <code>errno</code> can be set to <code>EIO</code> if a physical I/O error occurs. Can return extended errors. Can return Guardian file-system error numbers in <code>errno</code> .	Attempting to write to a Guardian file that is locked with Guardian locks returns -1 and sets <code>errno</code> to <code>EGUARDIANLOCKED</code> .
writenv() Writes to a file from scattered buffers.	Can return Guardian file-system error numbers in <code>errno</code> .	Attempting to write to a Guardian file that is locked with Guardian locks returns -1 and sets <code>errno</code> to <code>EGUARDIANLOCKED</code> .

Guardian File-System Procedures

The following table displays information about each Guardian procedure that you can use to manage files. Procedures that have been superseded by other procedures are not shown. The columns of the table contain the following:

Guardian Procedure

The name of the procedure and a brief description.

OSS Notes

Additional considerations or restrictions when the Guardian procedure is used on OSS files.

For details about a procedure, refer to the *Guardian Procedure Calls Reference Manual*.

Table 9: Guardian File-System Procedures

Guardian Procedure	OSS Notes
AWAITIO[X] Checks for completion of read operations pending on the \$RECEIVE file.	
FILE_ALTERLIST_ Changes certain attributes of a Guardian disk file that are normally set when the file is created.	Fails with file-system error 1163 when the file is an OSS file.

Table Continued



FILE_AWAITIO64_ Beginning with the H06.24 and J06.13 RVUs, checks for completion of read operations pending on the \$RECEIVE file. The FILE_AWAITIO64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the AWAITIOXL procedure.	
FILE_COMPLETE_ Completes one previously initiated I/O operation for a Guardian file or returns ready information for one OSS file.	For an OSS file descriptor: The <code>Z^Count^Tranferred</code> field is overlaid by <code>Z^Completion^Type</code> and the <code>Z^Tag</code> field contains 0D. Completion on an OSS file descriptor means checking for readiness. The operation of checking for readiness is equivalent to calling the OSS <code>select()</code> function. Can be interrupted by an OSS signal, which returns an error 4004.
FILE_COMPLETE_GETINFO_ Provides information about the set of files that are enabled for completion by FILE_COMPLETE_.	
FILE_COMPLETE_SET_ Enables a set of Guardian and OSS files for completion by subsequent calls to FILE_COMPLETE_.	For OSS, the type of operation (read, write, or exception) can be specified. Disabling a file descriptor of -1 means no OSS descriptors will be completed in consecutive FILE_COMPLETE_ calls; all OSS descriptors will be removed from the set of enabled files.
FILE_CONTROL64_ Beginning with the H06.24 and J06.13 RVUs, performs device-dependent I/O operations. The FILE_CONTROL64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the CONTROL procedure.	
FILE_CONTROLBUF64_ Beginning with the H06.24 and J06.13 RVUs, performs device-dependent I/O operations requiring a data buffer. The FILE_CONTROLBUF64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the CONTROLBUF procedure.	
FILE_CREATE_ Creates a Guardian disk file specifying commonly used disk file properties.	Fails with file-system error 1163 when the file is an OSS file.

Table Continued



FILE_CREATELIST_ Creates a Guardian disk file specifying properties in addition to commonly used disk file properties.	Fails with file-system error 1163 when the file is an OSS file.
FILE_GETINFO_ Gets information about a file by number.	Bit 15 of the <i>flags</i> parameter = 0 if the file is a Guardian file, 1 if the file is an OSS file.
FILE_GETINFOBYNAME_ Gets information about a file by name.	Bit 15 of the <i>flags</i> parameter = 0 if the file is a Guardian file, 1 if the file is an OSS file.
FILE_GETINFOLIST_ Gets detailed information about a file.	If the <i>item-list</i> parameter specifies item 161 and the <i>result</i> parameter returns 1, the file is an OSS file. For systems running J-series RVUs, H06.08 and later H-series RVUs, or G06.29 and later G-series RVUs only, if the <i>item-list</i> parameter specifies item 169 and the <i>result</i> parameter returns 11 bit on, OSS ACL file security is in effect. Access to record position pointers in 32-bit fields (item codes 11 and 12) using this API fails with the error FEBIGKEYS on an OSS file that was opened using 64-bit elections. An alternative is to access the record position pointers in 64-bit fields (item codes 201 and 202). Other item-list parameter items are also specific to OSS files. Refer to procedure description in the <i>Guardian Procedure Calls Reference Manual</i> for OSS information.
FILE_GETINFOLISTBYNAME_ Gets detailed information about a file by name.	If the <i>item-list</i> parameter specifies item 161 and the <i>result</i> parameter returns 1, the file is an OSS file. Access to record position pointers in 32-bit fields (item codes 11 and 12) using this API fails with the error FEBIGKEYS on an OSS file that was opened using 64-bit elections. An alternative is to access the record position pointers in 64-bit fields (item codes 201 and 202).
FILE_GETLOCKINFO_ Gets information about locks held or pending on a local disk file or local disk volume.	This procedure operates only on Guardian files. Error 0 is returned if the file is an OSS file.
FILE_GETOPENINFO_ Gets information about the opens of one disk file on a disk device, all the opens on a disk device, or the opens of certain nondisk devices.	When FILE_GETOPENINFO_ is given a Guardian filename that refers to an OSS file, the primary process handle returned is for the initial opener. The process handle might no longer exist.

Table Continued



FILE_LOCKFILE64_ Beginning with the H06.24 and J06.13 RVUs, excludes other users from accessing a file (and any records within that file). The FILE_LOCKFILE64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the LOCKFILE procedure.	
FILE_LOCKREC64_ Beginning with the H06.24 and J06.13 RVUs, excludes other users from accessing a record at the current position. The FILE_LOCKREC64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the LOCKREC procedure.	
FILE_OPEN_ Opens a file.	Set <i>options</i>. <10> to 1 to open an OSS file by its OSS pathname. OSS files can be opened only with shared exclusion mode. See also <u>Accessing OSS Files Larger Than 2 GB</u>. OSS SEEP consultation, beginning with the J06.15 and H06.26 RVUs. For OSS SEEP details, see <u>Accessing OSS SEEP-Protected Files</u>.
FILE_OPEN_CHKPT_ Opens a designated file in the backup Guardian process.	Always fails for OSS processes.
FILE_PURGE_ Deletes a disk file that is not open.	Fails with file-system error 1163 when the file is an OSS file.
FILE_READ64_ Beginning with the H06.24 and J06.13 RVUs, returns data from an open file to the application process's data area. The FILE_READ64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the READX procedure.	
FILE_READLOCK64_ Beginning with the H06.24 and J06.13 RVUs, sequentially locks and reads records in a disk file. The FILE_READLOCK64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the READLOCKX procedure.	

Table Continued



FILE_READUPDATE64_

Beginning with the H06.24 and J06.13 RVUs, reads data from a disk or process file in anticipation of a subsequent write to the file. The FILE_READUPDATE64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the READUPDATEXL procedure.

FILE_READUPDATELOCK64_

Beginning with the H06.24 and J06.13 RVUs, used for random processing of records in a disk file. The FILE_READUPDATELOCK64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the READUPDATELOCKX procedure.

FILE_RENAME_

Changes the name of an open disk file.

Fails with file-system error 564 when the file is an OSS file.

FILE_REPLY64_

Beginning with the H06.24 and J06.13 RVUs, sends a reply message to a message received earlier in a corresponding call to READUPDATE[X]XL/FILE_READUPDATE64_ on the \$RECEIVE file. The FILE_REPLY64_ procedure can be called even if there are incomplete nowait I/O operations pending on \$RECEIVE. The FILE_REPLY64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the REPLYXL procedure.

FILE_SETMODENOWAIT64_

Beginning with the H06.24 and J06.13 RVUs, used to set device-dependent functions in a nowait manner on nowait files. The FILE_SETMODENOWAIT64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the SETMODENOWAIT procedure.

FILE_UNLOCKFILE64_

Beginning with the H06.24 and J06.13 RVUs, unlocks a disk file and any records in the file currently locked by the user. The FILE_UNLOCKFILE64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the UNLOCKFILE procedure.

Table Continued



FILE_UNLOCKREC64_
Beginning with the H06.24 and J06.13 RVUs, unlocks a record currently locked by the user. The FILE_UNLOCKREC64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the UNLOCKREC procedure.

FILE_WRITE64_
Beginning with the H06.24 and J06.13 RVUs, writes data from an array in the application program to an open file. The FILE_WRITE64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the WRITEX procedure.

FILE_WRITEREAD64_
Beginning with the H06.24 and J06.13 RVUs, for files opened in a no-waited manner, control is returned to the caller after the request is initiated. In this case, the count read is not returned and the caller needs to call FILE_AWAITIO64_O or FILE_COMPLETEL_O to complete the I/O and obtain the count read. For files opened in a waited manner, the completion is waited for, at which time the data has been transferred to the caller's buffer. Extends the capabilities of the FILE_WRITEREAD_ procedure.

FILE_WRITEUPDATE64_
Beginning with the H06.24 and J06.13 RVUs, transfers data from an array in the application program to a file. The FILE_WRITEUPDATE64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the WRITEUPDATEX procedure.

FILE_WRITEUPDATEUNLOCK64_
Beginning with the H06.24 and J06.13 RVUs, used for random processing of records in a disk file. The FILE_WRITEUPDATEUNLOCK64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the WRITEUPDATEUNLOCKX procedure.

Table Continued



FILENAME_FINDNEXT64_ Beginning with the H06.24 and J06.13 RVUs, returns the next name in a set of named entities that was defined by a call to the FILENAME_FINDSTART_ procedure. The FILENAME_FINDNEXT64_ procedure is intended for use with 64-bit extended addresses, but it can be used with 32-bit addresses as well. Extends the capabilities of the FILENAME_FINDNEXT_ procedure.	
FILENAME_TO_PATHNAME_ Converts a Guardian filename into an OSS pathname.	There is too much OSS information to summarize here. Refer to procedure description in the <i>Guardian Procedure Calls Reference Manual</i> for OSS information.
LOCKFILE Excludes other users from access to a Guardian file.	Fails with file-system error 2 when the file is an OSS file.
LOCKREC Excludes other users from access to a record.	Fails with file-system error 2 when the file is an OSS file.
PATHNAME_TO_FILENAME_ Maps an OSS pathname to the name of the Guardian ZYQ filename of the corresponding regular file.	There is too much OSS information to summarize here. Refer to procedure description in the <i>Guardian Procedure Calls Reference Manual</i> for OSS information.
READLOCK[X] Sequentially locks and reads records in a Guardian disk file, just like combinations of LOCKREC and READ[X].	Fails with file-system error 2 when the file is an OSS file.
READUPDATELOCK[X] Performs random processing of records in a Guardian disk file.	Fails with file-system error 2 when the file is an OSS file.
SETMODE Sets device-dependent functions.	Fails with file-system error 2 when the file is an OSS file.
WRITEUPDATEUNLOCK[X] Alters, then unlocks the contents of a Guardian record at the current position.	Fails with file-system error 2 when the file is an OSS file.

Managing Processes

You can create, control, and terminate OSS processes as you would in a standard UNIX environment. In addition to the standard set of UNIX process-management functions, there are several Hewlett Packard Enterprise extension functions that provide flexibility in propagating attributes during process creation.

OSS processes have a full set of Guardian attributes as well as OSS-specific attributes. Most Guardian process-management procedures can access OSS processes, and some OSS process-management functions can access Guardian processes.

This section discusses the following topics:

Topic	Content
<u>Common and Unique Characteristics</u>	Summary of the differences between UNIX and OSS process management.
<u>Process Attributes</u>	Which OSS attributes Guardian processes have, examples of Guardian attributes that are important to OSS processes, and a table comparing attributes in OSS and Guardian processes.
<u>Process-Management Interoperability</u>	Capability of OSS functions to manage Guardian processes and of Guardian procedures to manage OSS processes.
<u>Creating and Controlling Processes</u>	OSS functions and Guardian procedures to create, suspend, activate, and terminate processes.
<u>Querying the Process Environment</u>	OSS functions and Guardian procedures to get information about both OSS and Guardian processes and their environments.
<u>Modifying the Process Environment</u>	OSS functions and Guardian procedures to modify characteristics about both OSS and Guardian processes and their environments.
<u>Performance Considerations</u>	Suggestions regarding the use of processmanagement functions to improve application performance.
<u>OSS Process-Management Functions</u>	List of each OSS process-management function with notes about its OSS implementation and use on Guardian processes.
<u>Guardian Process-Management Procedures</u>	List of each Guardian process-management procedure with notes about its use on OSS processes.

Common and Unique Characteristics

OSS process management looks like typical UNIX process control, but there are some important extensions added for interoperability between the OSS and Guardian environments. These extensions and any differences between OSS process-management functions and typical UNIX process control are summarized in **Table 13: OSS Process-**



Management Functions. **Table 14: Guardian Process-Management Procedures** summarizes the behavior of Guardian process-management procedures when used to operate on OSS processes.

Features Common to UNIX and OSS Process Management

Common features include those discussed in the following subsections:

- **Process Creation Functions**
- **Process Execution Priority-Control Functions**

Process Creation Functions

You can create OSS processes with the `fork()` function. The attributes of the parent process are propagated to the child process, and the child process has an OSS process ID that uniquely identifies it. An OSS process can send signals to and receive signals from other OSS processes. You can query and modify the process environment using functions such as `getpid()` and `putenv()`. OSS processes receive notification when a child process terminates with the `SIGCHLD` signal and can use the `wait()` and `waitpid()` functions.

Beginning with the H06.24 and J06.13 RVUs, you can create 64-bit OSS processes. For information about 64-bit OSS processes, see **64-Bit OSS Processes**.

Dynamic Launcher Program and **Dynamic Echo Server Program** illustrate typical use of the `fork()`, `execv()`, and `wait()` functions in an application. These programs demonstrate one of the restrictions of using a member of the standard `exec` set of functions: you are limited to launching new processes in the same processor; refer to **Performance Considerations** for less restricted examples.

The launcher program in **Dynamic Launcher Program** is compiled as a linked program file named `dlaunch1`. The server program in **Dynamic Echo Server Program** is compiled as a linked program file named `dserver`. (A requester program, shown in **Using AF_INET in an OSS Requester Process**, is also compiled as a linked program file named `requester2`.) All programs are secured for execution in the current working directory.

NOTE:

These examples use the `AF_INET` sockets functions, which provide an interface to NonStop TCP/IP. The programs could also be written using the `AF_INET6` sockets functions to interface to NonStop TCP/IPv6. For information and examples of `AF_INET6` sockets usage, refer to the *TCP/IP Programming Manual*.

In this application, `dlaunch1` creates an `AF_INET` socket and listens for incoming connections from one or more copies of `requester2`. Upon receipt of a connection, `dlaunch1` uses `fork()` to create a duplicate of itself as a child process.

The child process duplicates itself again with another call to `fork()` to create a grandchild. The grandchild overwrites itself by starting `server`, using an `execv()` call. The child process exits immediately so that the `dlaunch1` call to the `wait()` function prevents the child process from becoming a zombie process and wasting processor resources.

Dynamic Launcher Program

```
/* dlaunch1.c
 *
 * Simple OSS example using OSS socket APIs for communication
 * between requester and server.
 *
 * dlaunch1 (dynamic launcher 1) is an inetd-like program which
 * accepts socket connections and exec's a dserver process for
 * each connection.
 *
 * dserver reads stdin and echoes to stdout. dlaunch1 and dserver
```



```

* write progress messages and error messages to stderr.
*
* dlaunch1 and the multiple dserver processes run on the same cpu.
*/

#define _XOPEN_SOURCE_EXTENDED 1          /* needed for OSS sockets */

#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <sys/time.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netdb.h>

/* Guardian APIs */
#include <cextdecs.h(PROCESSHANDLE_GETMINE_, PROCESSHANDLE_DECOMPOSE_)>

short cpu, phandle[10];    /* our cpu and process handle */
int newpid, fd, listenfd;  /* process id, file descriptor, listening fd */
int readycount, bytesread; /* ready fd count, recv byte count */
struct sockaddr_in addr;   /* socket address (family,addr,port) */
struct timeval timeval;    /* timeout structure for select() call */
fd_set fdset;             /* set of fds for select() call */

#define IDLE_PERIOD 180    /* 180 second timeout value */

int main(int argc, char **argv)
{
    /* get process handle and cpu number */
    PROCESSHANDLE_GETMINE_(phandle);
    PROCESSHANDLE_DECOMPOSE_(phandle, &cpu);

    /* print startup message */
    fprintf(stderr, "dlaunch1 starting in cpu %d\n", cpu);

    /* create AF_INET stream socket */
    if ((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        fprintf(stderr, "socket() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* Get server port from the command line or use default 12345 */
    if (argc > 1)
        addr.sin_port = (short)atoi(argv[1]); /* use command line arg */
    else
        addr.sin_port = 12345;                  /* use default */

    addr.sin_family = AF_INET;                  /* internet address family */

```

```

addr.sin_addr.s_addr = gethostid(); /* assume server on our host */

/* bind socket to address */
if (bind(fd, (const struct sockaddr *)&addr, sizeof(addr)) < 0)
{
    fprintf(stderr, "bind() call failed [errno %d]\n", errno);
    exit(1);
}

/* make the socket a listening socket */
if (listen(fd, 5) < 0)
{
    fprintf(stderr, "listen() call failed [errno %d]\n", errno);
    exit(1);
}

/* fd usage
 * 0 stdin    - not used by dlaunch1, set up for dserver
 * 1 stdout   - not used by dlaunch1, set up for dserver
 * 2 stderr   - used by dlaunch1 for messages, inherited by dserver
 * 3 listenfd - used by dlaunch1 for connections,
 *             not inherited by dserver
 */

close(STDIN_FILENO);
close(STDOUT_FILENO);
listenfd = fd;
fcntl(listenfd, F_SETFD, FD_CLOEXEC); /* don't propagate fd to children */
FD_ZERO(&fdset);

/* loop until error or timeout */
while (1)
{
    /* initialize timeout and bitmask for select() call */
    timeval.tv_sec = IDLE_PERIOD;
    timeval.tv_usec = 0;
    FD_SET(listenfd, &fdset);
    readycount = select(listenfd+1, &fdset, NULL, NULL, &timeval);

    /* if timeout then exit */
    if (readycount == 0)
    {
        fprintf(stderr, "dlaunch1 stopping - timeout\n");
        exit(0);
    }
    /* accept new connection */
    if ((fd = accept(listenfd, (struct sockaddr *)NULL,
        (size_t *)NULL)) < 0)
    {
        fprintf(stderr, "accept() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* setup stdin and stdout */
    dup2(fd, STDIN_FILENO); /* fd is already STDIN_FILENO */
    dup2(fd, STDOUT_FILENO); /* setup stdout */
}

```

```

/* launch a dserver process to handle the connection */
/* note - fork and exec are expensive operations and only */
/*         apply to the same cpu as the parent process */
if ((newpid = fork()) < 0)
{
    fprintf(stderr, "fork() failed [errno %d]\n", errno);
    exit(1);
}
else if (newpid == 0)
{
    /* we are child ... create grandchild to avoid zombie processes */
    if ((newpid = fork()) < 0)
    {
        fprintf(stderr, "fork() failed [errno %d]\n", errno);
        exit(1);
    }
    else if (newpid == 0)
    {
        /* we are grandchild ... exec server process with stdin
         * and stdout set to connection and stderr set to
         * launcher's stderr
         */
        execv("./dserver", argv); /* no return from exec */
    }
    exit(0); /* child goes away */
}
else
{
    /* we are parent */
    fprintf(stderr, "dserver exec'ed in cpu %d\n", cpu);
    wait(&newpid); /* wait for child to avoid zombie
                     * child process */
    close(STDIN_FILENO);
    close(STDOUT_FILENO);
}
} /* end while (1) */
}

```

The original `dlaunch1` process closes the file descriptors associated with the `requester2` connection. The grandchild process, `dserver`, which has inherited file descriptors for the connection, handles all work associated with the connection. When `dserver` terminates, it no longer has a parent process (because its parent, the child process, already terminated) and cannot become a zombie process.

Note that `dserver` is a dynamic server, launched only when needed and terminating when its function is completed. Such servers conserve processor resources if run infrequently. If servers need to be launched frequently, a static iterative server, such as that shown in **Static Echo Server** or in **Using AF_INET in an OSS Requester Process**, uses processor resources more efficiently.

Using two terminal sessions, sample dialog for the terminal launching `dlaunch1` might be:

```

/home/software/chrisbl/cabpubs: ./dlaunch1
dlaunch1 starting in cpu 8
dserver exec'ed in cpu 8
dserver [cpu 8] - starting
dserver [cpu 8] - message received on fd 0 = hello from first requester
dserver [cpu 8] - stopping [input closed]
dserver exec'ed in cpu 8
dserver [cpu 8] - starting

```

```

dserver [cpu 8] - message received on fd 0 =  hello from another requester
dserver [cpu 8] - stopping [input closed]
dlaunch1 stopping - timeout
/home/software/chrisbl/cabpubs:

```

while the terminal running the copy of requester2 might show:

```

/home/software/chrisbl/cabpubs: ./requester2
Requester starting
?hello from first requester
Read 27 bytes back from server
?
/home/software/chrisbl/cabpubs: ./requester2
Requester starting
?hello from another requester
Read 29 bytes back from server
?
/home/software/chrisbl/cabpubs:

```

Dynamic Echo Server Program

```

/* dserver.c
 *
 * Simple OSS server example using OSS APIs for communication
 * between requester and server.
 *
 * dserver is a dynamic echo server. A dserver process is launched
 * for each requester. dserver reads stdin and echoes on stdout.
 * dserver exits when idle for 60 seconds.
 */

#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <sys/time.h>

/* Guardian APIs */
#include <cextdecs.h(PROCESSHANDLE_GETMINE_, PROCESSHANDLE_DECOMPOSE_)>

short cpu, phandle[10]; /* our cpu and process handle */
int bytesread, readycount; /* ready fd count, recv byte count */
struct timeval timeval; /* timeout structure for select() call */
fd_set fdset; /* set of fds for select() call */

#define IDLE_PERIOD 60 /* 60 seconds */
#define BUFF_LEN 1024 /* buffer size */
char msg_buff[BUFF_LEN+1]; /* single buffer is enough */

int main(int argc, char **argv)
{
    /* get process handle and cpu number */
    PROCESSHANDLE_GETMINE_(phandle);
    PROCESSHANDLE_DECOMPOSE_(phandle, &cpu);

```




```

/* print startup message */
fprintf(stderr, "dserver [cpu %d] - starting\n", cpu);

FD_ZERO(&fdset);

/* loop until error or timeout or input closed */
while (1)
{
    /* initialize timeout and bitmask for select() call */
    timeval.tv_sec = IDLE_PERIOD;
    timeval.tv_usec = 0;
    FD_SET(STDIN_FILENO, &fdset);
    readycount = select(STDIN_FILENO+1, &fdset, NULL, NULL, &timeval);

    /* if timeout then exit */
    if (readycount == 0)
    {
        fprintf(stderr, "dserver [cpu %d] - stopping [timeout]\n", cpu);
        exit(0);
    }
    /* fd ready so read data */
    if ((bytesread = read(STDIN_FILENO, msg_buff, BUFF_LEN)) < 0)
    {
        fprintf(stderr, "dserver [cpu %d] - "
                    "read(fd %d) failed [errno %d]\n",
                    cpu, STDIN_FILENO, errno);
        exit(1);
    }
    else if (bytesread == 0) /* check for input closed */
    {
        fprintf(stderr, "dserver [cpu %d] - "
                    "stopping [input closed]\n",
                    cpu);
        exit(1);
    }
    else /* data received */
    {
        /* give progress message */
        msg_buff[bytesread] = '\0'; /* guarantee string NULL-terminated */
        fprintf(stderr, "dserver [cpu %d] - "
                    "message received on fd %d = %s",
                    cpu, STDIN_FILENO, msg_buff);

        /* echo on stdout */
        if (write(STDOUT_FILENO, msg_buff, bytesread) < 0)
        {
            fprintf(stderr, "dserver [cpu %d] - "
                        "write(fd %d) failed [errno %d]\n",
                        cpu, STDOUT_FILENO, errno);
            exit(1);
        }
    }
} /* end while (1) */
}

```

Process Execution Priority-Control Functions

You can change the process scheduling priority, called the nice value, with the `nice()` function.

Determining and Changing Process Priority using `getpriority()` and `nice()` provides sample code to demonstrate the use of the `getpriority()` function to determine the process priority of the processes that are members of the specified group. The sample code also demonstrates the use of `nice()` to change the current process priority by the increment specified in the command line.

Determining and Changing Process Priority using `getpriority()` and `nice()`

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/resource.h>

extern int errno;

int getpri(int which, id_t who)
{
    int cval;

    errno = 0;
    cval = getpriority(which, who);
    if(errno) {
        fprintf(stderr, "getpriority error, args = %d, %d\n", which, who);
        exit(1);
    }
    return(cval);
}

int main(int argc, char *argv[])
{
    int prval, pval, retcode;

    /*
     * Read the value of the process priority increment from the command line.
     * If the value is not provided, print usage message and exit.
     * If the value is invalid, print error message and exit.
     */
    if(argc > 1) {
        prval = atoi(argv[1]);
        if(abs(prval) > (2*NZERO-1)) {
            fprintf(stderr, "Invalid priority increment %s\n", argv[1]);
            exit(1);
        }
    } else {
        fprintf(stderr, "Usage: nicegetp value [0 to 150]\n");
        exit(1);
    }

    /* Print current process priority value. */
    pval = getpri(PRIO_PROCESS, 0);
    printf("Current process priority value = %d\n", pval);
    /* Print current process group priority value. */
```



```

    pval = getpri(PRIO_PGRP, 0);
    printf("Current process group priority value = %d\n", pval);
/* Print process priority value associated with this user. */
    pval = getpri(PRIO_USER, 0);
    printf("Process priority value associated with user = %d\n", pval);

/* Set current process priority value */
    errno = 0;
    pval = nice(prval);
    if(errno) {
        fprintf(stderr, "Can't change process priority by %d\n", prval);
        exit(1);
    }

    printf("New nice value = %d\n", pval);
    pval = getpri(PRIO_PROCESS, 0);
    printf("Process priority set to %d\n", pval);
    return(0);
}

```

Features Unique to OSS Process Management

- OSS processes have an OSS process ID and a process handle. An OSS process ID is a nonnegative integer that uniquely identifies a process within a node. A process handle is a data structure ten words long that uniquely identifies a process within a NonStop network.
- There are Hewlett Packard Enterprise extension functions. The `tdm_fork()`, `tdm_execve()`, `tdm_execvep()`, `tdm_spawn()`, and `tdm_spawnp()` functions allow you to set attributes when a process is created that you cannot set with the standard `fork()` and `exec` set of functions. For example, the `tdm_fork()` function allows you to either retain the attribute values inherited from the parent or specify other values. The `fork()` function does not allow you to overwrite the values of the inherited attributes.
- OSS processes have Guardian attributes. In addition to OSS-specific attributes, OSS processes have a full set of Guardian attributes, such as process handle and Guardian user ID.
- Most Guardian process-management procedures can access OSS processes. The process handle of an OSS process allows most Guardian process-management procedures to manipulate OSS processes, including changing the priority of a process, getting information about it, and terminating it.
- Some OSS functions can access Guardian processes. For example, OSS functions can provide the system-dependent system limits, times, and user ID of a Guardian process.
- After a process is created, its execution priority cannot be modified using an OSS function. In the current release, you must use a Guardian procedure to change process priority.
- Processes running in the OSS environment have their `nice()` scheduling priorities determined using UNIX conventions. The scheduling priority for processes running in the Guardian environment is defined as increasing as the priority number increases. This convention is the opposite of the convention used on UNIX systems, where a lower priority number means a higher scheduling priority.



Process Attributes

All processes, OSS and Guardian, have Guardian attributes and some OSS attributes. OSS processes are basically Guardian processes with some additional OSS-specific attributes, such as OSS process ID.

OSS Attributes in Guardian Processes

The OSS attributes of Guardian processes include an extended security block that allows Guardian processes to access OSS and Guardian objects belonging to other related groups and group IDs. Guardian processes also have some OSS file system attributes that allow Guardian processes to access OSS objects and do OSS file system work.

A Guardian process has a process handle but not an OSS process ID. This means that an OSS process cannot send signals to a Guardian process and that a Guardian process cannot belong to a process group or session.

Guardian Attributes in OSS Processes

There are many Guardian attributes that an OSS child process inherits from its parent. When you create a process using the OSS Hewlett Packard Enterprise extension functions such as `tdm_fork()`, `tdm_execve()`, and `tdm_spawn()`, you can set or overwrite inherited Guardian attributes by specifying values for the fields in the input structure that contains Guardian attributes. Examples of these attribute fields are `pe_cpu` (processor), `pe_debug_options` (debug options), and `pe_hometerm` (home terminal).

Some Guardian attributes are more important for you to manipulate than others. The following lists the attributes you might want to set when you create a process using the OSS Hewlett Packard Enterprise extension functions. For more information about setting these attributes, see the process-creation functions in the reference pages either online or in the *Open System Services System Calls Reference Manual*.

- **Processor.** Specifying the processor on which a process executes allows you to distribute an application across processors. `tdm_fork()` does not allow you to specify a value for the processor attribute, but the `tdm_exec` and `tdm_spawn` sets of functions do. You cannot change the processor in which a process executes after the process is created.
- **Debug options.** Debug options can instruct a debugging program to create a saveabend file if the process terminates abnormally. A saveabend file contains data-area and file-status information at the time of the failure. Debug options also can force the child process to enter a debugging program after returning from the `tdm_fork()` function. You cannot change the debug options of a process after the process is created.
- **Home terminal.** Making the home terminal different from the controlling terminal allows you to avoid information collisions. For example, you can make another terminal or terminal emulator window (other than the controlling terminal) the home terminal, and a debugging program can display messages there. Use the `PROCESS_SETSTRINGINFO_` procedure to change the home terminal of a process after it is created.
- **Priority.** Changing the execution priority of a process changes the position of the process in the ready queue. Use the `PROCESS_SETINFO_` procedure to change the priority of a process after it is created.
- **DEFINES.** Specifying a set of DEFINES to be propagated to a new process lets you pass information to the process. For example, you can specify an SQL catalog to be used by an SQL process. Use Guardian procedures, such as `DEFINEADD` and `DEFINDELETE`, or OSS utilities, such as `add_define` and `del_define`, to change the DEFINES of a process after it is created.

Comparing OSS and Guardian Process Attributes

The following table summarizes some important process attributes that OSS and Guardian processes have in common and those they do not share. Notice that except for a program type of Guardian, OSS processes almost always have Guardian process attributes while Guardian processes lack several OSS-specific attributes.



Table 10: OSS and Guardian Process Attribute Comparison

Process Attribute	Part of an OSS Process?	Part of a Guardian Process?
Ancestors		
Guardian ancestor	Sometimes	Sometimes
Guardian MOM/GMOM	Sometimes	Sometimes
OSS parent	Always	Never
Groups		
OSS group leader	Always	Never
OSS session leader	Always	Never
Identification		
Guardian creator access ID	Always	Always
Guardian process access ID	Always	Always
Guardian process handle	Always	Always
Guardian process name	Sometimes	Sometimes
Guardian user ID	Always	Always
OSS group ID	Always	Always
OSS parent process ID	Always	Never
OSS process ID	Always	Never
OSS user ID	Always	Always
Program type		
Guardian	Never	Always
OSS	Always	Never
Terminals		
Guardian home terminal	Sometimes	Sometimes
OSS controlling terminal	Sometimes	Never
nice scheduling priority	Always	Sometimes

Process-Management Interoperability

In general, you should manage OSS processes with OSS functions, because your applications are more portable. However, there might be cases in which you want to use Guardian procedures to manage OSS processes or OSS functions to manage Guardian processes. For example, if you want to use a product or subsystem that runs only in the Guardian environment, you might need to manage a process from the opposite environment.

The process handle of an OSS process allows most Guardian process-management procedures to access it. On the other hand, a Guardian process does not have an OSS process ID, which is required by many OSS process-management functions. This means that some OSS process-management functions cannot access Guardian processes.



The following table lists and provides a short description of the Guardian process-management procedures that can access OSS processes. Note that the only principal Guardian process-management procedure missing from this list is `PROCESS_LAUNCH_`, because it can create only Guardian processes. **Table 14: Guardian Process-Management Procedures** table provides a summary of all Guardian process-management procedures.

Procedures that are indicated as “superseded” in the *Guardian Procedure Calls Reference Manual* are not included in this table, because you should not use them for new program development.

Table 11: Guardian Process-Management Procedures That Can Access OSS Processes

Guardian Procedure	Description
<code>CHILD_LOST_</code>	Examines a system message to determine whether a process has stopped
<code>FILENAME_TO_PROCESSHANDLE_</code>	Converts a (process) filename to a process handle
<code>OSS_PID_NULL_</code>	Returns a null OSS process ID
<code>PROCESS_ACTIVATE_</code>	Returns a suspended process to a ready state
<code>PROCESS_DEBUG_</code>	Invokes the debugging facility on a process
<code>PROCESS_GETINFO_</code>	Gets a limited set of information about a process
<code>PROCESS_GETINFOLIST_</code>	Gets detailed information about a process or set of processes
<code>PROCESS_GETPAIRINFO_</code>	Obtains information about a named process or process pair
<code>PROCESS_SETINFO_</code>	Modifies a single nonstring attribute of a process
<code>PROCESS_SETSTRINGINFO_</code>	Modifies a single string-form attribute of a process
<code>PROCESS_SPAWN[64]_</code>	Creates an OSS process
<code>PROCESS_STOP_</code>	Terminates a process
<code>PROCESS_SUSPEND_</code>	Places a process into a suspended state
<code>PROCESSHANDLE_COMPARE_</code>	Compares two process handles and reports whether they are identical, represent different processes of the same process pair, or are different
<code>PROCESSHANDLE_DECOMPOSE_</code>	Returns one or more parts of a process handle
<code>PROCESSHANDLE_GETMINE_</code>	Gets the process handle of the calling process
<code>PROCESSHANDLE_NULLIT_</code>	Initializes a process handle to a null value

Table Continued



Guardian Procedure	Description
PROCESSHANDLE_TO_CRTPID_	Converts a process handle to the corresponding Guardian process ID (CRTPID)
PROCESSHANDLE_TO_FILENAME_	Converts a process handle to a (process) filename
PROCESSHANDLE_TO_STRING_	Converts a process handle to a process string, which can identify a process or set of processes
PROCESSNAME_CREATE_	Returns a unique process name that is suitable for passing to the PROCESS_CREATE_ procedure
PROCESSTRING_SCAN_	Scans an input string for a process string and returns the corresponding process handle or a single component of the process string converted to internal form
SETLOOPTIMER 1	Sets the process loop-timer value of the calling process
SIGACTION_INIT_ 2	Establishes the initial state of signal handling for the calling process
SIGACTION_RESTORE_ 3	Restores the signal-handling state stored by a call to SIGACTION_SUPPLANT_
SIGACTION_SUPPLANT_ 4	Saves the current signal-handling state and allows a subsystem to take over signal handling temporarily
SIGJMP_MASKSET_ 5	Saves the process signal mask in a jump buffer

¹ Also available as the C function SETLOOPTIMER().

² Also available as the C function SIGACTION_INIT().

³ Also available as the C function SIGACTION_RESTORE().

⁴ Also available as the C function SIGACTION_RESTORE().

⁵ Also available as the C function SIGACTION_MASKSET().

The following table lists and provides a short description of the OSS process-management functions that can access Guardian processes. **Table 13: OSS Process-Management Functions** table provides a summary of all OSS process-management functions.

NOTE: To determine which RVUs support an OSS function, see **Documented OSS Functions**.



Table 12: OSS Process-Management Functions That Can Access Guardian Processes

OSS Function	Description
<code>abort()</code>	Generates a software signal to terminate the calling process
<code>alarm()</code>	Sets or changes a timer that expires at a specified time in the future
<code>_exit()</code> <code>exit()</code>	Terminates the calling process
<code>getegid()</code>	Gets the effective group ID of the calling process
<code>getenv()</code>	Returns the value of an environment variable
<code>geteuid()</code>	Gets the effective user ID of the calling process
<code>getgid()</code>	Gets the real group ID of the calling process
<code>getgroups()</code>	Gets the group list of the calling process
<code>getsid()</code>	Gets the group ID of the process that is the session leader of the specified process
<code>getuid()</code>	Gets the real user ID of the calling process
<code>longjmp()</code>	Performs a nonlocal goto
<code>nftw()</code> <code>nftw64()</code>	Traverses a file tree
<code>pause()</code>	Suspends the calling process until a signal is received
<code>putenv()</code>	Sets an environment variable
<code>raise()</code>	Sends a signal to the calling process
<code>setgid()</code>	Sets the group ID of the calling process
<code>setjmp()</code>	Saves the current execution context
<code>setitimer()</code>	Sets or changes an optionally-renewable timer that expires at a specified time in the future. Only real-time timer is supported.
<code>setpgpr()</code>	Creates a new session when the calling process is not a process group leader, and sets the process group ID of the calling process

Table Continued

OSS Function	Description
<code>setregid()</code>	Sets the real and effective group IDs of the current process
<code>setreuid()</code>	Sets the real and effective user IDs of the current process
<code>setsid()</code>	Creates a new session and sets the process group ID
<code>setuid()</code>	Sets the user ID of the calling process
<code>sigaction()</code>	Specifies the action to be taken upon delivery of a signal
<code>sigaddset()</code>	Adds a signal to a signal set
<code>sigdelset()</code>	Deletes a signal from a signal set
<code>sigemptyset()</code>	Initializes a signal set to exclude all signals
<code>sigfillset()</code>	Initializes a signal set to include all signals
<code>sigismember()</code>	Tests whether a signal is a member of a signal set
<code>siglongjmp()</code>	Performs a nonlocal goto
<code>signal()</code>	Specifies the action to be taken upon delivery of a signal
<code>sigpending()</code>	Returns the set of signals that are blocked from delivery and pending to the calling process
<code>sigprocmask()</code>	Changes or examines a process signal mask
<code>sigsetjmp()</code>	Saves the current execution context and can save the current process signal mask
<code>sigsuspend()</code>	Changes a process signal mask and suspends the calling process
<code>sleep()</code>	Suspends the calling process for a specified interval of time
<code>times()</code>	Gets calling process and child process times
<code>uname()</code>	Gets information identifying the system on which the calling process runs

Signals Interoperability

Signals are software interrupts that provide a way of handling asynchronous events, such as a user at a terminal typing an interrupt key, timer expiration, detection of a hardware fault, or abnormal termination of a process.



Using Standard Signals Functions

Both OSS processes and Guardian native processes can send, receive, and handle signals using the functions in the OSS API, which conform to the POSIX.1 standard. However, there are some important differences (mostly with respect to how Guardian processes interact with signals), which are summarized as follows:

- Control of a process over itself and other processes

An OSS process can control, or send a signal to, itself using a function such as `raise()`, `abort()`, or `sleep()`. An OSS process can also exercise control over another OSS process by sending a signal to it using the `kill()` function.

A Guardian native process can use most of the signals functions in the OSS API to send, receive, and handle signals but only for the purpose of exercising control over itself. A Guardian native process cannot send a signal to or receive a signal from another Guardian or OSS process using the `kill()` function.

- Signals in TNS processes and native processes

Any OSS process (native or G-series TNS) can use OSS signals functions. Only a Guardian native process can use OSS signals functions.

- Interrupts from a terminal as signals

When a user presses an interrupt key at a terminal, an OSS process receives the interrupt as a signal. A Guardian process receives the interrupt as a system message and must read its `$RECEIVE` file to get the interrupt.

- Process signal mask

The process signal mask of an OSS process is initialized to that of its parent. The process signal mask of a Guardian process is initialized so that no signals are blocked from delivery to it.

- Default signal handler

The Common Run-Time Environment (CRE) sets up a default signal handler for a Guardian program written in C. The CRE does not set up a default signal handler for an OSS program written in C.

Using Hewlett Packard Enterprise Signals Extension Functions

Guardian TNS processes receive traps to indicate run-time events requiring immediate attention. Guardian native processes receive signals to indicate conditions known as traps in Guardian TNS processes.

The Hewlett Packard Enterprise signals extensions are provided as migration and convenience tools that allow native processes to catch signals corresponding to trap conditions in TNS processes. The signals extensions provide shortcuts to the same basic functions as provided by the standard signal interfaces.

Both OSS and Guardian native processes can use the signals extensions, but you probably won't have much reason to use them because you are concerned with handling the full range of signals, not just those known as traps in Guardian TNS programs. Traps are a subset of all signals. Also, the standard signals functions are portable and the Hewlett Packard Enterprise extensions are not.

The signals extensions are available in C and pTAL. The C versions are `SIGACTION_INIT_()`, `SIGJMP_MASKSET_()`, `SIGACTION_SUPPLANT_()`, and `SIGACTION_RESTORE_()`.

For information about how to use these functions, see the *Guardian Programmer's Guide*.

Examples in the following subsections show how to manage processes with both the OSS and Guardian APIs. For information about OSS functions, see the *Open System Services System Calls Reference Manual* and the *Open System Services Library Calls Reference Manual*. For information about Guardian procedures, see the *Guardian Procedure Calls Reference Manual*.



Creating and Controlling Processes

The OSS API allows you to create OSS processes and to control (through signals functions) both OSS and Guardian processes. The Guardian API provides procedures to create and control both OSS and Guardian processes.

Using the OSS API

You can create only OSS processes with OSS functions. Both Guardian and OSS processes can be controlled through signals functions with one exception: the `kill()` function. Only OSS processes can send or receive signals using the `kill()` function, because `kill()` needs the OSS process ID of the intended recipient in order to send a signal to it. (A Guardian process does not have an OSS process ID.)

NOTE:

You can use OSS signals functions with Guardian native processes; you cannot use them with Guardian TNS processes.

Creating OSS Processes Using Hewlett Packard Enterprise Extension Functions

The `tdm_fork()`, `tdm_execve()`, `tdm_execvep()`, `tdm_spawn()`, and `tdm_spawnp()` functions allow you to set attributes when a process is created that you cannot set with the standard `fork()` and `exec` set of functions.

You can create OSS processes using the `tdm_spawn()` or `tdm_spawnp()` function or by calling the `fork()` or `tdm_fork()` function followed by one of the `exec` or `tdm_exec` set of functions.

Application performance is better if you use `tdm_spawn()` or `tdm_spawnp()`. These functions also allow you to specify inheritance information for the child process in the `inherit` structure, and the `fd_count` and `fd_map` parameters allow you to control the file descriptors the child process inherits from the parent.

Examples of both methods of creating processes are shown for purposes of comparison.

Creating an OSS Process Using `tdm_fork0` and `tdm_execvep0` creates a named process with a system-generated name using `tdm_fork()`. The program then passes a different process name and the processor on which the child process will execute to `tdm_execvep()`.

Creating an OSS Process Using `tdm_fork0` and `tdm_execvep0`

```
#include <tdmext.h>      /* Note 1 */
#include <errno.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>

/* Set defaults for input and output structures.      Note \2 */
struct process_extension
    procect = DEFAULT_PROCESS_EXTENSION;
struct process_extension_results
    procreults = DEFAULT_PROCESS_EXTENSION_RESULTS;
extern char **environ;
char *argv[10];

int main(void)
{
    char buf[4096];
    pid_t pid;
    int status, cnt;
```



```

char *ptr;

/* Get parameters for the child process from the command line. */
printf("%s "); /* print prompt (printf requires %s to print %) */
while (fgets(buf, 4096, stdin) != NULL) {
    ptr = buf;
    buf[strlen(buf) - 1] = '\0'; /* replace newline with NULL */

/* Build the parameter list for the child process. */
    argv[0] = buf;
    cnt = 1;
    while(*ptr) {
        if(*ptr == ' ') {
            *ptr = '\0';
            argv[cnt++] = ptr + 1; /* one blank between parameters */
        }
        ptr++;
    }
    argv[cnt] = NULL;

/* Request system-generated process name and create child process. */
    proctx.pe_name_options = _TPC_GENERATE_NAME;
    if ( (pid = tdm_fork(&proctx, &proctxresults)) < 0 ) {

/* If tdm_fork() fails, print diagnostic messages. */
        fprintf(stderr, "pid = %o\n", pid);
        fprintf(stderr, "proctxresults.pr_len = %o\n", \
            proctxresults.pr_len);
        fprintf(stderr, "proctxresults.pr_pid = %o\n", \
            proctxresults.pr_pid);
        fprintf(stderr, "proctxresults.pr_errno = %o\n", \
            proctxresults.pr_errno);
        fprintf(stderr, "proctxresults.pr_TPCError = %o\n", \
            proctxresults.pr_TPCError);
        fprintf(stderr, "proctxresults.pr_TPCdetail = \
            %o\n", proctxresults.pr_TPCdetail);
        fprintf(stderr, "errno = %d\n", errno);
        perror("tdm_fork error");

/* Pass values for the child process to execvp(). */
    } else if (pid == 0) { /* child process executing */
        printf("trying execute of %s\n", buf);
        printf("argv[0]=%s,argv[1]=%s,argv[2]=%s\n", \
            argv[0],argv[1],argv[2]);

/* Supply a name and processor number for the child process. */
        proctx.pe_name_options = _TPC_NAME_SUPPLIED; /* Note 3 */
        proctx.pe_process_name = "/G/HEINZ"; /* set process name */
        proctx.pe_cpu = 1; /* set processor number */

/* Pass parameter values to the child process. */
        tdm_execvp(buf, argv, environ, &proctx, &proctxresults);

/* If process execution failed, print error message. */
        fprintf(stderr, "couldn't execute: %s\n", buf); /* Note 4 */
        exit(2);
    }
}

```

```

/* The parent process waits for the child process to terminate. */
    if ( (pid = waitpid(pid, &status, 0)) < 0)
        perror("waitpid error");
    printf("%s\n");
}
exit(0);
}

```

Note 1

The `tdmext.h` library header file defines the input structure containing Guardian process attributes to be assigned to the new process. It also defines the output structure containing optional process identification and error information.

Note 2

To set attributes in the new process, you must declare the `proext` (input) and `proresults` (output) structures and initialize them with default values. Then you can set attributes using literals defined in `tdmext.h`.

Note 3

To set the process name, request a system-generated process name before calling `tdm_fork()` and pass your process name to `tdm_evecvop()`. If you attempt to pass your name to `tdm_fork()` and use the same name in the call to `tdm_execvop()`, you get an error stating that the process already has a name.

Note 4

After the call to `tdm_fork()`, the child process has its own unique OSS process ID, which does not change. It also has a process handle that does change after the call to `tdm_execvop()`.

Creating an OSS Process Using `tdm_spawnop()` creates a named process with a system-generated name using `tdm_spawnop()`. By setting fields in the `inherit` structure, the child process becomes a member of a group other than the parent's process group, and the child process is the leader of the new process group.

Creating an OSS Process Using `tdm_spawnop()`

```

#include <tdmext.h>
#include <spawn.h>    /* Note 1 */
#include <errno.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>

/* Set defaults for input and output structures */
struct process_extension
    proext = DEFAULT_PROCESS_EXTENSION;
struct process_extension_results
    proresults = DEFAULT_PROCESS_EXTENSION_RESULTS;
struct inheritance inherit;
extern char **environ;
char *argv[10];

int main(void)
{
    char buf[4096];
    pid_t pid;
    int status, cnt;
    int fd_count, fd_map[20];

```



```

char *ptr;

/* Get parameters for the child process from the command line. */
printf("%% "); /* print prompt (printf requires %% to print %) */
while (fgets(buf, 4096, stdin) != NULL) {
    ptr = buf;
    buf[strlen(buf) - 1] = '\0'; /* replace newline with NULL */

/* Build the parameter list for the child process. */
    argv[0] = buf;
    cnt = 1;
    while(*ptr) {
        if(*ptr == ' ') {
            *ptr = '\0';
            argv[cnt++] = ptr + 1; /* one blank between parameters */
        }
        ptr++;
    }
    argv[cnt] = NULL;
    fd_map[0] = 0; /* Note 2 */
    fd_map[1] = 1;
    fd_map[2] = 2;
    fd_count = 3;

/* Request system-generated process name. */
    proctext.pe_name_options = _TPC_GENERATE_NAME;
/* Set up child process as new group leader. */
    inherit.flags = SPAWN_SETGROUP;
    inherit.pgroup = SPAWN_NEWPGROUP;
    printf("trying spawn of %s\n", buf);
/* Spawn child process with supplied parameters. */
    if((pid=tdm_spawn(buf, fd_count, fd_map, &inherit,
        argv, environ, &proctext, &procresults))<0) {
/* If unsuccessful, print diagnostic messages. */
        fprintf(stderr, "pid = %o\n", pid);
        fprintf(stderr, "procresults.pr_len = %o\n",
            procresults.pr_len);
        fprintf(stderr, "procresults.pr_pid = %o\n",
            procresults.pr_pid);
        fprintf(stderr, "procresults.pr_errno = %o\n",
            procresults.pr_errno);
        fprintf(stderr, "procresults.pr_TPCError = %o\n",
            procresults.pr_TPCError);
        fprintf(stderr, "procresults.pr_TPCdetail = %o\n",
            procresults.pr_TPCdetail);
        fprintf(stderr, "errno = %d\n", errno);
        fprintf(stderr, "couldn't spawn: %s\n", buf);
    }

/* The parent process waits for the child process to terminate. */
    if ( (pid = waitpid(pid, &status, 0)) < 0)
        perror("waitpid error");
    printf("%% ");
}
exit(0);
}

```

Note 1

The `spawn.h` library header file defines the `inherit` structure, which allows you to specify the process group ID and signal mask of the new process and a list of signals that the child process will take default action on.

Note 2

The `fd_count` and `fd_map` parameters allow you to control the file descriptors the child process inherits from the parent. `fd_count` specifies the number of file descriptors designated by `fd_map`. `fd_map` specifies how the file descriptors in the parent process map to the file descriptors in the child process. That is, the file descriptor specified in `fd_map[0]` is copied to file descriptor 0 (zero) in the child process, and so on.

Using the Guardian API

Using Guardian procedures, you can create and control both OSS and Guardian processes. See the *Guardian Programmer's Guide* for a discussion of the Hewlett Packard Enterprise signals extension procedures.

Creating and Controlling Guardian Processes

Creating and Controlling a Guardian Process Using Guardian Procedures manages a named Guardian process from the OSS environment by creating it, suspending it, returning it to a ready state, and terminating it.

Creating and Controlling a Guardian Process Using Guardian Procedures

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <dlaunch.h>
#include <zsysc>
#include <cextdecs.h>(PROCESS_LAUNCH_, PROCESS_SUSPEND_, \
                    PROCESS_ACTIVATE_, PROCESS_STOP_)>

/* Initialize input parameter structure to default values. */
process_launch_parms_def proc_param = P_L_DEFAULT_PARMS;
/* Process results structure. */
zsys_ddl_msg_proccreate_def proc_results;

char *pname;
short errordetail;
short outlen;
char *proc_name;

int main(int argc, char *argv[]) {
    short retcode;
    /* If the name of a Guardian program to run is not provided, use a
       default program name. */
    if(argc > 1)
        pname = argv[1];
    else
        pname = "$system.systools.vols";
    proc_name = "$heinz";          /* make the process name unique */

    /* Initialize input parameters without default values. */
    proc_param.program_name = pname;
    proc_param.program_name_len = (short)strlen(pname);
    proc_param.process_name = proc_name;
    proc_param.process_name_len = (short)strlen(proc_name);
```



```

    proc_param.name_options = ZSYS_VAL_PCREATOPT_NAMEINCALL;

/* Create a new Guardian process. */
retcode = PROCESS_LAUNCH_ (
    &proc_param,          /* input parameter list */
    &errordetail,         /* details of error */
    &proc_results,        /* output results */
    sizeof(proc_results), /* maximum size of output */
    &outlen               /* actual size of output */
);

/* If unsuccessful, print error message and exit. */
if (retcode != 0) {
    fprintf(stderr, "retcode = %d\n", retcode);
    fprintf(stderr, "Error detail = %d\n", errordetail);
    exit(1);
}
printf("Process %s created successfully\n", proc_name);
sleep(10);      /* wait ten seconds */

/* Suspend the child process. */
retcode = PROCESS_SUSPEND_
    ((short *)&proc_results.z_phandle,
     );
if (retcode != 0) {
    fprintf(stderr, "Process suspend failure, code = %d\n", retcode);
    exit(1);
}
printf("Process %s suspended for 10 seconds\n", proc_name);
sleep(10);      /* wait ten seconds */

/* Reactivate the child process. */
retcode = PROCESS_ACTIVATE_
    ((short *)&proc_results.z_phandle,
     );
if (retcode != 0) {
    fprintf(stderr, "Process activate failure, code = %d\n",
             retcode);
    exit(1);
}
printf("Process %s activated for 10 seconds\n", proc_name);
sleep(10);      /* wait ten seconds */

/* Terminate the child process. */
retcode = PROCESS_STOP_
    ((short *)&proc_results.z_phandle,
     , /* process specifier */
     , /* normal termination option */
     , /* completion code */
     , /* termination information */
     , /* subsystem ID */
     , /* text to be sent as part of process
        termination system message */
     /* message length */
    );
if (retcode != 0) {
    fprintf(stderr, "Process termination failure, code = %d\n",

```



```

        retcode);
    exit(1);
}
printf("Process %s terminated\n", proc_name);
exit(0);
}

```

Creating and Controlling OSS Processes

To create and control an OSS process using the Guardian API, you must use the `PROCESS_SPAWN[64]_` procedure. It creates an OSS process and can be called by both Guardian and OSS processes, although it is most commonly called by Guardian processes, such as monitor programs, that must manage both Guardian and OSS processes.

When the calling process is a Guardian monitor process pair, the monitor can ensure that an OSS child process is restarted if it stops. In this way, the OSS child process is considered to be a persistent process. Writing applications as persistent processes is one way to achieve application-level fault tolerance.

`PROCESS_SPAWN[64]_` also allows you to create an OSS process in a `nowait` manner, which means that the `PROCESS_SPAWN[64]_` procedure returns as soon as process creation is initiated. The calling process continues processing while the child process is being created.

The process that calls `PROCESS_SPAWN[64]_` is a Guardian parent, regardless of whether it is an OSS or Guardian process, and it receives process-termination notification through Guardian system messages instead of the `SIGCHLD` signal. The parent process reads system messages from `$RECEIVE`, which is a special file through which a process receives system messages or messages from other processes. The created process is not the POSIX child of the created process.

Monitoring OSS Processes Reading Guardian System Messages

A Guardian parent process must monitor an OSS child process by reading Guardian system messages from `$RECEIVE`. (An OSS parent process could monitor its child process by reading Guardian system messages, but it would be very unusual.)

When monitoring an OSS child process by reading Guardian system messages, it is important for the parent process to verify that the child process is still alive after any processor failure because:

- The parent process does not receive process-termination messages for any unnamed OSS processes running in a particular processor if that processor fails.
- An OSS process can move to another processor using the `tdm_execve()` or `tdm_execvep()` function. The parent receives a termination message so it can determine that an exec function took place.

To monitor an OSS process effectively by reading Guardian system messages, a parent process must do the following:

- Call the `MONITORCPUS` procedure regularly to be notified of any processor failure.
- If a processor fails, call the `PROCESS_GETINFOLIST_` procedure to check on the status of the OSS child process.

In the following example (*Creating an OSS Process Using `PROCESS_SPAWN_`*), a Guardian process creates an OSS process using `PROCESS_SPAWN[64]_`. While doing so, the parent Guardian process opens three standard I/O streams for the child OSS process by explicitly specifying them in the `fdinfo` parameter. The parent process monitors the status of the OSS child process by checking for processor failures and reading system messages on its `$RECEIVE` file. If a processor fails, the parent verifies that the OSS process is still alive by verifying the OSS process ID of the child process is still valid. This example can run only as a native process because it calls the `gfileno()` function. A TNS process must call the `fdtogfn()` function instead. A native process cannot call `fdtogfn()`.

This example must be compiled in the OSS environment and executed from the Guardian environment:



1. Compile the program

```
c89 oss_process_spawn.c -Wextensions -Wsystype=guardian -Wtarget=tns/e -o psguard
```

2. Copy the object file to the Guardian environment

3. Run the object file in the Guardian environment

```
TACL> Run PSGUARD
```

Creating an OSS Process Using PROCESS_SPAWN_

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <limits.h>
#include <errno.h>
#include <spawn.h>
#include <tdmext.h>
#include <zsysc>
#include <cextdecs.h(FILE_OPEN_, FILE_GETINFO_, READX, MONITORCPUS,\
    PROCESS_SPAWN_, PROCESS_GETINFOLIST_, FILENAME_TO_PATHNAME_)>

/* Structures for standard I/O file descriptor information */
fdentry_def *fdentry; /* structure pointer for fdentry */
fdinfo_def *fdinfo; /* structure pointer to fdinfo block
                     with one fdentry */
fdinfo_def fdinit = DEFAULT_FDINFO; /* initialize structure */
/* Structure for inheritance values */
struct inheritance inherit;
/* Set defaults for process spawn input/output structures */
struct process_extension
    proc_ext = DEFAULT_PROCESS_EXTENSION;
struct process_extension_results
    proc_results = DEFAULT_PROCESS_EXTENSION_RESULTS;

/* Structure used for process termination message */
union {
    short msgcode; /* message code */
    zsys_ddl_msgg_procdeath_def procdeath; /* termination message format */
} msg;

extern char **environ;
#define FILE_MAX ZSYS_VAL_LEN_FILENAME /* maximum characters in file name */
char *progrname; /* Name of program to be executed */
pid_t pid; /* OSS process ID */
char *pargv[4]; /* arguments for program to be run */
short error; /* error code, if any */
char inpath[FILE_MAX]; /* standard input file name */
char outpath[FILE_MAX]; /* standard output file name */
char errpath[FILE_MAX]; /* standard error file name */
char *recvname = "$RECEIVE"; /* $RECEIVE file */
short recvfile; /* file number for $RECEIVE */

/*
 * This procedure converts an internal file descriptor to an OSS pathname.
```

```

*/

short fd_to_pathname(FILE *fp, char *pathname)
{
    short  err;
    char   extfname[FILE_MAX];
    short  len;
    short  pathnamelen;
#ifdef D30
    err = FILE_GETINFO_((short)fdtogfn((short)fileno(fp)), /*lasterr*/ ,
#else
    err = FILE_GETINFO_((short)gfileno(fp), /*lasterr*/ ,
#endif
                        extfname, (short)sizeof(extfname)-1,
                        &len);

    if (err == ENOERR) {
        extfname[len] = '\0';
        err = FILENAME_TO_PATHNAME_(extfname, len,
                                    pathname, PATH_MAX,
                                    &pathnamelen);

        if (err == ENOERR)
            pathname[pathnamelen] = '\0';
    }
    return (err);
}

/*
 * This procedure returns nonzero if the specified OSS PID identifies
 * an existing process. Process handle is retrieved as a test.
 */
int valid_pid(pid_t pid)
{
    short err;
    short err_detail;
    short len;
    short phandle[10];
    short ret_attr = 48; /* get process handle */
    err = PROCESS_GETINFOLIST_(*cpu*/, /*pin*/, /*node*/, /*nodelen*/, /*phandle*/,
                              &ret_attr, 1, phandle, (short)sizeof(phandle), &len,
                              &err_detail, 3 /* get info for PID specified */ ,
                              /*srchattr*/, /*srchattrcnt*/,
                              /*srchval*/, /*srchvalcnt*/, pid);
    return ( err == 0 );
}

int main(int argc, char *argv[]) {
    int i = 1; /* argument counter */
    int remote = 0; /* remote = 0 for local command execution */
                  /* remote = 1 for remote command execution */
    char *pathstring; /* search PATH string */
    char *progrname;

    /* If the name of an OSS program is not provided,
       use the default program "who" */
    if(argc > 1)
        progrname = argv[1];
    else

```

```

        progname = "who";

/* Get standard file pathnames */
if((error = fd_to_pathname(stdin, inpath)) != ENOERR
    || (error = fd_to_pathname(stdout, outpath)) != ENOERR
    || (error = fd_to_pathname(stderr, errpath)) != ENOERR) {
    fprintf(stderr, "Bad file number\n");
    exit(1);
}

/* Build the fdinfo structure header */
/* Allocate memory for structure with three fdentry's */
fdinfo = (fdinfo_def *) malloc(sizeof(fdinit)
                               + 2 * sizeof(fdinit.z_fdentry));

/* Copy initialized structure entries into new memory area */
memcpy(fdinfo, &fdinit, sizeof(fdinit));
fdinfo->z_len = sizeof(fdinit);      /* size of structure */
fdinfo->z_timeout = -1;              /* no timeout to be used */
fdinfo->z_umask = -1;                /* use creation mask of caller */
fdinfo->z_cwd = "/tmp";               /* initial working directory */
fdinfo->z_fdcount = 3;                /* number of files to be opened */
fdentry = &fdinfo->z_fdentry;

/* Build the fdinfo entry for stdin */
fdentry[0].z_fd = 0;                /* file descriptor number */
fdentry[0].z_dupfd = -1;             /* no dup of file desc. */
fdentry[0].z_name = inpath;          /* standard in file name */
fdentry[0].z_oflag = O_RDONLY;       /* open as read only */
fdentry[0].z_mode = 0755;            /* file permission mode */

/* Build the fdinfo entry for stdout */
fdentry[1].z_fd = 1;
fdentry[1].z_dupfd = -1;
fdentry[1].z_name = outpath;
fdentry[1].z_oflag = O_WRONLY;
fdentry[1].z_mode = 0755;

/* Build the fdinfo entry for stderr */
fdentry[2].z_fd = 2;
fdentry[2].z_dupfd = -1;
fdentry[2].z_name = errpath;
fdentry[2].z_oflag = O_RDWR;
fdentry[2].z_mode = 0755;

/* Dummy inheritance structure */
inherit.flags = 0;

/* Build program argument list */
pargv[0] = progname;
if(argc <= 2)
    pargv[1] = NULL;
else
    while(pargv[i] = argv[i+1])
        i++;

/* Assign search PATH string based on whether the program/process is
be run on a remote system or a local system */
if(remote)
    pathstring = "/E/native/bin:/E/native/usr/bin:/E/native/usr/local/bin";
else
    pathstring = "/bin:/usr/bin:/usr/local/bin";

```

```

/* Create the new OSS process and run program */
pid = PROCESS_SPAWN_(
    progname,          /* OSS pathname of program to be run */
    fdinfo,            /* file descriptor information structure */
    pargv,             /* pointer to argument strings */
    environ,           /* pointer to array of addr. of env. values */
    &inherit,          /* inheritance structure */
    sizeof(inherit),   /* length of structure */
    &proc_ext,         /* process extension structure */
    &proc_results,     /* process results structure */
    ,                  /* nowait tag (not used in this example) */
    pathstring         /* use PATH search string */
);
/* If unsuccessful, print out error message and exit */
if (pid == -1) {
    fprintf(stderr, "OSS errno = %d\n", proc_results.pr_errno);
    fprintf(stderr, "Guardian error = %d\n", proc_results.pr_TPCerror);
    fprintf(stderr, "Error detail = %d\n", proc_results.pr_TPCdetail);
    exit(1);
}
/* Wait for death of child process */
MONITORCPUS(-1); /* Request CPU status messages */
if (error = FILE_OPEN_(recvname, (short)strlen(recvname), &recvfile))
    exit(1);
/* Read system messages on $RECEIVE */
do {
    READX(recvfile, (char *)&msg.msgcode, (short)sizeof(msg));
    FILE_GETINFO_(recvfile, &error);
} while (!error
    || error == 6 /* system message */
    && !(msg.msgcode == ZSYS_VAL_SMSG_PROCDEATH /* death message */
        && msg.procdeath.z_ossid == pid /* process ID match */
        && msg.procdeath.z_completion_code
            != ZSYS_VAL_PROCDEATH_CHILDEXEC) /* completion code */
    && !(msg.msgcode == ZSYS_VAL_SMSG_CPUDOWN /* CPU down message check*/
        && !valid_pid(pid))); /* child process ID invalid*/
printf("Child process terminated\n");
exit(0);
}

```

Creating an OSS Process using PROCESS_SPAWN64_

```

#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <limits.h>
#include <errno.h>
#include <spawn.h>
#include <tdmext.h>
#include <zsysc>
#include <cextdecs(FILE_OPEN_, FILE_GETINFO_, FILE_READ64_, MONITORCPUS,\
    PROCESS_SPAWN64_, PROCESS_GETINFOLIST_, FILENAME_TO_PATHNAME_)>

/* Structures for standard I/O file descriptor information */
fdentry64_def *fdentry; /* structure pointer for fdentry64 */

```

```

fdinfo64_def *fdinfo; /* structure pointer to fdinfo64 block with one fdentry */

fdinfo_def fdinit = DEFAULT_FDINFO64; /* initialize structure */
/* Structure for inheritance values */
struct inheritance inherit;
/* Set defaults for process spawn input/output structures */
struct process_extension
    proc_ext = DEFAULT_PROCESS_EXTENSION;
struct process_extension_results
    proc_results = DEFAULT_PROCESS_EXTENSION_RESULTS;
/* Structure used for process termination message */
union {
    short msgcode; /* message code */
    zsys_ddl_msg_procdeath_def procdeath; /* termination message format */
} msg;

extern char **environ;
#define FILE_MAX ZSYS_VAL_LEN_FILENAME /* maximum characters in file name */
char *programe; /* Name of program to be executed */
pid_t pid; /* OSS process ID */
char *pargv[4]; /* arguments for program to be run */
short error; /* error code, if any */
char inpath[FILE_MAX]; /* standard input file name */
char outpath[FILE_MAX]; /* standard output file name */
char errpath[FILE_MAX]; /* standard error file name */
char *recvname = "$RECEIVE"; /* $RECEIVE file */
short recvfile; /* file number for $RECEIVE */

/*
 * This procedure converts an internal file descriptor to an OSS pathname.
 */

short fd_to_pathname(FILE *fp, char *pathname)
{
    short err;
    char extfname[FILE_MAX];
    short len;
    short pathnamelen;

    err = FILE_GETINFO_((short)gfileno(fp), /*lasterr*/,
                        extfname, (short)sizeof(extfname)-1,
                        &len);
    if (err == ENOERR) {
        extfname[len] = '\0';
        err = FILENAME_TO_PATHNAME_(extfname, len,
                                    pathname, PATH_MAX,
                                    &pathnamelen);

        if (err == ENOERR)
            pathname[pathnamelen] = '\0';
    }
    return (err);
}

/*
 * This procedure returns nonzero if the specified OSS PID identifies
 * an existing process. Process handle is retrieved as a test.
 */
int valid_pid(pid_t pid)

```

```

{
    short err;
    short err_detail;
    short len;
    short phandle[10];
    short ret_attr = 48; /* get process handle */

    err = PROCESS_GETINFOLIST_(*cpu*/,/*pin*/,/*node*/,/*nodelen*/,/*phandle*/,
                               &ret_attr, 1, phandle, (short)sizeof(phandle), &len,
                               &err_detail, 3 /* get info for PID specified */ ,
                               /*srchattrib*/, /*srchattribcnt*/,
                               /*srchval*/, /*srchvalcnt*/, pid);

    return ( err == 0 );
}

int main(int argc, char *argv[]) {
    int i = 1; /* argument counter */
    int remote = 0; /* remote = 0 for local command execution */
                    /* remote = 1 for remote command execution */
    char *pathstring; /* search PATH string */
    char *progname;
    /* If the name of an OSS program is not provided,
       use the default program "who" */
    if(argc > 1)
        progname = argv[1];
    else
        progname = "who";
    /* Get standard file pathnames */
    if((error = fd_to_pathname(stdin, inpath)) != ENOERR
        || (error = fd_to_pathname(stdout, outpath)) != ENOERR
        || (error = fd_to_pathname(stderr, errpath)) != ENOERR) {
        fprintf(stderr, "Bad file number\n");
        exit(1);
    }
    /* Build the fdinfo structure header */
    /* Allocate memory for structure with three fdentry's */
    fdinfo = (fdinfo64_def *) malloc(sizeof(fdinit)
                                     + 2 * sizeof(fdinit.z_fdentry));

    /* Copy initialized structure entries into new memory area */
    memcpy(fdinfo, &fdinit, sizeof(fdinit));
    fdinfo->z_len = sizeof(fdinit); /* size of structure */
    fdinfo->z_timeout = -1; /* no timeout to be used */
    fdinfo->z_umask = -1; /* use creation mask of caller */
    fdinfo->z_cwd = "/tmp"; /* initial working directory */
    fdinfo->z_fdcount = 3; /* number of files to be opened */
    fdentry = &fdinfo->z_fdentry;

    /* Build the fdinfo entry for stdin */
    fdentry[0].z_fd = 0; /* file descriptor number */
    fdentry[0].z_dupfd = -1; /* no dup of file desc. */
    fdentry[0].z_name = inpath; /* standard in file name */
    fdentry[0].z_oflag = O_RDONLY; /* open as read only */
    fdentry[0].z_mode = 0755; /* file permission mode */
    /* Build the fdinfo entry for stdout */
    fdentry[1].z_fd = 1;
    fdentry[1].z_dupfd = -1;
    fdentry[1].z_name = outpath;

```

```

fdentry[1].z_oflag = O_WRONLY;
fdentry[1].z_mode = 0755;
/* Build the fdinfo entry for stderr */
fdentry[2].z_fd = 2;
fdentry[2].z_dupfd = -1;
fdentry[2].z_name = errpath;
fdentry[2].z_oflag = O_RDWR;
fdentry[2].z_mode = 0755;

/* Dummy inheritance structure */
inherit.flags = 0;

/* Build program argument list */
pargv[0] = progname;
if(argc <= 2)
    pargv[1] = NULL;
else /* copy command arguments */
    while(pargv[i] = argv[i+1])
        i++;
/* Assign search PATH string based on whether the program/process is
   be run on a remote system or a local system */
if(remote)
    pathstring = "/E/native/bin:/E/native/usr/bin:/E/native/usr/local/bin";
else
    pathstring = "/bin:/usr/bin:/usr/local/bin";

/* Create the new OSS process and run program */
pid = PROCESS_SPAWN64_(
    progname, /* OSS pathname of program to be run */
    fdinfo, /* file descriptor information structure */
    pargv, /* pointer to argument strings */
    environ, /* pointer to array of addr. of env. values */
    &inherit, /* inheritance structure */
    sizeof(inherit), /* length of structure */
    &proc_ext, /* process extension structure */
    &proc_results, /* process results structure */
    , /* nowait tag (not used in this example) */
    pathstring /* use PATH search string */
);

/* If unsuccessful, print out error message and exit */
if (pid == -1) {
    fprintf(stderr, "OSS errno = %d\n", proc_results.pr_errno);
    fprintf(stderr, "Guardian error = %d\n", proc_results.pr_TPCError);
    fprintf(stderr, "Error detail = %d\n", proc_results.pr_TPCdetail);
    exit(1);
}
/* Wait for death of child process */
MONITORCPUS(-1); /* Request CPU status messages */
if (error = FILE_OPEN_(recvname, (short)strlen(recvname), &recvfile))
    exit(1);

/* Read system messages on $RECEIVE */
do {
    error = FILE_READ64_(recvfile, (char *)&msg.msgcode, (short)sizeof(msg));
} while (!error
    || error == 6 /* system message */
    && !(msg.msgcode == ZSYS_VAL_SMSG_PROCDEATH /* death message */

```



```

    && msg.procdeath.z_ossid == pid /* process ID match */
    && msg.procdeath.z_completion_code
    != ZSYS_VAL_PROCDEATH_CHILDEXEC) /* completion code */
    && !(msg.msgcode == ZSYS_VAL_SMSG_CPUDOWN /* CPU down message check*/
    && !valid_pid(pid)); /* child process ID invalid*/
printf("Child process terminated\n");
exit(0);
}

```

Querying the Process Environment

The OSS API and the Guardian API both allow you to get information about both OSS and Guardian processes and their environments.

Using the OSS API

OSS functions provide information about process times (`times()`), environment variables (`getenv()`), user IDs (`getuid()`, `geteuid()`), groups and group IDs (`getgroups()`, `getgid()`, `getegid()`), and system limits (`sysconf()`) for OSS and Guardian processes. Because Guardian processes do not have OSS process IDs and cannot belong to process groups, the `getpid()`, `getppid()`, and `getpgrp()` functions operate only on OSS processes. You can call `sysconf()` from a Guardian process, but it is not meaningful to do so.

Getting Information About System Limits Using `sysconf()` gets the system-dependent system limits of a process and sends their values to the standard output file. The system limits shown in this example are not in the `limits.h` header file because they are not available at compile time.

Getting Information About System Limits Using `sysconf()`

```

#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

static void pr_sysconf(char *, int);
int main (int argc, char *argv[])
{
    /* Retrieve system-dependent system limits. */
    pr_sysconf("ARG_MAX", _SC_ARG_MAX);
    pr_sysconf("CHILD_MAX", _SC_CHILD_MAX);
    pr_sysconf("clock ticks/second", _SC_CLK_TCK);
    pr_sysconf("NGROUPS_MAX", _SC_NGROUPS_MAX);
    pr_sysconf("OPEN_MAX", _SC_OPEN_MAX);
    #ifdef _SC_STREAM_MAX
        pr_sysconf("STREAM_MAX", _SC_STREAM_MAX);
    #endif
    #ifdef _SC_TZNAME_MAX
        pr_sysconf("TZNAME_MAX", _SC_TZNAME_MAX);
    #endif
    pr_sysconf("_POSIX_JOB_CONTROL", _SC_JOB_CONTROL);
    pr_sysconf("_POSIX_SAVED_IDS", _SC_SAVED_IDS);
    pr_sysconf("_POSIX_VERSION", _SC_VERSION);
    exit(0);
}

/* This function displays system-dependent limits to standard output.*/
static void pr_sysconf(char *mesg, int name)

```



```

{
    long val;
    fputs(mesg, stdout);
    errno = 0;
    if ( (val = sysconf(name)) < 0) {
        if (errno != 0) {
            printf("sysconf error\n");      /* not a valid constant name */
            exit(1);
        }
        fputs(" (not defined)\n", stdout); /* value is indeterminate */
    } else
        printf(" %ld\n", val);
}

```

Using the Guardian API

The Guardian procedures that provide information about OSS processes as well as Guardian processes are `PROCESS_GETINFO_` and `PROCESS_GETINFOLIST_`.

Beginning with the H06.24 and J06.13 RVUs, additional 64-bit process attributes are available. For information about 64-bit process attributes, see **New PROCESS_GETINFOLIST_ Attributes**.

The `PROCESS_GETINFO_` procedure returns a limited set of information about a specified process, including the process handle, execution priority, home terminal, type (OSS or Guardian), swap file and program file statistics, and the OSS process ID if it is an OSS process.

The `PROCESS_GETINFOLIST_` procedure returns detailed information about a process or set of processes that meets specified criteria. These criteria identify processes for which information is returned.

Getting Information About a Process Using PROCESS_GETINFO_ gets information about an OSS or Guardian process by obtaining the process handle for the process and then passing the process handle to `PROCESS_GETINFO_`. If no process name is provided from the command line, information about the current process is retrieved and printed.

Getting Information About a Process Using PROCESS_GETINFO_

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <cextdecs.h(FILENAME_TO_PROCESSHANDLE_, \
                    PROCESS_GETINFO_, PROCESSHANDLE_GETMINE_)>

short processhandle[10];
short mom_processhandle[10];
short gmom_processhandle[10];
short fnlen, priority, htlen, caid, paid, jobid, proglen, swaplen;
short errordetail;
long long proc_time;
char proc_fname[64];
char hometerm[64];
char programfile[64];
char swapfile[64];
short proctype;
long osspid;

int main(int argc, char *argv[]) {
    short retcode;

    /* Get the process name from the command line.  If not supplied, use the

```

```

current process. Use the process name to get the process handle. */
if(argc > 1) {
    retcode = FILENAME_TO_PROCESSHANDLE_(
        argv[1], /* process filename */
        (short) strlen(argv[1]), /* length of process filename */
        processhandle /* process handle */
    );
    if(retcode != 0) {
        fprintf(stderr, "Invalid process name %s\n", argv[1]);
        exit(1);
    }
} else {
    printf("Using current process\n");
    retcode = PROCESSHANDLE_GETMINE_(processhandle);
    if (retcode != 0) {
        fprintf(stderr, "Can't get current process handle\n");
        exit(1);
    }
}

/* Get process information and print the values. */
retcode = PROCESS_GETINFO_(
    processhandle, /* process handle */
    proc_fname, /* process filename */
    64, /* maximum length */
    &fnlen, /* length of filename */
    &priority, /* current execution priority */
    mom_processhandle, /* process handle of MOM process */
    hometerm, /* home terminal */
    64, /* maximum length */
    &htlen, /* length of hometerm name */
    &proc_time, /* execution time in usec */
    &caid, /* creator access ID */
    &paid, /* process access ID */
    gmom_processhandle, /* process handle of GMOM */
    &jobid, /* job ID */
    programfile, /* program filename */
    64, /* maximum length */
    &proglen, /* length of program filename */
    swapfile, /* swap filename */
    64, /* maximum length */
    &swaplen, /* length of swap filename */
    &errordetail, /* details of any errors */
    &proctype, /* process type (1=OSS,0=Guardian) */
    &osspid /* OSS process ID, if type=1 */
);

/* If unsuccessful, print error message and exit. */
if (retcode != 0) {
    fprintf(stderr, "retcode = %d\n", retcode);
    fprintf(stderr, "Failure details = %d\n",errordetail);
    exit(1);
}
printf("Process filename = %s\n", proc_fname);
printf("Process priority = %d\n", priority);
printf("Home terminal = %s\n", hometerm);
printf("Process execution time = %ld microseconds\n", proc_time);

```

```

printf("CAID = %d, PAID = %d\n", caid, paid);
printf("Job ID = %d\n", jobid);
printf("Program filename = %s\n", programfile);
printf("Swap filename = %s\n", swapfile);
if(proctype == 1)
    printf("OSS process ID = %d\n", osspid);
exit(0);
}

```

Getting Information About a Process Using PROCESS_GETINFOLIST_ gets information about an OSS process by passing the OSS process ID to PROCESS_GETINFOLIST_. If no OSS process ID is provided from the command line, the OSS process ID of the current process is obtained. Process information is returned in the `values_list` buffer and printed.

Getting Information About a Process Using PROCESS_GETINFOLIST_

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <cextdecs.h(PROCESS_GETINFOLIST_)>

short processhandle[10];
short attr_list[] = { /* list of attributes for which values are
                      to be returned */
    21, /* real group ID */
    22, /* real user ID */
    80, /* effective group ID */
    23, /* effective user ID */
    90, /* OSS process ID */
    94, /* OSS parent process ID */
    26, /* OSS session leader ID */
    98, /* OSS group leader PID */
    27, /* OSS controlling terminal */
    111, /* current size of native heap area */
    112, /* maximum size of native heap area */
    31, /* wait state */
    32, /* process state */
    48 /* process handle */
};

short attr_count; /* number of attributes */
short values_list[8000]; /* buffer of return values */
short values_len; /* size of return values information */
short errordetail; /* additional error information */
short srch_option; /* search option */
short srch_attr_list[] = { 28 }; /* attribute code specifying search
                                criteria. 28 is process type (OSS
                                or Guardian). */

short srch_attr_count; /* number of search criteria */
short srch_values_list[1024]; /* buffer of return values */
short srch_values_len; /* size of return values */
char ctty[100]; /* controlling terminal */
pid_t osspid; /* OSS process ID */

int main(int argc, char *argv[]) {
    int i, nbytes;
    short retcode;

```



```

/* Get information for the process specified by the OSS process ID. */
if(argc > 1) {
    osspid = atoi(argv[1]);
} else
    osspid = getpid();
attr_count = (short) (sizeof(attr_list)/sizeof(short));
srch_option = 3; /* return info for process whose
                  OSS process ID is specified */

retcode = PROCESS_GETINFOLIST_(
    , /* processor */
    , /* process ident number (PIN) */
    , /* node name */
    , /* length of node name */
    , /* process handle */
    attr_list, /* attribute list to be returned */
    attr_count, /* number of attributes */
    values_list, /* list of output values */
    1024, /* maximum number of values */
    &values_len, /* length of list returned */
    &errordetail, /* details of any errors */
    srch_option, /* search for OSS process */
    , /* attribute codes used in search */
    , /* number of attributes used */
    , /* match values found */
    , /* length of list returned */
    osspid /* OSS process ID */
);

/* If unsuccessful, print error message and exit. */
if (retcode != 0) {
    fprintf(stderr, "retcode = %d\n", retcode);
    fprintf(stderr, "Failure details = %d\n",errordetail);
    exit(1);
}

printf("Printing selected process attributes:\n\n");
i = 0; /* start at beginning of the buffer printing out
        the values of process attributes returned */
printf("Real group ID = %d\n", (*(gid_t*)(&values_list[i])));
i += (sizeof(gid_t)/sizeof(short));
printf("Real user ID = %d\n", (*(uid_t*)(&values_list[i])));
i += (sizeof(uid_t)/sizeof(short));
printf("Effective group ID = %d\n", (*(gid_t*)(&values_list[i])));
i += (sizeof(gid_t)/sizeof(short));
printf("Effective user ID = %d\n", (*(uid_t*)(&values_list[i])));
i += (sizeof(uid_t)/sizeof(short));
printf("OSS process ID = %d\n", (*(pid_t*)(&values_list[i])));
i += (sizeof(pid_t)/sizeof(short));
printf("Parent process ID = %d\n", (*(pid_t*)(&values_list[i])));
i += (sizeof(pid_t)/sizeof(short));
printf("Session leader = %d\n", (*(pid_t*)(&values_list[i])));
i += (sizeof(pid_t)/sizeof(short));
printf("Group leader = %d\n", (*(pid_t*)(&values_list[i])));
i += (sizeof(pid_t)/sizeof(short));
nbytes = values_list[i++];

```

```

    strncpy(ctty, (char *)&values_list[i], nbytes);
    printf("Control tty = %s\n", ctty);
    i += (nbytes+1)/(sizeof(short));
    printf("Current size of native heap area = %d bytes\n", values_list[i]);
    i += 2;    /* 2 short's */
    printf("Maximum size of native heap area = %d bytes\n", values_list[i]);
    i += 2;    /* 2 short's */
    printf("Process wait state = %o\n", values_list[i++]);
    printf("Process state = %o\n", values_list[i++]);
    printf("Process handle = %o,%o,%o,%o,%o\n",
           values_list[i],
           values_list[i+1],
           values_list[i+2],
           values_list[i+3],
           values_list[i+4]);
    exit(0);
}

```

Modifying the Process Environment

The OSS API and the Guardian API both allow you to modify characteristics about both OSS and Guardian processes and their environments.

Using the OSS API

OSS functions can change information about process environment variables (`putenv()`), user IDs (`setuid()`), and group IDs (`setgid()`) for OSS and Guardian processes. Because Guardian processes do not have OSS process IDs and cannot belong to process groups or sessions, the `setsid()` and `setpgid()` functions operate only on OSS processes.

Setting the User ID of the Current Process Using `setuid()` sets the user ID of the current process to the user ID of the user whose name is supplied at the command line. This program must be run using the super ID.

Setting the User ID of the Current Process Using `setuid()`

```

#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pwd.h>

int main(int argc, char *argv[])
{
    uid_t uid;
    struct passwd *pp;
    int retcode;

    /* Read the name of the user from the command line.
       If the user name is not valid, print an error message and exit.
       If the user name is not provided, print a usage message and exit. */
    if(argc > 1) {
        pp = getpwnam(argv[1]);
        if (!pp) {
            fprintf(stderr, "Unknown user name\n");
            exit (1);
        }
    }
}

```



```

    } else {
        fprintf(stderr, "Usage: procinf2 user name\n" );
        exit (1);
    }

/* Set user ID to ID of user name entered at the command line. */
uid = pp->pw_uid;

/* Print user ID before and after changing it. */
printf("Current UID = %d\n", getuid());
retcode = setuid(uid);
printf("New UID = %d\n", getuid());
if(retcode != 0) {
    fprintf(stderr, "Can't set UID to %d\n", uid);
    exit(1);
} else
    exit(0);
}

```

Using the Guardian API

The `PROCESS_SETINFO_` and `PROCESS_SETSTRINGINFO_` procedures allow you to change information about OSS as well as Guardian processes. `PROCESS_SETINFO_` operates on integer process attributes, and `PROCESS_SETSTRINGINFO_` operates on string attributes.

Modifying Information About the Current Process Using `PROCESS_SETINFO_` and `PROCESS_SETSTRINGINFO_` changes the execution priority of the current process using `PROCESS_SETINFO_` and sets the home terminal to a new window using `PROCESS_SETSTRINGINFO_`.

Modifying Information About the Current Process Using `PROCESS_SETINFO_` and `PROCESS_SETSTRINGINFO_`

```

/* Modifying info about the current process using PROCESS_SETINFO_ and */
/* PROCESS_SETSTRINGINFO_ (works on H-series RVUs only) */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <cextdecs.h(PROCESS_SETINFO_,PROCESS_SETSTRINGINFO_)>

short priority = 150;
short oldpriority, oldvaluelen;
char *hometerm;

int main(int argc, char *argv[]) {
    short retcode;

/* Get name of home terminal from command line.
   If none provided, use default setting. */
    if (argc > 1)
        hometerm = argv[1];
    else
        #ifdef _TNS_E_TARGET
            hometerm = "$ZTNT.#PTY00D4" ;    /* Note 1 */
        #else
            hometerm = "$ZTN0.#PTY00D4" ;    /* Note 1 */
        #endif

/* Set process execution priority to a specified value. */
    retcode = PROCESS_SETINFO_(

```



```

        ,                /* process handle of current process */
        ,                /* process specifier */
        (short)42,       /* current priority attribute code */
        &priority,       /* priority value */
        (short)1,       /* length of value (short) */
        &oldpriority,    /* returned value of priority */
        (short)1,
        &oldvaluelen     /* length of returned value */
    );

/* If unsuccessful, print error message and exit. */
if (retcode != 0) {
    fprintf(stderr, "retcode = %d\n", retcode);
    exit(1);
}
printf("Current process priority set to %d\n", priority);
/* Set process home terminal to new value. */
retcode = PROCESS_SETSTRINGINFO_(
    ,                /* process handle of current process */
    ,                /* process specifier */
    5,               /* home terminal attribute code */
    hometerm,        /* new home terminal string value */
    (short)strlen(hometerm), /* length of string */
);

/* If unsuccessful, print error message and exit. */
if (retcode != 0) {
    fprintf(stderr, "retcode = %d\n", retcode);
    exit(1);
}
printf("Home terminal set to %s\n", hometerm);
exit(0);
}

```

Note 1

You can get the name of a terminal using the `ttyname()` function or the `tty` command. OSS terminal names are in the format `/G/ztnt/#pty00nn`. The `PROCESS_SETINFO_` procedure requires the Guardian form of a terminal name, which is in the format:

- `$ZTN0.#PTY00xx` for systems running G-series RVUs.
- `$ZTNT.#PTY00xx` for systems running H-series RVUs.

To convert from an OSS pathname format to a Guardian filename format, use the `PATHNAME_TO_FILENAME_` procedure.

Performance Considerations

You can improve the performance of your applications with respect to process management by following these recommendations:



- Minimize the use of the process-creation functions. There is a lot of system overhead involved in creating processes, so use static server and demon processes whenever possible. Create a process and do not terminate it until the program no longer needs it.
- Distribute the workload across processors. Specify the processor using the Hewlett Packard Enterprise extension functions when creating a process.
- Create processes in one step instead of two. Use the `tdm_spawn()` or `tdm_spawnp()` function instead of calling the `fork()` or `tdm_fork()` function followed by one of the `exec` or `tdm_exec` set of functions.

The following subsections illustrate these performance considerations and actions:

- **Using Dynamic Servers Spread Across Available Processors**
- **Using Static Servers Spread Across Available Processors**

Using Dynamic Servers Spread Across Available Processors

In contrast, **Dynamic Launcher Program to Spawn Dynamic Server** offers a more flexible solution to the programs in the sample application of **Dynamic Launcher Program** and **Dynamic Echo Server Program**. This new program uses the `tdm_spawn()` function, instead of one of the `fork()` calls and the `execv()` call, to allow for scalability to other processors and for slightly better process-creation performance.

The launcher program in **Dynamic Launcher Program to Spawn Dynamic Server** is compiled as a linked program file named `dlaunch2`. The dynamic server program in **Dynamic Echo Server Program** is compiled as a linked program file named `dserver`. (A requester program, shown in **Using AF_INET in an OSS Requester Process**, is also compiled as a linked program file named `requester2`.) All programs are secured for execution in the current working directory.

In this application, `dlaunch2` creates an `AF_INET` socket and listens for incoming connections from one or more copies of `requester2`. Upon receipt of a connection, `dlaunch2` starts `dserver` in another available processor, using a `tdm_spawn()` call. `dlaunch2` closes the file descriptors associated with its `requester2` connection. `dserver`, which has inherited file descriptors for the `requester2` connection, handles all work associated with the connection.

Using two terminal sessions, sample dialog for the terminal launching `dlaunch2` might be:

```
/users/chris: ./dlaunch2
dlaunch2 starting in cpu 2
dserver spawn'ed in cpu 0
dserver [cpu 0] - starting
dserver [cpu 0] - message received on fd 0 = hello 1
dserver [cpu 0] - message received on fd 0 = goodbye 1
dserver [cpu 0] - stopping [input closed]
dserver spawn'ed in cpu 1
dserver [cpu 1] - starting
dserver [cpu 1] - message received on fd 0 = hello 2
dserver [cpu 1] - message received on fd 0 = goodbye 2
dserver [cpu 1] - stopping [input closed]
dlaunch2 stopping - timeout
```

while the terminal running the copy of `requester2` might show:

```
/users/chris: ./requester2
Requester starting
?hello 1
Read 8 bytes back from server
?goodbye 1
Read 10 bytes back from server
```



```

?/users/chris: ./requester2
Requester starting
?hello 2
Read 8 bytes back from server
?goodbye 2
Read 10 bytes back from server
?
/users/chris:

```

Dynamic Launcher Program to Spawn Dynamic Server

```

/* dlaunch2.c
 *
 * Simple OSS example using OSS socket APIs for communication
 * between requester and server.
 *
 * dlaunch2 (dynamic launcher 2) is an inetd-like program which
 * accepts socket connections and spawn's a dserver process for
 * each connection.
 *
 * dserver reads stdin and echoes to stdout. dlaunch2 and dserver
 * write progress messages and error messages to stderr.
 *
 * dlaunch2 starts the dserver processes run on different cpus.
 */

#define _XOPEN_SOURCE_EXTENDED 1          /* needed for OSS sockets */

#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <netinet/in.h>

#include <tdmext.h>

/* Guardian APIs */
#include <cextdecs.h>(PROCESSHANDLE_GETMINE_,      \
                    PROCESSHANDLE_DECOMPOSE_,     \
                    PROCESSORSTATUS)>

short cpucount;          /* cpucount - # cpu in system */
short cpuindex;          /* cpuindex - current cpu for dserver */
short cpumask;           /* cpumask - bit mask of available cpus */
short cpu, phandle[10];  /* our cpu and process handle */
int newpid, fd, listenfd; /* process id, file descriptor,
                          * listening fd */
int readycount, bytesread; /* ready fd count, recv byte count */
struct sockaddr_in addr;   /* socket address (family,addr,port) */
struct timeval timeval;    /* timeout structure for select() call */
fd_set fdset;              /* set of fds for select() call */
#define IDLE_PERIOD 180    /* 180 second timeout value */

```



```

extern char **environ;
static char *childargv[] = {"dserver", NULL};
struct inheritance          inherit;
struct process_extension    peparm = DEFAULT_PROCESS_EXTENSION;
struct process_extension_results perslt =
                                DEFAULT_PROCESS_EXTENSION_RESULTS;

int main(int argc, char **argv)
{
    /* get system cpu config */
    cpumask = (short) PROCESSORSTATUS();
    cpucount = (short) (PROCESSORSTATUS() >> 16);
    cpuindex = -1;

    /* get process handle and cpu number */
    PROCESSHANDLE_GETMINE_(phandle);
    PROCESSHANDLE_DECOMPOSE_(phandle, &cpu);

    /* print startup message */
    fprintf(stderr, "dlaunch2 starting in cpu %d\n", cpu);

    /* create AF_INET stream socket */
    if ((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        fprintf(stderr, "socket() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* Get server port from the command line or use default 12345 */
    if (argc > 1)
        addr.sin_port = (short)atoi(argv[1]); /* use command line arg */
    else
        addr.sin_port = 12345;                /* use default */

    addr.sin_family      = AF_INET;          /* internet address family */
    addr.sin_addr.s_addr = gethostid(); /* assume server on our host */

    /* bind socket to address */
    if (bind(fd, (const struct sockaddr *)&addr, sizeof(addr)) < 0)
    {
        fprintf(stderr, "bind() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* make the socket a listening socket */
    if (listen(fd, 5) < 0)
    {
        fprintf(stderr, "listen() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* fd usage
    * 0 stdin    - not used by dlaunch2, set up for dserver
    * 1 stdout   - not used by dlaunch2, set up for dserver
    * 2 stderr   - used by dlaunch2 for messages,
    *              inherited by dserver
    * 3 listenfd - used by dlaunch2 for connections,

```

```

*           not inherited by dserver
*/

close(STDIN_FILENO);
close(STDOUT_FILENO);
listenfd = fd;
fcntl(listenfd, F_SETFD, FD_CLOEXEC); /* don't propagate fd to children */
FD_ZERO(&fdset);

/* loop until error or timeout */
while (1)
{
    /* initialize timeout and bitmask for select() call */
    timeval.tv_sec = IDLE_PERIOD;
    timeval.tv_usec = 0;
    FD_SET(listenfd, &fdset);
    readycount = select(listenfd+1, &fdset, NULL, NULL, &timeval);

    /* if timeout then exit */
    if (readycount == 0)
    {
        fprintf(stderr, "dlaunch2 stopping - timeout\n");
        exit(0);
    }

    /* accept new connection */
    if ((fd = accept(listenfd, (struct sockaddr *)NULL,
        (size_t *)NULL)) < 0)
    {
        fprintf(stderr, "accept() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* setup stdin and stdout */
    dup2(fd, STDIN_FILENO); /* fd is already STDIN_FILENO */
    dup2(fd, STDOUT_FILENO); /* setup stdout */

    /* launch a dserver process to handle the connection */
    /* note - spawn is an expensive operation and is more */
    /* flexible than exec */

    /* find next available cpu */
    do {
        cpuindex = (++cpuindex == cpucount) ? 0 : cpuindex;
    } while (((cpumask >> (15-cpuindex)) & 1) == 0);

    peparm.pe_cpu = cpuindex;
    inherit.flags = 0;

    if ((newpid = fork()) < 0)
    {
        fprintf(stderr, "fork() failed [errno %d]\n", errno);
        exit(1);
    }
    else if (newpid == 0)
    {
        /* we are child ... spawn grandchild to avoid zombie processes */

```

```

        tdm_spawn(childargv[0], 0, NULL, &inherit,
                  childargv, environ, &peparm, &perslt);
        exit(0); /* child goes away */
    }
    else
    {
        /* we are parent */
        fprintf(stderr, "dserver spawn'ed in cpu %d\n", peparms.pe_cpu);
        wait(&newpid); /* wait for child to avoid zombie child process */
        close(STDIN_FILENO);
        close(STDOUT_FILENO);
    }
} /* end while (1) */
}

```

Using Static Servers Spread Across Available Processors

Dynamic Launcher Program to Spawn Dynamic Server can be further optimized as a launcher for iterative static servers, as shown in **Launcher Program to Spawn Static Servers**. This program uses AF_UNIX sockets and launches a static server in each processor as soon as it starts; that provides maximum scalability immediately and slightly better responsiveness by eliminating the process-creation delay when handling each new connection.

The server program in **Launcher Program to Spawn Static Servers** is compiled as a linked program file named `slaunch1`. The server program in **Static Echo Server** is compiled as a linked program file named `sserver`. (A requester program, shown in **Using AF_INET in an OSS Requester Process**, is also compiled as a linked program file named `requester2`.) All programs are secured for execution in the current working directory.

In this application, `slaunch1` creates an AF_INET socket and listens for incoming connections from one or more copies of `requester2`. It then starts a copy of `sserver` in each available processor.

Launcher Program to Spawn Static Servers

```

/*slaunch1.c

* Simple OSS example using OSS sockets APIs for communication
* between requester and server.
*
* slaunch1 (static launcher 1) is an inetd-like program which
* starts an sserver (static server) process on each available cpu
* and then passes each incoming socket connection to a sserver process.
*
* slaunch1 and sserver write progress messages and error messages
* to stderr.
*/

#define _XOPEN_SOURCE_EXTENDED 1          /* needed for OSS sockets */

#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <sys/time.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>

```

```

#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netdb.h>

#include <tdmext.h>

/* Guardian APIs */
#include <cextdecs.h>(PROCESSHANDLE_GETMINE_, \
                    PROCESSHANDLE_DECOMPOSE_, \
                    PROCESSORSTATUS)>

short cpucount;          /* cpucount - # cpu in system */
short cpuindex;          /* cpuindex - current cpu for sserver */
short cpumask;           /* cpumask - bit mask of available cpus */
short cpu, phandle[10];  /* our cpu and process handle */
int pairfd[2];           /* socketpair fd values */
int serverfd[16];        /* server fd indexed by cpu number */
int newpid, fd, listenfd; /* process id, file descriptor, listening fd */
int readycount, bytesread; /* ready fd count, recv byte count */
struct sockaddr_in addr;  /* socket address (family,addr,port) */
struct timeval timeval;   /* timeout structure for select() call */
fd_set fdset;            /* set of fds for select() call */
struct msghdr fdmsghdr;   /* msghdr for sendmsg() call */
struct iovec iov[1];      /* io vector for sendmsg() call */
struct {                 /* ancillary data for sendmsg() call */
    struct cmsghdr fdcmshdr; /* ancillary data header */
    int fd;                 /* ancillary data (one fd) */
} fdadata;

#define IDLE_PERIOD 180 /* 180 second timeout value */

extern char **environ;
static char *childargv[] = {"sserver", NULL};

struct inheritance inherit;
struct process_extension peparm = DEFAULT_PROCESS_EXTENSION;
struct process_extension_results perslt = DEFAULT_PROCESS_EXTENSION_RESULTS;

int main(int argc, char **argv)
{
    /* get system cpu config */
    cpumask = (short) PROCESSORSTATUS();
    cpucount = (short) (PROCESSORSTATUS() >> 16);
    cpuindex = -1;

    /* get process handle and cpu number */
    PROCESSHANDLE_GETMINE_(phandle);
    PROCESSHANDLE_DECOMPOSE_(phandle, &cpu);

    /* print startup message */
    fprintf(stderr, "slaunch1 starting in cpu %d\n", cpu);

    /* create AF_INET stream socket */
    if ((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        fprintf(stderr, "socket() call failed [errno %d]\n", errno);
        exit(1);
    }
}

```

```

/* Get server port from the command line or use default 12345 */
if (argc > 1)
    addr.sin_port = (short)atoi(argv[1]); /* use command line arg */
else
    addr.sin_port = 12345;                /* use default */

addr.sin_family      = AF_INET;          /* internet address family */
addr.sin_addr.s_addr = gethostid(); /* assume server on our host */

/* bind socket to address */
if (bind(fd, (const struct sockaddr *)&addr, sizeof(addr)) < 0)
{
    fprintf(stderr, "bind() call failed [errno %d]\n", errno);
    exit(1);
}

/* make the socket a listening socket */
if (listen(fd, 5) < 0)
{
    fprintf(stderr, "listen() call failed [errno %d]\n", errno);
    exit(1);
}

listenfd = fd;
fcntl(listenfd, F_SETFD, FD_CLOEXEC); /* don't propagate fd on spawn */
FD_ZERO(&fdset);

/* fd usage
 * 2 stderr    - used by slaunch1 for messages,
 *              inherited by sserver
 * 3 listenfd  - used by slaunch1 for connections, not inherited by sserver
 * y fd        - accepted connection, passed to sserver and closed
 * x serverfd[cpu] - slaunch1 fd corresponding to sserver stdin
 */

/* spawn a static server on each available cpu */
for (cpuindex = 0; cpuindex < cpucount; cpuindex++)
{
    if ((cpumask >> (15-cpuindex)) & 1)
    {
        /* use AF_UNIX socket pair for slaunch1/sserver fd passing */
        if (socketpair(AF_UNIX, SOCK_STREAM, 0, pairfd) < 0)
        {
            fprintf(stderr, "socketpair() call failed [errno %d]\n",
                        errno);
            exit(1);
        }

        /* setup stdin and stdout */
        dup2(pairfd[1], STDIN_FILENO); /* setup server stdin */
        dup2(pairfd[1], STDOUT_FILENO); /* setup server stdout */
        close(pairfd[1]);
        serverfd[cpuindex] = pairfd[0];
        fcntl(pairfd[0], F_SETFD, FD_CLOEXEC); /* don't propagate fd on spawn */

        inherit.flags = 0;
        peparm.pe_cpu = cpuindex;
    }
}

```

```

fprintf(stderr,
        "sserver spawn'ed in cpu %d\n", peparm.pe_cpu);
if ((newpid = fork()) < 0)
{
    fprintf(stderr, "fork() failed [errno %d]\n", errno);
    exit(1);
}
else if (newpid == 0)
{
    /* we are child ... spawn grandchild to avoid zombie processes */
    if (tdm_spawn(childargv[0], 0, NULL, &inherit,
                  childargv, environ, &peparm, &perslt) < 0)
    {
        fprintf(stderr,
                "tdm_spawn() to cpu %d failed [errno %d]\n",
                cpuindex, errno);
    }
    exit(0); /* child goes away */
}
else
{
    /* we are parent */
    wait(&newpid); /* wait for child to avoid zombie child process */
}
}
}

close(STDIN_FILENO);
close(STDOUT_FILENO);
cpuindex = 0;

/* prepare fd passing message */
fdmsghdr.msg_name      = NULL;
fdmsghdr.msg_namelen    = 0;

fdmsghdr.msg_iov        = iov;
fdmsghdr.msg_iovlen     = 1;
iov[0].iov_base         = &cpuindex; /* sent but not used */
iov[0].iov_len          = sizeof(cpuindex);

fdmsghdr.msg_control    = &fdadata;
fdmsghdr.msg_controllen = sizeof(fdadata);
fdadata.fdcmsghdr.cmsg_len = sizeof(fdadata);
fdadata.fdcmsghdr.cmsg_level = SOL_SOCKET;
fdadata.fdcmsghdr.cmsg_type = SCM_RIGHTS;
fdadata.fd              = -1; /* set prior to each
                               * sendmsg() call */

/* loop until error or timeout */
while (1)
{
    /* initialize timeout and bitmask for select() call */
    timeval.tv_sec = IDLE_PERIOD;
    timeval.tv_usec = 0;
    FD_SET(listenfd, &fdset);
    readycount = select(listenfd+1, &fdset, NULL, NULL, &timeval);

    /* if timeout then exit */

```



```

    if (readycount == 0)
    {
        fprintf(stderr, "slaunch1 stopping - timeout\n");
        exit(0);
    }

    /* accept new connection */
    if ((fd = accept(listenfd, (struct sockaddr *)NULL,
        (size_t *)NULL)) < 0)
    {
        fprintf(stderr, "accept() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* find next available server */
    do {
        cpuindex = (++cpuindex == cpucount) ? 0 : cpuindex;
    } while ((cpumask >> (15-cpuindex)) & 1 == 0);

    /* pass socket fd to next available server */
    fdadata.fd = fd;
    if (sendmsg(serverfd[cpuindex], &fdmsghdr, 0) < 0)
    {
        fprintf(stderr, "sendmsg() call failed [errno %d]\n", errno);
        exit(1);
    }
    close(fd);
} /* end while (1) */
}

```

`slaunch1` creates an `AF_UNIX` socket pair to use for interprocess communication with each of its servers. (Note that `slaunch1` fails if the OSS local sockets server has not been started on your node.)

`slaunch1` uses `fork()` to create a duplicate of itself for each available processor, and then starts a copy of `sserver` in that processor using a `tdm_spawn()` call. `slaunch1` closes unneeded file descriptors and then passes the file descriptor of a `requester2` session that needs service to the next available copy of `sserver`, cycling among all of the copies and spreading the workload across all processors.

Each copy of `sserver` inherits an `AF_UNIX` socket to receive the file descriptor of a `requester2` process connection and handles all work associated with that connection. Note that `sserver` is an iterative static server, like the program in [Using AF_INET in an OSS Server Process](#), and handles multiple requestors.

The `sserver` process runs continuously in the background until it is idle for an arbitrary length of time. `sserver` could easily be modified to run until killed by an operator.

Using two terminal sessions, sample dialog for the terminal launching `slaunch1` might be:

```

/users/chris: ./slaunch1
slaunch1 starting in cpu 1
sserver spawn'ed in cpu 0
sserver [cpu 0] - starting
sserver spawn'ed in cpu 1
sserver [cpu 1] - starting
sserver spawn'ed in cpu 2
sserver [cpu 2] - starting
sserver spawn'ed in cpu 3
sserver [cpu 3] - starting
sserver [cpu 1] - new connection on fd 3
sserver [cpu 1] - message received on fd 3 = hello
sserver [cpu 1] - connection on fd 3 closed

```

```

sserver [cpu 2] - new connection on fd 3
sserver [cpu 2] - message received on fd 3 = hello from another requester
sserver [cpu 2] - connection on fd 3 closed

```

while the terminal running the copy of requester2 might show:

```

/users/chris: ./requester2
Requester starting
?hello
Read 7 bytes back from server
?
/users/chris: ./requester2
Requester starting
?hello from another requester
Read 29 bytes back from server
?
/users/chris:

```

Static Echo Server

```

/* sserver.c
 *
 * Simple OSS server example using OSS APIs for communication
 * between requester and server.
 *
 * sserver is a static echo server. A sserver process is launched
 * for each cpu with stdin being used for fd passing from slaunch
 * to sserver. When an fd is passed to sserver, it adds it to an
 * fdset and then echoes the input on that connection.
 */

#define _XOPEN_SOURCE_EXTENDED 1          /* needed for OSS sockets */

#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <sys/time.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netdb.h>

#include <tdmext.h>

/* Guardian APIs */
#include <cextdecs.h>(PROCESSHANDLE_GETMINE_, \
                    PROCESSHANDLE_DECOMPOSE_, PROCESSORSTATUS)>

short cpu, phandle[10];    /* our cpu and process handle */
int fd, fdpassfd, newfd, highfd, fdcount, readycount, bytesread;
struct sockaddr_in addr;   /* socket address (family,addr,port) */
struct timeval timeval;    /* timeout structure for select() call */

```



```

fd_set fdset, rdset;          /* set of fds, set of fds ready for reading */
struct msghdr  fdmsghdr;      /* msghdr for recvmsg() call */
struct iovec   iov[1];        /* io vector for recvmsg() call */
struct {
    struct cmsghdr fdcmsghdr; /* ancillary data header */
    int            fd;        /* ancillary data (one fd) */
} fdadata;
#define IDLE_PERIOD 60        /* 60 seconds for timeout */
#define BUFF_LEN 1024        /* buffer size */
char msg_buff[BUFF_LEN+1];    /* single buffer is enough for all connections */

int main(int argc, char **argv)
{
    /* get process handle and cpu number */
    PROCESSHANDLE_GETMINE_(phandle);
    PROCESSHANDLE_DECOMPOSE_(phandle, &cpu);

    /* print startup message */
    fprintf(stderr, "sserver [cpu %d] - starting\n", cpu);

    fdcount = 1;
    highfd = STDIN_FILENO;
    fdpassfd = STDIN_FILENO;

    FD_ZERO(&fdset);
    FD_SET(fdpassfd, &fdset);    /* initially fdset has only one active fd */

    /* prepare fd passing message */
    fdmsghdr.msg_name = NULL;
    fdmsghdr.msg_namelen = 0;

    fdmsghdr.msg_iov = iov;
    fdmsghdr.msg_iovlen = 1;
    iov[0].iov_base = msg_buff; /* data ignored when passing fd */
    iov[0].iov_len = sizeof(msg_buff);

    fdmsghdr.msg_control = &fdadata;
    fdmsghdr.msg_controllen = sizeof(fdadata);
    fdadata.fdcmsghdr.cmsg_len = sizeof(fdadata);
    fdadata.fdcmsghdr.cmsg_level = SOL_SOCKET;
    fdadata.fdcmsghdr.cmsg_type = SCM_RIGHTS;
    fdadata.fd = -1; /* set by each recvmsg() call */

    /* Find a ready socket ... */
    while (1)
    {
        /* initialize timeout and bitmask for select() call */
        timeval.tv_sec = IDLE_PERIOD;
        timeval.tv_usec = 0;
        memcpy(&rdset, &fdset, sizeof(fdset));
        readycount = select(highfd+1, &rdset, NULL, NULL, &timeval);

        /* if timeout then exit */
        if (readycount == 0)
        {
            fprintf(stderr, "sserver [cpu %d] - "
                        "stopping [timeout]\n",

```

```

        cpu);
    exit(0);
}

/* find first fd that was marked ready for reading */
for (fd = 0; !FD_ISSET(fd, &rdset); fd++)
{
    ; /* for loop does all the work */
}

/* if ready fd is the fd passing socket, accept new fd */
if (fd == fdpassfd)
{
    if ((bytesread = recvmsg(fd, &fdmsghdr, 0)) < 0)
    {
        fprintf(stderr, "sserver [cpu %d] - "
                    "recvmsg(fd %d) failed [errno %d]\n",
                    cpu, fd, errno);
        exit(1);
    }
    else if (bytesread == 0)
    {
        fprintf(stderr, "sserver [cpu %d] - "
                    "stopping [stdin unexpectedly closed]\n",
                    cpu);
        exit(1);
    }

    /* add fd to set of active fds */
    newfd = fdadata.fd;
    FD_SET(newfd, &fdset);
    fdcount++;
    if (newfd > highfd)
        highfd = newfd;
    fprintf(stderr, "sserver [cpu %d] - "
                "new connection on fd %d\n",
                cpu, newfd);
}
else /* else ready fd is an existing connected socket */
{
    /* read data from connected socket which is ready
     * for reading */
    if ((bytesread = recv(fd, msg_buff, BUFF_LEN, 0)) < 0)
    {
        fprintf(stderr, "sserver [cpu %d] - "
                    "recv(fd %d) failed [errno %d]\n",
                    cpu, fd, errno);
        exit(1);
    }
    else if (bytesread == 0) /* peer closed connection */
    {
        /* close socket and remove fd from active fd set */
        fprintf(stderr, "sserver [cpu %d] - "
                    "connection on fd %d closed\n",
                    cpu, fd);
        close(fd);
        FD_CLR(fd, &fdset);
    }
}

```

```

        fdcount--;
    }
    else /* data received */
    {
        msg_buff[bytesread] = '\0'; /* make string NULL-terminated */
        fprintf(stderr, "sserver [cpu %d] - "
            "message received on fd %d = %s",
            cpu, fd, msg_buff);
        /* echo back to sender */
        if (send(fd, msg_buff, bytesread, 0) < 0)
        {
            fprintf(stderr, "sserver [cpu %d] - "
                "send(fd %d) failed [errno %d]\n",
                cpu, fd, errno);
            exit(1);
        }
    }
} /* end handle ready fd */
} /* end while (1) */
}

```

OSS Process-Management Functions

Table 13: OSS Process-Management Functions displays information about each OSS function that you can use to manage processes. The columns of the table contain the following information:

OSS Function

The name of the function and a brief description.

OSS Notes

Notes about the OSS implementation of the function. The note “Can return extended errors” means that the function can return errors that are Hewlett Packard Enterprise extensions to the XPG4 specification. Refer to the function’s reference page for further information.

Beginning with the J06.15 and H06.26 RVUs, a partner or customer OSS SEEP is supported and can participate in access-control decisions for OSS objects. (For OSS SEEP details, see **Accessing OSS SEEP-Protected Files**.) The note “OSS SEEP consultation” indicates that the OSS name server consults the OSS SEEP when this function is called for operations on files in OSS SEEP-protected filesets and the OSS SEEP is running.

Guardian Notes

Notes about using the function on a Guardian process.

The information in this table is only a summary of the information in the reference page for a function. For details, refer to the reference pages either online or in the *Open System Services System Calls Reference Manual* and the *Open System Services Library Calls Reference Manual*.

NOTE: To determine which RVUs support an OSS function, see **Documented OSS Functions**.



Table 13: OSS Process-Management Functions

OSS Function	OSS Notes	Guardian Notes
<code>abort()</code> Generates a software signal to terminate the calling process.		Calls the Guardian ABEND procedure when called from a Guardian TNS process.
<code>alarm()</code> Sets or changes a timer that expires at a specified time in the future.		Results in undefined behavior when called from a Guardian TNS process.
<code>execl()</code> Replaces the current process image with a new process image, using a pathname, a set of environment strings, and <code>**environ</code> .	Can return extended errors. Propagates open file descriptors except those opened using a Guardian function or procedure. For 64-bit OSS process-creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u> . OSS SEEP consultation.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>execle()</code> Replaces the current process image with a new process image, using a pathname, a set of environment strings, and an undeclared <code>envp</code> array.	Can return extended errors. Propagates open file descriptors except those opened using a Guardian function or procedure. For 64-bit OSS process-creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u> . OSS SEEP consultation.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>execvp()</code> Replaces the current process image with a new process image, using a filename, a set of environment strings, and <code>**environ</code> .	Can return extended errors. Propagates open file descriptors except those opened using a Guardian function or procedure. For 64-bit OSS process-creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u> . OSS SEEP consultation.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.

Table Continued

OSS Function	OSS Notes	Guardian Notes
execv() Replaces the current process image with a new process image, using a pathname, an argv array, and **environ.	Can return extended errors. Propagates open file descriptors except those opened using a Guardian function or procedure. For 64-bit OSS process-creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u> . OSS SEEP consultation.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
execve() Replaces the current process image with a new process image, using a pathname, an argv array, and an undeclared envp array.	Can return extended errors. Propagates open file descriptors except those opened using a Guardian function or procedure. For 64-bit OSS process-creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u> . OSS SEEP consultation.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
execvp() Replaces the current process image with a new process image, using a filename, an argv array, and **environ.	Can return extended errors. Propagates open file descriptors except those opened using a Guardian function or procedure. For 64-bit OSS process-creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u> . OSS SEEP consultation.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
_exit() exit() Terminate the calling process.	Child processes of a terminated OSS process are assigned a parent process ID of 1.	A Guardian process has no OSS process ID and therefore has no wait (<code>wait()</code> or <code>waitpid()</code>) considerations.
fexecve() Replaces the current process image with a new process image, using a file descriptor, an argv array, and undeclared envp array.	Can return extended errors. Propagates open file descriptors except those opened using a Guardian function or procedure.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>fork()</code> Creates an OSS process.	Can return extended errors. The child process shares directory streams with the parent. They share the same block of directory entries, but each stream can be used by only one of the processes. Propagates open file descriptors except those opened using a Guardian function or procedure. For 64-bit OSS process-creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u> .	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>getegid()</code> Gets the effective group ID of the calling process.		A process's effective group ID can be different from the group number in its process access ID (PAID).
<code>getenv()</code> Returns the value of an environment variable.		
<code>geteuid()</code> Gets the effective user ID of the calling process.		
<code>getgid()</code> Gets the real group ID of the calling process.		A process's real group ID can be different from the group number in its creator access ID (CAID).
<code>getgroups()</code> Gets the group list of the calling process.	The effective group ID might not occur in the returned group ID list if the effective group ID has been changed by executing a set-group-ID program or by calling the <code>setgid()</code> function.	
<code>getpgrp()</code> Gets the OSS process group ID of the calling process.		Sets Guardian trap number 5 when called from a Guardian process.
<code>getpid()</code> Gets the OSS process ID of the calling process.		Sets Guardian trap number 5 when called from a Guardian process.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>getppid()</code> Gets the OSS parent process ID of the calling process.	If the parent process terminates or the calling process was created by a Guardian process, it returns a parent process ID of 1.	Sets Guardian trap number 5 when called from a Guardian process.
<code>getsid()</code> Gets the group ID of the process that is the session leader of the specified process	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value and sets Guardian trap number 5 when called from a Guardian process.
<code>getuid()</code> Gets the real user ID of the calling process.		A process's real user ID is not always numerically equal to its creator access ID.
<code>kill()</code> Sends a signal to a process or group of processes.	Can return extended errors. Applications should not count on all system processes receiving signals. There are several constraints in sending signals if there is Safeguard protection on OSS processes.	Cannot send signals to Guardian processes. Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>longjmp()</code> Performs a nonlocal goto.		
<code>nftw()</code> <code>nftw64()</code> Traverses a file tree.	Can return extended errors. <code>nftw64()</code> can access both small files and large files.	Guardian files that are visible to the <code>opendir()</code> function can be traversed. Security controls on Guardian objects do not affect the outcome of a call to <code>nftw()</code> . Two Guardian file system file numbers (not necessarily the next two available) are allocated for the root directory and the current working directory. The current working directory is assigned from the VOLUME attribute of the Guardian environment <code>=_DEFAULTS DEFINE</code> . The use of static memory by the process increases slightly.
<code>pause()</code> Suspends the calling process until a signal is received.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian TNS process.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>putenv()</code>		
Sets an environment variable.		
<code>raise()</code>	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian TNS process.
Sends a signal to the calling process.		
<code>setgid()</code>		
Sets the group ID of the calling process.		
<code>setitimer()</code>		Not available for Guardian TNS process.
Sets or changes an optionally-renewable timer that expires at a specified time in the future. Only real-time timer is supported.		
<code>setjmp()</code>		
Saves the current execution context.		
<code>setpgid()</code>	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value and sets Guardian trap number 5 when called from a Guardian process.
Sets the process group ID of the calling process.		
<code>setpgrp()</code>	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value and sets Guardian trap number 5 when called from a Guardian process.
Creates a new session when the calling process is not a process group leader, and sets the process group ID of the calling process.		
<code>setregid()</code>		
Sets the real and effective group IDs of the current process		
<code>setreuid()</code>		
Sets the real and effective user IDs of the current process		
<code>setsid()</code>	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value and sets Guardian trap number 5 when called from a Guardian process.
Creates a new session and sets the process group ID.		
<code>setuid()</code>	The value of the user ID must be in the range 0 through 65535.	
Sets the user ID of the calling process.		

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>sigaction()</code> Specifies the action to be taken upon delivery of a signal.	Can return extended errors. Hewlett Packard Enterprise has defined several new signals.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian TNS process.
<code>sigaddset()</code> Adds a signal to a signal set.	Can return extended errors.	Not meaningful when called from a Guardian TNS process.
<code>sigdelset()</code> Deletes a signal from a signal set.	Can return extended errors.	Not meaningful when called from a Guardian TNS process.
<code>sigemptyset()</code> Initializes a signal set to exclude all signals.	Can return extended errors.	Not meaningful when called from a Guardian TNS process.
<code>sigfillset()</code> Initializes a signal set to include all signals.	Can return extended errors.	Not meaningful when called from a Guardian TNS process.
<code>sigismember()</code> Tests whether a signal is a member of a signal set.	Can return extended errors.	Not meaningful when called from a Guardian TNS process.
<code>siglongjmp()</code> Performs a nonlocal goto.		Results in undefined behavior when called from a Guardian TNS process.
<code>signal()</code> Specifies the action to be taken upon delivery of a signal.	Can return extended errors. Hewlett Packard Enterprise has defined several new signals.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian TNS process.
<code>sigpending()</code> Returns the set of signals that are blocked from delivery and pending to the calling process.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian TNS process.
<code>sigprocmask()</code> Changes or examines a process signal mask.	Can return extended errors. Hewlett Packard Enterprise has defined several new signals.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian TNS process.
<code>sigsetjmp()</code> Saves the current execution context and can save the current process signal mask.		Results in undefined behavior when called from a Guardian TNS process.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>sigsuspend()</code> Changes a process signal mask and suspends the calling process.	Can return extended errors. Hewlett Packard Enterprise has defined several new signals.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian TNS process.
<code>sleep()</code> Suspends the calling process for a specified interval of time.	There are several implementation-defined features with respect to the <code>SIGALRM</code> signal.	Results in undefined behavior when called from a Guardian TNS process.
<code>tdm_execve()</code> <code>tdm_execvep()</code> Replaces the current OSS process image with a new process image (with Hewlett Packard Enterprise extensions).	Is an extension to the XPG4 specification. Is similar to the <code>exec</code> set of functions; also allows you to retain the attribute values inherited from the parent process or to specify other values. OSS SEEP consultation.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>tdm_fork()</code> Creates a new OSS process (with Hewlett Packard Enterprise extensions).	Is an extension to the XPG4 specification. Is similar to the <code>fork()</code> function; also allows you to retain the attribute values inherited from the parent process or to specify other values. Does not propagate file locks. For 64-bit OSS process-creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u>.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>tdm_spawn()</code> <code>tdm_spawnp()</code> Creates a new OSS process with a specified executable image (with Hewlett Packard Enterprise extensions).	Can return extended errors. Is an extension to the XPG4 specification. Produces a new OSS process by combining the function of <code>tdm_execve()</code> or <code>tdm_execvep()</code> with <code>tdm_fork()</code> , but with the following additions: - Parameters control the file descriptors the child process inherits from the parent process. - The inherit structure allows the process group ID and signal mask of the child process to be specified in addition to a list of signals that the child process takes default action on. - Does not propagate file locks. For 64-bit OSS process-creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u> . OSS SEEP consultation.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>times()</code> Gets calling process and child process times.	Can return extended errors. There is no overflow limit for the fields in the <code>tms</code> structure.	Does not maintain child times. Accumulates all processor time as user time. Might not maintain elapsed time correctly if the processor's internal clock is reset.
<code>uname()</code> Gets information identifying the system on which the calling process runs.	Can return extended errors.	
<code>wait()</code> <code>waitpid()</code> Wait for a child process to terminate.	If a parent process terminates without waiting for all of its child processes to terminate, the remaining child processes are assigned a parent process ID of 1.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.



Guardian Process-Management Procedures

The following table displays information about each Guardian procedure that you can use to manage processes. Procedures that have been superseded by other procedures are not shown. The columns of the table contain the following:

Guardian Procedure

The name of the procedure and a brief description.

OSS Notes

Additional considerations or restrictions when the Guardian procedure is used on OSS processes.

For details about a procedure, refer to the *Guardian Procedure Calls Reference Manual*.

Table 14: Guardian Process-Management Procedures

Guardian Procedure	OSS Notes
CHILD_LOST_ Examines a system message to determine whether a process has stopped.	An OSS parent process must read its \$RECEIVE file for system messages. If an OSS process is reading Guardian system messages to monitor an OSS process, it might appear that the monitored process is alive when it has stopped. See <u>Monitoring OSS Processes Reading Guardian System Messages</u> .
FILENAME_TO_PROCESSHANDLE_ Converts a Guardian (process) filename to a process handle.	
OSS_PID_NULL_ Returns a null OSS process ID.	
PROCESS_ACTIVATE_ Returns a suspended process to a ready state.	Has the same effect as calling the OSS <code>kill()</code> function with <code>SIGCONT</code> as the value of the <i>signal</i> parameter.
PROCESS_DEBUG_ Invokes the debugging facility on a process.	There are several requirements an OSS process must meet to use the debugging facility.
PROCESS_GETINFO_ Gets a limited set of information about a process.	
PROCESS_GETINFOLIST_ Gets detailed information about a process or set of processes.	Beginning with the H06.24 and J06.13 RVUs, additional 64-bit process attributes are available. See <u>New PROCESS_GETINFOLIST_Attributes</u> for details.
PROCESS_GETPAIRINFO_ Obtains information about a named process or process pair.	Prior to the J06.14 and H06.25 RVUs, OSS processes cannot belong to process pairs. For OSS process pairs details, see <u>OSS Process Pairs</u> .

Table Continued



Guardian Procedure	OSS Notes
PROCESS_LAUNCH_ Creates a Guardian process.	Cannot create an OSS process.
PROCESS_SETINFO_ Modifies a single nonstring attribute of a process.	If used to change the priority of an OSS process, the same security rules apply as for the OSS <code>kill()</code> function.
PROCESS_SETSTRINGINFO_ Modifies a single string-form attribute of a process.	
PROCESS_SPAWN[64]_ Creates an OSS process.	Cannot create a Guardian process. Can be used to create a backup process for an OSS process pair. For OSS process pairs details, see <u>OSS Process Pairs</u>. For 64-bit OSS process creation details, beginning with the H06.24 and J06.13 RVUs, see <u>64-Bit OSS Processes</u>. OSS SEEP consultation, beginning with the J06.15 and H06.26 RVUs. For OSS SEEP details, see <u>Accessing OSS SEEP-Protected Files</u>.
PROCESS_STOP_ Terminates a process.	An OSS parent process receives a <code>SIGCHLD</code> signal and the OSS process termination status. A Guardian parent process receives a process-termination system message. When used to stop a process other than the caller, the process handle must be specified. The security rules are the same as when using the OSS <code>kill()</code> function to terminate a process.
PROCESS_SUSPEND_ Places a process into a suspended state.	Has the similar effect to calling the OSS <code>kill()</code> function with <code>SIGSTOP</code> as the value of the <i>signal</i> parameter.
PROCESSHANDLE_COMPARE_ Compares two process handles and reports whether they are identical, represent different processes of the same process pair, or are different.	Prior to the J06.14 and H06.25 RVUs, OSS processes cannot belong to process pairs. For details of OSS process pairs, see <u>OSS Process Pairs</u> .
PROCESSHANDLE_DECOMPOSE_ Returns one or more parts of a process handle.	
PROCESSHANDLE_GETMINE_ Gets the process handle of the calling process.	
PROCESSHANDLE_NULLIT_ Initializes a process handle to a null value.	

Table Continued



PROCESSHANDLE_TO_CRTPID_ Converts a process handle to the corresponding Guardian process ID (CRTPID).	
PROCESSHANDLE_TO_FILENAME_ Converts a process handle to a Guardian (process) filename.	
PROCESSHANDLE_TO_STRING_ Converts a process handle to a process string, which can identify a process or set of processes.	
PROCESSNAME_CREATE_ Returns a unique process name that is suitable for passing to the PROCESS_CREATE_ or PROCESS_SPAWN[64]_ procedures.	
PROCESSOR_GETINFOLIST_ Gets configuration information and statistics about a processor.	Beginning with the H06.24 and J06.13 RVUs, an additional attribute is provided to determine whether the specified (local or remote) system is capable of running 64-bit (LP64) processes.. See <u>New PROCESSOR_GETINFOLIST Attribute</u> for details.
PROCESSTRING_SCAN_ Scans an input string for a process string and returns the corresponding process handle or a single component of the process string converted to internal form.	
SETLOOPTIMER Sets the process loop-timer value of the calling process.	When the process loop-timer expires for an OSS process, a SIGTIMEOUT signal is generated.
SIGACTION_INIT_ Establishes the initial state of signal handling for the calling process.	
SIGACTION_RESTORE_ Restores the signal-handling state stored by a call to SIGACTION_SUPPLANT_.	
SIGACTION_SUPPLANT_ Saves the current signal-handling state and allows a subsystem to take over signal handling temporarily.	
SIGJMP_MASKSET_ Saves the process signal mask in a jump buffer.	



OSS Process Pairs

Beginning with the J06.14 and H06.25 RVUs, the NonStop Operating System supports process pairs in which the processes operate in the Open System Services (OSS) environment. The `PROCESS_SPAWN[64]_` procedure can create an OSS backup process. This feature provides the NonStop characteristics of process pairs in the OSS environment.

NOTE: Though it is possible to create an OSS process pair on older RVUs, the feature is unsupported prior to the T9050 SPRs that participate in the J06.14 and H06.25 RVUs.

OSS Process Pairs Overview

An OSS process pair consists of two processes, in different processors (CPUs), identified by the same Guardian process name and working together to provide a single facility from the point of view of their clients. One process is called **primary** and performs the work of the application. The other process is called **backup** and shadows the primary. Upon failure of the primary process, or its processor, the backup process takes over. The primary periodically sends state (checkpoints) to the backup, so that the backup has the necessary information to continue the work. The operating system automatically reroutes client requests to the backup if the primary fails.

An OSS process pair is created when a single named process creates a backup process. The creating process becomes the primary. The backup and primary processes must be in different processors. Both processes have the same Guardian process name. Only a named OSS process can create an OSS backup, and it can do so only by calling the `PROCESS_SPAWN[64]_` procedure. An OSS process cannot create a Guardian backup process, and vice versa.

For information about the `PROCESS_SPAWN[64]_` procedure, see the *Guardian Procedure Calls Reference Manual*.

OSS Process Pair Considerations

Only the active-backup model is available in an OSS process pair. Passive backup is not supported—in Guardian or OSS—for C/C++ programs using the normal heap. Therefore, an OSS process pair does not use the `CHECKMONITOR` procedure or procedures in the `CHECKPOINT...` family. Design and implementation of an active-backup protocol and determination of when to checkpoint a particular state is application-specific.

Being an OSS process allows the members of the pair to use OSS-specific features, such as pthreads and OSS semaphores. (For pthreads, the application developer must devise appropriate active-backup strategy for a multithreaded process.) Note that on NonStop, pthreads is a user-mode facility; individual threads are not independently dispatched (that is, not “kernel” threads), so only one can be active on a processor at a time. There are no facilities to checkpoint a hidden pthread state.

Designing, implementing, and testing a correct OSS process pair is not a simple task. Often it is preferable to use middleware subsystems that provide the necessary fault-tolerance features. However, sometimes a tailored process pair is the best or most efficient mechanism. For more information about process-pair programming, see the *Fault-Tolerant Programming in C* chapter in the *Guardian Programmer's Guide*.

One reason to use an OSS process pair is to ease interactions with other OSS processes in a multi-process OSS application. For example, an OSS process opens OSS files for `stdin`, `stdout`, and `stderr` streams. Some functions, such as `freopen()`, do not interoperate between OSS and Guardian files; even `freopen_oss()` does not permit a Guardian process to reopen a Guardian `stdout` stream as an OSS stream.

OSS Process Pair Creation

Beginning with the J06.14 and H06.25 RVUs, the `PROCESS_SPAWN[64]_` procedure accepts the value `ZSYS_VAL_PCREATOPT_CALLERSNAME (3)` in the `z_nameoptions` (also known as `pe_name_options`) field of the `process-extension` parameter to create an OSS backup process. The `z_processname` (`pe_process_name`) field is ignored and should be `NULL` (zero, the default). The caller must be a named OSS process. The first parameter, `oss-program-file` (the `pathname`), is typically absent or null, so that the backup process runs the same program as the primary.



NOTE: The `PROCESS_SPAWN[64]` procedure takes several parameters that are pointers to structures, which are defined in both the `tdmext.h` header file and the ZSYSDDL family of header files (including ZSYSTAL for TAL/pTAL and ZSYSC for C/C++). The `process-extension` structure has members with names `pe_...` defined in `tdmext.h`. The member called `pe_name_options` is also called `Z^NAME^OPTIONS` in ZSYSTAL and `z_name_options` in ZSYSC. `ZSYS_VAL_PCREATOPT_CALLERSNAME` is defined in ZSYSC but not in `tdmext.h`. For details about these structures and other `PROCESS_SPAWN[64]` parameters, see the *Guardian Procedure Calls Reference Manual*.

Because the process is created with `PROCESS_SPAWN[64]`, it has Guardian ancestry, and not OSS ancestry.

OSS Process Attributes

The characteristics of processes in OSS process pairs are:

- Both the primary and backup processes have the same process name.
- The `PROCESS_GETPAIRINFO` procedure works the same as it does for Guardian.
- The program can reside in either the Guardian or the OSS file system.
- Each process has its own PID, just as it has its own CPU, PIN identity.
- The primary and backup processes communicate the same way as usual, using (Guardian) opens of the backup process in the primary and `$RECEIVE` in the backup.
- If the OSS process pair implements a server, the primary also opens `$RECEIVE` and reads client requests from it.
- To enable automatic message switching, `$RECEIVE` must be opened with receive depth greater than 0.

OSS process-creation functions invoked by a member of an OSS process pair have the following effects:

- `fork()` creates a new process that inherits the usual attributes from its parent. The process name is not inherited; therefore, the new process is not relevant to the original process pair.
- Functions in the `exec...` family associate a new program image with the current PID. Operationally, a new process is created (with a different CPU, PIN identity), and the process performing the `exec` terminates.

! **IMPORTANT:** A member of an OSS process pair should not perform `exec`; if it does, that original member no longer exists.

- `tdm_spawn()` creates a new process that is not relevant to the pair.
- `PROCESS_SPAWN[64]` is similar to `tdm_spawn()`, unless it has the `name` option that creates a backup process.

Process Status Display

Beginning with the J06.14 and H06.25 RVUs, the TACL STATUS command provides the following process-status identifiers:

- “Y” – OSS backup process
- “X” – OSS process
- “B” – Guardian backup process

Previous versions of TACL display “X” for any OSS process, including a backup.



Known Restrictions

The `crtlns.h` facilities are not supported for OSS. The `_ns_start_backup()` function creates a Guardian backup process. All the other `_ns_...` functions deal with (Guardian) file state; these functions work in OSS processes provided the target files are Guardian.

Because the primary and backup processes' PIDs are different:

- The backup process cannot “take over” OSS file locks.
- A signal sent with the `kill` command or `kill()` function affects only the process designated.

NOTE: This list is not exhaustive.

Other Considerations

To a first approximation, an OSS process pair is a Guardian process pair with PIDs. There are no NonStop enhancements to OSS files, sockets, and other related OSS objects.

For the activities special to process pairs, the application uses the underlying Guardian facilities, which were designed for NonStop programming. The relevant features (such as receive/sync depth and sync IDs) of the Guardian (Enscribe) file system are available.

OSS processes do not receive startup, assign, and param messages from their parent. `PROCESS_SPAWN[64]` accepts parameters for the vector of `arg` and `env` strings; the parent (primary) can easily pass on its own arguments and environment to the child (backup).

To the extent that the PID is used only for identification, as in the `shm...` shared memory facilities, or the `sem...` semaphore mechanism, it can be considered local to the process.

Hewlett Packard Enterprise recommends that a process pair protect itself from unwanted external signals by blocking them, so the use of PID in `kill()` becomes irrelevant.

File Number Conflicts

A difficulty can occur in an OSS process pair, because an OSS process has two special “files” open—the `root` directory and the current working directory. Each of these objects consumes a file number in the process. Typically, when `PROCESS_SPAWN[64]` creates a new OSS process, `root` and `cwd` are assigned file numbers 1 and 2, respectively. The standard in/out/error files get numbers 3, 4, and 5, respectively. Additional opens use additional numbers. For example, this is the situation in a new shell process created by the `OSH` utility.

If a shell process executes the `cd` command, a new `cwd` object is created with the next available file number, and the old one is closed. For example, `cwd` becomes file #7 (#6 was already in use for another file), and file #2 becomes unused. (An example of this situation can be seen in a debugging session using the `elnspect info opens` command.) Suppose the shell now forks and execs a new process that becomes the primary of an application, and that process opens a disk file, then creates its backup and attempts to backup open that file. (Backup opens use the same file number as in the primary.) Since file #2 was available, the disk file received that number. But file #2 is not available in the backup, because it is occupied by `cwd`, so the backup open fails. One way to avoid this issue is to launch the primary process with `PROCESS_SPAWN[64]`, rather than running it from the shell.

A workaround is to execute an even number of `cd` commands in the shell. In the example above, if the shell user follows the `cd` command that moved `cwd` from #2 to #7 with a second `cd` command (even to the same directory), a third `cwd` object is opened using file #2, which is now the first available, and file #7 is closed.



Managing Memory

The term “memory management” has broad scope. The phrase applies to the actions of the operating system, which associates ranges of virtual addresses with physical addresses and with aggregations of data. Memory management applies somewhat differently to the actions in the user environment, including the run-time library.

Examples of operating-system actions include:

- Establishing the memory environment of a process
- Including segments of code and data for the program and any libraries it uses
- Creating explicit data segments, some of which can exist beyond the context of a single process

Examples of user-level actions include:

- Explicit user-program actions such as allocating data areas from a heap
- Implicit actions such as:
 - The placement of global and static variables
 - The automatic creation and placement of local variables

These implicit placements occur as the program and libraries are compiled and linked.

A key memory-management concept in the NonStop system is the **segment**, defined as a collection of information identified and managed as an entity with consistent access and other properties. Abstractly, a segment can exist independently of a process, but when associated with a process it occupies a contiguous range of virtual address space.

The system automatically establishes segments for the global data of the program, instance data of any native libraries, and text (code) segments for the program and libraries. In C/C++ and some other languages, the system establishes a process heap.

NOTE:

For information about memory layout on NonStop systems, see the *Managing Memory* chapter in the *Guardian Programmer's Guide*.

Flat, Selectable, and Extended Segments

The term “extended” is significant only in the context of TNS processes. The only nonextended data segment in user address space is the TNS user data segment, and there are no nonextended data segments in a native process. For information about TNS processes, see **TNS Processes**.

In the TNS architecture, extended data segments are selectable. Most segments on modern systems are flat; however, the NonStop system continues to support selectable segments.

Flat Segments

Flat segments have distinct addresses. All flat segments allocated in a process are available to that process simultaneously.

With the exception of the automatic extended data segment of TNS processes, described below, all the segments allocated automatically by the system are flat.



NOTE:

The adjective “flat” is applied to segments on NonStop systems only to mean “not selectable.”

Selectable Segments

Selectable segments all share the same address range; all selectable segments have the same starting address (%2000000 or 0x80000). Thus, only one selectable segment is available to a process at any given time. You select a segment—make a segment current—with the `SEGMENT_USE_` procedure. For information about using extended segments in application programs, see the *Managing Memory* chapter in the *Guardian Programmer's Guide*.

Native Processes

In a native process, all system-supplied segments are flat. The system provides a data segment for the program's global data and the 32-bit heap, and a segment for the instance data of each DLL or SRL in the process that has global data. Optionally (specified at link time), TNS/E and TNS/X programs and DLLs can have separate segments for constant and variable data. In a 64-bit TNS/E (or TNS/X) OSS process (supported beginning with the H06.24 and J06.13 RVUs and L-series), the heap occupies a separate segment in 64-bit address space.

There are also system-provided stack segments, a main stack for running procedures in user state, and a priv stack for running procedures in privileged state (for example, portions of the operating system). Other segments are supplied for system use; some in the process address space and others that are global—visible to all processes.

TNS Processes

In the legacy TNS environment, the process has a User Data Segment of up to 128 KB at address 0, and often an automatic extended data segment of up to 127.5 MB. TNS OSS processes can exist on TNS/R systems but not TNS/E or TNS/X systems. TNS OSS processes use the wide-data model (ILP32): `int`, `long`, and pointer data types all occupy 32 bits. In TNS C, the wide-data model is a variant of the large-memory model. TNS processes using other data models, or any Guardian TNS processes, are of interest to OSS programmers only if they want to share data segments with such processes. For information about the wide-data model and the large-memory model, see the *C/C++ Programmer's Guide for NonStop Systems* in the NTL.

A process using the large-memory model has an automatic (compiler-generated) extended data segment, with segment ID 1028, which holds the heap, most global and static data, and local aggregate data. This segment is selected automatically and must remain selected to access this data. Therefore, Hewlett Packard Enterprise strongly discourages explicitly selecting other selectable segments in such a process. For additional information, see *Using Selectable Segments in TNS Processes* in the *Guardian Programmer's Guide*.

On TNS/R, TNS/E, and TNS/X systems, TNS processes also have native stack segments. These stacks are used when the process transitions into native mode to invoke native procedures in the operating system.

User-Created Segments

In addition to the segments provided automatically by the system, the user can cause explicit segments to be created and shared. Separate mechanisms are supplied by Guardian and OSS APIs.

OSS Shared Memory Segments

The following OSS functions manage flat shared-memory segments:

- `shmget ( )` – Creates a new segment or shares an existing one.
- `shmat ( )` – Attaches the segment to the process.



- `shmdt()` – Detaches the segment.
- `shmctl()` – Performs other actions on the segment, including removing it. The segment persists until explicitly removed.

Segment sharing is specified using an identifier called **shmid**, returned by `shmget()`, and passed to `shmat()`.

Permission to share the segment is based on OSS protection mechanisms: the creator specifies whether the segment can be shared by the same user, the user group, or anyone.

For information about the set of `shm*` functions, see the *Open System Services System Calls Reference Manual* in the NTL.

Guardian Memory Segments

The `SEGMENT_ALLOCATE_` Guardian procedure can create and share either flat or selectable segments. By default, it establishes a selectable segment. On servers running H06.20 and later H-series RVUs, or J06.09 and later J-series RVUs, `SEGMENT_ALLOCATE64_` provides a superset of `SEGMENT_ALLOCATE_` facilities. By default, `SEGMENT_ALLOCATE64_` establishes a flat segment in 64-bit address space; parameters can specify a 32-bit flat segment or a selectable segment, (which is always in 32-bit address space).

Both procedures can create segments backed either by the Kernel Managed Swapspace Facility, KMSF, or by individual temporary or permanent files. Both procedures support sharing of Guardian segments, specified by file name or by (segment ID and) PIN.

For share-by-PIN, the caller specifies the PIN of a process (in the same processor) to share the segment in that process using the same segment ID. Permission is based on the process-access-ID attributes of the two processes.

For share-by-filename, the caller specifies the file name of the backing file for the segment; the contents of the file are mapped directly into memory, and the file-backed memory segment is shared among all processes allocating it. Permission is based on access attributes for the file and the access ID of this process. If the segment is read-only, it can be shared by processes in multiple processors, and the file can also be opened for reading. Otherwise, all processes sharing the segment must be in the same processor, and the file cannot simultaneously be open for reading or writing. Guardian segments shared by file name provide some of the same abilities of the `mmap()` function, which is found in many UNIX and Linux systems. For more information, see *Memory Mapping* in the *Open System Services System Porting Guide*.

Other procedures of interest for managing Guardian memory segments include:

- `ADDRESS_DELIMIT[64]_`
- `MOVEX`
- `SEGMENT_DEALLOCATE_`
- `SEGMENT_GETINFO[64]_`
- `SEGMENT_GETINFOSTRUCT_`
- `SEGMENT_RESIZE_`
- `SEGMENT_USE_`

For information about these procedures, see the *Guardian Procedure Calls Reference Manual* in the NTL.

Memory-Management Interoperability

Guardian and OSS processes can share memory segments with each other. Both Guardian and OSS processes can call `SEGMENT_ALLOCATE_` and related Guardian procedure calls: they can share segments as described for `SEGMENT_ALLOCATE_`. On servers running L-series or J06.12 or later J-series RVUs or H06.23 or later H-series RVUs,



both Guardian processes and OSS processes can call the OSS shared memory functions; they can share segments as described for `shmget()`. For more information about these functions, see [Using Shared Memory](#).

The shared memory segments created and managed by OSS `shm*` functions are distinct from segments created by the Guardian `SEGMENT_ALLOCATE_` and related procedure calls. Both kinds of segments can be created and shared, but one kind cannot be shared with the other.

In the OSS environment, communication between processes using shared memory, message queues, semaphores, and pipe functions is considered to be *interprocess communication*—rather than memory management. OSS interprocess-communication functions are discussed in [Interprocess Communication](#).

Pool Functions

Frequently, portions of a memory segment are utilized for independent purposes. A prime example is the heap: the C standard functions `malloc()`, `calloc()`, `realloc()`, and `free()`, and the C++ operators `new` and `delete`, manage portions of the process heap. These facilities are the same in Guardian and OSS processes. In addition to the standard functions, the Common Runtime Environment library also offers some functions for control and diagnosis of the heap:

- `heap_check()` – Checks the consistency of the run-time heap.
- `heap_check_always()` – Controls whether the run-time heap consistency is checked automatically.
- `heap_min_block_size()` – Specifies the minimum block size to attempt to use when allocating from the heap.

For various reasons, a program might need to perform similar sub-allocations from some other segment. Beginning with the H06.20 and J06.09 RVUs, the system provides a set of efficient Guardian procedures to manage a heap or pool defined by the program. These procedures accommodate 64-bit addresses, and can manage multiple disjoint areas (typically in different segments) as a single pool.

- `POOL64_AUGMENT_` – Adds a disjoint memory area to the pool.
- `POOL64_CHECK_` – Performs a diagnostic scan of the pool.
- `POOL64_CHECK_SHRINK_` – Checks whether the pool can be made smaller while retaining its contents.
- `POOL64_DEFINE_` – Specifies the virtual address space and initializes the pool.
- `POOL64_DIMINISH_` – Removes an augmentation area from the pool.
- `POOL64_GET_` – Allocates a block of space from the pool, analogous to `malloc()`.
- `POOL64_GETINFO_` – Reads summary statistics from the pool.
- `POOL64_PUT_` – Returns a previously allocated block to the pool, analogous to `free()`.
- `POOL64_RESIZE_` – Shrinks or enlarges a single-segment pool.

For more information about these 64-bit pool procedures, see the *Guardian Procedure Calls Reference Manual* in the NTL.

NOTE: Another set of pool procedures is available on all supported NonStop systems. They support only 32-bit addressing, and are less efficient than the `POOL64_*` procedures. The 32-bit pool procedures are named `POOL_*`, and are also documented in the *Guardian Procedure Calls Reference Manual*.



Managing Time

This section describes the time-management features that are common to both UNIX and the OSS API and those that are unique to the OSS API.

You can use OSS functions and Guardian procedures to query the system clock, obtain process times, and set process timers.

This section discusses the following topics:

Topic	Content
<u>Common and Unique Characteristics</u>	Summary of the differences between OSS and UNIX time management.
<u>Process Attributes</u>	The objects on which you can use time-management functions and procedures.
<u>Querying the System Clock</u>	How to obtain and format the system date and time.
<u>Querying Process Times</u>	How to obtain process times.
<u>Limiting Processor Time Use</u>	How to use the SETLOOPTIMER procedure in an OSS program.
<u>OSS Time-Management Functions</u>	List of each OSS time-management function with notes about the OSS implementation and its use on Guardian objects.
<u>Guardian Time-Management Procedures</u>	List of each Guardian time-management procedure with notes about its use on OSS objects.

Common and Unique Characteristics

OSS time management is the same as UNIX time management, except that you cannot set or alter the system clock with an OSS function.

Features Common to UNIX and OSS Time Management

The OSS API enables you to:

- Obtain the system time with the `time()` function
- Obtain process times with the `clock()` and `times()` functions
- Set alarms with the `alarm()` and `ualarm()` functions
- Using the `setitimer()` function you can set the timer for automatic repetition. Using the `getitimer()` function you can obtain the current timer values.

The OSS API also enables you to format the time and perform time calculations with the following functions:



Function	Summary
<code>asctime()</code>	Converts the date and time to a string.
<code>ctime()</code>	Converts time units.
<code>difftime()</code>	Computes the difference between two calendar time values.
<code>gmtime()</code>	Converts a time value to broken-down Coordinated Universal Time (UTC) time.
<code>localtime()</code>	Converts a time value to broken-down local time.
<code>mktime()</code>	Converts a broken-down time into time since the Epoch (00:00:00 UTC January 1, 1970 A.D.).
<code>strftime()</code>	Converts a date and time to a string.
<code>strptime()</code>	Converts a character string to a time value.
<code>tzset()</code>	Sets the time-zone conversion information.
<code>utime()</code>	Sets file access and modification times.
<code>wcsftime()</code>	Converts a date and time to a wide character string.

Features Unique to OSS Time Management

The OSS API does not provide functions that set or alter the system time or obtain information about processor times. It also does not provide an internal timer.

Time-Management Interoperability

You can use Guardian time-management procedures from the OSS environment.

Guardian system clock procedures and OSS system clock functions can be called from both Guardian and OSS processes. Because the OSS and Guardian time formats are incompatible, only Guardian system clock procedures work on Guardian time formats and only OSS system clock functions work on OSS time formats.

You can use the `MYPROCESSTIME`, `PROCESS_GETINFO_`, and `PROCESS_GETINFOLIST_` procedures and the Measure product to obtain time information about an OSS process. The `PROCESS_GETINFO_` and `PROCESS_GETINFOLIST_` procedures are especially useful in that they can obtain information not just about the calling process, but about another process that you specify.

When a Guardian process calls the OSS `times()` function, only restricted information is returned: times are available for Guardian processes but not for processes that act on behalf of a Guardian process.

You can use the `utime()` function to change Guardian file access and modification times of unstructured files.

The `alarm()` and `ualarm()` functions use OSS signals to notify OSS processes and do not work with Guardian processes, because OSS signals cannot be sent to Guardian processes.

You can use the Guardian `SETLOOPTIMER` procedure to limit processor time use in OSS processes.



Querying the System Clock

You query the system date and time in the OSS environment with the `time()` function as you would in the UNIX operating system. The timestamp you get from the `time()` function is accurate to the nearest second. If you need greater accuracy in OSS programs, you can call Guardian system date and time procedures and obtain timestamps accurate to the centisecond or microsecond.

Guardian timestamps differ from the OSS timestamp as shown in the following table.

Table 15: Guardian and OSS Timestamps

Feature	OSS Timestamp	Guardian 48-Bit Timestamp	Guardian 64-Bit Julian Timestamp
Precision	Second	Centisecond	Microsecond
Beginning of Time	00:00:00 UTC January 1, 1970 A.D. (the Epoch)	00:00 December 31, 1974 A.D.	12:00 (noon) UTC January 1, 4713 B.C.

Guardian system clock procedures work only on Guardian timestamp formats, and OSS system time-management functions work only on OSS timestamp formats. For example, the `INTERPRETJULIANDAYNO` procedure requires a Julian day number, which does not exist in the OSS API. Likewise, the `strftime()` function requires an OSS time object.

To obtain a timestamp time with an accuracy to the microsecond in an OSS program, you must use the Guardian `JULIANTIMESTAMP` procedure.

Querying Process Times

You can obtain the execution time of processes from the OSS API with the `times()` or `clock()` function. The `times()` function provides more information than the `clock()` function.

An OSS process calls the `times()` or `clock()` function in the same way a UNIX process would. If your program is running as a Guardian process, you can also obtain the execution time of that Guardian process with the `times()` or `clock()` function. When a Guardian process calls the `times()` or `clock()` function, it cannot obtain the execution time of its OSS child processes.

The processor time of a Guardian process is accumulated as user time.

Limiting Processor Time Use

You can use the Guardian `SETLOOPTIMER` procedure to limit the processor time consumed by an OSS program (as opposed to the elapsed time that you would limit with the `alarm()` function). **Using SETLOOPTIMER in an OSS Program** demonstrates the use of `SETLOOPTIMER` to control the processor time used by a compute-bound program. The time limit can be set to a hundredth of a second.

Using SETLOOPTIMER in an OSS Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <cextdecs.h (SETLOOPTIMER)>

int timelimit = 247; /* time-out for processor processing
                     set to 2.47 seconds */
```

```

/* Catch time-out signal. */
void catchtimeout(arg) {
    printf("Program used more than %d hundredths of a seconds\n",\
        timelimit);
    exit(1);
}

struct sigaction action;

int main(int argc, char *argv[]) {
    int i;

    /* Set up processor processing time-out signal-catching routine. */
    action.sa_handler = catchtimeout;
    sigaction(SIGTIMEOUT, &action, NULL);

    /* Set processor processing time-out to specified limit. */
    SETLOOPTIMER((short)timelimit);
    for(i=0;i < 100000000; i++) i=i;    /* compute loop */
    return(0);
}

```

OSS Time-Management Functions

OSS Time-Management Functions table displays information about each OSS function that you can use to manage time. The columns of the table contain the following:

OSS Function

The name of the function and a brief description.

OSS Notes

Notes about the OSS implementation of the function. The note “Can return extended errors” means that the function can return errors that are Hewlett Packard Enterprise extensions to the XPG4 specification. Refer to the function’s reference page for further information.

Guardian Notes

Notes about using the function when it is called from a Guardian process.

The note “FDM set” (for file, directory, and memory) means that the function is one of a set of functions that have the following effects when the first of them is called from the Guardian environment:

- Two Guardian file-system numbers (not necessarily the next two available) are allocated for the root directory and the current working directory. These file numbers cannot be closed by calling the Guardian FILE_CLOSE_ procedure.
- The current working directory is assigned from the VOLUME attribute of the Guardian environment =_DEFAULTS DEFINE.
- The use of static memory by the process increases slightly.
- The functions in the FDM set are access(), chdir(), chmod(), chown(), chroot(), creat(), creat64(), fchmod(), fchown(), fstatvfs(), fstatvfs64(), ftok(), ftw(), ftw64(), getcwd(), glob(), lchmod(), lchown(), link(), lstat(), lstat64(), mkdir(), mkfifo(), mknod(), nftw(), nftw64(), open(), open64(), opendir(), pathconf(), pipe(),



readlink(), rename(), rename_oss(), rmdir(), select(), socket(), socketpair(), stat(), stat64(), statvfs(), statvfs64(), symlink(), unlink(), and utime() .

The information in this table is only a summary of the information in the reference page for a function. For details, refer to the reference pages either online or in the *Open System Services System Calls Reference Manual* and the *Open System Services Library Calls Reference Manual*.

Using gettimeofday() example shows the use of the gettimeofday() function to get date to the nearest microsecond. The function is provided for compatibility with BSD programs.

Using gettimeofday

```
#include <sys/time.h>
#include <time.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    struct timeval tv;
    struct timezone tz;
/*
    Read the system values for the current time and time zone.
    The time is expressed in seconds and microseconds since
    midnight January 1, 1970.
*/
    gettimeofday(&tv, &tz);
    printf("The current time is:\n%s", ctime(&tv.tv_sec));
    printf("and %d microseconds after midnight January 1, 1970\n", tv.tv_usec);
    printf("Daylight savings time value = %d\n", tz.tz_dsttime);
    printf("Local time zone in minutes west of Greenwich = %d\n",
           tz.tz_minuteswest);
    return(0);
}
```

Table 16: OSS Time-Management Functions

OSS Function	OSS Notes	Guardian Notes
alarm() Sets or changes the time-out value for an interval timer.	A SIGLIMIT signal is delivered to the process if the function call fails.	Does not work with Guardian TNS processes because OSS signals cannot be sent to a Guardian TNS process.
asctime() Converts a date and time to a string. Refer to ctime().		
clock() Reports processor time used.		
ctime() Converts time units.		

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>difftime()</code> Computes the difference between two calendar time values. Refer to <code>ctime()</code> .		
<code>ftime(3)</code> Gets date and time.	Can be called only by native processes.	
<code>getdate()</code> Converts user format date and time.	Can be called only by native processes.	Can be called only by native processes.
<code>getitimer()</code> obtains the current timer values.		Not available for Guardian TNS process.
<code>gettimeofday()</code> Gets date and time.		Cannot be used from the Guardian environment.
<code>gmtime()</code> Converts a time value to broken-down Coordinated Universal Time (UTC) time. Refer to <code>ctime()</code> .		
<code>localtime()</code> Converts a time value to broken-down local time. Refer to <code>ctime()</code> .		
<code>mktime()</code> Converts a broken-down time into time since the Epoch (00:00:00 UTC January 1, 1970 A.D.). Refer to <code>ctime()</code> .		
<code>setitimer()</code> Sets or changes an optionally-renewable timer that expires at a specified time in the future. Only real-time timer is supported.		Not available for Guardian TNS process.
<code>strftime()</code> Converts a date and time to a string.		
<code>strptime()</code> Converts a character string to a time value.		

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>time()</code> Obtains the time in seconds since the Epoch (00:00:00 UTC January 1, 1970 A.D.).		
<code>times()</code> Obtains process and child process elapsed times.	Time values are approximate. Times are not calculated for a process if it costs more to compute the running time than to run the process itself. There is no overflow limit for the fields in the <code>tms</code> structure. Can return extended errors.	When called from a Guardian process, the following restrictions apply: • Child times are not maintained. • All processor time is accumulated as user time. • Elapsed time might not be correct if the processor's internal clock has been reset.
<code>tzset()</code> Sets the time-zone conversion information. Refer to <code>ctime()</code> .		
<code>utime()</code> Sets file access and modification times.	Can return extended errors.	FDM set. The file access time is not updated by the <code>FILE_OPEN_</code> procedure.
<code>wcsftime()</code> Converts a date and time to a wide character string. Refer to <code>strftime()</code> .		

Guardian Time-Management Procedures

The following table displays information about each Guardian procedure that you can use to manage time. Measure procedures and procedures that have been superseded by other procedures are not shown. The columns of the table contain the following:

Guardian Procedure

The name of the procedure and a brief description.

OSS Notes

Additional considerations or restrictions when the Guardian procedure is called from an OSS process.

For details about a procedure, refer to the *Guardian Procedure Calls Reference Manual*.



Table 17: Guardian Time-Management Procedures

Guardian Procedure	OSS Notes
ADDDSTTRANSITION	Allows the super ID to add an entry to the daylight-saving time (DST) transition table.
CANCELPROCESSTIMEOUT	Cancels a process-time timer previously initiated by a call to the SIGNALPROCESSTIMEOUT procedure.
CANCELTIMEOUT	Cancels a process-time timer previously initiated by a call to the SIGNALTIMEOUT procedure.
COMPUTEJULIANDAYNO	Converts a Gregorian calendar date on or after January 1, 0001, to a Julian day number.
COMPUTETIMESTAMP	Converts a Gregorian calendar date into a 64-bit Julian timestamp.
CONVERTPROCESSTIME	Converts the process time returned by the PROCESSTIME, MYPROCESSTIME, and PROCESSINFO procedures into hours, minutes, seconds, milliseconds, and microseconds.
CONVERTTIMESTAMP	Converts a Greenwich mean time (GMT) timestamp to or from a local-time-based timestamp.
CPUTIMES	Returns the amount of time since system load that a given processor has spent in the following states: process busy, interrupt busy, and idle.
INTERPRETINTERVAL	Converts a fixed variable containing a number of microseconds into a combination of days, hours, minutes, seconds, milliseconds, and microseconds.
INTERPRETJULIANDAYNO	Converts a Julian day number to the year, month, and day.
INTERPRETTIMESTAMP	Converts a 64-bit Julian timestamp into a Gregorian date and time of day.

Table Continued

JULIANTIMESTAMP	Returns either a Julian timestamp or the number of microseconds elapsed since the last system load.	
MYPROCESSTIME	Returns the process execution time of the calling process.	
PROCESS_GETINFO_	Obtains information, including the execution time, about a specified process.	
PROCESS_GETINFOLIST_	Obtains information, including the execution time, about a specified process.	Some attributes obtain OSS time information.
PROCESSOR_GETINFOLIST_	Obtains information about a specified processor.	Some attributes obtain OSS time information.
SETLLOPTIMER	Enables you to limit the processor time a process is allowed.	
SETSYSTEMCLOCK	Allows you to change the system clock.	
SIGNALPROCESSTIMEOUT	Sets a timer based on process execution time.	
SIGNALTIMEOUT	Sets a timer to a given number of units of elapsed time.	
TIME	Provides the current date and time in integer form.	
TIMESTAMP	Provides the internal form of the interval clock of the processor where the application is running.	

²1 Applies to both AF_UNIX Release 2 sockets and earlier AF_UNIX sockets. However, AF_UNIX Release 2 sockets must be of the same mode to communicate with each other.

Interprocess Communication

Interprocess-communication mechanisms allow processes to communicate within the same processor, between processors, and between nodes. This section discusses the following topics:

Topic	Content
<u>Available Methods</u>	Summary of the mechanisms available to an OSS application.
<u>Common and Unique Characteristics</u>	Summary of the differences between UNIX and OSS interprocess communication.
<u>Interprocess-Communication Interoperability</u>	Summary of OSS and Guardian processes ability to communicate using OSS and Guardian interprocess-communication methods
<u>Performance Considerations</u>	Suggestions regarding the use of interprocess-communication mechanisms to improve application performance.
<u>OSS Interprocess-Communication Functions</u>	List of each OSS interprocess-communication function with notes about its OSS implementation and its use when called from Guardian processes.
<u>Guardian Interprocess-Communication Procedures</u>	List of each Guardian interprocess-communication procedure with notes about its use when called from OSS processes.

Available Methods

The OSS application program interface (API) provides access to the sockets, pipes, FIFOs, shared memory, semaphores, and message queues found on most UNIX systems. An OSS application also has access to Guardian APIs that provide additional interprocess-communication methods such as Guardian sockets and the Guardian requester/server interface known as the \$RECEIVE file. Some of the OSS and Guardian interprocess communication methods also allow OSS processes to communicate with Guardian processes.

The following table lists the interprocess-communication methods available in Open System Services using only OSS APIs, and the capabilities of each relevant to communication between processors and nodes. The methods are listed in approximate decreasing order of usage on UNIX systems.



Table 18: Interprocess Communication Using OSS APIs

IPC method	Within a processor	Between processors	Between Expand nodes	With other systems
OSS AF_INET and AF_INET6 sockets	Yes	Yes	Yes	Yes
OSS AF_UNIX sockets ¹	Yes	Yes	No	No
Pipes	Yes	Yes	No	No
FIFOs	Yes	Yes	Yes	No
Message queues	Yes	Yes	No	No
OSS signals	Yes	Yes	No	No
Shared memory	Yes	No	No	No
OSS semaphores	Yes	No	No	No

¹Applies to both AF_UNIX Release 2 sockets and earlier AF_UNIX sockets. However, AF_UNIX Release 2 sockets must be of the same mode to communicate with each other. See [Using AF_UNIX Sockets](#).

The following table lists the interprocess-communication methods available to an OSS process using mixed OSS and Guardian APIs, and the capabilities of each relevant to communication between processors and nodes. The methods are listed in an order consistent with [Table 18: Interprocess Communication Using OSS APIs](#).

Table 19: Interprocess Communication Using OSS and Guardian APIs

IPC method	Within a processor	Between processors	Between Expand nodes	With other systems
Guardian AF_INET and AF_INET6 sockets	Yes	Yes	Yes	Yes
OSS AF_INET and AF_INET6 sockets	Yes	Yes	Yes	Yes
OSS AF_UNIX sockets ¹	Yes	Yes	No	No
Pipes (with OSS processes only)	Yes	Yes	No	No

Table Continued

IPC method	Within a processor	Between processors	Between Expand nodes	With other systems
FIFOs	Yes	Yes	Yes	No
Message queues (with OSS processes only)	Yes	Yes	No	No
OSS signals	Yes	Yes	No	No
Shared memory ²	Yes	No	No	No
Guardian binary semaphores	Yes	No	No	No
OSS semaphores (with OSS processes only)	Yes	No	No	No
\$RECEIVE	Yes	Yes	Yes	No

¹Applies to both AF_UNIX Release 2 sockets and earlier AF_UNIX sockets. However, AF_UNIX Release 2 sockets must be of the same mode to communicate with each other. See [Using AF_UNIX Sockets](#).

²On servers running L-series or J06.12 or later J-series RVUs or H06.23 or later H-series RVUs, Guardian processes can use OSS function calls to access shared memory. See [Using Shared Memory](#) for more information.

Common and Unique Characteristics

OSS interprocess communication includes most UNIX interprocess-communication methods. The extensions and differences of OSS interprocess-communication functions compared to typical UNIX interprocess communication are summarized in the table [OSS Process-Management Functions](#). The table [Guardian Process-Management Procedures](#) summarizes the behavior of Guardian interprocess-communication procedures when called from OSS processes.

Features Common to UNIX and OSS Interprocess Communication

Sockets are the most widely used API for communication between processes running on different systems. The semantics used by sockets applications vary from UNIX system to UNIX system.

OSS sockets are based on the versions of UNIX systems developed at the University of California, Berkeley, by the Berkeley Software Distribution (BSD) group. The OSS implementations (BSD 4.3 for AF_INET and AF_INET6, and BSD 4.3+ for AF_UNIX) have limited extensions based on the XPG4 standard. OSS sockets obey semantics consistent with the POSIX.1G draft standard.

For a good introduction to the use of sockets, refer to a commercial text such as *UNIX Network Programming, Second Edition*, by W. Richard Stevens.

Pipes and FIFOs are memory-resident interprocess channels through which OSS processes can communicate. OSS processes having a common ancestor can communicate through a pipe, but common ancestry is not required for OSS processes to communicate through a FIFO, which is a named pipe.

OSS processes can share memory segments using shared-memory functions. OSS semaphores allow multiple processes to synchronize access to shared memory segments to avoid compromising the integrity of the data in those segments.

OSS processes can also communicate by sending and receiving messages using OSS message queues.



Features Unique to OSS Interprocess Communication

The following is a summary of the features that are unique to OSS interprocess communication:

- On servers running L-series or J06.12 or later J-series RVUs, or H06.23 or later H-series RVUs, there is a limit of approximately 2000 OSS shared memory segments that can be attached by one process. The exact number of OSS shared memory segments that can be attached per process depends on the process' usage of its address space.

On servers running earlier J-series RVUs, earlier H-series RVUs, or G-series RVUs, a process can attach no more than 13 segments at one time. If an attached segment is larger than 32 MB the maximum number of attached segments is reduced.
- OSS processes can share and synchronize access to memory segments within the same processor.
- The OSS sockets API supports features that allow the use of alternate TCP processes within a node.
- The `ftok()` function, which generates an OSS interprocess-communication key, is an optional function in XPG4, Version 2, and is included in the OSS API.
- The OSS sockets functions do not return the `errno` value `ENOSR`.
- The OSS sockets functions can return `errno` values that are Hewlett Packard Enterprise extensions to the XPG4 specification.
- `socketmark()`, `socket_transport_name_get()`, and `socket_transport_name_set()` are Hewlett Packard Enterprise extensions to the XPG4 specification
- OSS processes can use Guardian AF_INET and AF_INET6 sockets APIs.
- Beginning with the H06.24 and J06.13 RVUs, access to 64-bit memory is supported on:
 - 64-bit OSS processes
 - 32-bit OSS processes
 - Native Guardian processes

For information about sharing 64-bit memory, see [**Sharing Memory Between 32-Bit and 64-Bit Processes**](#).

Interprocess-Communication Interoperability

OSS and Guardian processes can communicate as described in the following subsections:

- [**Using Guardian AF_INET and AF_INET6 Sockets, and OSS AF_INET and AF_INET6 Sockets**](#)
- [**Using AF_UNIX Sockets**](#)
- [**Using FIFOs**](#) (named pipes)
- [**Using Shared Memory**](#)
- [**Using the \\$RECEIVE File**](#).

Guardian processes can use OSS sockets APIs, with the restrictions described in [**Table 20: OSS Interprocess-Communication Functions**](#). The table includes a short description of the OSS sockets functions.

OSS and Guardian processes cannot communicate using message queues, because only OSS processes can call the message queue functions. OSS and Guardian processes cannot use AF_UNIX sockets through the Guardian sockets API because Guardian sockets do not support the AF_UNIX address family.



Using Guardian AF_INET and AF_INET6 Sockets, and OSS AF_INET and AF_INET6 Sockets

OSS and Guardian processes can communicate within the same node and between nodes or systems using either:

- Guardian AF_INET or AF_INET6 sockets functions and semantics
- or
- OSS AF_INET or AF_INET6 sockets functions and semantics.

The AF_INET sockets functions use the services provided by NonStop TCP/IP; the AF_INET6 sockets functions use the services provided by NonStop TCP/IPv6. The two interfaces are similar, but NonStop TCP/IPv6 allows a larger IP address, and hence, access to a greater number of systems and devices.

The Guardian version of sockets was also derived from BSD sockets (Berkeley sockets) and provide the general function of BSD sockets, but have modified APIs and behavior. Guardian sockets are not POSIX-conformant and are less portable in code than OSS sockets, so converting a UNIX or POSIX application to use Guardian sockets might require source or algorithm changes. The use of Guardian sockets for NonStop TCP/IP, Parallel Library TCP/IP, and NonStop TCP/IPv6 is described in the *TCP/IP Programming Manual*.

OSS processes can communicate with Guardian processes within the same node and between nodes or systems through AF_INET or AF_INET6 sockets when either process uses the OSS sockets functions.

The examples in the following subsections illustrate some of these uses.

Communication Using Only OSS APIs

Using AF_INET in an OSS Server Process and **Using AF_INET in an OSS Requester Process** should give you an idea of how the OSS AF_INET communication mechanism works and provide you with a template you can expand in your applications.

Using AF_INET in an OSS Server Process and **Using AF_INET in an OSS Requester Process** show an OSS server process and an OSS requester process using OSS AF_INET functions to communicate.

NOTE:

These examples use the AF_INET sockets functions, which provide an interface to NonStop TCP/IP. The programs could also be written using the AF_INET6 sockets functions to interface to NonStop TCP/IPv6. For information and examples of AF_INET6 sockets usage, refer to the *TCP/IP Programming Manual*.

The server program in **Using AF_INET in an OSS Server Process** is compiled as a linked program file named `server2` that is secured for execution in the current working directory. The requester program in **Using AF_INET in an OSS Requester Process** is compiled as a linked program file named `requester2` that is secured for execution in the current working directory.

The server process can be started in the background from the OSS shell using the command:

```
run ./server2 &
```

After you start the server process, the requester process in **Using AF_INET in an OSS Requester Process** can be started in the foreground from the OSS shell using the command:

```
./requester2
```

Alternatively, both processes can be started in the foreground from separate terminal sessions. The following sample dialog illustrates the processes run from two terminal sessions. The server terminal session for the program in **Using AF_INET in an OSS Server Process** appears as:

```
/home/software/chrisbl/cabpubs: ./server2
Server starting
Message received = hello
```

```
Message received = goodbye
Server stopping
/home/software/chrisbl/cabpubs:
```

while the requester terminal session for the program in **Using \$RECEIVE in an OSS Requester Process** appears as:

```
/home/software/chrisbl/cabpubs: ./requester2
Requester starting
?hello
Read 6 bytes back from server
?goodbye
Read 8 bytes back from server
?
/home/software/chrisbl/cabpubs:
```

Note that the `server2` program is an iterative server, performing its functions for multiple requestors so that process-creation delays are minimized and processor usage is somewhat optimized. For examples of more traditional dynamic and static server applications, refer to **Performance Considerations**.

For more information about using OSS sockets, refer to the individual function reference pages either online or in the *Open System Services System Calls Reference Manual* and the *Open System Services Library Calls Reference Manual*.

Using AF_INET in an OSS Server Process

```
/* server2.c
 *
 * Simple OSS server example using OSS socket APIs
 * for communication between requester and server.
 *
 * This server accepts AF_INET stream socket connections on
 * a listening port. Data requests are handled by printing
 * the data and then replying back to the requester with the
 * original data. The server exits when idle for 60 seconds.
 */

#define _XOPEN_SOURCE_EXTENDED 1          /* needed for OSS sockets */

#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <sys/time.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netdb.h>

int fd, listenfd, newfd, highfd, fdcount, readycount, bytesread;
struct sockaddr_in addr; /* socket address (family,addr,port) */
struct timeval timeval; /* timeout structure for select() call */
fd_set fdset, rdset; /* set of fds, set of fds ready for reading */

#define IDLE_PERIOD 60 /* 60 seconds */
#define BUFF_LEN 1024 /* buffer size */
```



```

char msg_buff[BUFF_LEN+1]; /* one buffer is enough for all connections */

int main(int argc, char **argv)
{
    printf("Server starting\n");

    /* create AF_INET stream socket ... if error, then exit(1) */
    if ((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        printf("socket() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* Get server port from the command line or use default 12345 */
    if (argc > 1)
        addr.sin_port = (short)atoi(argv[1]); /* use command line arg */
    else
        addr.sin_port = 12345; /* use default */

    addr.sin_family = AF_INET; /* internet address family */
    addr.sin_addr.s_addr = gethostid(); /* assume server on our host */

    /* bind socket to address ... if error, then exit(1) */
    if (bind(fd, (const struct sockaddr *)&addr, sizeof(addr)) < 0)
    {
        printf("bind() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* make the socket a listening socket... if error then exit(1) */
    if (listen(fd, 5) < 0)
    {
        printf("listen() call failed [errno %d]\n", errno);
        exit(1);
    }

    fdcount = 1;
    highfd = fd;
    listenfd = fd;
    FD_ZERO(&fdset);
    FD_SET(listenfd, &fdset);

    /* Find a ready socket ... */
    while (1)
    {
        /* initialize timeout and bitmask for select() call */
        timeval.tv_sec = IDLE_PERIOD;
        timeval.tv_usec = 0;
        memcpy(&rdset, &fdset, sizeof(fdset));
        readycount = select(highfd+1, &rdset, NULL, NULL, &timeval);

        /* if timeout then exit(0) */
        if (readycount == 0)
        {
            printf("Server stopping\n");
            exit(0);
        }
    }
}

```

```

/* find first fd that was marked ready for reading */
for (fd = 0; fd <= highfd; fd++)
{
    if (FD_ISSET(fd, &rdset))
        break;
}
/* if fd is listening socket, accept incoming connection */
if (fd == listenfd)
{
    /* accept new connection ... if error, then exit(1) */
    if ((newfd = accept(fd, (struct sockaddr *)NULL, (size_t *)NULL)) < 0)
    {
        printf("accept() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* add fd to fd set */
    fdcount++;
    FD_SET(newfd, &fdset);
    if (newfd > highfd)
        highfd = newfd;
}
else /* else fd is an existing connection */
{
    if ((bytesread = recv(fd, msg_buff, BUFF_LEN, 0)) <= 0)
    {
        /* if recv error then exit(1) */
        if (bytesread < 0)
        {
            printf("recv() call failed [errno %d]\n", errno);
            exit(1);
        }
    }
    else /* bytesread == 0 ... peer closed connection */
    {
        close(fd);
        fdcount--;
        FD_CLR(fd, &fdset);
    }
}
else /* data received */
{
    msg_buff[bytesread] = '\0'; /* make string
                                NULL-terminated */
    printf("Message received = %s", msg_buff);

    /* echo back to sender ... if error then exit(1) */
    if (send(fd, msg_buff, bytesread, 0) < 0)
    {
        printf("send() call failed [errno %d]\n", errno);
        exit(1);
    }
}
} /* end else fd is an existing connection */
} /* end while (1) */
}

```


Using AF_INET in an OSS Requester Process

```
/* requester2.c
 *
 * Simple OSS requester example using OSS socket APIs
 * for communication between requester and server.
 *
 * This requester accepts terminal input and sends it to a
 * server using a send() call and then waits for a response using
 * a recv() call. When the server echoes the data back to the
 * requester, the recv() call completes and the requester prompts
 * for more terminal input. When terminal input ends with EOF,
 * the socket connection is closed and the requester stops.
 */

#define _XOPEN_SOURCE_EXTENDED 1          /* needed for OSS sockets */

#include <unistd.h>
#include <fcntl.h>
#include <tal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netdb.h>

int fd, bytesread;
struct sockaddr_in addr;      /* socket address (family,addr,port) */

#define BUFF_LEN 1024        /* data buffer length */
char msg_buff[BUFF_LEN+1];  /* data buffer */

int main(int argc, char **argv) {

    printf("Requester starting\n");

    /* create AF_INET stream socket ... if error, then exit(1) */
    if ((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        printf("socket() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* Get server port from the command line or use default 12345 */
    if (argc > 1)
        addr.sin_port = (short)atoi(argv[1]); /* use command line arg */
    else
        addr.sin_port = 12345;                  /* use default */

    addr.sin_family = AF_INET; /* internet address family */
    addr.sin_addr.s_addr = gethostid(); /* assume server on our host */

    /* connect to server ... if error, then exit(1) */
```

```

if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
{
    printf("connect() failed [errno %d]\n", errno);
    exit(1);
}

/* Prompt user, read data, send data to server and wait for reply */
while (1) {
    printf("?");
    fgets(msg_buff, BUFF_LEN, stdin);
    if (feof(stdin))
        exit(0);
    bytesread = (int)strlen(msg_buff);

    /* send line to server... if error then exit(1) */
    if (send(fd, msg_buff, bytesread, 0) < 0)
    {
        printf("send() call failed [errno %d]\n", errno);
        exit(1);
    }
    /* expect echo from server... if close or error then
       exit(1) */
    if ((bytesread = recv(fd, msg_buff, bytesread, 0)) <= 0)
    {
        if (bytesread == 0)
            printf("Server closed connection\n");
        else
            printf("recv() call failed [errno %d]\n", errno);
        exit(1);
    }
    printf("Read %d bytes back from server\n", bytesread);
}
}

```

Communication Using Mixed APIs

Using AF_INET in a Guardian Server Process and **Using AF_INET in an OSS Requester Process** should give you an idea of how the AF_INET communication mechanism works when different sets of APIs are used and provide you with a template you can expand in your applications.

Using AF_INET in a Guardian Server Process and **Using AF_INET in an OSS Requester Process** show a Guardian server process and an OSS requester process using Guardian AF_INET and OSS AF_INET functions, respectively, to communicate.

NOTE:

These examples use the AF_INET sockets functions, which provide an interface to NonStop TCP/IP. The programs could also be written using the AF_INET6 sockets functions to interface to NonStop TCP/IPv6. For information and examples of AF_INET6 sockets usage, refer to the *TCP/IP Programming Manual*.

The server program in **Using AF_INET in a Guardian Server Process** is compiled as a linked program file named `server2g` that is secured for execution in the current working directory. The requester program in **Using AF_INET in an OSS Requester Process** is compiled as a linked program file named `requester2` that is secured for execution in the current working directory.

The server process can be started in the background from the OSS shell using the command:

```
run ./server2g &
```

After you start the server process, the requester process in [Using AF_INET in an OSS Requester Process](#) can be started in the foreground from the OSS shell using the command:

```
./requester2
```

Alternatively, both processes can be started in the foreground from separate terminal sessions. The following sample dialog illustrates the processes run from two terminal sessions. The server terminal session for the program in [Using AF_INET in a Guardian Server Process](#) appears as:

```
/home/software/chrisbl/cabpubs: ./server2g
Server starting
Message received = hello
Message received = goodbye
Server stopping
/home/software/chrisbl/cabpubs:
```

while the requester terminal session for the program in [Using \\$RECEIVE in an OSS Requester Process](#) appears as:

```
/home/software/chrisbl/cabpubs: ./requester2
Requester starting
?hello
Read 6 bytes back from server
?goodbye
Read 8 bytes back from server
?
/home/software/chrisbl/cabpubs:
```

Using AF_INET in a Guardian Server Process

```
/* server2g.c
 *
 * Simple OSS server example using Guardian socket APIs
 * for communication between requester and server.
 *
 * This server accepts AF_INET stream socket connections on
 * a listening port. Data requests are handled by printing
 * the data and then replying back to the requester with the
 * original data. The server exits when idle for 60 seconds.
 *
 * Note - Guardian sockets do not have Posix semantics.
 * select() is not available and nonblocking calls require
 * starting operations with a nowait socket call such as
 * recv_nw() and then completing the operation with a call
 * to AWAITIOX(). Each operation requires a dedicated buffer.
 */

#define _GUARDIAN_SOCKETS          /* needed for Guardian sockets */

#include <unistd.h>
#include <fcntl.h>
#include <tal.h>
#include <cextdecs.h(AWAITIOX,FILE_CLOSE)>
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <sys/time.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netdb.h>

short fd, listenfd, newfd;          /* note - fd is really Guardian fnum */
short bytesread;
long tag;
int  addrlen;
struct sockaddr_in addr;             /* socket address (family,addr,port) */

enum tag { bind_tag = 1, accept_tag, recv_tag, send_tag };

#define NOWAIT          1
#define SYNC            1
#define FOREVER         -1L
#define IDLE_PERIOD     6000 /* 60 seconds = 6000 x 0.01 second */

#define MAX_FDS         100
#define BUFF_LEN        1024
char *msg_buff_ptr[MAX_FDS+1];      /* need one buffer per socket */
char *bufferptr;                  /* pointer to current buffer */
int main(int argc, char **argv)
{
    printf("Server starting\n");

    /* create AF_INET stream socket ... if error then exit(1) */
    if ((fd = (short)socket_nw(AF_INET, SOCK_STREAM, 0,
                              NOWAIT, SYNC )) < 0)
    {
        printf("socket_nw() call failed [errno %d]\n", errno);
        exit(1);
    }

    /* Get server port from the command line or use default 12345 */
    if (argc > 1)
        addr.sin_port = (short)atoi(argv[1]); /* use command line arg */
    else
        addr.sin_port = 12345;                  /* use default */

    addr.sin_family      = AF_INET;              /* internet address family */
    addr.sin_addr.s_addr = gethostid();          /* assume server on our host */

    /* bind socket to address ... if error then exit(1) */
    if (bind_nw(fd, (struct sockaddr *)&addr, sizeof(addr), bind_tag) < 0)
    {
        printf("bind_nw() call failed [errno %d]\n", errno);
        exit(1);
    }
    AWAITIOX(&fd, , , , FOREVER);
}

```

```

/* start a nowait accept ... if error then exit(1) */
addrlen = sizeof(addr);
if (accept_nw(fd, (struct sockaddr *)&addr, &addrlen, accept_tag) < 0)
{
    printf("accept_nw() call failed [errno %d]\n", errno);
    exit(1);
}

listenfd = fd;

/* Find a finished nowait operation */
while (1)
{
    fd = -1;
    AWAITIOX(&fd, (long *) &bufferptr, &bytesread, &tag,
            IDLE_PERIOD);

    /* if timeout then exit(0) */
    if (fd == -1)
    {
        printf("Server stopping\n");
        exit(0);
    }

    /* if incoming connection, accept connection */
    if (fd == listenfd)
    {
        /* create a new socket ... if error then exit(1) */
        if ((newfd = (short) socket_nw(AF_INET, SOCK_STREAM, 0,
                                     NOWAIT, SYNC) ) < 0)
        {
            printf("socket_nw() call failed [errno %d]\n", errno);
            exit(1);
        }
        AWAITIOX(&newfd, , , , FOREVER); /* should finish quickly */

        /* if too many connections then exit(1) */
        if (newfd > MAX_FDS)
        {
            printf("Too many connections\n");
            exit(1);
        }
        msg_buff_ptr[newfd] = malloc(BUFF_LEN+1);

        /* accept the connection using the filled in addr
         * buffer from the finished accept_nw() call. */

        if (accept_nw2(newfd, (struct sockaddr *)&addr, accept_tag) < 0)
        {
            printf("accept_nw2() call failed [errno %d]\n", errno);
            exit(1);
        }
        AWAITIOX(&newfd, , , , FOREVER); /* should finish quickly */

        /* need to post a recv() for AWAITIOX() to complete */
        if (recv_nw(newfd, bufferptr, BUFF_LEN, 0, recv_tag) < 0)

```

```

    {
        printf("recv() call failed [errno %d]\n", errno);
        exit(1);
    }
    /* need to post an accept for the next connection */
    addrlen = sizeof(addr);
    if (accept_nw(fd, (struct sockaddr *)&addr, &addrlen, accept_tag) < 0)
    {
        printf("accept_nw() call failed [errno %d]\n", errno);
        exit(1);
    }
}
else /* else data or error on an existing connection */
{
    if (bytesread == 0) /* no data - error or close */
    {
        /* close our end */
        FILE_CLOSE_(fd);          /* not close(fd)
                                   but FILE_CLOSE_(fd) */
        free(msg_buff_ptr[fd]);
        msg_buff_ptr[fd] = NULL;
    }
    else /* data was received */
    {
        /* so print data and echo it back to sender */
        bufferptr[bytesread] = '\0'; /* make string NULL-terminated */
        printf("Message received = %s", bufferptr);

        if (send_nw(fd, bufferptr, bytesread, 0, send_tag) < 0)
        {
            printf("send_nw() call failed [errno %d]\n", errno);
            exit(1);
        }
        AWAITIOX(&fd, , , , FOREVER); /* should finish quickly */

        /* need to post a recv() for AWAITIOX() to complete */
        if (recv_nw(fd, bufferptr, BUFF_LEN, 0, recv_tag) < 0)
        {
            printf("recv_nw() call failed [errno %d]\n", errno);
            exit(1);
        }
    }
} /* end else data was received */
} /* end while (1) */
}

```

Using AF_UNIX Sockets

An OSS process can communicate with a Guardian process using AF_UNIX sockets only when both processes use the OSS sockets APIs because only the OSS sockets APIs support the AF_UNIX address family.

Systems running RVUs earlier than J06.05 or H06.16 are using AF_UNIX Release 1 sockets and software.

Systems running J06.05 and later J-series RVUs or H06.16 and later H-series RVUs are using AF_UNIX Release 2 sockets and software. For AF_UNIX Release 2 software:



- The OSS sockets local server, \$ZPLS, is replaced by the OSS local server 2 servers. Although the name \$ZPLS is used as a transport-provider name, \$ZPLS is not a process that runs on the system.
- An OSS local server 2 server \$ZLSnn (where nn is the processor number) for each processor is started automatically when the processor loads.
- Sockets can be created in one of two modes: compatibility mode and portability mode:
 - Compatibility mode maintains the semantics of AF_UNIX Release 1 sockets to allow applications to migrate to AF_UNIX Release 2 software without changes.
 - Portability mode provides behaviors that are similar to other AF_UNIX implementations, such as Linux and HP-UX.
 - Sockets are created in compatibility mode by default. The mode of the socket is determined by the name of the transport provider process when the socket is created. After a socket is created, the mode of the socket cannot be changed.
 - Sockets created in compatibility mode can communicate with other sockets created in compatibility mode but cannot communicate with sockets created in portability mode.
 - Sockets created in portability mode can communicate with other sockets created in portability mode but cannot communicate with sockets created in compatibility mode.
 - The `accept()` function creates a socket in the same mode as the listening socket whose file descriptor is passed.

Compatibility Mode

Sockets created in compatibility mode have these characteristics and behaviors, which are the same as the characteristics and behaviors for AF_UNIX Release 1 sockets:

- If a relative name is passed to `connect()` then subsequent `getpeername()` calls return that same relative name. That is, whatever pathname is passed to `connect()` is returned, byte-for-byte, in subsequent calls to `getpeername()`, including path names beginning with `/E/local_system/` and names containing `/./` or `/../`.
- If the file to which a socket is bound is removed or renamed, calls to `getsockname()` fail with the error `FE_ENOENT`.
- If the file to which a datagram socket is bound is removed or renamed, and then a send operation is performed on the socket, a `recvfrom()` or `recvmsg()` call on the receiving socket returns a null address.
- A call to `listen()` with a *backlog* parameter less than or equal to 0 (zero) causes subsequent `connect()` calls to fail with the `ECONNREFUSED` error unless the socket has a pending `accept()` call.

You can create sockets in compatibility mode using one of these methods:



- Use the default transport-provider name. For systems running AF_UNIX Release 2 software, the default AF_UNIX transport-provider name is \$ZPLS, which results in sockets created in compatibility mode.
- Use the Guardian DEFINE =_AFUNIX_PROCESS_NAME to specify \$ZPLS as the transport provider name. This Guardian define must be a MAP DEFINE with a FILE attribute string of the desired AF_UNIX transport-provider name. For example:

```
TACL> ADD DEFINE =_AFUNIX_PROCESS_NAME, CLASS MAP, FILE $ZPLS
```

or

```
OSH> add_define =_AFUNIX_PROCESS_NAME class=MAP file=\$ZPLS
```

NOTE: Although the name \$ZPLS is used as a transport-provider name, \$ZPLS is not a process that runs on the system.

- Use the `socket_transport_name_set()` function to specify \$ZPLS as the transport provider name. This method overrides the default transport-provider name and allows you to create sockets in either mode within the same application.

Portability Mode

Sockets created in portability mode have these characteristics and behaviors:

- If a relative name is passed to `connect()` then subsequent `getpeername()` calls return the corresponding absolute name. That is, the absolute path name returned by `getpeername()` has the *local_system/* /E/ prefix (if present) stripped, all symbolic links resolved, all */./* strings suppressed, and all */../* strings resolved.
- If the file to which a socket is bound is removed or renamed, calls to `getsockname()` continue to return the original absolute path name.
- If the file to which a datagram socket is bound is removed or renamed, and then a send operation is performed on the socket, a `recvfrom()` or `recvmsg()` call on the receiving socket returns the original pathname in the address parameter.
- Portability mode sockets are more efficient because `getsockname()`, `recvfrom()` and `recvmsg()` no longer require an interaction with an OSS name server.
- A call to `listen()` with a *backlog* parameter less than or equal to 0 (zero) causes subsequent blocking `connect()` calls to block until there is a corresponding `accept()` call. This differs from the behavior for compatibility mode, in which calls to `connect()` fail with the ECONNREFUSED error unless the socket has a pending `accept()` call.

You can create sockets in portability mode using one of these methods:



- Use the Guardian `DEFINE =_AFUNIX_PROCESS_NAME` to specify `$ZAFN2` as the default transport-provider name. This Guardian define must be a MAP DEFINE with a FILE attribute string of the desired `AF_UNIX` transport-provider name. For example:

```
TACL> ADD DEFINE =_AFUNIX_PROCESS_NAME,CLASS MAP,FILE $ZAFN2
```

or

```
OSH> add_define =_AFUNIX_PROCESS_NAME class=MAP file=\$ZAFN2
```

NOTE: Although the name `$ZAFN2` is used as a transport-provider name, `$ZAFN2` is not a process that runs on the system.

- Use the `socket_transport_name_set()` function to specify `$ZAFN2` as the transport-provider name. This method overrides the default transport-provider name and allows you to create sockets in either mode within the same application.

Using FIFOs

OSS and Guardian processes can communicate using FIFOs:

- Within the same processor
- Between processors
- Between nodes (by way of the `/E` directory)

However, they cannot communicate using unnamed pipes. Unnamed pipes can only be used to communicate between OSS processes.

Using Shared Memory

In the Guardian environment, both flat and selectable memory segments can be created and shared. Guardian processes can share and synchronize access to memory using Guardian shared memory and semaphore functions. OSS processes can share and synchronize access to memory with other OSS processes using OSS shared memory and semaphore functions. The shared memory segments created and managed by OSS `shm*` functions are distinct from segments created by the Guardian `SEGMENT_ALLOCATE_` and related procedure calls. Both kinds of segments can be created and shared, but one kind cannot be shared with the other.

Both Guardian and OSS processes can call `SEGMENT_ALLOCATE_` and related Guardian procedure calls: they can share segments as described for `SEGMENT_ALLOCATE_`.

On servers running L-series or J06.12 or later J-series RVUs or H06.23 or later H-series RVUs:

- Both Guardian processes and OSS processes can call the OSS shared memory functions, which are `shmat()`, `shmctl()`, `shmdt()`, and `shmget()`. They can share segments as described for `shmget()`.
- The number of shared memory identifiers per CPU is limited to 1000 on J-series and on L-series prior to L22.09, and to 4000 on L22.09 or later L-series RVUs.

On servers running J06.11 or earlier J-series RVUs, H06.22 or earlier H-series RVUs, or G-series RVUs:



- Only OSS processes can call `shmget()` and related functions; Guardian process calls to these functions fail and `errno` is set to the value of `[ENOTOSS]`.
- A shared memory segment created by `shmget()` can contain up to 128 megabytes (MB).
- A process can attach no more than 13 segments at one time. If an attached segment is larger than 32 MB the maximum number of attached segments is reduced.

The distinction between flat and selectable segments, and how to use these shared memory segments, is discussed in detail in **Managing Memory**. Using shared memory is an efficient IPC mechanism that can be used when large amounts of data need to be shared between an OSS process and a Guardian process running on the same processor.

Sharing 64-Bit Memory

Beginning with the H06.24 and J06.13 RVUs, access to 64-bit memory is supported on:

- 64-bit OSS processes
- 32-bit OSS processes
- Native Guardian processes

For information about sharing 64-bit memory, see **Sharing Memory Between 32-Bit and 64-Bit Processes**.

Coordinating Memory Sharing

Shared-memory communication generally requires synchronization of critical regions, to serialize access to and update of the shared data. OSS processes can use semaphores, via the `sem*` functions. Both OSS and Guardian processes can use Binary Semaphores, manipulated with the `BINSEM_*O` procedures; see the *Summary of Guardian Binary Semaphore Procedures* in the *Guardian Programmer's Guide*.

Use One API to Manipulate a Memory Segment

You cannot use both OSS functions and Guardian procedures to manipulate the same memory segment. For example, an OSS process cannot call a Guardian procedure, such as `SEGMENT_ALLOCATE_`, to allocate a memory segment and then call an OSS function, such as `shmctl()`, to manipulate the same segment.

Considerations for TNS Processes

Hewlett Packard Enterprise strongly recommends that a G-series OSS TNS process use flat segments and not explicit selectable segments, because in a C program using `xmem` (the large or wide model), the TNS data heap and most global data reside in a selectable segment, and you cannot access both selectable segments at the same time. (The data heap and global data in native processes are not in a selectable segment.)

See **Managing Memory** for more information about NonStop operating system extended data (memory) segments.

Using the \$RECEIVE File

OSS and Guardian processes can use Guardian procedures to communicate between nodes, between processors, and within the same processor through `$RECEIVE`. `$RECEIVE` is a special file in the Guardian filesystem through which a process can receive system messages or messages from other processes.

The basic algorithm requires a server process to open its `$RECEIVE` file and accept messages from one or more requester processes that open the server process file (in UNIX terms, each requester opens the server process as a special file in the Guardian file system). The server replies to received messages as necessary and requester processes see the replies as coming from the server process file. The use of `$RECEIVE` is described in the *Guardian Programmer's Guide*.

Using \$RECEIVE in an OSS Server Process and **Using \$RECEIVE in an OSS Requester Process** should give you an idea of how the `$RECEIVE` interprocess communication mechanism works and provide you with a template you can expand in



your applications. The *Guardian Programmer's Guide* contains detailed instructions and examples on using \$RECEIVE, and you should read it before expanding these examples.

The server program in **Using \$RECEIVE in an OSS Server Process** is compiled as a linked program file named `server1` that is secured for execution in the current working directory. The requester program in **Using \$RECEIVE in an OSS Requester Process** is compiled as a linked program file named `requester1` that is secured for execution in the current working directory.

The server process can be started in the background from the OSS shell using the command

```
run -name=/G/echo ./server1 &
```

which `/G/echo` gives the running server process the process name \$ECHO in the Guardian file system. That process name is used by requesters to open the server program.

After you start the server process, the requester process in **Using \$RECEIVE in an OSS Requester Process** can be started in the foreground from the OSS shell using the command:

```
.\requester1
```

Alternatively, both processes can be started in the foreground from separate terminal sessions. The following sample dialog illustrates the processes run from two terminal sessions. The server terminal session for the program in **Using \$RECEIVE in an OSS Server Process** appears as:

```
/home/software/chrisbl/cabpubs: run -name=/G/echo ./server1
Server starting
Message received = hello
Message received = goodbye
Server stopping
/home/software/chrisbl/cabpubs:
```

while the requester terminal session for the program in **Using \$RECEIVE in an OSS Requester Process** appears as:

```
/home/software/chrisbl/cabpubs: ./requester1
Requester starting
?hello
Read 6 bytes back from server
?goodbye
Read 8 bytes back from server
?
/home/software/chrisbl/cabpubs:
```

Using \$RECEIVE in an OSS Server Process

```
/* server1.c
 *
 * Simple OSS server example using Guardian file input/output
 * APIs for communication between requester(s) and server.
 *
 * This server accepts requests on $RECEIVE. Data requests are
 * handled by printing the data and then replying back to the
 * requester with the original data. Open and close requests
 * are handled by updating an opener count. The server exits
 * when idle for 60 seconds.
 */

#include <unistd.h>
#include <fcntl.h>
#include <tal.h>
#include <cextdecs.h(FILE_OPEN_, READUPDATEX, REPLYX, FILE_CLOSE_, \
                    AWAITIOX, FILE_GETINFO_)>
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

#define NOWAITDEPTH 1          /* enable nowait operations */
#define RECVDEPTH 1           /* enable replies on $RECEIVE */
#define IDLE_PERIOD 6000      /* 60 seconds = 6000 x 0.01 second */

short retcode; /* return code for APIs that return short */
_cc_status CC; /* return code for APIs that return condition code */

short fnum, bytesread, error, openers;
char *filename = "$RECEIVE"; /* file name for incoming requests */

#define BUFF_LEN 1024 /* buffer size */
union msg
{
    short sys_msg[50]; /* buffer viewed as system message */
    char data[BUFF_LEN+1]; /* buffer viewed as user data */
} msg_buff; /* single buffer is enough for all connections */

int main(int argc, char **argv)
{
    printf("Server starting\n");

    /* The server process opens its $RECEIVE file for nowait IO. */
    retcode = FILE_OPEN_(
        filename, /* filename */
        (short)strlen(filename), /* filename length */
        &fnum, /* file number returned */
        0, /* access mode (r/w) */
        0, /* exclusion mode (shared) */
        NOWAITDEPTH, /* nowait depth (nowait enabled) */
        RECVDEPTH, /* receive depth (1 msg) */
        0); /* options (sys msgs enabled) */
    if (retcode != 0) /* if error, exit(1) */
    {
        fprintf(stderr, "FILE_OPEN_() to %s failed\n", filename);
        fprintf(stderr, "Return code = %d\n", retcode);
        exit(1);
    }

    /* Read data from $RECEIVE ... handle system msgs and user msgs */
    openers = 0;
    while (1)
    {
        CC = READUPDATEX(fnum, /* file number */
            msg_buff.data, /* msg buffer */
            BUFF_LEN, /* length */
            &bytesread); /* number of bytes read */
        if (_status_ne(CC)) /* if error, exit(1) */
        {
            fprintf(stderr, "READUPDATEX() to %s failed\n", filename);
            exit(1);
        }
    }
}

```

```

/* Wait for I/O completion on $RECEIVE. */
CC = AWAITIOX(&fnum,                                /* file number */
              ,                                       /* buffer address */
              &bytesread,                            /* count transferred */
              ,                                       /* tag */
              IDLE_PERIOD);                          /* time limit (60 sec) */

/* get last file error ... 6 = sys msg, 40 = timeout */
FILE_GETINFO_(fnum, &error);

if (error != 0 && error != 6 && error != 40) /* if error then exit(1) */
{
    fprintf(stderr, "AWAITIOX() to %s failed\n", filename);
    exit(0);
}

/* if timeout then exit(0) */
if (error == 40)
{
    FILE_CLOSE_(fnum);
    printf("Server stopping\n");
    exit(0);
}

/* If a system message then handle and reply. */
if (error == 6) /* system message received */
{
    switch (msg_buff.sys_msg[0]) /* first short is msg type */
    {
        case -103: /* open message */
            openers++;
            break;
        case -104: /* close message */
            openers--;
            break;
        default: /* other system message */
            break;
    }

    CC = REPLYX(msg_buff.data, 0); /* reply with 0 data bytes */
    if (_status_ne(CC)) /* if error, exit(1) */
    {
        fprintf(stderr, "REPLYX() to %s failed\n", filename);
        exit(1);
    }
}

else /* else must be a user request */
{
    msg_buff.data[bytesread] = '\0'; /* make string NULL-terminated */
    printf("Message received = %s", msg_buff.data);
    CC = REPLYX(msg_buff.data, bytesread); /* reply with data */
    if (_status_ne(CC)) /* if error, exit(1) */
    {
        fprintf(stderr, "REPLYX() to %s failed\n", filename);
        exit(1);
    }
}

```

```

    }
} /* end while (!done) */
}

```

Using \$RECEIVE in an OSS Requester Process

```

/* requester1.c
 *
 * Simple OSS requester example using Guardian file input/output
 * APIs for communication between requester and server.
 *
 * This requester accepts terminal input and sends it to a
 * server using a waited WRITEREADX() call. When the server
 * echoes the data back to the requester, the WRITEREADX() call
 * completes and the requester prompts for more terminal input.
 * When terminal input ends with EOF, the server is closed and
 * the requester stops.
 */

#include <unistd.h>
#include <fcntl.h>
#include <tal.h>
#include <cextdecs.h(FILE_OPEN_,FILE_CLOSE_, \
                    WRITEREADX,FILE_GETINFO_)>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

short retcode; /* return code for APIs that return short */
_cc_status CC; /* return code for APIs that return condition code */

#define BUFF_LEN 1024 /* buffer length */
char msg_buff[BUFF_LEN+1]; /* buffer */
char *procname; /* pointer to server process name */
short fnum, bytesread, error;

int main(int argc, char **argv)
{
    /* Get server process name from the command line,
     * else assign default name $ECHO. */
    if (argc > 1)
        procname = argv[1];
    else
        procname = "$ECHO";

    printf("Requester starting\n");

    /* Requester process opens the server process for waited IO. */
    retcode = FILE_OPEN_(
        procname, /* server process name */
        (short)strlen(procname), /* process name length */
        &fnum, /* file number returned */
        0, /* access mode (r/w) */
        0, /* exclusion mode (shared) */
        0); /* nowait depth (waited IO) */
    if (retcode != 0) /* if error, exit(1) */
    {

```

```

        fprintf(stderr, "FILE_OPEN_() to %s failed\n", procname);
        fprintf(stderr, "Return code = %d\n", retcode);
        exit(1);
    }

    /* Prompt user, read data, send data to server and wait for reply */
    while (1)
    {
        printf("?");
        fgets(msg_buff, BUFF_LEN, stdin); /* read user input */
        if (feof(stdin)) /* if EOF */
        {
            FILE_CLOSE_(fnum); /* close server */
            exit(0); /* and exit(0) */
        }
        bytesread = (short)strlen(msg_buff); /* includes newline */
        CC = WRITEREADX(fnum, /* file number */
                       msg_buff, /* msg buffer */
                       bytesread, /* size of string to server */
                       BUFF_LEN, /* length of buffer */
                       &bytesread); /* number read back */
        if (_status_ne(CC)) /* if error, exit(1) */
        {
            fprintf(stderr, "WRITEREADX() to %s failed\n", procname);
            retcode = FILE_GETINFO_(fnum, &error);
            if (retcode == 0)
                fprintf(stderr, "File error = %d\n", error);
            else
                fprintf(stderr, "FILE_GETINFO() failed, retcode = %d\n",
                        retcode);
            exit(1);
        }
        printf("Read %d bytes back from server\n", bytesread);
    } /* end while(1) */
}

```

Performance Considerations

You can improve the performance of your applications with respect to interprocess communication by following these recommendations.

For input and output in general:

There can be delays due to the blocking nature of many input/output calls. It can take an indeterminate amount of time to complete a blocked call. Use the `select()` function or nonblocking input/output when indeterminate waits are not desirable. Refer to **Using AF_INET in an OSS Server Process** for an example of the use of the `select()` call.

For pipes and FIFOs:

- Whenever possible, keep processes that use pipes and FIFOs on the same processor. When processes reside on the same processor, there is no message system traffic. When processes reside on different processors, a message system I/O path is created, which slightly degrades performance. However, keeping processes on different processors can provide better load balancing and scalability for your applications.
- Make the size of the data buffer exchanged through pipes and FIFOs equal to or less than the size of the `PIPE_BUF` system variable. A buffer the size of `PIPE_BUF` or smaller can be written in one I/O operation.



An application can force the OSS file system to attempt to resize the buffer used for a specific pipe or FIFO file by calling the `write()` or `writew()` function for a write of more than `PIPE_BUF` bytes. If resizing occurs, subsequent reads of up to 52 kilobytes can occur in a single operation. Because all pipes and FIFOs in a processor have buffers allocated from a fixed amount of memory, increasing the buffer size for one pipe or FIFO to improve its throughput reduces the number of pipes and FIFOs that can exist simultaneously in that processor.

For semaphores:

Include deadlock-avoidance techniques in your applications.

For sockets:

- OSS `AF_UNIX` Release 2 sockets have improved performance over `AF_UNIX` Release 1 sockets because `AF_UNIX` Release 2 sockets have fewer system processes involved in the delivery of each message. The following list shows the order of increasing overhead (decreasing performance) for `AF_UNIX` Release 2 processes and sockets:
 1. Both processes are in the same processor and their sockets were created in that processor.
 2. The processes are in different processors and each process is using a socket created in its processor.
 3. One of the processes resides in a processor other than where its socket was created.
 4. Both processes reside in processors other than where their socket was created.
- OSS `AF_UNIX` Release 1 sockets and OSS `AF_INET` and `AF_INET6` sockets involve multiple system processes in the data path for each message, even if both ends of the connection are on the same processor; as a result, the socket overhead is much higher than for other interprocess communication mechanisms. The following tradeoffs exist for the use of OSS `AF_UNIX` Release 1 sockets, `AF_INET` sockets, and `AF_INET6` sockets:
 - If you must have portability and POSIX standard sockets semantics, use OSS sockets.
 - If you require improved performance with sockets-like semantics but portability is not a primary consideration, consider using Guardian sockets. The use of Guardian sockets for NonStop TCP/IP, Parallel Library TCP/IP, and NonStop TCP/IPV6 is described in the *TCP/IP Programming Manual*.
 - If you require even higher performance and can abandon portability and sockets semantics, consider using `$RECEIVE`. Refer to the *Guardian Programmer's Guide* for more information about programs that use `$RECEIVE`.
 - If you require maximum performance with portability and can abandon sockets semantics, use OSS pipes or FIFO files. OSS pipes and FIFOs typically provide a direct data path without intervening system processes.

OSS Interprocess-Communication Functions

Table **Table 20: OSS Interprocess-Communication Functions** displays information about each OSS function that you can use for interprocess communication. The columns of the table contain the following:

OSS Function

The name of the function and a brief description.

OSS Notes

Notes about the OSS implementation of the function. The note “Can return extended errors” means that the function can return errors that are Hewlett Packard Enterprise extensions to the XPG4 specification. Refer to the function’s reference page for further information.

Beginning with the J06.15 and H06.26 RVUs, a partner or customer OSS Security Event-Exit Process (SEEP) is supported and can participate in access-control decisions for OSS objects. (For OSS SEEP details, see **Accessing OSS**)



SEEP-Protected Files.) The note “OSS SEEP consultation” indicates that the OSS name server consults the OSS SEEP when this function is called for operations on files in OSS SEEP-protected filesets and the OSS SEEP is running.

Guardian Notes

Notes about using the function when it is called from a Guardian process. The note “FDM set” (for file, directory, and memory) means that the function is one of a set of functions that have the following effects when the first of them is called from the Guardian environment:

- Two Guardian file-system numbers (not necessarily the next two available) are allocated for the root directory and the current working directory. These file numbers cannot be closed by calling the Guardian FILE_CLOSE_ procedure.
- The current working directory is assigned from the VOLUME attribute of the Guardian environment =_DEFAULTS DEFINE.
- The use of static memory by the process increases slightly.
- The functions in the FDM set are `access()`, `chdir()`, `chmod()`, `chown()`, `chroot()`, `creat()`, `creat64()`, `faccessat()`, `fchdir()`, `fchmod()`, `fchmodat()`, `fchown()`, `fchownat()`, `fdopendir()`, `fstatat()`, `fstatat64()`, `fstatvfs()`, `fstatvfs64()`, `ftok()`, `ftw()`, `ftw64()`, `getcwd()`, `glob()`, `lchmod()`, `lchown()`, `link()`, `linkat()`, `lstat()`, `lstat64()`, `mkdir()`, `mkdirat()`, `mkfifo()`, `mkfifoat()`, `mknod()`, `mknodat()`, `nftw()`, `nftw64()`, `open()`, `openat()`, `open64()`, `openat64()`, `opendir()`, `pathconf()`, `pipe()`, `pread()`, `pread64_()`, `pwrite()`, `pwrite64_()`, `readlink()`, `readlinkat()`, `rename()`, `renameat()`, `rename_oss()`, `rmdir()`, `select()`, `socket()`, `socketpair()`, `stat()`, `stat64()`, `statvfs()`, `statvfs64()`, `symlink()`, `symlinkat()`, `unlink()`, and `utime()`.

`symlink()`, `unlink()`, and `utime()`.

This table only summarizes the information in the reference page for a function. For details, refer to the reference pages either online or in the *Open System Services System Calls Reference Manual* and the *Open System Services Library Calls Reference Manual*.

NOTE: Some functions also have reentrant versions. For a complete list of supported functions, including information about RVU support for each function, see **Documented OSS Functions**.

Table **Table 21: OSS Interprocess-Communication-Related Functions** displays information about functions useful when managing interprocess communication. The columns of the table contain the same kinds of information as in table **Table 20: OSS Interprocess-Communication Functions**.

Table 20: OSS Interprocess-Communication Functions

OSS Function	OSS Notes	Guardian Notes
<code>accept()</code> Accepts a new connection on a socket.	Can return extended errors.	FDM set.
<code>bind()</code> Binds a name to a socket.	Can return extended errors. OSS SEEP consultation.	

Table Continued

OSS Function	OSS Notes	Guardian Notes
<code>connect()</code> Connects a socket.	Can return extended errors. OSS SEEP consultation.	
<code>ftok()</code> Generates an OSS interprocess communications (IPC) key.	Can return extended errors.	FDM set.
<code>getsockname()</code> Gets the locally bound name of a socket.	Can return extended errors.	
<code>getsockopt()</code> Gets socket options.	Can return extended errors. Does not support <code>SO_LINGER</code> or <code>SO_ACCEPTCONN</code> .	
<code>listen()</code> Listens for socket connections and limits the backlog of incoming connections.	Can return extended errors.	
<code>mkfifo()</code> Creates a FIFO file.	Can return extended errors. Fails and sets <code>errno</code> to the <code>EINVAL</code> value if: - Bits in the mode parameter other than file permission bits and <code>S_IFIFO</code> are set. - Attempt is made to create a FIFO in the <code>/G</code> file system. Sets the group ID of the new FIFO to the group ID of the parent directory if the <code>S_ISGID</code> flag is set in that directory; otherwise, the group ID is set to the effective group ID of the calling process. <code>G</code> file system. OSS SEEP consultation.	FDM set.
<code>msgctl()</code> Performs message queue control operations.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>msgget()</code> Creates or returns the identifier for a message queue.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>msgrcv()</code> Reads a message from a message queue.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>msgsnd()</code> Sends a message to a message queue.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>pclose()</code> Closes a pipe to a process that was opened using <code>popen()</code> .	Can return extended errors.	
<code>pipe()</code> Creates an interprocess channel.	Can return extended errors.	FDM set.
<code>popen()</code> Initiates a pipe to a process.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
<code>recv()</code> Receives a message from a connected socket.	Can return extended errors.	
<code>recv64()</code> Receives a message from a connected socket.	Can return extended errors. Beginning with the H06.24 and J06.13 RVUs.	
<code>recvfrom()</code> Receives a message from a socket.	Can return extended errors.	
<code>recvfrom64()</code> Receives a message from a socket.	Can return extended errors. Beginning with the H06.24 and J06.13 RVUs.	
<code>recvmsg()</code>) Receives a message from a socket using a message structure.	Can return extended errors.	
<code>recvmsg64()</code> Receives a message from a socket using a message structure.	Can return extended errors. Beginning with the H06.24 and J06.13 RVUs.	
<code>semctl()</code> Performs semaphore control operations.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.

Table Continued



OSS Function	OSS Notes	Guardian Notes
semget () Creates a new semaphore set ID or returns the ID of an existing semaphore set.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
semop () Performs semaphore operations.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process.
send () Sends a message to a connected socket.	Can return extended errors.	
send64 () Sends a message to a connected socket.	Can return extended errors. Beginning with the H06.24 and J06.13 RVUs.	
sendmsg () Sends a message to a socket using a message structure.	Can return extended errors.	
sendmsg64 () Sends a message to a socket using a message structure.	Can return extended errors. Beginning with the H06.24 and J06.13 RVUs.	
sendto () Sends a message to a socket.	Can return extended errors.	
sendto64 () Sends a message to a socket.	Can return extended errors. Beginning with the H06.24 and J06.13 RVUs.	
setsockopt () Sets socket options.	Can return extended errors. Nonzero values other than 1 can be used to set Boolean options.	
shmat () Attaches a shared memory segment to the address space of the calling process.	Can return extended errors. Memory segments are not propagated to a child process by any of the <code>exec</code> , <code>tdm_exec</code> , or <code>tdm_spawn</code> sets of functions or when the child resides in a different processor from the parent. See Using Shared Memory for shared memory details.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process on servers running J06.11 or earlier J-series RVUs, H06.22 or earlier H-series RVUs, or G-series RVUs. See Using Shared Memory for shared memory details.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>shmctl()</code> Performs shared memory control operations.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process on servers running J06.11 or earlier J-series RVUs, H06.22 or earlier H-series RVUs, or G-series RVUs. See Using Shared Memory for shared memory details.
<code>shmdt()</code> Detaches a shared memory segment.	Can return extended errors.	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process on servers running J06.11 or earlier J-series RVUs, H06.22 or earlier H-series RVUs, or G-series RVUs. See Using Shared Memory for shared memory details.
<code>shmget()</code> Creates a new shared memory segment or returns the identifier of an existing shared memory segment.	Can return extended errors. Reuses no-longer-used shared memory identifiers after a long time elapses. A shared memory segment is shared through the Kernel-Managed Swap-space Facility. If a processor fails, all shared memory segments in that processor are lost. The maximum number of shared memory identifiers per CPU is defined in the “Number of Shared Segments and Identifiers” section of the <code>shmget(2)</code> man page. On servers running L-series or J06.12 or later J-series RVUs, or H06.23 or later H-series RVUs, there is no configured limit to the size of a shared memory segment. The size of the shared memory segment is limited by system resources only. On servers running earlier J-series RVUs, earlier H-series RVUs, or G-series RVUs, a shared memory segment can contain up to 128 megabytes (MB).	Sets <code>errno</code> to the <code>ENOTOSS</code> value when called from a Guardian process on servers running J06.11 or earlier J-series RVUs, H06.22 or earlier H-series RVUs, or G-series RVUs. See Using Shared Memory for shared memory details.
<code>shutdown()</code> Shuts down socket send or receive operations.	Can return extended errors.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>socketmark()</code> Determines whether a socket is at the out-of-band mark.	Hewlett Packard Enterprise extension to XPG4 specification.	
<code>socket()</code> Creates an endpoint for communications.	Can return extended errors.	FDM set.
<code>socketpair()</code> Creates a pair of connected sockets.	Can return extended errors. Has memory segment considerations.	FDM set.
<code>socket_transport_name_get()</code> Gets the name of the transport-provider process.	Hewlett Packard Enterprise extension to XPG4 specification.	
<code>socket_transport_name_set()</code> Sets the name of the transport-provider process.	Hewlett Packard Enterprise extension to XPG4 specification.	

Table 21: OSS Interprocess-Communication-Related Functions

OSS Function	OSS Notes	Guardian Notes
<code>endhostent()</code> Closes the network host database.	Availability of database might require system manager action.	
<code>endnetent()</code> Closes the network database.	Availability of database might require system manager action.	
<code>endprotoent()</code> Closes the connection to the network protocols database.	Availability of database might require system manager action.	
<code>endservent()</code> Closes the connection to the network services database.	Availability of database might require system manager action.	
<code>freeaddrinfo()</code> Frees a specified address information structure previously allocated by <code>getaddrinfo()</code> .		

Table Continued



OSS Function	OSS Notes	Guardian Notes
<pre>freehostent()</pre> Frees <code>hostent</code> structure previously allocated by <code>getipnodebyaddr</code> or <code>getipnodebyname</code>.		
<pre>gai_sterr()</pre> Aids applications in printing error messages returned by <code>getaddrinfo()</code>.		
<pre>getaddrinfo()</pre> Converts hostnames and service names into socket address structures.		
<pre>gethostbyaddr()</pre> Gets a network host entry by address.	Availability of database might require system manager action.	
<pre>gethostbyname()</pre> Gets a network host entry by name.	Availability of database might require system manager action.	
<pre>gethostent()</pre> Gets the next entry in the network host database.	Availability of database might require system manager action.	
<pre>gethostid()</pre> Gets the identifier for the current host.	Availability of database might require system manager action.	
<pre>getipnodebyaddr()</pre> Gets a network host entry by address.		
<pre>getipnodebyname()</pre> Gets host information based on IP node name.		
<pre>getnameinfo()</pre> Translates a protocol-independent host address to a host name and gives the service name.		
<pre>getnetbyaddr()</pre> Gets a network entry by address.	Availability of database might require system manager action.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>getnetbyname()</code> Gets a network entry by name.	Availability of database might require system manager action.	
<code>getnetent()</code> Gets the next entry in the network database.	Availability of database might require system manager action.	
<code>getpeername()</code> Gets the name of the peer socket.	Can return extended errors.	
<code>getprotobyname()</code> Gets a protocol entry by name.	Availability of database might require system manager action.	
<code>getprotobynumber()</code> Gets a protocol entry by number.	Availability of database might require system manager action.	
<code>getprotoent()</code> Gets the next protocol entry.	Availability of database might require system manager action.	
<code>getservbyname()</code> Gets a network services entry by name.	Availability of database might require system manager action.	
<code>getservbyport()</code> Gets a network services entry by port number.	Availability of database might require system manager action.	
<code>getservent()</code> Gets the next entry in the network services database.	Availability of database might require system manager action.	
<code>if_freenameindex()</code> Frees dynamic memory allocated by the <code>if_nameindex</code> function.		
<code>if_indextoname()</code> Maps an interface index to its corresponding name.		
<code>if_nameindex()</code> Gets all interface names and indexes.		
<code>if_nametoindex()</code> Maps an interface name to its corresponding index.		

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>htonl()</code>	Converts a 32-bit integer from host byte order to Internet network byte order.	
<code>htons()</code>	Converts a 16-bit integer from host byte order to Internet network byte order.	
<code>inet_addr()</code>	Constructs an integer Internet address from a character string in Internet dot notation.	
<code>inet_lnaof()</code>	Extracts the local network address from an Internet host address.	
<code>inet_makeaddr()</code>	Constructs an integer Internet address from an Internet network number and a local network address.	
<code>inet_netof()</code>	Extracts the network number from an Internet host address.	
<code>inet_network()</code>	Translates a character string in Internet dot notation to an Internet network address.	
<code>inet_ntoa()</code>	Translates an integer Internet address into a character string in Internet dot notation.	
<code>inet_ntop()</code>	Converts a binary internet address to a character value.	
<code>inet_pton()</code>	Converts a character string to an IPv4 or IPv6 binary address.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>ntohl()</code> Converts a 32-bit integer from Internet network byte order to host byte order.		
<code>ntohs()</code> Converts a 16-bit integer from Internet network byte order to host byte order.		
<code>sethostent()</code> Opens the network host database and sets the next entry pointer to the first entry in the database.	Availability of database might require system manager action.	
<code>setnetent()</code> Opens the network database and sets the next entry pointer to the first entry in the database.	Availability of database might require system manager action.	
<code>setprotoent()</code> Opens the protocols database and sets the next entry to the first entry in the database.	Availability of database might require system manager action.	
<code>setservent()</code> Opens the network services database and sets the next entry pointer to the first entry in the database.	Availability of database might require system manager action.	

Guardian Interprocess-Communication Procedures

Guardian interprocess-communication procedures include those for use of:

- Guardian sockets, as described in the *TCP/IP Programming Manual* for NonStop TCP/IP, Parallel Library TCP/IP, and NonStop TCP/IPv6.
- Shared memory
- Binary semaphores
- The requester/server \$RECEIVE file method

Table 22: Guardian Interprocess-Communication Procedures displays information about each Guardian procedure that you can use for interprocess communication using the latter three methods. Procedures that have been superseded by other procedures are not shown. The columns of the table contain the following:

Guardian Procedure

The name of the procedure and a brief description.



OSS Notes

Additional considerations or restrictions when the Guardian procedure is called from an OSS process.

Table 23: Guardian Interprocess-Communication-Related Procedures displays information about Guardian procedures useful when performing interprocess communication. The columns of the table contain the same kind of information as in **Table 22: Guardian Interprocess-Communication Procedures**. For details about a procedure, refer to the *Guardian Procedure Calls Reference Manual* and the *Guardian Programmer's Guide*.

Table 22: Guardian Interprocess-Communication Procedures

Guardian Procedure	OSS Notes
AWAITIO[X] Checks for completion of nowait operations pending on one or all Guardian file opens (using file numbers), including operations on a server process file.	
BINSEM_CLOSE_ Closes a binary semaphore.	Closes only a Guardian semaphore, not an OSS semaphore.
BINSEM_CREATE_ Creates, opens, and locks a binary semaphore.	Creates, opens, and locks only a Guardian semaphore, not an OSS semaphore.
BINSEM_FORCELOCK_ Forces a lock on a binary semaphore.	Forces a lock on only a Guardian semaphore, not an OSS semaphore.
BINSEM_ISMINE_ Reports whether this process owns (has locked) the semaphore.	Beginning with the H06.23 and J06.12 RVUs.
BINSEM_LOCK_ Locks a binary semaphore.	Locks only a Guardian semaphore, not an OSS semaphore.
BINSEM_OPEN_ Opens a binary semaphore.	Opens only a Guardian semaphore, not an OSS semaphore.
BINSEM_UNLOCK_ Relinquishes a lock on a binary semaphore.	Relinquishes a lock on only a Guardian semaphore, not an OSS semaphore.
CANCEL Cancels the oldest outstanding nowait operation on a specified Guardian open, such as a server process file.	
CANCELREQ Cancels a a specific outstanding nowait operation on a specified Guardian file open, such as a server process file.	
FILE_AWAITIO64_ Completes a previously initiated I/O operation.	Beginning with the H06.24 and J06.13 RVUs.

Table Continued



FILE_CLOSE_ Closes a Guardian file open, such as a server process file.	
FILE_GETINFO_ Provides error information and characteristics about a Guardian file open, such as a server process file.	
FILE_GETRECEIVEINFO_ Returns information about the last message read from the \$RECEIVE file of the calling process.	
FILE_OPEN_ Establishes a Guardian file open with a file number. Can be used by:	OSS SEEP consultation, beginning with the J06.15 and H06.26 RVUs. For OSS SEEP details, see Accessing OSS SEEP-Protected Files .
• A requester process to open a server process file • A server process to open its \$RECEIVE file	
FILE_READUPDATE64_ Reads data from a disk or process file in anticipation of a subsequent write to the file and extends the capabilities of READUPDATE(XL).	Beginning with the H06.24 and J06.13 RVUs.
FILE_REPLY64_ Sends a reply message to a message received earlier in a corresponding call to READUPDATE[X XL]/FILE_READUPDATE64_ on the \$RECEIVE file. The FILE_REPLY64_ procedure can be called even if there are incomplete nowait I/O operations pending on \$RECEIVE.	Beginning with the H06.24 and J06.13 RVUs.
FILE_WRITEREAD64_ For files opened in a no-waited manner, control is returned to the caller after the request is initiated. In this case, the count read is not returned and the caller needs to call FILE_AWAITIO64_O or FILE_COMPLETEL_O to complete the I/O and obtain the count read. For files opened in a waited manner, the completion is waited for, at which time the data has been transferred to the caller's buffer.	Beginning with the H06.24 and J06.13 RVUs.
READUPDATE[X] Reads a message from the \$RECEIVE file of a server process, expecting to reply to the message sender in a subsequent call to REPLY[X].	

Table Continued

Guardian Procedure	OSS Notes
REPLY[X] Replies through the \$RECEIVE file of a server process to a message that was previously read by READUPDATE[X].	
SEGMENT_ALLOCATE_ Allocates a 32-bit flat or selectable memory segment.	Allocates both flat and selectable segments.
SEGMENT_ALLOCATE64_ By default, allocates a 64-bit flat memory segment. Optionally allocates a 32-bit flat or selectable segment.	Beginning with the H06.20 and J06.09 RVUs.
SEGMENT_DEALLOCATE_ Deallocates a flat or selectable memory segment.	Deallocates both flat and selectable segments.
SEGMENT_USE_ Makes a selectable segment current (a flat segment need not be made current). It also returns the last selectable segment in use.	
WRITEREAD[X] Sends a message to a server process and waits for a reply (assuming waited I/O).	

Table 23: Guardian Interprocess-Communication-Related Procedures

Guardian Procedure	OSS Notes
CONTROL Issues CONTROL operations to a process that simulates an input/output device.	
CONTROLBUF Issues CONTROLBUF operations to a process that simulates an input/output device.	
FILE_CONTROL64_ Performs device-dependent I/O operations.	Beginning with the H06.24 and J06.13 RVUs.
FILE_CONTROLBUF64_ Performs device-dependent I/O operations requiring a data buffer.	Beginning with the H06.24 and J06.13 RVUs.
FILE_READ64_ Returns data from an open file to the application process's data area using 64-bit addresses.	Beginning with the H06.24 and J06.13 RVUs.

Table Continued



FILE_SETMODENOWAIT64_ Set device-dependent functions in a nowait manner on nowait files.	Beginning with the H06.24 and J06.13 RVUs.
FILE_WRITE64_ Writes data from an array in the application program to an open file using 64-bit addresses.	Beginning with the H06.24 and J06.13 RVUs.
MESSAGESTATUS Checks the \$RECEIVE file of a server process to see whether a specific message has been canceled.	
READ[X] Reads information from the \$RECEIVE file of a server process.	
SETMODE Issues SETMODE functions to a process that simulates an input/output device.	
SETMODENOWAIT Performs the same functions as SETMODE but in a nowait manner.	
SETPARAM Issues SETPARAM operations to a process that simulates an input/output device.	
WRITE[X] Sends a message to a server process and waits for a reply (assuming waited input/output).	

Managing I/O

This section describes how to manage OSS tape, terminal, and printer I/O. It discusses the following topics:

Topic	Content
<u>Common and Unique Characteristics</u>	Summary of the differences between OSS and UNIX I/O.
<u>I/O Interoperability</u>	The objects on which you can use OSS I/O functions and Guardian procedures.
<u>Tape I/O</u>	Tape I/O using pax and the Guardian API.
<u>Terminal I/O</u>	Terminal I/O using the OSS API.
<u>Using OSSTTY to Redirect Input and Output to Guardian Objects</u>	Using the OSSTTY facility to redirect the OSS standard files to Guardian processes and EDIT files.
<u>Printer I/O</u>	Printing OSS files using the OSS and Guardian APIs.
<u>OSS I/O Management Functions</u>	List of each OSS I/O function with notes about its OSS implementation and its use on Guardian objects.
<u>Guardian I/O Management Procedures</u>	List of each Guardian I/O procedure with notes about its use on OSS objects.

The major difference between managing I/O with the OSS API and managing I/O with the UNIX operating system is that the OSS environment offers limited I/O features. For greater control over most I/O operations, you must use Guardian procedures.

Common and Unique Characteristics

OSS tape I/O and printing are handled by OSS utilities. There are no direct OSS programmatic tape and printer interfaces. For programmatic control, you must use the Guardian API.

Features Common to UNIX and OSS Terminal I/O Management

OSS terminal I/O is handled as with UNIX network terminals. As in UNIX systems, the name of the controlling terminal is `/dev/tty`. Both canonical and noncanonical input are supported.

Features Unique to OSS Terminal I/O Management

Some terminal parameter values have no effect, because all access to the OSS environment is through the Telserv server and a VT100 terminal emulation or through OSSTTY. The OSS environment supports two types of windows: static windows, which are opened with the Subsystem Control Facility (SCF), and dynamic windows, which Telserv or OSSTTY creates when it receives a connection request.



I/O Interoperability

OSS terminal I/O functions operate only on the terminal device `/dev/tty`. The functions `cfsetispeed()` and `cfsetospeed()` have no effect on the network terminals (they affect only local terminals). The function `tcsetattr` affects both the Telserv and OSSTTY terminals.

There are no OSS I/O functions that apply to printers or tape drives.

Guardian I/O procedures operate only on Guardian devices.

Tape I/O

The OSS environment has no programmatic access to tapes. To access tapes in the OSS environment you must use the `pax` utility. You can, however, use Guardian procedures to write OSS files and output directly to tape and read them directly from tape.

Tape I/O With `pax`

The `pax` utility reads, writes, and lists files and directories to and from archive files. An archive file can be a disk or tape file. You can use the `pax` utility from a C program by invoking it with the `popen()` function. For information about the `pax` utility, refer to the `pax(1)` reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.

Using the `pax` Utility From a Program shows how to use the `pax` utility from a program. It reads the names of files to be backed up from the user's terminal input and passes the list of files to the `pax` utility using `popen()`.

NOTE:

Normally, you or system operators read from and write to tape archives using commands and utilities from the command line prompt or shell scripts. This example shows how you can accomplish the same task programmatically, even though it might be unusual to do so.

Using the `pax` Utility From a Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

FILE *fpout;
#define MAXLINE 132
char paxbuf[MAXLINE];
char line[MAXLINE];

int main (int argc, char *argv[]) {

/* The program is called with name of a backup (archive) file.
   Make sure the correct number of arguments are entered.
   If not, print a usage message and exit.
*/
    if(argc < 2) {
        fprintf(stderr, "Usage: paxopen <backup file>\n");
        exit(1);
    }
/* Create a string for popen() with the necessary arguments
   to the pax command to use it as a filter. */
    sprintf(paxbuf, "pax -w -f %s", argv[1]);
/* Open the pax utility. */
```



```

    if((fpout = popen(paxbuf, "w")) == NULL) {
        fprintf(stderr, "popen error on %s\n", paxbuf);
        exit(1);
    }

    /* Copy the names of the files to be backed up to the pax filter. */
    for(;;) {
        fprintf(stderr, "File to back up (^Y for end of list): ");
        if(fgets(line, MAXLINE, stdin) == NULL)
            break;
        if(fputs(line, fpout) == EOF) {
            fprintf(stderr, "fputs error to pipe\n");
            exit(1);
        }
    }

    /* Check for errors on input processing. */
    if(ferror(stdin)) {
        fprintf(stderr, "fgets error\n");
        exit(1);
    }

    /* Check for errors in the execution of the pax command. */
    if(pclose(fpout) == -1) {
        fprintf(stderr, "pclose error\n");
        exit(1);
    }
    printf("Backup file %s created\n", argv[1]);
    exit(0);
}

```

Tape I/O With the Guardian API

To use a programmatic interface to tape or to use labeled tapes, you must use Guardian procedures.

Labeled tapes have the following advantages over unlabeled tapes:

- A labeled tape contains information about the contents of the tape that prevent undesired overwriting of the tape.
- Automatic cartridge systems use only labeled tapes.
- Many systems and vendors use only labeled tapes.
- You can access labeled tapes on different systems than the one your file is on.

To perform read or write operations on an OSS file on a labeled tape, follow these steps:

1. Open the desired OSS file.
2. Open the proper tape file by passing the name of a DEFINE that has the appropriate attributes as the *filename* parameter to the `FILE_OPEN_` procedure.
3. Position the tape to available space on the tape.
4. Perform read or write operations on the tape with the `READX` or `WRITEX` procedure.
5. Close access to the tape with the `FILE_CLOSE_` procedure.

DEFINES are file-system elements that provide a means for passing information, such as attributes, to a process. DEFINES allow attributes to be grouped and named. Attributes can then be passed to a process or retrieved by specifying the name of the DEFINE. For information about DEFINES and programming with tapes, refer to the *Guardian Programmer's Guide*.



Terminal I/O

The only terminals supported by OSS are Telserv windows and OSSTTY windows, so terminal I/O is the same as that for a UNIX network terminal. OSS also supports local terminals through OSSTTY, although OSSTTY has the look and feel of a remote terminal. The OSS terminal interface is described in detail in the `termios(4)` and `tty(7)` reference pages either online or in the *Open System Services System Calls Reference Manual*.

Accessing Terminals From the OSS Environment

The only way to access the OSS environment interactively is through an OSSTTY window or through a Telserv virtual terminal, or window, with VT100 emulation.

Static and Dynamic Windows

Telserv provides two kinds of windows: static and dynamic.

- A static window is opened with Telserv SCF commands and with OSSTTY, and can exist independently of a connection to the OSS environment. (The OSSTTY windows `#stdin`, `#stdout`, and `#stderr` are considered to be static because they are preconfigured and cannot be changed while OSSTTY is running.)
- A dynamic window is opened by Telserv in response to a connection request. OSSTTY does not support dynamic windows.

For additional information about Telserv windows, refer to the *Telserv Guide* and the *TCP/IP Configuration and Management Manual*. For additional information about OSSTTY windows, refer to **Using OSSTTY to Redirect Input and Output to Guardian Objects** and to the *Open System Services Management and Operations Guide*.

Blocking and Nonblocking Opens

Both blocking and nonblocking opens are supported for static windows:

- Blocking open: For a Telserv window, the function returns successfully when the connection to the window is requested and established. For OSSTTY windows, the function returns successfully when the availability of the redirection target is established.
- Nonblocking open: For a Telserv window, the function returns successfully if a Telserv connection to the window is already established when the request is made. For an OSSTTY window, the function does not depend on the availability of the redirection target.

For dynamic windows, only blocking opens are supported. You cannot open a dynamic window unless a connection has already been established.

Blocking and Nonblocking Terminal I/O

For J-series RVUs, H06.06 and later H-series RVUs, and G06.27 and later G-series RVUs, nonblocking behavior is supported for Telserv and OSSTTY terminal objects for these functions:

- `select()`
- `FILE_COMPLETE_()`
- `FILE_COMPLETE_SET_()`
- `FILE_COMPLETE_GETINFO_()`

If you attempt to use these functions on a terminal process running on a system that is running RVUs earlier than G06.27 or H06.06:



- `select()` returns the error `ENOTSUP`.
- `FILE_COMPLETE_()` and `FILE_COMPLETE_SET_()` return the error `FEINVALIDOP`.

Nonblocking behavior for the `open()` and `fcntl()` functions continues to be supported.

Each processor in the system has a terminal helper process that provides support for nonblocking I/O for terminal devices. Terminal helpers are named processes with names of the form `$ZTTnn`, where `nn` is the processor number. Terminal helper processes are started automatically when the processor is started. If the terminal helper process stops or terminates abnormally, the processor halts.

Terminal Parameters

Most of the control characters in the `c_cflag` field of the `termios` structure in the `termios.h` header file are ignored. The `HUPCL` flag of the `c_flag` field affects both local and remote terminals. For more information, refer to the `termios(4)` and `tty(7)` reference pages either online or in the *Open System Services System Calls Reference Manual*.

Also, the following functions have no effect:

- `cfsetispeed()`
- `cfsetospeed()`
- `csendbreak()`

Control Characters

The default values of control characters are defined in the `termios.h` header file. The definitions are in the array specified by the `c_cc` field of the `termios` structure. According to the standards that the OSS environment conforms to, these values can vary from implementation to implementation.

The OSS API has two terminal I/O modes: canonical mode, in which input data is assembled into lines, and noncanonical mode, in which input data is not assembled into lines.

Canonical mode control characters, their default values, associated flags, and indexes into the array specified by the `c_cc` field of the `termios` structure are shown in the following table.

Table 24: Canonical Mode Control Characters and Associated Flags in `termios.h`

Control Character	Default Value	Associated Flag	Index in <code>c_cc</code>
EOF	<code>%X04 ^D</code>	<code>ICANON</code>	<code>VEOF</code>
EOL	<code>%X0A ^M</code>	<code>ICANON</code>	<code>VEOL</code>
ERASE	<code>%X08 ^H</code>	<code>ICANON</code>	<code>VERASE</code>
INTR	<code>%X03 ^C</code>	<code>ISIG, NOFLSH</code>	<code>VINTR</code>
KILL	<code>%X15 ^U</code>	<code>ICANON</code>	<code>VKILL</code>

Table Continued



Control Character	Default Value	Associated Flag	Index in <code>c_cc</code>
QUIT	%X1C ^\	ISIG, NOFLSH	VQUIT
START	%X11 ^Q	IXON/IXOFF	VSTART
STOP	%X13 ^S	IXON/IXOFF	VSTOP
SUSP	%X1A ^Z	ISIG, NOFLSH	VSUSP

Noncanonical mode control characters, their default values, associated flags, and indexes into the array specified by the `c_cc` field of the `termios` structure are shown in the following table.

Table 25: Noncanonical Mode Control Characters and Associated Flags in `termios.h`

Control Character	Default Value	Associated Flag	Index in <code>c_cc</code>
INTR	%X03 ^C	ISIG, NOFLSH	VINTR
MIN	%X00	ICANON	VMIN
QUIT	%X1C ^\	ISIG, NOFLSH	VQUIT
START	%X11 ^Q	IXON	VSTART
STOP	%X13 ^S	IXON	VSTOP
SUSP	%X1A ^Z	ISIG, NOFLSH	VSUSP
TIME	%X00	ICANON	VTIME

Because OSSTTY has no control over the interface from which it reads data, the STOP and START characters in the stream are discarded. OSSTTY supports flow control for applications when directed to do so programmatically through `tcflow()` (TCION and TCIOFF flags).

Accessing Terminals From the Guardian Environment

You might need to access a terminal in an OSS program using Guardian procedures. **Accessing an OSS Terminal With Guardian Procedures in the OSS API** shows the use of Guardian procedures to read from and write to a Telserv window in an OSS program. The program uses `FILE_OPEN_` to open the Telserv window and then uses `WRITEREADX` to write to the Telserv window and read data back. This OSS program uses Guardian I/O and therefore cannot use OSS job-control features. For example, signals are handled during OSS file I/O calls, such as `EINTR` during a `read()` call, but are deferred during Guardian file I/O calls, such as a `WRITEREADX` call.

Accessing an OSS Terminal With Guardian Procedures in the OSS API

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <tal.h>
#include <cextdecs.h>(FILE_OPEN_, READX, WRITEX, WRITEREADX, \
```



```

PROCESS_GETINFO_,FILE_CLOSE_)>

short filenum;          /* file code number */
short retcode;          /* procedure call return code */
_cc_status CC;          /* condition code */
char *terminal;         /* name of terminal */
char hometerm[64];      /* name of home terminal */
short htlen;            /* length of home terminal name */
char buffer[134];       /* buffer for input/output line */
char *buf;              /* pointer to buffer string */
short bytesread;        /* bytes read */

int main(int argc, char *argv[]) {
/* Get the Guardian terminal filename from the command argument. */
    if (argc > 1)
        terminal = argv[1];
    else {
        retcode = PROCESS_GETINFO_(          /* get Home terminal name */
            ,                               /* process handle */
            ,                               /* process filename */
            ,                               /* maximum length of filename */
            ,                               /* actual length of filename */
            ,                               /* execution priority */
            ,                               /* process handle of MOM */
            hometerm,                       /* home terminal name */
            64,                             /* maximum length of hometerm name */
            &htlen                          /* actual length of hometerm name */
        );
/* Check if PROCESS_GETINFO_ was successful. */
        if (retcode != 0) {
            fprintf(stderr, "PROCESS_GETINFO_ failed: retcode = %d\n", \
                retcode);
            exit(1);
        }
        terminal = hometerm;
    }

    retcode = FILE_OPEN_(terminal,          /* name of Guardian terminal */
        (short)strlen(terminal),            /* terminal name length */
        &filenum,                           /* file number returned */
        0,                                 /* open for read/write access */
    );
/* Check if the open was successful. */
    if (retcode != 0) {
        fprintf(stderr, "Can't open %s\n", terminal);
        fprintf(stderr, "retcode = %d\n", retcode);
        exit(1);
    }
/* Write to the terminal using WRITEX. */
    CC = WRITEX(filenum,                    /* terminal file number */
        buf = "Type in a line followed by ENTER:", /* output string */
        (short)strlen(buf)                  /* size of string */
    );
/* Check for successful terminal write operation. */
    if (_status_ne(CC)) {
        fprintf(stderr, "WRITEX to terminal failed: %s\n", terminal);
        exit(1);
    }
}

```

```

    }
    /* Read input from the terminal using READX. */
    CC = READX(filenum,          /* terminal file number */
               buffer,          /* input buffer */
               sizeof(buffer),   /* size of buffer */
               &bytesread,      /* bytes read */
               );
    /* Check for successful terminal read operation. */
    if (_status_ne(CC)) {
        fprintf(stderr, "READX from terminal failed: %s\n", terminal);
        exit(1);
    }
    /* Write and read from the terminal in one operation. */
    buffer[0] = ':'; buffer[1] = '\0'; /* set up colon (:) prompt */
    CC = WRITEREADX(filenum,     /* terminal file number */
                    buffer,      /* output string */
                    1,           /* size of output string */
                    132,         /* maximum buffer size */
                    &bytesread,  /* actual bytes read */
                    );
    if (_status_ne(CC)) {
        fprintf(stderr, "WRITEREADX from terminal failed: %s\n", \
                    terminal);
        exit(1);
    }
    buffer[bytesread] = 0;
    printf("%s\n", buffer);          /* print out data just read */
    FILE_CLOSE_(filenum);           /* close terminal file */
    exit(0);
}

```

Using OSSTTY to Redirect Input and Output to Guardian Objects

The OSSTTY facility provides a way for OSS processes to interact with the Guardian environment. OSSTTY enables OSS processes to redirect their standard input, standard output, and standard error files to Guardian processes and, for standard output only, to Guardian EDIT files. Before the implementation of OSSTTY, the OSS environment could be accessed only through a Telserv TELNET session. Applications running in the OSS environment could interact with only one Guardian process, namely Telserv. The OSSTTY process emulates a terminal, and eliminates the dependency OSS applications have on TELNET sessions.

The following figure gives a high-level view of OSSTTY and its role as an intermediary between the OSS and Guardian environments.



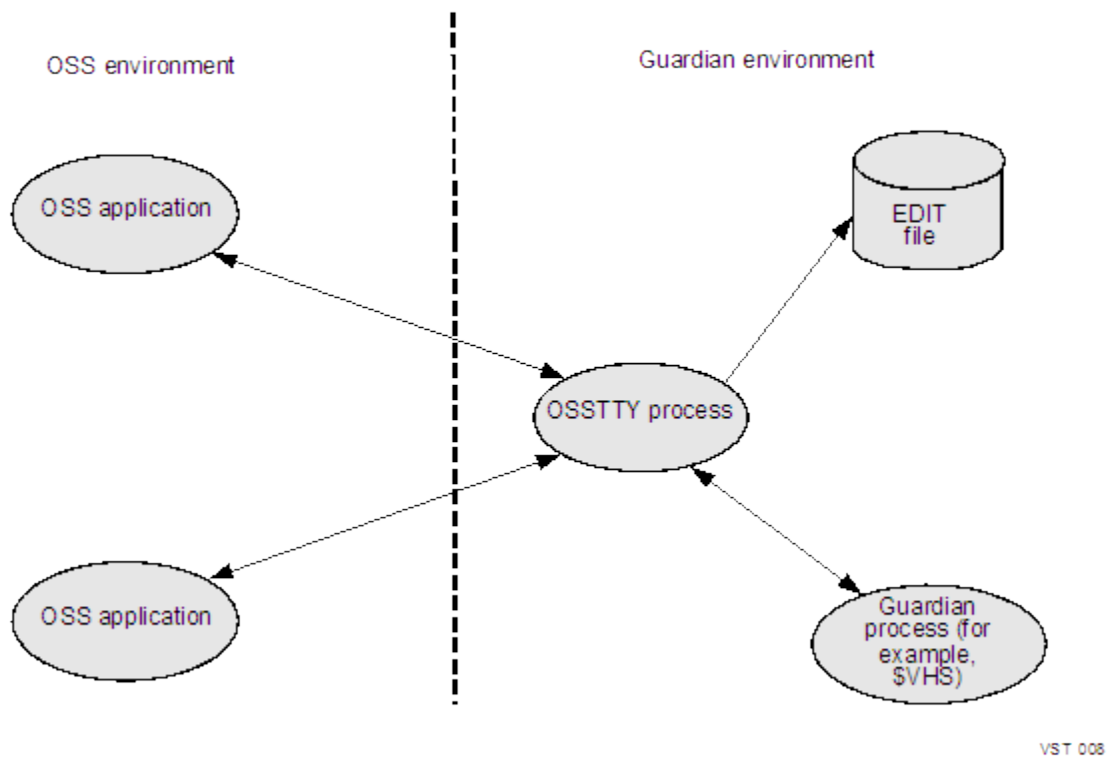


Figure 6: OSSTTY Interaction With OSS and Guardian Environments

OSSTTY supports three standard qualifiers: `#stdin`, `#stdout`, and `#stderr`. OSS applications can redirect their standard input, standard output, and standard error to OSSTTY by using these qualifiers:

Qualifier	Description
<code>/G/name/#stdin</code>	Redirects standard input data to the OSSTTY process identified by <i>name</i> .
<code>/G/name/#stdout</code>	Redirects standard output data to the OSSTTY process identified by <i>name</i> .
<code>/G/name/#stderr</code>	Redirects standard error data to the OSSTTY process identified by <i>name</i> .

OSSTTY, in turn, redirects data from these qualifiers to Guardian objects that you specify when starting OSSTTY.

Starting OSSTTY

You can start OSSTTY either from the TACL prompt or through the OSH command line. The following subsections present simple forms of the commands for starting OSSTTY. For detailed descriptions of all OSSTTY run-time options, see the *Open System Services Management and Operations Guide*.

Starting OSSTTY From the TACL Prompt

You can start OSSTTY by entering a `run` command at the TACL prompt. A general form of a `run` command that starts an OSSTTY process is as follows:

```
TACL> run osstty/ name process-name in input out output term hometerm
```

This command starts a named OSSTTY process. The `in`, `out`, and `term` parameters specify the Guardian targets of the OSSTTY process.

The value specified for `input` can be either of the following:

- A Guardian process; for example, `$vhs` (Guardian virtual hometerm subsystem) or `$zhome` (Guardian reliable home terminal).
- Any Telserv terminal.

The OSSTTY process will redirect all data received from the specified `input` to any process that uses `$process-name . #stdin` (Guardian process) or `/G/process -name/ #stdin` (OSS process) as its standard input.

The value specified for `output` can be any of the following:

- A Guardian process; for example, `$vhs` (Guardian virtual hometerm subsystem) or `$zhome` (Guardian reliable home terminal).
- Any Telserv terminal.
- A Guardian EDIT file.

OSSTTY will redirect to the specified `output` all data received from any process that uses `$process-name . #stdout` (Guardian process) or `/G/process -name/ #stdout` (OSS process) as its standard output.

The value specified for `hometerm` can be either of the following:

- A Guardian process; for example, `$vhs` (Guardian virtual hometerm subsystem) or `$zhome` (Guardian reliable home terminal).
- Any Telserv terminal.

OSSTTY will redirect to the specified `hometerm` all error messages received from any process that uses `$process-name . #stderr` (Guardian process) or `/G/process -name/ #stderr` (OSS process) as its standard error.

Starting OSSTTY on the OSH Command Line

You can start OSSTTY by specifying the `-osstty` flag on the `osh` command line. The `osh` command initially routes the input, output, and error file information of the OSS process it launches to its own standard input, output, and error files. You can specify standard input, output, and error files for the child process on the `osh` command as run options or by using the redirection operators (`<`, `>`, `2>`); for example:

```
TACL> osh/ in input, out output, term hometerm/ -osstty
```

```
TACL> osh -osstty > output
```

The values for `input`, `output`, and `hometerm` are as described under [Starting OSSTTY From the TACL Prompt](#).

When `-osstty` is specified on the `osh` command line, OSSTTY is launched only if the OSS shell determines that it is necessary. The OSS shell makes this determination based on the standard files specified in the run options or through



redirection operators: if any one of the files specified is not accessible to the child process, then OSSTTY is launched. For example, in the following command, OSSTTY is launched because OSS cannot access the Guardian EDIT file `myfile` as an output file:

```
TACL> osh/out $data.mysubvol.myfile/ -osstty
```

If the files specified as run options or through redirection operators are accessible to OSS, then OSSTTY is not launched. For example, the following command does not launch OSSTTY because Guardian EDIT files are accessible to OSS processes as input:

```
TACL> osh/ in $data.mysubvol.myfile/ -osstty
```

Once started, OSSTTY becomes the standard input, standard output, and standard error of the OSS process, and redirects input, output, and error data to the Guardian targets specified as run options or through redirection operators.

Examples of OSSTTY Usage

Following are some typical examples of OSSTTY usage.

Redirecting Input From a Guardian Process to an OSS Process

In this example, OSSTTY is used to direct input from `$vhs` (Guardian virtual hometerm subsystem) to an ftp client running in OSS.

```
TACL> run osstty /name $tty, in $vhs, nowait/  
TACL> osh  
$/usr/ucb/ftp </G/tty/#stdin
```

OSSTTY is started with its standard input set to `$vhs`. In the OSS environment, an ftp client is started, with redirection used to receive input from `/G/tty/#stdin`. Input received through `$vhs` is received by OSSTTY and redirected to `/G/tty/#stdin`. The `nowait` parameter is used to return control to the user immediately after OSSTTY is initiated.

The technique illustrated in this example could be used to create an OSS application that receives input interactively from the Guardian environment (through `$vhs`).

Redirecting Output From OSS to a Guardian EDIT file

The following example illustrates redirecting output from an OSS command to a Guardian EDIT file.

```
TACL> run osstty /name $tty, out editfil, nowait/  
TACL> osh -c "ls > /G/tty/#stdout"
```

The `run` command starts an OSSTTY process named `$tty`, with its standard output set to `editfil` (assume `editfil` is a Guardian EDIT file). The `osh` command starts an OSS shell and runs an `ls` command that redirects its output to `/G/tty/#stdout/`. OSSTTY receives the output and redirects it to its standard output, which is set to `EDITFIL`.

Redirecting Output From OSS to a Telserv Terminal

In the following example, output from an `ls` command is redirected, through OSSTTY, to a Telserv terminal named `$ztn0.#ptycasu1`.

```
TACL> run osstty /name $tty, out $ztn0.#ptycasu1, nowait/  
-noserver -idletime 0  
TACL> osh
```



```
$ ls -l > /G/tty/#stdout
```

The `-noserver` and `-idletime` parameters cause OSSTTY to terminate after the `ls` command completes.

Executing a Shell Script and Redirecting Output to a Guardian Process

The following example executes an OSS shell script and uses OSSTTY to direct the output to a Guardian process. The Guardian process is a logger process that sends all input data from its `$RECEIVE` file to an EDIT file.

The following command starts the shell script `/home/stu01/user_report` with output redirected to the standard output file.

```
TACL> osh -c "/home/stu01/user_report" > /G/tty/#stdout
```

The following command starts the Guardian logger process, named `logger`, with `logfile` as its output file.

```
TACL> run logger /name $log/ logfile
```

The following command starts OSSTTY, with the logger process specified for the `out` and `term` parameters. OSSTTY will redirect its standard output (`/G/tty/#stdout`) and standard error (`/G/tty/#stderr`) to the logger process.

```
TACL> run osstty /name $tty, out $log, term $log, nowait/
```

The logger process receives input through `$RECEIVE` and sends the data to the log file `logfile`.

Example Logger Program Used With OSSTTY shows the source code for the logger program.

Example Logger Program Used With OSSTTY

```
/* Logger Program used with OSSTTY */

#pragma runnable, symbol, nolist, inspect
/* This is a simple logger program. It does the following:

- Opens its $RECEIVE for interprocess communication.

- Creates an EDIT file that is provided as run-time argument
  during instantiation. It is used as a log file.

- Accepts an OPEN from an application and logs the data that
  is being written to it.
```

On the reception of the system CLOSE message, the process closes the log file and terminates.

Note: The program must be started with the log file name as an argument.

Do not post a READ on the started process.

The log file will be purged and recreated by the process.

Messages other than OPEN and CLOSE are not handled.

Operations are Waited. */

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <tal.h>
#include <cextdecs(FILE_OPEN_, OPENEDIT_, READUPDATEX, REPLY, \
    CLOSEALLEDIT, FILE_PURGE_, FILE_GETINFO_, WRITEEDIT)>

/* local macros and defines */

#define MAX_USRMSG 8*1024
#define RECV_QMAX          4047
#define FNAME32_LEN        64
#define O_WAITED_OPEN      0x00

/* public data */

short rcv_fd;
short U_data[4*1024];
short edit_fd ;
short rcv_cnt;

/* public functions */

void file_create(char* file_name);
short tty_info_fnum( short fd);
void dol_RECEIVE_opener(void);
short tty_open_file(char *name, short name_len, short nwio,
    short nwopen, short excl, short access, long sync,
    short *error);

/* private functions */

void user_message(void);
void system_message(void);

main(int argc, char** argv) {
    short ret_fd;
    _cc_status read_status ;
    short error = 0;

    /* Create the EDIT file */
    file_create(argv[1]);

    /* Open $RECEIVE for interprocess communication */
    dol_RECEIVE_opener();

    /* Exit only when close received from OSSTTY */
    for (;;)
    {
        /* Read from $RECEIVE in waited manner */
        read_status = READUPDATEX(rcv_fd,
                                (char *)U_data,
                                MAX_USRMSG,
                                &rcv_cnt);

        /* A CCG on $RECEIVE means that a system message has
           been received */
        if( _status_gt(read_status)) {

```

```

        error = tty_info_fnum (rcv_fd);
        if(error == 6)
            system_message ();
    }
    else
        user_message( );
}
}
/* Log the message from OSSTTY to the EDIT file */
void user_message(void) {
    short error = 0 ;
    error = WRITEEDIT(edit_fd, ,
                      (char *)U_data,
                      rcv_cnt);

    if (error) {
        printf("Unable to write to EDIT file\n");
        exit(0);
    }
    REPLY();
}
/* Service the Open and Close system message */
void system_message(void) {
    switch(U_data[0]) {
        case -103: /* Open message */
        {
            REPLY();
            break;
        }
        case -104: /* Close Message */
        {
            CLOSEALLEEDIT();
            REPLY();
            exit(0);
            break;
        }
        default:
            REPLY();
            break;
    }
}

/* Create the log file */
void file_create(char* file_name ) {
    short error = 0;
    FILE_PURGE_((char *)file_name,
                (short)strlen(file_name));
    error = OPENEDIT_((char *)file_name,
                     (short)strlen(file_name),
                     &edit_fd,
                     2/*Write Only*/,
                     0/* shared*/,
                     0,/*nowait depth*/
                     15);

    if (error) {
        printf("Unable to Created EDIT File\n");
        exit(0);
    }
}

```

```

}

short tty_info_fnum(short fd) {
    short error;
    (void) FILE_GETINFO_(fd, &error );
    return error;
}

/* Open $RECEIVE in waited manner for interprocess
communication */
void dol_RECEIVE_opener(void) {
    short error = 0;
    if (( rcv_fd = tty_open_file (" $RECEIVE", 8, 0,
                                O_WAITED_OPEN,
                                O/*Shared*/,
                                O/*Read Write*/,
                                RECV_QMAX, &error)) < 0 ) {
        printf("Can't open $RECEIVE\n");
    }
}

short tty_open_file (char *name, short name_len, short nwio, short nwopen,
                    short excl, short access, long sync, short *error)
{
    char fname [FNAME32_LEN + 1];
    short fd;
    short options = 0;

    strncpy (fname, name, name_len);
    fname[name_len]='\0';
    *error = FILE_OPEN_ (fname, name_len,
                        &fd, access, excl, nwio, (short) sync,
                        options);
    return fd;    /* fd = -1 if FILE_OPEN failed */
}

```

Running an Interactive OSS Session From a TACL Macro

This example shows a method for using OSSTTY to run multiple interactive OSS sessions from the Guardian environment. The example behaves similarly to the the Guardian Define Process facility and the TACL #INLINE facility. The example consists of a TACL program that starts an OSS shell, starts an OSSTTY process, submits OSS commands to the shell, and uses OSSTTY to redirect output, prompts, and error messages to the Guardian environment.

The program takes into account the fact that OSS shell commands issue input prompts through the standard error file. The program redirects the shell's standard error file to the standard output, then tests each line of output to determine whether it is a prompt or some other message.

Note the following about using this program:

- If you give the program a process name qualified with the system name, such as `\mysys.$sh`, the program fails with a TACL error.
- If #INFORMAT is set to TACL, you must escape TACL special characters like the vertical bar with a tilde: `~|`
- If you run a utility, such as `cat`, `SCF`, or `VPROC`, the "be" and "print" routines will hang.

The TACL program consists of the following procedures.



Procedure	Description
SH <i>\$name</i>	Starts a named OSSTTY process and an OSS shell process, setting up the necessary redirection.
UNSH <i>\$name</i>	Removes the specified previously defined OSSTTY/OSS shell environment.
TELL <i>\$name command-string</i>	Sends a single command to the specified previously defined OSSTTY/OSS shell environment. Also displays any output from a prior command.
PRINT <i>\$name</i>	Displays any output from the specified previously defined OSSTTY/OSS shell environment.
BE <i>\$name</i>	Enters interactive mode with the specified previously defined OSSTTY/OSS shell environment. The user can enter commands and receive responses repeatedly, until <Ctrl-y> is entered. Entering <Ctrl-y> causes the procedure to exit interactive mode, but leaves the OSSTTY/OSS shell environment intact.
RESET <i>\$name</i>	Informs a previously defined OSSTTY environment that there is an interactive prompt outstanding. (Used in the event that there is a prompt outstanding, but the procedures do not recognize the prompt.)

The following steps illustrate the use of these procedures:

1. load the TACL macro (required once per TACL session).

```
TACL> load /keep 1/ sh
```

2. Start an OSSTTY process named *\$n1* and an OSS shell that uses *\$n1*.

```
TACL> sh $n1
```

3. Send a `pwd` command to the shell and OSSTTY. There is no response because none is queued.

```
TACL> tell $n1 pwd
```

4. Send a `cd` command to the shell and OSSTTY. The queued response from the prior command is displayed.

```
TACL> tell $n1 cd/usr/bdir/webserver
/G/SYSTEM/ADIR
```

The TACL procedures are shown in **Running an Interactive OSS Session From the Guardian Environment**.

Running an Interactive OSS Session From the Guardian Environment

```
== TACL macro for an interactive OSS session from the Guardian environment
== file: sh
== Start an OSSTTY process and OSS shell
?section sh routine
#push pname
[#case [#argument /value pname/ processname /syntax/
        otherwise]
|1|
```

```

#set pname [#shiftstring /up/ [pname]]
[#case [#argument end
        otherwise]

|1|
  [#if [#processexists [pname]] |then|
    #output Process [pname] already exists -- terminating.
    #output
    #pop pname
    #return
  ] == end if

|otherwise|
  #output Extraneous input -- terminating.
  #output
  #pop pname
  #return
] == end case

|2|
  #output Expecting a nonexistent process name -- terminating.
  #output
  #pop pname
  #return
] == end case
#set pname [#shiftstring /down/ [pname]]
#chardel pname 1 for 1
[#push [pname]^in^buf [pname]^out^buf [pname]^prompt^buf [pname]^need^prompt
  [pname]^status stdin stdout
] == end push
#set stdin /G/[pname]/#stdin
#set stdout /G/[pname]/#stdout
sink [#newprocess $system.system.osstty /name ${pname}, nowait,
      inv [pname]^in^buf dynamic, outv [pname]^out^buf/
      -server]
sink [#newprocess $system.system.osh /status [pname]^status, nowait/
      -p /bin/sh < [stdin] > [stdout] 2> [stdout]]

sink [#wait [pname]^out^buf]
#set [pname]^prompt^buf [#lineget [pname]^out^buf 1]

#set [pname]^need^prompt -1
#pop pname stdin stdout

== Remove previously defined OSSTTY/OSS shell environment
?section unsh routine
#frame
#push pname prog name
[#case [#argument /value pname/ processname /syntax/
        otherwise]

|1|
  #set pname [#shiftstring /up/ [pname]]
  [#case [#argument end
          otherwise]

|1|
|otherwise|
  #output Extraneous input -- terminating.
  #output

```

```

        #unframe
        #return
    ] == end case

|2|
    #output Expecting a process name -- terminating.
    #output
    #unframe
    #return
] == end case
#set name [pname]
#chardel name 1 for 1
[#if not [#variableinfo /existence/ [name]^in^buf] |then|
    #output Nothing to get rid of.
    #output
    #unframe
    #return
] == end if
[#if [#processexists [pname]] |then|
    #set prog [#fileinfo /file/ [#processinfo /programfile/
                                [pname]]]

    [#if not [#match osstty [prog]] |then|
        #output [pname] is not an OSSTTY process.
        #output
        #unframe
        #return
    ] == end if

    [#if [#emptyv [name]^status] |then|
        sink [#stop [#variableinfo /process/ [name]^status]]
    ] == end if

    sink [#stop [pname]]
] == end if
#pop [name]^in^buf [name]^out^buf [name]^prompt^buf [name]^need^prompt
#pop [name]^status
#unframe

?section tell routine
#frame
#push pname out^line

#if [#argument /value pname/ processname]
#chardel pname 1 for 1
[#if not [#variableinfo /existence/ [pname]^in^buf] |then|
    #output Process not known.
    #output
    #unframe
    #return
] == end if

#set [pname]^out^buf
#append [pname]^in^buf [#rest]
sink [#wait [pname]^out^buf [pname]^status]
#set [pname]^need^prompt -1
print ${pname}

```



```

[#if not [#variableinfo /existence/ [pname]^status] |then|
    == print complained. Bail-out now
    #unframe
    #return
] == end if not variableinfo existence pname^status then

#unframe

== Display output from previously defined OSSTTY/OSS shell environment
?section print routine
#frame
#push pname out^line

#if [#argument /value pname/ processname]
#if [#argument end]
#chardel pname 1 for 1
[#if not [#variableinfo /existence/ [pname]^in^buf] |then|
    #output Process not known.
    #output
    #unframe
    #return
] == end if

[#loop |while| [[pname]^need^prompt] |do|
    #delay 1 == Get out of the way of the shell
    #set out^line [#extract [pname]^out^buf]
    [#if not [#emptyv [pname]^status] |then|
        #set [pname]^need^prompt 0
        #output Process $[pname] died ([[pname]^status]). Cleaning up.
        #output
        #outputv [pname]^out^buf
        unsh $[pname]
        #unframe
        #return
    ] == end if not emptyv pname^status then

    [#if [#match [[pname]^prompt^buf] [out^line]] |then|
        #set [pname]^need^prompt 0
    |else|
        #outputv out^line
    ] == end if
] == end loop
#unframe

== Enter interactive mode with previously defined OSSTTY/OSS
== shell environment
?section be routine
#frame
#push pname done in^line out^line

#if [#argument /value pname/ processname]
#chardel pname 1 for 1
[#if not [#variableinfo /existence/ [pname]^in^buf] |then|
    #output Process not known.
    #output
    #unframe
    #return
]

```



```

] == end if

#set done 0
[#loop |while| not [done] |do|
  #inputv in^line [pname]^prompt^buf
  [#if [#inputeof] |then|
    #set done -1
  |else|
    #set [pname]^out^buf
    #appendv [pname]^in^buf in^line
    #set [pname]^need^prompt -1
    sink [#wait [pname]^out^buf [pname]^status]

    print ${pname}

    [#if not [#variableinfo /existence/ [pname]^status] |then|
      == print complained that the process died. Bail-out now
      #unframe
      #return
    ] == end if not variableinfo /existence/ pname^status then

  ] == end if
] == end loop
#unframe

== Inform previously defined OSSTTY/OSS shell environment of
== prompt outstanding
?section reset routine
#frame
#push pname

#if [#argument /value pname/ processname]
#chardel pname 1 for 1
[#if not [#variableinfo /existence/ [pname]^in^buf] |then|
  #output Process not known.
  #output
  #unframe
  #return
] == end if
#set [pname]^need^prompt 0
#unframe

```

Printer I/O

The OSS API has no function that accesses printers or spoolers directly. You cannot write directly to printers with the OSS API. To print from an OSS program or get information about a print job, you access the Guardian spooler with the `lp` or `lpstat` utility.

You can, however, write OSS files and output directly to a printer or to the Guardian spooler using the Guardian API, as described later in this subsection.

Printing With the OSS API

There are no OSS functions that have direct access to a printer or printer spooler. To print an OSS file from a program, you can use the `popen()` function to call the `lp` or `lpstat` utility. For information about these utilities, refer to the



lp(1) and lpstat(1) reference pages either online or in the *Open System Services Shell and Utilities Reference Manual*, or refer to the *Open System Services User's Guide*.

Printing an OSS File With the lp Utility shows how to print to the default printer with a program. It generates text that is output to the default printer using the `popen()` function. The contents of the specified input pathname is printed on the default line printer with its name on the banner page.

Printing an OSS File With the lp Utility

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

FILE *fpin, *fpout;
#define MAXLINE 132
char lpbuf[MAXLINE];
char line[MAXLINE];

int main (int argc, char *argv[]) {

    if(argc < 2) {
        fprintf(stderr, "Usage: lpopen <pathname>\n");
        exit(1);
    }
    /* Open the input file. */
    if((fpin = fopen(argv[1], "r")) == NULL) {
        fprintf(stderr, "Can't open %s\n", argv[1]);
        exit(1);
    }
    /* Create a string for popen() with the name of the file to be
       printed in the title. */
    sprintf(lpbuf, "lp -t %s", argv[1]);

    /* Open the line printer program. */
    if((fpout = popen(lpbuf, "w")) == NULL) {
        fprintf(stderr, "popen error on %s\n", lpbuf);
        exit(1);
    }
    /* Copy the contents of the file to the line printer filter. */
    while(fgets(line, MAXLINE, fpin) != NULL) {
        if(fputs(line, fpout) == EOF) {
            fprintf(stderr, "fputs error to pipe\n");
            exit(1);
        }
    }
    /* Check for errors on input file processing. */
    if(ferror(fpin)) {
        fprintf(stderr, "fgets error\n");
        exit(1);
    }
    /* Check for errors in execution of lp command. */
    if(pclose(fpout) == -1) {
        fprintf(stderr, "pclose error\n");
        exit(1);
    }
    printf("File %s output to line printer\n", argv[1]);
}
```



```

    exit(0);
}

```

Printing OSS Files With the Guardian API

You can send OSS files and output to the Guardian spooler or send them directly to a printer using Guardian procedures.

Sending an OSS File to the Spooler

You can send OSS files and output to the Guardian print spooler and take advantage of the spooler features. For example, your program can examine the queues of available printers and move a job to a printer whose queue becomes unexpectedly shorter because of a job cancellation, or away from a printer that is experiencing delays. You can also access spoolers that are on other systems than the file you want to print.

Sending an OSS File to the Guardian Spooler shows how to send an OSS file to the spooler and how to control the spooler from an OSS program using Guardian spooler procedures. The printer is opened using `FILE_OPEN_`, then `SPOOLSTART` is called to establish a level-3 spooling session using a level-3 buffer to compress and block data. The `SPOOLCONTROL` procedure is used to send control codes to the level-3 buffer, and the `SPOOLWRITE` procedure is used to compress and block data in the level-3 buffer. When the level-3 buffer is full, the buffer is written to the collector. The `SPOOLEND` procedure is used to write any remaining data from the level-3 buffer and complete the job being spooled at level 3. The printer name is specified in Guardian filename format (for example, `$s.#default`), because printer names are inaccessible using the OSS pathname format.

For information about the `SPOOLCONTROL`, `SPOOLEND`, `SPOOLSTART`, and `SPOOLWRITE` procedures, as well as other spooler procedures, refer to the *Spooler Programmer's Guide*.

Sending an OSS File to the Guardian Spooler

```

/* Example:  Sending an OSS file to the Guardian Spooler      */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <cextdecs.h(FILE_OPEN_,FILE_CLOSE_,\
                    SPOOLSTART, SPOOLCONTROL,\
                    SPOOLWRITE,SPOOLEND)>

#define MAXLINE 132
short filenum;      /* file code number */
short retcode;      /* procedure call return code */
char *printer;      /* name of printer */
char *pathname;     /* name of file to print */
short size;         /* number of bytes used in buffer */
FILE *infile;       /* input file to be printed */

#ifdef _TNS_E_TARGET || _TNS_R_TARGET
    char buffer[MAXLINE]; /* buffer for print line */
#else
    char _lowmem buffer[MAXLINE];
#endif

short l3buffer[512]; /* level-3 buffer for spooler,
                    note that it is not lowmem, even in CISC */

int main(int argc, char *argv[]) {
    /* Get the Guardian printer filename and the name of the file to print.*/
    if (argc > 1)

```

```

    printer = argv[1];
else
    printer = "$s.#default";

if (argc > 2)
    pathname = argv[2];
else
    pathname = __FILE__;

if((infile = fopen(pathname, "r")) == 0) {
    perror("Can't open file");
    printf("File name is %s\n", pathname);
    exit(1);
}

retcode = FILE_OPEN_(printer, /* name of Guardian printer */
                    (short)strlen(printer), /* printer name length */
                    &filenum, /* file number returned */
                    2, /* open for write access */
                    1, /* open for exclusive use */
                    );

/* Check if open was successful. */
if (retcode != 0) {
    fprintf(stderr, "Can't open %s\n", printer);
    fprintf(stderr, "retcode = %d\n", retcode);
    exit(1);
}

/* Call SPOOLSTART to specify the job's attributes. */
/* in NMC, the "16-bit" and "32-bit" level-3 buffers are equivalent
   Using the "32-bit" version makes the code work better with CISC C */
retcode = SPOOLSTART(
    filenum, /* file number of spooler job file */
    , /* old 16-bit level-3 spooler buffer */
    , /* stick with default location */
    , /* form name for the job */
    , /* report name for the job */
    , /* number of copies */
    , /* page size (in lines) */
    047, /* flag - attributes of job */
    /* 040 - hold after bit on */
    /* 007 - job priority = 7 */
    , /* owner of job */
    , /* maximum number of lines allowed for job */
    , /* maximum number of pages allowed for job */
    , /* filename of spooler collector */
    , /* alternative parameter to collector filenum */
    l3buffer /* 32 bit pointer to l3 buffer */
);

if (retcode != 0) {
    fprintf(stderr, "SPOOLSTART failed, retcode = %d\n", retcode);
    exit(1);
}

/* Position the paper to top of form. */
retcode = SPOOLCONTROL( /* send control code to spooler */
    , /* 16-bit version of level-3 buffer */
    1, /* forms control mode */
    0, /* top of form control code */

```

```

        &size, /* number of bytes used in level-3 buffer*/
        l3buffer /* 32-bit version of level-3 buffer */
    );
if (retcode != 0) {
    fprintf(stderr, "SPOOLCONTROL failed on printer: %s\n", printer);
    fprintf(stderr, "retcode = %d\n", retcode);
    exit(1);
}

/* Read input file, one line at a time, and send to spooler. */
while(fgets(buffer, MAXLINE, infile)) { /* get line from input file */
    retcode =
        SPOOLWRITE(, /* 16-bit version of level-3 buffer */
#ifdef _TNS_E_TARGET || _TNS_R_TARGET
            (short *)buffer,
#else
            (short _near *)buffer, /* print line */
#endif
            (short)strlen(buffer), /* size of print line */
            &size, /* number of bytes used in level-3 buffer */
            l3buffer /* data is compressed and blocked
                        into the level-3 buffer */
        );

    /* Check for successful spooler write operation. */
    if (retcode != 0) {
        fprintf(stderr, "SPOOLWRITE failed: %s\n", printer);
        fprintf(stderr, "retcode = %d\n", retcode);
        exit(1);
    }
}

/* Position the paper to top of form again. */
retcode = SPOOLCONTROL( /* send control code to spooler */
    , /* 16-bit level-3 buffer */
    1, /* forms control mode */
    0, /* top of form control code */
    &size, /* number of bytes used in level-3 buffer*/
    l3buffer /* 32-bit level-3 buffer */
);
if (retcode != 0) {
    fprintf(stderr, "SPOOLCONTROL failed: %s\n", printer);
    fprintf(stderr, "retcode = %d\n", retcode);
    exit(1);
}

retcode = SPOOLEND( /* complete the spool job */
    , /* 16-bit level-3 buffer */
    0104, /* flag - attributes of job */
        /* 0100 - hold bit on */
        /* 0004 - job priority = 4 */
    l3buffer
);
if (retcode != 0) {
    fprintf(stderr, "SPOOLEND error %d\n", retcode);
    exit(1);
}

```

```

fclose(infile);          /* close input file */
FILE_CLOSE_(filenum);    /* close printer file */
printf("File %s spooled to %s printer\n", pathname, printer);
exit(0);

return 0; /* keep compiler happy */
}

```

Sending an OSS File Directly to a Printer

Your application can send an OSS file or output directly to a printer.

Sending an OSS File Directly to a Printer Using the Guardian API shows how to send an OSS file directly to a printer. You must specify the printer using the Guardian filename format; you cannot access a printer by using its name in the /G directory. For further information on working with printers directly, refer to the *Guardian Programmer's Guide*. In this example, the printer is opened using the Guardian FILE_OPEN_ procedure, control messages are sent using the CONTROL procedure, and data is written using the WRITEX procedure. This example uses the escape sequences for a 5515/5516/5518 printer.

Control codes are dependent on the printer. They are listed in the *Guardian Programmer's Guide* and in manuals for the printer you are using.

Sending an OSS File Directly to a Printer Using the Guardian API

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <tal.h>
#include <cextdecs.h(FILE_OPEN_, CONTROL, WRITEX, FILE_CLOSE_)>

short filenum;          /* file code number */
short retcode;          /* procedure call return code */
_cc_status CC;          /* condition code */
char *printer;          /* name of printer */
char *pathname;         /* name of file to print */
char buffer[134];       /* buffer for print line */
FILE *infile;           /* input file to be printed */

int main(int argc, char *argv[]) {

/* Get the Guardian printer filename and the name of the file to print.*/
if (argc > 1)
    printer = argv[1];
else
    printer = "$s.#default";
if (argc > 2)
    pathname = argv[2];
else
    pathname = "/usr/include/stdio.h";
if((infile = fopen(pathname, "r")) == 0) {
    perror("Can't open file");
    exit(1);
}
retcode = FILE_OPEN_(printer,          /* name of Guardian printer */
    (short)strlen(printer),            /* printer name length */
    &filenum,                          /* file number returned */
    2,                                /* open for write access */

```

```

        1,                      /* open for exclusive use */
    );
/* Check if open was successful. */
if (retcode != 0) {
    fprintf(stderr, "Can't open %s\n", printer);
    fprintf(stderr, "retcode = %d\n", retcode);
    exit(1);
}

/* Position the paper to top of form. */
CC = CONTROL(filenum,          /* send control code to printer */
    1,                        /* forms control mode */
    0                         /* top of form control code */
);
if (_status_ne(CC)) {
    fprintf(stderr, "Illegal control code to printer: %s\n", \
        printer);
    exit(1);
}

/* Read the input file, one line at a time, and send it to printer. */
while(fgets(buffer, 132, infile)) { /* get line from input file */
    CC = WRITEEX(filenum,          /* write buffer to printer */
        buffer,                    /* printer buffer */
        (short)strlen(buffer)      /* size of text string */
    );
/* Check for successful printer write operation. */
    if (_status_ne(CC)) {
        fprintf(stderr, "Write to printer failed: %s\n", printer);
        exit(1);
    }
}

/* Position the paper to top of form again. */
CC = CONTROL(filenum,          /* send control code to printer */
    1,                        /* forms control mode */
    0                         /* top of form control code */
);
if (_status_ne(CC)) {
    fprintf(stderr, "Illegal control code to printer: %s\n", \
        printer);
    exit(1);
}
fclose(infile);                /* close input file */
FILE_CLOSE_(filenum);          /* close printer file */
exit(0);
}

```

OSS I/O Management Functions

The following table displays information about each OSS function that you can use to manage I/O. The columns of the table contain the following:

OSS Function

The name of the function and a brief description.



OSS Notes

Notes about the OSS implementation of the function. The note “Can return extended errors” means that the function can return errors that are Hewlett Packard Enterprise extensions to the XPG4 specification. Refer to the function’s reference page for further information.

Guardian Notes

Notes about using the function from a Guardian process.

The information in this table is only a summary of the information in the reference page for a function. For details, refer to the reference pages either online or in the *Open System Services Library Calls Reference Manual*.

NOTE: To determine which RVUs support an OSS function, see [Documented OSS Functions](#).

Table 26: OSS I/O Functions

OSS Function	OSS Notes	Guardian Notes
<code>cfgetispeed()</code> Gets the input baud rate for a terminal.		
<code>cfgetospeed()</code> Gets the output baud rate for a terminal.		
<code>cfsetispeed()</code> Sets the input baud rate for a terminal.	Does not affect baud rate.	
<code>cfsetospeed()</code> Sets the output baud rate for a terminal.	Does not affect baud rate.	
<code>ctermid()</code> Generates the pathname for a controlling terminal.	If the parameter is a null pointer, the string is stored in an internal static area and the address is returned. The next call to this function overwrites the contents of the internal static area. Can return extended errors.	For Guardian TNS processes, it returns an empty string and <code>errno</code> is set to <code>ENOTOSS</code> .
<code>fpathconf()</code> Retrieves file implementation characteristics.	Can return extended errors.	
<code>isatty()</code> Tests for a terminal.	Can return extended errors.	
<code>pathconf()</code> Retrieves file implementation characteristics.	Can return extended errors.	

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>tcdrain()</code> Waits for output to complete.	Can return extended errors.	
<code>tcflow()</code> Performs flow control functions.	Can return extended errors.	
<code>tcflush()</code> Flushes nontransmitted output data and nonread input data.	Can return extended errors.	
<code>tcgetattr()</code> Gets the attributes associated with a terminal.	Can return extended errors.	
<code>tcsendbreak()</code> Sends a “break” to an asynchronous serial data line.	Can return extended errors.	
<code>tcsetattr()</code> Sets the attributes associated with a terminal.	Can return extended errors.	
<code>ttynam()</code> Gets the name of a terminal.	Can return extended errors.	

Guardian I/O Management Procedures

The following table displays information about each Guardian procedure that you can use to manage I/O. Procedures that have been superseded by other procedures are not shown. Spooler procedures are not shown; for information about these, see the *Spooler Programmer's Guide*. The columns of the table contain the following:

Guardian Procedure

The name of the procedure and a brief description.

OSS Notes

Additional considerations or restrictions when the Guardian procedure is used on OSS processes.

For details about a procedure, refer to the *Guardian Procedure Calls Reference Manual*.



Table 27: Guardian I/O Procedures

Guardian Procedure	OSS Notes
CHECKSETMODE Allows the primary process of a process pair to propagate SETMODE operations to the backup process of the pair.	
CONTROL Performs device-dependent I/O operations.	
CONTROLBUF Performs device-dependent I/O operations that require a data buffer.	
DEVICE_GETINFOBYLDEV_ Obtains the logical and physical attributes of a device.	
FILE_CONTROL64_ Performs device-dependent I/O operations.	Beginning with the H06.24 and J06.13 RVUs.
FILE_CONTROLBUF64_ Performs device-dependent I/O operations requiring a data buffer.	Beginning with the H06.24 and J06.13 RVUs.
FILE_SETMODENOWAIT64_ Set device-dependent functions in a nowait manner on nowait files.	Beginning with the H06.24 and J06.13 RVUs.
LABELEDTAPESUPPORT Provides a way for nonprivileged programs to determine whether labeled-tape processing is enabled on the system.	
SETMODE Sets device-dependent procedures.	
SETMODENOWAIT Sets device-dependent procedures in a nowait manner on nowait files.	

Using Logging Mechanisms

This section describes how to log information in the OSS environment.

You can log information in two ways in the OSS environment:

- With the OSS logging functions
- With Event Management Service (EMS) procedures

Use the OSS logging functions in portable applications. These functions enable you to use the function calls of the Berkeley Software Distribution (BSD) UNIX logging mechanism to send log messages. However, rather than sending log messages to a logging demon, the OSS logging mechanism sends log messages to EMS.

EMS is a collection of Guardian processes, tools, and interfaces that collects and distributes messages. EMS relays OSS log messages to destinations such as the system operator's message terminal (which is somewhat like a UNIX console). The destinations of the log messages are determined by the manager of the system that your application is running on and are set up using EMS.

Use EMS procedures in OSS programs that are written specifically for the OSS environment. EMS procedures enable you to provide more concise and meaningful events than would be produced using the OSS logging functions. You use EMS by calling Guardian procedures. EMS procedures are not discussed in this guide. For information about using EMS, refer to the *EMS Manual*.

OSS processes can receive EMS messages by starting a consumer distributor process in the same way a Guardian process can. An OSS process must use Guardian file-system and process-control procedures to start and communicate with the consumer distributor.

This section discusses the following topics:

Topic	Content
<u>Common and Unique Characteristics</u>	Summary of the differences between OSS and UNIX logging mechanisms.
<u>Logging Interoperability</u>	The processes in which you can use OSS logging functions.
<u>How to Log Information With the OSS API</u>	How to use the OSS logging functions.
<u>OSS Logging-Mechanism Functions</u>	List of each OSS logging mechanism function with notes about its OSS implementation and use on Guardian processes.

Common and Unique Characteristics

The behavior of the OSS logging functions are the same as those of the BSD UNIX logging functions, but the ways in which the BSD and OSS functions perform their tasks are different.

Features Common to UNIX and OSS Logging

The OSS logging functions are the same as the BSD logging functions, but they might be somewhat different from message-logging functions in other varieties of the UNIX operating system.

The OSS API provides the logging functions shown in the following table.



Table 28: OSS Logging Functions

Function	Description
<code>closelog()</code>	Closes the connection to the collector
<code>openlog()</code>	Initializes logging parameters
<code>setlogmask()</code>	Sets the EMS event log mask
<code>syslog()</code>	Logs messages

These functions are described in `closelog(3)`, `openlog(3)`, `setlogmask(3)`, and `syslog(3)` reference pages, which are available either online or in the *Open System Services Library Calls Reference Manual*.

Features Unique to OSS Logging

The OSS logging mechanism sends event messages to an EMS collector process rather than to a `syslogd` process as used on some versions of the UNIX operating system. A collector process gathers EMS events and their associated tokens and sends them to destinations determined by the system manager. One of the EMS tokens contains message text.

Authorization, demon, kernel, and mail facilities are in the Guardian environment and do not generate OSS events or messages.

Some priorities, facilities, and options used in logging on BSD versions of the UNIX operating system have no effect in the OSS environment. For example, the `LOG_NOWAIT` option of the `logopt` parameter of the `openlog()` function has no effect.

A child process uses the collector that the parent process passes to it in the `EMS_COLLECTOR` environment variable.

For detailed information about event messages, the kinds of events to log, and how messages are routed, refer to the *EMS Manual*.

Logging Interoperability

You can call OSS logging functions from a Guardian module that is part of an OSS process. You cannot use the `LOG_PID` option of the `openlog()` function on a Guardian process.

OSS processes have the same access to EMS procedures as do Guardian processes. Both OSS processes and Guardian processes can:

- Call any of the EMS procedures for constructing events
- Open collectors and send them events using the `WRITEREAD` or `WRITEREADX` procedures
- Start a consumer distributor and receive EMS event messages

An OSS process must use Guardian file-system and process-control procedures to start and communicate with the consumer distributor. For further information about using EMS, refer to the *EMS Manual*.

How to Log Information With the OSS API

You log information by using the OSS `syslog()` function. If you want to use the default settings, simply call the `syslog()` function.



You can customize logging by performing one or both of the following actions before you call `syslog()`:

- Call the `setlogmask()` function to specify the kinds of events to be logged.
- Call the `openlog()` function to initialize logging parameters.

You can explicitly close the EMS collector process you are using with the `closelog()` function.

The OSS logging functions use the parameters and default parameter values shown in the following table. The parameters and default values are described in detail later in this section.

Table 29: OSS Logging Function Parameters and Default Values

Parameter	of Function	Default Value	Description of Default Value
<i>facility</i>	<code>openlog()</code>	<code>LOG_USER</code>	Logs messages generated by user processes and displays the facility string <code>USER</code> .
<i>ident</i>	<code>openlog()</code>	<code>"syslog"</code>	Causes the identity string "syslog" to appear in messages.
<i>level</i>	<code>syslog()</code>	<code>LOG_INFO</code>	Assigns the level <code>LOG_INFO</code> to the event and displays the level string <code>INFO</code> .
<i>logopt</i>	<code>openlog()</code>	<code>LOG_ODELAY</code>	Opens a collector process only when the first message is logged.
<i>maskpri</i>	<code>setlogmask()</code>	<code>LOG_UPTO (LOG_DEBUG)</code>	Logs events of all priorities.
<i>message</i>	<code>syslog()</code>		Points to the display format and text that you supply.

The values of these parameters appear in an OSS logging message in the following order:

facility level[,PID = pid] (identity) : message-text

The content of *message-text* is pointed to by the *message* parameter.

The values of the *logopt* and *maskpri* parameters do not appear in OSS log messages.

The OSS process ID appears in the message if you specify the `LOG_PID` option in the *logopt* parameter of the `openlog()` function, as described in [logopt Parameter](#).



Specifying a Collector

A user of your application can specify an EMS collector process from the shell by setting the OSS environment variable `EMS_COLLECTOR` to the desired collector. You must export the variable before using the collector. You use the `export` command to both set the value of the variable and export it as follows:

```
export EMS_COLLECTOR=\$COL
```

In this example, the backslash (`\`) is required to prevent the shell from interpreting `$COL` as a shell variable, and `$COL` represents the name of the collector.

You can have event messages sent to `$0`, the local primary collector, if, for some reason a message cannot be sent to the collector specified by a user with the `EMS_COLLECTOR` environment variable. To send events to `$0` as a backup, specify the `LOG_CONS` option in the `logopt` parameter of the `openlog()` function, as described in [logopt Parameter](#).

Specifying the Severity Level of Logged Events

You can specify the severity level of events you want logged by setting the `maskpri` parameter of the `setlogmask()` function. For example, setting the `maskpri` parameter to `LOG_WARNING` causes only events of severity `LOG_WARNING` to be logged.

You can use the `LOG_UPTO` operator in the `maskpri` parameter to select the level through which events are logged. For example, setting `maskpri` to `LOG_UPTO (LOG_INFO)` causes all events except those of `LOG_DEBUG` to be logged. There is no default value for `maskpri`, so you must set this value yourself.

The following table lists the names and event numbers of the `level` parameter in descending order of severity. The `level` parameter is also displayed as a string in the message, and this string is also listed in the following table.

Table 30: Logging Level Event Numbers and Strings

Level	EMS Event Number	Message String
LOG_EMERG	5	EMERGENCY (most severe)
LOG_ALERT	6	ALERT
LOG_CRIT	7	CRITICAL
LOG_ERR	8	ERROR
LOG_WARNING	9	WARNING
LOG_NOTICE	10	NOTICE
LOG_INFO	11	INFO
LOG_DEBUG	12	DEBUG (least severe)



Initializing Logging Parameters

The `openlog()` function initializes the Event Management Service (EMS) event log parameters that are used by the OSS logging mechanism. It enables you to change the values of the parameters listed in the following table if you do not want to use the default values.

Table 31: `openlog()` Parameters Used by OSS Logging Mechanism

Parameter	Description
<i>facility</i>	The part of the system you specify as having generated the event message
<i>ident</i>	An event subject string that appears in the message
<i>logopt</i>	Logging options

facility Parameter

The *facility* parameter enables you to specify the part of the system that generates the event message. The default facility is `LOG_USER`; this is the only facility that has meaning in the OSS environment. Although authorization, demon, kernel, and mail facilities are in the Guardian environment and do not generate OSS events or messages, you can use the `openlog()` function to specify any facility.

The `openlog()` function changes the default facility only if the *facility* parameter is nonzero. The value of `LOG_KERN` is 0, so to change the facility to `LOG_KERN`, change its value to 1 using the exclusive-or operator, as shown in the following example:

```
openlog(LOG_KERN | 1);
```

The *facility* parameter is also displayed as a string in the OSS log message. The facilities and their strings are listed in the following table:

Table 32: OSS Logging Facilities

Facility	Message String	Description
<code>LOG_AUTH</code>	<code>AUTH</code>	Messages generated by the <code>login</code> , <code>su</code> , and other shell commands or utilities belonging to a user authorization system.
<code>LOG_CRON</code>	<code>CRON</code>	Messages generated by a clock demon.
<code>LOG_DAEMON</code>	<code>DAEMON</code>	Messages generated by system demons.
<code>LOG_KERN</code>	<code>KERNEL</code>	Messages generated by the kernel.
<code>LOG_LOCAL[0-7]</code>	<code>LOCAL[0-7]</code>	Messages generated by local facilities. This option is reserved for local site use.
<code>LOG_LPR</code>	<code>LPR</code>	Messages generated by a line printer subsystem.

Table Continued



Facility	Message String	Description
LOG_MAIL	MAIL	Messages generated by a mail subsystem.
LOG_NEWS	NEWS	Messages generated by a network news subsystem.
LOG_SYSLOG	SYSLOG	Messages generated by a syslogd demon.
LOG_USER	USER	Messages generated by user processes. This is the default value.
LOG_UUCP	UUCP	Messages generated by a UNIX-to-UNIX copy program (UUCP) subsystem.

identity Parameter

The *identity* parameter enables you to specify an event subject string to appear in the OSS log message. The default subject string is “syslog”.

logopt Parameter

The *logopt* parameter enables you to specify logging options. You can specify any set of options. To specify more than one option, separate the options with the OR operator (|), as in the following example:

```
openlog(LOG_CONS | LOG_PID);
```

The default value of *logopt* is LOG_ODELAY, which waits until the first message is logged before opening a collector. The following table lists the logging options.

Table 33: OSS Logging Options

Option	Description
LOG_CONS	Sends the event message to the local primary collector, \$0, if the message cannot be sent to the collector specified by the EMS_COLLECTOR environment variable
LOG_NDELAY	Opens the connection to the collector process immediately
LOG_NOWAIT	Has no effect
LOG_ODELAY	Waits until the first message is logged before opening the connection to the collector process
LOG_PERROR	Has no effect
LOG_PID	Includes the OSS process ID of the calling process in each event message

If you specify both LOG_NDELAY and LOG_ODELAY, LOG_NDELAY takes effect.



NOTE: Do not use the LOG_PID option on a Guardian process. If you do, the `syslog()` function will call the `openlog()` function and fail with Guardian trap 5, because Guardian processes have no OSS process ID.

Logging a Message

You call the `syslog()` function whenever you want to log a message from your program. For suggestions about the kinds of events and messages to log, refer to the *EMS Manual*.

You can use the formatting codes that `printf()` recognizes in a `syslog()` call. **Calling syslog() With Formatting Codes** shows a code fragment with a call to `syslog()` using these codes.

Calling syslog() With Formatting Codes

```
fahr = 50;

syslog( LOG_DEBUG
        , "%4d Fahrenheit -- %6.1f Centigrade "
        , fahr
        , (5.0/9.0)*(fahr - 32 ) );
```

This call produces the following message:

```
USER DEBUG : 50 Fahrenheit -- 10.0 Centigrade
```

You can use the `%m` formatting code to include the current error number. **Calling syslog() With the %m Formatting Code** shows a code fragment with a call to `syslog()` using `%m`.

Calling syslog() With the %m Formatting Code

```
len = read( master_fd , buf, MAX_READ_SIZE );
if ( len < 0 )
{
    syslog( LOG_DEBUG, "READ Error %m on Master File" );
}
```

This call produces a message such as the following if there is an error:

```
USER DEBUG : READ Error 4209 on Master File
```

where 4209 is an OSS file-system error number. You can obtain the related message string with the `perror()` or `strerror()` functions.

Use of System Logging Functions provides a complete program showing the use of the system logging functions in an OSS program.

Use of System Logging Functions

```
/* Using System logging functions */

#include <fcntl.h>
#include <syslog.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    char *ident="Test X"; /* Identity string, "syslog" by default */
    int logopt=LOG_CONS|LOG_PID|LOG_ODELAY; /* Logging options */
    int facility=LOG_USER; /* Part of system generating event message */
    int maskpri=LOG_UPTO(LOG_INFO); /* EMS event log mask */
    char buf[512];
    int fd;
```

```

/* Open the system log with the appropriate options
   - identity string
   - logging option
   - generated by user program by default
*/
openlog(ident, logopt, facility);

/* Set the Event Management Service (EMS) event log mask */
setlogmask(maskpri);

/* Write a "Daemon X started" message. */
syslog(LOG_INFO, "Daemon X started.\n");
/* Make sure the correct number of arguments are entered
   in starting up the daemon.
   If not, print a usage message and exit. */

if(argc < 2) {
    syslog(LOG_ERR, "Usage: syslog pathname\n");
    exit (1);
}
/* If the file cannot be opened, log an error message and exit. */
if((fd = open(argv[1], O_RDONLY)) < 0) {
    syslog(LOG_ERR, "Can't open %s\n", argv[1]);
    exit(1);
}
/* If the file is not at least 512 bytes long, log a read error. */
if(read(fd, buf, 512) != 512) {
    syslog(LOG_ERR, "Read error %m on key daemon file.\n");
    exit(1);
}
close(fd);
/* Write a "Daemon X stopped" message. */
syslog(LOG_INFO, "Daemon X stopped.\n");
/* Close connection to EMS collector process */
closelog();
exit(0);
}

```

Closing a Collector

To close the connection to the EMS collector process opened by `openlog()` or `syslog()`, call the `closelog()` function.

Logging Parameters and EMS Tokens

OSS logging function parameters correspond to the EMS tokens listed on the `syslog(3)` reference page, which is available either online or in the *Open System Services Library Calls Reference Manual*. The values of the tokens can be changed by the functions described in this section, but the tokens themselves cannot be changed with the OSS API.

The following table lists logging function parameters with their corresponding EMS tokens.



Table 34: OSS Logging Function Parameters and Corresponding EMS Tokens

Parameter	Token
<i>facility</i>	ZOSS-TKN-FACILITY
<i>identity</i>	ZOSS-TKN-IDENT
<i>level</i>	ZOSS-TKN-LEVEL
<i>message</i>	ZEMS-TKN-TEXT

The OSS process ID value displayed in log messages corresponds to the value of the ZOSS-TKN-PID token. This parameter is included in the event message only if LOG_PID is specified to `openlog()`.

OSS Logging-Mechanism Functions

The following table displays information about each OSS function that you can use to manage logging. The columns of the table contain the following:

OSS Function

The name of the function and a brief description.

OSS Notes

Notes about the OSS implementation of the function.

Guardian Notes

Notes about using the function on a Guardian process.

The information in this table is only a summary of the information in the reference page for a function. For details, refer to the reference pages either online or in the *Open System Services System Calls Reference Manual* and the *Open System Services Library Calls Reference Manual*.

Table 35: OSS Logging-Mechanism Functions

OSS Function	OSS Notes	Guardian Notes
<code>closelog()</code> Closes the collector process that was opened by <code>openlog()</code> or <code>syslog()</code> .		
<code>openlog()</code> Initializes logging parameters.		If you specify the LOG_PID option on a Guardian process, the <code>syslog()</code> function fails with Guardian trap 5.

Table Continued



OSS Function	OSS Notes	Guardian Notes
<code>perror()</code> Writes a message explaining the last error encountered by a library function.		
<code>setlogmask()</code> Sets the EMS event log mask.		
<code>strerror()</code> Accesses a message explaining the function error.	Upon successful completion, this function returns a pointer to the generated message string. If the error number is not valid, <code>errno</code> is set to the <code>EINVAL</code> value.	
<code>syslog()</code> Logs messages.		If you specify <code>thLOG_PID</code> option with the <code>openlog()</code> function on a Guardian process, the <code>syslog()</code> function fails with Guardian trap 5.

Using Subsystems From OSS

There are Hewlett Packard Enterprise subsystems that you can access from OSS programs. This section summarizes considerations or limitations in using Hewlett Packard Enterprise subsystems from your OSS application programs. The section also provides references to manuals that discuss these subsystems in detail.

The following subsections discuss programmatic access from an OSS application to the corresponding subsystems:

- **Communications Subsystems**
- **Subsystems Supporting the Subsystem Programmatic Interface (SPI)**
- **NonStop SQL/MP**
- **NonStop SQL/MX**
- **NonStop TS/MP**
- **NonStop TMF**
- **NonStop Tuxedo System**
- **NonStop Distributed Computing Environment (DCE)**
- **Simple Network Management Protocol (SNMP)**

Communications Subsystems

A communications subsystem provides users with access to a set of communications services. It consists of one or more processes running on a system and possibly a protocol module running on a communications controller.

Examples of Hewlett Packard Enterprise communications subsystems include Transmission Control Protocol/Internet Protocol (TCP/IP), Tandem LAN Access Method (TLAM), X.25 Access Method (X25AM), SNAX Extended Facility (SNAX/XF), Open Systems Interconnection/Application Services (OSI/AS), and Open Systems Interconnection/Message Handling System (OSI/MHS).

You can access Hewlett Packard Enterprise communications subsystems from OSS applications, but for some operations you must use Guardian I/O procedures. You will probably want to use `nowait` I/O, which allows a program to continue executing in parallel with read or write operations. `Nowait` I/O is not available using OSS I/O functions. You also need to read the `$RECEIVE` file if the subsystem sends status messages to `$RECEIVE`. Reading the `$RECEIVE` file requires using Guardian procedures.

Because you have to use Guardian procedures for significant portions of your program, it is easier if you use Guardian I/O procedures exclusively to access communications subsystems instead of mixing Guardian I/O procedures and OSS I/O functions.

For more information about Hewlett Packard Enterprise communications subsystems, see the manual sets that describe the subsystems you need to access. These manual sets can include introductory, configuration, programming, and reference manuals.

Subsystems Supporting the Subsystem Programmatic Interface (SPI)

The Subsystem Programmatic Interface (SPI) provides a common interface for the management of Hewlett Packard Enterprise subsystems by site-written applications. SPI provides the path for management applications to exchange command-specific messages and event messages with most subsystems and with many modules of the Subsystem Control Facility (SCF).



SPI is available to C and COBOL programs. The primary requirement is to include the definitions found in the appropriate Guardian environment header file (in the case of C programs, `$$SYSTEM.ZSPIDEF.ZSPIC`) when compiling the application. This requirement can be met by creating a copy of the needed file in the OSS file system and including a reference to the file in the form appropriate to your programming language; you can even partially automate that action by including an entry for the action in a `make` file. For example, the following `make` file entry:

```
zspi.h: /G/system/zspidef/zspic
      cp /G/system/zspidef/zspic ./zspi.h
```

creates a dependency on the presence of the necessary file in the user's local directory and makes sure that any updates made to the Guardian copy of the file also appear in the copy used to compile the application. Adding an `include` statement to the C program source code completes the task of providing access to the SPI features.

Alternatively, you could request that your system administrators create a `zspidef` directory within `/usr/include` and copy the necessary files from the Guardian file system into that directory each time updates are installed on the system.

For more information about SPI, refer to the *SPI Programming Manual* and the *SPI Common Extensions Manual*.

NonStop SQL/MP

The NonStop SQL/MP relational database management system uses the standard Structured Query Language (SQL) to describe and manipulate data. SQL/MP can be used for online transaction processing, decision support systems, scalable electronic commerce applications, and other business applications.

You can access SQL databases from an OSS program as you do from a Guardian program, using embedded SQL statements. You can also get information about SQL objects, catalogs, and programs using Guardian procedure calls. However, in the Guardian environment, you must use several products to compile, link, and optimize a program. In the OSS environment, you can use the `c89` (but not the `c99`) utility to perform all of these functions in a single `c89` command.

You do need to invoke the SQL compiler as well as the C compiler in the `c89` command, and the SQL compiler must be version 315 or newer. For information about the `c89` utility, see the `c89 (1)` reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.

For more information about writing OSS application programs that access SQL/MP databases, see the *SQL/MP Programming Manual for C*.

NonStop SQL/MX

The NonStop SQL/MX relational database management system uses the standard Structured Query Language (SQL) to describe and manipulate data. SQL/MX can be used for online transaction processing, decision support systems, scalable electronic commerce applications, and other business applications.

You can access SQL databases from an OSS program using embedded SQL statements. You can also get information about SQL objects, catalogs, and programs. You can use the `c89` or the `c99` utility to compile, link, and optimize a program in a single `c89` or `c99` command.

You do need to invoke the SQL/MX compiler as well as the C compiler in the `c89` or the `c99` command. For information about the `c89` utility, see the `c89 (1)` reference page either online or in the *Open System Services Shell and Utilities Reference Manual*. For information about the `c99` utility, see the `c99 (1)` reference page either online or in the *Open System Services Shell and Utilities Reference Manual*.

For more information about writing OSS application programs that access SQL/MX databases, see the *SQL/MX Programming Manual for C and COBOL*.



NonStop TS/MP

NonStop Transaction Services/MP (TS/MP) provides online transaction processing. TS/MP can create and monitor OSS servers, but it does not support OSS requesters.

An OSS server process must read its \$RECEIVE file to get messages from requester processes. To read the \$RECEIVE file, you must use Guardian procedures.

OSS and Guardian server classes can be managed by the same PATHMON process. However, because all server processes in a server class use the same object-code file, OSS and Guardian server processes cannot reside in the same server class. OSS server processes and server classes are configured and managed in the same way as Guardian servers and server classes, although some of the configuration parameters are different. Either the PATHCOM interactive interface or the Subsystem Programmatic Interface (SPI) can be used.

To allow for management of OSS server processes in the PATHMON environment, OSS server programs must be coded to follow the standard protocol for Pathway servers: that is, the server must stop itself when the LINKMON process or the terminal control process closes it on behalf of the last requester that has a link to it.

TS/MP uses the PROCESS_SPAWN_ procedure to create OSS server processes. If you configure and use OSS server classes in TS/MP, you might see some PROCESS_SPAWN_ errors, returned within PATHMON error message 1159.

For more information about how OSS server processes interact with TS/MP, see the *TS/MP Pathsend and Server Programming Manual*. For configuration information, see the *TS/MP System Management Manual*.

NonStop TMF

NonStop Transaction Management Facility (TMF) provides transaction protection, database consistency, and database recovery. It does this by managing database transactions, keeping track of database activity through the use of audit trails, and providing database recovery methods.

TMF cannot audit, or protect, OSS files. You can access protected Guardian files from the OSS environment, but you must use Guardian procedures to do so.

For more information about the Guardian procedures you can use to access files protected by the TMF product, see the *TMF Application Programmer's Guide*.

NonStop Tuxedo System

The NonStop Tuxedo system is the Hewlett Packard Enterprise implementation of the Novell, Inc., Tuxedo enterprise transaction processing system. Major features of the NonStop Tuxedo implementation include the use of the Hewlett Packard Enterprise core services (NonStop operating system, TS/MP, and TMF) to provide the Hewlett Packard Enterprise fundamentals (scalability, availability, and manageability) for Tuxedo applications. The NonStop Tuxedo system also provides support for transaction management and monitoring, client/server communications, buffer management, and management and administration of server processes.

Because TS/MP servers open the \$RECEIVE file to get messages from requester processes, you might expect that NonStop Tuxedo servers also open \$RECEIVE, but they do not. NonStop Tuxedo server applications use Tuxedo functions to get input from requester processes.

For more information about programming in the NonStop Tuxedo processing environment, see the *NonStop TUXEDO System Application Development Guide*.

NOTE:

NonStop Tuxedo applications run as OSS processes.



NonStop Distributed Computing Environment (DCE)

NonStop Distributed Computing Environment is an integrated set of services that supports the development and execution of distributed applications between heterogeneous networked computers. Each DCE environment (called a cell) maintains at least the following core DCE services:

- DCE Threads
- DCE Host Services
- DCE Cell Directory Service
- DCE Time Service
- DCE Security Service

With the exception of DCE Threads, all the core DCE services require administration in one way or another. Some services, such as CDS and DCE Security Service, usually need more managing than, for example, DCE Time Service, which needs practically no intervention after you have set it up. If your DCE cell consists of just a few computers and their users, you could probably manage the naming, time, and security needs of users, programs, and host systems by logging on to individual hosts to perform any necessary administration tasks. However, most cells will consist of many, perhaps hundreds or even thousands, of computers and their users.

Refer to *OSF DCE Administration Guide-Core Components*, *OSF DCE Administration Guide--Introduction*, *OSF DCE Application Development Guide-Core Components*, *OSF DCE Application Development Guide-Introduction and Style Guide*, and *OSF DCE Problem Determination Guide* for more information.

Simple Network Management Protocol (SNMP)

The Simple Network Management Protocol (SNMP) originated in the Internet community in the late 1980s as a means for managing TCP/IP and Ethernet networks. Because of its relative simplicity, SNMP has since gained widespread acceptance as a protocol for managing devices from multiple vendors on a network.

Hewlett Packard Enterprise uses subagents in its SNMP implementation. SNMP subagents are responsible for handling a particular collection of resources. Some subagents are implemented as independent processes, and some are bound into other processes, such as the SNMP agent process.

Hewlett Packard Enterprise offers a Subagent Toolkit that helps programmers generate subagents that interoperate with the SNMP agent process. Subagents make customer-written applications manageable by SNMP managers. Refer to the *SNMP Subagent Programmer's Guide* for more information about the Subagent Toolkit.

Hewlett Packard Enterprise also offers a Manager Services toolkit that allows C and C++ programmers to create SNMP managers that run as NonStop operating system processes in either the Guardian or OSS environment. Refer to the *SNMP Subagent Programmer's Guide* for information about the Manager Services toolkit.

Refer to the *SNMP Configuration and Management Manual*, the *SNMP Manager Programmer's Guide*, and the *SNMP Subagent Programmer's Guide* for more information.



Managing OSS Security

This section provides a brief overview of the features available for program control of OSS process and file security. The following topics are discussed:

Topic	Content
<u>Common and Unique Characteristics</u>	Summary of the differences between OSS and UNIX security management
<u>Functions Available for Security Management</u>	Describes the C functions available for managing the security of OSS objects
<u>Security Auditing of OSS files</u>	Describes the application program interfaces (APIs) that can be audited when performing OSS file access and process control activities
<u>Example of Managing file Ownership</u>	Provides a simple example of managing file ownership in the OSS environment
<u>Using OSS Access Control Lists (ACLs)</u>	Describes OSS access control lists and their use.
<u>Restricted-Access Filesets and File Privileges</u>	Describes restricted-access filesets and how to access files in these filesets.

Common and Unique Characteristics

Basic file security is the same for the OSS environment as on a UNIX system. files are accessed according to a file mode and access permissions, as described in the *Open System Services User's Guide* and in **Managing Files** of this guide.

Certain differences might require you to code the security management portions of a program or shell script in a manner specific to the OSS environment. For example:

- OSS user and group administration occurs through tools based in the Guardian environment, such as the Safeguard command interpreter (SAFECON) program or third-party products. OSS user and group administration does not use any of the following files or directories, which can be the target of security intruders:

```
/etc/groups
/etc/passwd
/etc/security
/etc/shadow
```

Standard C functions do provide access to information needed from the security database. However, the database files themselves are not available in the OSS file system.

- OSS administration of device access does not use files in the `/dev` directory that are available on some UNIX systems, such as:

```
console
fd
```



kmem
ttyda

Again, these files can be the target of security intruders.

- The OSS environment does not provide common UNIX default user names and user IDs unless they are explicitly created by a site administrator. However, equivalent OSS user names and user IDs do exist. For example, the privileges normally associated with the UNIX user name `root` and the user ID of 0 exist for the OSS user ID of 65535 (the super ID), which is normally the user `SUPER.SUPER`.
- OSS file auditing mechanisms and policies are implemented through the Guardian environment Safeguard product instead of such UNIX commands or utilities as:

`/etc/reboot`
`/etc/shutdown`
`/etc/syslog`
`passwd`

Prior to the J06.15 and H06.26 RVUs, OSS file auditing features are not available through third-party Security Event-Exit Process (SEEP) programs.

- Access to OSS auditing logs occurs through the Safeguard audit reduction tool (SAFEART) program.
- The OSS implementation of object security does not conform to POSIX.6 draft 12, IEEE Standard 1003-1e, or IEEE Standard 1003-2c. In particular, the OSS environment does not provide access control list APIs except as described in **OSS ACLs**.

Figure 7: Major Components and Interfaces for OSS Security Management illustrates the major components and interfaces of OSS security management mentioned in this section.

Refer to the *Security Management Guide* for an overview of both Guardian and OSS security. Refer to the *Safeguard Audit Service Manual* for a description of the SAFEART program and the information logged for audited OSS files.

The commercial text *Practical UNIX & Internet Security* listed in **About This Document** contains many suggestions for writing secure programs. Those suggestions include which functions to avoid in best-practice program designs.

Beginning with the J06.15 and H06.26 RVUs, a partner or customer OSS SEEP is supported and can participate in access-control decisions for OSS objects. For OSS SEEP details, see **Accessing OSS SEEP-Protected Files**.

Functions Available for Security Management

The OSS environment provides most of the basic XPG4 functions used by C or C++ programs for process and file security.

OSS Security-Related Functions lists and provides a short description of the C functions related to security. Many of these functions have been described in other sections of this guide in the context of file management or process management; some of these functions are not normally regarded as security functions but have security implications when used.

NOTE: To determine which RVUs support an OSS function, see **Documented OSS Functions**.



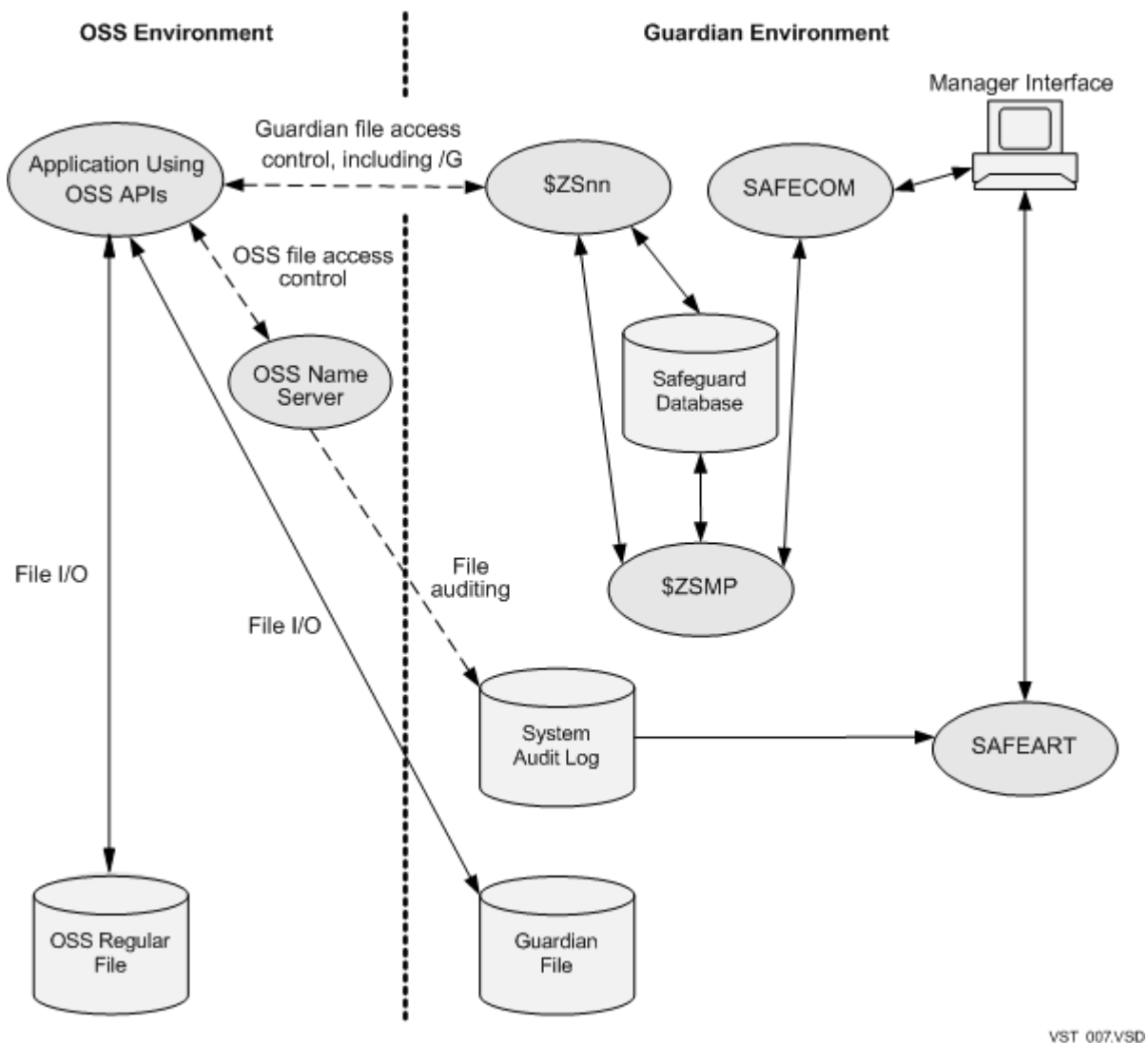


Figure 7: Major Components and Interfaces for OSS Security Management

Table 36: OSS Security-Related Functions

OSS Function	Security-Related Behavior
<code>acl()</code>	Changes file access permissions for a list of users and groups. OSS SEEP consultation. ¹
<code>chmod()</code>	Changes file access permissions.
<code>fchmod()</code>	OSS SEEP consultation. ¹
<code>lchmod()</code>	

Table Continued



OSS Function	Security-Related Behavior
<code>chown()</code>	Changes file ownership.
<code>fchown()</code>	OSS SEEP consultation. ¹
<code>lchown()</code>	
<code>crypt()</code>	Encrypts a password or other character data.
<code>cuserid()</code>	Gets the user name associated with the real user ID of the current process.
<code>encrypt()</code>	Encrypts or decrypts an array.
<code>endgrent()</code>	Closes the group database.
<code>fstatvfs()</code>	Gets fileset access information.
<code>fstatvfs64()</code>	
<code>getegid()</code>	Gets the effective group ID of the calling process.
<code>geteuid()</code>	Gets the effective OSS user ID of the calling process.
<code>getgid()</code>	Gets the real group ID of the calling process.
<code>getsid()</code>	Gets the group ID of the process that is the session leader of the specified process.
<code>getgrent()</code>	Gets group information from the group database.
<code>getgroups()</code>	Gets the list of groups to which the current process belongs.
<code>getlogin()</code>	Gets the login name (user name) for the current terminal session.
<code>getpass()</code>	Reads a string of characters (such as a password) from a terminal without echoing the characters entered.
<code>getpgrp()</code>	Gets the process-group ID of the calling process.
<code>getpid()</code>	Gets the OSS process ID of the calling process.
<code>getppid()</code>	Gets the OSS process ID of the parent process for the calling process.
<code>getuid()</code>	Gets the real OSS user ID of the calling process.
<code>lstat()</code>	Gets the OSS user ID for the owner of a file or symbolic link.
<code>lstat64()</code>	

Table Continued



OSS Function	Security-Related Behavior
<code>mkdir()</code>	Creates a directory with specified access permissions and default OSS user ID and group ID values. OSS SEEP consultation. ¹
<code>rmdir()</code>	Removes a directory subject to access permissions. OSS SEEP consultation. ¹
<code>setfilepriv()</code>	Sets the file privilege attribute for a file. The file privilege attribute is used in addition to the file permissions. For more information, see Restricted-Access Filesets and File Privileges .
<code>setgid()</code>	Sets the effective group ID of the calling process.
<code>setgrent()</code>	Resets the group name key to get group information from the group database.
<code>setkey()</code>	Sets an encryption or decryption key.
<code>setpgid()</code>	Changes the process-group ID for job control.
<code>setpgrp()</code>	Creates a new session when the calling process is not a process group leader, and sets the process group ID of the calling process.
<code>setregid()</code>	Sets the real and effective group ID.
<code>setreuid()</code>	Sets the real and effective user ID.
<code>setsid()</code>	Sets the process-group ID for a new terminal session.
<code>setuid()</code>	Sets the effective OSS user ID of the calling process.
<code>stat()</code>	Gets the OSS user ID for the owner of a file.
<code>stat64()</code>	
<code>fstat()</code>	
<code>fstat64()</code>	
<code>fstatat()</code>	
<code>fstatat64()</code>	

¹Beginning with the J06.15 and H06.26 RVUs, a partner or customer OSS Security Event-Exit Process (SEEP) is supported and can participate in access-control decisions for OSS objects. For OSS SEEP details, see **Accessing OSS SEEP-Protected Files**.

The Guardian environment provides separate procedures for managing Guardian process and Guardian file security. Refer to the *Security Management Guide* for an overview of both Guardian and OSS security interfaces and activities.



Security Auditing of OSS files

Your application program does not need to provide its own file auditing mechanism. Your system manager can enable file auditing on a specific fileset. When auditing is enabled, certain application program activities for any file in that fileset can automatically generate entries in the audit file for the system where the file is located.

The following subsections describe:

- **How to Identify an Audited file in Audit Records**
- **What Operations and Objects Are Audited**
- **Considerations for File Auditing**

How to Identify an Audited file in Audit Records

You can write either an OSS or Guardian application program to process audit records. Audit files are structured files and are easiest to process with Guardian APIs. The structure and Guardian file naming conventions for audit files, the field names and object types, and the format of audit file records are described in the *Safeguard Audit Service Manual*.

The audit record for an audited file event identifies the file being audited with both its absolute OSS pathname and the internal name used for it by Guardian access mechanisms. The internal name includes the creation version serial number (CRVSN) so that a specific instance of a file can be identified. An audit entry looks like:

```
pathname=$volume_name.subvolume_name.file_identifier:crvsn
```

For example:

```
/bin/ed=$DATA13.ZYQ00001.Z0004G7:1934568735
```

where 1934568735 is the CRVSN of the file being audited. For more information on the use of the CRVSN with files, refer to the *Open System Services Management and Operations Guide*.

What Operations and Objects Are Audited

Table **Table 37: OSS Function Calls Audited When Used With Audited Filesets** and table **Guardian Procedure Calls Audited When Used With Audited Filesets** provides a brief overview of the information logged for audited OSS files and available through the SAFEART program. Table **Guardian Procedure Calls Audited When Used for Process Control** lists the audited Guardian procedures that can be used from an OSS program.

Manipulating an audited Guardian file through OSS function calls and the /G directory also causes log entries; the information logged for Guardian files is controlled by Guardian auditing policies rather than by membership in an audited fileset. Similarly, manipulation of Guardian files through Guardian procedure calls from an OSS program can be audited through Guardian policies, which are not discussed in this guide.

NOTE: To determine which RVUs support an OSS function, see **Documented OSS Functions**.



Table 37: OSS Function Calls Audited When Used With Audited Filesets

OSS Function	Attributes or Actions Audited
<code>access()</code>	The value of the <code>access_mode</code> parameter. Use on files in the <code>/G</code> directory is also audited.
<code>faccessat()</code>	
<code>acl()</code>	The number of ACL entries and the value of each changed ACL entry before and after the call.
<code>bind()</code>	For <code>AF_UNIX</code> sockets, the values of the <code>access_mode</code> parameter, the OSS user ID, group ID, and <code>rdev</code> .
<code>chmod()</code>	The values of the file mode before and after the call.
<code>fchmod()</code>	
<code>fchmodat()</code>	
<code>lchmod()</code>	
<code>chown()</code>	The values of the OSS user ID, group ID, and file mode before and after the call.
<code>fchown()</code>	Use on files in the <code>/G</code> directory is also audited.
<code>fchownat()</code>	
<code>lchown()</code>	
<code>connect()</code>	For <code>AF_UNIX</code> sockets, the values of the OSS user ID and group ID. Only audited beginning with the J06.15 and H06.26 RVUs.
<code>creat()</code>	For all files, the value of the file mode, the OSS user ID, the group ID, and <code>rdev</code> .
<code>creat64()</code>	For regular files, the value of the open flags.
<code>execl()</code>	The value of the OSS user ID and group ID. Use on files in the <code>/G</code> directory is also audited.
<code>execle()</code>	
<code>execlp()</code>	
<code>execv()</code>	
<code>execve()</code>	
<code>fexecve()</code>	
<code>execvp()</code>	
<code>fork()</code>	
	The value of the OSS user ID and group ID.

Table Continued

OSS Function	Attributes or Actions Audited
<code>kill()</code>	The signal sent, the real OSS user ID and effective OSS user ID of the sender, the OSS process ID or process group ID, the process handle, and the saved set OSS user ID of the target process. If the target process is a member of a process group, the audit information is logged for all the processes affected by the call when those processes can be determined.
<code>link()</code> <code>linkat()</code>	The name and the link count of the linked-to file, and the new filename.
<code>mkdir()</code> <code>mkdirat()</code>	The value of the file mode, the OSS user ID, the group ID, and <code>rdev</code> .
<code>mkfifo()</code> <code>mkfifoat()</code>	The value of the file mode, the OSS user ID, the group ID, and <code>rdev</code> .
<code>mknod()</code> <code>mknodat()</code>	The value of the file mode, the OSS user ID, the group ID, and <code>rdev</code> .
<code>open()</code> <code>openat()</code> <code>open64()</code>	The value of the open flags and the value of the file mode before and after the call. Use on files in the <code>/G</code> directory is also audited.
<code>opendir()</code> <code>fdopendir()</code>	The value of the open flags and the value of the file mode before and after the call.
<code>rename()</code> <code>renameat()</code>	The old and new pathname values.
<code>rmdir()</code>	For a link count of zero, the value of the file mode, the OSS user ID, the group ID, the <code>mtime</code>, <code>ctime</code>, <code>size</code>, and <code>rdev</code>. For a link count that is not zero, the value of the link count after the call.
<code>setfilepriv()</code>	The value of the file privilege attribute before and after the call. Use on files in the <code>/G</code> directory is also audited.
<code>setgid()</code>	The value of the real, effective, and saved-set group ID before and after the call.
<code>setpgid()</code>	The value of the process-group ID before and after the call.
<code>setpgrp()</code>	The value of the process-group ID before and after the call.
<code>setregid()</code>	The value of the real, effective, and saved-set OSS group ID before and after the call

Table Continued



OSS Function	Attributes or Actions Audited
<code>setuid()</code>	The value of the process-group ID before and after the call.
<code>setreuid()</code>	The value of the real, effective, and saved-set OSS user ID before and after the call.
<code>setuid()</code>	The value of the real, effective, and saved-set OSS user ID before and after the call.
<code>symlink()</code> <code>symlinkat()</code>	The contents of the symbolic link and the value of the file mode, the OSS user ID, the group ID, and <code>rdev</code> .
<code>tdm_execve()</code> <code>tdm_execvpe()</code>	The process name and the value of the OSS user ID and group ID. Use on files in the <code>/G</code> directory is also audited.
<code>tdm_fork()</code>	The process name and the value of the OSS user ID and group ID.
<code>tdm_spawn()</code> <code>tdm_spawnp()</code>	The process name and the value of the OSS user ID and group ID. Use on files in the <code>/G</code> directory is also audited.
<code>unlink()</code> <code>unlinkat()</code>	For a link count of zero, the value of the file mode, the OSS user ID, the group ID, the <code>mtime</code> , <code>ctime</code> , <code>size</code> , and <code>rdev</code> . For a link count that is not zero, the value of the link count after the call .
<code>utime()</code>	The values of <code>mtime</code> , <code>atime</code> , and <code>ctime</code> before and after the call. Use on files in the <code>/G</code> directory is also audited.

Table 38: Guardian Procedure Calls Audited When Used With Audited Filesets

Procedure Call	Attributes or Actions Audited
<code>FILE_OPEN_</code> <code>SPT_FILE_OPEN</code>	Resolution of a supplied pathname to a regular file in an audited fileset; use of an internal OSS filename; all Guardian file audit attributes.
<code>PROCESS_SPAWN[64]_</code>	Resolution of a supplied pathname to a regular file in an audited fileset; use of an internal OSS filename; all Guardian file audit attributes.



Table 39: Guardian Procedure Calls Audited When Used for Process Control

Procedure Call	Attributes or Actions Audited
FILE_OPEN_ SPT_FILE_OPEN	The value of the open flags and the value of the file mode before and after the call
PROCESS_SPAWN[64]_	The process name and the value of the OSS user ID and group ID

Considerations for File Auditing

The `/G` and `/E` directories and Network file System (NFS) filesets cannot be audited. Audited filesets cannot be mounted for NFS client use. By default, Restricted-access filesets are audited.

Prior to the J06.15 and H06.26 RVUs:

- OSS sockets function calls such as `connect()` are not audited.
- The `chdir()` and `chroot()` function calls are not audited. The method used to audit pathnames uses absolute names from the system root directory, so these functions do not require auditing.

Beginning with the J06.15 and H06.26 RVUs:

- The `connect()` function is audited for file operations in OSS SEEP-protected filesets.
- The `chdir()` and `chroot()` functions are audited for file operations in OSS SEEP-protected filesets.

For those file operations that involve OSS SEEP consultation, the audit record is generated with the final ruling after the OSS SEEP consultation. (For a list of all OSS SEEP operations, see **OSS SEEP System and Library Calls**.)

The `pipe()` function call is not audited because its effects are transitory.

When a directory that is a component of one or more fileset mount point pathnames is renamed, that renaming must be propagated to all of the affected filesets. This propagation is not part of the renaming operation; propagation occurs after completion of the `rename()` function call. If both of the following are true:

- An affected fileset and its mount point are managed by different OSS name server processes
- The renaming is followed immediately by an audited operation on a file in that fileset

Then the audit record produced by the latter operation might include a pathname that does not reflect the renaming because the renaming propagation has not completed. For example, if an audited fileset is mounted on `/usr/src/projects/mine` and the following function calls occur:

```
rename("/usr/src/projects", "/usr/src/tasks");
open("/usr/src/tasks/mine/main.c");
```

Then the pathname recorded for the audited `open()` call might be `/usr/src/projects/mine/main.c` instead of `/usr/src/tasks/mine/main.c`.

The propagation delay depends on processor and system workloads and is normally a second or less.



Example of Managing file Ownership

Using OSS Security Functions provides an example of code demonstrating the use and effect of the `getuid()`, `geteuid()`, `getgid()`, `getegid()`, `setgid()`, `setregid()`, and `chown()` functions in managing file ownership.

Using OSS Security Functions

```
/* Using OSS Security functions (without ACLs) */

#include <stdio.h>
#include <limits.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdlib.h>
#include <grp.h>
#include <pwd.h>
/*
 * Print out the group number in decimal followed by (groupname)
 */
void printgroup(gid_t groupid)
{
    unsigned long    lt;                /* temp */
    struct group *grpPtr;               /* pointer to group info */

    lt = (unsigned long)groupid;        /* make the gid a long */
    printf(" %lu(",lt);                /* print it */
    grpPtr = getgrgid(groupid);        /* get group structure */
/* print question marks if group name is unknown */
    if(grpPtr == NULL) {
        printf("??????");
        return;
    }
    printf("%s)",grpPtr->gr_name);      /* print group name */
    return;
}
/*
 * Print out the user ID in decimal followed by (username)
 */
void printuser(uid_t userid)
{
    unsigned long    lt;                /* temp */
    struct passwd *pwPtr;               /* pointer to user info */

    lt = (unsigned long)userid;        /* make the uid a long */
    printf(" %lu(",lt);                /* print the number */
    pwPtr = getpwuid(userid);          /* get the information */
/* print question marks if user ID is not known */
    if(pwPtr == NULL) {
        printf("??????");
        return;
    }
    printf("%s)",pwPtr->pw_name);
    return;
}
```

```

void printsupgroups()
{
    int      ngroups;                /* number of supplemental groups */
    gid_t    grouplist[NGROUPS_MAX]; /* list of group IDs */
    int      i;
    gid_t    gid;

    /* Get the number of supplementary group IDs in use */
    ngroups = getgroups(NGROUPS_MAX, grouplist);
    if(ngroups == -1) {
        perror("getgroups() failed");
        return;
    }
    printf("The following supplementary groups are available:\n");
    for(i = 0; i < ngroups; i++) {
        gid = grouplist[i];
        printf("\t");
        printgroup(gid);
        printf("\n");
    }
    return;
}

struct stat statbuf;

int main(int argc, char **argv)
{
    uid_t    uid;
    gid_t     gid, egid;
    char      *login;
    char      termid[L_ctermid];
    int fd;

    /* Get new effective group ID from command line.
       Default value set to real group ID value.
    */
    if(argc > 1)
        egid = atoi(argv[1]);
    else
        egid = getgid();
    /* Print out information about the user */
    login = getlogin();
    if (login == NULL)
        printf("Login name is not known\n");
    else
        printf("Login name is '%s'\n", login);
    ctermid(termid);
    printf("Terminal pathname is '%s'\n", termid);
    uid = getuid();
    printf("Real UID is");
    printuser(uid);
    uid = geteuid();
    printf(" and effective UID is");
    printuser(uid);
    gid = getgid();
    printf("\nReal group ID is");
    printgroup(gid);

```

```

    gid = getegid();
    printf(" and effective group ID is ");
    printgroup(gid);
    printf("\n");
    printsupgroups();

/* Create a new file with effective group ID of this process.
   Then set the effective group ID of this process to one of
   the supplemental group values. Use chown to set the group ID
   of the file to the new effective group ID. The real group ID
   value is left unchanged.
*/
    if((fd = open("junkfile", O_RDWR|O_CREAT, 0666)) < 0) {
        fprintf(stderr, "Can't create file 'junkfile'\n");
        exit(1);
    }
    close(fd);
    if(stat("junkfile", &statbuf) < 0) {
        fprintf(stderr, "Can't get status of file 'junkfile'\n");
        exit(1);
    }
    printf("junkfile created with uid = %d, gid = %d\n",
        statbuf.st_uid, statbuf.st_gid);
    if(setregid( (gid_t)-1, egid) < 0) {
        fprintf(stderr, "Can't change effective group ID of process.\n");
        exit(1);
    }
    if(chown("junkfile", (uid_t)-1, egid) < 0) {
        fprintf(stderr, "Can't change group ID of file.\n");
        exit(1);
    }
    if(stat("junkfile", &statbuf) < 0) {
        fprintf(stderr, "Can't get status of file 'junkfile'\n");
        exit(1);
    }
    printf("junkfile chown'ed with uid = %d, gid = %d\n",
        statbuf.st_uid, statbuf.st_gid);
    gid = getgid();
    printf("Real group ID is");
    printgroup(gid);
    gid = getegid();
    printf("and effective group ID is");
    printgroup(gid);
    printf("\n");
    return(0);
}

```

This program produces the following output (software.abcde (1,5) is a member of SUPER group):

```

/usr/abc: run a.out 255
Login name is SOFTWARE.ABCDE
Terminal pathname is /dev/tty
real UID is 261(SOFTWARE.ABCDE) and effective UID is 261(SOFTWARE.ABCDE)
Real group ID is 1(SOFTWARE) and effective group ID is 1(SOFTWARE)
The following supplementary groups are available:
    1(SOFTWARE)
    255(SUPER)

```



```
junkfile created with uid = 261, gid = 1
junkfile chowned with uid = 261, gid = 255
Real group ID is 1(SOFTWARE) and effective group ID is 255(SUPER)
```

Using OSS Access Control Lists (ACLs)

OSS access control lists (ACLs) are supported for certain files in Version 3 catalog OSS filesets on systems running J-series RVUs, H06.08 and later H-series RVUs, or G06.29 and later G-series RVUs.

OSS already enables nonprivileged users or processes, such as file owners, to allow or deny other users access to files and other objects as determined by their user identity, group identity, or both. This level of control is accomplished by setting or manipulating a file's permission bits to grant or restrict access by owner, group, and others (see the `chmod(2)` reference page).

ACLs offer a greater degree of selectivity than permission bits. ACLs allow a process whose effective user ID matches the file owner, super ID, or a member of the Safeguard SECURITY-OSS- ADMINISTRATOR security group to permit or deny access to a file to a list of specific users and groups.

ACLs are supported as a superset of the UNIX operating system discretionary access control (DAC) mechanism for files, but not for other objects such as interprocess communication (IPC) objects.

All OSS system calls that include pathnames are subject to the ACLs on any directory or file in the path.

OSS ACLs:

- Are supported in Version 3 and later catalog versions of OSS filesets.
- Are supported for directories, regular files, first-in, first-out (FIFO) files, and bound AF_UNIX sockets.
- Support up to 150 ACL entries.
- Support separate permissions for up to 146 additional users and groups.
- Support default ACL inheritance (see **ACL Inheritance**).
- Are based on the POSIX 1003.1e draft standard and the HP-UX implementation of ACLs.
- Are not supported by the OSS Network File System (NFS) for J06.08 and earlier J-series RVUs, H06.19 and earlier H-series RVUs, or G-series RVUs. Any attempt by NFS clients to access OSS objects protected by OSS ACLs that contain optional ACL entries is denied.
- Are supported by the OSS NFS for J06.09 and later J-series RVUs and H06.20 and later H-series RVUs, depending upon the NFSPERMMAP attribute value for the fileset that contains the object. For information about the NFSPERMMAP attribute, see **OSS Network File System (NFS) and ACLs**.
- Beginning with the J06.15 and H06.26 RVUs, a partner or customer OSS Security Event-Exit Process (SEEP) is supported and can participate in access-control decisions for OSS objects. The OSS SEEP returns a file-access ruling for file operations in OSS SEEP-protected filesets, if the OSS SEEP is running. The OSS SEEP's YES ruling does not override denials due to a POSIX ACL. For details of OSS SEEP results, see **Final Result of the Operation**.

Definitions

Control of access to data is a key concern of computer security. These definitions, based on the *Department of Defense Trusted Computer System Evaluation Criteria*, explain the concepts of access control and its relevance to OSS security features:

access

A specific type of interaction between a subject and an object that results in the flow of information from one to the other. Subjects include persons, processes, or devices that cause information to flow among objects or change the



system state. Objects include files (ordinary files, directories, special files, FIFOs, and so on) and IPC features (shared memory, message queues, semaphores, and sockets).

access control list (ACL)

An access control list is a set of user.group, mode entries associated with a file that specifies permissions for all possible combinations of user IDs and group IDs.

ACL entry

An entry in an ACL that specifies access rights for a file owner, owning group, group class, additional user, additional group, or all others.

change permissions

The right to alter DAC information (permission bits or ACL entries). Change permission is granted to object (file) owners and to privileged users.

discretionary access control (DAC)

A means of restricting access to objects based on the identity of subjects, groups to which they belong, or both. The controls are discretionary because a subject with a certain access permission is able to pass that permission (perhaps indirectly) to any other subject.

mode

Three bits in each ACL entry that represent read, write, and execute or search permissions.

privilege

The ability to ignore access restrictions and change restrictions imposed by security policy and implemented in an access control mechanism. In OSS, the super ID is the only user ID that can ignore access restrictions. However, the super ID and any member of the Safeguard SECURITY-OSS-ADMINISTRATOR security group can change the ownership and access permissions (standard UNIX permissions or ACL entries) of a file.

Access Control List Entries

An ACL consists of a set of one-line entries that specify permissions for a file. Each entry specifies for one user-ID or group-ID a set of access permissions, including read, write, and execute/search.

To understand the relationship between access control lists and traditional file permissions, consider the following file and its permissions:

```
-rwxr-xr-- james admin datafile
```

For this file:

- The owner is the user `james`.
- The group is `admin`.
- The name of the file is `datafile`.
- The file owner permissions are `rwx`.
- The file group permissions are `r-x`.
- The file other permissions are `r--`.

In an ACL, user and group IDs are represented by names or numbers, as found in the user authentication database and group database for the system.



ACL Notation

Supported commands that manage ACLs recognize these symbolic representations:

```
[d[efault]:]u[ser]:[uid]:perm
[d[efault]:]g[roup]:[gid]:perm
[d[efault]:]c[lass]:perm
[d[efault]:]o[ther]:perm
```

An ACL entry prefixed with `d:` or `default:` can occur only in ACLs for directories. The prefix indicates that the remainder of the entry is not to be used in determining the access rights to the directory but instead is to be applied to any files or subdirectories created in the directory (see [ACL Inheritance](#)).

The `uid` and `gid` fields contain either numeric user or group IDs, or their corresponding character strings from the authentication database and group database for the system.

The `perm` field indicates access permission either in symbolic form, as a combination of `r`, `w`, `x`, and `-` (dash), or in numeric form, as an octal value of 0 through 7 representing the sum of 4 for read permission, 2 for write permission, and 1 for execute permission.

Types of ACL Entries

An ACL can contain several types of entries:

- **Base ACL Entries**
- **Class Entry**
- **Actual ACL Entries**
- **Default ACL Entries**

Base ACL Entries

The base ACL entries grant permissions equivalent to standard UNIX permissions. When an ACL consists of the four base ACL entries only, it is called a minimal ACL, and the permissions for the class and other ACL entries are equal. The `chmod()` and `acl()` functions can change base ACL entries.

Table 40: Base ACL Entries

Notation	Entry Type	Description
<code>user::perm</code>	USER_OBJ	Permissions for the owner of the object
<code>group::perm</code>	GROUP_OBJ	Permissions for the owning group of the object
<code>class:perm</code>	CLASS_OBJ	The maximum permissions granted to the file group class
<code>other:perm</code>	OTHER_OBJ	Permissions for other users

Class Entry

The class ACL entry, which is a base ACL entry, was created to preserve compatibility with programs that only use `chmod()`. Because of the class ACL entry, existing programs that use `chmod()` can work with file that have ACL entries for optional users and optional groups in addition to the owning group.



The class entry acts as an upper bound for file permissions. In an ACL that contains optional group entries or optional user entries, the class entry specifies the maximum permissions that can be granted to:

- Members of the owning group
- Any additional user entries (optional users)
- Any additional group entries (members of any optional groups)

The class entry is useful because it allows you to restrict the permissions for all of the other ACL entries by changing only one ACL entry. If optional `user` or optional `group` ACL entries are present, the `chmod` command changes the permissions of the `class` ACL entry instead of the permissions of the owning group. This behavior allows programs that use the `chmod` command to support files or directories that have permissions for additional users and groups.

Optional ACL Entries

Optional ACL entries are ACL entries other than the base ACL entries. Optional ACL entries grant permissions beyond the standard UNIX permissions and can be used to further allow or deny access to the file. A file or directory is considered to have an ACL only if optional ACLs are present. In OSS, you can specify up to 146 optional ACL entries in an ACL. You use the `setacl` command or the `acl()` system call to set ACL entries.

Table 41: Actual (Nondefault) Optional ACL Entries

Notation	Entry Type	Description
<code>user:uid:perm</code>	USER	Permissions for the user specified by <i>uid</i>
<code>group:gid:perm</code>	GROUP	Permissions for the group specified by <i>gid</i>

Examples of actual optional ACL entries:

`u:mary:rwX`

Grant read, write, and execute access to the user `mary`.

`user:george:---`

Deny access to the user `george`.

`g:writers:rw-`

Grant read and write access to all members of the group `writers`.

Actual ACL Entries

The base ACL entries, optional user ACL entries, and optional group ACL entries are considered actual ACL entries because they actually control access to the associated file or directory. These ACL entries are also called nondefault ACL entries.

Default ACL Entries

Default ACL entries are allowed for directories only. Default ACL entries do not determine who can access the directory. Instead, default ACL entries affect the access permissions for files or directories created in the directory (see [**ACL Inheritance**](#)). All default ACL entries are optional ACL entries.



Table 42: Default ACL Entries

Notation	Entry Type	Description
<code>default:user::perm</code>	DEF_USER_OBJ	Default permissions for the object owner
<code>default user:uid:perm</code>	DEF_USER	Default permissions for additional users specified by <i>uid</i>
<code>default:group::perm</code>	DEF_GROUP_OBJ	Default permissions for members of the owning group of the object
<code>default:group:gid:perm</code>	DEF_GROUP	Default permissions for members of the additional group specified by <i>gid</i>
<code>default:class:perm</code>	DEF_CLASS_OBJ	Default maximum permissions granted to the file group class
<code>default:other:perm</code>	DEF_OTHER_OBJ	Default permissions granted to other users

The permissions for these entries in the parent directory, modified by the file-creation mode, the umask, or both, become the permissions for the base ACL entries for a new file when the new file inherits ACL entries from the parent directory:

- `default:user::perm` (DEF_USER_OBJ)
- `default:group::perm` (DEF_GROUP_OBJ)
- `default:class:perm` (DEF_CLASS_OBJ)
- `default:other:perm` (DEF_OTHER_OBJ)

These entries are sometimes referred to as base default ACL entries.

ACL Uniqueness

Entries are unique in each ACL. An ACL can contain only one of each type of base entry, and only one entry for any given user or group ID. Likewise, an ACL can contain only one of each type of default base entry and only one default entry for any given user or group ID.

ACL Inheritance

The permissions, including access control list entries, if any, for a newly created file are determined by:

- Whether the fileset of the created file supports OSS ACLs
- Whether the system on which the process is running supports OSS ACLs
- Whether the parent directory of the created file contains default ACL entries
- The file-creation mode (*mode*)
- The process umask (*umask*)

If the fileset does not support OSS ACLs, the permissions of the created file are the *mode* bitwise-ANDed with the complement of the *umask*.



If the fileset supports OSS ACLs, but the system on which the process is running does not support OSS ACLs, and the parent directory for the created file does not have default ACL entries, the permissions of the created file are the *mode* bitwise-ANDed with the complement of the *umask*.

If the fileset supports OSS ACLs, but the system on which the process is running does not support OSS ACLs, and the parent directory for the created file has default ACL entries:

- The permissions for the base ACL entries of the created file or directory are determined by a combination of the file-creation mode and the default base ACL entries of the parent directory as follows:

Permissions Type	Derived From
USER_OBJ permissions	The DEF_USER_OBJ permissions bitwise-ANDed with the <i>modeuser</i> permissions bitwise-ANDed with the complement of the <i>umaskuser</i> permissions
GROUP_OBJ permissions	The DEF_GROUP_OBJ permissions
CLASS_OBJ permissions	The DEF_CLASS_OBJ permissions bitwise-ANDed with the <i>modegroup</i> permissions bitwise-ANDed with the complement of the <i>umaskgroup</i> permissions
OTHER_OBJ permissions	The DEF_OTHER_OBJ permissions bitwise-ANDed with the <i>modeother</i> permissions bitwise-ANDed with the complement of the <i>umaskother</i> permissions

- The default optional ACL entries for the parent directory of the created file are added to the ACL of the created file as actual (nondefault) optional ACL entries.
- If the created file is a directory, all of the default ACL entries of the parent directory are added to the ACL of the new directory. This behavior allows ACL entries to be inherited by files and directories created under this new directory.

If both the fileset for the created file and the system in which the process is running support OSS ACLs, and the parent directory for the created file does not have default ACL entries, the permissions of the created file are the *mode* bitwise-ANDed with the complement of the *umask*.

If both the fileset for the created file and the system in which the process is running support OSS ACLs, and the parent directory of the created file contains default ACL entries:

- The permissions for the base ACL entries of the created file or directory are determined by a combination of the file-creation mode and the default base ACL entries of the parent directory as follows:

Permissions Type	Derived From
USER_OBJ permissions	The DEF_USER_OBJ permissions bitwise-ANDed with the <i>modeuser</i> permissions
GROUP_OBJ permissions	The DEF_GROUP_OBJ permissions

Table Continued



Permissions Type	Derived From
GROUP_OBJ permissions	The DEF_CLASS_OBJ permissions bitwise-ANDed with the <i>modegroup</i> permissions
OTHER_OBJ permissions	The DEF_OTHER_OBJ permissions bitwise-ANDed with the <i>modeother</i> permissions

- The default optional ACL entries for the parent directory of the created file are added to the ACL of the created file as actual (nondefault) optional ACL entries.
- If the created file is a directory, all of the default ACL entries of the parent directory are copied to the ACL of the new directory. This behavior allows default ACL entries to be inherited by files and directories created under this new directory.

For security reasons, if an ACL contains default ACL entries, all of the default base ACL entries should be provided. During ACL inheritance, if any default base ACL entries are missing, the permissions for the missing default base ACL entries are derived as follows:

Permissions Type	Derived From
DEF_USER_OBJ permissions	The complement of the <i>umask</i> user permissions
DEF_GROUP_OBJ permissions	The complement of the <i>umask</i> group permissions
DEF_CLASS_OBJ permissions	The complement of the <i>umask</i> group permissions
DEF_OTHER_OBJ permissions	The complement of the <i>umask</i> other permissions

Examples of ACL Inheritance

Directory */a* has the following ACL, as reported by the `getacl` command:

```
# file: /a
# owner: alpha
# group: uno
user::rwx
group::rwx
class:rwx
other:rwx
default:user:beta:r--
default:user:gamma:r--
default:group:dos:---
default:group:tres:---
```

In this example, the ACL for a new file created in the directory */a* includes the default ACL entries for directory */a* as actual (nondefault) ACL entries:

```
# file: /a/newfile
# owner: creator_uid
# group: creator_gid
```



```

user::rwuser:
beta:r--
user:gamma:r--
group::r--
group:dos:---
group:tres:---
class:r--
other:r--

```

In this example, a new directory, `dir` is created in the `/a` directory. The default ACL entries of the parent directory, `/a`, are added to the ACL of the new subdirectory twice: first as actual (nondefault) ACL entries and second as the default ACL entries. This behavior ensures that default ACLs propagate downward as trees of directories are created. This example shows the ACL of the new directory, `dir`:

```

# file: /a/dir
# owner: creator_uid
# group: creator_gid
user::rwx
user:beta:r--
user:gamma:r--
group::r-x
group:dos:---
group:tres:---
class:r-x
other:r-x
default:user:beta:r--
default:user:gamma:r--
default:group:dos:---
default:group:tres:---

```

Access Check Algorithm

To determine the permissions granted to an accessing process, the operating system checks for matching IDs in the following order:

Procedure

1. If the EUID of the process is the same as the owner of the file, grant the permissions specified in the `user::` entry of the ACL. Otherwise, continue to the next check.
2. If the EUID matches the UID specified in one of the additional `user:uid:` ACL entries, grant the permissions specified in that entry bitwise-ANDed with the permissions specified in the `class` entry. Otherwise, continue to the next check.
3. If the EGID or a supplementary GID of the process matches the owning GID of the file or one of the GIDs specified in any additional `group:gid:` ACL entries, grant the permissions specified in the `class` entry bitwise-ANDed with the result of bitwise-ORing together all of the permissions in all matching `group` entries. Otherwise, continue to the next check.
4. Grant the permissions specified in the `other:` ACL entry.

Because the checks are performed in this order and the ID match checking stops when a match is found, you can use optional user or group ACL entries with restrictive permissions to deny access to certain users or groups.

If the EGID, the supplementary GIDs of the process, or both match the GIDs of multiple group ACL entries for a file, the process is granted the permissions of all of the matching group entries, restricted by the permissions in the `class` entry. For example, assume that the effective user ID for a process represents the user `beta`, and the group IDs for that process represent group membership only in the `dos` and `tres` groups. In this example, that process is allowed to open the



file /a/file with read/write access, because the `group: dos:` entry granted read access, the `group: tres:` entry granted write access, and the `class:` entry allowed read and write access.

```
# file: /a/file
# owner: creator_uid
# group: creator_gid
user::rw-
group::rw-
group:dos:r--
group:tres:-w-
class:rw-
other:r--
```

ACL Operations Supported

- The `acl ( )` system call sets, retrieves, or counts ACLs.
- The `setacl` command sets or modifies ACLs.
- The `getacl` command retrieves ACLs.
- The `-acl` option of the `find` command locates files with certain ACL properties.
- The `cp`, `cpio -p`, `mv`, and `pax -rw` commands copy ACLs with the source files to the destination files.
- The Backup and Restore 2 utility backs up ACLs with the files on tape and restores ACLs with the files back to disk.

ACL Interaction with `stat ( )`

The `st_mode` field summarizes the access rights to the file. It differs from file permission bits only if the file has one or more optional ACL entries. If one or more optional ACL entries are present in the ACL of the file, the permissions specified in the class entry of the ACL are returned as the permissions for group in the `st_mode` field. Because of this behavior, programs that use the `stat ( )` or `chmod ( )` functions and are ignorant of ACLs are more likely to produce expected results. The `st_acl` field indicates the presence of optional ACL entries in the ACL for the file. The `st_basemode` field provides the owning user permissions, owning group permissions, and other permissions for the file.

ACL Interaction with `chmod ( )`

Using the `chmod ( )` function to set the group permission bits affects the `class:` entry for the file, which in turn affects the permissions granted by additional `user:uid:` and `group:gid:` entries. In particular, using `chmod ( )` to set file permission bits to all zeros removes all access to the file, regardless of permissions granted by any additional `user:uid:` or `group:gid:` entries. If the `chmod ( )` function is used on an object that does not have optional ACL entries, both the `class:` ACL entry and the owning group (`group: :`) ACL entry permission bits are changed to the new group permissions value.

ACL Interaction with `chown ( )`

If you use the `chown ( )` function to change the owner or owning group of a file to a user ID or group ID that has an existing `user:uid:` or `group:gid:` entry in the ACL for the file, those existing entries are not removed from the ACL. However, those existing entries no longer have any effect, because the `user: :` or `group: :` entries take precedence.

OSS Network File System (NFS) and ACLs

For J06.09 and later J-series RVUs and H06.20 and later H-series RVUs, access by the OSS Network File System (NFS) to OSS objects that have OSS ACLs that contain optional ACL entries can be allowed, depending upon the `NFSPERMMAP` attribute value for the OSS fileset that contains the object.



The system software on NFS Version 2 (NFS V2) client systems makes its own access decisions based on its interpretation of the permissions bits of the object. Because NFS Version 2 does not provide support for ACLs, the ACL entries must be mapped to the nine basic permissions bits used for objects in NFS. An object that is protected by an ACL cannot reflect the correct access for all users in these nine permission bits. It might be that access that would be granted by the mapped permission bits is actually denied explicitly by the ACL. It might also be that access that seems to be denied by the mapped permission bits is, in fact, granted explicitly by the ACL. The NFSPERMMAP attribute value selects the algorithm used to map the OSS ACL permissions for the object to the standard permissions (rwxrwxrwx) expected for the object by NFS V2 clients. For information about the NFSPERMMAP attribute, see the *Open System Services Management and Operations Guide*.

NOTE:

If an OSS fileset has objects protected by OSS ACLs, that fileset must be mounted from the NFS client systems using mount options that disable write buffering. Disabling write buffering on the NFS client system causes NFS write requests from the NFS client systems to contain the actual effective user id of the writing client process, which enables the NonStop system to properly enforce the OSS ACL permissions. For example, the mapped NFS permissions can indicate to an NFS client system that a user has write permission to a file when in fact that specific user is denied in the OSS ACL on the NonStop system. In this case, an NFS write request that is denied by an OSS ACL on the NonStop system is likely to be reported to the NFS client application as an EIO or EPERM error.

Because of the behavior of some NFS V2 clients, if you do not disable write buffering, the server might not receive the correct user ID information from the NFS client, which can result in write requests being denied or data being written to a file by a client that should have been denied write access. See the description of the OSS fileset NFSPERMMAP attribute in the *Open System Services NFS Management and Operations Guide*.

For more information about OSS NFS file system security, see the *Overview of NFS for Open System Services* and the *Open System Services NFS Management and Operations Guide*.

Header file

The `sys/acl.h` header file defines the following constants to govern the number of entries per ACL:

NACLENTRIES

The maximum number of entries per ACL, including base entries

NACLBASE

The number of base entries

For compatibility with HP-UX, the variable name `NACLVENTRIES` is provided as an alias for `NACLENTRIES`.

The ACL structure `struct acl` is also defined and includes these fields:

```
int a_type; /* type of entry */

uid_t a_id; /* group ID */

unsigned short a_perm; /* see <unistd.h> */
```

The `sys/acl.h` header defines the set of valid values for the `a_type` field in addition to the valid values for the `cmd` parameter of the `acl( )` function.

The `sys/aclv.h` header file contains the `sys/acl.h` header file and is provided for compatibility with HP-UX.

Example Program For ACLs

Using `acl0` and `aclsort0` provides simple examples of using the `acl( )` and `aclsort( )` APIs.



Using `acl()` and `aclsort()`

```
/* This program provides simple examples of acl(2) and aclsort(3) usage.
 * It adds a GROUP ACL entry (with read permissions) to the ACL of the
 * file. The file pathname and group ID number are passed as command
 * arguments.
 * To run:
 * addACLgroup <pathname> <group ID number>
 * This program performs the following steps:
 * 1. Acquires the count of ACL entries in the ACL on the file
 *    using acl(ACL_CNT).
 * 2. Allocates memory for the ACL buffer using malloc().
 * 3. Acquires the existing ACL on the file using acl(ACL_GET).
 * 4. Adds a new GROUP ACL entry to the end of the ACL buffer.
 * 5. Calls aclsort() to sort the ACL entries in the ACL buffer
 *    into the proper order.
 * 6. Sets the new ACL on the file using acl(ACL_SET).
 * If you run this program twice on the same file, it will report
 * an error in aclsort() as you are trying to add a second group ACL entry
 * for the same group id. aclsort() points to the ACL entry in error.
 */
```

```
#include <stdlib.h>
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <acl.h>
#include <errno.h>
```

```
#define READPERM 4
#define CALCCLASS 1
```

```
typedef struct acl acl_t;
void printAcl( char *header, acl_t *aclEnt, int count )
```

```
{
    int i;
    printf("%s\n",header);
    for (i= 0; i < count; i++) {
        printf("acl entry %d ", i);
        printf("\ta_type = %d ", aclEnt[i].a_type );
        printf("\ta_id = %d ", aclEnt[i].a_id );
        printf("\ta_perm = %o\n", aclEnt[i].a_perm );
    }
}
```

```
main( int argc, char *argv[])
{
    acl_t *aclEnt = 0;      /* pointer to ACL buffer */
    char *pathname = 0;     /* pointer to pathanme */
    int prevCount, newCount; /* counts of ACL entries */
    int groupId;            /* group ID number for new ACL entry */
    int error;              /* error variable */

    pathname = argv[1];     /* get ptr to pathname command argument */
    groupId = atoi(argv[2]); /* get groupId command argument value */
```

```

printf("Input pathname = %s, input groupId = %d\n", pathname, groupId);

/* find out how many ACL entries in the existing ACL on the object */
if (( prevCount = acl(pathname, ACL_CNT, NACLENTRIES, aclEnt)) == -1 ) {
    printf("acl(ACL_CNT) error= %d, text = %s\n", errno, strerror(errno));
    return 1;
}
printf("Number of ACL entries = %d\n", prevCount);

/* Allocate space, reserving 1 extra ACL entry for the new GROUP entry */

newCount = prevCount + 1;
if (( aclEnt = (acl_t *) malloc( newCount * sizeof(acl_t))) == 0 ) {
    printf("malloc error= %d, text = %s\n", errno, strerror(errno));
    return 1;
}

/* Acquire the existing ACL on the object */

if ((prevCount = acl(pathname, ACL_GET, prevCount, aclEnt)) == -1 ) {
    printf("acl(ACL_GET) error= %d, text = %s\n", errno, strerror(errno));
    free(aclEnt);
    return 1;
}
printAcl("Existing ACL entries", aclEnt, prevCount);

/* add new GROUP acl entry at the end of the ACL */

aclEnt[ newCount-1 ].a_type = GROUP;
aclEnt[ newCount-1 ].a_id = groupId;
aclEnt[ newCount-1 ].a_perm = READPERM;
printAcl("New ACL entries before aclsort()",aclEnt, newCount);

/* sort all of the ACL entries into proper order for acl( ACL_SET) */

if ((error = aclsort(newCount, CALCCLASS, aclEnt)) != 0 ) {
    printf("aclsort() error = %d\n", error);
    free(aclEnt);
    return 1;
}
printAcl("New ACL entries after aclsort()", aclEnt, newCount);

/* now set the new ACL on the object */

if ((error = acl(pathname, ACL_SET, newCount, aclEnt)) == -1 ) {
    printf("acl(ACL_SET) error= %d, text = %s\n", errno, strerror(errno));
    free(aclEnt);
    return 1;
}
free( aclEnt );
return 0;
}

```

Restricted-Access Filesets and File Privileges

On systems running J06.11 or later J-series RVUs or H06.22 or later H-series RVUS, OSS Version 3 catalog filesets can be configured as restricted-access filesets. Restricted-access filesets deny the super ID (255,255 in the Guardian environment, 65535 in the OSS environment) special access privileges.

However, because some applications still require special privileges for accessing restricted-access file sets, the J06.11 and H06.22 RVUs introduce the file privilege file attribute, which is used for executable files, user libraries, and ordinary DLLs. System DLLs (provided as part of the system files) and public DLLs (installed by the system administrator) do not require file privileges.

The combination of the Safeguard security groups and file privileges restricts the use of special access privileges in restricted-access filesets to certain system management tasks (like backing up and restoring files using the Backup and Restore 2 product) and to customer-designated programs.

Restricted-Access Filesets

Applications that depend on the super ID for file access are subject to additional restrictions for files that are in restricted-access filesets. When accessing a file in a restricted-access fileset, the super ID is restricted by the same file permissions and owner privileges as any other user ID: It has no special privileges unless the executable file started by the super ID has the PRIVSETID file privilege (see **File Privilege File Attribute**). Therefore the super ID is denied special access privileges when using functions such as these:

```
access ()
acl ()
bind ()
chdir ()
chmod (), lchmod (), and fchmod ()
chown (), lchown (), and fchown ()
creat (), and creat64 ()
link ()
mkfifo ()
mknod ()
open (), and open64 ()
opendir ()
readdir (), readdir_dt, and readdir64 ()
remove ()
rename_oss ()
rmdir ()
setfilepriv ()
unlink ()
utime ()
```

Instead of the super ID, it is members of the Safeguard SECURITY-OSS-ADMINISTRATOR (SOA) security group, sometimes in combination with executable files that have file privileges, that have the appropriate privileges to directly perform the operations permitted to the super ID in unrestricted OSS filesets.

Members of the SOA security group have the same privileges to manage files in restricted-access filesets as they have to manage files in unrestricted filesets. That is, when they are locally-authenticated, they can use these functions and commands on files they do not own:

```
acl (ACL_SET)
chdir ()
chmod (), lchmod (), and fchmod ()3
chown (), lchown (), and fchown ()3
```



```
opendir()  
readdir() and readdir_dt
```

Network File System (NFS) clients are not granted SOA group privileges, even if these clients are accessing the system with a user ID that is a member of the SOA security group.

To use all the functions a super ID would be allowed to use in an unrestricted fileset on a file that is in a restricted-access fileset, the executable file (including any user libraries or ordinary DLLs) that uses the functions must **both**:

- Be started by a locally-authenticated member of the SECURITY-OSS-ADMINISTRATOR group
- Have the PRIVSOARFOPEN file privilege

Restricted-access filesets have the RESTRICTEDACCESS attribute set to a value other than DISABLED. By default, restricted-access filesets are audited. For more information about restricted-access filesets, see the *Open System Services Management and Operations Guide*.

³If the executable file that uses this function does not have the PRIVSOARFOPEN privilege, the set-user-ID and set-group-ID bits of the file mode (04000 and 02000 respectively) of the file in the restricted-access fileset are cleared.

File Privilege File Attribute

On systems running J06.11 or later J-series RVUs or H06.22 or later H-series RVUs, files have an additional file privilege attribute that specifies which special privileges, if any, a file has when accessing files in a restricted-access fileset. For example:

- The executable files for the Backup and Restore 2 product can be given the PRIVSOARFOPEN file privilege to allow a locally-authenticated member of the Safeguard SECURITY-OSS-ADMINISTRATOR (SOA) group to back up and restore files that are in a restricted-access fileset (see **PRIVSOARFOPEN File Privilege**).
- An executable file can be given the PRIVSETID file privilege to allow the super ID to use a privileged switch operation, such as `setgid()` or `setuid()`, to switch to another user ID or group ID and, based on the permissions for that ID, access a file in a restricted-access fileset (see **PRIVSETID File Privilege**).

File privileges:

- Can be set on any OSS regular file or Guardian disk file, whether or not that file is in a restricted-access fileset. However, file privileges only have impact on executable code files, user libraries, and private DLLs.

A process created from an executable file has the privileges of that executable file. Any user library or private DLL used by that process must also have the file privileges of the executable file:

- If an executable file has the PRIVSOARFOPEN file privilege and is started by a member of the Safeguard SOA group, then any user library or ordinary DLL used by that process must also have the PRIVSOARFOPEN privilege. Otherwise, an error is reported when the process attempts to load that library or DLL.
- If the executable file has the PRIVSETID file privilege and is started by the super ID, then any user library or ordinary DLL loaded by the process must also have the PRIVSETID file privilege. Otherwise, an error is reported when the process attempts to load that library or DLL.

System DLLs (provided as part of the system files) and public DLLs (installed by the system administrator) do not require file privileges.

- Are ignored when accessing files that are not in a restricted-access fileset.
- Can be set only by members of the Safeguard SECURITY-PRV-ADMINISTRATOR (SPA) group, using the `setfilepriv` command or the `setfilepriv()` function. In addition, a member of the SPA group can use the



`initfilepriv` command to set the appropriate privileges for the executable files of the Backup and Restore 2 product.

- Are removed whenever a file is modified. When an executable file, user library, or private DLL that requires or has file privileges is newly installed or modified, a member of the SPA group must set or restore those file privileges. This requirement allows the SPA group members to control modifications to files that allow privileged access to files in restricted-access filesets.

Use the `getfilepriv` command to get information about the file privileges for a file. For information about the `getfilepriv` command, see the `getfilepriv(1)` reference page either online or in the *Open System Service Shell and Utilities Reference Manual*.

For information about the `setfilepriv` command, see the `setfilepriv(1)` reference page either online or in the *Open System Service Shell and Utilities Reference Manual*. For more information about the `setfilepriv()` function, see the `setfilepriv(2)` reference page either online or in the *Open System Service System Calls Reference Manual*.

PRIVSOARFOPEN File Privilege

The PRIVSOARFOPEN file privilege allows a process to directly access any file in a restricted-access fileset on the local system, but only if that executable file has been started by a locally-authenticated member of the Safeguard SECURITY-OSS-ADMINISTRATOR (SOA) group. If an executable file has the PRIVSOARFOPEN file privilege and is started by a member of the Safeguard SOA group, then any user library or ordinary DLL used by that process must also have the PRIVSOARFOPEN privilege. Otherwise, an error is reported when the process attempts to load that library or DLL.

If an executable with the PRIVSOARFOPEN is started by a user who is not a member of the SOA group, then that process is created without the PRIVSOARFOPEN privilege.

The PRIVSOARFOPEN file privilege can be inherited by child processes created using `fork()` because the parent and child process share the same executable. Any child processes created by other process creation functions or procedure calls (such as `exec()` or `PROCESS_CREATE_`) acquire their file privileges from that target executable file.

The most common use for this file privilege is to allow a SECURITY-OSS-ADMINISTRATOR to use the Backup and Restore 2 product to back up files that are in restricted-access filesets. It is not required that the executable file be in the restricted-access fileset.

File privileges are removed from a file if the file is changed (such as by being opened for writing).

PRIVSETID File Privilege

The PRIVSETID file privilege allows the locally-authenticated super ID to start a process from an executable file and use a privileged switch operation, such as `setgid()` or `setuid()`, to switch to another user ID or group ID (without a password) and, based on the permissions for that ID, access files in restricted-access filesets. It is not required that the executable file be in the restricted-access fileset.

If the executable file has the PRIVSETID file privilege and is started by the super ID, then any user library or ordinary DLL loaded by the process must also have the PRIVSETID file privilege. Otherwise, an error is reported when the process attempts to load that library or DLL.

The PRIVSETID file privilege can be inherited by child processes created using `fork()` because the parent and child process share the same executable. Any child processes created by other process creation functions or procedure calls (such as `exec()` or `PROCESS_CREATE_`) acquire their file privileges from that target executable file.

If an executable without the PRIVSETID file privilege performs a privileged switch ID operation, the process is unconditionally denied access to files in the restricted-access fileset.

File privileges are removed from a file if the file is changed (such as by being opened for writing).



Authenticated Logons, Authorized Privileged Switch ID Operations, and Restricted-Access Filesets

The user ID and group ID of a process must have been locally-authenticated with a password (at logon) in order for a process to access a files in a restricted-access fileset.

A process performs a privileged switch ID operation when it uses a function such as `setgid()` or `setuid()` to switch to another user ID or group ID (without a password).

If a process performs a privileged switch ID operation and the process does not have the PRIVSETID file privilege, then that privileged switch ID operation is considered unauthorized and that process is denied access to files in restricted-access filesets. Additionally, any child process subsequently created by that process is denied access to files in restricted-access filesets.

Thus, for a process to access a file in a restricted-access fileset, the logon user ID and group ID of the process must have been locally-authenticated, and any privileged switch ID operation performed by the process (and its process ancestors) must have been authorized by the PRIVSETID file privilege on the executable file or files. A process is not permitted to acquire SOA or SPA privileges as the result of a privileged switch ID operation. The SOA or SPA user ID must have been locally-authenticated with a password (at logon).

OSS SEEP Programming

This section provides the following OSS Security Event-Exit Process (SEEP) information:

- **OSS SEEP Management**
- **OSS SEEP Consultation**
- **OSS SEEP System and Library Calls**
- **Final Result of the Operation**
- **OSS SEEP Interprocess-Communication Messages**
- **OSS SEEP Design**
- **OSS SEEP-Related EMS Events**

An OSS SEEP is a user-written process that participates in additional file-access authorization for operations on Version 3 catalog filesets that are OSS SEEP-protected and the OSS SEEP is running. The OSS name server passes requests to the OSS SEEP for file operations and the OSS SEEP returns a ruling to the OSS name server for enforcement.

For an overview of OSS SEEP, see **Accessing OSS SEEP-Protected Files**.

OSS SEEP Management

Subsystem Control Facility (SCF) commands provide the interface for configuration of the OSS SEEP. For details of the OSS SEEP-related SCF commands and attributes, see the *Open System Services Management and Operations Guide*.

The OSS name server controls starting and stopping the OSS SEEP.

Starting an OSS SEEP

The OSS name server automatically starts the associated OSS SEEP with the configured process-attribute values for each of the following conditions:

- When the OSS name server's SEEPENABLED attribute is set to ON, or changed from OFF to ON, and an OSS SEEP-protected fileset is mounted under it
- During fileset mount, if all of these conditions are true:



- The fileset is OSS SEEP-protected.
- The associated OSS name server's SEEPENABLED attribute value is ON.
- The OSS SEEP is not already running.
- When the SEEPPROTECTED attribute of a mounted fileset is set or changed from OFF to ON, if the OSS name server's SEEPENABLED attribute is ON and the OSS SEEP is not already running
- Upon receiving an OSS SEEP death message due to the OSS SEEP's CPU failure or other reasons
- During OSS name server takeover for either of the following conditions:
 - If the OSS SEEP goes down along with the primary OSS name server due to a CPU failure
 - If the OSS SEEP needs to be started based on the current OSS SEEP state

The following is done for the OSS SEEP start:

1. Create the process for the OSS SEEP
2. Open the OSS SEEP, send the startup parameter message, and close the OSS SEEP
3. Open the OSS SEEP for access-authorization requests
4. Send the handshake message, and after an OSS SEEP reply, change the OSS SEEP's state to RUNNING

The OSS SEEP runs with a userid of SUPER.SUPER.

The backup OSS name server does a backup open of the OSS SEEP when the primary OSS name server completes the OSS SEEP start.

Each time the OSS name server starts the OSS SEEP, it first attempts to kill any process with the same process name. An EMS message notes this action. If the OSS name server determines that the OSS SEEP was not started, it does not send messages to the OSS SEEP until it successfully kills the process and restarts it. Be careful to avoid name collisions.

Stopping an OSS SEEP

When the OSS SEEP is required to be stopped, the OSS name server does a close of the OSS SEEP and waits for the process death message. The OSS name server expects the OSS SEEP to perform any necessary cleanup and stop itself upon receiving the final close message, but will kill it if it does not stop in a timely manner. Each of the following conditions trigger an OSS SEEP stop:

- When the OSS name server's SEEPENABLED attribute is changed from ON to OFF
- During fileset unmount, if it is the only OSS SEEP-protected fileset mounted in the controlling OSS name server
- When the SEEPPROTECTED attribute of a mounted fileset is changed from ON to OFF, if the OSS SEEP is running and if there are no other OSS SEEP-protected filesets mounted under the OSS name server

The OSS name server performs a graceful stop by closing the OSS SEEP. After receiving an OSS SEEP death message, the OSS SEEP's state is changed to STOPPED. The OSS SEEP is killed if the OSS SEEP death message is not received within a predefined timeout period of 30 seconds.

The backup OSS name server does a close of the OSS SEEP when the primary OSS name server closes the OSS SEEP, which results in the graceful stop of the OSS SEEP.

OSS SEEP Consultation

The OSS name server consults the OSS SEEP for operations on files in OSS SEEP-protected Version 3 catalog filesets, if the OSS SEEP is running. After the path name resolution, an authorization request containing the final resolved path name



is sent to the OSS SEEP. The OSS SEEP is expected to respond with one of the following rulings within the configured timeout period:

- YES — access approved
- NO — access denied
- NORECORD — no OSS SEEP ruling

The OSS name server communicates with the OSS SEEP by opening the OSS SEEP and then sending messages using the file system APIs. Two message types are supported, one for the handshake message and the other for the authorization request. (During the OSS SEEP startup, the OSS name server sends the OSS SEEP a handshake message to determine the OSS SEEP's readiness to accept authorization requests.)

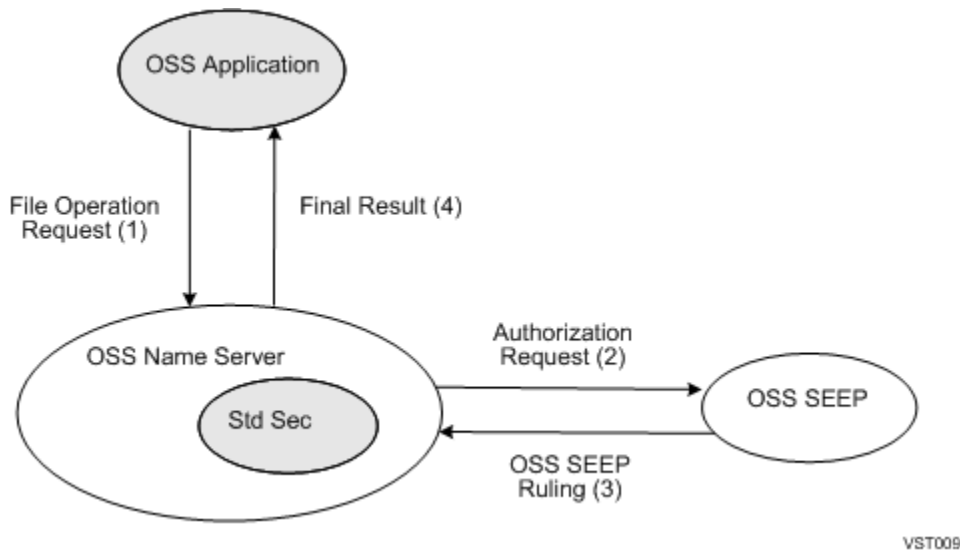


Figure 8: OSS SEEP Consultation Flow

Depending on the OSS SEEP's response, the OSS name server either immediately responds to the request with a security error or continues with the standard security evaluation. For examples of final results for security evaluations, see **Final Result of the Operation**.

Considerations for OSS SEEP consultation are:

- The OSS name server does not consult the OSS SEEP for access to /E and /G filesets. The OSS SEEP consultation for /E path names happens at the target node. For /G files, the access authorization is handled by Safeguard.
- The OSS name server does not consult the OSS SEEP for search authorization on directories in the pathname.
- The directory-search authorization result on a directory in an OSS SEEP-protected fileset is processed as follows:
 - If a directory-search authorization error occurs during pathname resolution due to a POSIX ACL on a directory in an OSS SEEP-protected fileset, the operation is failed immediately, as is done for a normal fileset.
 - If a directory-search authorization error occurs during pathname resolution due to standard OSS permissions on a directory in an OSS SEEP-protected fileset, the OSS name server remembers the error and proceeds with the rest of the pathname resolution and the OSS SEEP consultation. The directory-search authorization error is returned as the result of the operation if any of the following conditions occurs:
 - A fileset that is not OSS SEEP-protected is entered during pathname resolution
 - The OSS SEEP replies with NORECORD

- A path error (PATHDOWN error) is encountered while communicating with the OSS SEEP
- The request to the OSS SEEP times out

For all other cases, any directory-search authorization error is overridden by the OSS SEEP's ruling.

OSS SEEP System and Library Calls

The OSS SEEP returns access-authorization rulings for file operations in Version 3 catalog filesets that are OSS SEEP-protected, if the OSS SEEP is running. For operations that include multiple pathnames, the OSS SEEP is consulted once per system or library call. The following OSS system calls, library calls, and Guardian procedure calls consult the OSS SEEP for access authorization. Note that the operation type passed to the OSS SEEP for each function and procedure is also listed.

```
access(2) - ee-oss-op-access*
faccessat(2) - ee-oss-op-access*
acl(2) - ee-oss-op-acl**
bind(2) - ee-oss-op-bind
chdir(2) - ee-oss-op-chdir
fchdir(2) - ee-oss-op-chdir
chmod(2) - ee-oss-op-chmod
fchmodat(2) - ee-oss-op-chmod
chown(2) - ee-oss-op-chown
fchownat(2) - ee-oss-op-chown
chroot(2) - ee-oss-op-chroot
connect(2) - ee-oss-op-connect
creat(2) - ee-oss-op-open or ee-oss-op-open64
creat64(2) - ee-oss-op-open64
execl(2) - ee-oss-op-exec
execle(2) - ee-oss-op-exec
execvp(2) - ee-oss-op-exec
execv(2) - ee-oss-op-exec
execve(2) - ee-oss-op-exec
fexecve() - ee-oss-op-exec
execvp(2) - ee-oss-op-exec
FILE_OPEN_ - ee-oss-op-fileopen
fchmod(2) - ee-oss-op-chmod
fchown(2) - ee-oss-op-chown
fopen(3) - ee-oss-op-open or ee-oss-op-open64
fopen64(3) - ee-oss-op-open64
lchmod(2) - ee-oss-op-chmod
lchown(2) - ee-oss-op-chown
link(2) - ee-oss-op-link
linkat(2) - ee-oss-op-link
mkdir(2) - ee-oss-op-mkdir
mkdirat(2) - ee-oss-op-mkdir
mkfifo(3) - ee-oss-op-mkfifo
mkfifoat(3) - ee-oss-op-mkfifo
mknod(2) - ee-oss-op-mknod
mknodat(2) - ee-oss-op-mknod
```



```

open(2) - ee-oss-op-open or ee-oss-op-open64
openat(2) - ee-oss-op-open or ee-oss-op-open64
open64(2) - ee-oss-op-open64
openat64(2) - ee-oss-op-open64
opendir(3) - ee-oss-op-opendir
fdopendir(3) - ee-oss-op-opendir
PROCESS_SPAWN[64]_ - ee-oss-op-exec
remove(3) - ee-oss-op-rmdir or ee-oss-op-unlink
rename(2) - ee-oss-op-rename
renameat(2) - ee-oss-op-rename
rename_oss(2) - ee-oss-op-rename
rmdir(2) - ee-oss-op-rmdir
symlink(2) - ee-oss-op-symlink
symlinkat(2) - ee-oss-op-symlink
tdm_execve(2) - ee-oss-op-exec
tdm_execvep(2) - ee-oss-op-exec
tdm_spawn(2) - ee-oss-op-exec
tdm_spawnp(2) - ee-oss-op-exec
unlink(2) - ee-oss-op-unlink
utime(2) - ee-oss-op-utime

```

* OSS SEEP consultation for access types other than F_OK

** OSS SEEP consultation only for ACL_SET

Final Result of the Operation

After receiving the OSS SEEP's result for an authorization request, the OSS name server either continues with the standard security evaluation or immediately responds to the request with a security error.

The following decision table illustrates the final results of operations for the `acl` (ACL_SET), `chown` (2), `fchown` (2), `lchown` (2), `chmod` (2), `fchmod` (2), `lchmod` (2), `chdir` (2), and `opendir` (3) system and library calls.

OSS SEEP Result	POSIX ACL Result	Standard OSS Permissions Result	Directory Search Denial ¹	SOA Member	Final Result
YES	YES	N/A	N/A	N/A	Success
YES	NO	N/A	N/A	NO	Failure (because ACL result of NO)
YES	NO	N/A	N/A	YES	Success (because SOA member overrules ACL result of NO)
YES	None	N/A	N/A	N/A	Success

Table Continued



OSS SEEP Result	POSIX ACL Result	Standard OSS Permissions Result	Directory Search Denial ¹	SOA Member	Final Result
NO	N/A	N/A	N/A	NO	Failure (because OSS SEEP result of NO)
NO	N/A	N/A	N/A	YES	Success (because SOA member overrides OSS SEEP result of NO)
NORECORD	N/A	N/A	YES	N/A	Failure (because directory search denial)
NORECORD	YES	N/A	NO	N/A	Success
NORECORD	NO	N/A	NO	NO	Failure (because ACL result of NO)
NORECORD	NO	N/A	NO	YES	Success (because SOA member overrides ACL result of NO)
NORECORD	None	YES	NO	N/A	Success
NORECORD	None	NO	NO	NO	Failure (because standard permissions result of NO)
NORECORD	None	NO	NO	YES	Success (because SOA member overrides standard permissions result of NO)

¹Directory-search authorization error due to Standard OSS Permissions

The following decision table illustrates the final results of operations for all other system and library calls in **OSS SEEP System and Library Calls**, other than the `acl (ACL_SET)`, `chown (2)`, `fchown (2)`, `lchown (2)`, `chmod (2)`, `fchmod (2)`, `lchmod (2)`, `chdir (2)`, and `opendir (3)` calls.



OSS SEEP Result	POSIX ACL Result	Standard OSS Permissions Result	Directory Search Denial ¹	Final Result
YES	YES	N/A	N/A	Success
YES	NO	N/A	N/A	Failure (because ACL result of NO)
YES	None	N/A	N/A	Success
NO	N/A	N/A	N/A	Failure (because OSS SEEP result of NO)
NORECORD	N/A	N/A	YES	Failure (because directory search denial)
NORECORD	YES	N/A	NO	Success
NORECORD	NO	N/A	NO	Failure (because ACL result of NO)
NORECORD	None	YES	NO	Success
NORECORD	None	NO	NO	Failure (because standard permissions result of NO)

¹Directory-search authorization error due to Standard OSS Permissions

The OSS name server performs its normal standard OSS permissions or POSIX ACLs evaluation upon a NORECORD ruling from the OSS SEEP.

NOTE: An OSS SEEP response timeout is treated as NORECORD for all users.

OSS SEEP Interprocess-Communication Messages

The OSS name server communicates with the OSS SEEP by opening the process and then sending messages using the standard file system API WRITEREAD[X]. The message is divided into three sections: Header_Data, Subject_Data, and Message_Data.

Two message types are supported, one for the handshake message and the other for the authorization request. The handshake message does not contain the subject data and message data sections.

The OSS SEEP's response consists of only the header data section.



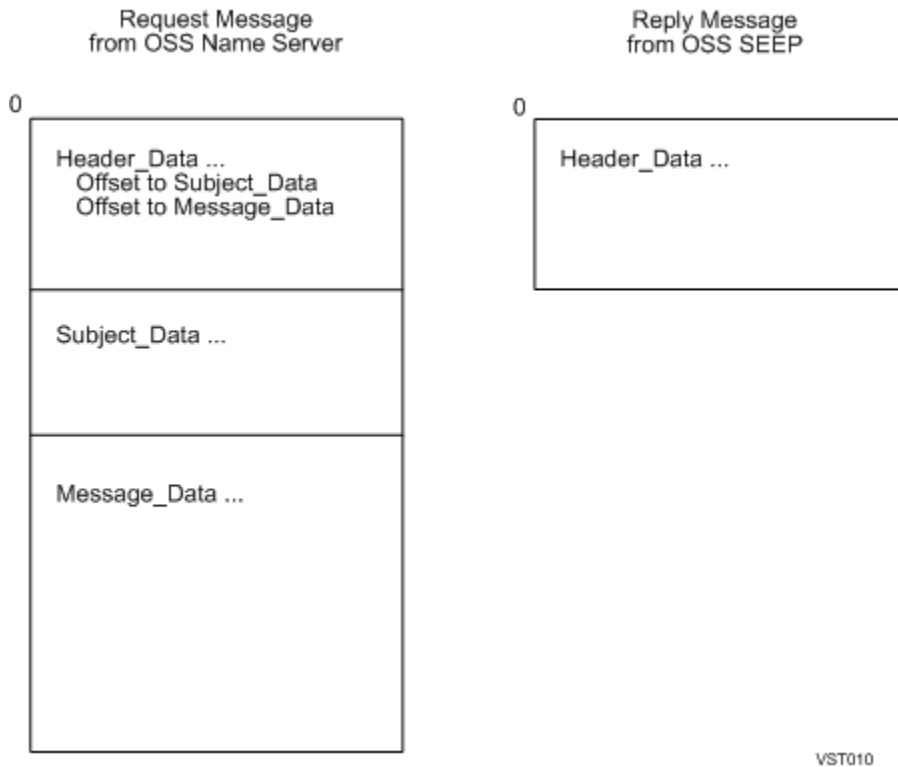


Figure 9: OSS SEEP Message Buffer

The message structure is published in DDL form for use by OSS SEEP developers. The message structures are defined in the new SEEPDDL module.

The OSS SEEP provides its response and status information by altering fields in the Header_Data and returning it to the OSS name server. The OSS SEEP does not return the Subject_Data. The OSS SEEP might return the Message_Response_Data, depending on the type of event.

Header_Data

The following table shows the structure of the header data sent from the OSS name server to the OSS SEEP. The header is always present. When the OSS SEEP responds to an event request, it is expected to alter these fields in the header data:

- Error
- Status
- SubjectData
- MessageData

NOTE: The field names in the following table are pseudonyms and do not exactly match the names as specified in the DDL. For variable names, see the DDL output. The DDL is located in the ZOSS.SEPPDDL file.

Table 43: Header_Data

Field	Type	Description
Base	INT[0:-1]	The base from which the offsets to other data areas are calculated. Base indicates the allocation of a placeholder, not data. It is used for reference for all offsets and VAR-STRING fields within the messages.
EventType	short (2 bytes)	An enumeration describing the type of Message_Data in the message. The valid types are HANDSHAKE (for handshake message) and OSS_ACCESS_CONTROL (for authorization request).
MsgVersion	short (2 bytes)	The version of this message structure. The OSS SEEP is responsible for checking the message version and rejecting any message it does not recognize. The rejection is returned in the Error field of this header. Currently, this value is 1.
MinMsgVersion	short (2 bytes)	The oldest version of the message that must be supported by the OSS SEEP in order to interpret this message. Currently, this value is 1.
SubjectData	short (2 bytes)	The byte offset to the subject data associated with this message. It must be an even number. Set to 0 in the handshake request and in the response from the OSS SEEP.
MessageData	short (2 bytes)	The byte offset to the message data associated with this message. It must be an even number. Set to 0 in the handshake request and in the response from the OSS SEEP.
SSID	SSID-STRUCT	Subsystem ID of the sender in standard SSID format. This is the OSS SSID).
Timestamp	Fixed (8 bytes)	The time the message is sent, in Greenwich mean time (GMT).
OriginalSystemNumber	int (4 bytes)	The system number of the system originating the request.
Filler	int (4 bytes)	Unused in OSS SEEP.

Table Continued

Field	Type	Description
Error	short (2 bytes)	A return value that indicates the OSS SEEP's response to the message. The value is always 0 when the message is sent. Valid return values are as follows: 0 = OK – The request was successfully processed and a valid status is returned 3501 = Message size exceeded maximum expected message size 3503 = OSS SEEP does not support this message data type 3505 = OSS SEEP's supported message version is lower than MinMsgVersion of this message
Status	short (2 bytes)	Status of the reply message. This field is valid only for the OSS_ACCESS_CONTROL message type and if the Error field contains the value 0. The valid values are: 1 = YES 2 = NORECORD 3 = NO
FEA	22 bytes	A future expansion area; currently filled with zeros.

Subject_Data

The following table shows the structure of the subject data sent from the OSS name server to the OSS SEEP. This subject data is always present. It is not returned by the OSS SEEP.

NOTE: The field names in the following table are pseudonyms and do not exactly match the names as specified in the DDL. For variable names, see the DDL output. The DDL is located in the ZOSS.SEPPDDL file.

Table 44: Subject_Data

Field	Type	Description
UserName	Var-String	The subject's user name, in external format. This will be an alias if the user is logged on as an alias.
UserID	int (4 bytes)	The user ID associated with the user name. This is the effective user ID.
CAID	int (4 bytes)	CAID of the subject.
SubjectTerm	Var-String	Home terminal of the subject.

Table Continued



Field	Type	Description
Domain	int (4 bytes)	The security domain from which the request originated. Currently, this is the same as the OriginSystemNumber.
SubjectProcessName/ID	phandle (20 bytes)	The process handle of the subject process.
AuthType	enum	Indicates whether the subject is unauthenticated, locally authenticated, or remotely authenticated. Values are: 0 = unauthenticated 1 = reserved (not used) 2 = locally authenticated 3 = remotely authenticated
IsUndeniable	Boolean	Indicates if the subject is an undeniable user ¹ .
IsAlias	Boolean	True if the UserName is an alias.
RealUserID	int (4 bytes)	The subject's real user ID.
SavedUserID	int (4 bytes)	The subject's saved user ID.
RealGroupID	int (4 bytes)	The subject's real group ID.
EffectiveGroupID	int (4 bytes)	The subject's effective group ID.
SavedGroupID	int (4 bytes)	The subject's saved group ID.
AuthNode	Int (4 bytes)	Last authenticated node number for a remote subject, zero otherwise. Valid only if associated AuthNodeValid is set (True).
GroupList	VAR-STRING	The subject's supplementary group list.
AuthNodeValid	BOOLEAN	True indicates the AuthNode field contains a valid remote node value.
IsSOA	Boolean	Indicates if the user is an SOA member.

Table Continued



Field	Type	Description
IsSPA	Boolean	Indicates if the user is an SPA member.
SubjectType	enum	The subject type. Values are: 1 = NSK 2 = NSF Currently, only NSK is supported.
IsSetUid	Boolean	Indicates if the process has changed the user ID since last logon.
FEA	42 bytes	A future expansion area; currently filled with zeros.

¹Super.Super and SOA members are considered as undeniable users for unrestricted filesets. For restricted-access filesets, only SOA members are considered as undeniable users

Message_Data (Access Control)

The following table shows the structure of the message data sent from the OSS name server to the OSS SEEP for an access-control event. The message data is present only for an access-control event. The OSS SEEP does not return this data in its response.

NOTE: The field names in the following table are pseudonyms and do not exactly match the names as specified in the DDL. For variable names, see the DDL output. The DDL is located in the ZOSS.SEEPDDL file.

Table 45: Message_Data (Access Control)

Field	Type	Description
AXSObjectType	enum	The OSS object type. The current values defined in <code>zoss-obj-type</code> are: directory, symlink, FIFO, terminal, regular file, and socket.
AXSObjectName	Var-String	The absolute path name of the file.
AXSOperation	enum	The attempted file operation. See the <code>zoss-operation</code> definitions for the possible values.
NewObjectName	Var-string	The new path name. Valid only for operations involving two path names (for example, the <code>rename()</code> and <code>link()</code> functions).
NewObjectExists	Boolean	TRUE indicates the new path name points to an existing file. Valid only for the <code>rename()</code> function.

Table Continued



Field	Type	Description
CurrentOwner	int (4 bytes)	The OSS object's current owner.
AXSNewOwner	int (4 bytes)	The OSS object's new owner. Valid only for operations that change ownership, such as <code>chown()</code> .
CurrentGroup	int (4 bytes)	The OSS object's current group.
NewGroup	int (4 bytes)	The OSS object's new group. Valid only for the <code>chgrp()</code> function.
Mode	int (4 bytes)	The file mode.
NewMode	int (4 bytes)	The new file mode for the <code>chmod()</code> function. Also the open or access type for the <code>open()</code> , <code>open64()</code> , <code>FILE_OPEN_</code> and <code>access()</code> APIs.
IsRestrictedFileset	Boolean	TRUE indicates the file belongs to a restricted-access fileset.
FEA	24 bytes	A future expansion area; currently filled with zeros.

OSS SEEP Design

The following restrictions and considerations for the OSS SEEP design are applicable to both the OSS SEEP process and its child processes.

- The OSS SEEP must be a Guardian process.
- The OSS SEEP must be multithreaded and must not perform waited I/O.
- The OSS SEEP must not use any OSS functions that involve OSS name server interaction.
- The OSS SEEP must consider the final close message from the OSS name server as a stop message and do a graceful shutdown of itself.
- Do not specify `$SYSTEM.SYSTEM.NULL` as the `SEEP_PROGFILENAME` attribute.
- The OSS SEEP must open its `$RECEIVE` queue and respond to the handshake message from the OSS name server in order to complete the initialization.
- In the case of a primary OSS name server failure and backup takeover during the initialization phase, the OSS SEEP might receive a second handshake message, which is sent from the new primary. The OSS SEEP is expected to reply to this handshake message.
- The OSS SEEP must set a certain system attribute. For more information, contact the HPE GNSC.



- The OSS SEEP is required to call `PROCESS_GETINFOLIST_` to determine whether the process performing the file operation is an OSS process (or not) and to retrieve the OSS pathname of that program. (The OSS name server does not include this information in the message sent to the OSS SEEP.)

! **IMPORTANT:** Attribute 158 must be used in the call to `PROCESS_GETINFOLIST_` to obtain the fully qualified OSS program pathname for the OSS program. Using attribute 93 can lead to a deadlock between the OSS SEEP process and the OSS name server process.

- The OSS SEEP must not use `FILE_GETINFOLISTBYNAME_` item codes 238 through 246 for OSS regular files, which are introduced in L18.08 RVU.

OSS SEEP-Related EMS Events

The OSS name server generates EMS events to \$0 and \$ZLOG for each of the following conditions:

- The OSS SEEP is successfully started. The EMS event reports the OSS SEEP state change from STOPPED to RUNNING.
- The OSS SEEP is stopped or an OSS SEEP death occurs. The EMS event reports the OSS SEEP state change from RUNNING to STOPPED.
- The OSS SEEP failed to start. The EMS event reports the OSS SEEP state change from STOPPED to STOPPED with the error indicating the reason for failure.
- While attempting to start the OSS SEEP with the configured process name, if there is already a process running with the same name, the OSS name server stops the process and starts the OSS SEEP with that name.
- There is an error sending an authorization request to the OSS SEEP. The EMS event reports a communication error with the OSS SEEP.

After the EMS event is generated for an OSS SEEP's failure to start, the same EMS event is not generated for a predefined hold-off period of 2 minutes.

NOTE:

The `SCF STATUS SERVER` command can also be used to view the OSS SEEP's current state and the most recent error encountered by the OSS name server during the OSS SEEP's creation and communication.



64-Bit Support in OSS and Guardian

The NonStop L-series RVUs, H06.24 RVU (and later H-series RVUs) and the J06.13 RVU (and later J-series RVUs) offer several features to enable the full power of 64-bit computing. These RVUs include tools to help you transition from 32-bit applications to 64-bit OSS applications.

NonStop TNS/E systems running H06.24 and later H-series RVUs or J06.13 and later J-series RVUs, and NonStop TNS/X systems running L-series RVUs provide the following 64-bit support:

- Creation of 64-bit OSS processes

NOTE:

Only 64-bit OSS processes are supported. 64-bit Guardian processes are not supported.

- Access to 64-bit memory from these processes:
 - 64-bit OSS processes
 - 32-bit OSS processes
 - Native Guardian processes

Up to 510 GB of virtual memory is available for these types of OSS and Guardian processes on NonStop TNS/E systems running H06.24 and later H-series RVUs or J06.13 and later J-series RVUs, and NonStop TNS/X systems running L-series RVUs. (The total 510 GB of virtual memory includes 508 GB of 64-bit-addressable memory and 2 GB of 32-bit-addressable memory.)

NOTE:

The limit for 64-bit addressable memory is well below 508 GB and is primarily limited by the swap space configured, the physical memory on the system, the locality of access within the address space, and the number of processes contending for these resources.

Most programs are expected to remain as 32-bit programs. Only programs that can benefit from more virtual memory—and require a robust set of OSS functions (such as `malloc()` and `shm*()`) to access that memory—should be considered as candidates for migration to 64-bit programs.

This chapter explains the wide-ranging impacts for 32-bit and 64-bit OSS applications in both the OSS and Guardian environments.

Features of 64-Bit Support

Features of 64-bit support on NonStop TNS/E and TNS/X systems are:

- Existing 32-bit applications can coexist with 64-bit applications and run on the same processor without modification.
- 64-bit OSS processes have their main heap in 64-bit addressable space. The total size of the heap and any 64-bit flat segments allocated by the process can be a maximum of 508 GB.
- Most public DLLs have two versions, one for use by 32-bit processes and one for use by 64-bit processes.
- The OSS APIs, and other necessary Guardian APIs, have been enhanced to handle 64-bit arguments. The maximum read/write size for process files has been increased to 2 MB. The maximum read/write size for other file types has not changed from previous RVUs.
- 64-bit OSS applications have access to OSS sockets.



- Native C/C++ compilers are provided for creating 64-bit OSS applications.
- Measure can be used to analyze the performance of 64-bit programs.
- Tools are provided to debug and diagnose problems on a 64-bit platform.
- Larger snapshot files can be used for post-mortem analysis of 64-bit process failures.
- 64-bit threaded applications must use the Posix User Threads library.
- 32-bit and 64-bit processes can communicate through inter-process communication (IPC) messages and shared memory.

64-Bit OSS Processes

A 32-bit OSS process is created when running an OSS program that was compiled under the ILP32 data model, which is the default data model. (See [ILP32 and LP64 Data Models](#) for details of the OSS data models.)

A 64-bit OSS process is created from a running OSS program in any of the following events:

- A call is made to one of the `exec()` or `tdm_exec()` family of functions, where the program file named in the path argument is compiled and linked under the LP64 data model. The caller can be either a 32-bit or a 64-bit process.
- A call is made to `tdm_spawn()` or `tdm_spawnp()`, where the program file named in the file argument is compiled and linked under the LP64 data model.
- A call is made to `fork()` from an already existing 64-bit process. (The child process is a copy of the caller/parent process.)

Additionally, a 64-bit OSS process can be created by calling the `PROCESS_SPAWN[64]_Guardian` procedure, where the program file in the OSS program-file argument is compiled and linked under the LP64 data model. The caller can be either a native Guardian process, a 32-bit OSS process, or a 64-bit OSS process that was developed in a mixed-data-model programming environment. For information about mixed-data-model programming, see [Mixed Data-Model Programming](#). The 64-bit OSS process can be created on any processor in the Expand network with the restriction that the OSS program file must reside on the node where the process is being created. (The processor for the 64-bit OSS target process must be running L-series, H06.24 or J06.13 or greater.)

NOTE: A 64-bit OSS process that calls the `PROCESS_SPAWN[64]_Guardian` procedure must be developed in a mixed-data-model environment. That is, all pointers for `PROCESS_SPAWN[64]_Guardian` parameters must be allocated from 32-bit addressable memory using the `malloc32()` function. For information about using `PROCESS_SPAWN[64]_Guardian` in a mixed-data-model environment, see the *Guardian Procedure Calls Reference Manual* in the NTL.

A 64-bit process has its resources allocated as follows:

- Allocated from 32-bit addressable memory:
 - The main stack
 - The priv stack
 - The program file and DLL text
 - The application's and DLL's global data
 - The 32-bit addressable heap (allocated with `malloc32()`)
- Allocated from 64-bit addressable memory:



- The 64-bit addressable heap (allocated by default with `malloc()`)
- SysV Shared Memory (SHM)
- 64-bit flat segments from `SEGMENT_ALLOCATE64_`

Process Type Attribute

The type of process—32-bit or 64-bit—is recorded in the process control block (PCB) during process creation and is provided by the `PROCESS_GETINFOLIST_` Guardian procedure. The attribute code for determining if a process is 64-bit is 155 (`ZSYS_VAL_PINF_IS_64BIT`).

New `PROCESS_GETINFOLIST_` Attributes

New `PROCESS_GETINFOLIST_` 64-Bit-Wide Attributes

`PROCESS_GETINFOLIST_` has new 64-bit attributes that provide information about the 64-bit addresses and values used by a process. Applications that obtain attributes from a 64-bit process should use these new 64-bit attribute codes rather than the equivalent 32-bit attribute codes. These 64-bit attribute codes can also be used to obtain values from a 32-bit process. The new 64-bit attributes are:

Table 46: New `PROCESS_GETINFOLIST_` 64-Bit-Wide Attributes

Code	Attribute	Data Type	Description
142	<code>ZSYS_VAL_PINF_MAINSTACK_ORG64</code>	<code>INT(64)</code>	64-bit origin of the MAIN stack
143	<code>ZSYS_VAL_PINF_MAINSTACK_SIZE64</code>	<code>INT(64)</code>	Current MAIN stack size
144	<code>ZSYS_VAL_PINF_MAINSTACK_MAX64</code>	<code>INT(64)</code>	Maximum size of the MAIN stack
145	<code>ZSYS_VAL_PINF_PRIVSTACK_ORG64</code>	<code>INT(64)</code>	64-bit origin of the privileged stack
146	<code>ZSYS_VAL_PINF_PRIVSTACK_SIZE64</code>	<code>INT(64)</code>	Current privileged stack size
147	<code>ZSYS_VAL_PINF_PRIVSTACK_MAX64</code>	<code>INT(64)</code>	Maximum privileged stack size
148	<code>ZSYS_VAL_PINF_GLOBAL_ORG64</code>	<code>INT(64)</code>	64-bit address start of global data
149	<code>ZSYS_VAL_PINF_GLOBAL_SIZE64</code>	<code>INT(64)</code>	Size of global data
150	<code>ZSYS_VAL_PINF_HEAP_ORG64</code>	<code>INT(64)</code>	Start of the 64-bit heap area
151	<code>ZSYS_VAL_PINF_HEAP_SIZE64</code>	<code>INT(64)</code>	Current size of the 64-bit heap area
152	<code>ZSYS_VAL_PINF_HEAP_MAX64</code>	<code>INT(64)</code>	Maximum size of the 64-bit heap area
153	<code>ZSYS_VAL_PINF_SPACE_GUARANT64</code>	<code>INT(64)</code>	Guaranteed swap/memory space

Table Continued



Code	Attribute	Data Type	Description
155	ZSYS_VAL_PINF_IS_64BIT	INT(32)	Non-zero for a 64-bit process
156	ZSYS_VAL_PINF_64BIT_SEGMENTS	INT(32)	Number of 64-bit segments

The above PROCESS_GETINFOLIST_ attributes are identical to their 32-bit counterparts except that the values returned are 64-bit values rather than 32-bit values (except for ZSYS_VAL_PINF_IS_64BIT and ZSYS_VAL_PINF_64BIT_SEGMENTS).

PROCESS_GETINFOLIST_ Superseded 32-Bit Attributes

The PROCESS_GETINFOLIST_ equivalent 32-bit-only attributes are:

Table 47: PROCESS_GETINFOLIST_ Superseded 32-Bit-Only Attributes

Code	Attribute	Data Type	Description
102	ZSYS_VAL_PINF_MAINSTACK_ORG	INT(32)	32-bit origin of the MAIN stack
103	ZSYS_VAL_PINF_MAINSTACK_SIZE	INT(32)	Current MAIN stack size
104	ZSYS_VAL_PINF_MAINSTACK_MAX	INT(32)	Maximum size of the MAIN stack
105	ZSYS_VAL_PINF_PRIVSTACK_ORG	INT(32)	32-bit origin of the privileged stack
106	ZSYS_VAL_PINF_PRIVSTACK_SIZE	INT(32)	Current privileged stack size
107	ZSYS_VAL_PINF_PRIVSTACK_MAX	INT(32)	Maximum privileged stack size
108	ZSYS_VAL_PINF_GLOBALS_ORG	INT(32)	32-bit address start of global data
109	ZSYS_VAL_PINF_GLOBALS_SIZE	INT(32)	Size of global data
113	ZSYS_VAL_PINF_SPACE_GUARANTEE	INT(32)	Guaranteed swap/memory space

The above PROCESS_GETINFOLIST_ equivalent 32-bit-only attributes:

- Are superseded and Hewlett Packard Enterprise recommends using the equivalent new 64-bit attribute—described in **Table 46: New PROCESS_GETINFOLIST_ 64-Bit-Wide Attributes**—that has a 64-bit data type and can return all possible values.
- For attributes that represent a size or maximum size: If the returned value can be represented by a 32-bit integer, the value will be returned as a 32-bit data type in the *ret-values-list* parameter. Otherwise, a value of `-1D` will be returned in the *ret-values-list* parameter.
- For attributes that represent an address: If the returned value can be represented as a unique 32-bit address, the value will be returned as a 32-bit data type in the *ret-values-list* parameter. Otherwise, a value of `(char *) 0xFFFFC0000` will be returned in the *ret-values-list* parameter.

PROCESS_GETINFOLIST_ Equivalent 32-bit Heap Attributes



The PROCESS_GETINFOLIST_ equivalent 32-bit attributes for the heap are:

Table 48: PROCESS_GETINFOLIST_ Equivalent 32-Bit Heap Attributes

Code	Attribute	Data Type	Description
110	ZSYS_VAL_PINF_HEAP_ORG	INT(32)	Start of the 32-bit heap area
111	ZSYS_VAL_PINF_HEAP_SIZE	INT(32)	Current size of the 32-bit heap area
112	ZSYS_VAL_PINF_HEAP_MAX	INT(32)	Maximum size of the 32-bit heap area

The above PROCESS_GETINFOLIST_ heap attributes are not superseded; however, they only refer to the 32-bit heap so that they are distinguished from the 64-bit heap attributes.

See the *Guardian Procedure Calls Reference Manual* in the NTL for details of the PROCESS_GETINFOLIST_ procedure.

New PROCESSOR_GETINFOLIST_ Attribute

The PROCESSOR_GETINFOLIST_ procedure has a new attribute that applications can use to determine whether the specified (local or remote) system is capable of running 64-bit (LP64) processes.

Table 49: New PROCESSOR_GETINFOLIST_ Attribute

Code	Name	Data Type	Description
80	ZSYS_VAL_CINF_64BIT_SUPPORT	INT(32)	Non-zero for 64-bit process support

See the *Guardian Procedure Calls Reference Manual* in the NTL for details of the PROCESSOR_GETINFOLIST_ procedure.

64-Bit Heap and 32-Bit Heap for 64-Bit OSS Processes

64-Bit Heap for 64-Bit Processes

The default Heap Manager for 64-bit OSS processes manages space in the 64-bit addressable part of process-private virtual memory. A 64-bit OSS process accesses the 64-bit heap when calling the following functions:

- `calloc()`
- `free()`
- `malloc()`
- `realloc()`
- `heap_check()`
- `heap_check_always()`

NOTE: The 64-bit heap is separate from the 32-bit heap, which is accessed with equivalent OSS 32-bit memory functions, such as `malloc32()` and `free32()`. For more information about the 32-bit heap, see the following section **32-Bit Heap for 64-Bit Processes**.



The default maximum heap size for 64-bit OSS processes is 12 GB. There are three ways to change the maximum heap size from 12 GB:

- At link time, using the `-set heap_max` option with the `eld` linker
- At run time:
 - `PROCESS_SPAWN[64]_tdm_spawn()`, and the `tdm_exec*()` family of OSS functions allow the maximum heap size to be specified.
 - The OSS `run` utility can accept a new `-maxheap=N` option, where *N* is a positive integer for the maximum heap size and defaults to megabytes (MB), but can be (optionally) specified with either an “MB” or “GB” (gigabytes) suffix.

64-bit and 32-bit processes can share 64-bit memory. See [Sharing Memory Between 32-Bit and 64-Bit Processes](#) for additional details.

32-Bit Heap for 64-Bit Processes

A secondary Heap Manager for 64-bit OSS processes manages space in the 32-bit addressable part of process-private virtual memory (the same address range utilized by the Heap Manager for 32-bit applications). 64-bit OSS processes can access the secondary (32-bit) heap space through the following new OSS functions:

- `calloc32()`
- `free32()`
- `malloc32()`
- `realloc32()`
- `heap_check32()`
- `heap_check_always32()`

ILP32 and LP64 Data Models

The data model currently supported by NonStop C/C++ compilers on RVUs preceding H06.24 and J06.13 is called *ILP32*, and this is the default native data model on all H-series and J-series RVUs. This name indicates that `int`, `long`, and pointer data types are 32 bits wide.

Beginning with the H06.24 and J06.13 RVUs, C/C++ also supports a 64-bit data model, called **LP64**. This name indicates that `long` and pointer data types are 64 bits wide. In this model, `int` remains 32 bits wide. The following table shows the size (in bits) for the various C/C++ data types in the ILP32 and LP64 data models.

Table 50: NonStop ILP32 and LP64 Data Models

Data Type	ILP32	LP64
char	8	8
short	16	16
int	32	32

Table Continued



Data Type	ILP32	LP64
long	32	64
long long	64	64
void *	32	64
enum	32	32
& (address of)	32 ¹	64
float	32	32
double	64	64

¹This is a 64-bit value if the operand is the address of an object referenced through a 64-bit pointer: &ptr64->field.

The default data model for native processes is ILP32. Though not explicitly named in this guide for previous RVUs, this data model has been supported since the inception of TNS/R, TNS/E, and TNS/X native processes.

For information about the C/C++ compiler options that specify the LP64 data model in the OSS, Windows, and the Guardian environment, see the *C/C++ Programmer's Guide for NonStop Systems* in the NTL.

Sharing Memory Between 32-Bit and 64-Bit Processes

Software applications can share data segments between 32-bit and 64-bit processes with these restrictions:

- Segments can be shared by *pin* and *segment-id*, or by *filename*, using the `SEGMENT_ALLOCATE_` or `SEGMENT_ALLOCATE64_Guardian` procedures. Segments can also be shared by *shmid*, using the `shm*()` set of OSS functions.

The Guardian and OSS interfaces operate on different segments; a segment created by `SEGMENT_ALLOCATE64_` cannot be shared using

`shm*()` and vice versa.

`SEGMENT_ALLOCATE[64]_` supports share-by-file and share-by-pin, which implicitly uses the same *segment-id*. `shmget()` supports share-by-*shmid*.

- When sharing a segment, the sharers are not required to share it at the same address.
- Shared segments allocated by the `shm*()` set of OSS APIs are allocated as flat segments.
- Selectable segments can be shared using `SEGMENT_ALLOCATE_` and `SEGMENT_ALLOCATE64_`, but only as selectable segments, and always at the same address.
- If the segment contains pointers, they should be defined using either the `_ptr32` or `_ptr64` keyword. If those pointers refer to objects within a shared segment, then the segment should be mapped at the same address by all sharers.
- Avoid the use of the `long` data type for declaring data items allocated within the segment, because the width of `long` is dependent on the data model (ILP32 or LP64) of the code accessing that data.

NOTE: 32-bit processes—whether OSS or Guardian—can only access 64-bit memory through flat segments that are allocated using the `SEGMENT_ALLOCATE64_` Guardian procedure.



You can use `SEGMENT_ALLOCATE64_` to share 64-bit memory segments between 32-bit and 64-bit processes. `SEGMENT_ALLOCATE64_` is similar to `SEGMENT_ALLOCATE_` and it allocates a flat segment in the 64-bit space. For information about `SEGMENT_ALLOCATE64_`, see the *Guardian Procedure Calls Reference Manual* in the NTL.

TNS and native processes can share segments in 32-bit address space, using `SEGMENT_ALLOCATE_` with either the share-by-pin or share-by-filename method. TNS/E and TNS/X 32-bit or 64-bit native processes can also use `SEGMENT_ALLOCATE64_` to share 32-bit segments. TNS, TNS/E, and TNS/X native processes cannot share `shm*()` segments with each other. On TNS/R systems, native and TNS OSS processes can share `shm*()` segments, but native Guardian processes cannot share them with OSS processes (native or TNS).

! **IMPORTANT:** Hewlett Packard Enterprise does not recommend attempting to share selectable segments among any of the following:

- Programs using the wide-memory model, such as:
 - TNS C
 - COBOL
 - FORTRAN
- TNS TAL programs that uses global or local data with .EXT indirection

For additional information about sharing selectable segments, see *Using Selectable Segments in TNS Processes* in the *Guardian Programmer's Guide*.

The following new 64-bit segment routines have been added, beginning with the H06.20 and J06.09 RVUs:

- `ADDRESS_DELIMIT64_`
- `REFPARAM_BOUNDSCHECK64_`
- `SEGMENT_ALLOCATE64_`
- `SEGMENT_GETINFO64_`
- `SEGMENT_GETINFOSTRUCT_`
- `SEGMENT_RESIZE64_`
- `SEGMENT_GET_PREF_ALIGN_`
- `SEGMENT_GET_MIN_ALIGN_`

A 32-bit process can allocate virtual memory using `SEGMENT_ALLOCATE64_`. It can then define a 64-bit pool on that memory using `POOL64_DEFINE_`, if desired, so that it can allocate and de-allocate from that memory dynamically. All access to the 64-bit segment must be done using 64-bit pointers.

64-bit pool routines can be used for dynamic allocation and de-allocation (`pool64.h`) and provide improved performance over traditional NonStop pool management. A pool need not be a contiguous block of virtual memory. The pool can be increased by allocating an additional segment and augmenting the pool to include that segment.

For more information about 64-bit pool functions, see **Pool Functions**.

C/C++ Language Features Supporting 64-Bit Addressing

Beginning with the H06.24 and J06.13 RVUs, and L-series, the C/C++ compilers hosted on Guardian, OSS, and Windows support a command-line directive to specify the data model in the resulting compilation. The syntax is slightly different between the Guardian compilers and the other compilers. For detailed information about the NonStop C/C++ compilers in the OSS, Windows, and Guardian environments, see the latest *C/C++ Programmer's Guide for NonStop Systems* in the NTL.



New 64-bit features have been added to the NonStop C/C++ compilers in the OSS, Windows, and the Guardian environments, beginning with the H06.24 and J06.13 RVUs. The following table describes these new 64-bit features. For detailed information about the NonStop C/C++ compilers in the OSS, Windows, and the Guardian environments, see the *C/C++ Programmer's Guide for NonStop Systems* in the NTL.

Table 51: NonStop C/C++ Compiler 64-Bit Features

Feature	What it Does
<code>-WlIp32</code>	OSS and Windows compiler option that specifies the ILP32 data model, in which the <code>int</code> , <code>long</code> , and pointer data types are 32 bits wide. This is the default data model.
<code>ilp32</code>	Guardian compiler option that specifies the ILP32 data model, in which the <code>int</code> , <code>long</code> , and pointer data types are 32 bits wide. This is the default data model.
<code>-Wlp64</code>	OSS and Windows compiler option that specifies the LP64 data model, in which the <code>int</code> data type is 32 bits wide, but <code>long</code> and pointer data types are 64 bits wide
<code>lp64</code>	Guardian compiler option that specifies the LP64 data model, in which the <code>int</code> data type is 32 bits wide, but <code>long</code> and pointer data types are 64 bits wide
<code>-Wmigration_check=32to64</code>	OSS and Windows compiler option that emits warning messages for valid C/C++ code that potentially can behave in an unexpected manner when code designed for ILP32 is compiled using the LP64 data model
<code>migration_check 32to64</code>	Guardian compiler option that emits warning messages for valid C/C++ code that potentially can behave in an unexpected manner when code designed for ILP32 is compiled using the LP64 data model

Table Continued



Feature	What it Does
<code>__ILP32</code>	Feature test macro that is automatically defined and set to 1 when the data model specified (explicitly or implicitly) is ILP32
<code>__LP64</code>	Feature test macro that is automatically defined and set to 1 when the data model specified is LP64
<code>_ptr32</code>	Data-type modifier (keyword) used in a pointer declaration that specifies the pointer is unconditionally 32 bits wide. See <u>New Pointer Modifiers</u> for details.
<code>_ptr64</code>	Data-type modifier (keyword) used in a pointer declaration that specifies the pointer is unconditionally 64 bits wide. See <u>New Pointer Modifiers</u> for details.
<code>_ptr64_to_ptr32()</code>	Intrinsic (builtin) that makes sure a program can safely truncate a passed 64-bit pointer to a 32-bit pointer. If the passed pointer is a sign-extended 32-bit pointer, its low-order 32 bits are returned; otherwise, a run-time trap occurs. See <u>New C/C++ builtins</u> for details.
<code>_is_32bit_addr()</code>	Intrinsic (<code>builtin.h</code>) that returns a non-zero value if the passed pointer is a sign-extended 32-bit address —returns a zero, otherwise. See <u>New C/C++ builtins</u> for details.

Table Continued



Feature	What it Does
<code>#pragma p32_shared</code>	The <code>p32_shared</code> pragma applies to class, struct, or union definitions. It indicates that within the definition, unqualified pointer declarations are 32 bit pointers—regardless of the data model. See New pragma Directives for details.
<code>#pragma p64_shared</code>	The <code>p64_shared</code> pragma applies to class, struct, or union definitions. It indicates that within the definition, unqualified pointer declarations are 64bit pointers—regardless of the data model. See New pragma Directives for details.

NOTE: The 64-bit object files containing the C/C++ `main` procedure are:

- Guardian environment:
`cmain64`
- OSS environment:
`cmain64.o`

The following CCOMP options are not allowed with LP64:

- SQL
- SYSTYPE GUARDIAN

The following CPPCOMP options are not allowed with LP64:

- VERSION2
- SYSTYPE GUARDIAN

The following c89 options are not allowed with `-Wlp64`:

- `-Wsql`
- `-Wsystype=guardian`
- `-Wversion2`



The following `c99` option is not allowed with `-Wlp64`:

```
-Wsystype=guardian
```

32-bit and 64-bit versions of DLLs are provided. (See **32-Bit and 64-Bit Public DLLs** for details.) The specified data model (ILP32 or LP64) for the C/C++ compilers and the `el` or `xld` linker causes the correct run-time DLLs to be selected if the build file uses the *shorthand* DLL name. For example, if `“-l CRE”` is in the build file, rather than `“-l ZCREDLL”`, the linker selects the appropriate 32-bit or 64-bit DLL (ZCREDLL or YCREDLL, respectively and in TNS/X systems XCREDLL or WCREDLL, respectively), depending on whether the ILP32 or LP64 data model was specified for the compilation.

32-Bit and 64-Bit Applications Constraints

Consider these constraints for 32-bit and 64-bit applications:

- When sharing memory segments between 32-bit and 64-bit processes, interpretation of bits inside the memory segment and their semantics is the application’s responsibility.
- Programs must be compiled using either the ILP32 or the LP64 data model and you cannot mix the two data models into one program.
- Migrating a 32-bit application to a 64-bit application might require source code changes.
- Migrating a 32-bit application to a 64-bit application requires minor build-script changes.
- The virtual memory available for heap data is expanded. The virtual memory available for program text, DLL text, and global data is not expanded.

! **IMPORTANT:** A 64-bit process’s stack is currently 32-bit addressable. However, this might not be true in the future; therefore, Hewlett Packard Enterprise does not recommend the use of 32-bit pointers referring to local variables in a 64-bit program.

Fallback Considerations

If no new 64-bit features were used after migrating from a preceding release to the H06.24 or J06.13 RVU, there will be no fallback impact and no application changes are required.

If 64-bit features were used in either the H06.24 or J06.13 RVU, and then a fallback to a preceding release occurs, then one of the following is required:

- Applications that were compiled in the LP64 data model must be rebuilt in the ILP32 data model.
- The previous 32-bit binaries must be used.

64-Bit Support for OSS and Guardian APIs

The 64-bit programming model supports coexistence of 32-bit and 64-bit applications on TNS/E and TNS/X systems. These applications in different data models can communicate using IPC mechanisms, including shared memory segments. TNS/E and TNS/X 32-bit non-privileged object files will continue to run without recompiling or relinking.

Changed OSS Functions

Beginning with the H06.24 and J06.13 RVUs, all OSS functions in the *Open System Services System Calls Reference Manual* and the *Open System Services Library Calls Reference Manual* have pointers and `long` data types widened to 64 bits and are callable from 64-bit OSS programs. 64-bit OSS applications can continue to use the same function name when calling these functions.



The large file model is assumed in 64-bit OSS programs. File offsets are always 64 bits. For information about large file aware APIs, see [Large File Aware APIs](#).

32-Bit and 64-Bit Public DLLs

The following public DLLs have both 32-bit and 64-bit versions that are used by 32-bit and 64-bit processes, respectively. 32-bit public DLLs begin with the letter “Z” and 64-bit public DLLs begin with the letter “Y”. On L-series systems, the public DLLs begin with the letters “X” and “W” for 32-bit and 64-bit versions, respectively.

Table 52: 32-Bit and 64-Bit Public DLLs

Product Number	Product Name	Public DLL (32-Bit/64-Bit)
T0248	XDR STREAMS	ZXDRDLL/YXDRDLL (XXDRDLL/ WXDRDLL on TNS/X systems)
T0372/T8655	NATIVE SOCKET LIB	ZINETDLL/YINETDLL
T0379	LIGHTWEIGHT RESOLVER LIBRARY	ZLWRDLL/YLWRDLL
T1069	OSS ENCRYPTION ROUTINES USA	ZOSSCDLL/YOSSCDLL
T1269	COMMON RUNTIME ENVIRONMENT	ZCREDLL/YCREDLL (XCREDLL/ WCREDLL on TNS/X systems)
T1280	POSIX USER THREADS	ZPUTDLL/YPUTDLL
T2831	C++ COMMON RTL	ZCPPCDLL/YCPPCDLL (XCPPCDLL/ WCPPCDLL on TNS/X systems)
T2833	C++ RUNTIME V3	ZCPP3DLL/YCPP3DLL (XCPP3DLL/ WCPP3DLL on TNS/X systems)
T6572	ICONV TABLES	ZICNVDLL/YICNVDLL
T7956	OSSFSLIB II	ZOSSHDLL/YOSSHDLL
T8160	OSS SUBJECT API	ZSECDLL/YSECDLL
T8372	I18N LOCALES	ZI18NDLL/YI18NDLL
T8432	C RUNTIME	ZCRTLDLL/YCRTLDLL (XCRTLDLL/ WCRTLDLL on TNS/X systems)
T9050	NONSTOP KERNEL	ZOSSKDLL/YOSSKDLL
T9228	FILSYS LIBRARIES	ZOSSFDLL/YOSSFDLL

Table Continued



Product Number	Product Name	Public DLL (32-Bit/64-Bit)
T9627	OSSFSLIB	ZOSSEDLL/YOSSEDLL
T9661	MC RUNTIME LIBRARY	ZUTILDLL/YUTILDLL (XUTILDLL/ WUTILDLL on TNS/X systems)

Neutral Public DLLs

The following public DLLs are data-model neutral. These neutral public DLLs can be used by both 32-bit and 64-bit processes.

Table 53: Data-Model-Neutral Public DLLs

Product Number	Product Name	Public DLL (32-Bit/64-Bit)
T0428	RLD-Native PIC Run-time Linker	ZRLDDLL
T0694	CIPMON	ZCIPDLL
T0747	TNS/E Code Profiling Utilities	ZPGODLL
T0747	TNS/X Code Profiling Utilities	XPGODLL
T1264	TCP/IPV6 Subsystem	ZTCP6DLL

SQL/MX

For the H06.24 and J06.13 RVUs, SQL/MX does not currently support 64-bit OSS processes. Please refer to SQL/MX documentation for the availability of 64-bit support.

SQL/MP is not supported for 64-bit OSS processes.

New OSS Function in YPUTDLL

A new `PUT_FILE_WRITEREAD_()` function is added to the YPUTDLL 64-bit PUT Library. This new function is used as a wrapper routine for the `FILE_WRITEREAD_` and `FILE_WRITEREAD64_` Guardian procedures. For information about `PUT_FILE_WRITEREAD_()`, see the *Open System Services Library Calls Reference Manual* in the NTL.

For information about thread-aware Enscribe functions, see **Table 73: Thread-Aware Enscribe Functions**.

New Guardian Procedures and OSS Functions

New Guardian Procedures with 64-Bit Functionality

The following new Guardian procedures have 64-bit functionality, beginning with the H06.24 and J06.13 RVUs (except for the POOL64* procedures, which were released in the H06.20 and J06.09 RVUs).

`ADDRESS_DELIMIT64_`
`DNUMIN64_`
`FILE_AWAITIO64_`
`FILE_CONTROL64_`
`FILE_CONTROLBUF64_`



FILE_LOCKFILE64_
 FILE_LOCKREC64_
 FILE_READ64_
 FILE_READLOCK64_
 FILE_READUPDATE64_
 FILE_READUPDATELOCK64_
 FILE_REPLY64_
 FILE_SETMODENOWAIT64_
 FILE_UNLOCKFILE64_
 FILE_UNLOCKREC64_
 FILE_WRITE64_
 FILE_WRITEREAD64_
 FILE_WRITEUPDATE64_
 FILE_WRITEUPDATEUNLOCK64_
 FILENAME_FINDNEXT64_
 POOL64_AUGMENT_
 POOL64_CHECK_
 POOL64_CHECK_SHRINK_
 POOL64_DEFINE_
 POOL64_DIMINISH_
 POOL64_GET_
 POOL64_GETINFO_
 POOL64_PUT_
 POOL64_RESIZE_
 REFPARAM_BOUNDSCHECK64_
 SEGMENT_ALLOCATE64_
 SEGMENT_GET_MIN_ALIGN_
 SEGMENT_GET_PREF_ALIGN_
 SEGMENT_GETINFO64_
 SEGMENT_GETINFOSTRUCT_
 SEGMENT_RESIZE64_

New OSS Functions

The following new OSS functions are available, beginning with the H06.24 and J06.13 RVUs.

calloc32()
 free32()
 heap_check32()
 heap_check_always32()
 malloc32()
 memchr64()
 memcmp64()
 memcpy64()
 memmove64()
 memset64()
 PUT_BEGINTRANSACTION_EXT_
 PUT_FILE_WRITEREAD_
 PUT_SERVERCLASS_DIALOG_BEGINL_
 PUT_SERVERCLASS_DIALOG_SENDL_
 read64_
 realloc32()



```

recv64_()
recvfrom64_()
recvmsg64_()
send64_()
sendmsg64_()
sendto64_()
SPT_BEGINTRANSACTION_EXT_()
SPT_SERVERCLASS_DIALOG_BEGINL_()
SPT_SERVERCLASS_DIALOG_SENDL_()
strcat64()
strchr64()
strcmp64()
strcpy64()
strlen64()
strncat64()
strncmp64()
strncpy64()
strrchr64()
strstr64()
write64()

```

Guardian Procedures Callable from 64-Bit OSS Processes

A large number of Guardian procedures can be called from 64-bit OSS processes. Additionally, a new set of Enscribe I/O APIs has been created with the following characteristics:

- Directly return a file management error
- Accept 64-bit buffers
- Have 64-bit no-wait tags so that 64-bit addresses can be used as tags

The following Guardian procedures can be called from 64-bit OSS processes:

```

ACTIVATERECEIVETRANSID
ADDRESS_DELIMIT64_
BEGINTRANSACTION
BEGINTRANSACTION_EXT_
BINSEM_CLOSE_
BINSEM_CREATE_
BINSEM_FORCELOCK_
BINSEM_LOCK_
BINSEM_OPEN_
BINSEM_UNLOCK_
CANCELREQL
CHANGELIST
CLOSE
COMPUTEJULIANDAYNO
COMPUTETIMESTAMP
COMPUTETRANSID
CONTIME
CONTROLMESSAGESYSTEM
CONVERTASCIIEBCDIC

```



CONVERTPROCESSTIME
 CONVERTTIMESTAMP
 CPUTIMES
 DAYOFWEEK
 DEBUG
 DEFINE_MODE_
 DEFINEADD
 DEFINEINFO
 DEFINEREADATTR
 DEFINESATTR
 DEFINESETLIKE
 DNUMIN64_
 DNUMOUT
 DST_GETINFO_
 DST_TRANSITION_ADD_
 DST_TRANSITION_DELETE_
 DST_TRANSITION_MODIFY_
 EMSADDSUBJECT
 EMSADDSUBJECTMAP
 EMSADDTOKENMAP
 EMSADDTOKENS
 EMSINIT
 EMSINITMAP
 ERRNO_GET_
 FILE_AWAITIO64_
 FILE_CONTROL64_
 FILE_CONTROLBUF64_
 FILE_COMPLETE_GETINFO_
 FILE_COMPLETE_SET_
 FILE_COMPLETEL_
 FILE_LOCKFILE64_
 FILE_LOCKREC64_
 FILE_CLOSE_
 FILE_CREATE_
 FILE_CREATELIST_
 FILE_GETINFO_
 FILE_GETINFOBYNAME_
 FILE_GETINFOLIST_
 FILE_GETINFOLISTBYNAME_
 FILE_GETLOCKINFO_
 FILE_GETRECEIVEINFO_
 FILE_GETRECEIVEINFOL_
 FILE_OPEN_
 FILE_READ64_
 FILE_READLOCK64_
 FILE_READUPDATE64_
 FILE_READUPDATELOCK64_
 FILE_RENAME_
 FILE_REPLY64_
 FILE_SETKEY_



FILE_SETLASTERROR_
 FILE_SETMODENOWAIT64_
 FILE_SETPOSITION_
 FILE_UNLOCKFILE64_
 FILE_UNLOCKREC64_
 FILE_WRITE64_
 FILE_WRITEREAD64_
 FILE_WRITEUPDATE64_
 FILE_WRITEUPDATEUNLOCK64_
 FILENAME_COMPARE_
 FILENAME_DECOMPOSE_
 FILENAME_EDIT_
 FILENAME_FINDFINISH_
 FILENAME_FINDNEXT64_
 FILENAME_FINDSTART_
 FILENAME_RESOLVE_
 FILENAME_TO_OLDFILENAME_
 FILENAME_TO_PATHNAME_
 FILENAME_TO_PROCESSHANDLE_
 FNAMECOLLAPSE
 FNAMEEXPAND
 FP_IEEE_DENORM_GET_
 FP_IEEE_DENORM_SET_
 FP_IEEE_ENABLES_GET_
 FP_IEEE_ENABLES_SET_
 FP_IEEE_ENV_CLEAR_
 FP_IEEE_ENV_RESUME_
 FP_IEEE_EXCEPTIONS_GET_
 FP_IEEE_EXCEPTIONS_SET_
 FP_IEEE_ROUND_GET_
 FP_IEEE_ROUND_SET_
 GETCPCBINFO
 GETSYSTEMSERIALNUMBER
 GETTMPNAME
 GETTRANSID
 GETTRANSINFO
 HALTPOLL
 HEADROOM_ENSURE_
 HIST_FORMAT_
 HIST_GETPRIOR_
 HIST_INIT_
 JULIANTIMESTAMP
 INTERPRETINTERVAL
 INTERPRETJULIANDAYNO
 INTERPRETTIMESTAMP
 INTERPRETTRANSID
 IPUAFFINITY_CONTROL_
 IPUAFFINITY_GET_
 IPUAFFINITY_SET_
 LABELEDTAPESUPPORT



MBCS_CHAR_
 MBCS_TESTBYTE_
 MESSAGESYSTEMINFO
 MONITORCPUS
 MONITORNET
 MONITORNEW
 MYPROCESSTIME
 MYSYSTEMNUMBER
 NODE_GETCOLDLOADINFO_
 NODENAME_TO_NODENUMBER_
 NODENUMBER_TO_NODENAME_
 NSK_FLOAT_IEEE32_TO_TNS32_
 NSK_FLOAT_IEEE64_TO_TNS32_
 NSK_FLOAT_IEEE64_TO_TNS64_
 NSK_FLOAT_TNS32_TO_IEEE32_
 NSK_FLOAT_TNS32_TO_IEEE64_
 NSK_FLOAT_TNS64_TO_IEEE64_
 OLDFILENAME_TO_FILENAME_
 OPEN
 OSS_PID_NULL_
 PATHNAME_TO_FILENAME_
 PROCESS_ACTIVATE_
 PROCESS_CREATE_
 PROCESS_DELAY_
 PROCESS_DEBUG_
 PROCESS_FINALIZE_
 PROCESS_GETINFO_
 PROCESS_GETINFOLIST_
 PROCESS_GETPAIRINFO_
 PROCESS_LAUNCH_
 PROCESS_SETINFO_
 PROCESS_SETSTRINGINFO_
 PROCESS_STOP_
 PROCESS_SUSPEND_
 PROCESSHANDLE_COMPARE_
 PROCESSHANDLE_DECOMPOSE_
 PROCESSHANDLE_GETMINE_
 PROCESSHANDLE_NULLIT_
 PROCESSHANDLE_TO_FILENAME_
 PROCESSHANDLE_TO_STRING_
 PROCESSNAME_CREATE_
 PROCESSOR_GETINFOLIST_
 PROCESSOR_GETNAME_
 PROCESSORSTATUS
 PROCESSORTYPE
 RAISE_
 REFPARAM_BOUNDSCHECK64_
 REMOTEPROCESSORSTATUS
 REMOTETOSVERSION
 SEGMENT_ALLOCATE64_



SEGMENT_DEALLOCATE_
 SEGMENT_GETINFO64_
 SEGMENT_GETINFOSTRUCT_
 SEGMENT_GET_MIN_ALIGN_
 SEGMENT_GET_PREF_ALIGN_
 SEGMENT_RESIZE64_
 SEGMENT_USE_
 SERVERCLASS_SEND_
 SERVERCLASS_DIALOG_BEGIN_
 SERVERCLASS_DIALOG_SEND_
 SERVERCLASS_SEND_INFO_
 SERVERCLASS_SENDL_
 SETLOOPTIMER
 SETSTOP
 SETSYSTEMCLOCK
 SIGACTION_INIT_
 SIGACTION_RESTORE_
 SIGACTION_SUPPLANT_
 SIGSETJMP_
 SPOOLCONTROL
 SPOOLEND
 SPOOLSETMODE
 SPOOLSTART
 SPOOLWRITE
 SSINIT
 SSPUT
 STACK_ALLOCATE_
 STACK_DEALLOCATE_
 STATUSTRANSACTION
 STRING_UPSHIFT_
 SYSTEMENTRYPOINT_RISC_
 SYSTEMENTRYPOINTLABEL
 TEXTTOTRANSID
 TIME
 TIMER_START_
 TIMER_STOP_
 TIMESTAMP
 TMF_BEGINTAG_FROM_TXHANDLE_O
 TMF_GET_TX_ID_O
 TMF_GET_EXTTRANSID_O
 TMF_GETTXHANDLE_O
 TMF_SETTXHANDLE_O
 TMF_SUSPEND_O
 TMF_SUSPEND_EXT_O
 TMF_TXBEGIN_O
 TMF_TXHANDLE_FROM_BEGINTAG_O
 TMF_VERSION_O
 TRANSIDTOTEXT
 TS_NANOSECS_
 TS_UNIQUE_COMPARE_



TS_UNIQUE_CONVERT_TO_JULIAN_
TS_UNIQUE_CREATE_
VRO_SET_

NOTE: Not all of the NonStop Kernel (NSK) Guardian procedures have been converted to accept 64-bit pointers; therefore, 64-bit OSS programs need to use 32-bit pointers to access NSK Guardian procedures that are not in the above list. The 32-bit heap is used for this purpose. See **32-Bit Heap for 64-Bit Processes** for details of a 64-bit OSS process using the 32-bit heap.

OSS Functions Callable from 32-bit Processes Using 64-bit Pointers

Beginning with the H06.24 and J06.13 RVUs, enhancements have been made so that native Guardian and 32-bit OSS processes on TNS/E systems can access 64-bit address space, such as performing OSS file and socket I/O operations to and from 64-bit memory segments.



IMPORTANT:

All of the following OSS functions can also be called from 64-bit OSS processes.

The following OSS functions can be called from native Guardian and 32-bit OSS processes on TNS/E systems using 64-bit pointers:

```
memchr64 ()  
memcmp64 ()  
memcpy64 ()  
memmove64 ()  
memset64 ()  
read64_ ()  
recv64_ ()  
recvfrom64_ ()  
recvmsg64_ ()  
send_nw2_64_ ()  
send64_ ()  
sendmsg64_ ()  
sendto64_ ()  
strcat64 ()  
strchr64 ()  
strcmp64 ()  
strcpy64 ()  
strlen64 ()  
strncat64 ()  
strncmp64 ()  
strncpy64 ()  
strrchr64 ()  
strstr64 ()  
write64_ ()
```

32-Bit and 64-Bit Object Files

This section provides considerations for 32-bit and 64-bit object files, beginning with the H06.24 and J06.13 RVUs.



Linker Object Files

In NonStop TNS/E and TNS/X systems, 32-bit object files use the ELF64 format, where all addresses in the ELF structures occupy 64 bits. The C/C++ compilers and tools categorize binaries in one of the following ways:

- 32 bit
- 64 bit
- Neutral

OSS and Guardian linkfiles can be linked together to create either an OSS or a Guardian loadfile. You can have Guardian modules in an OSS loadfile and vice versa. 64-bit applications can link with 64-bit public DLLs and neutral DLLs. 32-bit applications can link with 32-bit public DLLs and neutral DLLs.

ENOFT has been enhanced to display the data model.

```
enoft> filehdr
```

```
***** ELF File Header
```

```
Ident:                ELF64-bit Big_Endian NSK
Type:                 Linkfile
Target Machine:       IA64
...
Flags:                (0x830000)
                     NO_SHARED_GLOBALIZED
                     Neutral DATA MODEL
```

Data Models for Object Files

Beginning with the H06.24, J06.13 RVUs, and L-series RVUs, the `-set` option for the `eld` and `xld` linker has a new attribute named `data_model`. The values allowed for this option are:

- `ilp32` (Creates an object file that uses the ILP32 data model. All existing object files have this data model.)
- `lp64` (Creates an object file that uses the LP64 data model.)
- `neutral` (Creates an object file with an unspecified data model.)

By default, when `eld` or `xld` links together a set of linkfiles:

- An error occurs if the set of input linkfiles contains both ILP32 and LP64 linkfiles.
- The data model of the output file is ILP32 if the input linkfiles are ILP32 linkfiles.
- The data model of the output file is LP64 if the input linkfiles are LP64 linkfiles.

If the `-set data_model ilp32` (or `lp64`) option is given, then an error occurs if any of the input linkfiles were compiled in the opposite data model. For example, if the `-set data_model ilp32` is specified, you cannot input LP64 linkfiles.

If the `-set data_model neutral` option is specified, then the output file is always marked neutral, and the input linkfiles can contain both ILP32 and LP64 linkfiles. This is allowed when `eld` or `xld` is building a linkfile or a DLL; however, a program can never be neutral.

The `-set data_model neutral` option is always imposed if `eld` or `xld` is building an implicit DLL. See the *eld and xld Manual* in the NTL for additional details of the `-set data_model` option



The `eld` and `xld` linkers attempt to find the public DLL registry file whenever they are creating a loadfile. The public DLL registry file (`zreg` file) is built by the Public DLL Installation Tool (PLINSTL). PLINSTL is run on NonStop systems by DSM/SCM whenever a new version of a public DLL is installed or when an implicit DLL is rebuilt. All of the 32-bit, 64-bit, and neutral public DLL's are listed in the same `zreg` file. See the *DLL Programmer's Guide for TNS/E and TNS/X Systems* in the NTL for additional details about finding DLLs on TNS/E and TNS/X systems.

DLLs can be 32-bit, 64-bit or Neutral. The rules for loading 32-bit, 64-bit, and neutral DLLs are:

- A 64-bit DLL may only be loaded into a 64-bit process.
- A 32-bit DLL may only be loaded into a 32-bit process.
- A Neutral DLL may be loaded into either a 32-bit or a 64-bit process.
 - Neutral DLLs may depend only on other neutral DLLs.
 - The Implicit DLLs are neutral.
 - The Networking DLLs loaded by QIO are neutral.

See **Neutral Public DLLs** for the list of neutral DLLs.

A *DLL type mismatch* occurs when the linker searches for a 32-bit or 64-bit DLL, and the linker encounters a DLL of the opposite model. In this case:

1. The linker will:
 - a. Issue a warning that a DLL of the incorrect model was found
 - b. Remember the location of the incorrect DLL
 - c. Continue searching
2. If a DLL of the correct model (or a neutral DLL) is subsequently found, that DLL is used. If a correct or neutral DLL is not found, the linker issues a second warning and attempts to resolve symbols using the incorrect DLL. This is done under the assumption that a DLL of the correct model might still be found by the run-time loader at run time.
3. If the run-time loader finds a DLL of the opposite model, it ignores that file and continues searching.

Mixed Data-Model Programming

A program that contains both 32-bit and 64-bit pointers is a *mixed data-model program*. The use of the `_ptrnn` modifiers and the `p64_shared` pragma results in a mixed data-model program. For information about `_ptr32` and `_ptr64`, see **New Pointer Modifiers**.

Mixed data-model programming uses one of the following methods:

- The ILP32 data model with explicit 64-bit pointers to access data outside of the 32-bit addressable range
- The LP64 data model with explicit 32-bit pointers to access data in the 32-bit addressable range

For example, access to 64-bit memory segments from native Guardian and 32-bit OSS processes on TNS/E or TNS/X systems requires mixed data-model programming.

! **IMPORTANT:** A 32-bit process cannot use a 64-bit address to call a 32-bit function or procedure. For example, a 32-bit process cannot use a 64-bit pointer to call `recvmsg()`. Instead, the 32-bit process must use a 64-bit pointer and call `recvmsg64()`.

See **OSS Functions Callable from 32-bit Processes Using 64-bit Pointers** for the 64-bit functions that can be called from 32-bit applications.

New Pointer Modifiers

The majority of applications are built using either the ILP32 or the LP64 data model. The entire set of operating system APIs are not available in a 64-bit variant, and there is limited support for creating 32-bit pointers in the LP64 data model and 64-bit pointers in the ILP32 data model.

New pointer modifiers are supported that explicitly specify the size of a pointer. The `_ptr64` modifier is used to declare a 64-bit pointer and the `_ptr32` modifier is used to declare a 32-bit pointer. These modifiers can be used in both the ILP32 data model and the LP64 data model.

Examples of `_ptr64` and `_ptr32` Pointer Modifiers

```
char _ptr64 *Ptr1;    /* Ptr1 is a 64-bit pointer to a character. */
char _ptr32 *Ptr2;    /* Ptr2 is a 32-bit pointer to a character. */
```

New pragma Directives

There is limited support for the combining of modules compiled with different data models into a single loadfile. This feature is specifically limited to code that is intended for a data-model-neutral DLL. This feature is not allowed for user-level applications. It is the programmer's responsibility to insure that declarations and definitions are consistent between separately compiled modules that are to be combined. Two new pragmas are provided to aid in this.

NOTE: No link-time support is available that detects potential mismatches.

`#pragma p32_shared`

The new `#pragma p32_shared` pragma applies to class, struct, or union definitions. It indicates that within the definition, unqualified pointer declarations are 32 bit pointers—regardless of the data model. For C, this only affects pointers declared as fields. For C++, this also applies to pointers declared (implicitly or explicitly) for member function parameters (including the implicit `this` parameter) and for any pointers used in class and member function definitions.

It also disallows components whose layout or size are not data-model invariant. This means that `long` is disallowed as a component. Any class, struct, or union component must be declared as either `p32_shared` or `p64_shared`. Any base class of a `p32_shared` class must also be a `p32_shared` class.

`#pragma p64_shared`

The new `#pragma p64_shared` pragma applies to class, struct, or union definitions. It indicates that within the definition, unqualified pointer declarations are 64 bit pointers—regardless of the data model. For C, this only affects pointers declared as fields. For C++, this also applies to pointers declared (implicitly or explicitly) for member function parameters (including the implicit `this` parameter) and for any pointers used in class and member function definitions.

It also disallows components whose layout or size are not data-model invariant. This means that `long` is disallowed as a component. Any class, struct, or union component must be declared as either `p32_shared` or `p64_shared`. Any base class of a `p64_shared` class must also be a `p64_shared` class.

Mixed Data-Model Prototypes in `cextdecs.h`

For the `cextdecs.h` file, only prototypes that are supported for mixed data-model use (for example, a 32-bit process using 64-bit pointers) have pointers with the `_ptr64` modifier. Most `cextdecs.h` prototypes do not have mixed data-model support, and their unqualified pointers default to the pointer width for the data model used at compile time—either ILP32 or LP64.



NOTE:

Procedures not modified to support 64-bit pointers (for example, the DEFINESAVE procedure) have `_ptr32` in their prototypes.

Mixed Data-Model Programming Example

The following code is an example of mixed data-model programming. This code is an example of a 64-bit application needing to call a routine that is a 32-bit-only API. The prototype declarations have been extracted from `cextdecs.h`.

Mixed Data-Model Programming Example

```
#pragma section definesave
#if ( _TANDEM_ARCH_ > 1)
_tal _extensible short DEFINESAVE (
    const char _ptr32 *, /* IN */
    short _ptr32 *,      /* OUT */
    short ,              /* IN */
    short _ptr32 *,      /* OUT */
    short                /* IN */
);
#else
_tal _extensible short DEFINESAVE (
    const char _far *,   /* IN */
    short _far *,        /* OUT */
    short ,              /* IN */
    short _far *,        /* OUT */
    short                /* IN */
);
#endif

/* This is where the program text starts */
#include <cextdecs.h(DEFINESAVE)>
...
struct savedDef {
    char name[24];
    short length;
    short data[1024];
}

struct savedDef _ptr32 * savedDefine;
short result;

savedDefine =
    (struct savedDef _ptr32 *)malloc32( sizeof savedDef );

if ( ! savedDefine) {
...
}

strcpy( savedDefine->name, "MYDEFINE" );

result = DEFINESAVE( savedDefine->name,
                    savedDefine->data,
                    1024,
                    &savedDefine->length );
```

```

if ( result != 0 ) {
    ...
}

```

New C/C++ builtins

`_is_32bit_addr`

Mixed data-model programming often requires testing a 64-bit address to determine if it is a sign-extended 32-bit address. So that there is a uniform way for doing that, this new builtin is provided.

```
int _is_32bit_addr( void _ptr64 * );
```

The `_is_32bit_addr` function returns a non-zero value if the passed pointer is a sign-extended 32-bit address. Otherwise, a zero is returned.

`_ptr64_to_ptr32`

This new builtin makes sure a program can safely truncate a 64-bit pointer to a 32-bit pointer.

```
void _ptr32 * _ptr64_to_ptr32( void _ptr64 * );
```

If the passed pointer is a sign-extended 32-bit pointer, its low-order 32 bits are returned. Otherwise, a run-time trap occurs.

New OSS String Manipulation Functions

The `string.h` file contains prototypes for new string manipulation functions that accept 64-bit pointers and data lengths. These functions return pointers that are also 64-bits wide.

- `memchr64()`
- `memcmp64()`
- `memcpy64()`
- `memmove64()`
- `memset64()`
- `strcat64()`
- `strchr64()`
- `strcmp64()`
- `strcpy64()`
- `strlen64()`
- `strncat64()`
- `strrchr64()`
- `strncmp64()`
- `strncpy64()`
- `strstr64()`

See the *Open System Services Library Calls Reference Manual* in the NTL for details of these new functions.



File and Socket I/O APIs 32-Bit Support for 64-Bit Segments

Beginning with the H06.24 and J06.13 RVUs, native Guardian and OSS processes on TNS/E systems and beginning with L-series on TNS/X systems, can perform OSS file and socket I/O operations directly to and from 64-bit memory segments. Both native Guardian and OSS processes can call the following functions to perform I/O directly to and from a 64-bit memory segment.

OSS Interfaces

- `read64_()`
- `recv64_()`
- `recvfrom64_()`
- `recvmsg64_()`
- `send64_()`
- `sendmsg64_()`
- `sendto64_()`
- `write64_()`

Guardian Socket Interfaces

- `recv64`
- `recvfrom64_`
- `recvmsg64_`
- `recv_nw64_`
- `recvfrom_nw64_`
- `t_recvfrom_nw64_`
- `send64_`
- `sendmsg64_`
- `sendto64_`
- `send_nw64_`
- `sendmsg_nw64_`
- `sendto_nw64_`
- `t_sendto_nw64_`

NOTE:

Both 32-bit and 64-bit callers can pass 32-bit pointers and 64-bit pointers to these interfaces. The passed 32-bit pointers are automatically sign-extended to 64 bits wide. The passed 64-bit pointers are used as is.

For information about 64-bit memory segments, see [**Sharing Memory Between 32-Bit and 64-Bit Processes**](#).



64-Bit Data Types and Data Structures Changes

Changes have been made to fundamental data types and data structures for the LP64 data model, beginning with the H06.24 and J06.13 RVUs.

64-Bit Data Types Changes

The following fundamental data types have been changed for the LP64 data model.

- **gid_t** - For LP64, the **gid_t** type in `/usr/include/sys/types.h` is changed from **long** to **int**.

```
#ifdef __LP64
    typedef int gid_t;
#else
    typedef long gid_t;
#endif
```
- **in_addr_t** - For LP64, the **in_addr_t** type in `/usr/include/netdb.h` is changed from **unsigned long** to **unsigned int**.

```
#ifdef __LP64
    typedef unsigned int in_addr_t;
#else
    typedef unsigned long in_addr_t;
#endif
```
- **mode_t** - For LP64, the **mode_t** type in `/usr/include/sys/types.h` is changed from **unsigned long** to **unsigned int**.

```
#ifdef __LP64
    typedef unsigned int mode_t;
#else
    typedef unsigned long mode_t;
#endif
```
- **uid_t** - For LP64, the **uid_t** type in `/usr/include/sys/types.h`, is changed from **long** to **int**.

```
#ifdef __LP64
    typedef int uid_t;
#else
    typedef long uid_t;
#endif
```
- **size_t** - For LP64, the **size_t** type in `/usr/include/sys/types.h`, is changed from **unsigned int** to **unsigned long**.

```
#ifdef __LP64
    typedef unsigned long size_t;
#else
    typedef unsigned int size_t;
#endif
```
- **ssize_t** - For LP64, the **ssize_t** type in `/usr/include/sys/types.h`, is changed from **int** to **long**.

```
#ifdef __LP64
    typedef long ssize_t;
#else
```



```
typedef int ssize_t;
#endif
```

- `useconds_t` – For LP64, the `useconds_t` type in `/usr/include/sys/types.h`, is changed from `unsigned int` to `unsigned long`.

```
#ifdef __LP64
typedef unsigned long useconds_t;
#else
typedef unsigned int useconds_t;
#endif
```

OSS Functions with New `socklen_t` Data Type

The `socklen_t` data type has been added and arguments for socket functions that previously used the `size_t` type now use the `socklen_t` type, which is `unsigned int` in both the ILP32 and LP64 data models. The following functions need the `socklen_t` cast (to suppress a warning) for LP64 compilations; there is no change for ILP32.

- `bind()`
- `connect()`
- `setsockopt()`

Here is an LP64 example for the `bind()` function:

```
if (bind(fd, (const struct sockaddr *)&addr, (socklen_t)sizeof(addr)) < 0 )
...
```

However, the following functions need more than a cast:

- `accept()`
- `getpeername()`
- `getsockname()`
- `getsockopt()`

These functions require a change in data type, such as the following `getsockname()` example.

getsockname() Data Type Change for LP64

In this example, the current ILP32 code is:

```
size_t length;
struct sockaddr addr;

getsockname( fd, &addr, &length );
```

For LP64, the ILP32 code would be changed as follows:

```
socklen_t length;
struct sockaddr addr;

getsockname( fd, &addr, &length );
```

Note that the LP64 code would compile and work correctly with the ILP32 data model.

For information about these socket functions, see the *Open System Services System Calls Reference Manual* in the NTL.



64-Bit Data Structures Changes and Additions

Beginning with the H06.24 and J06.13 RVUs, the following data structures have been changed. Some of the structures are changed only for the LP64 data model, which is specified at compile time. For listings of these data structures, see the corresponding `.h` file.

- `fd_set` structure – The `fds_bits` array type in the `fd_set` structure in `/usr/include/sys/time.h` is changed from `long` to `int`.
- `flock` structure – For LP64, the `l_sysid` field type in the `flock` structure in `/usr/include/fcntl.h` is changed from `long` to `int`.
- `flock64` structure – For LP64, the `l_sysid` field type in the `flock64` structure in `/usr/include/fcntl.h` is changed from `long` to `int`.
- `in6_addr` structure – For LP64, the `sa6_laddr` field type in the `in6_addr` structure in `/usr/include/netinet/in6.h` is changed from `unsigned long` to `unsigned int`.
- `sockaddr_in6` structure – For LP64, the `sin6_flowinfo` and `sin6_scope_id` field types in the `sockaddr_in6` structure in `/usr/include/netinet/in6.h` are changed from `unsigned long` to `unsigned int`.
- `process_extension` structure – For LP64, there are changes to the `process_extension` structure in `/usr/include/tdmext.h` to allow passing 64-bit pointers and specifying 64-bit values for the maximum heap size.
- `process_launch_parms` structure – For LP64, there are changes to provide a new 64-bit version of the `process_launch_parms` structure in `/usr/include/dlaunch.h`.
- `stack_t` and `sigaction` structures – For LP64, there are changes to the `stack_t` and `sigaction` structures in `/usr/include/signal.h` to allow for 64-bit pointers.
- `NSK_stack_t` and `ucontext_t` structures – For LP64, there are changes to the `NSK_stack_t` and `ucontext_t` structures in `/usr/include/tdmsig.h` to allow for 64-bit pointers.

The following 64-bit data structures have been added, beginning with the H06.24 and J06.13 RVUs.

- `iovec64` structure – This new `iovec64` structure has the same fields as the `iovec` structure. This structure is in `/usr/include/sys/uio.h` and is used by the new `msghdr64` structure.
- `msghdr64` structure – This new `msghdr64` structure has the same fields as the `msghdr` structure. This structure is in `/usr/include/sys/socket.h` and is used by the new `sendmsg64()` and `recvmsg64()` functions.
- `stat` structure – This new `stat` structure has the same fields as the `stat64` structure, with the exception of 8-byte time fields. This structure is in `/usr/include/sys/stat.h`.
- `statvfs` structure – This new `statvfs` structure has the same fields as the `statvfs64` structure, with the exception of `unsigned long` fields. This structure is in `/usr/include/sys/statvfs.h`.

64-Bit OSS Callers of HIST_INIT_

64-bit OSS callers of `HIST_INIT_` must use the new 64-bit version of the `NSK_histWorkspace` data structure and must pass `HistVersion3` for the `version` parameter. For information about 64-bit OSS process calls to `HIST_INIT_`, see the *Guardian Procedure Calls Reference Manual* in the NTL.



New 64-Bit Data Structures for Process Creation

Some data structures contain `#ifdef _LP64` inside the data structure and around certain fields. This can cause the fields to be different sizes (and potentially different offsets) between ILP32 and LP64 compilations. Additionally, some data structures are a different size between ILP32 and LP64 compilations.

New 64-Bit Data Structures for Guardian Process Creation

The `PROCESS_LAUNCH_` procedure creates only Guardian processes; however, it can be called from a native Guardian process, a 32-bit OSS process, or a 64-bit OSS process on TNS/E and TNS/X systems.

For 64-bit callers, the *param-list* argument specifies the address of either the `zsys_ddl_plaunch_parms_def` data structure or the new `zsys_ddl_plaunch_parms64_def` 64-bit data structure that contains all of the input fields for `PROCESS_LAUNCH_`. The `zsys_ddl_plaunch_parms64_def` data structure should be used by 64-bit applications.

This new `zsys_ddl_plaunch_parms64_def` 64-bit data structure is in `ZSYSDDL` and related files (for example, `zsysc`).

For information about the `PROCESS_LAUNCH_` procedure, see the *Guardian Procedure Calls Reference Manual* in the NTL.

New 64-Bit Data Structures for OSS Process Creation

The `PROCESS_SPAWN[64]` Guardian procedure and the `tdm_exec*`() and `tdm_spawn*`() sets of OSS functions create OSS processes. New 64-bit data structures are provided for 64-bit callers. Two examples of new 64-bit data structures are `zsys_ddl_processextension64_def` (in `ZSYSC`) and `process_extension` (in `tdmext.h`).

By default, 32-bit callers to `PROCESS_SPAWN[64]` and these OSS functions get a `process_extension` data structure that only allows specifying a 32-bit value for the maximum heap size. 64-bit callers get the 64-bit version of the `process_extension` data structure. 32-bit C/C++ callers can compile using the `#define _PROCEX32_64BIT 1` feature test macro, or an equivalent compiler command option, to get the 64-bit version of the `process_extension` data structure that allows specifying larger values for maximum heap size when creating a 64-bit process.

NOTE: `_PROCEX32_64BIT` is only used in `tdmext.h`; it is not in `ZSYSC`. Also, the parameters to `PROCESS_SPAWN[64]` have not been widened to 64 bits. All pointer parameters to `PROCESS_SPAWN[64]` must be in 32-bit addressable space.

For information about the `PROCESS_SPAWN[64]` procedure, see the *Guardian Procedure Calls Reference Manual* in the NTL. For information about the sets of `tdm_exec*`() and `tdm_spawn*`() OSS functions, see the *Open System Services System Calls Reference Manual* in the NTL.

Converting 32-Bit Applications to 64-Bit Applications

The advantages of 64-bit applications are:

- The string-manipulation and file and socket I/O APIs available for 32-bit applications that access 64-bit data segments are very limited. There are many more string-manipulation and file and socket I/O APIs available for 64-bit applications, which access 64-bit data segments. See **[New OSS String Manipulation Functions](#)** and **[File and Socket I/O APIs 32-Bit Support for 64-Bit Segments](#)** for details of the available string-manipulation and file and socket I/O APIs available for 64-bit data segments in 32-bit applications.
- Access to 64-bit data segments from 32-bit applications requires the use of proprietary APIs. Porting an existing 64-bit application would be much easier than converting the application to the ILP32 data model with 64-bit segment access.



The following topics in this section provide the required steps to convert 32-bit applications to 64-bit applications.

Step 1: Identify Programs that Need to be Compiled in the LP64 Data Model

The first step in converting 32-bit applications to 64-bit applications is to identify the code that would benefit from 64-bit computing. See **Features of 64-Bit Support** for the advantages of 64-bit applications.

64-bit main programs can only call 64-bit DLLs or neutral DLLs. Therefore, DLL providers for 64-bit applications must transition their DLLs to the LP64 or neutral data model.

Step 2: Identify Non-Portable Constructs

You can use the following tools to find non-portable code when transitioning 32-bit applications to the LP64 data model:

- NonStop C/C++ compilers
- CodeCheck – developed by ABRAXAS Software
- Open Systems Portability Checker (OSPC) – developed by Knowledge Software
- FlexeLint – developed by Gimpel Software

Step 3: Using the C/C++ Compiler to Identify Needed Source Code Changes

The C/C++ compiler has the ability to detect a number of 64-bit porting issues. For some issues, the compiler emits a diagnostic by default. For example, any direct assignment from a 64-bit pointer to a 32-bit pointer results in the error:

```
error(611): a value of type "long-pointer-type"
           " cannot be
assigned to an entity of type "short-pointer-type"
           "
```

In addition, the C/C++ compiler provides a feature to enable additional warnings to be emitted. These warnings detect valid C/C++ code that potentially can behave in an “unexpected” manner when code designed for the ILP32 data model is compiled using the LP64 data model. This feature is controlled by the following command line options.

- Windows and OSS environment: `-Wmigration_check=32to64`
- Guardian environment: `migration_check 32to64`

An example of one of these diagnostics is: Any conversion (using either a cast or a direct assignment) from a `long` to an `int` results in the warning:

```
warning(2412): 64 bit migration: type conversion may truncate value
```

Another example is: Any conversion from a pointer to a `long` to a pointer to an `int` or from a pointer to an `int` to a pointer to a `long` results in the warning:

```
warning(2414): 64 bit migration: type conversion may cause target of
pointers to have a different size
```

Hewlett Packard Enterprise recommends that code be compiled with the migration check warnings enabled, and each warning examined, before compiling with `-Wlp64`.

64-Bit Programming Guidelines

When transitioning to 64-bit mode, strive to maintain a single set of source files and header files for both data models. Consider the following guidelines before porting to 64-bit mode:

- *Data Truncation* – Truncation problems can happen when assignments are made between 64-bit and 32-bit data items. Because `int`, `long`, and pointer 32-bit data types are all the same size in ILP32, mixed assignments between



these data types do not present any special concerns. However, in the LP64 data model, `long` and pointer data types are no longer the same size as `int`. In LP64, truncation occurs when pointer or `long` are assigned to `int`.

In LP64, truncation can occur during:

- Initialization
- Assignments
- Parameter passing
- Return statements
- Casts

To prevent data truncation:

- Avoid assigning a `long` to an `int`
 - Avoid storing a pointer in an `int`
 - Avoid truncating function return values
 - Use appropriate print specifiers
- *Data Type Promotion* – When comparisons and arithmetic operations are performed between variables and constants with different data types, the C compiler first converts these types to compatible types. For example, when a `short` is compared to a `long`, the `short` is first converted to a `long`. This conversion process is called *data type promotion*.

! **IMPORTANT:** Avoid arithmetic between signed and unsigned numbers

Certain data type promotions result in signed numbers being treated as unsigned numbers. When this happens, you can occasionally get unexpected results. For example, in ILP32:

```
long result;
int i = -2;
unsigned int j = 1;
result = i + j;
```

The ILP32 intermediate result (an unsigned `int`) and the final result (a signed `long`) have the same internal representation because they are both 32 bits. Since the final result is signed, the answer is `-1`.

Under the LP64 data model, the results are different. When the 32-bit intermediate result (an unsigned `int`) is converted to the 64-bit final result (a signed `long`), the left 32 bits are zero-filled. This results in a very large 64-bit positive number.

- *Pointers* – Avoiding pointer corruption is an important concern when migrating to LP64:
 - Assigning a 32-bit hexadecimal constant or an `int` to a pointer results in an invalid address and might cause errors when the pointer is de-referenced.
 - Casting a pointer to an `int` results in truncation.
 - Casting an `int` to a pointer can cause errors when the pointer is de-referenced.
 - Casts between `long*` to `int*` are problematic because the object of a `long` pointer is 64 bits in size, but the object of an `int` pointer is only 32 bits in size.
 - Functions that return pointers, when declared implicitly, can return truncated values.
 - Comparing an `int` to a pointer can cause unexpected results.



- Avoid pointer arithmetic between `long` and `int`. Pointer arithmetic is a source of difficulty in migration.
- Avoid casting a pointer to an `int` or an `int` to a pointer.
- Avoid storing a pointer in an `int`.
- Avoid truncating function return values.
- *Data Alignment and Data Sharing* – Data alignment rules determine where fields are located in memory. There are differences between the LP64 data alignment rules and the ILP32 data alignment rules.

In ILP32, pointer and `long` data types are 32 bits wide and are aligned on 32-bit boundaries. In LP64, pointer and `long` are 64 bits wide and are aligned on 64-bit boundaries.

Applications that do not consider alignment differences between ILP32 and LP64 can have trouble sharing binary data. Data exchanged between ILP32 and LP64 programs, whether via files, remote procedure calls, or other messaging protocols, might not be aligned as expected.

- `#pragma shared2/shared8` — When compiled with the LP64 data model, the compiler will issue an error for components with either a `long` or pointer data type that does not have an explicit size modifier. (However, these types of components are still allowed for the ILP32 data model.)
- *Hardcoded Constants* – When a program with hexadecimal constants is ported from ILP32 to LP64, the data types assigned to the constants might change. The following table illustrates some common hex constants and their types:

Table 54: Hex Constants and Types

Constant	ILP32 Data Model	LP64 Data Model
<code>0x7fffffff</code>	<code>int</code>	<code>int</code>
<code>0xffffffffL</code>	<code>long</code>	<code>long</code>
<code>0x80000000</code>	<code>unsigned int</code>	<code>unsigned int</code>
<code>0x80000000L</code>	<code>unsigned long</code>	<code>long</code>

In LP64, 32-bit hexadecimal constants might not set pointers or masks to the correct value. In LP64, the first 32 bits of 64-bit pointers contain significant information.

- Avoid using literals and masks that assume 32 bits
- Avoid hardcoding size of data types
- Avoid hardcoding bit shift values
- Avoid hardcoding constants with `malloc()` and `malloc32()`
- *Bit Shifts and Bit Masks* – Bit shifts and bit masks are sometimes coded with the assumption that the operations are performed in variables that have the same data type as the result. In cases such as:

```
a = b operation c
```

The data type used for the intermediate result of the operation depends on the types of `b` and `c`. The intermediate result is then promoted to the type of `a`. If the result requires 64 bits, but `b` and `c` are 32-bit data types, then the intermediate result either overflows or is truncated before being assigned to `a`.

In the following example, the left operand 1 is a small numeric constant that the compiler treats as a 32-bit value in both ILP32 and LP64:

```
unsigned long y;  
y = (1 << 32); /* Overflows in both data models. */
```

This bit shift uses a 32-bit data type as the intermediate result. In 64-bit mode, the operation overflows and the final result is *undefined*.

You can use suffixes such as L and UL for long and unsigned long if you need long constants. For example, in 64-bit mode, the above code fragment can be changed to:

```
y = (1L << 32); /* 2^32 in LP64. Overflows in ILP32. */
```

Step 4: Compile in the LP64 Data Model

To compile in the LP64 data model, update makefiles with the appropriate options:

- -Wlp64: For Windows and OSS compiles.
- lp64: For Guardian compiles.
- See [Linker Object Files](#) for details of eld -set data_model options.

If you are building 64-bit applications, verify that 64-bit versions of the NonStop DLLs are installed.

Compiling Code for Both ILP32 and LP64 Data Models

If you have code that needs to be compiled for both ILP32 and LP64 data models and the code is slightly different between the two models, you can use the `__LP64` feature test macro, which is automatically set to 1 when either the -Wlp64 or the LP64 compiler option is specified. For example:

```
#if __LP64  
    64-bit code goes here  
#else  
    32-bit code goes here  
#endif
```

64-Bit TNS/E and TNS/X Debugger Support

Beginning with the H06.24 and J06.13 RVUs, Native Inspect is derived directly from the industry standard GDB debugger, version 6.8. The GUI debugging interface is provided by an enhanced version of Eclipse that uses Native Inspect as its server.

You have two choices for debugging 64-bit programs: command-line-based Native Inspect and the Eclipse-based NonStop Development Environment for Eclipse (NSDEE) GUI. To use the NSDEE debugger, the application must be built by the NSDEE IDE, either locally (using Windows hosted cross-compilers) or remotely using TNS/E or TNS/X compilers. NSDEE uses Native Inspect to implement debugging operations.

Both Native Inspect and Eclipse-based debugging have been enhanced to support debugging of 64-bit processes. Native Inspect provides full debugging support for 64-bit processes, including:

- Live process debugging
- Postmortem debugging using snapshot files (see [Snapshot Support](#))
- 64-bit Tool Control Language (TCL) (see [Tool Control Language \(TCL\) Support](#))
- Memory leak detection



- Function invocations
- 64-bit memory access breakpoints

See the debugger information in **OSS Tools** for more details.

Tool Control Language (TCL) Support

The version of Tool Control Language (TCL) is upgraded to version 8.5, and it natively supports 64-bit arithmetic. The multi-precision arithmetic library, MPFORMAT, no longer needs to be used for 64-bit arithmetic. It is therefore designated as “deprecated”, and it might not be supported in future releases.

Snapshot Support

Process snapshot files, file code 130, store information about a process’ state for later use in postmortem debugging. Snapshot files are created:

- Automatically – when a process with the `SAVEABEND` attribute set terminates via `ABEND`
- Manually – by a debugger user who wants to save the process state during a debugging session

Snapshot files that are created for 64-bit processes and processes containing 64-bit data segments use a new 64-bit-capable format. These files continue to have a file code of 130, but can only be used with H06.24 (and later H-series RVUs) or J06.13 (and later J-series RVUs) versions of Native Inspect and NSDEE. These 64-bit-capable snapshot files cannot be used with Visual Inspect.

A 64-bit snapshot file can be either an Enscribe Format 1 file (maximum of 2 GB) or an Enscribe Format 2 file (maximum of 1 TB minus 4 GB).

NOTE: Snapshot files created for 64-bit processes and processes containing 64-bit data segments are based on the industry standard ELF64 format, also referred to as “core files” on UNIX systems. This format uses industry-standard IA-64 Linux/GCC data structures. Because of this open format, tools can be created using supplied header files to extract information from snapshot files.

Reading Snapshot Files

The Snapshot Read/Write routines (NonStop T0745) continue to be available to read 32-bit snapshot files.

64-bit snapshot files can be accessed directly using the ELF64 core file headers included with T0745.

Snapshot File Compression

64-bit snapshot files can be compressed when they are created using the `save` command:

```
$ save filename [compression] [!]
```

Where *compression* can be specified as `bzip2`, `gzip`, or `none`.

The `snapshot` command automatically detects compression and acts accordingly.

There are significant time and space trade-offs with compression:

- The `none` option is the fastest, but (the compressed dump) takes the most space.
- The `gzip` option is the next fastest, but takes up less space than `none`.
- The `bzip2` option is the slowest, but takes up the least space.



In the absence of a `save` option, and in the *saveabend* case, compression is selectively applied based on the uncompressed file size.

NOTE: Dumps can be compressed after the fact by using the standard utilities from the ITUG Library.

eInspect Support

Beginning with the H06.24 and J06.13 RVUs, features of Hewlett Packard Enterprise eInspect are:

- Re-based to GDB 6.8 from GDB 5.0
- Re-based to TCL 8.5 from TCL 8.0 and it supports 64-bit arithmetic
- Can be used to debug 64-bit processes
- Can be used to analyze both 32-bit and 64-bit snapshot files

eGarth Support

Beginning with the H06.24 and J06.13 RVUs, eGarth creates 64-bit snapshot files from a CPU dump for processes having 64-bit segments.

Visual Inspect Support

Visual Inspect can be used to debug 32-bit processes with 64-bit segments; however, it does not support live debugging of a 64-bit process. Also, Visual Inspect cannot be used to analyze the new 64-bit format snapshot files that contain one or more 64-bit segments.

An attempt to debug a 64-bit process or open a 64-bit-capable snapshot file is rejected with an error.

The Native Inspect debugger starts when specifying a 64-bit OSS program as the `ossprog` parameter for either the `run -debug` or `runv` command, because Visual Inspect does not currently support 64-bit programs.

NOTE:

Visual Inspect is not supported on TNS/X systems (L-series RVUs).



Using the Standard POSIX Threads Library

Standard POSIX Threads is a user-space implementation of IEEE POSIX Standard 1003.1c pthreads for use by native C and C++ applications in the OSS environment on NonStop servers. The POSIX threads standard is officially described in International Standard ISO/IEC 9945-1; 1996 (E) IEEE Std 1003.1,1996 (Incorporating ANSI/IEEE Stds 1003.1-1990, 1003.1b-1993, 1003.1c-1995, and 1003.1i-1995). For simplicity, this chapter refers to the “Standard document.” The Standard document is available for purchase online at <http://standards.ieee.org/>.

To use threads, you should be familiar with programming in the OSS environment and with writing threaded applications. See **About This Document** for a list of books on programming with threads. See the *Open System Services Porting Guide* for information about porting existing threaded applications and for differences between the previous standard and Standard POSIX Threads.

- **Standard POSIX Threads**
- **Threading Considerations**
- **Single Versus Multiple Application Threads**
- **Jacket Routines**
 - **SERVERCLASS_SEND and SERVERCLASS_SENDL Jacket Routines**
 - **TMF Transaction Jacket Routines**
- **Reference Pages for Thread Functions**
- **Threaded Application Examples**
- **Thread-Aware and Nonblocking OSS Functions**
- **Reentrant OSS Functions**
- - **Thread-Aware Toolkit Functions and Types**
 - **Thread-Aware \$RECEIVE I/O Functions**
- **Thread-Aware Enscribe Functions**
- **OSS File Descriptor and Timer Callbacks**

Standard POSIX Threads

Threads are scheduled for execution by the Standard POSIX Threads library, not by the NonStop operating system. All threads created within a process share the same process address space.

With Standard POSIX Threads, one thread can never be preempted by another thread. You must ensure that threads relinquish control either by explicitly calling the `sched_yield()` function or by waiting on a mutex or condition variable.

All resources of a process—for example, open files or memory—are either global or private, as shown in the following table. Private resources can be accessed only by a specific thread. Global resources can be accessed by all threads. Access to global resources by threads should be synchronized using mutex or condition variables.



Table 55: Process Resources

Resource	Type	Description
Thread stack	Private	The memory where all thread-local variables are stored
Thread program counter	Private	The address of the instruction that the thread is about to execute
Thread context	Private	The data maintained by the threads library on behalf of a thread; contains the per-thread signal mask, which is used by the <code>pthread_kill()</code> function
All external data	Global	Includes process and module global resources
All static internal variables	Global	
Heap	Global	
File descriptors and file state	Global	Includes file and record locks
SQL cursors	Global	
Environment variables	Global	
Signal mask	Global	The per-process signal mask, used by the <code>kill()</code> function

Compiling and Linking Considerations

To use the Standard POSIX Threads library, you must:

- Include this header file:

```
/usr/include/spthread.h
```

The `spthread.h` file contains prototypes of the Standard POSIX Threads interfaces.

- Specify the `_SPT_MODEL_` define. This define is required for systems running H06.21 or later H-series RVUs or J06.10 or later J-series RVUs.
- For TNS/R programs, when linking with `nld` or `ld`, specify `ZSPTURL`, the Standard POSIX Threads shared run-time library.
- For TNS/E programs, when linking with `eld`, specify `ZSPTDLL`, the Standard POSIX Threads DLL, before specifying any public libraries, such as `ZSECDLL`.
- For TNS/X programs, when linking with `xld`, specify `XSPTDLL`, the Standard POSIX Threads DLL, before specifying any public libraries, such as `XSECDLL`.

These libraries are located in the current `sysnn` subvolume.

Beginning with RVU G06.22, T1248 supports both the VERSION2 and VERSION3 C++ libraries. Additionally, take special steps to ensure that exception handling will continue to function properly. For compiling a TNS/R C++ program utilizing

T1248 and the C++ exception handling mechanism, link the object `/usr/lib/sptcpp.o`. This object ensures that exception handling will continue to work, regardless of the C++ version. However, for programming exclusively in C, or in C++ on a TNS/E or TNS/X system, linking `/usr/lib/sptcpp.o` is not necessary.

Error Reporting Changed From Previous Standard

Standard POSIX Threads functions return error indications as return values, instead of setting `errno`. The error values returned are in many cases the values that were stored in `errno` in Draft 4. Standard POSIX Threads supports only the error values described in the Standard document.

If an error condition occurs, a pthread routine returns an integer value indicating the type of error. For example, a call to the Draft 4 implementation of `pthread_cond_destroy()` that returned a `-1` and set `errno` to `EBUSY` now returns `EBUSY` as the routine return value.

Reporting errors by returning error values is a global change from the previous standard (Draft 4), in which errors are reported in `errno`. If you are porting a threaded application that adheres to Draft 4, you must rewrite the error handling routines. The **Threaded Application Examples** demonstrates error handling with Standard POSIX Threads. See the Open System Services Porting Guide for a list of the supported return values as well as other porting issues.

Attributes Defined by Threads Macros

The attributes of Standard POSIX Threads are determined by macros in `unistd.h`. The following table lists the principal macros that influence the Hewlett Packard Enterprise implementation for NonStop servers.

Table 56: Attributes of Standard POSIX Threads Defined by Macros

Macro or Attribute	Description in Standard POSIX Threads
Mutex and condition variable initialization attributes: <code>_POSIX_THREAD_PROCESS_SHARED</code>	Not defined. A mutex or condition variable can be operated on only by threads in the same process that initialized the mutex or condition variable.
Thread scheduling attribute: <code>_POSIX_THREAD_PRIORITY_SCHEDULING</code>	Not defined. Threads are created with the <code>PTHREAD_SCOPE_PROCESS</code> scheduling contention scope by default. Threads contend directly with other threads within their process. This attribute cannot be altered. The scheduling allocation domain size is 1. Threads are always bound to the same processor as the parent thread.
Thread scheduling attribute: <code>inheritsched</code>	Has the value <code>PTHREAD_EXPLICIT_SCHED</code> . The scheduling policy and associated attributes, such as priority, are set to the corresponding values from the attributes object during thread creation. This attribute can be altered to <code>PTHREAD_INHERIT_SCHED</code> (so that the scheduling policy and attributes are inherited from the creating thread) by using <code>pthread_attr_setinheritsched()</code> .

Table Continued



Macro or Attribute	Description in Standard POSIX Threads
Thread scheduling attribute: <code>schedpolicy</code>	The default value is <code>SCHED_FIFO</code> . This attribute cannot be altered.
Mutex initialization scheduling attributes: <code>_POSIX_THREAD_PRIO_INHERIT</code> <code>_POSIX_THREAD_PRIO_PROTECT</code>	Not defined. The <code>prioceiling</code> and <code>protocol</code> attributes are not defined.
Thread creation attribute: <code>_POSIX_THREAD_ATTR_STACKSIZE</code>	Defined. The thread creation stack size attribute can be set in the attribute object.
Thread creation attribute: <code>_POSIX_THREAD_ATTR_STACKADDR</code>	Not defined. The location of the storage to be used for the stack of the created thread cannot be altered.
Thread creation attribute: <code>detachstate</code>	The default value is <code>PTHREAD_CREATE_JOINABLE</code> (that is, storage associated with the thread is reclaimed when the thread that created it calls a <code>join()</code>). This attribute can be altered using <code>pthread_attr_setdetachstate()</code>.

Signal Handling

There are two basic types of signals:

- Synchronous signals represent events that occur inside a process.

Synchronous signals occur immediately after the instruction that caused the event—for example, attempting to divide by zero. A synchronous signal is associated with the thread that is executing when the event occurs. A thread can install signal handlers (using the `sigaction()` function) for the synchronous signals that occur during its execution. For terminating signals, the default behavior is to terminate the process.

- Asynchronous signals represent events external to a process.

A thread can wait for a set of asynchronous signals using the `spt_sigwait()` function. If a thread is waiting for a type of signal that is received, the thread is awakened and if a signal handler has been installed, it is invoked, and the signal is considered handled; otherwise, the default signal action is taken. For a terminating signal, the default behavior is to terminate the process. For J-series RVUs, H06.06 and later H-series RVUs, and G06.29 and later G-series RVUs, you can define signal-catching functions for asynchronous signals. See **Thread-Aware Signal Handling**.

The supported signal numbers are listed in the following table. For more information about the Standard POSIX Threads implementation of signals, see **Using Signals**.

Table 57: Signals Processed by Threads

Name	Number	Type	Default Action	Description
SIGABEND	31 ¹	Process-level	Terminate with <code>saveabend</code>	Abnormal termination
SIGABRT	6	Synchronous	Terminate with <code>saveabend</code>	Abort process

Table Continued



Name	Number	Type	Default Action	Description
SIGALRM	14	Alarm signal	Terminate process	Alarm clock
SIGCHLD	18	Process-level	Discard signal	Child has stopped or terminated
SIGCONT	28	Process-level	Continue execution	Continue execution
SIGFPE	8	Synchronous	Terminate with saveabend	Arithmetic overflow
SIGHUP	1	Process-level	Terminate process	Hangup
SIGILL	4	Synchronous	Terminate with saveabend	Invalid instruction
SIGINT	2	Asynchronous	Terminate process	Interrupt
SIGIO	7	Synchronous	Discard signal	IO possible or completed
SIGKILL	9 ¹	Process-level	Terminate process	Kill
SIGLIMIT	27	Synchronous	Terminate with saveabend	NonStop operating system limits trap
SIGMEMERR	22	Synchronous	Terminate with saveabend	Uncorrectable memory error
SIGMEMMGR	24	Synchronous	Terminate with saveabend	Memory manager read error
SIGNOMEM	23	Synchronous	Terminate with saveabend	No memory available
SIGPIPE	13	Asynchronous ²	Terminate process	Write on a pipe, no one to read it
SIGQUIT	3	Asynchronous	Terminate with saveabend	Quit
SIGRECV	19	Synchronous	Discard signal	Message queued on \$RECEIVE (currently not used)
SIGSEGV	11	Synchronous	Terminate with saveabend	Invalid address reference
SIGSTK	25	Synchronous	Terminate with saveabend	Stack overflow
SIGSTOP	20 ¹	Process-level	Terminate process	Stop
SIGTERM	15	Asynchronous	Terminate process	Software termination signal

Table Continued



Name	Number	Type	Default Action	Description
SIGTIMEOUT	26	Process-level ³	Terminate process	Process loop timer time-out
SIGTSTP	21	Process-level	Terminate process	Interactive stop
SIGTTIN	29	Process-level	Stop process	Background read attempted from controlling terminal
SIGTTOU	30	Process-level	Stop process	Background write attempted to controlling terminal
SIGUNCP	10	Process-level ³	Discard signal	Uncooperative process. Supported for servers running H-series RVUs only.
SIGURG	5	Asynchronous ²	Discard signal	Urgent condition on I/O channel
SIGUSR1	16	Asynchronous	Terminate process	User-defined signal 1
SIGUSR2	17	Asynchronous	Terminate process	User-defined signal 2
SIGWINCH	12	Synchronous	Discard signal	Terminal device window size changed

¹Signal cannot be caught or ignored.

²For H06.20 and earlier H-series RVUs and J06.09 and earlier J-series RVUs, this is a synchronous signal

³Not supported on G-series RVUs

Thread-Aware Signal Handling

For J-series RVUs, H06.06 and later H-series RVUs, and G06.29 and later G-series RVUs, the Hewlett Packard Enterprise implementation of Standard POSIX Threads catches both externally generated OSS signals and synchronous signals, which are result of an event that occurs inside a process, and deliver them to the correct thread.

By default, thread-aware signal behavior is disabled, and signals are handled at the process level. To enable thread-aware signal handling, export the SPT_THREAD_AWARE_SIGNAL environmental variable from within the shell to the value 1:

```
> export SPT_THREAD_AWARE_SIGNAL=1
```

The SIGPIPE and SIGURG signals are synchronous for H06.20 and earlier H-series RVUs and J06.09 and earlier J-series RVUs, and asynchronous for H06.21 and later H-series RVUs and J06.10 and later J-series RVUs.

Types of Signals

Signals are categorized as synchronous, asynchronous, or process level:

Synchronous signals result from an event that occurs inside a process and are delivered synchronously with that event. These signals are delivered to current active thread. If current active thread has not installed the signal handler, then default actions are performed. Signals in this category are:⁴



SIGABRT
SIGFPE
SIGILL
SIGIO
SIGLIMIT
SIGMEMERR
SIGMEMMGR
SIGNOMEM
SIGSEGV
SIGSTK
SIGWINCH

Asynchronous signals result from an event that is external to the process and are delivered at any point in the execution of a thread. These signals are delivered to the active threads in FIFO order. If no active threads have installed a signal handler for the signal, the process-level actions described in **Table 57: Signals Processed by Threads** are performed. Signals in this category are:

SIGINT
SIGPIPE⁴
SIGQUIT
SIGTERM
SIGURG ⁴
SIGUSR1
SIGUSR2

For process level signals only process level actions can be performed. Signal handlers for these signals are registered directly to the operating system, so an existing handler for a given signal is overwritten when a new signal handler is defined for that signal. Signals in this category are:

SIGABEND (can't be handled by a user-defined signal handler)
SIGCONT
SIGKILL (can't be handled by a user-defined signal handler)
SIGSTOP (can't be handled by a user-defined signal handler)
SIGTSTP (process level action is performed)
SIGTIMEOUT (not supported on G-series RVUs)
SIGTTIN
SIGTTOU
SIGUNCP (supported on H-series RVUs only)

⁴The SIGPIPE and SIGURG signals are synchronous for H06.20 and earlier H-series RVUs and J06.09 and earlier J-series RVUs, and asynchronous for H06.21 and later H-series RVUs and J06.10 and later J-series RVUs.

Thread-Aware Signal Handling APIs

These APIs are supported for J-series RVUs, H06.06 and later H-series RVUs, and G06.29 and later G-series RVUs:



API	Description
<code>spt_signal()</code>	Allows the calling thread to change the action to be taken when a specific signal is delivered to the thread issuing this function call. To enable thread-aware signal handling, export the <code>SPT_THREAD_AWARE_SIGNAL</code> environmental variable from within the shell to the value 1. If thread-aware signal handling is not enabled, signals are handled at the process level. For C applications, an application call to <code>signal()</code> is automatically mapped to <code>spt_signal()</code> when you compile the application using the <code>#define _SPT_THREAD_SIGNAL</code> feature test macro or an equivalent compiler command option. For C++ applications, an application call to <code>signal()</code> is automatically mapped to <code>spt_signal()</code> when you compile the application using the <code>#define _SPT_THREAD_SIGNAL_PRAGMA</code> feature test macro or an equivalent compiler command option.
<code>spt_alarm()</code>	Arranges for a <code>SIGALRM</code> signal to be delivered to the process from a thread in the specified number of seconds. To enable thread-aware signal handling, export the <code>SPT_THREAD_AWARE_SIGNAL</code> environmental variable from within the shell to the value 1. If thread-aware signal handling is not enabled, alarms are handled at the process level. For C applications, an application call to <code>alarm()</code> is automatically mapped to <code>spt_alarm()</code> when you compile the application using the <code>#define _SPT_THREAD_SIGNAL</code> feature test macro or an equivalent compiler command option. For C++ applications, an application call to <code>alarm()</code> is automatically mapped to <code>spt_alarm()</code> when you compile the application using the <code>#define _SPT_THREAD_SIGNAL_PRAGMA</code> feature test macro or an equivalent compiler command option.

Limitations

Do not deliver signals like `SIGALRM` and `SIGCHLD` using the `kill` command or the `raise()` function because these functions do not provide enough information about the process IDs for the parent and child processes. Instead, use the `spt_alarm()` function to raise alarms.

If thread-aware signal handling is enabled for a threaded application, and the application is linked to libraries that are not thread-aware and that change the action to be taken when a specific signal is delivered, then the application might show unexpected behavior.

Examples

The following example provides an example of handling the synchronous signal `SIGFPE`.

The output of the example depends on whether thread-aware signal handling is enabled:

- If thread-aware signal handling not enabled, the signal handler defined in the example, `sigband`, is not executed, and signal 31 (`SIGABEND`) is delivered to the process.
- If thread-aware signal handling is enabled, the signal handler defined in the example is executed, and signal 8 (`SIGFPE`) is received by the thread. On successful return from a signal-catching function for a `SIGFPE`, `SIGILL`,

SIGLIMIT, SIGMEMERR, SIGMEMMGR, SIGNOMEM, SIGSEGV, or SIGSTK signal that was not generated by a `kill()` or `raise()` function call, a process receives a SIGABEND signal and terminates.

Handling the Synchronous Signal SIGFPE

```
/* This test program demonstrates handling the */
/* SIGFPE synchronous signal. */
/* You must also export the environment variable */
/* SPT_THREAD_AWARE_SIGNAL to 1 to */
/** enable thread-aware signal handling */

#define SPT_THREAD_AWARE /** enables thread aware behavior */
#define SPT_THREAD_SIGNAL /** maps signal() to spt_signal() */
#include <spthread.h>

/* Define a signal handler for this thread */
void sighand(int signo);
int x=10,y=0,z;

void *threadfunc(void *parm)

/* Perform the illegal operation of dividing by zero and */
/* raise the SIGFPE signal. The SIGFPE signal is sent to */
/* the generic handler. */
{
    signal(SIGFPE,sighand);
    z = x/y;
    printf("\nI am in thread1\n");
    return NULL;
}

int main(int argc, char **argv)
{
    pthread_t    thread1;
    int          rc;
    printf("Enter Testcase - %s\n", argv[0]);

    /* create the thread */
    pthread_create(&thread1, NULL, threadfunc, NULL);
    sleep(1);
    printf("Wait for threads to complete\n");
    pthread_join(thread1, NULL);
    printf("Main completed\n");
    return 0;
}

void sighand(int signo)
{
    printf("Signal received is %d\n", signo);
}
```

Handling the Synchronous Signal SIGINT creates a thread that waits for asynchronous signal SIGINT.



Sample input:

```
> kill -2 process-name
```

or

```
<CTRL-C>
```

Both of these inputs generate a SIGINT signal.

The output of **Handling the Synchronous Signal SIGINT** depends on whether thread-aware signal handling is enabled:

- If thread-aware signal handling is not enabled, the program terminates, which is the default behavior when signal 2 (SIGINT) is received. Sample output:
Enter Testcase - ./Test2
spt_sigaction returned 0
Wait for threads to complete
- If thread-aware signal handling is enabled, the signal handler of `thread1` is executed and `thread1` no longer waits for a signal. Sample output:

```
Enter Testcase - ./Test2
spt_sigaction returned 0
Wait for threads to complete
Signal received is 2
spt_sigsuspend returned -1
Main completed.
```

This behavior differs from the behavior when thread-aware signal handling is not enabled because the generic handler catches the signal delivered to a process and delivers the signal to the correct thread(s).

Handling the Synchronous Signal SIGINT

```
/* Program which demonstrates behavior of Asynchronous */
/* signal SIGINT */

/** You must also export the environment variable */
/** SPT_THREAD_AWARE_SIGNAL to 1 to */
/** enable thread-aware signal handling */

#define SPT_THREAD_AWARE /** enables thread aware behavior */
                        /** and maps functions to spt_versions */
#include <spthread.h>

void sighand(int signo);

struct sigaction sigact_t;

sigset_t waitset;

void *threadfunc(void *parm)
{
    int rc1, rc2;
    sigfillset( &waitset );
    sigdelset( &waitset, SIGINT );
    sigact_t.sa_handler = sighand;
    rc1 = sigaction(SIGINT, &sigact_t, NULL);
```

```

    printf("\nspt_sigaction returned %d\n",rc1);
    rc2=sigsuspend(&waitset);
    printf("\nspt_sigsuspend returned %d\n",rc2);
    return NULL;
}

int main(int argc, char **argv)
{
    pthread_t      thread1;
    int            rc;
    printf("Enter Testcase - %s\n", argv[0]);
    pthread_create(&thread1, NULL, threadfunc, NULL);
    sleep(1);
    printf("Wait for threads to complete\n");
    pthread_join(thread1, NULL);
    printf("Main completed\n");
    return 0;
}

void sighand(int signo)
{
    printf("Signal received is %d\n",signo);
}

```

Handling the SIGCHLD Signal demonstrates handling the SIGCHLD signal. This example creates two threads, each of which call `fork()`. When the child process executes, the process raises the SIGCHLD signal.

The output of **Handling the SIGCHLD Signal** depends on whether thread-aware signal handling is enabled:

- If thread-aware signal handling is not enabled, the SIGCHLD signal results in the default action described in **Table 57: Signals Processed by Threads** (discard the signal). In this example, `spt_sigaction()` fails because it is not supported. Sample output:

```

spt_sigaction for thread1 returned -1
spt_sigaction for thread2 returned -1
Inside Child of Thread1
Inside Child of Thread2
Main Completed

```

- If thread-aware signal handling is enabled, the SIGCHLD signal is delivered to the correct thread, and `spt_sigaction()` function in the example defines the action to be taken. Sample output:

```

spt_sigaction for thread1 returned 0
spt_sigaction for thread2 returned 0
Inside Child of Thread2
Signal received for Thread2 is 18
Inside Child of Thread1
Signal received for Thread1 is 18
Main Completed

```

Handling the SIGCHLD Signal

```

/* This program demonstrates the handling of the */
/* SIGCHLD signal */

```

```

/** You must also export the environment variable */
/** SPT_THREAD_AWARE_SIGNAL to 1 to */
/** enable thread-aware signal handling */
#define SPT_THREAD_AWARE /** enables thread aware behavior */
                        /** and maps functions to spt_ versions */
#include <sptthread.h>
struct sigaction      sigact_t, osigact_t;
sigset_t waitset;
int rc;

void sig_cld1(int);
void sig_cld2(int);
void *do_work1(void *arg)
{
    pid_t pid;
    sigact_t.sa_handler = sig_cld1;
    sigact_t.sa_flags = 0;
    rc = spt_sigaction(SIGCHLD, &sigact_t, NULL);
    printf("\nspt_sigaction for thread1 returned %d\n", rc);
    if ((pid = fork()) < 0)
        printf("ERROR IN FORK\n");
    else if (pid == 0)
    {
        sleep(2);
        printf("\nInside Child of Thread1\n");
        exit(0);
    }
    spt_waitpid(pid, NULL, NULL);
    return NULL;
}

void *do_work2(void *arg)
{
    pid_t pid;
    osigact_t.sa_handler = sig_cld2;
    osigact_t.sa_flags = 0;
    rc = spt_sigaction(18, &osigact_t, NULL);
    printf("\nspt_sigaction for thread2 returned %d\n", rc);
    if ((pid = fork()) < 0)
        printf("ERROR IN FORK\n");
    else if (pid == 0)
    {
        sleep(1);
        printf("\nInside Child of Thread2\n");
        exit(0);
    }
    spt_waitpid(pid, NULL, NULL);
    return NULL;
}

void main()
{
    pthread_t tid1, tid2;
    pthread_create(&tid1, NULL, do_work1, NULL);
    pthread_create(&tid2, NULL, do_work2, NULL);
    sched_yield ();
}

```

```

    pthread_join(tid1,NULL);
    pthread_join(tid2,NULL);
    printf ("\nMain Completed\n");
}

void sig_cld1(int signo)
{
    printf("\nSignal received for Thread1 is %d\n",signo);
}

void sig_cld2(int signo)
{
    printf("\nSignal received for Thread2 is %d\n",signo);
}

```

Handling SIGALRM Signals provides an example of SIGALRM handling. It creates two threads that wait for a SIGALRM signal.

The output of **Handling SIGALRM Signals** depends on whether thread-aware signal handling is enabled:

- If thread-aware signal handling not enabled, the entire process terminates when the SIGALRM signal is received. Sample output:

```

spt_sigaction for thread1 returned 0
spt_sigaction for thread2 returned 0

```

- If thread-aware signal handling is enabled, SIGALRM signal is delivered to the correct thread, and the sighand1() and sighand2() functions define the actions to be taken. Sample output:

```

spt_sigaction for thread1 returned 0
spt_sigaction for thread2 returned 0
Signal received for thread1 is 14
spt_sigsuspend for thread1 returned -1
spt_sigwait returned 0
spt_sigwait for thread2 returned for signal 14
Main Completed

```

Handling SIGALRM Signals

```

/*Program demonstrating handling the SIGALRM signal */

/** You must also export the environment variable  **/
/** SPT_THREAD_AWARE_SIGNAL                        **/
/** to 1 to enable thread-aware signal handling  **/
#define SPT_THREAD_AWARE /** enables thread aware behavior and maps **/
    /** functions other than alarm() and signal() to their spt_ versions**/
#define SPT_THREAD_SIGNAL /** maps alarm() to spt_alarm()**/
#include <spthread.h>
void sighand1(int);
void sighand2(int);
struct sigaction sigact_t;
int rc;
void *do_work(void *arg)
{
    sigset_t set;
    sigfillset(&set);
    sigdelset(&set,SIGALRM);
    sigact_t.sa_handler = sighand1;
    rc = spt_sigaction(SIGALRM,&sigact_t,NULL);
}

```

```

    printf("\n    spt_sigaction for thread1 returned %d\n",rc);
    alarm(5);
    rc = spt_sigsuspend(&set);
    printf("\nspt_sigsuspend for thread1 returned %d\n",rc);
    return NULL;
}

void *do_work1(void *arg)
{
    sigset_t set;
    int sig;
    sigemptyset(&set);
    sigaddset(&set,SIGALRM);
    sigact_t.sa_handler = sighand2;
    rc=spt_sigaction(SIGALRM,&sigact_t,NULL);
    printf("\nspt_sigaction for thread2 returned %d\n",rc);
    alarm(6);
    rc=spt_sigwait(&set,&sig);
    printf("\nspt_sigwait returned %d\n",rc);
    printf("\nspt_sigwait for thread2 returned for signal %d\n",sig);
    return NULL;
}

int main()
{
    pthread_t tid1,tid2;
    pthread_create(&tid1,NULL,do_work,NULL);
    pthread_create(&tid2,NULL,do_work1,NULL);
    sched_yield();
    pthread_join(tid1,NULL);
    pthread_join(tid2,NULL);
    printf("\nMain Completed\n");
}

void sighand1(int signo)
{
    printf("Signal received for thread1 is %d\n",signo);
}

void sighand2(int signo)
{
    printf("Signal received is %d\n",signo);
}

```

Threads Portability

Standard POSIX Threads follow the POSIX threads Standard document. Using thread functions whose names end with `_np` (non-portable) decreases the portability of a program, because these functions are extensions to the standard. Using the jacket routines provided by Hewlett Packard Enterprise also reduces portability of a threaded application; see **Jacket Routines** for more information about the jacket routines.



Numerical Limits in Standard POSIX Threads

The following table lists numerical limits imposed by the standard implementation. The symbolic constants listed in the table are defined in `limits.h` with the values shown in the table. These symbolic constants refer to the most restrictive value for each feature.

Table 58: Numerical Limits in Standard POSIX Threads

Symbolic Constant	Description	Value
<code>_POSIX_ARG_MAX</code>	Maximum length of the arguments for one of the <code>exec</code> functions, in bytes, including environment data	4096
<code>_POSIX_CHILD_MAX</code>	Maximum number of simultaneous processes for one real user ID	6
<code>_POSIX_FD_SETSIZE</code>	Maximum number of descriptors that a process can examine	<code>_POSIX_OPEN_MAX</code>
<code>_POSIX_HIWAT</code>	Maximum number of bytes that can be buffered on a socket for sending or receiving	<code>_POSIX_PIPE_BUF</code>
<code>_POSIX_LINK_MAX</code>	Maximum value of a file's link count	8
<code>_POSIX_MAX_CANON</code>	Maximum number of bytes in a terminal canonical input queue	255
<code>_POSIX_MAX_INPUT</code>	Maximum number of bytes for which space will be available in a terminal input queue	255
<code>_POSIX_NAME_MAX</code>	Maximum number of bytes in a filename	14
<code>_POSIX_NGROUPS_MAX</code>	Maximum number of simultaneous supplementary group IDs for one process	0
<code>_POSIX_OPEN_MAX</code>	Maximum number of files that one process can have open at one time	256
<code>_POSIX_PATH_MAX</code>	Maximum number of bytes in a pathname	255
<code>_POSIX_PIPE_BUF</code>	Maximum number of bytes that can be written atomically when writing to a pipe	512
<code>_POSIX_QLIMIT_</code>	Maximum number of connections that can be queued on a socket	1
<code>_POSIX_SSIZE_MAX</code>	Maximum value that can be stored in an object of type <code>ssize_t</code>	32,767

Table Continued



Symbolic Constant	Description	Value
_POSIX_STREAM_MAX	Maximum number of streams that one process can have open at one time	8
_POSIX_TZNAME_MAX	Maximum number of bytes supported for time zone names	3

Keeping Application Design Simple

Because of the inherent complexity of multithreaded programming, application designers should follow these recommendations:

- Write programs with a single application thread when possible.
- Keep multiple application threads independent of each other when possible. If this is not possible, keep the interdependencies to a minimum.

Threading Considerations

Application designers should be aware of the following general threading considerations when developing a Standard POSIX Threads application:

- **Sharing Process Resources**
- **Using Standard Libraries**
- **Addressing the Thread Stack**
- **Making Process-Blocking Calls**
- **Using Nowait Input or Output**
- **Yielding Control**
- **Using Signals**
- **Spawning a New Process**
- **Isolating Faults**

Sharing Process Resources

Access by threads to shared process resources such as global data, the heap, and files must be synchronized using mutexes and condition variables. **Table 55: Process Resources** table lists global and private process resources.

Using Standard Libraries

No Need for Jacket Routines or Special Header Files

Standard POSIX Threads execute until they relinquish control. A thread can never be preempted while executing code in a standard library. Therefore, an application does not need to protect standard libraries with global mutexes or use special header files. Standard libraries do not require jacket routines or other modifications to make them thread-safe or reentrant.



The thread library does provide jacket routines for calls to Pathway servers (SERVERCLASS_SEND and SERVERCLASS_SEND_INFO) and for TMF transactions, as described in [Jacket Routines](#).

No Asynchronous Cancels

Standard POSIX Threads does not support the asynchronous receipt of cancels; a thread can never exit prematurely from a standard library due to cancellation. Therefore, an application does not need to protect the run-time state of standard libraries by disabling cancelability before making a call into the libraries.

Beware of Pointers to Static Library Data

Some libraries contain functions that return pointers to static data. In this case, one thread could invalidate another thread's pointer. For example, the `getpwuid()` function returns a pointer to information about a user, contained in a static variable in the library. This static variable is overlaid on the next call to this procedure by any thread.

When using functions that return pointers to static library data, immediately copy the data to thread-private memory.

Beware of Libraries That Maintain Context Across Calls

Some libraries maintain context across calls. For example, the `getpwent()` function returns information about the next user in the user database, with the current position kept implicitly in the library. The `setpwent()` function ensures that the next call to `getpwent()` returns the first user entry. A program can get unexpected results if two different threads use these functions without synchronizing their calls.

You can use a mutex to protect implicit context maintained by a standard library, for example by bracketing a series of library calls with the lock and unlock of a mutex. This technique should be used only for short periods of time.

Standard libraries that use an explicit handle to identify the intended context on each call can be used without interference between threads. File operations that use file descriptors follow this model.

Some libraries allow multiple contexts but do not support explicit context identification on all calls. Instead, the library allows the caller to establish a current context for the process. With these libraries, the application is responsible for ensuring that the correct context is made current while a thread is executing. For example, the TMF transaction-demarcation jacket routines use this method, relying on an implicit current transaction for the process.

Addressing the Thread Stack

Memory for function variables that are not declared static is allocated from the thread-private stack. This memory is addressable only while the thread is executing. It is not addressable by other threads; attempting to read or write variables located in another thread's stack causes unpredictable results.

All program data that is to be shared by multiple threads must be external data, static internal data, or data allocated from the heap. This data should be protected by a mutex, as appropriate.

The Standard provides a set of functions to manage thread-specific data. One use of these functions is to maintain thread global data, such as state. The interface allows you to declare destructor functions that are invoked at thread run-down. These functions are distinct from cancellation clean-up handlers.

Making Process-Blocking Calls

When a thread calls a Guardian procedure that blocks the process, all other threads in the process are also prevented from executing. If a thread is blocked for a significant period of time, it can delay or even prevent the execution of time-critical run-time functions.

Examples of programs that can block include:

- Pathway servers, which can wait indefinitely while reading \$RECEIVE
- The Inspect, Native Inspect, and Visual Inspect symbolic debuggers, whose use can block a process indefinitely



Using Nowait Input or Output

Programs that use threads cannot call the Guardian `AWAITIO` or `AWAITIOX` procedure with `-1` as a file parameter (applying the call to the oldest incomplete operation pending on each file). `AWAITIOX` can be used to poll for completion on a specific file, although a thread must complete all nowait input or output operations before yielding control to another thread. In summary:

- No calls to `AWAITIO[X]()` with a file number of `-1` are made outside of pthreads.
- No calls to `AWAITIO[X]()` with a file number registered with pthreads are made outside of pthreads.

Yielding Control

Because Standard POSIX Threads are never preempted, a call to the `sched_yield()` function might be needed to allow time-critical run-time threads to execute. This function releases the processor to another thread of the same or higher priority. The calling thread may get control immediately if no threads of the same or higher priority are ready to run.

Be careful using the `sched_yield()` function. Misuse causes unnecessary context switching, which increases overhead without allowing the other threads to execute. For example, it is counterproductive for a thread to yield control while it has a needed resource locked.

Using Signals

- The following signal features are not supported:
 - Job Control Signals. The stop a process or continue a process actions implied by these signals are also not supported.
 - The Real-time Signals Extension option.
- Use the `sigaction()` function, not the `signal()` function, to specify the action to be taken when a given signal is received.
- The threads library contains jacket routines for the following OSS system and library functions:

```
sigaction()
sigpending()
sigsuspend()
pause()
sleep()
```

In addition, `spt_alarm()` and `spt_signal()` are provided for thread aware signal handling (see **Thread-Aware Signal Handling**).

- The `pthread_kill()` function provides a mechanism for synchronously directing a signal to a thread in the calling process.
- A signal is handled in the context of the given thread. However, the signal action (terminating or stopping) might affect the process as a whole. See **Thread-Aware Signal Handling**.
- Signals sent with the `pthread_kill()` function are queued in FIFO order for the target thread. More than one instance of the same signal might be pending for a thread. However, applications should not rely on this ordering.
- You must specify whether a signal listed in **Table 57: Signals Processed by Threads** as generating a saveabend actually generates a saveabend by specifying the `-Wsaveabend` flag in `c89` or `c99`, or `saveabend` in `nld`, `ld`, `eld`, or `xld`.



- If a signal is delivered to a thread waiting on a condition variable, then upon return from the signal handler, the thread will resume waiting for the condition variable as if it were not interrupted.
- Non thread-aware libraries linked to the application should not change the signal handler installed by the Standard POSIX Thread Library as it might result in unexpected application behavior.

Using Signals With the `setjmp()` and `longjmp()` Functions

Using the `setjmp()` and `longjmp()` functions can interfere with signal handling. The `longjmp()` function causes a thread to jump to the execution context saved by a previous call to the `setjmp()` function, but it does not restore the signal mask of the process. If the `longjmp()` function is called within a signal handler, the signal mask used in the signal handler is retained when execution resumes at the `setjmp()` location. The exception mechanism uses the `setjmp()` and `longjmp()` functions and behaves similarly.

The `sigsetjmp()` and `siglongjmp()` functions have an option to save and restore the signal mask. This option can cause signals that were enabled by another thread to be inadvertently masked.

When handling signals:

- Do not use the `raise()` function for an exception, and do not use the `longjmp()` function to exit a synchronous signal handler. Simply returning from a signal handler is safe and is the recommended practice.
- Use exception handlers instead of the `setjmp()` and `longjmp()` functions for threaded OSS applications.
- Do not use the option of the `sigsetjmp()` and `siglongjmp()` functions that saves and restores the signal mask. Doing so can cause signals enabled by another thread to be masked.

Spawning a New Process

Using the `fork()` or `tdm_fork()` function in a multithreaded environment can cause problems.

Applications can use either of these functions safely as follows: A parent process that has already called a function can also use either of the fork functions to create a child process if the child process calls any of the `exec` functions, the `tdm_execve()`, or `tdm_execep()` function immediately upon its creation.

A threaded application might alternatively use `pthread_atfork()` to declare as many as three fork handlers: a prepare handler called before the fork, a parent handler called after the fork in the parent process, and a child handler called after the fork in the child process.

When a fork is called, only the calling thread is duplicated in the child process. The expected usage of `pthread_atfork()` is for the prepare handler to acquire all mutex locks of interest to the child, and the other two fork handlers to release them.

Beginning with the H06.24 and J06.13 RVUs, 64-bit OSS processes are supported. See [64-Bit OSS Processes](#) for details of creating 64-bit OSS processes.

Isolating Faults

Because all threads created by a process share the process address space, there is no address space protection between the application threads in a process. One application thread can corrupt data, causing other threads or the entire process to fail.

Single Versus Multiple Application Threads

OSS threaded applications can be classified as follows, in order of increasing complexity:



- **Programs With a Single Application Thread**
- **Programs With Multiple Independent Application Threads**
- **Programs With Multiple Interdependent Application Threads**

Programs With a Single Application Thread

Externally, programs with a single application thread behave like standard, sequential (nonthreaded) programs. The program begins execution in the `main()` procedure; it then calls other procedures that, in turn, can call still other procedures. This type of application includes simple OSS client programs.

In single-threaded applications:

- Process resources can be used freely.
No synchronization is needed. The application can freely use process resources such as process and module globals, the heap, file descriptors, SQL cursors, and the signal mask. Mutexes and condition variables are not needed.
- Standard libraries can be used freely.
The application can share access to library data, including implicit context maintained across multiple library calls. For example, the application thread can use the TMF transaction jacket routines and no other thread can change the current transaction of the process.
- One application thread cannot block another from executing. Process-blocking operations have minimal effect.
- One application thread cannot corrupt another.
The thread does not share its address space with any other application threads. The application thread is protected by the address-space protection provided for the process, offering fault isolation.

The following general threading considerations affect single-threaded applications but have less effect than for processes with multiple application threads:

- Using process-blocking calls
Although process-blocking calls will not affect other will not affect other application threads, they can interfere with time-critical threads.
- Isolating faults and yielding control
If the application thread can execute for long periods (more than a second) without relinquishing control, the application might still need to use the `sched_yield()` function to allow time-critical threads to execute.

These general threading considerations apply to single-threaded applications:

- Using nowait input or output as described in **Using Nowait Input or Output**.
- Spawning a process as discussed in **Spawning a New Process**.

Programs With Multiple Independent Application Threads

Programs containing multiple threads can improve performance by providing concurrent execution. An application thread is independent if it does not explicitly share application resources that exist in the process environment, such as application file opens, SQL cursors, or modifiable application data structures.

This type of application includes simple OSS servers that maintain the application state in a database. Such a server simplifies server design because then application state does not need to be maintained in the server and thus most



servers can be context-free. In cases where context must be maintained across multiple requests, the context is generally private to the client and should not be shared.

Threading Considerations describes some considerations for developing an application with multiple application threads.

Programs With Multiple Interdependent Application Threads

An application thread is interdependent if it shares application resources that exist in the process environment, such as application file opens, SQL cursors, and modifiable application data structures. Applications with multiple interdependent application threads must synchronize access to the shared application resources.

This type of application includes OSS servers that maintain application state in the server to improve performance or that share resources, such as file opens, to reduce resource consumption. This type of application also includes client programs that use parallelism.

Programs with multiple interdependent application threads can be synchronized by using mutexes to reserve access to a shared resource and by using condition variables to wait for a shared resource to be in a certain state.

Poor performance can result if inappropriate synchronization techniques are used. Synchronizing at too coarse a level can overly restrict concurrency and cause a server to serialize processing, dramatically increasing the application's service time. Synchronizing at too fine a level can result in an increase in the processor cycles spent on initializing, obtaining, and releasing mutexes and dispatching threads.

Jacket Routines

A jacket routine invokes a nonblocking (nowait) version of the system call and suspends the thread until the thread's operation is complete. Standard POSIX Threads provides jacket routines for `SERVERCLASS_SEND` and `SEVERCLASS_SENDL` calls and for TMF transaction demarcation.

Full reference and syntax information for the jacket routines is available online in the OSS environment using the `man` command.

SERVERCLASS_SEND and SERVERCLASS_SENDL Jacket Routines

The `SPT_SERVERCLASS_SEND()` function in the threads library sends a message to and receives a reply from a server process in a Pathway server class. This jacket routine invokes a nowait version of the system `SERVERCLASS_SEND` procedure. The thread that makes the call is suspended until the send operation is complete.

The `SPT_SERVERCLASS_SENDL()` function, which is supported on J06.07 and later J-series RVUs and H06.18 and later H-series RVUs only, also sends a message to and receives a reply from a server process in a Pathway server class. This jacket routine invokes a nowait version of the system `SERVERCLASS_SENDL` procedure. The thread that makes the call is suspended until the send operation is complete. This function allows for messages up to two megabytes long when Expand-over-ServerNet is used. This function does not support using Expand-over-IP.

These functions can be used with the `SPT_SERVERCLASS_SENDL()` function:

```
spt_INITRECEIVE()  
spt_RECEIVEREADL()  
spt_REPLYXL()
```

Another jacket routine, the `SPT_SERVERCLASS_SEND_INFO()` function returns information about the last `SPT_SERVERCLASS_SEND()` or `SPT_SERVERCLASS_SENDL()` call that was attempted or completed by the current thread. The `SPT_SERVERCLASS_SEND_INFO()` function ensures that the thread gets the information for the last message sent by the thread rather than by the process.

The `SERVERCLASS_SENDL` jacket routines (`SPT_SERVERCLASS_SENDL()`, `spt_INITRECEIVE()`, `spt_RECEIVEREADL()`, and `spt_REPLYXL()`) are included in the Standard POSIX Threads library on J06.07 and later J-series RVUs, H06.18 and later H-series RVUs, and L-series only:

```
c89 -o program program .o -lsptdll
```



```
c89 -o program program .o -lxsptdll
```

The SERVERCLASS_SEND_ jacket routines are included in the Standard POSIX Threads library that can be bound to an application requiring them:

```
c89 -o program program .o -lzsptsrl (TNS/R programs)
c89 -o program program .o -lzsptdll (TNS/E programs)
c89 -o program program .o -lxsptdll (TNS/X programs)
```

SERVERCLASS_DIALOG Jacket Routines

The Pathsend SERVERCLASS_DIALOG jacket routines are used to:

- Establish a dialog with a server instance in a Pathway server class
- Send and receive messages within the dialog
- Abort or end the dialog

The Context-Sensitive Pathway requester can be written using the thread-aware SERVERCLASS_DIALOG jacket routines to communicate with a server instance in the Pathway server class. The Pathway requester initiates the dialog using SPT_SERVERCLASS_DIALOG_BEGIN_ () routine and sends the first message of the dialog. This thread-aware jacket routine returns the dialog ID that is used in subsequent operations to communicate with the Pathway server. The Pathway requester can invoke multiple sends using SPT_SERVERCLASS_DIALOG_SEND_ () within the dialog to send data to the server for processing. If the requester encounters an error while sending the data, the requester can abort the dialog using the SPT_SERVERCLASS_DIALOG_ABORT_ () routine.

The Pathway requester can use multiple SPT_SERVERCLASS_DIALOG_SEND_ () calls to have all requests serviced by the same server and all operations performed in a single context.

If the requester encounters Pathsend errors during the server class operation, the requester can call SPT_SERVERCLASS_SEND_INFO_ () to obtain information about the Pathsend error and file-system error.

Either the requester or the server can abort the dialog, but only the server can end it. The server ends the dialog by returning file-system error 0 (FEOK). The requester then calls SPT_SERVERCLASS_DIALOG_END_ () to clean up resources after the server has ended the dialog. To abort the dialog, the requester calls SPT_SERVERCLASS_DIALOG_ABORT_ ().

The SERVERCLASS_DIALOG_ jacket routines are included in the Standard POSIX Threads library, which can be bound to an application requiring the routines:

```
c89 -o program program .o -lzsptsrl (TNS/R programs)
c89 -o program program .o -lzsptdll (TNS/E programs)
c89 -o program program .o -lxsptdll (TNS/X programs)
```

TMF Transaction Jacket Routines

The following Transaction Management Facility (TMF) transaction jacket routines invoke the TMF calls and manage the multiple concurrent transactions rooted in a process:

```
SPT_ABORTTRANSACTION ()
SPT_BEGINTRANSACTION ()
SPT_ENDTRANSACTION ()
SPT_RESUMETRANSACTION ()
```

The syntax and semantics of these function calls are the same as those of the similarly named TMF calls.

When using the TMF transaction jacket routines:



- The `transaction_tag` parameter of the `SPT_BEGINTRANSACTION()` function is required.
- Each thread can have at most one current transaction. The system `RESUMETRANSACTION` procedure is automatically called during thread dispatch to maintain the correct transaction context.
- Each transaction is initially associated with only the thread that created it by calling the `SPT_BEGINTRANSACTION()` function.
- Another thread can be associated with a specific transaction by calling the `SPT_RESUMETRANSACTION()` function and specifying the `transaction_tag` value returned by the initial `SPT_BEGINTRANSACTION()` function call.
- If a thread is created, it does not inherit its parent's transaction; the created thread's transaction context is nil.
- If a thread terminates or is destroyed while it has an active transaction, that transaction is then inaccessible from the process. If cancellation is a possibility, use exception handlers for `pthread_cancel_e`. Alternatively, use the `pthread_cleanup_push()` and `pthread_cleanup_pop()` functions to ensure that the transaction is terminated if the thread exits or is cancelled.
- If a transaction is aborted by either an operator, network partition, or another process, an error is reported when the initiating thread attempts to operate on the transaction.
- A process using the TMF transaction jacket routines must not call:
 - The TMF procedures `BEGINTRANSACTION`, `ABORTTRANSACTION`, `ENDTRANSACTION`, or `RESUMETRANSACTION`
 - The SQL/MP `BEGIN-WORK` and `COMMIT-WORK` procedures
- A process can perform up to the default of 100 concurrent transactions, with up to 1000 transactions maximum declared using the `spt_setTMFConcurrentTransaction()` function. If the maximum is exceeded, a call to the `spt_setTMFConcurrentTransaction()` function fails, returning error number 83 (too many transactions).

Reference Pages for Thread Functions

For full syntax and usage information about the Standard POSIX Threads functions, see the *Open System Services System Calls Reference Manual*.

Full reference and syntax information for the `SERVERCLASS_SEND` and TMF transaction demarcation jacket routines is available online in the OSS environment using the `man` command.

Standard POSIX Threads does not support every thread function listed in the reference pages; for example, the various `pthread_rwlock`, `pthread_nsg`, `pthread_rad`, and `pthread_exc` functions are not part of the Standard. Many of the supported functions are changed as well. Differences between the previous standard and the current standard are described in the *Open System Services Porting Guide*.

All the thread functions are listed in [Documented OSS Functions](#) and [API Interoperability in Native Processes](#).

Threaded Application Examples

The following example application creates a threaded time-of-day server socket that listens for connections and upon connection, creates a thread to handle the connection.

Threaded Time-of-Day Server Socket

```
/* Threaded time-of day server socket. */
/* If compiled for TNS/R, specify the ZSPTSRL compilation option. */
```



```

/* If compiled for TNS/E, specify the ZSPTDLL, the standard POSIX threads */
/* If compiled for TNS/X, specify the XSPTDLL, the standard POSIX threads */
/* shared run-time library. */

#include <assert.h>
#include <fcntl.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <strings.h>

/*
Defines thread-aware functions calls to the threads package:
*/

#define SPT_THREAD_AWARE
#include <sptthread.h>

/*
Defines a short macro to convert file descriptors to non-blocking (T1248's
thread-aware functions are thread-aware only if the file descriptor is non-
blocking):
*/

#define fd_make_nonblock(fd) \
    assert(fcntl(fd, F_SETFL, fcntl(fd, F_GETFL) | O_NONBLOCK) != -1)

/*
Defines the port the server will run on:
*/

#define SERVER_PORT 3000

/*
Shows the worker thread that is created after each accept:
*/

void *worker_thread(void* sock_fd)
{
    char buffer[1024];
    time_t now;

    time(&now);
    strcpy(buffer, ctime(&now));
    send(*(int *)sock_fd, &buffer, strlen(buffer), 0);
    close(*(int *)sock_fd);
    return NULL;
}

int main(int argc, char *argv[])
{
    int inet_addr_port = SERVER_PORT;
    int sock=-1, accept_fd=-1;
    size_t size;
    struct sockaddr_in servaddr, accept_addr;

```



```

/*
Defines one pthread_t per request we intend to handle:
*/

pthread_t my_thread;
pthread_attr_t my_attr;

/*
Turns on concurrency:
*/

if(pthread_setconcurrency(1))
{
    perror("pthread_setconcurrency");
    exit(-1);
}

/*
Initializes attribute:
*/

if(pthread_attr_init(&my_attr))
{
    perror("pthread_attr_init");
    exit(-1);
}

/*
Sets detach Attribute:
*/

if(pthread_attr_setdetachstate(&my_attr, PTHREAD_CREATE_DETACHED))
{
    perror("pthread_attr_setdetachstate");
    exit(-1);
}

/*
Creates socket:
*/

if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
{
    perror("Socket Error");
    exit(-1);
}

/*
Sets up Bind:
*/

bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(inet_addr_port);

```

```

    if (bind(sock, (struct sockaddr *) &servaddr, sizeof(servaddr)) < 0)
    {
        perror("Bind Error");
        exit(-1);
    }

    fd_make_nonblock(sock);

    /*
    Sets socket to listen:
    */

    if (listen(sock, 1) < 0)
    {
        perror("Listen Error");
        exit(-1);
    }

    /*
    Starts server:
    */

    while(1)
    {
        bzero(&accept_addr, sizeof(accept_addr));
        accept_fd = accept(sock, NULL, NULL);
        if (accept_fd < 0)
        {
            perror("Accept Error");
            exit(-1);
        }

        /*
        Dispatches a thread to handle service request:
        */

        if(pthread_create(&my_thread, &my_attr, &worker_thread, &accept_fd)
            {
                perror("pthread_create");
                exit(-1);
            }
        } /* while */
    } /* main */

```

Threaded Time-of-Day Server Socket Using \$RECEIVE also creates a threaded time-of-day server socket, but using \$RECEIVE.

Threaded Time-of-Day Server Socket Using \$RECEIVE

```

#include <errno.h>
#include <assert.h>
#include <cextdecs.h(FILE_OPEN_, READX, FILE_CLOSE_, WRITEX, FILE_GETINFO_)>
#include <cextdecs.h(FILE_GETRECEIVEINFO_)>
#include <cextdecs.h(REPLYX)>
#include <cextdecs.h(READUPDATEX)>

```

```

#define SPT_THREAD_AWARE
/*
Must come after cextdecs.h:
*/
#include <spthread.h>

void * server_thread(void* target)
{
char buffer[1024];
time_t now;
int error;

    time(&now);
    strcpy(buffer, ctime(&now));

    /*
A compiler warning will occur regarding the next line, which is OK:
*/

    error = spt_REPLYX(buffer, (short) strlen(buffer), NULL,
        (short)target, 0);

    fprintf(stderr, "server_thread: spt_REPLYX: error: %d\n", error);
    fflush(stderr);
    return NULL;
}

int main(void)
{

    pthread_t my_thread;
    pthread_attr_t my_attr;
    char *filename = "$RECEIVE";
    char buffer[1024];
    long bytesread, error;
    short receive_info[17];
    short dialog_info;
    short target;
    short filenum;

    /*
Sets concurency level:
*/

    if(pthread_setconcurrency(1))
    {
        perror("pthread_setconcurrency");
        exit(-1);
    }

    /*
Initializes attribute:
*/

    if(pthread_attr_init(&my_attr))

```

```

        {
            perror("pthread_attr_init");
            exit(-1);
        }

/*
Sets detach state for thread attribute:
*/

if(pthread_attr_setdetachstate(&my_attr,PTHREAD_CREATE_DETACHED)!=0)

    {
        perror("pthread_attr_setdetachstate");
        exit(-1);
    }

error = FILE_OPEN_(filename, (short)strlen(filename), &filenum,
    0, , 1, 10, 1);
fprintf(stderr,"main: FILE_OPEN_: error: %d\n",error);
assert(error == 0);
error = spt_INITRECEIVE(filenum, 10);
fprintf(stderr,"main: spt_initReceive: error: %d\n",error);
fflush(stderr);
assert(error == 0);

while (1)

{
    error = spt_RECEIVEREAD(filenum, buffer, (short)sizeof(buffer),
        &bytesread, -1, receive_info, &dialog_info);
    fprintf(stderr,"main_thread: spt_RECEIVEREAD: error: %d\n",error);

    target = receive_info[2];

    if(pthread_create(&my_thread, &my_attr, &server_thread, 0
        (void *)target))

        {
            perror("pthread_create");
            exit(-1);
        }

    } /* while */
} /* main */

```

Thread-Aware and Nonblocking OSS Functions

If a function must wait for an I/O operation to complete, the function blocks its caller (either a thread or a process) from continuing until the I/O operation completes. A function that blocks all threads while it waits for I/O to complete is a process-blocking function. Process-blocking functions are not thread aware. If a thread calls a process-blocking function, the calling thread and all the other threads in the process are prevented from continuing until the I/O operation is complete and the function returns a status indication. Standard OSS I/O library functions are not thread-aware.

In contrast to a process-blocking function, a thread-aware function blocks only the thread calling that function, instead of blocking all threads in the process. The other threads in the process can continue doing work while the calling thread

waits for I/O to complete. A thread-aware function is sometimes called a thread-blocking function, because it blocks only the calling thread.

Sometimes it is not enough for a function to be thread-aware. There can be times when you do not want even the calling thread to wait for I/O to complete. Instead, you want the function to either succeed immediately or to fail immediately. Functions that do not block the calling thread or the other threads in the process are nonblocking functions. Nonblocking I/O functions are sometimes called asynchronous I/O functions.

The behavior of an OSS I/O library function can vary according to these factors:

- Whether the file descriptor is for a regular file or a non-regular file.
- Whether the function was designed to be process-blocking, thread-aware (thread-blocking), or nonblocking for that type of file.
- Whether the file descriptor passed to the function is nonblocking. A file descriptor is nonblocking when one of these conditions is true:
 - The file descriptor has the `O_NONBLOCK` flag set.
 - The file descriptor has been set to non-blocking mode by setting the argument to the *request* parameter to `F_SETFL` when calling the `fcntl()` function.

NOTE: In this section, thread-aware and nonblocking thread-aware functions are listed explicitly. However, instead of calling these functions directly, Hewlett Packard Enterprise recommends that you use preprocessor directives when compiling your application. See **Preprocessor Directives Automatically Map Standard System Calls to Thread-Aware or Nonblocking Thread-Aware System Calls**.

Regular Files

For regular files, the `spt_*z()` set of functions are thread-aware. The `spt_*z()` set of functions block the calling thread but allow the other threads to run, whether or not the file descriptor is nonblocking. These functions never return the errors `EAGAIN`, `EINPROGRESS`, or `EWOULDBLOCK` for regular files, so there are no nonblocking thread-aware functions for regular files.

In contrast, the `spt_*()` and `spt_*x()` sets of functions are not thread-aware for regular files. For regular files, these functions are always process-blocking.

The `spt_*z()` set of functions are supported on systems running J06.04 and later J-series RVUs and H06.15 and later H-series RVUs, and are listed in the rightmost column, labeled Thread-Aware (Regular Files) and Nonblocking Thread-Aware (Non-Regular Files) Functions, of **System and Equivalent Thread-Aware Functions**.

In addition to the functions listed in **System and Equivalent Thread-Aware Functions**, these additional functions, although not thread-aware, support the use of `spt_*z()` set of thread-aware functions by serializing file operations on an open file:

```
spt_fstat64z()  
spt_fstatz()  
spt_lseek64z()  
spt_lseekz()
```

Serialization of File Operations On an Open File

The version of the Standard POSIX Threads library that supports the `spt_*z()` set of functions serializes file operations on an open file. If a thread calls one of the `spt_*z()` set of functions to access a file that already has a file operation in progress by a different thread, the calling thread is blocked until the prior file operation is complete. For example:



Procedure

1. Thread A calls `spt_readz()` to read the open file `MyDataFile`. The file does not have an operation in progress, so thread A starts reading the file.
2. Thread B calls `spt_writez()` to write to the same open file `MyDataFile`. Because the read operation started by thread A is still in progress, thread B is blocked and must wait for thread A's read operation to complete.

Non-Regular Files

For non-regular files, there are functions that can provide thread-aware behavior or both nonblocking and thread-aware behavior:

- The `spt_*()` set of functions are thread aware only if the file descriptor is nonblocking. The behavior of these functions depend on how the file descriptor `O_NONBLOCK` flag is set:
 - If `O_NONBLOCK (fd flag)` is set, then the call will not block the other calling threads. It might block the calling thread depending on the function call.
 - If `O_NONBLOCK (fd flag)` is not set, then most functions block the entire process. Some functions, like `spt_fd_read_ready()` and `spt_fd_write_ready()`, will block only the calling thread.

The `spt_*()` set of functions are listed in the Thread-Aware Functions (Non-Regular Files Only) column of **System and Equivalent Thread-Aware Functions**

- The `spt_*x()` set of functions are thread-aware whether or not the file descriptor is nonblocking. If the `O_NONBLOCK` flag is set, these functions are also nonblocking: they do not block the calling thread or the other threads, and they return one of these errors to the application: `EAGAIN`, `EINPROGRESS`, or `EWouldBlock`. Otherwise, these functions block the calling thread but allow other threads to run. These functions remove the need for the application to explicitly set the `O_NONBLOCK` flag of the file descriptor to enable thread-aware behavior.

The `spt_*x()` set of functions are available on systems running J06.03 and later J-series RVUs, H06.05 and later H-series RVUs, and G06.28 and later G-series RVUs, and are listed in the Nonblocking Thread-Aware Functions (Non-Regular Files Only) column of **System and Equivalent Thread-Aware Functions**.

- For non-regular files, the `spt_*z()` set of functions have the same behavior as `spt_*x()` functions. The `spt_*z()` set of functions are thread-aware whether or not the `O_NONBLOCK` flag of the file descriptor is set. If the `O_NONBLOCK` flag is set, these functions are also nonblocking: they do not block the calling thread or the other threads, and they return one of these errors to the application: `EAGAIN`, `EINPROGRESS`, or `EWouldBlock`. Otherwise, these functions block the calling thread but allow other threads to run.

The `spt_*z()` set of functions are supported on systems running J06.04 and later J-series RVUs and H06.15 and later H-series RVUs, and are listed in the rightmost column, Thread-Aware (Regular Files) and Nonblocking Thread-Aware (Non-Regular Files) Functions, of **System and Equivalent Thread-Aware Functions**



Table 59: Comparison of `spt_*`, `spt_*x`, and `spt_*z` Functions

Function Set	Behavior for Non-Regular Files	Behavior for Regular Files
<code>spt_*()</code>	File descriptor is nonblocking: ◦ Function is thread-aware (other threads can continue to run) ◦ Function blocks the calling thread. File descriptor is not nonblocking: this function is process-blocking (all threads in the process must wait for I/O to complete).	Always process-blocking (all threads in the process must wait for I/O to complete).
<code>spt_*x()</code>	Always thread-aware. Can be nonblocking for the calling thread. File descriptor is nonblocking: Function is nonblocking thread-aware (does not block any threads, including the calling thread): it returns an error instead of blocking the thread. File descriptor is not nonblocking: ◦ Function is thread-aware (other threads can continue to run) ◦ Function blocks the calling thread.	Always process-blocking (all threads in the process must wait for I/O to complete)
<code>spt_*z()</code>	Always thread-aware. Can be nonblocking for the calling thread. File descriptor is nonblocking: Function is nonblocking thread-aware (does not block any threads, including the calling thread): it returns an error instead of blocking the thread. File descriptor is not nonblocking: ◦ Function is thread-aware (other threads can continue to run) ◦ Function blocks the calling thread.	Always thread-aware:¹ ◦ Function is thread-aware (other threads can continue to run) ◦ Function blocks the calling thread. I/O operations by multiple threads on the same open file are serialized.

¹The functions `spt_fstat64z()`, `spt_fstatz()`, `spt_lseek64z()`, and `spt_lseek64z()` are not thread aware but support the `spt_*z()` thread-aware functions for regular files by serializing file operations on an open file.

Equivalent System, Thread-Aware, and Nonblocking Thread-Aware Functions

The table **System and Equivalent Thread-Aware Functions** lists standard functions that have equivalent thread-aware functions. For example:



- `read()` is a process-blocking function for both regular files and for non-regular files.
- `spt_read()` is a thread-aware function with this behavior:
 - For non-regular files, the file descriptor must be nonblocking for this function to be thread-aware. This function blocks the calling thread, but allows other threads to run.
 - For regular files, this function blocks the entire process.
- `spt_readx()` is a thread-aware function with this behavior:
 - For non-regular files, this function does not require the file descriptor to be nonblocking for this function to be thread-aware. If the file description is nonblocking, this function does not block the calling thread, and returns one of these errors to the application: EAGAIN, EINPROGRESS, or EWOULDBLOCK. Otherwise, this function blocks the calling thread but allows other threads to run.
 - For regular files, this function blocks the entire process.
- `spt_readz()` is a thread-aware function with this behavior:
 - For non-regular files, this function does not require the file descriptor to be nonblocking for this function to be thread-aware. If the file descriptor is nonblocking, this function does not block the calling thread, and returns one of these errors to the application: EAGAIN, EINPROGRESS, or EWOULDBLOCK. Otherwise, this function blocks the calling thread but allows other threads to run. (This is the same behavior as the `spt_readx()` function.)
 - For regular files, this function blocks the calling thread but allows other threads to run. This function does not require the file descriptor to be nonblocking for this function to be thread-aware.

NOTE: To determine which RVUs support an OSS function, see [Documented OSS Functions](#)

Table 60: System and Equivalent Thread-Aware Functions

Standard (Process-Blocking) OSS Functions	Thread-Aware Functions (Non-Regular Files Only)	Nonblocking Thread-Aware Functions (Non-Regular Files Only)	Thread-Aware (Regular Files) and Nonblocking Thread-Aware (Non-Regular Files) Functions
<code>accept()</code>	<code>spt_accept()</code>	<code>spt_acceptx()</code>	
—	<code>spt_alarm()</code>	—	
<code>close()</code>	<code>spt_close()</code>	<code>spt_closex()</code>	<code>spt_closez()</code>
<code>connect()</code>	<code>spt_connect()</code>	<code>spt_connectx()</code>	
<code>dup2()</code>	—	<code>spt_dup2x()</code>	
<code>fclose()</code>	<code>spt_fclose()</code>	<code>spt_fclosex()</code>	
<code>fcntl()</code>	<code>spt_fcntl()</code>	<code>spt_fcntlx()</code>	<code>spt_fcntlz()</code>
<code>fflush()</code>	<code>spt_fflush()</code>	<code>spt_fflushx()</code>	
<code>fgetc()</code>	<code>spt_fgetc()</code>	<code>spt_fgetc_x()</code>	
<code>fgets()</code>	<code>spt_fgets()</code>	<code>spt_fgetsx()</code>	

Table Continued

Standard (Process-Blocking) OSS Functions	Thread-Aware Functions (Non-Regular Files Only)	Nonblocking Thread-Aware Functions (Non-Regular Files Only)	Thread-Aware (Regular Files) and Nonblocking Thread-Aware (Non-Regular Files) Functions
<code>fgetwc()</code>	<code>spt_getwc()</code>	<code>spt_getwcx()</code>	
<code>fprintf()</code>	<code>spt_fprintf()</code>	<code>spt_printfx()</code>	
<code>fputc()</code>	<code>spt_fputc()</code>	<code>spt_fputcx()</code>	
<code>fputs()</code>	<code>spt_fputs()</code>	<code>spt_fputsx()</code>	
<code>fputwc()</code>	<code>spt_fputwc()</code>	<code>spt_fputwcx()</code>	
<code>fread()</code>	<code>spt_fread()</code>	<code>spt_freadx()</code>	
<code>fsync()</code>	—	—	<code>spt_fsyncz()</code>
<code>ftruncate()</code>	—	—	<code>spt_ftruncatez()</code>
<code>ftruncate64()</code>	—	—	<code>spt_ftruncate64z()</code>
<code>fwrite()</code>	<code>spt_fwrite()</code>	<code>spt_fwritex()</code>	
<code>getc()</code>	<code>spt_getc()</code>	<code>spt_getcx()</code>	
<code>getchar()</code>	<code>spt_getchar()</code>	<code>spt_getcharx()</code>	
<code>gets()</code>	<code>spt_gets()</code>	<code>spt_getsx()</code>	
<code>getw()</code>	<code>spt_getw()</code>	<code>spt_getwx()</code>	
<code>getwc()</code>	<code>spt_getwc()</code>	<code>spt_getwcx()</code>	
<code>getwchar()</code>	<code>spt_getwchar()</code>	<code>spt_getwcharx()</code>	
<code>printf()</code>	<code>spt_printf()</code>	<code>spt_printfx()</code>	
<code>putc()</code>	<code>spt_putc()</code>	<code>spt_putcx()</code>	
<code>putchar()</code>	<code>spt_putchar()</code>	<code>spt_putcharx()</code>	
<code>puts()</code>	<code>spt_puts()</code>	<code>spt_putsx()</code>	
<code>putw()</code>	<code>spt_putw()</code>	<code>spt_putwx()</code>	
<code>putwc()</code>	<code>spt_putwc()</code>	<code>spt_putwcx()</code>	
<code>putwchar()</code>	<code>spt_putwchar()</code>	<code>spt_putwcharx()</code>	
<code>read()</code>	<code>spt_read()</code>	<code>spt_readx()</code>	<code>spt_readz()</code>
<code>readv()</code>	<code>spt_readv()</code>	<code>spt_readvx()</code>	<code>spt_readvz()</code>
<code>recv()</code>	<code>spt_recv()</code>	<code>spt_recvx()</code>	
<code>recvfrom()</code>	<code>spt_recvfrom()</code>	<code>spt_recvfromx()</code>	
<code>recvmsg()</code>	<code>spt_recvmsg()</code>	<code>spt_recvmsgx()</code>	

Table Continued

Standard (Process-Blocking) OSS Functions	Thread-Aware Functions (Non-Regular Files Only)	Nonblocking Thread-Aware Functions (Non-Regular Files Only)	Thread-Aware (Regular Files) and Nonblocking Thread-Aware (Non-Regular Files) Functions
<code>select()</code>	<code>spt_select()</code> <code>spt_select_single_np()</code>	—	
<code>send()</code>	<code>spt_send()</code>	<code>spt_sendx()</code>	
<code>sendto()</code>	<code>spt_sendto()</code>	<code>spt_sendto()</code>	
<code>sendmsg()</code>	<code>spt_sendmsg()</code>	<code>spt_sendmsgx()</code>	
<code>sigaction()</code>	<code>spt_sigaction()</code>	—	
—	<code>spt_signal()</code>	—	
<code>sigpending()</code>	<code>spt_sigpending()</code>	—	
<code>sigsuspend()</code>	<code>spt_sigsuspend()</code>	—	
<code>wait()</code>	<code>spt_wait()</code>	—	
<code>sleep()</code>	<code>spt_sleep()</code>	—	
—	<code>spt_usleep()</code>	—	
<code>system()</code>	<code>spt_system()</code>	—	
<code>vfprintf()</code>	<code>spt_vfprintf()</code>	<code>spt_vfprintfx()</code>	
<code>vprintf()</code>	<code>spt_vprintf()</code>	<code>spt_vprintfx()</code>	
<code>waitpid()</code>	<code>spt_waitpid()</code>	—	
<code>write()</code>	<code>spt_write()</code>	<code>spt_writex()</code>	<code>spt_writez()</code>
<code>writew()</code>	<code>spt_writew()</code>	<code>spt_writewx()</code>	<code>spt_writewz()</code>

Preprocessor Directives Automatically Map Standard System Calls to Thread-Aware or Nonblocking Thread-Aware System Calls

When you use preprocessor directives when compiling the application, standard system calls are automatically mapped to the appropriate Standard POSIX Thread system call, eliminating the need to explicitly call `spt_*()`, `spt_*x()`, or `spt_*z()` functions. Using preprocessor directives is preferred to using explicit calls to thread-aware or nonblocking thread-aware functions.

Hewlett Packard Enterprise highly recommends using only one of these preprocessor directive sets in an application:

```
#define SPT_THREAD_AWARE
#define SPT_THREAD_AWARE_PRAGMA
```



```
#define SPT_THREAD_AWARE_NONBLOCK
#define SPT_THREAD_AWARE_PRAGMA_NONBLOCK

#define SPT_THREAD_AWARE_XNONBLOCK
#define SPT_THREAD_AWARE_PRAGMA_XNONBLOCK
```

Mixing modules compiled with different preprocessor directive sets or compiled without preprocessor directives related to thread-aware behavior in an application might cause unexpected results. For more information, see **The `spt_*zO` Set of Functions Must Be Used Together.**

This table summarizes the behaviors for each of these preprocessor directives for non-regular and for regular files:

Preprocessor Directive	Behavior for Non-Regular File Descriptors	Behavior for Regular File Descriptors
<code>#define SPT_THREAD_AWARE</code> or <code>#define SPT_THREAD_AWARE_PRAGMA</code>	Thread-aware operations for nonblocking file descriptors Process-blocking operations for blocking file descriptors	Process-blocking operations for all file descriptors
<code>#define SPT_THREAD_AWARE_NONBLOCK</code> or <code>#define SPT_THREAD_AWARE_PRAGMA_NONBLOCK</code>	Nonblocking thread-aware operations for nonblocking file descriptors	Process-blocking operations for all file descriptors
<code>#define SPT_THREAD_AWARE_XNONBLOCK</code> or <code>#define SPT_THREAD_AWARE_PRAGMA_XNONBLOCK</code>	Thread-aware operations for blocking file descriptors	Thread-aware operations for all file descriptors

Defining Thread-Aware Functions for Non-Regular Files

For C applications, a macro to map `interface ()` to `spt_interface ()` is available when you use the `#define SPT_THREAD_AWARE` preprocessor directive before including `spthread.h` or when you use an equivalent compiler command option to compile the application.

For C++ applications, an alias to map `interface ()` to `spt_interface ()` is available when you use the `#define SPT_THREAD_AWARE_PRAGMA` preprocessor directive before including `spthread.h` or when you use an equivalent compiler command option to compile the application.

The following example illustrates using the `SPT_THREAD_AWARE_PRAGMA` define:

```
#define SPT_THREAD_AWARE_PRAGMA
#include <iostream.h> nolist
#include <spthread.h> nolist
int main()
...
```

For information about including a preprocessor directive in an application by specifying the directive in a compiler command option, see the `c89 (1)` or the `c99 (1)` reference page online or in the *Open System Services Shell and Utilities Reference Manual*.

Applications can explicitly call the functions listed in the Thread-Aware Functions (Non-Regular Files Only) column of **System and Equivalent Thread-Aware Functions** table without setting any defines before including the `spthread.h` header file.

Defining Nonblocking Thread-Aware Functions for Non-Regular Files

The `spt_*x ()` set of nonblocking thread-aware functions for non-regular files are available on systems running J-series RVUs, H06.05 and later H-series RVUs, and G06.28 and later G-series RVUs.



For C applications, a macro to map `interface()` to `spt_interfacex()` is available when you use the `#define SPT_THREAD_AWARE_NONBLOCK` preprocessor directive before including `spthread.h` or when you use an equivalent compiler command option to compile the application.

For C++ applications, an alias to map `interface()` to `spt_interfacex()` is available when you use the `#define SPT_THREAD_AWARE_PRAGMA_NONBLOCK` preprocessor directive before including `spthread.h` or when you use an equivalent compiler command option to compile the application.

This example illustrates using the `SPT_THREAD_AWARE_NONBLOCK` preprocessor directive in an application:

```
#define SPT_THREAD_AWARE_NONBLOCK
#include <spthread.h>
main()
...
```

For information about including a preprocessor directive in an application by specifying the directive in a compiler command option, see the `c89(1)` or the `c99(1)` reference page online or in the *Open System Services Shell and Utilities Reference Manual*.

For non-regular files:

- The `spt_*z()` set functions have the same behavior as the `spt_*x()` set of functions.
- For C applications, the behavior of the function is the same whether you use the `#define SPT_THREAD_AWARE_NONBLOCK` or the `#define SPT_THREAD_AWARE_XNONBLOCK` preprocessor directive.
- For C++ applications, the behavior of the function is the same whether you use the `#define SPT_THREAD_AWARE_PRAGMA_NONBLOCK` or the `#define SPT_THREAD_AWARE_PRAGMA_XNONBLOCK` preprocessor directive.

The `spt_*z()` set of Nonblocking thread-aware functions for non-regular files are available on systems running J06.04 and later J-series RVUs and H06.15 and later H-series RVUs. To use these functions see **Defining Thread-Aware Functions for Regular Files**.

Applications can explicitly call the functions listed in the Nonblocking Thread-Aware Functions (Non-Regular Files Only) column of the table **System and Equivalent Thread-Aware Functions** without setting any defines before including the `spthread.h` header file.

Defining Thread-Aware Functions for Regular Files

Thread-aware functions for regular files are available on systems running J06.04 and later J-series RVUs and H06.15 and later H-series RVUs. These functions are also nonblocking thread-aware functions for non-regular files.

For C applications, a macro to map `interface()` to `spt_interfacez()` is available when you use the `#define SPT_THREAD_AWARE_XNONBLOCK` preprocessor directive before including `spthread.h` or when you use an equivalent compiler command option to compile the application.

For C++ applications, an alias to map `interface()` to `spt_interfacez()` is available when you use the `#define SPT_THREAD_AWARE_PRAGMA_XNONBLOCK` preprocessor directive before including `spthread.h` or when you use an equivalent compiler command option to compile the application.

NOTE: When you use the `#define SPT_THREAD_AWARE_XNONBLOCK` or `#define SPT_THREAD_AWARE_PRAGMA_XNONBLOCK`, if a function has a corresponding `spt_*x()` function but not a corresponding `spt_*z()` function, the function is mapped to the corresponding `spt_*x()` function.

The following example illustrates using the `SPT_THREAD_AWARE_XNONBLOCK` define:

```
#define SPT_THREAD_AWARE_XNONBLOCK
#include<spthread.h>
```

```
main()
...
```

For information about including a preprocessor directive in an application by specifying the directive in a compiler command option, see the `c89(1)` or the `c99(1)` reference page online or in the *Open System Services Shell and Utilities Reference Manual*.

Applications can explicitly call the functions listed in the rightmost column of **System and Equivalent Thread-Aware Functions** without setting any defines before including the `sptthread.h` header file.

The `spt_*z()` Set of Functions Must Be Used Together

The Standard POSIX Threads library serializes I/O functions on open files only for the `spt_*z()` set of functions. Therefore, if an `spt_*z()` function exists for a needed standard function, you should use the `spt_*z()` function instead of using `spt_*x()`, `spt_*()`, or unmapped standard functions directly.

For example, if you compile one module in a way that maps functions to corresponding functions in the `spt_*z()` set of functions and you compile another module in a way that maps functions to corresponding functions in the `spt_*()` or `spt_*x()` set of functions, you can get errors when the two modules operate on the same open file:

Procedure

1. Thread A calls `spt_readz()` to read the open file `MyDataFile`. The file does not have an operation in progress, so thread A starts reading the file.
2. Thread B calls `read()` to read to the same open file `MyDataFile`, but the module was not compiled using the `SPT_THREAD_AWARE_XNONBLOCK` preprocessor directive, so the call to `read()` was automatically mapped to `spt_read()`. Because the `spt_read()` function does not support serialization, `spt_read()` attempts to do the read and fails, returning the error `EALREADY`. The `EALREADY` error indicates that an operation is already in progress.

In addition, using modules that were not compiled with any of the preprocessor directives with modules that have been compiled using the `SPT_THREAD_AWARE_XNONBLOCK` or `SPT_THREAD_AWARE_PRAGMA_XNONBLOCK` preprocessor directives can cause unexpected results. For example, compiling a module so that it calls the unmapped standard function `close()` directly, instead of the mapped function `spt_closez()`, might leave a thread with a pending thread-aware regular file request suspended with no mechanism available to reschedule it for execution.

Using Thread-Aware Functions

File Stream Thread-Aware Function Usage shows using thread-aware functions with a file stream. If the stream is to be used with a nonblocking thread-aware function, it is not necessary to set the `O_NONBLOCK` flag.

File Stream Thread-Aware Function Usage

```
...
fs = fopen(path, mode);
fd = fileno(fs);
fcntl(fd, F_SETFL, fcntl(fd, F_GETFL) | O_NONBLOCK);
...
// Call thread-aware functions.
...
fclose(fs);
...
```

File Descriptor Thread-Aware Function Usage shows opening a file descriptor for use with a thread-aware function. If the file descriptor is to be used with a nonblocking thread-aware function, it is not necessary to set the `O_NONBLOCK` flag.

File Descriptor Thread-Aware Function Usage

```
...
filedes = open(path, oflag, mode | O_NONBLOCK);
...
// Call thread-aware functions.
...
close(filedes);
...
```

Open File Descriptor Thread-Aware Function Usage shows preparing an open file descriptor for use with a thread-aware function. If the file descriptor is to be used with a nonblocking thread-aware function, it is not necessary to set the `O_NONBLOCK` flag.

Open File Descriptor Thread-Aware Function Usage

```
...
fcntl(filedes, F_SETFL, fcntl(fd, F_GETFL) | O_NONBLOCK);
...
// Call thread-aware functions.
...
```

Pthreads Nonblocking Disk File I/O (SPT_THREAD_AWARE_NONBLOCK version) is a multithreaded program that performs I/O on multiple OSS regular files using the nonblocking thread-aware functions and guardian nowaited I/O.

NOTE:

For systems running J06.04 or later J-series RVUs or H06.15 or later H-series RVUs, using Guardian nowait I/O to perform I/O operations that block only the calling thread is not necessary if you change the define used in the sample program in **Pthreads Nonblocking Disk File I/O (SPT_THREAD_AWARE_NONBLOCK version)** from `SPT_THREAD_AWARE_NONBLOCK` to `SPT_THREAD_AWARE_XNONBLOCK`. See **Pthreads Thread-Aware Disk File I/O Using SPT_THREAD_AWARE_XNONBLOCK**.

Pthreads Nonblocking Disk File I/O (SPT_THREAD_AWARE_NONBLOCK version)

```
/*
 * sptdio.c - SPThreads Diskfile IO Example Program
 *
 * This Pthreads program demonstrates a multithreaded program that
 * can perform I/O on multiple OSS regular files (diskfiles) using
 * non-blocking OSS I/O and Guardian nowaited I/O.
 *
 * Note that OSS I/O on regular files is serialized,
 * even though the OSS non-blocking
 * interface is used. This is standard Unix behavior.
 *
 * The following threads can be created by this program:
 *   - An OSS read thread which uses uses OSS I/O to completely
 *     read a regular file.
 *   - An OSS write thread which uses uses OSS I/O to write
 *     1Mb of data to the regular file.
 *   - A Guardian read thread which uses Guardian nowait I/O to
 *     completely read a regular file.
 *   - A Guardian write thread which uses Guardian nowait I/O
 *     to write 1Mb of data to the regular file.
 *
 * USAGE SYNTAX:
 *
 * sptdio [-r <OSS-file>] [-w <OSS-file>] [-R <OSS-file>] [-W <OSS-file>]
```

```

*
*  FLAGS
*  -r <OSS-file>   Create a thread that uses OSS I/O to
*                  completely read the given OSS diskfile.
*  -w <OSS-file>   Create a thread that uses OSS I/O to write the
*                  given OSS diskfile.  1Mb of data is written to the file.
*  -R <OSS-file>   Create a thread that uses Guardian nowait I/O to
*                  completely read the given OSS diskfile.
*  -W <OSS-file>   Create a thread that uses Guardian nowait I/O to write
*                  the given OSS diskfile.  1Mb of data is written to
*                  the file.
*
*  EXAMPLE USAGE:
*
*      /home/user: run sptdio -r /tmp/r1 -w /tmp/w1 -R /tmp/r2 -W /tmp/w2
*
*      ==> All threads created...
*      ==> OSS_write_thread() wrote 1048576 bytes to file /tmp/w1
*      ==> OSS_read_thread() read 1581888 bytes from file /tmp/r1
*      ==> Grd_WRITE_thread() wrote 1048576 bytes to file /tmp/w2
*      ==> Grd_READ_thread() read 1581888 bytes from file /tmp/r2
*
*  OSS BUILD INSTRUCTIONS:
*
*  c89 sptdio.c -o sptdio -Wextensions -Wsystype=oss -g -D_XOPEN_SOURCE=1 \
*  -lzSPTDLL
*/

#include <limits.h>
#include <stdlib.h>
/* Use the non-blocking version of the thread aware library */
#define SPT_THREAD_AWARE_NONBLOCK
#include <spthread.h>

#include <tal.h>
#include <cextdecs.h(CANCELREQ, FILE_CLOSE_, FILE_GETINFO_, FILE_OPEN_,
\ READX, WRITE_X)>

/*****
/* LITERALS/DEFINES */
*****/

/* the maximum number of I/O threads created by this program */
#define MAX_THREADS 4

/* concurrency level value passed to pthread_setconcurrency() */
/* the value of 100000 sets the minimum scheduled quantum to */
/* 0.000001 seconds. */
#define CONCURRENCY_LEVEL 100000

```

```

/* I/O size used by all threads */
#define IOSIZE 8192

/* size of files that are written - 1Mb */
#define WRITE_FILESIZE 1048576

/*
 *-----
 * OSS_read_thread --
 *
 * Description:
 *
 * This thread reads an OSS file using OSS I/O.
 *
 * Note that OSS I/O on regular files is serialized and only one OSS I/O
 * per open can occur at a given time. Even though we use the
 * thread-aware library, the entire
 * program blocks while the OSS read() function performs the I/O.
 *
 * Results:
 *   none.
 *-----
 */

void *OSS_read_thread(void *OSS_read_file)
{
    int fd;
    int nbytes;
    char *iobuf;
    int ret;
    long long filesize = 0;

    /*-----
    /* prepare I/O buffer (page aligned, paged into physical memory) */
    /*-----

    if ((iobuf = malloc(IOSIZE)) == NULL)
    {
        printf("*** ERROR in OSS_read_thread: unable to allocate
            I/O buffer ***\n");
        exit(0);
    }
    memset(iobuf, 0, IOSIZE);

    /*-----
    /* open the OSS file */
    /*-----

    if ((fd = open(OSS_read_file, O_RDONLY | O_NONBLOCK)) < 0)
    {
        printf("*** ERROR in OSS_read_thread: open() of %s failed
            w/error %d ***\n", OSS_read_file, errno);

        exit(0);
    }

```



```

/*****
/* read the OSS file in a thread aware manner */
*****/

while (1)
{
    nbytes = read(fd, iobuf, IOSIZE);

    if (nbytes < 0)
    {
        if ((errno == EWOULDBLOCK) || (errno == EAGAIN))

/*****
/* NOTE that this code does not execute for regular (disk) files, because */
/* read() on a regular file never returns EWOULDBLOCK or EAGAIN          */
*****/

        ret = spt_fd_read_ready(fd, NULL);
        if (ret != 0)
        {
            printf("*** ERROR in OSS_read_thread:  spt_fd_read_ready()
                    failed w/err %d\n",ret);
            exit(0);
        }
    }
    else
    {
        printf("*** ERROR in OSS_read_thread:  read() failed w/error
                %d ***\n",errno);
        exit(0);
    }
    else if (nbytes == 0) /* EOF */
    {
        break;
    }
    else /* read successful */
    {
        filesize += nbytes;

/*****
/* When doing Non-Blocking OSS I/O to diskfiles, it is a good          */
/* idea to call sched_yield(), otherwise this thread won't give         */
/* up control until the file is completely written.  This is because    */
/* write() never returns EWOULDBLOCK for a diskfile.                    */
*****/

        sched_yield();
    }
} /* while */

close(fd);

```

```

    printf("==> OSS_read_thread() read %ld bytes from file %s\n", filesize,
           OSS_read_file);
    return(NULL);
}

```

```

/*
 *-----
 *
 * OSS_write_thread --
 *
 * Description:
 *
 * This thread writes an OSS file using OSS I/O.
 * Note that OSS I/O on regular files is serialized and only one OSS I/O
 * per open can occur at a given time. Even though we use the thread
 * aware library, the entire program blocks while the OSS write()
 * function performs the I/O.
 *
 * Results:
 * none.
 *-----
 */

```

```

void *OSS_write_thread(void *OSS_write_file)
{
    int fd;
    int nbytes;
    char *iobuf;
    int ret;
    long long currsz = 0;

    /*****
    /* prepare I/O buffer (page aligned, paged into physical memory) */
    *****/

    if ((iobuf = malloc(IOSIZE)) == NULL)
    {
        printf("*** ERROR in OSS_write_thread: unable to allocate
              I/O buffer ***\n");
        exit(0);
    }
    memset(iobuf, 0, IOSIZE);

```



```

/*****/
/* open the OSS file */
/*****/

if ((fd = open(OSS_write_file, O_WRONLY | O_NONBLOCK | O_CREAT |
O_TRUNC, S_IRWXU)) < 0)
{
    printf("*** ERROR in OSS_write_thread: open() of %s failed
        w/error %d ***\n", OSS_write_file, errno);
    exit(0);
}

/*****/
/* write the OSS file in a thread aware manner */
/*****/

while (currsz < WRITE_FILESIZE)
{
    nbytes = write(fd, iobuf, IOSIZE);
    if (nbytes < 0)
    {
        if ((errno == EWOULDBLOCK) || (errno == EAGAIN))
        {

/*****/
/* NOTE that this code does not execute for regular (disk) files, */
/* because write() on a regular file never returns EWOULDBLOCK or */
/* EAGAIN */
/*****/

            ret = spt_fd_write_ready(fd, NULL);
            if (ret != 0)
            {
                printf("*** ERROR in OSS_write_thread:
                    spt_fd_write_ready() failed w/error %d ***\n", ret);
                exit(0);
            }
        }
        else
        {
            printf("*** ERROR in OSS_write_thread:
                write() failed w/error %d ***\n", errno);
            exit(0);
        }
    }
    else /* write successful */
    {
        currsz += nbytes;

/*****/
/* When doing Non-Blocking OSS I/O to diskfiles, it is a good */

```

```

/* idea to call sched_yield(), otherwise this thread won't give */
/* up control until the file is completely written. This is */
/* because write() never returns EWOULDBLOCK for a diskfile. */
/*****/

    sched_yield();
}
} /* while */

close(fd);

printf("==> OSS_write_thread() wrote %d bytes to file %s\n",
        WRITE_FILESIZE, OSS_write_file);
return(NULL);
}

/* Grd_READ_thread --
*
* Description:
*
*   This thread reads an OSS file using Guardian Nowait I/O.
*
*   This function performs Guardian nowait I/O in a thread aware
*   manner, such that the entire program never blocks.
*   Note however that this thread blocks when it waits for I/O
*   completions.
*   Other threads are free to do work while this thread is blocked
*   waiting for an I/O completion.
*
*   Since Guardian nowait I/O is not serialized, this program can
*   have multiple outstanding Guardian I/O requests.
*
* Results:
*   none.
*
*-----
*/

void *Grd_READ_thread(void *Grd_READ_file)
{
    short fnum;
    long count_read;
    char *iobuf;
    short err;
    spt_error_t spte;
    long error;
    _cc_status cc;
    long tag;
    long long filesize = 0;

    if ((iobuf = malloc(IOSIZE)) == NULL)
    {
        printf("*** ERROR in Grd_READ_thread:  unable to allocate
                I/O buffer ***\n");
        exit(0);
    }
}

```

```

memset(iobuf, 0, IOSIZE);

/*****
/* open the OSS file using FILE_OPEN_() so we can do      */
/* Guardian nowait I/O                                   */
*****/

err = FILE_OPEN_(Grd_READ_file,
                 (short) sizeof(Grd_READ_file),
                 &fnum,
                 0 /* rd_wr */,
                 0 /* shared */,
                 1 /* nowait-depth */,
                 0 /* sync depth */,
                 0x0020 /* options - bit <10> set to allow open of an OSS pathname */);
if (err > 0)
{
    printf("*** ERROR in Grd_READ_thread: FILE_OPEN_() of %s failed\n", Grd_READ_file, (int) err);
    exit(0);
}

/*****
/* register the fnum */
*****/

spte = spt_regFile(fnum);
if (spte != SPT_SUCCESS)
{
    printf("*** ERROR in Grd_READ_thread: spt_regFile() failed w/error %d ***\n", (int) spte);
    exit(0);
}

/*****
/* Do Guardian nowait I/O in a thread aware manner */
*****/

while (1)
{
    tag = spt_generateTag();

    cc = READX(fnum, iobuf, (short) IOSIZE, /* count read */, tag);
    if (_status_ne(cc))
    {
        (void) FILE_GETINFO_(fnum, &err);
        printf("*** ERROR in Grd_READ_thread: READX() failed\n", (int) err);
        exit(0);
    }
    else /* CCE - read successfully initiated */
    {

```

```

    spte = spt_awaitio(fnum, tag, -1, &count_read, &error, NULL);
    if (spte != SPT_SUCCESS)
    {
        (void) CANCELREQ(fnum, tag);
        printf("*** ERROR in Grd_READ_thread: spt_awaitio() failed
w/error %d ***\n", (int) error);
        exit(0);
    }
    else if (error == 1) /* EOF */
        break;
    filesize += count_read;
}
} /* while */

spt_unregFile(fnum);
FILE_CLOSE_(fnum);

printf("==> Grd_READ_thread() read %ld bytes from file %s\n",
    filesize, Grd_READ_file);
return(NULL);
}

```

```

/*
*-----
*
* Grd_WRITE_thread --
*
* Description:
*
*   This thread writes an OSS file using Guardian Nowait I/O.
*
*   This function performs Guardian nowait I/O in a thread aware
*   manner, such that the entire program never blocks.
*   Note however that this thread blocks when it waits for
*   I/O completions.
*   Other threads are free to do work while this thread is
*   blocked waiting for an I/O completion.
*
*   Since Guardian nowait I/O is not serialized, this program can
*   have multiple outstanding Guardian I/O requests.
*
* Results:
*   none.
*-----
*/

```

```

void *Grd_WRITE_thread(void *Grd_WRITE_file)
{
    short fnum;
    long count_written;
    char *iobuf;
    short err;
    spt_error_t spte;
    long error;
    _cc_status cc;
    long tag;

```



```

off_t currsz = 0;
int fd;

if ((iobuf = malloc(IOSIZE)) == NULL)
{
    printf("*** ERROR in Grd_WRITE_thread:  unable to allocate
           I/O buffer ***\n");
    exit(0);
}
memset(iobuf, 0, IOSIZE);

/* create the OSS file before opening it with Guardian FILE_OPEN_() */
if ((fd = open(Grd_WRITE_file, O_WRONLY | O_NONBLOCK | O_CREAT |
O_TRUNC, S_IRWXU)) < 0)
{
    printf("*** ERROR in Grd_WRITE_thread:  open() of %s failed
           w/error %d ***\n", Grd_WRITE_file, errno);
    exit(0);
}
close(fd);

/*****
/* now open the OSS file using FILE_OPEN_() so we can do
/* Guardian nowait I/O
*****/

err = FILE_OPEN_(Grd_WRITE_file,
                 (short) sizeof(Grd_WRITE_file),
                 &fnum,
                 0 /* rd_wr */,
                 0 /* shared */,
                 1 /* nowait-depth */,
                 0 /* sync depth - NO CHECKPOINTING */,
                 0x0020 /* options - bit <10> set to open OSS pathname */);
if (err > 0)
{
    printf("*** ERROR in Grd_WRITE_thread:  FILE_OPEN_() of % failed
           w/error %d ***\n", Grd_WRITE_file, (int) err);
    exit(0);
}

/* register the fnum */
spte = spt_regFile(fnum);
if (spte != SPT_SUCCESS)
{
    printf("*** ERROR in Grd_WRITE_thread:  spt_regFile() failed
           w/error %d ***\n", (int) spte);
    exit(0);
}

/*****
/* Do Guardian nowait I/O in a thread aware manner */

```

```

/*****/

while (currsz < WRITE_FILESIZE)
{
    tag = spt_generateTag();

    cc = WRITEX(fnum, iobuf, (short) IOSIZE, /* count read */, tag);
    if (_status_ne(cc))
    {
        (void) FILE_GETINFO_(fnum, &err);
        printf("*** ERROR in Grd_WRITE_thread: WRITEX() failed
            w/error %d ***\n", (int) err);
        exit(0);
    }
    else /* CCE - write successfully initiated */
    {
        spte = spt_awaitio(fnum, tag, -1, &count_written, &error, NULL);
        if (spte != SPT_SUCCESS)
        {
            (void) CANCELREQ(fnum, tag);
            printf("*** ERROR in Grd_WRITE_thread: spt_awaitio() failed
                w/error %d ***\n", (int) error);
            exit(0);
        }
        currsz += count_written;
    }
} /* while */

spt_unregFile(fnum);
FILE_CLOSE_(fnum);

printf("==> Grd_WRITE_thread() wrote %d bytes to file %s\n",
    WRITE_FILESIZE, Grd_WRITE_file);

return(NULL);
}

int main(int argc, char *argv[])
{
    /* the following variables correspond to the OSS filenames specified */
    /* as command line parameters to the program */
    char OSS_read_filename[PATH_MAX];
    char OSS_write_filename[PATH_MAX];
    char Grd_READ_filename[PATH_MAX];
    char Grd_WRITE_filename[PATH_MAX];
    /* total number of threads that have been created */
    int nmbr_created_threads = 0;
    /* array of handles for each thread created */
    pthread_t thread_handle[MAX_THREADS];
    int i;

    /*****/
    /* Process command line arguments */
    /*****/

```



```

if (argc > 9)
{
    printf("Usage:  sptdio [-r <OSS-file>] [-w <OSS-file>] [-R <OSS-file>] [-W
        <OSS-file>]\n");
    exit(0);
}

/* verify that an even number of command line parameters was specified */
if ((argc - 1) % 2) == 1)
{
    printf("Usage:  sptdio [-r <OSS-file>] [-w <OSS-file>] [-R <OSS-file>] [-W
        <OSS-file>]\n");
    exit(0);
}

for (i = 1; i < argc; i = i + 2)
{
    if (strcmp(argv[i], "-r") == 0)
    {
        strcpy(OSS_read_filename, argv[i+1]);
    }
    else if (strcmp(argv[i], "-w") == 0)
    {
        strcpy(OSS_write_filename, argv[i+1]);
    }
    else if (strcmp(argv[i], "-R") == 0)
    {
        strcpy(Grd_READ_filename, argv[i+1]);
    }
    else if (strcmp(argv[i], "-W") == 0)
    {
        strcpy(Grd_WRITE_filename, argv[i+1]);
    }
    else
    {
        printf("Usage:  sptdio [-r <OSS-file>] [-w <OSS-file>] [-R <OSS-file>]
            [-W <OSS-file>]\n");
        exit(0);
    }
}

/*****
/* Set the concurrency level for the thread scheduler */
*****/

pthread_setconcurrency(CONCURRENCY_LEVEL);

/*****
/* Create all of the I/O threads */
*****/

for (i = 1; i < argc; i = i + 2)
{
    if (strcmp(argv[i], "-r") == 0)

```

```

    {
        pthread_create(&thread_handle[nmbr_created_threads++],
                      NULL, &OSS_read_thread, OSS_read_filename);
    }
    else if (strcmp(argv[i], "-w") == 0)
    {
        pthread_create(&thread_handle[nmbr_created_threads++],
                      NULL, &OSS_write_thread, OSS_write_filename);
    }
    else if (strcmp(argv[i], "-R") == 0)
    {
        pthread_create(&thread_handle[nmbr_created_threads++],
                      NULL, &Grd_READ_thread, Grd_READ_filename);
    }
    else if (strcmp(argv[i], "-W") == 0)
    {
        pthread_create(&thread_handle[nmbr_created_threads++],
                      NULL, &Grd_WRITE_thread, Grd_WRITE_filename);
    }
}

printf("\n==> All threads created...\n");

/*****
/* wait for the threads to complete - this is where control is passed */
/* to the threads */
*****/

for (i = 0; i < nmbr_created_threads; ++i)
{
    pthread_join(thread_handle[i], NULL);
}

} /* main() */

```

Pthreads Thread-Aware Disk File I/O Using SPT_THREAD_AWARE_XNONBLOCK is a multithreaded program similar to **Pthreads Nonblocking Disk File I/O (SPT_THREAD_AWARE_NONBLOCK version)** except that performs I/O on multiple OSS regular files using the thread-aware functions that are available when you use the define **SPT_THREAD_AWARE_XNONBLOCK** (available on systems running J06.04 and later J-series RVUs and H06.15 and later H-series RVUs). With these functions, you do not have to use Guardian functions to allow other threads to continue to work while another thread waits for I/O to complete on an open file.

Pthreads Thread-Aware Disk File I/O Using SPT_THREAD_AWARE_XNONBLOCK

```

/*
 * sptdio.c - SPThreads Diskfile IO Example Program
 *
 * This Pthreads program demonstrates a multithreaded program that
 * can perform I/O on multiple OSS regular files (diskfiles) using
 * thread-aware OSS I/O functions available on systems running
 * J06.04 and later J-series RVUs or H06.15 and later H-series RVUs.
 *
 * Note that OSS I/O on regular files is serialized.
 *
 * The following threads can be created by this program:
 *   - An OSS read thread which uses OSS I/O to completely

```

```

*      read a regular file.
*      - An OSS write thread which uses OSS I/O to write
*        1Mb of data to the regular file.
*
*  USAGE SYNTAX:
*
*  sptdio [-r <OSS-file>] [-w <OSS-file>]
*
*  FLAGS
*  -r <OSS-file>   Create a thread that uses OSS I/O to
*                   completely read the given OSS diskfile.
*  -w <OSS-file>   Create a thread that uses OSS I/O to write the
*                   given OSS diskfile.  1Mb of data is written to the file.
*
*  EXAMPLE USAGE:
*
*  /home/user: run sptdio -r /tmp/r1 -w /tmp/w1
*
*  ==> All threads created...
*  ==> OSS_write_thread() wrote 1048576 bytes to file /tmp/w1
*  ==> OSS_read_thread() read 1581888 bytes from file /tmp/r1
*
*  OSS BUILD INSTRUCTIONS:
*
*  c89 sptdio.c -o sptdio -Wextensions -Wsystype=oss -g -D_XOPEN_SOURCE=1 \
*  -lZSPTDLL
*/

#include <limits.h>
#include <stdlib.h>
/* Use the thread-aware library functions that are thread-aware
 * (thread-blocking instead of process-blocking) for regular files
 */
#define SPT_THREAD_AWARE_XNONBLOCK
#include <sptthread.h>

/*****/
/* LITERALS/DEFINES */
/*****/

/* the maximum number of I/O threads created by this program */
#define MAX_THREADS 4

/* concurrency level value passed to pthread_setconcurrency() */
/* the value of 100000 sets the minimum scheduled quantum to 0.000001 seconds. */
#define CONCURRENCY_LEVEL 100000

/* I/O size used by all threads */
#define IOSIZE 8192

/* size of files that are written - 1Mb */
#define WRITE_FILESIZE 1048576

/*

```

```

*-----
* OSS_read_thread --
*
* Description:
*
* This thread reads an OSS file using OSS I/O. Because this program
* uses the define SPT_THREAD_AWARE_XNONBLOCK, the read() function is
* mapped to the spt_readz() function, which is thread-aware and blocks
* the thread instead of the entire process. Other threads are free to
* do work while this thread is blocked waiting for an I/O completion.
*
*
* Results:
*   none.
*
*-----
*/

void *OSS_read_thread(void *OSS_read_file)
{
    int fd;
    int nbytes;
    char *iobuf;
    int ret;
    long long filesize = 0;

    /*****
    /* prepare I/O buffer (page aligned, paged into physical memory) */
    *****/

    if ((iobuf = malloc(IOSIZE)) == NULL)
    {
        printf("*** ERROR in OSS_read_thread:  unable to allocate
            I/O buffer ***\n");
        exit(0);
    }
    memset(iobuf, 0, IOSIZE);

    /*****
    /* open the OSS file */
    *****/

    if ((fd = open(OSS_read_file, O_RDONLY | O_NONBLOCK)) < 0)
    {
        printf("*** ERROR in OSS_read_thread:  open() of %s failed w/error %d
            ***\n", OSS_read_file, errno);

        exit(0);
    }

    /*****
    /* read the OSS file in a thread aware manner */
    *****/

```

```

while (1)
{
    nbytes = read(fd, iobuf, IOSIZE);

    if (nbytes < 0)
    {
        if ((errno == EWOULDBLOCK) || (errno == EAGAIN))
        {

/*****
/* NOTE that this code does not execute for regular (disk) files, */
/* because read() on a regular file never returns EWOULDBLOCK or */
/* EAGAIN */
*****/

            ret = spt_fd_read_ready(fd, NULL);
            if (ret != 0)
            {
                printf("*** ERROR in OSS_read_thread: spt_fd_read_ready()
                    failed w/err %d\n",ret);
                exit(0);
            }
        }
        else
        {
            printf("*** ERROR in OSS_read_thread: read() failed w/error
                %d ***\n",errno);
            exit(0);
        }
    }
    else if (nbytes == 0) /* EOF */
    {
        break;
    }
    else /* read successful */
    {
        filesize += nbytes;
    }
} /* while */

close(fd);

printf("==> OSS_read_thread() read %ld bytes from file %s\n", filesize,
    OSS_read_file);
return(NULL);
}

/*
*-----
*
* OSS_write_thread --
*
* Description:
*

```

```

* This thread writes an OSS file using OSS I/O. Because this program
* uses the define SPT_THREAD_AWARE_XNONBLOCK, the write() function is
* mapped to the spt_writez() function, which is thread-aware and blocks
* the thread instead of the entire process. Other threads are free to do
* work while this thread is blocked waiting for an I/O completion.
*
* Results:
*   none.
*
*-----
*/

void *OSS_write_thread(void *OSS_write_file)
{
    int fd;
    int nbytes;
    char *iobuf;
    int ret;
    long long currsz = 0;

    /*****
    /* prepare I/O buffer (page aligned, paged into physical memory) */
    *****/

    if ((iobuf = malloc(IOSIZE)) == NULL)
    {
        printf("*** ERROR in OSS_write_thread:  unable to allocate I/O buffer\n");
        exit(0);
    }
    memset(iobuf, 0, IOSIZE);

    /*****
    /* open the OSS file */
    *****/

    if ((fd = open(OSS_write_file, O_WRONLY | O_NONBLOCK | O_CREAT | O_TRUNC,
S_IRWXU)) < 0)
    {
        printf("*** ERROR in OSS_write_thread:  open() of %s failed w/error\n", OSS_write_file, errno);
        exit(0);
    }

    /*****
    /* write the OSS file in a thread-aware manner */
    *****/

    while (currsz < WRITE_FILESIZE)

```

```

{
    nbytes = write(fd, iobuf, IOSIZE);
    if (nbytes < 0)
    {
        if ((errno == EWOULDBLOCK) || (errno == EAGAIN))
        {

/*****
/* NOTE that this code does not execute for regular (disk) files, because */
/* write() on a regular file never returns EWOULDBLOCK or EAGAIN      */
*****/

            ret = spt_fd_write_ready(fd, NULL);
            if (ret != 0)
            {
                printf("*** ERROR in OSS_write_thread:  spt_fd_write_ready()
                        failed w/error %d ***\n", ret);
                exit(0);
            }
        }
        else
        {
            printf("*** ERROR in OSS_write_thread:  write() failed w/error %d
                    ***\n", errno);
            exit(0);
        }
    }
    else /* write successful */
    {
        currsz += nbytes;
    }
} /* while */

close(fd);

printf("==> OSS_write_thread() wrote %d bytes to file %s\n",
        WRITE_FILESIZE, OSS_write_file);
return(NULL);
}

int main(int argc, char *argv[])
{
    /* the following variables correspond to the OSS filenames specified */
    /* as command line parameters to the program                        */
    char  OSS_read_filename[PATH_MAX];
    char  OSS_write_filename[PATH_MAX];
    /* total number of threads that have been created */
    int nmbr_created_threads = 0;
    /* array of handles for each thread created */
    pthread_t thread_handle[MAX_THREADS];
    int i;

/*****
/* Process command line arguments */
*****/

```

```

/*****/

if (argc > 9)
{
    printf("Usage:  sptdio [-r <OSS-file>] [-w <OSS-file>]\n");
    exit(0);
}

/* verify that an even number of command line parameters was specified */
if ((argc - 1) % 2) == 1)
{
    printf("Usage:  sptdio [-r <OSS-file>] [-w <OSS-file>]\n");
    exit(0);
}

for (i = 1; i < argc; i = i + 2)
{
    if (strcmp(argv[i], "-r") == 0)
    {
        strcpy(OSS_read_filename, argv[i+1]);
    }
    else if (strcmp(argv[i], "-w") == 0)
    {
        strcpy(OSS_write_filename, argv[i+1]);
    }
    else {
        printf("Usage:  sptdio [-r <OSS-file>] [-w <OSS-file>]\n");
        exit(0);
    }
}

/*****/
/* Set the concurrency level for the thread scheduler */
/*****/

pthread_setconcurrency(CONCURRENCY_LEVEL);

/*****/
/* Create all of the I/O threads */
/*****/

for (i = 1; i < argc; i = i + 2)
{
    if (strcmp(argv[i], "-r") == 0)
    {
        pthread_create(&thread_handle[nmbr_created_threads++],
                      NULL, &OSS_read_thread, OSS_read_filename);
    }
    else if (strcmp(argv[i], "-w") == 0)
    {
        pthread_create(&thread_handle[nmbr_created_threads++],
                      NULL, &OSS_write_thread, OSS_write_filename);
    }
}

printf("\n==> All threads created...\n");

```




```

/*****
/* wait for the threads to complete - this is where control is
/* passes to the threads
*****/

for (i = 0; i < nmbr_created_threads; ++i)
{
    pthread_join(thread_handle[i], NULL);
}

} /* main() */

```

Reentrant OSS Functions

For systems running J-series RVUs, H06.05 and later H-series RVUs, and G06.28 and later G-series RVUs, several new reentrant functions are provided (**Thread-Aware Enscribe Functions**). Many existing functions are also reentrant. To determine if a function is reentrant, see the reference page for that function either online or in the *Open System Services System Calls Reference Manual*.

Many functions are unsafe in multithreaded applications, often because these functions return a pointer to a string or structure that is in static storage. A second call to the function overwrites the static storage (the results from the first call). Other issues for multithreaded applications include the use of global variables and maintaining context across calls.

In programs that use multiple threads, the same functions and the same resources might be accessed concurrently by several flows of control. To protect resource integrity, code written for multithreaded programs must be reentrant and thread safe.

A reentrant function does not hold static data over successive calls and does not return a pointer to static data. All data is provided by the caller of the function. Incoming signals are blocked until the reentrant function completes.

To determine which RVUs support an OSS function, see **Documented OSS Functions**.

NOTE: On systems running H06.21 or later H-series RVUs or J06.10 or later J-series RVUs, reentrant functions are also provided in the library that provides the base function. Therefore, to use the Standard POSIX Threads version of the reentrant function, when you link your application using `eld`, you must specify the Standard POSIX Threads library (ZSPTDLL) before you specify public libraries such as ZSECDLL. On L-series, when you link your application using `xld`, you must specify the Standard POSIX Threads library (XSPTDLL) before you specify public libraries such as XSECDLL.

Table 61: System and Equivalent Reentrant Functions

System Function	Reentrant Function
<code>asctime()</code>	<code>asctime_r()</code>
<code>ctermid()</code>	<code>ctermid_r()</code>
<code>ctime()</code>	<code>ctime_r()</code>
<code>gamma()</code>	<code>gamma_r()</code>
<code>getgrent()</code>	<code>getgrent_r()</code>

Table Continued



System Function	Reentrant Function
getgrgid()	getgrgid_r()
getgrnam()	getgrnam_r()
gethostbyaddr()	gethostbyaddr_r()
gethostbyname()	gethostbyname_r()
gethostent()	gethostent_r()
getlogin()	getlogin_r()
getnetbyaddr()	getnetbyaddr_r()
getnetbyname()	getnetbyname_r()
getnetent()	getnetent_r()
getprotobyname()	getprotobyname_r()
getprotobynumber()	getprotobynumber_r()
getprotoent()	getprotoent_r()
getpwent()	getpwent_r()
getpwnam()	getpwnam_r()
getpwuid()	getpwuid_r()
getservbyname()	getservbyname_r()
getservbyport()	getservbyport_r()
getservent()	getservent_r()
gmtime()	gmtime_r()
lgamma()	lgamma_r()
localtime()	localtime_r()
rand()	rand_r()
readdir()	readdir_r()
readdir_dt()	readdir_r_dt()
strtok()	strtok_r()
tmpnam()	tmpnam_r()
ttyname()	ttyname_r()

Example Program Using Reentrant Functions

Using the Reentrant Function `strtok_r` provides an example program that uses the `strtok_r()` reentrant function.

The program results in this output:

```
thread - 0 token - a
thread - 1 token - a
thread - 0 token - b
thread - 1 token - b
thread - 0 token - c
thread - 1 token - c
```



```

thread - 0 token - x
thread - 1 token - x
thread - 0 token - y
thread - 1 token - y
thread - 0 token - z
thread - 1 token - z

```

For comparison, the same program, using `strtok()`, instead of the reentrant version, `strtok_r()`, results in this output:

```

thread - 0 token - a
thread - 1 token - a
thread - 0 token - b
thread - 1 token - c
thread - 0 token - x
thread - 1 token - y
thread - 0 token - z
thread - 1 token - x
thread - 0 token - y
thread - 1 token - z

```

Using the Reentrant Function `strtok_r`

```

/* reent.c - Using the reentrant function strtok_r
 *
 * Compilation steps:
 *
 * /usr/bin/c89 -Wextensions -Wsystype=oss -c reent.c -o reent.o
 * /usr/bin/nld /usr/lib/crtmain.o reent.o ./libspt.a -obey \
 *             /usr/lib/libc.obey -lzcp1srl -o reent
 */

#include<pthread.h>
#include<stdio.h>
void *gettok(void *args)
{
    char str[25] = "a,b,c,";
    char str5[25] = "x,y,z,";
    char **lasts;
    char *str1 = NULL;
    char *ptr;
    lasts = &str1;

    ptr = strtok_r(str, ",", lasts);
    while(ptr != NULL)
    {
        printf("\n thread - %d token - %s", args, ptr);
        sleep(2);
        ptr = strtok_r(NULL, ",", lasts);
    }

    ptr = strtok_r(str5, ",", lasts);
    while(ptr != NULL)
    {
        printf("\n thread - %d token - %s", args, ptr);
        sleep(2);
        ptr = strtok_r(NULL, ",", lasts);
    }
}

```

```

    printf("\n");
}
void main()
{
    pthread_t T[2];
    int ret_value,t;
    for(t=0;t < 2;t++)
    {
        ret_value = pthread_create(&T[t],NULL,(void *)gettok,(void *)t);
        if (ret_value != 0)
        {
            printf("Error in creating the thread");
            exit(1);
        }
    }
    pthread_join(T[0],NULL);
    pthread_join(T[1],NULL);
    exit(0);
}

```

Thread-Aware Toolkit Functions and Types

Functions written using the thread-aware toolkit (in the following table) allow one or more threads to perform work on one or more files without any one thread unnecessarily blocking another thread. To enable these functions, you must include the `sptthread.h` header file. You can use these functions with either the `spt_*()` functions or the `spt_*x()` functions.

For details on each function, see the appropriate reference page in the *Open System Services System Calls Reference Manual*.

Table 62: Thread-Aware Toolkit Functions and Types

Function	Description
<code>spt_unregFile()</code>	Unregister the file number as one you will manage. Any threads waiting on file number I/O will awaken with <code>SPT_ERROR</code> and system error 16.
<code>spt_FileIOHandler_p()</code>	Callback type required by <code>spt_regFileIOHandler()</code> . A callback is executed in the context of the last running thread; that is, it executes on the stack of the last running thread.
<code>spt_regFileIOHandler()</code>	Register the file number as one you will manage through a user-supplied callback. Invoke this callback immediately after each I/O on <code>filenum</code> completes.
<code>spt_wakeup()</code>	Wake up a thread awaiting the tagged I/O on file number. The awakened thread returns from <code>spt_awaitio()</code> with <code>SPT_SUCCESS</code> .
<code>spt_defaultCallback()</code>	Default callback used by <code>spt_regFile()</code> . It calls <code>spt_wakeup()</code> and passes on the supplied parameters.

Table Continued



Function	Description
<code>spt_regFile()</code>	Register the file number as one you will manage through the default callback.
<code>spt_awaitio()</code>	Await a tagged IO on file number to complete, timeout, or be interrupted. I/O is never canceled by this function, and completes only if <code>SPT_SUCCESS</code> is returned. Multiple threads should not await the same tagged I/O on any given file number.
<code>spt_interrupt()</code>	Interrupt all threads awaiting I/O on file number. I/O is not cancelled by this function. Interrupted threads will return from <code>spt_awaitio()</code> with a return value of <code>SPT_ERROR</code> .
<code>spt_interruptTag()</code>	Interrupt the thread awaiting the tagged IO on file number. I/O is not cancelled by this function. Interrupted threads will return from <code>spt_awaitio()</code> with a return value of <code>SPT_ERROR</code> .
<code>spt_generateTag()</code>	Increment and return a static long appropriate for use as a tag. This long will eventually wrap and therefore return tags that may still be in use. For example, if a process calls <code>spt_generateTag()</code> 100 times every second, the wrap will occur on the 248th day.
<code>spt_fd_read_ready()</code>	Wait on a file descriptor to be read ready or have an exception pending.
<code>spt_fd_write_ready()</code>	Wait on a file descriptor to be write ready or have an exception pending.

Using Thread-Aware Toolkit Functions

The following examples show using thread-aware toolkit functions:

- Thread-aware toolkit program flow — [**Thread-Aware Toolkit Program Flow**](#).
- Simplified program flow for a thread-aware function — [**Simplified Program Flow For a Thread-Aware Function**](#).
- Thread-aware `read()` function — [**Thread-Aware read\(\) Function**](#)
- Nonblocking thread-aware `read()` function — [**Nonblocking Thread-Aware read\(\) Function**](#)
- Function using `spt_READX()` — [**Example READX\(\) Function Used In Thread-Aware Manner**](#)

Thread-Aware Toolkit Program Flow uses the default callback mechanism by registering the file number via `spt_regFile(2)`. In some cases, you may wish to provide your own callback using `spt_regFileIOHandler(2)`, which registers the file number as one you will manage through the callback you define.

Thread-Aware Toolkit Program Flow

```
...
FILE_OPEN(..., &filenum, ...);
spt_regFile(filenum);
...
// Do work on filenum.
...
```

```
spt_unregFile(filenum);
FILE_CLOSE(filenum);
...
```

Simplified Program Flow For a Thread-Aware Function

```
...
READX(filenum, ...);
spte = spt_awaitio(filenum, tag, -1, count_read, &error, NULL);
if (spte != SPT_SUCCESS) (void) CANCELREQ(filenum, tag);
return error;
...
```

Thread-Aware read() Function illustrates using a thread-aware `read()` function. This example assumes you are using the thread-aware library (`spt_*()` functions) instead of the nonblocking thread-aware library (`spt_*x()` functions) or the thread-aware library for regular files (`spt_*z()` functions). If you were to rewrite this example using the nonblocking thread-aware library, you could call `read()` and handle the `EWOULDBLOCK` error instead of checking the `O_NONBLOCK` flag of the file descriptor (see **Nonblocking Thread-Aware read() Function**).

Thread-Aware read() Function

```
/* This example assumes you're using the thread aware library. */
/* It checks to ensure that the O_NONBLOCK flag is set for the */
/* file descriptor. If O_NONBLOCK is set, it is safe to call spt_read() */
/* because spt_read() will return EWOULDBLOCK if the file descriptor is */
/* not ready. */

...
#define SPT_THREAD_AWARE /* maps read() to spt_read() */
#include <spthread.h>
...
ssize_t
my_read(int filedес, void *buffer, size_t nbytes)
{
    int before_flags;

    /* Get file descriptor's current flags: */
    if ((before_flags = fcntl(filedes, F_GETFL)) == -1) return -1;

    /* If file descriptor is non-blocking then just issue the call and
       return. */
    if (before_flags & O_NONBLOCK) return read(filedes, buffer, nbytes);

    /* Block this thread and wait until file descriptor is ready. */
    if ((errno = spt_fd_read_ready(filedes, NULL)) != 0) return -1;

    /* Issue the call and return. */
    return read(filedes, buffer, nbytes);
}
...
```

Nonblocking Thread-Aware read() Function illustrates using a nonblocking thread-aware `read()` function. This example assumes you are using the nonblocking thread-aware library (`spt_*x()` functions) instead of the nonblocking thread-aware library (`spt_*x()` functions) or the thread-aware library for regular files (`spt_*z()` functions). If you were to use this code with the thread-aware library instead of the nonblocking thread aware library, you risk blocking the whole process if the file descriptor does not have the `O_NONBLOCK` flag set.



Nonblocking Thread-Aware read() Function

```
/* This program fragment assumes you are using the nonblocking thread- */
/* aware library. If you used the thread-aware library */
/* (define SPT_THREAD_AWARE instead of define SPT_THREAD_AWARE_NONBLOCK), */
/* and the file descriptor does not have O_NONBLOCK set, */
/* the read() call could block the whole program. */

...
#define SPT_THREAD_AWARE_NONBLOCK /* maps read() to spt_readx() */
#include <sptthread.h>
...
{
    int rv;
    while (1)
    {
        rv = read(filedes, buffer, nbytes);

        /* If read() returned an error. */
        if (rv < 0)
        {
            /* If not read ready then wait (assumes EWOULDBLOCK would be sent.) */
            if ((errno == EWOULDBLOCK) || (errno == EAGAIN))
            {
                rv = spt_fd_read_ready(filedes, NULL);

                /* If spt_fd_read_ready() returned an error. */
                if (rv != 0)
                {
                    errno = rv;
                    return -1;
                }

                continue;
            }
        }

        break;
    }

    return rv;
}
```

Example READX() Function Used In Thread-Aware Manner shows a code fragment that uses the READX () function in a thread-aware manner, which is appropriate when you are not using the thread-aware library. An alternative to this approach is to use the SPT_READX () thread-aware function.

Example READX() Function Used In Thread-Aware Manner

```
{
    spt_error_t spte;
    long error;
    _cc_status cc;
    long tag;

    tag = spt_generateTag();
    cc = READX(filenum, buffer, read_count, , tag);
}
```

```

/* If READX() failed. */
if (_status_lt(cc))
{
    short short_error;

    (void) FILE_GETINFO_(filenum, &short_error);
    return short_error;
}

/* Loop forever. */
while (1)
{
    spte = spt_awaitio(filenum, tag, timelimit, count_read,
        &error, NULL);

    /* If spte is SPT_INTERRUPTED then try again. */
    if (spte != SPT_INTERRUPTED) break;
}

/* If spt_awaitio() was unsuccessful.*/
if (spte != SPT_SUCCESS) (void) CANCELREQ(filenum, tag);

return error;
}

```

Caveats to Using the Thread-Aware Toolkit

Proper operation of the thread-aware toolkit depends on the following conditions:

- No calls to `AWAITIO[X] O` with a file number of `-1` are made.
- No calls to `put_awaitio()` with a file number of `-1` are made.
- No calls to `AWAITIO[X] O` with a file number registered with the PUT library are made. Use `put_awaitio()` instead.
- No calls to `FILE_COMPLETE_SET_ O` are made.
- No calls to `FILE_COMPLETE_ O` are made using the permanent set.

Thread-Aware \$RECEIVE I/O Functions

\$RECEIVE thread-aware functions (**Table 63: Thread-Aware \$RECEIVE Functions and Types**) integrate the following basic \$RECEIVE I/O functions with the POSIX thread engine and support.

- Read \$RECEIVE (similar to `READUPDATEX`) so that it is thread-blocking, but not process-blocking.
- Read \$RECEIVE from one or multiple threads and returning all pertinent information (`FILE_GETRECEIVEINFO`) in a thread-safe form.
- Inherit transactions on \$RECEIVE and properly associating or disassociating the transaction with the calling thread.
- Block a reading thread if the number of messages read has exceeded receive depth.

For details on each function, see the appropriate reference page in the *Open System Services System Calls Reference Manual*.



The functions listed above provide only basic \$RECEIVE handling support, not higher-level capabilities associated with a \$RECEIVE package. You can build such higher-level functions using the primitives present in the thread-aware \$RECEIVE functions, but must tailor them for whatever domain-specific application they occur in. High-level capabilities **not supported** by the thread-aware \$RECEIVE functions include:

- Maintenance of an opener’s table
- Pathway server semantics (exit upon last close)
- Automatic resend of last replies for NonStop requesters
- Distribution of messages based on type or opener
- Any special handling of system messages (system messages are supported, but are treated as any other message)
- Encapsulation of the open or close of \$RECEIVE, or any SETMODE

Table 63: Thread-Aware \$RECEIVE Functions and Types

Function	Description
spt_INITRECEIVE ()	Register <i>filenum</i> being managed by the \$RECEIVE callback
spt_RECEIVEREAD ()	Thread-aware function specifically for reading \$RECEIVE
spt_REPLYX ()	Thread-aware <i>REPLYX ()</i>
spt_READX ()	Thread-aware <i>READX ()</i>

Using Thread-Aware \$RECEIVE Functions

Thread-Aware \$RECEIVE Program Flow through **\$RECEIVE Server** show using thread-aware \$RECEIVE functions.

Thread-Aware \$RECEIVE Program Flow shows the program flow for using a thread-aware \$RECEIVE function.

Thread-Aware \$RECEIVE Program Flow

```
...
FILE_OPEN(..., &filenum, ..., receive_depth, ...);
spt_INITRECEIVE(filenum, receive_depth);
...
// Do work on filenum.
...
spt_unregFile(filenum);
FILE_CLOSE(filenum);
...
```

Function with spt_RECEIVEREADQ and spt_REPLYXQ shows using the thread-aware functions *spt_RECEIVEREAD ()* and *spt_REPLYX ()* to perform the actual work on \$RECEIVE.

Function with spt_RECEIVEREADQ and spt_REPLYXQ

```
...
while (1)
{
    if (spt_RECEIVEREAD(...) != 0) break;
    ...
}
```



```

        // Do work.
        ...
        if (spt_REPLYX(...) != 0) break;
    }
    ...

```

\$RECEIVE Server shows a single thread calling the thread-aware `spt_RECEIVEREAD()` function. Using thread-aware \$RECEIVE functions allows one or more threads to perform work on \$RECEIVE, without a thread unnecessarily blocking another thread.

\$RECEIVE Server

```

#include <cextdecs.h(FILE_OPEN_)>
#include <sptthread.h>

short filenum;
void *
thread_start(void *parms)
{
    long bytesread, error;
    char buffer[2];
    short receive_info[17];
    while (1)
    {
        error = spt_RECEIVEREAD(filenum, buffer,
                                (short)sizeof(buffer), &bytesread, -1,
                                receive_info, NULL);
        if (error != 0) break;
        // Do real work here!
        error = spt_REPLYX("server's reply.", 15, NULL,
                           receive_info[2], 0);
        if (error != 0) break;
    }
    fprintf(stderr, "thread_start: error: %d\n", error);
    return NULL;
}

int
main()
{
    pthread_t thread1, thread2;
    char *filename = "$RECEIVE";

    if (FILE_OPEN_(filename, (short)strlen(filename), &filenum,
                  0, , 1, 2, 1) != 0)
    {
        fprintf(stderr, "main: FILE_OPEN_ failed.\n");
        exit(1);
    }

    if (spt_INITRECEIVE(filenum, 2) != 0)
    {
        fprintf(stderr, "main: spt_INITRECEIVE failed\n");
        exit(1);
    }

    if (pthread_create(&thread1, NULL, &thread_start, NULL) != 0)

```

```

        {
            perror("main: pthread_create");
            exit(1);
        }
    }
    exit(0)

    if (pthread_create(&thread2, NULL, &thread_start, NULL) != 0)
    {
        perror("main: pthread_create");
        exit(1);
    }

    if (pthread_join(thread1, NULL) != 0)
    {
        perror("main: pthread_join");
        exit(1);
    }

    if (pthread_join(thread2, NULL) != 0)
    {
        perror("main: pthread_join");
        exit(1);
    }
}

```

Caveats to Using Thread-Aware \$RECEIVE

Proper operation of thread-aware \$RECEIVE depends on the following conditions:

- No calls to AWAITIO[X] O with a file number of -1 are made.
- No calls to AWAITIO[X] O with a file number for \$RECEIVE are made.
- No calls to FILE_COMPLETE_SET_ O are made.
- No calls to FILE_COMPLETE_ O are made using the permanent set.

SQL/MX Considerations

Usage of SQL calls in an embedded SQL/MX application, linked to the PUT library for threaded support, will cause the thread executing the SQL calls to return control to the other threads only after completion of the SQL operations. This will result in all other threads in the process getting blocked. As a result, the embedded application will not be able to utilize multi-threading.

The usage of transactions in a multi-threaded application using embedded SQL/MX calls is not qualified and behavior is undefined.

The recommendation to fully avail the benefits of multi-threading with SQL/MX is to either use ODBC/MX for C/C++ applications or JDBC/MX T2/T4 for Java applications.

Thread-Aware Enscribe Functions

Thread-aware Enscribe functions are supported on L-series, J-series, H-series, and G-series RVUs. The following table provides thread aware wrappers for Enscribe procedure calls that enable an application to make use of Enscribe procedure calls without blocking the process.



Thread-aware Enscribe functions are thread-blocking, like functions in the thread-aware library, instead of process-blocking, like other Enscribe functions. In 64-bit multithreaded OSS applications, thread-aware Enscribe functions call the corresponding 64-bit Guardian API.

The parameters and the semantics for the new APIs are the same as those of Enscribe procedures, which are described in the *Guardian Procedure Calls Reference Manual*.

The thread-aware APIs for Enscribe procedure calls can:

- Open a file
- Read and write
- Cancel I/O operations issued on the file
- Lock and unlock the file or a record
- Execute device-dependent operations on an open file
- Set device-dependent functions in an open file

These functions must be explicitly called. There are no macros to map to these functions from the non-thread aware versions.

Table 64: Thread-Aware Enscribe Functions

Function	Description
SPT_CANCEL	Cancels the oldest incomplete operation on a file opened nowaited
SPT_CONTROL	Performs device-dependent I/O operations
SPT_FILE_CLOSE_	Closes an open file
SPT_FILE_OPEN_	Establishes a communication path between an application process and a file
SPT_READX	Returns data from an open file to the application process data area
SPT_READUPDATEX	Reads data from a disk or process file in anticipation of a subsequent write to the file
SPT_READLOCKX	Sequentially locks and reads records in a disk file
SPT_READUPDATELOCKX	Locks, then reads the record from the current position in the file in anticipation of a subsequent call to SPT_WRITEUPDATEX or SPT_WRITEUPDATEUNLOCKX
SPT_SETMODE	Sets device-dependent functions
SPT_WRITEREADX	Writes data to a file from an array in the application, then waits for data to be transferred back from the file
SPT_WRITEX	Writes data from an array in the application to an open file

Table Continued



Function	Description
SPT_WRITEUPDATEX	Transfers data from an array in the application to a file
SPT_WRITEUPDATEUNLOCKX	Performs random processing of records in a disk file
SPT_LOCKFILE	Excludes other users from accessing a file
SPT_UNLOCKFILE	Unlocks a disk file and any records in that file currently locked by the user
SPT_LOCKREC	Excludes other users from accessing a record at the current position
SPT_UNLOCKREC	Unlocks a record currently locked by the user

Using Thread-Aware Enscribe Functions

Using the Thread-Aware SPT_FILE_OPEN Function shows an example of using the thread-aware SPT_FILE_OPEN_. This function blocks the calling thread instead of the calling process.

Compilation:

```
/usr/bin/c89 -Wextensions -Wsystype=oss -c ens_thread.c -o ens_thread.o
```

```
nld /usr/lib/crtlmain.o ens_thread.o libspt.a -obey /usr/lib/libc.obey
-lzcplsr1 -o ens_thread
```

Output:

```
Hello, I am in Thread1
Calling SPT_FILE_OPEN_ - thread-blocking Enscribe call
Hello, I am Thread2
error returned in open : 0 filenum : 6
error in file close 0
```

In contrast, if you replace the calls to the thread-blocking SPT_FILE_OPEN_ function with calls to the process-blocking FILE_OPEN_ function, and change the printf() statement accordingly, the program results in the following output:

```
Hello, I am in Thread1
Calling FILE_OPEN_ - process-blocking Enscribe call
error returned in open : 0 filenum : 6
error in file close 0
Hello, I am Thread2
```

Using the Thread-Aware SPT_FILE_OPEN Function

```
/* Using the thread-aware SPT_FILE_OPEN function */
#include <cextdecs.h>
#include <sptthread.h>
#include <stdio.h>
#include <string.h>
#include <tal.h>

short filenum;

void *Thread2(void *args)
{
```



```

        printf("Hello, I am Thread2\n");
    }

void *Thread1(void *args)
{
    short error;
    char *name = "$SYSTEM.TEST.EX2";

    printf("\nHello, I am in Thread1\n");

    printf("Calling SPT_FILE_OPEN_ - thread-blocking Enscribe call \n");
    error = SPT_FILE_OPEN_(name, (short)strlen(name), &filenum);
    printf("error returned in open : %d filenum : %d\n", error, filenum);

    error = FILE_CLOSE_(filenum);
    printf("error in file close %d\n", error);
}

void main()
{
    pthread_t T[2];
    int ret_value;
    int t = 1;

    ret_value = pthread_create(&T[0], NULL, (void *)Thread1, (void *)t);
    if (ret_value != 0)
    {
        printf("Error in creating the thread");
        exit(1);
    }

    ret_value = pthread_create(&T[1], NULL, (void *)Thread2, (void *)t);
    if (ret_value != 0)
    {
        printf("Error in creating the thread");
        exit(1);
    }

    pthread_join(T[0], NULL);
    pthread_join(T[1], NULL);
    exit(0);
}

```

OSS File Descriptor and Timer Callbacks

Thread-aware functions listed in the following table allow registration and use of callbacks for OSS file descriptors and timer callbacks. For details on each function, see the appropriate reference page in the *Open System Services System Calls Reference Manual*.



Table 65: Thread-Aware Callback Functions and Types

Function	Description
<code>spt_OSSFileIOHandler_p</code>	Callback type required by <code>spt_regOSSFileIOHandler()</code> .
<code>spt_regOSSFileIOHandler</code>	Register the file descriptor as one that the user will manage through a user supplied callback.
<code>spt_unregOSSFileIOHandler</code>	Unregister file descriptor as one that the user will manage.
<code>spt_setOSSFileIOHandler</code>	Set interest in file descriptor.
<code>spt_regTimerHandler</code>	Register a user supplied timer callback.
<code>spt_TimerHandler_p</code>	Callback type required by <code>spt_regTimerHandler()</code> .

Using OSS and Timer Callbacks

OSS File Descriptor Callback shows an OSS file descriptor callback program flow. Typically, the callback would conditionally signal a thread that the file descriptor is read-ready.

OSS File Descriptor Callback

```
...
spt_regOSSFileIOHandler(fd, callback);
spt_setOSSFileIOHandler(fd, 1, 0, 0); // Only interested in read ready.
...
// Do work on file descriptor via user supplied callback.
...
spt_unregOSSFileIOHandler(fd);
...
```

Timer Callback Program Flow shows an OSS timer callback program flow. As soon as possible, the pthreads scheduler initially invokes the timer callback, with successive invocations of the callback driven by the callback's return value. For more information, see the `spt_TimerHandler_p(2)` reference page.

Timer Callback Program Flow

```
...
spt_regTimerHandler(callback);
...
```

Caveat to Using Callbacks

An OSS file descriptor callback takes precedent over all other operations that are waiting on the file descriptor. For example, if a thread is blocked on `spt_fd_read_ready()` and there is a callback registered for that particular file descriptor, the thread will never be awakened by the file descriptor being read-ready, even if the callback didn't set interest in read-ready.

Timer callbacks must not block and may be invoked sooner or later than requested.



Using the POSIX User Thread (PUT) Model Library

The POSIX User Model Thread library is a user-space implementation of IEEE Std 1003.1, 2004, POSIX System Application Program Interface for use by native C and C++ TNS/E-targeted and TNS/X-targeted applications in the OSS environment on NonStop servers. This library is supported on systems running H06.21 and later H-series RVUs, J06.10 and later J-series RVUs, and L-series RVUs.

Throughout the rest of this chapter:

- The POSIX User Model Thread library is referred to as the PUT Model library
- The Standard POSIX Threads library is referred to as the SPT library.

To use the PUT Model library, you should be familiar with programming in the OSS environment and with writing threaded applications. See **About This Document** for a list of books on programming with threads. See the *Open System Services Porting Guide* for information about porting existing threaded applications and for differences between the standard and the PUT Model library.

Threads are scheduled for execution by the PUT Model library, not by the NonStop operating system. All threads created within a process share the same process address space.

With the PUT Model library, one thread can never be preempted by another thread. You must ensure that threads relinquish control either by explicitly calling the `sched_yield()` function, by waiting on a mutex or condition variable, or by calling a thread-aware library function.

Compiling and Linking Considerations

Header Files

Include the following header file:

```
/usr/include/pthread.h
```

The `pthread.h` file contains prototypes of the PUT Model library interfaces.

Linking to the POSIX User Thread Model Library

Specify ZPUTDLL or XPUTDLL, the PUT Model library DLL, when linking with `eld` or `xld`. These libraries are located in the current `sysnn` subvolume.

You cannot use both the PUT Model library and the SPT library in the same application. If the Common Run-Time Environment (CRE) detects that a combination of incompatible public libraries is linked into an application, it displays a diagnostic message (run-time error 19) on the standard log file (`stderr`) and terminates process execution. For example, the message reporting that incompatible libraries ZSPTDLL and ZPUTDLL are both present is:

```
*** Run-time error 019 ***
Incompatible runtime libraries - ZSPTDLL vs. ZPUTDLL
```

The recovery action is to identify which library is the correct one and rebuild the application such that the conflicting one is excluded.

Defines

`_PUT_MODEL_`

(Required) You must use the `_PUT_MODEL_` define when building a threaded application based on the PUT Model library. Because thread-aware signal handling is always enabled in the PUT Model library, the `alarm()` and



`signal()` functions are mapped to the PUT Model library versions of these functions (similar to what the `SPT_THREAD_SIGNAL` and `SPT_THREAD_PRAGMA_SIGNAL` defines do for the SPT library). For more information about thread-aware signal handling, see [Thread-Aware Signal Handling](#).

`_PUT_SELECT_SINGLE`

This define maps `select()` to a variant of `select()` that checks the status of a single file descriptor using a faster algorithm. (This is equivalent to `SPT_SELECT_SINGLE` in the SPT library.)

Environment Variables

`PUT_THREAD_AWARE_REGULAR_IO_DISABLE`

Allows you to disable the thread aware regular file I/O behavior. For more information about thread-aware regular file I/O behavior, see [Thread-Aware and Nonblocking OSS Functions](#).

`PUT_PROTECTED_STACK_DISABLE`

Allows you to disable protected stack support in a 32-bit application. When you disable the protected stack support, the heap allocation method is used. This environment variable is not supported in 64-bit applications and results in undefined behavior if used. For more information about protected stacks, see [The Protected Thread Stack for Signals](#).

`PUT_THREAD_RSE_STACK_PERCENTAGE`

Allows you to set the register stack size percentage. If not set, the default value of 50 for a protected stack and 35 for a unprotected stack is taken. This variable is applicable for only TNS/E series of machines.

Standards Conformance

The POSIX threads standard is officially described in Institute of Electrical and Electronics Engineers, Inc., 2004 IEEE Std 1003.1-2001, IEEE Std 1003.1-2001/Cor 1-2002 and IEEE Std 1003.1-2001/Cor 2-2004. For simplicity, this manual refers to the “Standard document.” The Standard document is available for purchase online at <http://standards.ieee.org/>.

Portable pthreads functions provided by the PUT Model library (designated with either the `pthread_` or `PTHREAD_` prefix and without the `_np` or `_NP` suffix) comply with the IEEE Std 1003.1, 2004, POSIX System Application Program Interface.

In contrast to the SPT library, the PUT Model library does not provide these global variables because these global variables are not specified in the final standard:

```
pthread_attr_default
pthread_mutexattr_default
pthread_condattr_default
```

Error Reporting Changed From Draft 4 Standard

The PUT Model library functions return error indications as return values, instead of setting `errno`. The error values returned are in many cases the values that were stored in `errno` in Draft 4. The PUT Model library supports only the error values described in the Standard document.

If an error condition occurs, a pthread routine returns an integer value indicating the type of error. For example, a call to the Draft 4 implementation of `pthread_cond_destroy()` that returned a `-1` and set `errno` to `EBUSY` now returns `EBUSY` as the routine return value.

Reporting errors by returning error values is a global change from the previous standard (Draft 4), in which errors are reported in `errno`. If you are porting a threaded application that adheres to Draft 4, you must rewrite the error handling routines. The [Threaded Application Examples](#) demonstrates error handling with the PUT Model library. See the *Open System Services Porting Guide* for a list of the supported return values as well as other porting issues.



Exceptions and Extensions to the Standard Document

Although the PUT Model library version of `sigaction()` accepts the `SA_ONSTACK` flag, the PUT Model library determines which stack is used to run the thread signal handler. The PUT Model library does not allow you to establish a signal stack using the `sigaltstack()` function. For more information about this behavior, see **The Protected Thread Stack for Signals**.

These functions are not async-signal safe, so they should not be used in signal handlers:

```
alarm()
fork()
pause()
sigaction()
sigsuspend()
sleep()
wait()
waitpid()
```

Attributes Defined by Threads Macros

The attributes of the PUT Model library are determined by macros in `unistd.h`. The following table lists the principal macros that influence the Hewlett Packard Enterprise implementation for NonStop servers.

Table 66: Attributes of the POSIX User Thread Model Library Defined by Macros

Macro or Attribute	Description in the POSIX User Thread Model Library
<code>_POSIX_THREADS</code>	Implementation supports threads options. Defined as 0 (zero)
<code>_POSIX_THREAD_SAFE_FUNCTIONS</code>	Implementation supports thread-safe functions. Defined as 0 (zero).
<code>_POSIX_THREAD_PROCESS_SHARED</code>	A mutex or condition variable can be operated on only by threads in the same process that initialized the mutex or condition variable. Defined as -1.
<code>_POSIX_THREAD_PRIORITY_SCHEDULING</code>	Defined as 0 (zero) and only supports threads created with the <code>PTHREAD_SCOPE_PROCESS</code> attribute. Threads contend directly with other threads within their process. This attribute cannot be altered. The scheduling allocation domain size is 1. Threads are always bound to the same processor as the parent thread.

Table Continued



Macro or Attribute	Description in the POSIX User Thread Model Library
Thread scheduling attribute: <code>inheritsched</code>	Has the value <code>PTHREAD_EXPLICIT_SCHED</code> . The scheduling policy and associated attributes, such as priority, are set to the corresponding values from the attributes object during thread creation. This attribute can be altered to <code>PTHREAD_INHERIT_SCHED</code> (so that the scheduling policy and attributes are inherited from the creating thread) by using <code>pthread_attr_setinheritsched()</code> .
Thread scheduling attribute: <code>schedpolicy</code>	The default value is <code>SCHED_FIFO</code> . This attribute cannot be altered.
Mutex initialization scheduling attributes: <code>_POSIX_THREAD_PRIO_INHERIT</code> <code>_POSIX_THREAD_PRIO_PROTECT</code>	Defined as <code>-1</code> . The <code>prioceiling</code> and <code>protocol</code> attributes are not defined.
Thread creation attribute: <code>_POSIX_THREAD_ATTR_STACKSIZE</code>	Defined as <code>0</code> (zero). The thread creation stack size attribute can be set in the attribute object.
Attribute: <code>_POSIX_READER_WRITER_LOCKS</code>	Supports read and write locks. Defined as <code>-1</code> .
Thread creation attributes: <code>_POSIX_THREAD_CPUTIME</code> <code>_POSIX_THREAD_SPORADIC_SERVER</code>	Defined as <code>-1</code> . These attributes cannot be altered.
Thread creation attribute: <code>_POSIX_THREAD_ATTR_STACKADDR</code>	The location of the storage to be used for the stack of the created thread cannot be altered. Defined as <code>-1</code> .
Thread creation attribute: <code>detachstate</code>	The default value is <code>PTHREAD_CREATE_JOINABLE</code> (that is, storage associated with the thread is reclaimed when the thread that created it calls a <code>join()</code>). This attribute can be altered using <code>pthread_attr_setdetachstate()</code> .

Process Resources

All resources of a process—for example, open files or memory—are either global or private, as shown in the following table. Private resources can be accessed only by a specific thread. Global resources can be accessed by all threads. Access to global resources by threads should be synchronized using mutex or condition variables.



Table 67: Process Resources

Resource	Type	Description
Thread stack	Private	The memory where all thread-local variables are stored
Thread program counter	Private	The address of the instruction that the thread is about to execute
Thread context	Private	The data maintained by the threads library on behalf of a thread; contains the per-thread signal mask, which is used by the <code>pthread_kill()</code> function
All external data	Global	Includes process and module global resources
All static internal variables	Global	
Heap	Global	
File descriptors and file state	Global	Includes file and record locks
SQL cursors	Global	
Environment variables	Global	
Signal mask	Private	The per-thread signal mask, used by the <code>pthread_kill()</code> , <code>sigaction()</code> , <code>sigwait()</code> , <code>sigmask()</code> , and <code>sigsuspend()</code> functions
Signal stack	Global	The stack that is used to run the signal handler

Numerical Limits in the POSIX User Thread Model Library

Protected Stack Limits

The PUT Model library uses a protected stack for signals (see [The Protected Thread Stack for Signals](#)) and defines these symbolic constants related to the protected and unprotected stacks:

Symbolic Constant	Description	Value
PTHREAD_STACK_MAX_NP	Maximum number of bytes available for the (unprotected) thread stack.	33554432
PTHREAD_STACK_MAX_PROTECTED_STACK	Maximum number of bytes available for the protected stack.	16777216
PTHREAD_STACK_MIN_PROTECTED_STACK	Minimum number of bytes available for the protected stack.	49152

Limits Imposed by the Standard

The following table lists numerical limits imposed by the standard implementation. The symbolic constants listed in the table are defined in `limits.h` with the values shown in the table. These symbolic constants refer to the most restrictive value for each feature.

Table 68: Numerical Limits in the POSIX User Thread Model Library

Symbolic Constant	Description	Value
_POSIX_ARG_MAX	Maximum length of the arguments for one of the <code>exec</code> functions, in bytes, including environment data	4096
_POSIX_CHILD_MAX	Maximum number of simultaneous processes for one real user ID	6
_POSIX_DESTRUCTOR_ITERATIONS	Maximum number of attempts to destroy the thread-specific data of a thread on termination	4
_POSIX_FD_SETSIZE	Maximum number of descriptors that a process can examine	_POSIX_OPEN_MAX
_POSIX_HIWAT	Maximum number of bytes that can be buffered on a socket for sending or receiving	_POSIX_PIPE_BUF
_POSIX_KEYS_MAX	Maximum number of keys a process can use	1024
_POSIX_LINK_MAX	Maximum value of a file's link count	8
_POSIX_MAX_CANON	Maximum number of bytes in a terminal canonical input queue	255
_POSIX_MAX_INPUT	Maximum number of bytes for which space will be available in a terminal input queue	255
_POSIX_NAME_MAX	Maximum number of bytes in a filename	14

Table Continued



Symbolic Constant	Description	Value
_POSIX_NGROUPS_MAX	Maximum number of simultaneous supplementary group IDs for one process	0
_POSIX_OPEN_MAX	Maximum number of files that one process can have open at one time	16
_POSIX_PATH_MAX	Maximum number of bytes in a pathname	255
_POSIX_PIPE_BUF	Maximum number of bytes that can be written atomically when writing to a pipe	512
_POSIX_QLIMIT_	Maximum number of connections that can be queued on a socket	1
_POSIX_SSIZE_MAX	Maximum value that can be stored in an object of type <code>ssize_t</code>	32,767
_POSIX_STACK_MIN	Minimum number of bytes available for the thread stack.	4096
_POSIX_STREAM_MAX	Maximum number of streams that one process can have open at one time	8
_POSIX_THREAD_DESTRUCTOR_ITERATIONS	Maximum number of attempts to destroy the thread-specific data of a thread on termination	4
_POSIX_THREAD_KEYS_MAX	Maximum number of thread-specific data keys available per process	128
_POSIX_THREAD_THREADS_MAX	Maximum number of threads supported per process	64
_POSIX_TZNAME_MAX	Maximum number of bytes supported for time zone names	3
PTHREAD_DESTRUCTOR_ITERATIONS	Maximum number of attempts to destroy thread's thread-specific data on termination)	4
PTHREAD_KEYS_MAX	Maximum number of thread-specific data keys available per process	1024
PTHREAD_STACK_MIN	Minimum number of bytes available for the thread stack.	4096
PTHREAD_THREADS_MAX	Maximum number of threads supported per process	2048



Signals and Signal Handling

Types of Signals

There are two basic types of signals:

- Synchronous signals represent events that occur inside a process.
Synchronous signals occur immediately after the instruction that caused the event—for example, attempting to divide by zero. A synchronous signal is associated with the thread that is executing when the event occurs. A thread can install signal handlers (using the `sigaction()` function) for the synchronous signals that occur during its execution. For terminating signals, the default behavior is to terminate the process.
- Asynchronous signals represent events external to a process.
A thread can wait for a set of asynchronous signals using the PUT Model library version of the `sigwait()` function. If a thread is waiting for a type of signal that is received, the thread is awakened and if a signal handler has been installed, it is invoked, and the signal is considered handled; otherwise, the default signal action is taken. For a terminating signal, the default behavior is to terminate the process. You can define signal-catching functions for asynchronous signals. See **Thread-Aware Signal Handling**.

The supported signal numbers are listed in the following table. For more information about the PUT Model library implementation of signals, see **Using Signals**.

Table 69: Signals Processed by Threads

Name	Number	Type	Default Action	Description
SIGABEND	31 ¹ If the default action for a signal is to terminate with <code>saveabend</code> , then the process will terminate with SIGABEND signal and the actual signal can be obtained from the <code>saveabend</code> file.	Process-level	Terminate with <code>saveabend</code>	Abnormal termination
SIGABRT	6	Synchronous	Terminate with <code>saveabend</code>	Abort process
SIGALRM	14	Alarm signal	Terminate process	Alarm clock
SIGCHLD	18	Process-level	Discard signal	Child has stopped or terminated
SIGCONT	28	Process-level	Continue execution	Continue execution
SIGFPE	8	Synchronous	Terminate with <code>saveabend</code>	Arithmetic overflow

Table Continued



Name	Number	Type	Default Action	Description
SIGHUP	1	Process-level	Terminate process	Hangup
SIGILL	4	Synchronous	Terminate with saveabend	Invalid instruction
SIGINT	2	Asynchronous	Terminate process	Interrupt
SIGIO	7	Synchronous	Discard signal	IO possible or completed
SIGKILL	9 ¹	Process-level	Terminate process	Kill
SIGLIMIT	27	Synchronous	Terminate with saveabend	NonStop operating system limits trap
SIGMEMERR	22	Synchronous	Terminate with saveabend	Uncorrectable memory error
SIGMEMMGR	24	Synchronous	Terminate with saveabend	Memory manager read error
SIGNOMEM	23	Synchronous	Terminate with saveabend	No memory available
SIGPIPE	13	Asynchronous	Terminate process	Write on a pipe, no one to read it
SIGQUIT	3	Asynchronous	Terminate with saveabend	Quit
SIGRECV	19	Synchronous	Discard signal	Message queued on \$RECEIVE (currently not used)
SIGSEGV	11	Synchronous	Terminate with saveabend	Invalid address reference
SIGSTK	25	Synchronous	Terminate with saveabend	Stack overflow
SIGSTOP	20 ¹	Process-level	Terminate process	Stop
SIGTERM	15	Asynchronous	Terminate process	Software termination signal

Table Continued



Name	Number	Type	Default Action	Description
SIGTIMEOUT	26	Process-level	Terminate process	Process loop timer time-out
SIGTSTP	21	Process-level	Terminate process	Interactive stop
SIGTTIN	29	Process-level	Stop process	Background read attempted from controlling terminal
SIGTTOU	30	Process-level	Stop process	Background write attempted to controlling terminal
SIGUNCP	10	Process-level	Discard signal	Uncooperative process. Supported for servers running H-series RVUs only.
SIGURG	5	Asynchronous	Discard signal	Urgent condition on I/O channel
SIGUSR1	16	Asynchronous	Terminate process	User-defined signal 1
SIGUSR2	17	Asynchronous	Terminate process	User-defined signal 2
SIGWINCH	12	Synchronous	Discard signal	Terminal device window size changed

¹Signal cannot be caught or ignored.

Using Signals

- The following signal features are not supported:
 - Job Control Signals. The stop a process or continue a process actions implied by these signals are also not supported.
 - The Real-time Signals Extension option.
- Use the `sigaction()` function, not the `signal()` function, to specify the action to be taken when a given signal is received.
- These OSS system and library functions support thread-aware signal handling:

```
alarm()
sigaction()
signal
sigpending()
sigsuspend()
sigwait()
```

```

pause()
sleep()
usleep()
wait()
waitpid()

```

- The `pthread_kill()` function provides a mechanism for synchronously directing a signal to a thread in the calling process.

Signals sent with the `pthread_kill()` function are queued in FIFO order for the target thread. More than one instance of the same signal might be pending for a thread. However, applications should not rely on this ordering.

- A signal is handled in the context of the given thread. However, the signal action (terminating or stopping) might affect the process as a whole. See **Thread-Aware Signal Handling**.
- You must specify whether a signal listed in **Table 69: Signals Processed by Threads** as generating a saveabend actually generates a saveabend by specifying the `-Wsaveabend` flag in `c89` or `c99`, or `saveabend` in `nld`, `ld`, `eld`, or `xld`.
- If a signal is delivered to a thread waiting on a condition variable, then upon return from the signal handler, the thread will resume waiting for the condition variable as if it were not interrupted.
- Non thread-aware libraries linked to the application should not change the signal handler installed by the POSIX User Thread Model Library as it might result in unexpected application behavior.

The Protected Thread Stack for Signals

The PUT Model library does not allow you to establish a signal stack using the `sigaltstack()` function. Instead, the library allocates a protected signal stack for use by signal handlers and applications that use the PUT Model library can use the `sigaltstack()` function to get the status of the signal stack.

Thread-aware signal handling is enabled by default in the PUT Model library. The library allocates and installs a protected stack to allow external signals and synchronous signals to be delivered to a signal stack that is different from the thread stack.

- When an external asynchronous signal is delivered to the PUT library signal handler on the signal stack, the thread library signal handlers check to see if the current thread can handle the signal. If it can, then the thread library signal handler invokes the thread signal handler immediately on the same signal stack. If the current thread cannot handle the signal, the thread library signal handler finds the thread that can and queues the signal for that thread and return. The thread signal handlers for these queued signals are run either at the thread dispatch time or at the cancellation point on the main stack for the thread.
- When a synchronous signal is delivered to the thread library signal handler on the signal stack, the signal gets processed immediately. If the current thread can handle the signal, the thread signal handler gets invoked immediately on the same signal stack; otherwise, the thread library signal handler performs the default action associated with the signal and terminates the process.
- If nested signals are delivered to the thread library signal handler, they are delivered on the same signal stack as the signal handler being interrupted. If a nested thread signal handler is run by the thread library signal handler, it is run on the same signal stack as the thread signal handler that is being interrupted.

Using Signals With the `setjmp()` and `longjmp()` Functions

Using the `setjmp()` and `longjmp()` functions can interfere with signal handling. The `longjmp()` function causes a thread to jump to the execution context saved by a previous call to the `setjmp()` function, but it does not restore the signal mask of the process. If the `longjmp()` function is called within a signal handler, the signal mask used in the signal handler is retained when execution resumes at the `setjmp()` location. The exception mechanism uses the `setjmp()` and `longjmp()` functions and behaves similarly.



The `sigsetjmp()` and `siglongjmp()` functions have an option to save and restore the signal mask. This option can cause signals that were enabled by another thread to be inadvertently masked.

When handling signals:

- Do not use the `raise()` function for an exception, and do not use the `longjmp()` function to exit a synchronous signal handler. Simply returning from a signal handler is safe and is the recommended practice.
- Use exception handlers instead of the `setjmp()` and `longjmp()` functions for threaded OSS applications.
- Do not use the option of the `sigsetjmp()` and `siglongjmp()` functions that saves and restores the signal mask. Doing so can cause signals enabled by another thread to be masked.

Thread-Aware Signal Handling

The PUT Model library catches both externally generated OSS signals and synchronous signals, which are result of an event that occurs inside a process, and deliver them to the correct thread.

By default, thread-aware signal behavior is enabled.

Types of Signals

Signals are categorized as synchronous, asynchronous, or process level:

Synchronous signals result from an event that occurs inside a process and are delivered synchronously with that event. These signals are delivered to current active thread. If current active thread has not installed the signal handler, then default actions are performed. Signals in this category are:

SIGABRT
SIGFPE
SIGILL
SIGIO
SIGLIMIT
SIGMEMERR
SIGMEMMGR
SIGNOMEM
SIGSEGV
SIGSTK
SIGWINCH

Asynchronous signals result from an event that is external to the process and are delivered at any point in the execution of a thread. These signals are delivered to the active threads in FIFO order. If no active threads have installed a signal handler for the signal, the process-level actions described in the table **Table 69: Signals Processed by Threads** are performed. Signals in this category are:

SIGINT
SIGPIPE
SIGQUIT
SIGTERM
SIGURG



SIGUSR1
SIGUSR2

For process level signals only process level actions can be performed. Signal handlers for these signals are registered directly to the operating system, so an existing handler for a given signal is overwritten when a new signal handler is defined for that signal. Signals in this category are:

SIGABEND (cannot be handled by a user-defined signal handler)
SIGCONT
SIGKILL (cannot be handled by a user-defined signal handler)
SIGSTOP (cannot be handled by a user-defined signal handler)
SIGTSTP (process level action is performed)
SIGTIMEOUT
SIGTTIN
SIGTTOU
SIGUNCP

Limitations

Do not deliver signals like SIGALRM and SIGCHLD using the `kill` command or the `raise()` function because these functions do not provide enough information about the process IDs for the parent and child processes. Instead, use the PUT Model library version of the `alarm()` function to raise alarms.

Examples

Handling the Synchronous Signal SIGFPE provides an example of handling the synchronous signal SIGFPE. Because thread-aware signal handling is always enabled, the signal handler defined in the example is executed, and signal 8 (SIGFPE) is received by the thread. On successful return from a signal-catching function for a SIGFPE, SIGILL, SIGLIMIT, SIGMEMERR, SIGMEMMGR, SIGNOMEM, SIGSEGV, SIGABRT, or SIGSTK signal that was not generated by a `kill()` or `raise()` function call, a process receives a SIGABEND signal and terminates.

Handling the Synchronous Signal SIGFPE

```
/* This test program demonstrates handling the */
/* SIGFPE synchronous signal.                  */

#define _PUT_MODEL_ /** enables thread aware behavior **/

#include <pthread.h>
#include <signal.h>
#include <unistd.h>
#include "myio.h" /* contains my_printf which is process blocking */

/*Define a signal handler for this thread */
void sighand(int signo);
int x=10,y=0,z;

void *threadfunc(void *parm)

/* Perform the illegal operation of dividing by zero and */
/* raise the SIGFPE signal. The SIGFPE signal is sent to */
/* the generic handler. */
{
    signal(SIGFPE,sighand);
    z = x/y;
```

```

    my_printf("\nI am in thread1\n");
    return NULL;
}

int main(int argc, char **argv)
{
    pthread_t    thread1;
    my_printf("Enter Testcase - %s\n", argv[0]);

    /* create the thread */
    pthread_create(&thread1, NULL, threadfunc, NULL);
    sleep(1);
    my_printf("Wait for threads to complete\n");
    pthread_join(thread1, NULL);
    my_printf("Main completed\n");
    return 0;
}
void sighand(int signo)
{
    my_printf("Signal received is %d\n", signo);
}

```

Handling the Synchronous Signal SIGINT creates a thread that waits for asynchronous signal SIGINT.

Sample input:

```
> kill -2 process-name
```

or

```
<CTRL-C>
```

Both of these inputs generate a SIGINT signal.

In **Handling the Synchronous Signal SIGINT**, the signal handler of thread1 is executed and thread1 no longer waits for a signal. Sample output:

```

Enter Testcase - ./Test2
sigaction returned 0
Wait for threads to complete
Signal received is 2
sigsuspend returned -1
Main completed.

```

Handling the Synchronous Signal SIGINT

```

/* Program that demonstrates behavior of Asynchronous */
/*  signal SIGINT          */

#define _PUT_MODEL_ /** enables thread aware behavior */
#include <pthread.h>
#include <signal.h>
#include <unistd.h>
#include "myio.h" /** contains my_printf which is process-blocking*/

void sighand(int signo);

```

```

struct sigaction      sigact_t;

sigset_t waitset;

void *threadfunc(void *parm)

{

    int    rc1,rc2;
    sigfillset( &waitset );
    sigdelset( &waitset,SIGINT);
    sigact_t.sa_handler = sighand;
    rc1 = sigaction(SIGINT,&sigact_t,NULL);
    my_printf("\nsigaction returned %d\n",rc1);
    rc2=sigsuspend(&waitset);
    my_printf("\nsigsuspend returned %d\n",rc2);
    return NULL;

}

int main(int argc, char **argv)

{

    pthread_t      thread1;
    my_printf("Enter Testcase - %s\n", argv[0]);
    pthread_create(&thread1, NULL, threadfunc, NULL);
    sleep(1);
    my_printf("Wait for  threads to complete\n");
    pthread_join(thread1, NULL);
    my_printf("Main completed\n");
    return 0;

}

void sighand(int signo)
{
    my_printf("Signal received is %d\n",signo);
}

```

Handling the SIGCHLD Signal demonstrates handling the SIGCHLD signal. This example creates two threads, each of which call `fork()`. When the child process executes, the process raises the SIGCHLD signal. The SIGCHLD signal is delivered to the correct thread, and `sigaction()` function in the example defines the action to be taken. Sample output:

```

sigaction for thread1 returned 0
sigaction for thread2 returned 0
Inside Child of Thread2
Signal received for Thread2 is 18
Inside Child of Thread1
Signal received for Thread1 is 18
Main Completed

```

Handling the SIGCHLD Signal

```

/* This program demonstrates the handling of the */
/* SIGCHLD signal */

#define _PUT_MODEL_ /** enables thread aware behavior **/

```

```

#include <pthread.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include "myio.h" /** contains my_printf which is process-blocking**/

struct sigaction      sigact_t,osigact_t;
sigset_t waitset;
int rc;

void sig_cld1(int);
void sig_cld2(int);
void *do_work1(void *arg)
{
    pid_t pid;
    sigact_t.sa_handler =sig_cld1;
    sigact_t.sa_flags = 0;
    rc =sigaction(SIGCHLD,&sigact_t,NULL);
    my_printf("\nsigaction for thread1 returned %d\n",rc);
    if ((pid = fork()) < 0)
        my_printf("ERROR IN FORK\n");
    else if (pid == 0)
    {
        sleep(2);
        my_printf("\nInside Child of Thread1\n");
        exit(0);
    }
    waitpid(pid,NULL,NULL);
    return NULL;
}

void *do_work2(void *arg)
{
    pid_t pid;
    osigact_t.sa_handler =sig_cld2;
    osigact_t.sa_flags = 0;
    rc = sigaction(SIGCHLD,&osigact_t,NULL);
    my_printf("\nsigaction for thread2 returned %d\n",rc);
    if ((pid = fork()) < 0)
        my_printf("ERROR IN FORK\n");
    else if (pid == 0)
    {
        sleep(1);
        my_printf("\nInside Child of Thread2\n");
        exit(0);
    }
    waitpid(pid,NULL,NULL);
    return NULL;
}

int main()
{
    pthread_t tid1,tid2;
    pthread_create(&tid1,NULL,do_work1,NULL);

```

```

    pthread_create(&tid2, NULL, do_work2, NULL);
    sched_yield ();
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    my_printf ("\nMain Completed\n");
}

void sig_cld1(int signo)
{
    my_printf("\nSignal received for Thread1 is %d\n", signo);
}

void sig_cld2(int signo)
{
    my_printf("\nSignal received for Thread2 is %d\n", signo);
}

```

Handling SIGALRM Signals provides an example of SIGALRM handling. It creates two threads that wait for a SIGALRM signal. The SIGALRM signal is delivered to the correct thread, and the `sigand1()` and `sigand2()` functions define the actions to be taken. Sample output:

```

sigaction for thread1 returned 0
sigaction for thread2 returned 0
Signal received for thread1 is 14
sigsuspend for thread1 returned -1
sigwait returned 0
sigwait for thread2 returned for signal 14
Main Completed

```

Handling SIGALRM Signals

```

/*Program demonstrating handling the SIGALRM signal */

#define _PUT_MODEL_ /** enables thread aware behavior **/
#include <pthread.h>
#include <signal.h>
#include <unistd.h>
#include "myio.h" /** contains my_printf which is process-blocking**/

void sigand1(int);
void sigand2(int);
struct sigaction sigact_t;
int rc;
void *do_work(void *arg)
{
    sigset_t set;
    sigfillset(&set);
    sigdelset(&set, SIGALRM);
    sigact_t.sa_handler = sigand1;
    rc = sigaction(SIGALRM, &sigact_t, NULL);
    my_printf("\n    sigaction for thread1 returned %d\n", rc);
    alarm(5);
    rc = sigsuspend(&set);
    my_printf("\nsigsuspend for thread1 returned %d\n", rc);
    return NULL;
}

void *do_work1(void *arg)

```



```

{
    sigset_t set;
    int sig;
    sigemptyset(&set);
    sigaddset(&set, SIGALRM);
    sigact_t.sa_handler = sighand2;
    rc=sigaction(SIGALRM, &sigact_t, NULL);
    my_printf("\nsigaction for thread2 returned %d\n", rc);
    alarm(6);
    rc=sigwait(&set, &sig);
    my_printf("\nsigwait returned %d\n", rc);
    my_printf("\nsigwait for thread2 returned for signal %d\n", sig);
    return NULL;
}

int main()
{
    pthread_t tid1, tid2;
    pthread_create(&tid1, NULL, do_work, NULL);
    pthread_create(&tid2, NULL, do_work1, NULL);
    sched_yield();
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    my_printf("\nMain Completed\n");
}

void sighand1(int signo)
{
    my_printf("Signal received for thread1 is %d\n", signo);
}

void sighand2(int signo)
{
    my_printf("Signal received is %d\n", signo);
}

```

Thread-Aware and Nonblocking OSS Functions

If a function must wait for an I/O operation to complete, the function blocks its caller (either a thread or a process) from continuing until the I/O operation completes. A function that blocks all threads while it waits for I/O to complete is a process-blocking function. Process-blocking functions are not thread aware. If a thread calls a process-blocking function, the calling thread and all the other threads in the process are prevented from continuing until the I/O operation is complete and the function returns a status indication. Standard OSS I/O library functions are not thread-aware.

In contrast to a process-blocking function, a thread-aware function blocks only the thread calling that function, instead of blocking all threads in the process. The other threads in the process can continue doing work while the calling thread waits for I/O to complete. A thread-aware function is sometimes called a thread-blocking function, because it blocks only the calling thread.

Sometimes it is not enough for a function to be thread-aware. There can be times when you do not want even the calling thread to wait for I/O to complete. Instead, you want the function to either succeed immediately or to fail immediately. Functions that do not block the calling thread or the other threads in the process are nonblocking functions. Nonblocking I/O functions are sometimes called asynchronous I/O functions.

The behavior of an OSS I/O library function can vary according to these factors:

- Whether the file descriptor is for a regular file or a non-regular file.
- Whether the function was designed to be process-blocking, thread-aware (thread-blocking), or nonblocking for that type of file.
- Whether the file descriptor passed to the function is nonblocking. A file descriptor is nonblocking when one of these conditions is true:
 - The file descriptor has the `O_NONBLOCK` flag set.
 - The file descriptor has been set to non-blocking mode by setting the argument to the *request* parameter to `F_SETFL` when calling the `fcntl()` function.

Regular Files

For regular files, thread-aware functions block the calling thread but allow the other threads to run, whether or not the file descriptor is nonblocking. These functions never return the errors `EAGAIN`, `EINPROGRESS`, or `EWOULDBLOCK` for regular files, so there are no nonblocking thread-aware functions for regular files. **Table 70: System and Equivalent Thread-Aware Functions** lists the functions for which PUT Model library provides a version that is thread-aware for regular files. You can disable thread-aware I/O for regular files by setting the `PUT_THREAD_AWARE_REGULAR_IO_DISABLE` environment variable before you start the process.

In addition to the functions listed in **Table 70: System and Equivalent Thread-Aware Functions**, these additional functions, although not thread-aware, support the use of thread-aware functions by serializing file operations on an open file:

```
dup2 ()
fstat64 ()
fstat ()
lseek64 ()
lseek ()
```

Serialization of File Operations On an Open File

The PUT Model library supports a thread-aware set of functions that serialize file operations on an open regular file. If a thread calls one of the thread-aware functions to access a file that already has a file operation in progress by a different thread, the calling thread is blocked until the prior file operation is complete. For example:

Procedure

1. Thread A calls the thread-aware version of the `read()` function to read the open file `MyDataFile`. The file does not have an operation in progress, so thread A starts reading the file.
2. Thread B calls the thread-aware version `write()` function to write to the same open file `MyDataFile`. Because the read operation started by thread A is still in progress, thread B is blocked and must wait for thread A's read operation to complete.

Non-Regular Files

For non-regular files, there are functions that can provide thread-aware behavior or both nonblocking and thread-aware behavior:

The set of functions that are thread-aware are thread-aware whether or not the file descriptor is nonblocking. If the `O_NONBLOCK` flag is set, these functions are also nonblocking: they do not block the calling thread or the other threads, and they return one of these errors to the application: `EAGAIN`, `EINPROGRESS`, or `EWOULDBLOCK`. Otherwise, these



functions block the calling thread but allow other threads to run. These functions remove the need for the application to explicitly set the `O_NONBLOCK` flag of the file descriptor to enable thread-aware behavior. The thread-aware functions are listed in the Nonblocking Thread-Aware Functions (Non-Regular Files Only) column of **Table 70: System and Equivalent Thread-Aware Functions**.

Equivalent System, Thread-Aware, and Nonblocking Thread-Aware Functions

The PUT Model library does not provide analogs to `spt_*x()` or `spt_*z()` functions, such as `spt_readx()` or `spt_readz()`, whose purpose is to be non-blocking alternatives to standard system library functions. Instead, you refer to the standard function name and compile and link the application as described in **Compiling and Linking Considerations**. Additionally, the PUT Model library does not provide analogs to `spt_*()` functions, such as `spt_fprintf()`, whose purpose is to be non-blocking alternatives to the standard C run-time library functions. Instead, you refer to the standard function names, which are thread aware for H06.21 and later H-series RVUs and J06.10 and later J-series RVUs.

Table 70: System and Equivalent Thread-Aware Functions lists standard functions that have equivalent thread-aware functions. For example:

- `read()` is a process-blocking function for both regular files and for non-regular files if you are not using the PUT Model library.
- The PUT Model library version of `read()` is a thread-aware function with this behavior: For non-regular files, this function does not require the file descriptor to be nonblocking for this function to be thread-aware. If the file descriptor is nonblocking, this function does not block the calling thread, and returns one of these errors to the application: `EAGAIN`, `EINPROGRESS`, or `EWOULDBLOCK`. Otherwise, this function blocks the calling thread but allows other threads to run. For regular files, this function blocks the calling thread but allows other threads to run. This function does not require the file descriptor to be nonblocking for this function to be thread-aware.

NOTE: To determine which RVUs support an OSS function, see **Documented OSS Functions**.

Table 70: System and Equivalent Thread-Aware Functions

Standard (Process-Blocking) OSS Functions	Nonblocking and thread aware in PUT Model library for regular files	Nonblocking and thread aware in PUT Model library for non-regular files	Thread aware in PUT Model library (functions not related to file I/O)
<code>accept()</code>	—	yes	—
<code>alarm()</code>	—	—	yes
<code>close()</code>	yes	yes	—
<code>connect()</code>	—	yes	—
<code>dup2()</code>	—	yes	—
<code>fcntl()</code>	yes	yes	—
<code>fork()</code>	—	—	yes

Table Continued



Standard (Process-Blocking) OSS Functions	Nonblocking and thread aware in PUT Model library for regular files	Nonblocking and thread aware in PUT Model library for non-regular files	Thread aware in PUT Model library (functions not related to file I/O)
<code>fstat()</code>	yes	—	—
<code>fstat64()</code>	yes	—	—
<code>fsync()</code>	yes	—	—
<code>ftruncate()</code>	yes	—	—
<code>ftruncate64()</code>	yes	—	—
<code>lseek()</code>	yes	—	—
<code>lseek64()</code>	yes	—	—
<code>pause()</code>	—	—	yes
<code>pread()</code>	yes ¹	—	—
<code>pread64_()</code>	yes ¹	—	—
<code>pwrite()</code>	yes ¹	—	—
<code>pwrite64_()</code>	yes ¹	—	—
<code>read()</code>	yes	yes	—
<code>readv()</code>	yes	—	—
<code>recv()</code>	—	yes	—
<code>recvfrom()</code>	—	yes	—
<code>recvmsg()</code>	—	yes	—
<code>select()</code>	—	yes	—
<code>send()</code>	—	yes	—
<code>sendmsg()</code>	—	yes	—
<code>sendto()</code>	—	yes	—

Table Continued



Standard (Process-Blocking) OSS Functions	Nonblocking and thread aware in PUT Model library for regular files	Nonblocking and thread aware in PUT Model library for non-regular files	Thread aware in PUT Model library (functions not related to file I/O)
<code>sigaction()</code>	—	—	yes
<code>sigaltstack()</code>	—	—	yes
<code>signal()</code>	—	—	yes
<code>sigpending()</code>	—	—	yes
<code>sigsuspend()</code>	—	—	yes
<code>sigwait()</code>	—	—	yes
<code>sleep()</code>	—	—	yes
<code>usleep()</code>	—	—	yes
<code>wait()</code>	—	—	yes
<code>waitpid()</code>	—	—	yes
<code>write()</code>	yes	yes	—
<code>writew()</code>	—	—	yes

¹ Thread-aware functions available starting from L18.08/J06.23.

The Set of Thread-Aware I/O Functions Must Be Used Together

The PUT Model library serializes I/O functions on open files only for the thread-aware regular I/O set of functions the PUT Model library provides. Using modules that were not compiled to use the PUT Model library can cause unexpected results. For example, compiling a module so that it calls the unmapped standard function `close()` directly, instead of the PUT Model library version of `close()`, might leave a thread with a pending thread-aware regular file request suspended with no mechanism available to reschedule it for execution. For example, if you compile one module to use the PUT Model library and another module to not use the PUT Model library, you can get errors when the two modules operate on the same open file:

Procedure

1. Thread A calls the PUT Model library version (a thread-aware version of `read()`) to read the open file `MyDataFile`. The file does not have an operation in progress, so thread A starts reading the file.
2. Thread B calls standard `read()` system function (you did not use the PUT Model library), which is not thread-aware, to read to the same open file `MyDataFile`. Because the standard `read()` function does not support serialization, the standard version of `read()` attempts to do the read and fails, returning the error `EALREADY`. The `EALREADY` error indicates that an operation is already in progress.

Using Thread-Aware Functions

File Stream Thread-Aware Function Usage shows using thread-aware functions with a file stream. It is not necessary to set the `O_NONBLOCK` flag.

File Stream Thread-Aware Function Usage

```
...
fs = fopen(path, mode);
fd = fileno(fs);
fcntl(fd, F_SETFL, fcntl(fd, F_GETFL) | O_NONBLOCK);
...
// Call thread-aware functions.
...
fclose(fs);
...
```

File Descriptor Thread-Aware Function Usage shows opening a file descriptor for use with a thread-aware function. It is not necessary to set the `O_NONBLOCK` flag.

File Descriptor Thread-Aware Function Usage

```
...
filedes = open(path, oflag, mode | O_NONBLOCK);
...
// Call thread-aware functions.
...
close(filedes);
...
```

Open File Descriptor Thread-Aware Function Usage shows preparing an open file descriptor for use with a thread-aware function. It is not necessary to set the `O_NONBLOCK` flag.

Open File Descriptor Thread-Aware Function Usage

```
...
fcntl(filedes, F_SETFL, fcntl(fd, F_GETFL) | O_NONBLOCK);
...
// Call thread-aware functions.
...
```

Pthreads Thread-Aware Disk File I/O Using PUT Model Library is a multithreaded program that performs I/O on multiple OSS regular files using the thread-aware functions that are available when you use the define `_PUT_MODEL_`. With these functions, you do not have to use Guardian functions to allow other threads to continue to work while another thread waits for I/O to complete on an open file.

Pthreads Thread-Aware Disk File I/O Using PUT Model Library

```
/*
 * sptdio.c - Pthreads thread-aware Diskfile IO Example Program
 *
 * This Pthreads program demonstrates a multithreaded program that
 * can perform I/O on multiple OSS regular files (diskfiles) using
 * thread-aware OSS I/O functions
 * Note that OSS I/O on regular files is serialized.
 * Note: If you want process-blocking I/O behavior, export the
 * PUT_THREAD_AWARE_REGULAR_IO_DISABLE = 1.
 *
 * The following threads can be created by this program:
 * - An OSS read thread which uses OSS I/O to completely
```

```

*      read a regular file.
*      - An OSS write thread which uses OSS I/O to write
*        1Mb of data to the regular file.
*
*  USAGE SYNTAX:
*
*  sptdio [-r <OSS-file>] [-w <OSS-file>]
*
*  FLAGS
*  -r <OSS-file>   Create a thread that uses OSS I/O to
*                  completely read the given OSS diskfile.
*  -w <OSS-file>   Create a thread that uses OSS I/O to write the
*                  given OSS diskfile.
*                  1Mb of data is written to the file.
*
*  EXAMPLE USAGE:
*
*  /home/user: run sptdio -r /tmp/r1 -w /tmp/w1
*
*  ==> All threads created...
*  ==> OSS_write_thread() wrote 1048576 bytes to file /tmp/w1
*  ==> OSS_read_thread() read 1581888 bytes from file /tmp/r1
*
*  OSS BUILD INSTRUCTIONS:
*
*  c89 sptdio.c -o sptdio -Wextensions -Wsystype=oss -g -D_XOPEN_SOURCE=1 \
*  -lZPUTDLL
*/

#define _PUT_MODEL_
#include <limits.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include "myio.h" /* contains my_printf which is process blocking */
/* Use the thread-aware library functions that are thread-aware */
/* (thread-blocking instead of process-blocking) for regular files */

#include <pthread.h>

/*****/
/* LITERALS/DEFINES */
/*****/

/* the maximum number of I/O threads created by this program */
#define MAX_THREADS 4

/* concurrency level value passed to pthread_setconcurrency() */
/* the value of 100000 sets the minimum scheduled quantum */
/* to 0.000001 seconds. */
#define CONCURRENCY_LEVEL 100000

/* I/O size used by all threads */
#define IOSIZE 8192

```

```

/* size of files that are written - 1Mb */
#define WRITE_FILESIZE 1048576

/* -----
 * OSS_read_thread --
 *
 * Description:
 *
 * This thread reads an OSS file using OSS I/O. Because this program
 * uses the define _PUT_MODEL>, the read() function is
 * mapped to the thread-aware read() function, which blocks
 * the thread instead of the entire process.
 * Other threads are free to do
 * work while this thread is blocked waiting for an I/O completion.
 *
 * Results:
 *   none.
 *-----
 */

void *OSS_read_thread(void *OSS_read_file)
{
    int fd;
    int nbytes;
    char *iobuf;
    int ret;
    long long filesize = 0;

    /*****
    /* prepare I/O buffer (page aligned, paged into physical memory) */
    *****/

    if ((iobuf = malloc(IOSIZE)) == NULL)
    {
        my_printf("*** ERROR in OSS_read_thread:  unable to allocate
            I/O buffer ***\n");

        exit(0);
    }
    memset(iobuf, 0, IOSIZE);

    /*****
    /* open the OSS file */
    *****/

    if ((fd = open(OSS_read_file, O_RDONLY | O_NONBLOCK)) < 0)
    {
        my_printf("*** ERROR in OSS_read_thread:  open() of %s failed
            w/error %d \***\n", OSS_read_file, errno);

        exit(0);
    }

    /*****
    /* read the OSS file in a thread aware manner */
    *****/

```



```

/*****/

while (1)
{
    nbytes = read(fd, iobuf, IOSIZE);

    if (nbytes < 0)
    {
        if ((errno == EWOULDBLOCK) || (errno == EAGAIN))
        {

/*****/
/* NOTE that this code does not execute for regular (disk) files, */
/* because read() on a regular file never returns EWOULDBLOCK or EAGAIN */
/*****/

            ret = put_fd_read_ready(fd, NULL);
            if (ret != 0)
            {
                my_printf("*** ERROR in OSS_read_thread:
                    put_fd_read_ready() \ failed w/err %d\n",ret);
                exit(0);
            }
        }
        else
        {
            my_printf("*** ERROR in OSS_read_thread:  read() failed w/error
                \ %d ***\n",errno);
            exit(0);
        }
    }
    else if (nbytes == 0) /* EOF */
    {
        break;
    }
    else /* read successful */
    {
        filesize += nbytes;
    }
} /* while */

close(fd);

my_printf("==> OSS_read_thread() read %ld bytes from file %s\n",
    filesize, OSS_read_file);
return(NULL);
}

/* -----
*
* OSS_write_thread --

```

```

*
* Description:
*
* This thread writes an OSS file using OSS I/O. Because this program
* uses the define _PUT_MODEL_, the write() function is
* mapped to the thread-aware version of the write() function, which
* blocks the thread instead of the entire process.
* Other threads are free to do
* work while this thread is blocked waiting for an I/O completion.
*
* Results:
* none.
*
*-----
*/

void *OSS_write_thread(void *OSS_write_file)
{
    int fd;
    int nbytes;
    char *iobuf;
    int ret;
    long long currsz = 0;

    /*****
    /* prepare I/O buffer (page aligned, paged into physical memory) */
    *****/

    if ((iobuf = malloc(IOSIZE)) == NULL)
    {
        my_printf("*** ERROR in OSS_write_thread:  unable to
                allocate I/O buffer \ ***\n");
        exit(0);
    }
    memset(iobuf, 0, IOSIZE);

    /*****/
    /* open the OSS file */
    /*****/

    if ((fd = open(OSS_write_file, O_WRONLY | O_NONBLOCK | O_CREAT |
O_TRUNC, S_IRWXU)) < 0)
    {
        my_printf("*** ERROR in OSS_write_thread:  open() of %s failed
                w/error \ %d ***\n", OSS_write_file, errno);
        exit(0);
    }

    /*****/
    /* write the OSS file in a thread-aware manner */
    /*****/

    while (currsz < WRITE_FILESIZE)
    {
        nbytes = write(fd, iobuf, IOSIZE);

```

```

        if (nbytes < 0)
        {
            if ((errno == EWOULDBLOCK) || (errno == EAGAIN))
            {
                /*****
                /* NOTE that this code does not execute for regular (disk) */
                /* files, because write() on a regular file never returns */
                /* EWOULDBLOCK or EAGAIN */
                *****/

                ret = put_fd_write_ready(fd, NULL);
                if (ret != 0)
                {
                    my_printf("*** ERROR in OSS_write_thread:
                        put_fd_write_ready() \ failed w/error %d ***\n",
                        ret);
                    exit(0);
                }
            }
            else
            {
                my_printf("*** ERROR in OSS_write_thread:  write() failed
w/error %d \
                        ***\n", errno);
                exit(0);
            }
        }
        else /* write successful */
        {
            currsz += nbytes;
        }
    } /* while */

    close(fd);

    my_printf("==> OSS_write_thread() wrote %d bytes to file %s\n",
        WRITE_FILESIZE, OSS_write_file);
    return(NULL);
}

```

```

int main(int argc, char *argv[])
{
    /* the following variables correspond to the OSS filenames */
    /* specified as command line parameters to the program */
    char OSS_read_filename[PATH_MAX];
    char OSS_write_filename[PATH_MAX];
    /* total number of threads that have been created */
    int nmbr_created_threads = 0;
    /* array of handles for each thread created */
    pthread_t thread_handle[MAX_THREADS];
    int i;

```

```

/*****
/* Process command line arguments */
*****/

if (argc > 9)
{
    my_printf("Usage:  sptdio [-r <OSS-file>] [-w <OSS-file>]\n");
    exit(0);
}

/* verify that an even number of command line parameters */
/* was specified */
if ((argc - 1) % 2) == 1)
{
    my_printf("Usage:  sptdio [-r <OSS-file>] [-w <OSS-file>]\n");
    exit(0);
}

for (i = 1; i < argc; i = i + 2)
{
    if (strcmp(argv[i], "-r") == 0)
    {
        strcpy(OSS_read_filename, argv[i+1]);
    }
    else if (strcmp(argv[i], "-w") == 0)
    {
        strcpy(OSS_write_filename, argv[i+1]);
    }
    else {
        my_printf("Usage:  sptdio [-r <OSS-file>] [-w <OSS-file>]\n");
        exit(0);
    }
}

/*****
/* Create all of the I/O threads */
*****/

for (i = 1; i < argc; i = i + 2)
{
    if (strcmp(argv[i], "-r") == 0)
    {
        pthread_create(&thread_handle[nmbr_created_threads++],
                      NULL, &OSS_read_thread, OSS_read_filename);
    }
    else if (strcmp(argv[i], "-w") == 0)
    {
        pthread_create(&thread_handle[nmbr_created_threads++],
                      NULL, &OSS_write_thread, OSS_write_filename);
    }
}

my_printf("\n==> All threads created...\n");

```

```

/*****
/* wait for the threads to complete -
/* this is where control is passed to the threads
/* to the threads
*****/

for (i = 0; i < nmbr_created_threads; ++i)
{
    pthread_join(thread_handle[i], NULL);
}

} /* main() */

```

Thread-Aware User Events

User events are events that are defined by an application process to facilitate cooperation and coordination with other processes on the same CPU. One process defines and owns the event and can wait for the event to occur; other processes can cause that occurrence, subject to security considerations. The user events that are supported on NonStop is described in the *Guardian Procedure Calls Reference Manual*.

31 user events are supported from L15.02 RVU and 63 user events are supported from L18.02/J06.22 RVU onwards. The user event feature is extended to a thread level, where multiple threads can wait for a single or multiple events to occur. Threads are woken up as soon as an event occurs. To use the user event feature at thread level, applications must use the PUT Model library infrastructure and APIs. The applications must use the Guardian API calls `USEREVENT_AWAKE_()` or `USEREVENT64_AWAKE_()` to trigger the event occurrence.

Make sure that user events are not used as a mode of communication between threads of the same process.

For more information on usage of `USEREVENT_AWAKE_()` and `USEREVENT64_AWAKE_()`, see *Guardian Procedure Calls and Reference Manual*.

To enable multithread applications to wait for an user event, the following PUT Model library APIs are provided:

PUT_USEREVENT_SET_()

Defines the set of user events upon which a thread intends to wait. This API can be invoked multiple times. Since `PUT_USEREVENT_SET_()` API performs the registration, PUT Model library does not provide a separate API to establish a file number to access the user events.

put_awaitiou64()

Allows a thread to wait for any event registered by the thread.

PUT_USEREVENT_GET_()

Returns the event or the set of events that are registered by the thread.

User Event API usage

- The application thread calls the `PUT_USEREVENT_SET_()` API to register for a set of user events upon which it wants to wait by passing events as an event mask.
- The thread calls `put_awaitiou64()` API to wait for any registered event occurrence. The thread is blocked until the event of interest occurs or for a timeout whichever is earlier. The `tag` parameter returns the event occurred. In



case of multiple event occurrences, the `tag` parameter returns the highest-priority event. To consume all events, this call must be iterated. Note that the event is not in the mask form.

- Multiple threads can wait for the same event. On the occurrence of this event, all threads waiting for that event are woken up.
- Threads waiting for an event can be interrupted using the `put_interruptTag()` API. Ensure that the event number is passed in the `tag` parameter.
- When a thread is woken up on an event occurrence, that event is removed from the set of events that are registered by the thread. To wait for the same event to reoccur, the thread must register for the event once again by calling the `PUT_USEREVENT_SET_()` API.
- If an event registered by multiple threads occurs before any of the threads wait for the event, the first thread calling `put_waitiou64()` API receives the event.

Considerations

- PUT Model Library requires NonStop kernel support for user event (32-bit or 64-bit) to function.
- User events are represented as bit masks of a 64-bit field, thus supporting a maximum of 63 events per thread. If NonStop Kernel 64-bit support is not available and only 32-bit field support available, each thread can only register for a maximum of 31 events.
- Hewlett Packard Enterprise recommends not to use the *fnum* returned from `PUT_USEREVENT_SET_()`. *fnum* must be used only with `put_waitiou64()` or `put_interruptTag()` APIs.



WARNING: Closing *fnum* exhibits undefined behavior.

- Hewlett Packard Enterprise does not recommend using `PUT_USEREVENT_SET_()`, `put_waitiou64()`, and `PUT_USEREVENT_GET_()` user event APIs from a signal handler.

Examples

A thread registering for multiple events and waiting for a specific event

The following snippet demonstrates a thread registering for multiple events and waiting for a specific event:

```
void thread_func()
{
    long long mask, retmask;
    short error = 0;
    short filenum;
    long long tag64 = 0;
    int put_err = 0;

    /* Thread wants to register for events 60 and 61 */
    mask = 0x3000000000000000;

    /* This step will set the event and the filenum will be used
     * to wait for the event to occur.
     */
    error = PUT_USEREVENT_SET_(mask, 0, &filenum);
    if (error)
        return;
```



```

/* This step is to get the set of events the thread
 * has registered.
 */
retmask = PUT_USEREVENT_GET_();

/* If the thread wants to wait for event 61 to occur, pass it
 * in the call to put_awaitiou64().
 * Note: If the thread wants to wait for any one of the events
 *       it has registered for (60 and 61 in this case), it needs
 *       to pass -1 as tag64. When the call returns, tag64 is updated
 *       with the event number that occurred.
 */
tag64 = 61;
error = put_awaitiou64(filenum, &tag64,
                      -1, NULL, &put_err, NULL);
if(error)
    return;
else
    printf("Event occurred:%d\n", tag64);
}

```

Multiple threads registering for the same event and waiting for that event

The following snippet demonstrates multiple threads registering for the same event and waiting for that event. Threads T1 and T2 are registering for event 60 to occur:

```

=====
/* Thread T1 can call */
mask = 0x10000000000000000;
PUT_USEREVENT_SET_(mask, 0, &filenum);
.....
Error = put_awaitiou64(filenum, &tag64, -1, NULL, &put_err, NULL);

/* Thread T2 can call */
mask = 0x10000000000000000;
PUT_USEREVENT_SET_(mask, 0, &filenum);
.....
Error = put_awaitiou64(filenum, &tag64, -1, NULL, &put_err, NULL);

/* Similarly other threads T3, T4 ... can call the same APIs. */
=====

```

Registering multiple threads and waiting for any event occurrence

The following snippet demonstrates how to register multiple threads and waiting for any event occurrence:

```

=====
/* Threads want to register for event 1, 2, 15 and 47 */

mask = 0x8000000008006;
PUT_USEREVENT_SET_(mask, 0, &filenum);

/* To wait for any one event to occur */
tag64 = -1;
put_awaitiou64(filenum, &tag64, -1, NULL, &put_err, NULL);

/* The event occurred is available in tag64. */
=====

```



Multiple threads registering for multiple events and the event occurs before the wait is completed

The following snippet considers three threads (events 1, 2, 15) registering for the same set of events:

```
=====
/* Threads want to register for event 1,2,15 */

mask = 0x8006;
PUT_USEREVENT_SET_(mask,0,&filenum);

/* Before any of the threads call put_awaitiou64() event 2 occurs. */
/* Then Thread 1 calls */

tag64 = 2;
error = put_awaitiou64(filenum,&tag64,-1,NULL,&put_err,NULL);

/* Call immediately returns with event number 2 since the event has already occurred. */
/* Now Thread 2 calls put_awaitiou64(). */
/* The thread is blocked for event 2 to occur again. */
/* If -1 is passed as tag64, the thread is woken up if any of the events 1, 2 or 15 occurs. */
=====
```

Registering a thread and waiting for the same event multiple times

The following snippet demonstrates how a thread registers and waits for the same event multiple times:

```
=====
/* Thread registers for event 15 */

mask = 0x8000;
PUT_USEREVENT_SET_(mask,0,&filenum);

/* Thread waits on event 15 */

tag64 = 15;
error = put_awaitiou64(filenum,&tag64,-1,NULL,&put_err,NULL);

/* Event 15 occurs and thread is woken up */
/* Thread wants to wait for event 15 again and hence it has to register first */

PUT_USEREVENT_SET_(mask,0,&filenum);
tag64 = 15;
error = put_awaitiou64(filenum,&tag64,-1,NULL,&put_err,NULL);

=====
```

Threads Portability

The PUT Model library follows the POSIX Threads Standard document. Using thread functions whose names end with `_np` (non-portable) decreases the portability of a program, because these functions are extensions to the Standard document. Using the jacket routines provided by Hewlett Packard Enterprise also reduces portability of a threaded application; see **Jacket Routines (Nonblocking Versions of SERVERCLASS_ and TMF System Calls)** for more information about the jacket routines.

Thread Safety

The lists in this section apply to systems running J06.10 or later J-series RVUs or H06.21 or later H-series RVUs only:



- Functions listed in the *Open System Services System Calls Reference Manual* or in the *Open System Services Library Calls Reference Manual* are safe for use in threaded programs unless they are listed in **Functions That Are Not Thread Safe**.
- Functions that contain a cancellation point are listed in **Cancellation Points**.
- Functions that allow a cancellation point to occur are listed in **Potential Cancellation Points**.

Functions That Are Not Thread Safe

These functions are not safe for use in threaded applications:

```
asctime()
catgets()
chvol()
crypt()
ctime()
dbm_clearerr()
dbm_close()
dbm_delete()
dbm_error()
dbm_fetch()
dbm_firstkey()
dbm_nextkey()
dbm_open()
dbm_store()
dlerror()
drand48()
ecvt()
encrypt()
fcvt()
fopen_guardian()
fopen_std_file()
fopen64_guardian()
fopen64_std_file()
freopen_guardian()
freopen64_guardian()
ftw()
ftw64()
gamma()
gcvt()
get_assign_msg()
get_assign_msg_by_name()
get_param_by_name()
get_param_msg()
get_startup_msg()
getc_unlocked()
getchar_unlocked()
getdate()
getenv()
getopt()
gfileno()
```



gmtime()
hcreate()
hdestroy()
hsearch()
inet_ntoa()
l64a()
lgamma()
localeconv()
localtime()
lrand48()
mrnd48()
nl_langinfo()
nlist()
nftw()
nftw64()
putc_unlocked()
putchar_unlocked()
re_comp()
re_exec()
readdir()
readdir_dt()
readdir64()
readdir64_dt()
regcmp()
regex()
setkey()
strerror()
strtok()
system_guardian()
tmpfile_guardian()
tmpfile64_guardian()
tmpnam_guardian()
ttyname()
wcstombs()
wctomb()

Cancellation Points

These functions contain a cancellation point:

close()
fcntl() (with F_SETLK as *cmd*)
fsync()
pread()
pread64_
pwrite()
pwrite64_
read()
readv()
sigwait()



```
write()
writev()
```

Potential Cancellation Points

A cancellation point might occur when a thread is executing one of these functions:

```
catgets()
dbm_close()
dbm_delete()
dbm_fetch()
dbm_firstkey()
dbm_nextkey()
dbm_open()
dbm_store()
dlclose()
dlopen()
endhostent()
endnetent()
endprotoent()
endservent()
fclose()
fflush()
fgetc()
fgetpos()
fgetpos64()
fgets()
fgetwc()
fgetws()
fmtmsg()
fprintf()
fputc()
fputs()
fputwc()
fputws()
fread()
freehostent()
freopen()
freopen_oss()
freopen64()
freopen64_oss()
fscanf()
fseek()
fseeko()
fseeko64()
fsetpos()
fsetpos64()
ftell()
ftello()
ftello64()
ftw()
```



```

ftw64()
fwprintf()
fwrite()
fwscanf()
getaddrinfo()
getc()
getc_unlocked()
getchar()
getchar_unlocked()
getdate()
gethostbyaddr()
gethostbyaddr_r()
gethostbyname()
gethostbyname_r()
gethostbyname2()
gethostent()
gethostent_r()
getipnodebyaddr()
getipnodebyname()
getnameinfo()
getnetbyaddr()
getnetbyaddr_r()
getnetbyname()
getnetbyname_r()
getnetent()
getnetent_r()
getprotobyname()
getprotobyname_r()
getprotobynumber()
getprotobynumber_r()
getprotoent()
getprotoent_r()
getservbyname()
getservbyname_r()
getservbyport()
getservbyport_r()
getservent()
getservent_r()
getw()
getwc()
getwchar()
getwd()
glob()
iconv_close()
iconv_open()
lseek()
lseek64()
lwres_getaddrinfo()
lwres_gethostbyaddr()

```



```

lwres_gethostbyaddr_r()
lwres_gethostbyname()
lwres_gethostbyname_r()
lwres_gethostbyname2()
lwres_getipnodebyaddr()
lwres_getipnodebyname()
lwres_getnameinfo()
mkstemp()
nftw()
nftw64()
pclose()
perror()
popen()
printf()
putc()
putc_unlocked()
putchar()
putchar_unlocked()
puts()
putw()
putwc()
putwchar()
rewind()
scanf()
sethostent()
setnetent()
setprotoent()
setservent()
strerror()
strerror_r()
ungetc()
ungetwc()
vfprintf()
vfwprintf()
vprintf()
vwprintf()
wordexp()
wprintf()
wscanf()

```

Reentrant OSS Functions

For J06.10 and later J-series RVUs and H06.21 and later H-series RVUs, reentrant functions are provided by the public libraries where the base non-reentrant functions are defined. This is in contrast to earlier RVUs in which the reentrant functions were only defined in the SPT library (The functions in the SPT library are still available). Applications that use the PUT Model library use the reentrant functions that are defined in the public libraries (do not attempt to use the reentrant function in the SPT library: you cannot include both modules that use the PUT Model library and the SPT library in the same application).

Reentrant functions include the `_r` suffix in their names. To determine if a function is reentrant, see the reference page for that function either online or in the *Open System Services System Calls Reference Manual*.



Many functions are unsafe in multithreaded applications, often because these functions return a pointer to a string or structure that is in static storage. A second call to the function overwrites the static storage (the results from the first call). Other issues for multithreaded applications include the use of global variables and maintaining context across calls.

In programs that use multiple threads, the same functions and the same resources might be accessed concurrently by several flows of control. To protect resource integrity, code written for multithreaded programs must be reentrant and thread safe.

A reentrant function does not hold static data over successive calls and does not return a pointer to static data. All data is provided by the caller of the function.

Example Program Using Reentrant Functions

Using the Reentrant Function `strtok_r` provides an example program that uses the `strtok_r()` reentrant function.

The program results in this output:

```
thread - 0 token - a
thread - 1 token - a
thread - 0 token - b
thread - 1 token - b
thread - 0 token - c
thread - 1 token - c
thread - 0 token - x
thread - 1 token - x
thread - 0 token - y
thread - 1 token - y
thread - 0 token - z
thread - 1 token - z
```

For comparison, the same program, using `strtok()`, instead of the reentrant version, `strtok_r()`, results in this output:

```
thread - 0 token - a
thread - 1 token - a
thread - 0 token - b
thread - 1 token - c
thread - 0 token - x
thread - 1 token - y
thread - 0 token - z
thread - 1 token - x
thread - 0 token - y
thread - 1 token - z
```

Using the Reentrant Function `strtok_r`

```
/* reent.c - Using the reentrant function strtok_r
 *
 * Compilation steps:
 *
 * /usr/bin/c89 -Wextensions -Wsystype=oss -c reent.c -o reent.o
 * /usr/bin/nld /usr/lib/crtmain.o reent.o ./libspt.a -obey \
 * /usr/lib/libc.obey -lzcredll -o reent
 */

#define _PUT_MODEL_
#include <pthread.h>
#include <unistd.h>
#include <string.h>
```

```

#include <stdio.h>
#include <stdlib.h>
#include "myio.h" /* contains my_printf which is process blocking */

void *gettok(void *args)
{
    char str[25] = "a,b,c,";
    char str5[25] = "x,y,z,";
    char **lasts;
    char *str1 = NULL;
    char *ptr;
    lasts = &str1;

    ptr = strtok_r(str, ",", lasts);
    while(ptr != NULL)
    {
        my_printf("\n thread - %d token - %s", args, ptr);
        sleep(2);
        ptr = strtok_r(NULL, ",", lasts);
    }

    ptr = strtok_r(str5, ",", lasts);
    while(ptr != NULL)
    {
        my_printf("\n thread - %d token - %s", args, ptr);
        sleep(2);
        ptr = strtok_r(NULL, ",", lasts);
    }
    my_printf("\n");
}

int main()
{
    pthread_t T[2];
    int ret_value, t;
    for(t=0; t < 2; t++)
    {
        ret_value = pthread_create(&T[t], NULL, gettok, (void *)t);
        if (ret_value != 0)
        {
            my_printf("Error in creating the thread");
            exit(1);
        }
    }
    pthread_join(T[0], NULL);
    pthread_join(T[1], NULL);
    exit(0);
}

```

Thread-Aware Toolkit Functions and Types

Functions written using the thread-aware toolkit (as mentioned in the following table) allow one or more threads to perform work on one or more files without any one thread unnecessarily blocking another thread. To enable these functions, you must include the `pthread.h` header file. You can use these functions with the thread-aware functions in the PUT Model library.

For details on each function, see the appropriate reference page in the *Open System Services System Calls Reference Manual*.

Table 71: Thread-Aware Toolkit Functions and Types

Function	Description
<code>put_unregFile()</code>	Unregister the file number as one you will manage. Any threads waiting on file number I/O will awaken with <code>put_ERROR</code> and system error 16.
<code>put_FileIOHandler_p()</code>	Callback type required by <code>put_regFileIOHandler()</code> . A callback is executed in the context of the last running thread; that is, it executes on the stack of the last running thread.
<code>put_regFileIOHandler()</code>	Register the file number as one you will manage through a user-supplied callback. Invoke this callback immediately after each I/O on <code>filenum</code> completes.
<code>put_wakeup()</code>	Wake up a thread awaiting the tagged I/O on file number. The awakened thread returns from <code>put_awaitio()</code> with <code>put_SUCCESS</code> .
<code>put_defaultCallback()</code>	Default callback used by <code>put_regFile()</code> . It calls <code>put_wakeup()</code> and passes on the supplied parameters.
<code>put_regFile()</code>	Register the file number as one you will manage through the default callback.
<code>put_awaitio()</code>	Await a tagged IO on file number to complete, timeout, or be interrupted. I/O is never canceled by this function, and completes only if <code>put_SUCCESS</code> is returned. Multiple threads should not await the same tagged I/O on any given file number.
<code>put_interrupt()</code>	Interrupt all threads awaiting I/O on file number. I/O is not cancelled by this function. Interrupted threads will return from <code>put_awaitio()</code> with a return value of <code>put_ERROR</code> .
<code>put_interruptTag()</code>	Interrupt the thread awaiting the tagged IO on file number. I/O is not cancelled by this function. Interrupted threads will return from <code>put_awaitio()</code> with a return value of <code>put_ERROR</code> .
<code>put_generateTag()</code>	Increment and return a static long appropriate for use as a tag. This long will eventually wrap and therefore return tags that may still be in use. For example, if a process calls <code>put_generateTag()</code> 100 times every second, the wrap will occur on the 248th day.
<code>put_fd_read_ready()</code>	Wait on a file descriptor to be read ready or have an exception pending.
<code>put_fd_write_ready()</code>	Wait on a file descriptor to be write ready or have an exception pending.

Using Thread-Aware Toolkit Functions

The following examples show using thread-aware toolkit functions:

- Thread-aware toolkit program flow — **Thread-Aware Toolkit Program Flow**.
- Simplified program flow for a thread-aware function — **Simplified Program Flow For a Thread-Aware Function**
- Nonblocking thread-aware `read()` function — **Nonblocking Thread-Aware `read()` Function**
- Function using `put_READX()` — **Example `READX()` Function Used In Thread-Aware Manner**

Thread-Aware Toolkit Program Flow uses the default callback mechanism by registering the file number via `put_regFile(2)`. In some cases, you may wish to provide your own callback using `put_regFileIOHandler(2)`, which registers the file number as one you will manage through the callback you define.

Thread-Aware Toolkit Program Flow

```
...
FILE_OPEN(..., &filenum, ...);
put_regFile(filenum);
...
// Do work on filenum.
...
put_unregFile(filenum);
FILE_CLOSE(filenum);
...
```

Simplified Program Flow For a Thread-Aware Function

```
...
READX(filenum, ...);
pute = put_awaitio(filenum, tag, -1, count_read, &error, NULL);
if (pute != put_SUCCESS) (void) CANCELREQ(filenum, tag);
return error;
...
```

Nonblocking Thread-Aware `read()` Function illustrates using a nonblocking thread-aware `read()` function. If you were to disable thread-aware behavior by using the `PUT_THREAD_AWARE_REGULAR_IO_DISABLE` environment variable, you risk blocking the whole process if the file descriptor points to a regular file.

Nonblocking Thread-Aware `read()` Function

```
/* This program fragment assumes that you are using the      */
/* PUT Model library. For regular files, the read() function */
/* is thread-aware unless you have set the                   */
/* PUT_THREAD_AWARE_REGULAR_IO_DISABLE environment         */
/* variable. For non-regular files, the read()              */
/* function can be thread aware or thread aware nonblocking.*/
/* If the file descriptor has O_NONBLOCK set, the read()    */
/* function is thread-aware nonblocking, and might return   */
/* EWOULDBLOCK and EAGAIN                                  */

...
#define _PUT_MODEL_ /* maps read() to thread-aware read() */
#include <pthread.h>
```

```

#include <unistd.h>
#include <errno.h>
...
{
    int rv;
    while (1)
    {
        rv = read(filedes, buffer, nbytes);

        /* If read() returned an error. */
        if (rv < 0)
        {
            /* If not read ready then wait (assumes EWOULDBLOCK would be sent.) */
            if ((errno == EWOULDBLOCK) || (errno == EAGAIN))
            {
                rv = put_fd_read_ready(filedes, NULL);

                /* If put_fd_read_ready() returned an error. */
                if (rv != 0)
                {
                    errno = rv;
                    return -1;
                }

                continue;
            }
        }

        break;
    }

    return rv;
}

```

Example READX() Function Used In Thread-Aware Manner shows a code fragment that uses the `READX()` function in a thread-aware manner, which is appropriate when you are not using the thread-aware library. An alternative to this approach is to use the `put_READX()` thread-aware function.

Example READX() Function Used In Thread-Aware Manner

```

{
    put_error_t spte;
    long error;
    _cc_status cc;
    long tag;

    tag = put_generateTag();
    cc = READX(filenum, buffer, read_count, , tag);

    /* If READX() failed. */
    if (_status_lt(cc))
    {
        short short_error;

        (void) FILE_GETINFO_(filenum, &short_error);
        return short_error;
    }
}

```

```

/* Loop forever. */
while (1)
{
    spte = put_awaitio(filenum, tag, timelimit, count_read,
        &error, NULL);

    /* If spte is put_INTERRUPTED then try again. */
    if (spte != put_INTERRUPTED) break;
}

/* If put_awaitio() was unsuccessful.*/
if (spte != put_SUCCESS) (void) CANCELREQ(filenum, tag);

return error;
}

```

Caveats to Using the Thread-Aware Toolkit

Proper operation of the thread-aware toolkit depends on the following conditions:

- No calls to AWAITIO[X] O with a file number of -1 are made.
- No calls to AWAITIO[X] O with a file number registered with pthreads are made.
- No calls to FILE_COMPLETE_SET_ O are made.
- No calls to FILE_COMPLETE_ O are made using the permanent set.

Thread-Aware \$RECEIVE I/O Functions

\$RECEIVE thread-aware functions (**s**) integrate the following basic \$RECEIVE I/O functions with the POSIX thread engine and support.

- Read \$RECEIVE (similar to READUPDATEX) so that it is thread-blocking, but not process-blocking.
- Read \$RECEIVE from one or multiple threads and returning all pertinent information (FILE_GETRECEIVEINFO) in a thread-safe form.
- Inherit transactions on \$RECEIVE and properly associating or disassociating the transaction with the calling thread.
- Block a reading thread if the number of messages read has exceeded receive depth.

For details on each function, see the appropriate reference page in the *Open System Services System Calls Reference Manual*.

The functions listed above provide only basic \$RECEIVE handling support, not higher-level capabilities associated with a \$RECEIVE package. You can build such higher-level functions using the primitives present in the thread-aware \$RECEIVE functions, but must tailor them for whatever domain-specific application they occur in. High-level capabilities **not supported** by the thread-aware \$RECEIVE functions include:

- Maintenance of an opener's table
- Pathway server semantics (exit upon last close)
- Automatic resend of last replies for NonStop requesters
- Distribution of messages based on type or opener



- Any special handling of system messages (system messages are supported, but are treated as any other message)
- Encapsulation of the open or close of \$RECEIVE, or any SETMODE

Table 72: Thread-Aware \$RECEIVE Functions and Types

Function	Description
<code>put_INITRECEIVE()</code>	Register <code>filenum</code> being managed by the \$RECEIVE callback
<code>put_RECEIVEREAD()</code>	Thread-aware function specifically for reading \$RECEIVE
<code>put_REPLYX()</code>	Thread-aware <code>REPLYX()</code>
<code>put_READX()</code>	Thread-aware <code>READX()</code>

Using Thread-Aware \$RECEIVE Functions

Thread-Aware \$RECEIVE Program Flow through **\$RECEIVE Server** show using thread-aware \$RECEIVE functions.

Thread-Aware \$RECEIVE Program Flow shows the program flow for using a thread-aware \$RECEIVE function.

Thread-Aware \$RECEIVE Program Flow

```
...
FILE_OPEN(..., &filenum, ..., receive_depth, ...);
put_INITRECEIVE(filenum, receive_depth);
...
// Do work on filenum.
...
put_unregFile(filenum);
FILE_CLOSE(filenum);
...
```

Function with `put_RECEIVEREAD()` and `put_REPLYX()` shows using the thread-aware functions `put_RECEIVEREAD()` and `put_REPLYX()` to perform the actual work on \$RECEIVE.

Function with `put_RECEIVEREAD()` and `put_REPLYX()`

```
...
while (1)
{
    if (put_RECEIVEREAD(...) != 0) break;
    ...
    // Do work.
    ...
    if (put_REPLYX(...) != 0) break;
}
...
```

\$RECEIVE Server shows a single thread calling the thread-aware `put_RECEIVEREAD()` function. Using thread-aware \$RECEIVE functions allows one or more threads to perform work on \$RECEIVE, without a thread unnecessarily blocking another thread.



\$RECEIVE Server

```
#define _PUT_MODEL_
#include <cextdecs.h(FILE_OPEN_)>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include "myio.h" /* contains my_fprintf which is process blocking */

short filenum;
void *
thread_start(void *parms)
{
    long bytesread, error;
    char buffer[2];
    short receive_info[17];
    while (1)
    {
        error = put_RECEIVEREAD(filenum, buffer,
                                (short)sizeof(buffer), &bytesread, -1,
                                receive_info, NULL);
        if (error != 0) break;
        /* Do real work here! */
        error = put_REPLYX("server's reply.", 15, NULL,
                           receive_info[2], 0);
        if (error != 0) break;
    }
    my_fprintf(stderr, "thread_start: error: %d\n", error);
    return NULL;
}

int
main()
{
    pthread_t thread1, thread2;
    char *filename = "$RECEIVE";

    if (FILE_OPEN_(filename, (short)strlen(filename), &filenum,
                    0, , 1, 2, 1) != 0)
    {
        my_fprintf(stderr, "main: FILE_OPEN_ failed.\n");
        exit(1);
    }

    if (put_INITRECEIVE(filenum, 2) != 0)
    {
        my_fprintf(stderr, "main: put_INITRECEIVE failed\n");
        exit(1);
    }

    if (pthread_create(&thread1, NULL, &thread_start, NULL) != 0)
    {
        perror("main: pthread_create");
        exit(1);
    }
}
```

```

    }

    if (pthread_create(&thread2, NULL, &thread_start, NULL) != 0)
    {
        perror("main: pthread_create");
        exit(1);
    }

    if (pthread_join(thread1, NULL) != 0)
    {
        perror("main: pthread_join");
        exit(1);
    }

    if (pthread_join(thread2, NULL) != 0)
    {
        perror("main: pthread_join");
        exit(1);
    }
}

```

Caveats to Using Thread-Aware \$RECEIVE

Proper operation of thread-aware \$RECEIVE depends on the following conditions:

- No calls to AWAITIO[X] O with a file number of -1 are made.
- No calls to AWAITIO[X] O with a file number for \$RECEIVE are made.
- No calls to FILE_COMPLETE_SET_ O are made.
- No calls to FILE_COMPLETE_ O are made using the permanent set.

Jacket Routines (Nonblocking Versions of SERVERCLASS_ and TMF System Calls)

A jacket routine invokes a nonblocking (nowait) version of the system call and suspends the thread until the thread's operation is complete. The PUT Model library provides jacket routines for SERVERCLASS_SEND and SERVERCLASS_SENDL_ calls and for TMF transaction demarcation.

SERVERCLASS_SEND and SERVERCLASS_SENDL Jacket Routines

The PUT_SERVERCLASS_SEND_ () function in the POSIX User Threads Model library sends a message to and receives a reply from a server process in a Pathway server class. This jacket routine invokes a nowait version of the system SERVERCLASS_SEND_ procedure. The thread that makes the call is suspended until the send operation is complete.

The PUT_SERVERCLASS_SENDL_ () function also sends a message to and receives a reply from a server process in a Pathway server class. This jacket routine invokes a nowait version of the system SERVERCLASS_SENDL_ procedure. The thread that makes the call is suspended until the send operation is complete. This function allows for messages up to two megabytes long when Expand-over-ServerNet is used. This function does not support using Expand-over-IP.

These functions can be used with the PUT_SERVERCLASS_SENDL_ () function:

```

put_INITRECEIVE()
put_RECEIVEREADL()
put_REPLYXL()

```



Another jacket routine, the `PUT_SERVERCLASS_SEND_INFO_()` function returns information about the last `PUT_SERVERCLASS_SEND_()` or `PUT_SERVERCLASS_SENDL_()` call that was attempted or completed by the current thread. The `PUT_SERVERCLASS_SEND_INFO_()` function ensures that the thread gets the information for the last message sent by the thread rather than by the process.

SERVERCLASS_DIALOG Jacket Routines

The Pathsend `SERVERCLASS_DIALOG` jacket routines are used to:

- Establish a dialog with a server instance in a Pathway server class
- Send and receive messages within the dialog
- Abort or end the dialog

The Context-Sensitive Pathway requester can be written using the thread-aware `SERVERCLASS_DIALOG` jacket routines to communicate with a server instance in the Pathway server class. The Pathway requester initiates the dialog using `PUT_SERVERCLASS_DIALOG_BEGIN_()` routine and sends the first message of the dialog. This thread-aware jacket routine returns the dialog ID that is used in subsequent operations to communicate with the Pathway server. The Pathway requester can initiate multiple sends using `PUT_SERVERCLASS_DIALOG_SEND_()` within the dialog to send data to the server for processing. If the requester encounters an error while sending the data, the requester can abort the dialog using the `PUT_SERVERCLASS_DIALOG_ABORT_()` routine.

The Pathway requester can use multiple `PUT_SERVERCLASS_DIALOG_SEND_()` calls to have all requests serviced by the same server and all operations performed in a single context.

If the requester encounters Pathsend errors during the server class operation, the requester can call `PUT_SERVERCLASS_SEND_INFO_()` to obtain information about the Pathsend error and file-system error.

Either the requester or the server can abort the dialog, but only the server can end it. The server ends the dialog by returning file-system error 0 (FEOK). The requester then calls `PUT_SERVERCLASS_DIALOG_END_()` to clean up resources after the server has ended the dialog. To abort the dialog, the requester calls `PUT_SERVERCLASS_DIALOG_ABORT_()`.

The `SERVERCLASS_DIALOG` jacket routines are included in the PUT Model library, which can be bound to an application requiring the routines:

```
c89 -o program program .o -lzputdll
```

The Pathway requester can initiate the dialog in a nowait form using `SERVERCLASS_DIALOG_BEGIN_()` routine and then register the `scsend-op-num` obtained with the PUT Model library calling `put_regPathsendFile()`. The PUT library then completes the operation. In this case, the requester must ensure that all the operations on this dialog (`send`, `abort`) are completed using the Pathsend `SERVERCLASS_DIALOG` routines instead of Pathsend `SERVERCLASS_DIALOG` jacket routines.

TMF Transaction Jacket Routines

The following Transaction Management Facility (TMF) transaction jacket routines invoke the TMF calls and manage the multiple concurrent transactions rooted in a process:

```
PUT_ABORTTRANSACTION()  
PUT_BEGINTRANSACTION()  
PUT_BEGINTRANSACTION_EXT_()  
PUT_ENDTRANSACTION()  
PUT_RESUMETRANSACTION()  
PUT_TMF_GetTxHandle()  
PUT_TMF_JOIN()  
PUT_TMF_RESUME()
```



```
PUT_TMF_SetTxHandle()  
PUT_TMF_SUSPEND()
```

The syntax and semantics of these function calls are the same as those of the similarly named TMF calls.

Please refer to the man page for details. When using the TMF transaction jacket routines:

- Use `PUT_TMF_Init()` during application initialization before starting any transaction and this API opens the *TFILE* implicitly.
- The *transaction_tag* parameter of the `PUT_BEGINTRANSACTION()` function is required.
- Each thread can have at most one current transaction. The system `RESUMETRANSACTION` procedure is automatically called during thread dispatch to maintain the correct transaction context.
- Each transaction is initially associated with only the thread that created it by calling the `PUT_BEGINTRANSACTION()` function.
- Another thread can be associated with a specific transaction by calling the `PUT_RESUMETRANSACTION()` function and specifying the *transaction_tag* value returned by the initial `PUT_BEGINTRANSACTION()` function call.
- If a thread is created, it does not inherit its parent's transaction; the created thread's transaction context is nil.
- If a thread terminates or is destroyed while it has an active transaction, that transaction is then inaccessible from the process. If cancellation is a possibility, use exception handlers for `pthread_cancel_e`. Alternatively, use the `pthread_cleanup_push()` and `pthread_cleanup_pop()` functions to ensure that the transaction is terminated if the thread exits or is cancelled.
- If a transaction is aborted by either an operator, network partition, or another process, an error is reported when the initiating thread attempts to operate on the transaction.
- A process using the TMF transaction jacket routines must not call the file system procedure calls that pertain to TMF:
 - Any indirect invocation of `BEGINTRANSACTION()`, `BEGINTRANSACTION_EXT()`, and so on.
 - The SQL/MP `BEGIN-WORK` and `COMMIT-WORK` procedures.
- A process can perform up to the default of 100 concurrent transactions, with up to 1000 transactions maximum declared using the `put_setTMFConcurrentTransaction()` function. If the maximum is exceeded, a call to the `put_setTMFConcurrentTransaction()` function fails, returning error number 83 (too many transactions).
- `PUT_TMF_JOIN` allows the current thread to participate in a transaction initiated by another process without taking over as the `PUT_ENDTRANSACTION` caller for threaded application or `ENDTRANSACTION` caller for non-threaded application.

Thread-Aware Enscribe Functions

Table 73: Thread-Aware Enscribe Functions provide thread aware wrappers for Enscribe procedure calls that enable an application to make use of Enscribe procedure calls without blocking the process.

Thread-aware Enscribe functions are thread-blocking, like functions in the thread-aware library, instead of process-blocking, like other Enscribe functions.

Thread-aware Enscribe functions call the Guardian Procedure calls in 32-bit compilation. For 64-bit compilation corresponding 64-bit Guardian APIs are called by thread-aware Enscribe functions.

The parameters and the semantics for the new APIs are the same as those of Enscribe procedures, which are described in the *Guardian Procedure Calls Reference Manual*.



The thread-aware APIs for Enscribe procedure calls can:

- Open a file
- Read and write
- Cancel I/O operations issued on the file
- Lock and unlock the file or a record
- Execute device-dependent operations on an open file
- Set device-dependent functions in an open file

These functions must be explicitly called. There are no macros to map to these functions from the non-thread aware versions.

! **IMPORTANT:** The POSIX User Thread (PUT) Model library is the only 64-bit thread model supported.

Table 73: Thread-Aware Enscribe Functions

Function	Description
PUT_BEGINTRANSACTION_EXT_ ¹	Starts a new transaction associated with the current process and current thread
PUT_CANCEL	Cancels the oldest incomplete operation on a file opened nowaited
PUT_CONTROL	Performs device-dependent I/O operations
PUT_FILE_CLOSE_	Closes an open file
PUT_FILE_OPEN_	Establishes a communication path between an application process and a file
PUT_FILE_WRITEREAD_ ¹	Writes data to a process previously opened from an array and waits for data to be transferred back from the process
PUT_READX	Returns data from an open file to the application process data area
PUT_READUPDATEX	Reads data from a disk or process file in anticipation of a subsequent write to the file
PUT_READLOCKX	Sequentially locks and reads records in a disk file
PUT_READUPDATELOCKX	Locks, then reads the record from the current position in the file in anticipation of a subsequent call to PUT_WRITEUPDATEX or PUT_WRITEUPDATEUNLOCKX
PUT_SERVERCLASS_DIALOG_BEGIN_ ¹	Initiates the dialog and sends the first message of the dialog to the server instance in the Pathway serverclass (larger message version)

Table Continued

Function	Description
PUT_SERVERCLASS_DIALOG_SEN DL_ ¹	Initiates a send within the specified dialog to the server instance in the Pathway serverclass
PUT_SETMODE	Sets device-dependent functions
PUT_WRITEREADX	Writes data to a file from an array in the application, then waits for data to be transferred back from the file
PUT_WRITEX	Writes data from an array in the application to an open file
PUT_WRITEUPDTEX	Transfers data from an array in the application to a file
PUT_WRITEUPDATEUNLOCKX	Performs random processing of records in a disk file
PUT_LOCKFILE	Excludes other users from accessing a file
PUT_UNLOCKFILE	Unlocks a disk file and any records in that file currently locked by the user
PUT_LOCKREC	Excludes other users from accessing a record at the current position
PUT_UNLOCKREC	Unlocks a record currently locked by the user

¹Beginning with the H06.24 and J06.13 RVUs.

Using Thread-Aware Enscribe Functions

Applications can use a mix of process-blocking Enscribe functions and thread-aware Enscribe functions based on the requirement. An application that needs to initiate I/O from the callback function it registered must invoke process-blocking Enscribe functions, since the callback function must not block its invoking thread. The behavior of the process based on the functions invoked are as follows:

Thread blocking I/O

- File opened for no-waited I/O using PUT_FILE_OPEN_. If the I/O is initiated using an Enscribe function by specifying a tag, then it can be completed with a call to `put_awaitio()` by specifying the same tag.
- File opened for waited I/O using PUT_FILE_OPEN_ and the I/O initiated using thread-aware Enscribe function.
- File opened for no-waited I/O using FILE_OPEN_ and the file number registered using `put_regFileIOHandler()/put_regFile()`. If the I/O is initiated using an Enscribe function by specifying a tag, then it can be completed with a call to `put_awaitio()` by specifying the same tag. The I/O can be initiated using the thread-aware Enscribe function which takes care of its completion.
- File opened for waited I/O using FILE_OPEN_ and the file number registered using `put_regFileIOHandler()/put_regFile()` and the I/O initiated using thread-aware Enscribe function.

Process blocking I/O



- File opened for no-waited I/O using `PUT_FILE_OPEN_`. The I/O initiated using thread-aware `Enscribe` function or using `Enscribe` function without specifying a tag must be completed with a call to `AWAITIOX()`.
- File opened for waited I/O using `PUT_FILE_OPEN_` and the I/O initiated using `Enscribe` function.
- File opened for no-waited I/O using `FILE_OPEN_` and file number registered using `put_regFileIOHandler()/put_regFile()`. The I/O initiated using `Enscribe` function must be completed with a call to `AWAITIOX()`.
- File opened for waited I/O using `FILE_OPEN_` and file number registered using `put_regFileIOHandler()/put_regFile()` and the I/O initiated using `Enscribe` function.

Undefined behavior

- File opened for waited or no-waited I/O using `FILE_OPEN_` and the file number is not registered with PUT Model library. The I/O initiated using thread-aware `Enscribe` function results in undefined behavior. The recommendation is to use Guardian `Enscribe` functions.
- A call to `AWAITIOX()` that waits for any completion (where the `filenum` is passed as -1) will receive completions for IO initiated by any thread. There is no way for the completion to be returned to the PUT library, so the completion is effectively lost and the application will fail.

Using the Thread-Aware `PUT_FILE_OPEN` Function shows an example of using the thread-aware `PUT_FILE_OPEN_`. This function blocks the calling thread instead of the calling process.

Compilation:

```
/usr/bin/c89 -Wextensions -Wstype=oss -c ens_thread.c -o ens_thread.o

nld /usr/lib/crtmain.o ens_thread.o libspt.a -obey /usr/lib/libc.obey
-lzcp1srl -o ens_thread
```

Output:

```
Hello, I am in Thread1
Calling PUT_FILE_OPEN_ - thread-blocking Enscribe call
Hello, I am Thread2
error returned in open : 0 filenum : 6
error in file close 0
```

In contrast, if you replace the calls to the thread-blocking `PUT_FILE_OPEN_` function with calls to the process-blocking `FILE_OPEN_` function, and change the `my_printf()` statement accordingly, the program results in the following output:

```
Hello, I am in Thread1
Calling FILE_OPEN_ - process-blocking Enscribe call
error returned in open : 0 filenum : 6
error in file close 0
Hello, I am Thread2
```

Using the Thread-Aware `PUT_FILE_OPEN` Function

```
/* Using the thread-aware PUT_FILE_OPEN function */
#define _PUT_MODEL_
#include <cextdecs.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <tal.h>
#include "myio.h" /* contains my_printf which is process blocking */
```

```

short filenum;

void *Thread2(void *args)
{
    my_printf("Hello, I am Thread2\n");
}

void *Thread1(void *args)
{
    short error;
    char *name = "$SYSTEM.TEST.EX2";

    my_printf("\nHello, I am in Thread1\n");

    my_printf("Calling PUT_FILE_OPEN_ - thread-blocking Enscribe call \n");
    error = PUT_FILE_OPEN_(name, (short)strlen(name), &filenum);
    my_printf("error returned in open : %d filenum : %d\n", error, filenum);

    error = FILE_CLOSE_(filenum);
    my_printf("error in file close %d\n", error);
}

int main()
{
    pthread_t T[2];
    int ret_value;
    int t = 1;

    ret_value = pthread_create(&T[0], NULL, Thread1, (void *)t);
    if (ret_value != 0)
    {
        my_printf("Error in creating the thread");
        exit(1);
    }

    ret_value = pthread_create(&T[1], NULL, Thread2, (void *)t);
    if (ret_value != 0)
    {
        my_printf("Error in creating the thread");
        exit(1);
    }

    pthread_join(T[0], NULL);
    pthread_join(T[1], NULL);
    exit(0);
}

```

OSS File Descriptor and Timer Callbacks

Thread-aware functions listed in the following table allow registration and use of callbacks for OSS file descriptors and timer callbacks. For details on each function, see the appropriate reference page in the *Open System Services System Calls Reference Manual*.



Table 74: Thread-Aware Callback Functions and Types

Function	Description
<code>put_OSSFileIOHandler_p</code>	Callback type required by <code>put_regOSSFileIOHandler()</code> .
<code>put_regOSSFileIOHandler</code>	Register the file descriptor as one that the user will manage through a user supplied callback.
<code>put_unregOSSFileIOHandler</code>	Unregister file descriptor as one that the user will manage.
<code>put_setOSSFileIOHandler</code>	Set interest in file descriptor.
<code>put_regTimerHandler</code>	Register a user supplied timer callback.
<code>put_TimerHandler_p</code>	Callback type required by <code>put_regTimerHandler()</code> .

Using OSS and Timer Callbacks

OSS File Descriptor Callback shows an OSS file descriptor callback program flow. Typically, the callback would conditionally signal a thread that the file descriptor is read-ready.

OSS File Descriptor Callback

```
...
put_regOSSFileIOHandler(fd, callback);
put_setOSSFileIOHandler(fd, 1, 0, 0); // Only interested in read ready.
...
// Do work on file descriptor via user supplied callback.
...
put_unregOSSFileIOHandler(fd);
...
```

Timer Callback Program Flow shows an OSS timer callback program flow. As soon as possible, the pthreads scheduler initially invokes the timer callback, with successive invocations of the callback driven by the callback's return value. For more information, see the `put_TimerHandler_p(2)` reference page.

Timer Callback Program Flow

```
...
put_regTimerHandler(callback);
...
```

Caveat to Using Callbacks

An OSS file descriptor callback takes precedent over all other operations that are waiting on the file descriptor. For example, if a thread is blocked on `put_fd_read_ready()` and there is a callback registered for that particular file descriptor, the thread will never be awakened by the file descriptor being read-ready, even if the callback didn't set interest in read-ready.

Timer callbacks must not block and may be invoked sooner or later than requested.



Converting Applications that use the SPT Library

The Standard POSIX Thread (SPT) library continues to be supported. You can run existing SPT applications without changes. However, for H06.21 and later H-series RVUs and J06.10 and later J-series RVUs, you must specify the `_SPT_MODEL_` macro when you compile applications that use the SPT library. For more information about the SPT library, see [64-Bit Support in OSS and Guardian](#).

This section describes instructions and considerations for converting an application from using the SPT library to using the PUT Model library.

Build Scripts

Build scripts used to compile application modules must define macro `_PUT_MODEL_`. This macro determines which definitions are obtained from various header files such as `sys/types.h` and `stdio.h`. The build scripts must not define macro `_SPT_MODEL_`. The build scripts must also specify linking the PUT Model library (for example, the `c89 -lput` option) and must not specify linking the SPT library (for example, do not use the `-lspt` option).

Although dynamic linking is possible for the SPT library (a call to `dlopen()` can cause the SPT library to be linked during the execution of the application). Static linking is required for the PUT Model library. The run-time environment terminates execution of an application if a call to the `dlopen()` function requires the dynamic linking of the PUT Model library.

Header Files

You must change the header file in `#include` statements from `spthread.h` to `pthread.h`.

Standard Header Files

Because the `pthread.h` header file includes different standard header files than the `spthread.h` header file, you might need to change your application to explicitly include header files that were formerly provided by the `spthread.h` header file.

Table 75: Standard header files included in `spthread.h` and `pthread.h` header files

Header file	Included Standard Header Files
<code>spthread.h</code>	<code>errno.h</code> <code>fcntl.h</code> <code>sched.h</code> <code>stdio.h</code> <code>stdlib.h</code> <code>signal.h</code> <code>unistd.h</code> <code>wchar.h</code> <code>sys/socket.h</code> <code>sys/stat.h</code> <code>sys/time.h</code>
<code>pthread.h</code>	<code>sched.h</code> <code>setjmp.h</code> <code>sys/types.h</code>

It is important to investigate any warning diagnostics generated when a source text is compiled after changing an inclusion of `spthread.h` into an inclusion of `pthread.h`. For example, a warning such as function `"fcntl"`



declared implicitly reports that no explicit definition of `fcntl()` was declared, therefore the compiler assumed defaults for all function attributes. Consequently, calls to `fcntl()` are not mapped to the non-blocking variant of `fcntl()`. For a PUT Model application this situation causes an unneeded and unwanted suspension of the entire process. The remedy in such cases is to include whichever standard header file defines the function and to ensure that you have also defined the `_PUT_MODEL_` macro.

Nonstandard Header Files

The SPT library provides nonstandard headers in these header files:

```
spt_extensions.h
spt_types.h
spt_ucontext.h
```

The PUT Model library provides nonstandard headers in the header file:

```
put/put_extensions.h, which includes the header file put/put_types.h
```

The `put/put_extensions.h` header file contains definitions supporting the enhanced function behavior that SPT library applications obtain from the `spt_extensions.h` header file.

The `put/put_types.h` header file includes PUT Model library versions of the nonstandard definitions that applications that use the SPT library obtained from `spt_types.h`, with this difference:

The `spt_types.h` header file defines type `timespec_t` that refers to a nonstandard `struct timespec` data item whose first field has type `unsigned long`. The `put/put_types.h` header file does not define this type. Instead, the standard header file `time.h` defines type `timespec_t`, and specifies it as referring to a standard `struct timespec` data item whose first field has type `time_t` (which is equivalent to type `signed long`).

The PUT Model library does not implement a nonstandard header file that corresponds to `spt_ucontext.h` because the PUT Model library provides standard function behavior that supersedes the nonstandard function behavior provided by the SPT library.

Standard and Nonstandard Pthread Types

Some definitions in the `spthread.h` header files refer to nonstandard pthread types. To comply to the IEEE Standard (1003.1, 2004 edition) the definitions in `pthread.h` refer to standard POSIX pthread types (as defined in `sys/types.h`) only.

If an application module calls a pthread function whose return value or parameters had a nonstandard type and you want to conform to the standard, logic and data item definitions must be revised to use the corresponding standard types. If you want to continue to use the nonstandard types, they are defined in the PUT Model library header `put/put_types.h`.

Table 76: SPT Library Nonstandard Types and Corresponding PUT Model Library Standard Types

SPT Library Nonstandard Type	Corresponding PUT Model Library Standard Type
<code>pthread_address_t</code>	<code>void *</code>
<code>pthread_addr_t</code>	<code>void *</code>
<code>pthread_integer_t</code>	<code>int</code>
<code>pthread_startroutine_t</code>	<code>void * (*) (void *)</code>

Table Continued



SPT Library Nonstandard Type	Corresponding PUT Model Library Standard Type
<code>pthread_initroutine_t</code>	<code>void (*) (void)</code>
<code>pthread_destructor_t</code>	<code>void (*) (void *)</code>
<code>pthread_void_fn_ptr_t</code>	<code>void (*) (void)</code>

Global Variables

In contrast to the SPT library, the PUT Model library does not provide these global variables because these global variables are not specified in the IEEE Standard (1003.1, 2004 edition):

```
pthread_attr_default
pthread_mutexattr_default
pthread_condattr_default
```

Concurrency Level Defaults

The PUT Model library has a default concurrency level of 20. The SPT library has a default concurrency level of 0 (zero).

Differences in Functions and Function Behavior

Nonblocking Function Equivalents to `spt_*x` and `spt_*z` Use Standard Names

The PUT model library does not provide analogs to `spt_*()` functions, such as `spt_readz()` or `spt_readx()`, whose purpose is to be non-blocking alternatives to standard system functions. If an application module contains calls to such a function, you must modify it to refer to the standard function name, such as `read()` and you must include the appropriate standard header file. For more information about these functions, see [Thread-Aware and Nonblocking OSS Functions](#).

Nonstandard `spt_*` Functions Have `put_*` Equivalents

The SPT library provides some non-standard `spt_*()` and `SPT_` functions, such as `spt_select_single_np()` and `SPT_INITRECEIVE()`. The PUT Model library provide analogs to these functions that you can call directly. If an application module contains calls to such functions, they must be replaced with calls to the equivalent `put_*()` or `PUT_*()` function (for example, `put_select_single_np()` and `PUT_INITRECEIVE()`). Such changes are straightforward because only the function name is different; there is no change to parameter types or the function semantics. Reference pages for all the `put_*()` and `PUT_*()` functions are included in either the *Open System Services System Calls Reference Manual* or the *Open System Services Library Calls Reference Manual*.

The `put/put_extensions.h` header file provides the prototypes for nonstandard functions. If the macro `_TANDEM_SOURCE` is defined, directly or indirectly, then when the compiler reads `pthread.h` it reads `put/put_extensions.h` as well. Otherwise you must explicitly include the `put/put_extensions.h` header file in your application.

Standard `pthread_*` functions replace some nonstandard `pthread_*` functions

The PUT Model library provides a few standard `pthread_*O` functions that replace the nonstandard versions provided by the SPT library:



Table 77: Nonstandard pthread_ functions replaced by standard versions

Nonstandard SPT library function	Standard PUT Model library function
pthread_mutexattr_getkind_np()	pthread_mutexattr_gettype()
pthread_mutexattr_setkind_np()	pthread_mutexattr_settype()
pthread_attr_getguardsize_np()	pthread_attr_getguardsize()
pthread_attr_setguardsize_np()	pthread_attr_setguardsize()

Header File Locations for Reentrant Functions

The SPT library provides reentrant functions that are thread-safe alternatives to standard functions whose semantics make their use by a threaded program unsafe. The SPT header file `spt_extensions.h` provides the prototypes for all of these functions.

The PUT Model library does not provide any of the reentrant functions. Instead, each such function is provided by the same public library and header file that provides the non-reentrant version of the function. For example, the `time.h` header file defines both the `asctime()` and `asctime_r()` functions. Consequently, each PUT Library module that refers to a reentrant function must include the appropriate header.

Implementations of getgrent_r() and getpwent_r() Functions

The implementations of functions `getgrent_r()` and `getpwent_r()` in the OSS subject API library (ZSECDLL) conform to the standard and each of these functions have four parameters. The corresponding SPT functions have only the first three parameters. Consequently the logic of an application module that calls the `getgrent_r()` or `getpwent_r()` function requires revision. For more information about these functions, see the reference page for the function either online or in the *Open System Services Library Calls Reference Manual*.

Different Behavior for C Language I/O Functions

The SPT library intercepts calls to C language I/O functions (such as `fgetc()`, `fprintf()`, and so forth) made by SPT application modules and, because it processes certain outcomes internally, never reports that an operation failed because of a situation in which `errno` would be set to `EWOULDBLOCK` (or to values describing a few other similar situations).

Because the logic of the PUT Model library is different, calls to C language I/O functions can complete with `errno` set to values that report that an operation failed but that the operation might succeed if retried. Consequently, the logic of an application module that calls a C language I/O function might need to be revised if the input-output resource accessed by the applicable `FILE *` can terminate with `errno` set to one of these values. For information about the behaviors and possible `errno` values for a function, see the reference page for that function either online or in the *Open System Services Library Calls Reference Manual*.

The SPT library functions implement C language I/O behavior by calling the corresponding C run-time library (RTL) I/O functions. Those C RTL functions perform the actual I/O operations by calling process-blocking file system functions (for example, `fread()` calls `read()` to obtain data from the file). In contrast, when applications use the PUT Model library, the C RTL I/O function services calls non-blocking thread-library functions provided by the PUT Model library. This difference can invalidate application logic that depends upon the behavior of library logic.

For example, assume a client provides a logging facility that, to guarantee the log data written to the file is contiguous (that is, no other thread can intersperse any data), depends upon output operations being executed without the possibility of a thread switch:

- Calling `fprintf()` to output trace data to the log file is safe for an application that uses the SPT library but not for an application that uses the PUT Model library. An application that uses the PUT Model library `fprintf()` logic

could call a PUT Model library version of a write function that initiates a non-blocking output operation and then allows a different thread to become active.

- To maintain the SPT library behavior, the application logic must change. For example, you could replace the `fprintf()` call with a call to `sprintf()` followed by a call to `write()`. Note that even this example works only if the logging logic is in an application module compiled with macro `_PUT_MODEL_` undefined. (Compiling the module with macro `_PUT_MODEL_` defined would translate the textual reference to system function `write()` into code that calls the PUT non-blocking write function. The behavior would then be equivalent to that of `fprintf()`.)

An application module could call the SPT variant of a C language I/O function directly; for example, `spt_fread()`. Such calls must be replaced with calls to the base function; for example, `fread()`. The functions of this kind are:

```
fclose()
fflush()
fgetc()
fgets()
fgetwc()
fprintf()
fputc()
fputs()
fputwc()
fread()
fwrite()
getc()
getchar()
gets()
getw()
getwc()
getwchar()
printf()
putc()
putchar()
puts()
putw()
putwc()
putwchar()
vfprintf()
vprintf()
```

Thread-Aware Signal Handling

An application that uses the SPT library can either enable thread-aware signal handling or leave this behavior disabled. For an application that uses the PUT Model library, thread-aware signal handling is always enabled. Consequently, if an application that uses the SPT library is not prepared to handle signals, converting it to an application that uses the PUT Model library might require additional logic to handle thread-specific signals.

Use of Process Resources

There are many differences in the internal logic of the public libraries that provide run-time services. The resultant different (potentially dramatically different) patterns of usage of process resources can have an impact ranging from minor to severe. In particular, implementations of various functions in public libraries can consume larger amounts of memory when called from an application that uses the PUT Model library. For example, the function `getservent()` returns the address of a `struct servent` whose content is managed by the INET public library. For an application that uses the SPT library, every call modifies and returns the address of the same data space. For application that uses the

PUT Model library, the library allocates and manipulates a separate `struct servent` data space for each thread. The cumulative impact of all increases in memory allocated could affect the performance of, or in a severe case lead to the failure of, a successful application that was converted from using the SPT library to using the PUT Model library.

Threaded Application Programming

Keeping Application Design Simple

Because of the inherent complexity of multithreaded programming, application designers should follow these recommendations:

- Write programs with a single application thread when possible.
- Keep multiple application threads independent of each other when possible. If this is not possible, keep the interdependencies to a minimum.

Threading Considerations

Application designers should be aware of the following general threading considerations when developing a the PUT Model library application:

- **Sharing Process Resources**
- **Using Standard Libraries**
- **Addressing the Thread Stack**
- **Making Process-Blocking Calls**
- **Using Nowait Input or Output**
- **Yielding Control**
- **Using Signals**
- **Spawning a New Process**
- **Isolating Faults**

Sharing Process Resources

Access by threads to shared process resources such as global data, the heap, and files must be synchronized using mutexes and condition variables. **Table 67: Process Resources** lists global and private process resources.

Using Standard Libraries

No Need for Jacket Routines or Special Header Files

On systems running H06.21 or later H-series RVUs or J06.10 or later J-series RVUS, standard library functions do not require jacket routines, special header files, or other modifications to make them thread-safe or reentrant.

Jacket Routines for Functions Related to Pathway or TMF

The PUT Model library does provide jacket routines for calls to Pathway servers (`SERVERCLASS_SEND` and `SERVERCLASS_SEND_INFO`) and for TMF transactions, as described in **Jacket Routines (Nonblocking Versions of `SERVERCLASS_` and TMF System Calls)**.



Asynchronous Cancels are Possible

The PUT Model library supports the asynchronous receipt of cancel requests; a thread can exit from a standard library due to cancellation (see **Thread Safety**). Therefore, if you want to prevent cancellation, you must disable cancellation before making calls to library functions that contain cancellation points or potential cancellation points.

Beware of Pointers to Static Library Data

Functions that are not thread-aware might return pointers to static data. In this case, one thread could invalidate another thread's pointer. For example, in an application that uses the PUT Model library, the `getenv()` function returns a pointer to environmental information contained in a static variable in the library. The content of this variable can be changed by any thread.

When using functions that return pointers to static library data, immediately copy the data to thread-private memory.

Most of the standard library functions follow the POSIX standard to protect thread safety when PUT library is used. For more information about thread safety, see **Thread Safety**.

Beware of Libraries That Maintain Context Across Calls

Most of the standard library functions maintain thread-specific data and context when you use the PUT Model library. For more information about thread safety, see **Thread Safety**.

For non-thread-safe standard library functions, you can use a mutex to protect implicit context maintained by a standard library, for example by bracketing a series of library calls with the lock and unlock of a mutex. This technique should be used only for short periods of time.

Standard libraries that use an explicit handle to identify the intended context on each call can be used without interference between threads. File operations that use file descriptors follow this model.

Some libraries allow multiple contexts but do not support explicit context identification on all calls. Instead, the library allows the caller to establish a current context for the process. With these libraries, the application is responsible for ensuring that the correct context is made current while a thread is executing. For example, the TMF transaction-demarcation jacket routines use this method, relying on an implicit current transaction for the process.

Addressing the Thread Stack

Memory for function variables that are not declared static is allocated from the thread-private stack. This memory is addressable only while the thread is executing. It is not addressable by other threads; attempting to read or write variables located in another thread's stack causes unpredictable results.

All program data that is to be shared by multiple threads must be external data, static internal data, or data allocated from the heap. This data should be protected by a mutex, as appropriate.

The Standard provides a set of functions to manage thread-specific data. One use of these functions is to maintain thread global data, such as state. The interface allows you to declare destructor functions that are invoked at thread run-down. These functions are distinct from cancellation clean-up handlers.

The PUT Model library uses a protected stack for signals (see **Signals and Signal Handling**).

Making Process-Blocking Calls

When a thread calls a Guardian procedure that blocks the process, all other threads in the process are also prevented from executing. If a thread is blocked for a significant period of time, it can delay or even prevent the execution of time-critical run-time functions.

Examples of programs that can block include:

- Pathway servers, which can wait indefinitely while reading \$RECEIVE
- The Inspect, Native Inspect, and Visual Inspect symbolic debuggers, whose use can block a process indefinitely



Using Nowait Input or Output

Programs that use threads cannot call the Guardian `AWAITIO` or `AWAITIOX` procedure with `-1` as a file parameter (applying the call to the oldest incomplete operation pending on each file). `AWAITIOX` can be used to poll for completion on a specific file, although a thread must complete all nowait input or output operations before yielding control to another thread. In summary:

- No calls to `AWAITIO[X]()` with a file number of `-1` are made outside of pthreads.
- No calls to `AWAITIO[X]()` with a file number registered with pthreads are made outside of pthreads.

Yielding Control

Because the PUT Model library are never preempted, a call to the `sched_yield()` function might be needed to allow time-critical run-time threads to execute. This function releases the processor to another thread of the same or higher priority. The calling thread may get control immediately if no threads of the same or higher priority are ready to run.

Be careful using the `sched_yield()` function. Misuse causes unnecessary context switching, which increases overhead without allowing the other threads to execute. For example, it is counterproductive for a thread to yield control while it has a needed resource locked.

Spawning a New Process

Using the `fork()` or `tdm_fork()` function in a multithreaded environment can cause problems.

Applications can use either of these functions safely as follows: A parent process that has already called a function can also use either of the fork functions to create a child process if the child process calls any of the `exec` functions, the `tdm_execve()`, or `tdm_execep()` function immediately upon its creation.

A threaded application might alternatively use `pthread_atfork()` to declare as many as three fork handlers: a prepare handler called before the fork, a parent handler called after the fork in the parent process, and a child handler called after the fork in the child process.

When a fork is called, only the calling thread is duplicated in the child process. The expected usage of `pthread_atfork()` is for the prepare handler to acquire all mutex locks of interest to the child, and the other two fork handlers to release them.

Beginning with the H06.24 and J06.13 RVUs, 64-bit OSS processes are supported. See [64-Bit OSS Processes](#) for details of creating 64-bit OSS processes.

Isolating Faults

Because all threads created by a process share the process address space, there is no address space protection between the application threads in a process. One application thread can corrupt data, causing other threads or the entire process to fail.

Single Versus Multiple Application Threads

OSS threaded applications can be classified as follows, in order of increasing complexity:

- **Programs With a Single Application Thread**
- **Programs With Multiple Independent Application Threads**
- **Programs With Multiple Interdependent Application Threads**



Programs With a Single Application Thread

Externally, programs with a single application thread behave like standard, sequential (nonthreaded) programs. The program begins execution in the `main()` procedure; it then calls other procedures that, in turn, can call still other procedures. This type of application includes simple OSS client programs.

In single-threaded applications:

- Process resources can be used freely.

No synchronization is needed. The application can freely use process resources such as process and module globals, the heap, file descriptors, SQL cursors, and the signal mask. Mutexes and condition variables are not needed.

- Standard libraries can be used freely.

The application can share access to library data, including implicit context maintained across multiple library calls. For example, the application thread can use the TMF transaction jacket routines and no other thread can change the current transaction of the process.

- One application thread cannot block another from executing. Process-blocking operations have minimal effect.
- One application thread cannot corrupt another.

The thread does not share its address space with any other application threads. The application thread is protected by the address-space protection provided for the process, offering fault isolation.

The following general threading considerations affect single-threaded applications but have less effect than for processes with multiple application threads:

- Using process-blocking calls

Although process-blocking calls will not affect other will not affect other application threads, they can interfere with time-critical threads.

- Isolating faults and yielding control

If the application thread can execute for long periods (more than a second) without relinquishing control, the application might still need to use the `sched_yield()` function to allow time-critical threads to execute.

These general threading considerations apply to single-threaded applications:

- Using `nowait` input or output as described in **Using Nowait Input or Output**.
- Spawning a process as discussed in **Spawning a New Process**.

Programs With Multiple Independent Application Threads

Programs containing multiple threads can improve performance by providing concurrent execution. An application thread is independent if it does not explicitly share application resources that exist in the process environment, such as application file opens, SQL cursors, or modifiable application data structures.

This type of application includes simple OSS servers that maintain the application state in a database. Such a server simplifies server design because then application state does not need to be maintained in the server and thus most servers can be context-free. In cases where context must be maintained across multiple requests, the context is generally private to the client and should not be shared.

Threading Considerations describes some considerations for developing an application with multiple application threads.



Programs With Multiple Interdependent Application Threads

An application thread is interdependent if it shares application resources that exist in the process environment, such as application file opens, SQL cursors, and modifiable application data structures. Applications with multiple interdependent application threads must synchronize access to the shared application resources.

This type of application includes OSS servers that maintain application state in the server to improve performance or that share resources, such as file opens, to reduce resource consumption. This type of application also includes client programs that use parallelism.

Programs with multiple interdependent application threads can be synchronized by using mutexes to reserve access to a shared resource and by using condition variables to wait for a shared resource to be in a certain state.

Poor performance can result if inappropriate synchronization techniques are used. Synchronizing at too coarse a level can overly restrict concurrency and cause a server to serialize processing, dramatically increasing the application's service time. Synchronizing at too fine a level can result in an increase in the processor cycles spent on initializing, obtaining, and releasing mutexes and dispatching threads.

Threaded Application Examples

The following example application creates a threaded time-of-day server socket that listens for connections and upon connection, creates a thread to handle the connection.

Threaded Time-of-Day Server Socket

```
/* Threaded time-of day server socket. */
/* If compiled for TNS/E, specify the ZPUTDLL, */
/* If compiled for TNS/E, specify the XPUTDLL, */
/* the standard POSIX threads shared run-time library. */

#define _PUT_MODEL_
#define _XOPEN_SOURCE_EXTENDED 1
#include <assert.h>
#include <fcntl.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <unistd.h>
#include <strings.h>
#include <stdlib.h>
#include <string.h>

/*
Defines thread-aware functions calls to the threads package:
*/

#include <pthread.h>

/*
Defines a short macro to convert file descriptors to non-blocking
(thread-aware functions are thread-aware only if the file descriptor
is non-blocking):
*/

#define fd_make_nonblock(fd) \
    assert(fcntl(fd, F_SETFL, fcntl(fd, F_GETFL) | O_NONBLOCK) != -1)

/*
```



```

Defines the port the server will run on:
*/

#define SERVER_PORT 3000

/*
Shows the worker thread that is created after each accept:
*/

void *worker_thread(void* sock_fd)
{

    char buffer[1024];
    time_t now;

        time(&now);
        strcpy(buffer, ctime(&now));
        send(*(int *)sock_fd, &buffer, strlen(buffer), 0);
        close(*(int *)sock_fd);
        return NULL;
}

int main(int argc, char *argv[])

{
int inet_addr_port = SERVER_PORT;
int sock=-1, accept_fd=-1;
size_t size;
struct sockaddr_in servaddr, accept_addr;

/*
Defines one pthread_t per request we intend to handle:
*/

pthread_t my_thread;
pthread_attr_t my_attr;

    /*
    Turns on concurrency:
    */

    if(pthread_setconcurrency(1))
        {
            perror("pthread_setconcurrency");
            exit(-1);
        }

    /*
    Initializes attribute:
    */

    if(pthread_attr_init(&my_attr))
        {
            perror("pthread_attr_init");
            exit(-1);
        }

```



```

/*
Sets detach Attribute:
*/

if(pthread_attr_setdetachstate(&my_attr, PTHREAD_CREATE_DETACHED))
{
    perror("pthread_attr_setdetachstate");
    exit(-1);
}

/*
Creates socket:
*/

if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
{
    perror("Socket Error");
    exit(-1);
}

/*
Sets up Bind:
*/

bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(inet_addr_port);

if (bind(sock, (struct sockaddr *) &servaddr, sizeof(servaddr)) < 0)
{
    perror("Bind Error");
    exit(-1);
}

fd_make_nonblock(sock);

/*
Sets socket to listen:
*/

if (listen(sock, 1) < 0)
{
    perror("Listen Error");
    exit(-1);
}

/*
Starts server:
*/

while(1)
{
    bzero(&accept_addr, sizeof(accept_addr));
    accept_fd = accept(sock, NULL, NULL);
}

```

```

        if (accept_fd < 0)
        {
            perror("Accept Error");
            exit(-1);
        }

        /*
        Dispatches a thread to handle service request:
        */

        if(pthread_create(&my_thread,&my_attr,&worker_thread,&accept_fd))
        {
            perror("pthread_create");
            exit(-1);
        }
    } /* while */
}/* main */

```

Threaded Time-of-Day Server Socket Using \$RECEIVE also creates a threaded time-of-day server socket, but using \$RECEIVE.

Threaded Time-of-Day Server Socket Using \$RECEIVE

```

#define _PUT_MODEL_
#include <errno.h>
#include <stdlib.h>
#include <assert.h>
#include <cextdecs.h(FILE_OPEN_, READX, FILE_CLOSE_, WRITEX, FILE_GETINFO_)>
#include <cextdecs.h(FILE_GETRECEIVEINFO_)>
#include <cextdecs.h(REPLYX)>
#include <cextdecs.h(READUPDATEX)>
#include "myio.h" /** contains my_printf which is process-blocking**/

/*
Must come after cextdecs.h:
*/
#include <pthread.h>

void * server_thread(void* target)
{
    char buffer[1024];
    time_t now;
    int error;

    time(&now);
    strcpy(buffer, ctime(&now));

    /*
    A compiler warning will occur regarding the next line, which is OK:
    */

    error = put_REPLYX(buffer, (short) strlen(buffer), NULL,
        (short)target, 0);

    my_fprintf(stderr,"server_thread: put_REPLYX: error: %d\n",error);
    fflush(stderr);
    return NULL;
}

```

```

}

int main(void)
{
    pthread_t my_thread;
    pthread_attr_t my_attr;
    char *filename = "$RECEIVE";
    char buffer[1024];
    long bytesread, error;
    short receive_info[17];
    short dialog_info;
    short target;
    short filenum;

    /*
    Sets concurency level:
    */

    if(pthread_setconcurrency(1))
    {
        perror("pthread_setconcurrency");
        exit(-1);
    }

    /*
    Initializes attribute:
    */

    if(pthread_attr_init(&my_attr))
    {
        perror("pthread_attr_init");
        exit(-1);
    }

    /*
    Sets detach state for thread attribute:
    */

    if(pthread_attr_setdetachstate(&my_attr, PTHREAD_CREATE_DETACHED) != 0)
    {
        perror("pthread_attr_setdetachstate");
        exit(-1);
    }

    error = FILE_OPEN_(filename, (short)strlen(filename), &filenum,
        0, , 1, 10, 1);
    my_fprintf(stderr, "main: FILE_OPEN_: error: %d\n", error);
    assert(error == 0);
    error = put_INITRECEIVE(filenum, 10);
    my_fprintf(stderr, "main: put_INITRECEIVE: error: %d\n", error);
    fflush(stderr);
    assert(error == 0);

    while (1)

```

```

{
    error = put_RECEIVEREAD(filenum, buffer, (short)sizeof(buffer),
        &bytesread, -1, receive_info, &dialog_info);
    my_fprintf(stderr, "main_thread: put_RECEIVEREAD: error: %d\n", error);

    target = receive_info[2];

    if(pthread_create(&my_thread, &my_attr, &server_thread,
        (void *)target))
    {
        perror("pthread_create");
        exit(-1);
    }

    } /* while */
} /* main */

```

64-Bit Considerations for the PUT Model Library

Beginning with the H06.24 and J06.13 RVUs, the following OSS functions have been added:

- PUT_FILE_WRITEREAD_O
- PUT_BEGINTRANSACTION_EXT_O
- PUT_SERVERCLASS_DIALOG_BEGINL_O
- PUT_SERVERCLASS_DIALOG_SENDDL_O

See the *Open System Services System Calls Reference Manual* for details of these new OSS functions.



IMPORTANT:

The POSIX User Thread (PUT) model library is the only 64-bit thread model supported.

Thread Local Storage Usage

Starting with L20.10 RVU, PUT Model Library supports Thread Local Storage (TLS). TLS is supported only in C++11 programs. For compilation procedure, see THREAD and TLS section in *C/C++ Programmer's Guide for NonStop Systems*.

The following limitations might apply while designing a TLS application and linking it to PUT Model Library:

- If the thread-local variable is accessed inside a signal handler, I/O handler, timer handler, or any other callback function, the application might show unexpected behavior in some rare situations, for example, a signal being raised when all threads are blocked for I/O.
- Since the memory for thread-local variable is allocated from the heap, there might be a shortage of memory in the application.



Websites

General websites

Hewlett Packard Enterprise Support Center

<https://www.hpe.com/support/hpesc>

Contact Hewlett Packard Enterprise Worldwide

<https://www.hpe.com/assistance>

Subscription Service/Support Alerts

<https://www.hpe.com/support/e-updates>

Manuals for L-series

<https://www.hpe.com/info/nonstop-ldocs>

Manuals for J-series

<https://www.hpe.com/info/nonstop-jdocs>

For additional websites, see **[Support and other resources](#)**.

Support and other resources

Accessing Hewlett Packard Enterprise Support

- For live assistance, go to the Contact Hewlett Packard Enterprise Worldwide website:
<https://www.hpe.com/info/assistance>
- To access documentation and support services, go to the Hewlett Packard Enterprise Support Center website:
<https://www.hpe.com/support/hpesc>

Information to collect

- Technical support registration number (if applicable)
- Product name, model or version, and serial number
- Operating system name and version
- Firmware version
- Error messages
- Product-specific reports and logs
- Add-on products or components
- Third-party products or components

Accessing updates

- Some software products provide a mechanism for accessing software updates through the product interface. Review your product documentation to identify the recommended software update method.
- To download product updates:

Hewlett Packard Enterprise Support Center

<https://www.hpe.com/support/hpesc>

Hewlett Packard Enterprise Support Center: Software downloads

<https://www.hpe.com/support/downloads>

My HPE Software Center

<https://www.hpe.com/software/hpesoftwarecenter>

- To subscribe to eNewsletters and alerts:
<https://www.hpe.com/support/e-updates>
- To view and update your entitlements, and to link your contracts and warranties with your profile, go to the Hewlett Packard Enterprise Support Center **More Information on Access to Support Materials** page:
<https://www.hpe.com/support/AccessToSupportMaterials>





IMPORTANT: Access to some updates might require product entitlement when accessed through the Hewlett Packard Enterprise Support Center. You must have an HPE Onepass set up with relevant entitlements.

Remote support

Remote support is available with supported devices as part of your warranty or contractual support agreement. It provides intelligent event diagnosis, and automatic, secure submission of hardware event notifications to Hewlett Packard Enterprise, which initiates a fast and accurate resolution based on the service level of your product. Hewlett Packard Enterprise strongly recommends that you register your device for remote support.

If your product includes additional remote support details, use search to locate that information.

HPE Get Connected

<https://www.hpe.com/services/getconnected>

HPE Pointnext Tech Care

<https://www.hpe.com/services/techcare>

HPE Complete Care

<https://www.hpe.com/services/complecare>

Customer self repair

Hewlett Packard Enterprise customer self repair (CSR) programs allow you to repair your product. If a CSR part needs to be replaced, it will be shipped directly to you so that you can install it at your convenience. Some parts do not qualify for CSR. Your Hewlett Packard Enterprise authorized service provider will determine whether a repair can be accomplished by CSR.

For more information about CSR, contact your local service provider.

Warranty information

To view the warranty information for your product, see the links provided below:

HPE ProLiant and IA-32 Servers and Options

<https://www.hpe.com/support/ProLiantServers-Warranties>

HPE Enterprise and Cloudline Servers

<https://www.hpe.com/support/EnterpriseServers-Warranties>

HPE Storage Products

<https://www.hpe.com/support/Storage-Warranties>

HPE Networking Products

<https://www.hpe.com/support/Networking-Warranties>

Regulatory information

To view the regulatory information for your product, view the *Safety and Compliance Information for Server, Storage, Power, Networking, and Rack Products*, available at the Hewlett Packard Enterprise Support Center:

<https://www.hpe.com/support/Safety-Compliance-EnterpriseProducts>



Additional regulatory information

Hewlett Packard Enterprise is committed to providing our customers with information about the chemical substances in our products as needed to comply with legal requirements such as REACH (Regulation EC No 1907/2006 of the European Parliament and the Council). A chemical information report for this product can be found at:

<https://www.hpe.com/info/reach>

For Hewlett Packard Enterprise product environmental and safety information and compliance data, including RoHS and REACH, see:

<https://www.hpe.com/info/ecodata>

For Hewlett Packard Enterprise environmental information, including company programs, product recycling, and energy efficiency, see:

<https://www.hpe.com/info/environment>

Documentation feedback

Hewlett Packard Enterprise is committed to providing documentation that meets your needs. To help us improve the documentation, use the **Feedback** button and icons (located at the bottom of an opened document) on the Hewlett Packard Enterprise Support Center portal (**<https://www.hpe.com/support/hpesc>**) to send any errors, suggestions, or comments. All document information is captured by the process.



Documented OSS Functions

This appendix lists the following information for each OSS function.

Function	The name of the OSS function.	
Reference	Where the function is documented:	
	Library	The hard-copy reference page for the function is in the <i>Open System Services Library Calls Reference Manual</i> .
	System	The hard-copy reference page for the function is in the <i>Open System Services System Calls Reference Manual</i> .
HPE	Information about the Hewlett Packard Enterprise implementation of the function:	
	Yes	The function is either a Hewlett Packard Enterprise extension or the Hewlett Packard Enterprise implementation varies from the POSIX standard or XPG4 specification.
	Extended	The function can also return extended errors.
Native Only	TNS/R, TNS/E, TNS/X	The function is restricted to the TNS/R, TNS/E, and TNS/X native environments.
	TNS/E	The function is restricted to the TNS/E native environment. It is not supported in the TNS/R native environment.
	TNS/X	The function is restricted to the TNS/X native environment. It is not supported in the TNS/E or TNS/R native environments.
RVU Notes	H06.nn	Support for this function was added after the H06.03 RVU, beginning with the RVU listed. If there is no G-series RVU listed, this function is not supported on G-series RVUs. If there is no J-series RVU listed, this function is not supported on J-series RVUs.
	G06.nn	Support for this function was added after the G06.25 RVU, beginning with the RVU listed. If there is no H-series RVU listed, this function is not supported on H-series RVUs. If there is no J-series RVU listed, this function is not supported on J-series RVUs.
	J06.nn	Support for this function was added after the J06.03 RVU, beginning with the RVU listed. If there is no H-series RVU listed, this function is not supported on H-series RVUs. If there is no G-series RVU listed, this function is not supported on G-series RVUs.

Table Continued



L15.nn

Support for this function for L-series RVUs began with the L15.02 RVU. If there is no H-series RVU listed, this function is not supported on H-series RVUs. If there is no G-series RVU listed, this function is not supported on G-series RVUs. If there is no J-series RVU listed, this function is not supported on J-series RVUs.

[blank]

Support for this function for H-series RVUs began with the H06.03 RVU. Support for this function for G-series RVUs began with or before the G06.25 RVU. Support for this function on J-series RVUs began with the J06.03 RVU.

Function	Reference	HPE	Native Only	RVU Notes
a64l ()	Library		TNS/R, TNS/E, TNS/X	
abort ()	Library			
_abs ()	Library			
abs ()	Library			
accept ()	System	Extended		
access ()	System	Yes		
acl ()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.08, J06.03, L15.02
aclsort ()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.08, J06.03, L15.02
acos ()	Library			
acosf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
acosh ()	Library	Yes	TNS/R, TNS/E, TNS/X	
acoshf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
acoshl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
acosl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
advance ()	Library		TNS/R, TNS/E, TNS/X	
alarm ()	Library	Yes		

Table Continued

Function	Reference	HPE	Native Only	RVU Notes
<code>_alloca()</code>	Library	Yes	TNS/E, TNS/X	H06.22, J06.11, L15.02
<code>asctime()</code>	Library			
<code>asctime_r()</code>	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
<code>asin()</code>	Library			
<code>asinf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>asinh()</code>	Library	Yes	TNS/R, TNS/E, TNS/X	
<code>asinhf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>asinh1()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>asinl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>assert()</code>	Library	Yes		
<code>atan()</code>	Library			
<code>atanf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>atanh()</code>	Library	Yes	TNS/R, TNS/E, TNS/X	
<code>atanhf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>atanh1()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>atanl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>atan2()</code>	Library			
<code>atan2f()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>atan2l()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>atexit()</code>	Library			
<code>atof()</code>	Library			
<code>atoi()</code>	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
atol ()	Library			
atoll ()	Library			
basename ()	Library		TNS/R, TNS/E, TNS/X	
bcmp ()	Library		TNS/R, TNS/E, TNS/X	
bcopy ()	Library		TNS/R, TNS/E, TNS/X	
bind ()	System	Extended		
bsearch ()	Library			
btowc ()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
bzero ()	Library		TNS/R, TNS/E, TNS/X	
cabs ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cabsf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cabsl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cacos ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cacosf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cacosh ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cacoshf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cacoshl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cacosl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
calloc ()	Library	Yes		
calloc32 ()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
carg ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cargf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
cargl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
casin()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
casinf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
casinh()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
casinhf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
casinhl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
casinl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
catan()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
catanf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
catanh()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
catanhf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
catanhl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
catanl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
catclose()	Library			
catgets()	Library			
catopen()	Library			
cbrt()	Library		TNS/R, TNS/E, TNS/X	
cbrtf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cbrtl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ccos()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ccosf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ccosh()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>ccoshf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>ccoshl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>ccosl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>ceil()</code>	Library	Yes		
<code>ceilf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>ceill()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cexp()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cexpf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cexpl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cfgetispeed()</code>	Library			
<code>cfgetospeed()</code>	Library			
<code>cfsetispeed()</code>	Library			
<code>cfsetospeed()</code>	Library			
<code>chdir()</code>	System	Extended		
<code>chmod()</code>	System	Extended		
<code>chown()</code>	System	Extended		
<code>chroot()</code>	System	Yes		
<code>chvol()</code>	Library			
<code>cimag()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cimagf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cimagl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>clearerr()</code>	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
clock()	Library	Yes		
clog()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
clogf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
clogl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
close()	System	Yes		
closedir()	Library	Extended		
closelog()	Library			
compile()	Library		TNS/R, TNS/E, TNS/X	
confstr()	Library			
conj()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
conjf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
conjl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
connect()	System	Extended		
copysign()	Library		TNS/R, TNS/E, TNS/X	
copysignf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
copysignl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cos()	Library			
cosf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cosh()	Library			
coshf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
coshl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
cosl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>cpow()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cpowf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cpowl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cproj()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cprojf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>cprojl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>creal()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>crealf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>creall()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>creat()</code>	System			
<code>creat64()</code>	System	Yes		G06.29, H06.06, J06.03
<code>crypt()</code>	Library	Yes	TNS/R, TNS/E, TNS/X	
<code>csin()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>csinf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>csinh()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>csinhf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>csinh1()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>csinl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>csqrt()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>csqrtf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>csqrtl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>ctan()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
ctanf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ctanh()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ctanhf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ctanh1()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ctanl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ctermid()	Library	Extended		
ctermid_r()	Library	Extended	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
ctime()	Library			
ctime_r()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
cuserid()	Library	Extended	TNS/R, TNS/E, TNS/X	
dbm_clearerr()	Library	Extended	TNS/R, TNS/E, TNS/X	
dbm_close()	Library	Extended	TNS/R, TNS/E, TNS/X	
dbm_delete()	Library	Extended	TNS/R, TNS/E, TNS/X	
dbm_error()	Library	Extended	TNS/R, TNS/E, TNS/X	
dbm_fetch()	Library	Extended	TNS/R, TNS/E, TNS/X	
dbm_firstkey()	Library	Extended	TNS/R, TNS/E, TNS/X	
dbm_nextkey()	Library	Extended	TNS/R, TNS/E, TNS/X	
dbm_open()	Library	Extended	TNS/R, TNS/E, TNS/X	
dbm_store()	Library	Extended	TNS/R, TNS/E, TNS/X	
dec_to_longlong()	Library	Yes		
difftime()	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
dirname()	Library	Yes	TNS/R, TNS/E, TNS/X	
dirfd()	Library	Yes	TNS/E, TNS/X	J06.22, L18.02
div()	Library			
dlclose()	Library		TNS/R, TNS/E, TNS/X	
dLError()	Library		TNS/R, TNS/E, TNS/X	
dlopen()	Library		TNS/R, TNS/E, TNS/X	
dlresultcode()	Library	Yes	TNS/R, TNS/E, TNS/X	
dlsym()	Library		TNS/R, TNS/E, TNS/X	
drand48()	Library			
dup()	System	Yes		
dup2()	System	Yes		
ecvt()	Library			
edfseek()	Library	Yes		
edftell()	Library	Yes		
encrypt()	Library	Yes	TNS/R, TNS/E, TNS/X	
endgrent()	Library	Yes	TNS/R, TNS/E, TNS/X	
endhostent()	Library	Yes	TNS/R, TNS/E, TNS/X	
endnetent()	Library	Yes	TNS/R, TNS/E, TNS/X	
endprotoent()	Library	Yes	TNS/R, TNS/E, TNS/X	
endpwent()	Library			
endservent()	Library	Yes	TNS/R, TNS/E, TNS/X	
erand48()	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>erf()</code>	Library		TNS/R, TNS/E, TNS/X	
<code>erfc()</code>	Library		TNS/R, TNS/E, TNS/X	
<code>erfcf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>erfc1()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>erff()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>erfl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>exec()</code>	System			
<code>execl()</code>	System	Extended		
<code>execle()</code>	System	Extended		
<code>execlp()</code>	System	Extended		
<code>execv()</code>	System	Extended		
<code>execve()</code>	System	Extended		
<code>execvp()</code>	System	Extended		
<code>_exit()</code>	System	Yes		
<code>_Exit()</code>	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
<code>exit()</code>	Library			
<code>exp()</code>	Library			
<code>expf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>expl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>expm1()</code>	Library	Extended	TNS/R, TNS/E, TNS/X	
<code>expm1f()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>expm1l()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
exp2 ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
exp2f ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
exp2l ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fabs ()	Library	Yes		
fabsf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fabsl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
faccessat ()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
fchdir ()	Library	Yes	TNS/E, TNS/X	J06.22, L18.02
fchmod ()	System	Extended		G06.28, H06.05, J06.03
fchmodat ()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
fchown ()	System	Extended		G06.28, H06.05, J06.03
fchownat ()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
fclose ()	Library	Extended		
fcntl ()	System	Extended		
fcvt ()	Library		TNS/R, TNS/E, TNS/X	
fdim ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fdimf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fdiml ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fdopen ()	Library			
feclearexcept ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fegetenv ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fegetexceptflag ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
fegetround()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
feholdexcept()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
feof()	Library			
feraiseexcept()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ferror()	Library			
fesetenv()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fesetexceptflag()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fesetround()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fetestexcept()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
feupdateenv()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fexecve()	System	Extended	TNS/X	L18.02, J06.23
fflush()	Library	Extended		
ffs()	Library		TNS/R, TNS/E, TNS/X	
fgetc()	Library	Extended		
fgetpos()	Library	Extended		
fgetpos64()	Library	Yes		G06.29, H06.06, J06.03
fgets()	Library	Extended		
fgetwc()	Library	Extended		
fgetws()	Library	Extended		
fileno()	Library			
finite()	Library	Yes	TNS/R, TNS/E, TNS/X	
flockfile()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>floor()</code>	Library			
<code>floorf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>floorl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fma()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fmaf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fmal()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fmax()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fmaxf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fmaxl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fmin()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fminf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fminl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fmod()</code>	Library	Yes		
<code>fmodf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fmodl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fmtmsg()</code>	Library		TNS/R, TNS/E, TNS/X	
<code>fnmatch()</code>	Library	Extended		
<code>fopen()</code>	Library			
<code>fopendir()</code>	Library	Extended	TNS/E, TNS/X	J06.22, L18.02
<code>fopen_guardian()</code>	Library	Yes		
<code>fopen_oss()</code>	Library	Yes		
<code>fopen_std_file()</code>	Library	Yes		

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>fopen64()</code>	Library	Yes		G06.29, H06.06, J06.03
<code>fopen64_guardian()</code>	Library	Yes		G06.29, H06.06, J06.03
<code>fopen64_oss()</code>	Library	Yes		G06.29, H06.06, J06.03
<code>fopen64_std_file()</code>	Library	Yes		G06.29, H06.06, J06.03
<code>fork()</code>	System	Extended		
<code>fp_class()</code>	Library	Yes	TNS/R, TNS/E, TNS/X	
<code>fpathconf()</code>	Library	Extended		
<code>fpclassify()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fprintf()</code>	Library	Yes		
<code>fputc()</code>	Library	Extended		
<code>fputs()</code>	Library	Extended		
<code>fputwc()</code>	Library	Extended		
<code>fputws()</code>	Library	Extended		
<code>fread()</code>	Library	Extended		
<code>free()</code>	Library			
<code>free32()</code>	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
<code>freeaddrinfo()</code>	Library	Yes		
<code>freehostent()</code>	Library	Yes		
<code>freopen()</code>	Library			
<code>freopen_guardian()</code>	Library	Yes		
<code>freopen_oss()</code>	Library	Yes		
<code>freopen64()</code>	Library	Yes		G06.29, H06.06, J06.03

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
freopen64_guardian()	Library	Yes		G06.29, H06.06, J06.03
freopen64_oss()	Library	Yes		G06.29, H06.06, J06.03
frexp()	Library			
frexpf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
frexpl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
fscanf()	Library	Yes		
fseek()	Library	Yes		
fseeko()	Library	Yes		G06.29, H06.06, J06.03
fseeko64()	Library	Yes		G06.29, H06.06, J06.03
fsetpos()	Library			
fsetpos64()	Library	Yes		G06.29, H06.06, J06.03
fstat()	System	Extended		
fstatat()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
fstat64()	System	Yes		G06.29, H06.06, J06.03
fstatat64()	System	Yes	TNS/E, TNS/X	J06.22, L18.02
fstatvfs()	System	Yes		
fstatvfs64()	System	Yes		
fsync()	System	Extended		
ftell()	Library	Extended		
ftello64()	Library	Yes		G06.29, H06.06, J06.03
ftime()	Library		TNS/R, TNS/E, TNS/X	
ftok()	Library	Extended		

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>ftruncate()</code>	System	Yes		
<code>ftruncate64()</code>	System	Yes		G06.29, H06.06, J06.03
<code>ftrylockfile()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>ftw()</code>	Library	Extended		
<code>ftw64()</code>	Library	Yes		G06.29, H06.06, J06.03
<code>funlockfile()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>fwide()</code>	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
<code>fwprintf()</code>	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
<code>fwrite()</code>	Library	Extended		
<code>fwscanf()</code>	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
<code>gai_strerror()</code>	Library	Yes		
<code>gamma()</code>	Library			
<code>gamma_r()</code>	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
<code>gcvt()</code>	Library		TNS/R, TNS/E, TNS/X	
<code>get_assign_msg()</code>	Library	Yes		
<code>get_assign_msg_by_name()</code>	Library	Yes		
<code>get_max_assign_msg_ordinal()</code>	Library	Yes		
<code>get_param_by_name()</code>	Library	Yes		
<code>get_param_msg()</code>	Library	Yes		
<code>get_startup_msg()</code>	Library	Yes		
<code>getaddrinfo()</code>	Library	Yes		
<code>getc()</code>	Library	Extended		

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
getc_unlocked()	Library	Extended	TNS/E, TNS/X	H06.21, J06.10, L15.02
getchar()	Library	Extended		
getchar_unlocked()	Library	Extended	TNS/E, TNS/X	H06.21, J06.10, L15.02
getcwd()	Library	Extended		
getdate()	Library		TNS/R, TNS/E, TNS/X	
getdtablesize()	Library		TNS/R, TNS/E, TNS/X	
getegid()	System			
getenv()	Library	Yes		
getenv_r()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
geteuid()	System			
getgid()	System			
getgrent()	Library	Extended	TNS/R, TNS/E, TNS/X	
getgrent_r()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getgrgid()	Library	Extended		
getgrgid_r()	Library	Extended	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getgrnam()	Library	Extended		
getgrnam_r()	Library	Extended	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getgroups()	System			
gethostbyaddr()	Library	Yes	TNS/R, TNS/E, TNS/X	
gethostbyaddr_r()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
gethostbyname ()	Library	Yes	TNS/R, TNS/E, TNS/X	
gethostbyname_r ()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
gethostbyname2 ()	Library	Yes	TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
gethostent ()	Library	Yes	TNS/R, TNS/E, TNS/X	
gethostent_r ()	Library	Yes	TNS/R, TNS/E, TNS/X	
gethostid ()	Library	Yes	TNS/R, TNS/E, TNS/X	
gethostname ()	System		TNS/R, TNS/E, TNS/X	
getipnodebyaddr ()	Library	Yes		
getipnodebyname ()	Library	Yes		
getitimer ()	Library	Yes	TNS/E, TNS/X	J06.20, L16.05
getlogin ()	Library	Extended		
getlogin_r ()	Library		TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getnameinfo ()	Library	Yes		
getnetbyaddr ()	Library	Yes	TNS/R, TNS/E, TNS/X	
getnetbyaddr_r ()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getnetbyname ()	Library	Yes	TNS/R, TNS/E, TNS/X	
getnetbyname_r ()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getnetent ()	Library	Yes	TNS/R, TNS/E, TNS/X	
getnetent_r ()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getopt ()	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
getpagesize()	Library		TNS/R, TNS/E, TNS/X	
getpass()	Library			
getpeername()	System	Extended	TNS/R, TNS/E, TNS/X	
getpgid()	System	Extended	TNS/R, TNS/E, TNS/X	
getpgrp()	System			
getpid()	System			
getppid()	System			
getpriority()	System	Extended	TNS/R, TNS/E, TNS/X	
getprotobyname()	Library	Yes	TNS/R, TNS/E, TNS/X	
getprotobyname_r()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getprotobynumber()	Library	Yes	TNS/R, TNS/E, TNS/X	
getprotobynumber_r()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getprotoent()	Library	Yes	TNS/R, TNS/E, TNS/X	
getprotoent_r()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getpwent()	Library	Yes		
getpwent_r()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getpwnam()	Library	Extended		
getpwnam_r()	Library	Extended	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getpwuid()	Library	Extended		
getpwuid_r()	Library	Extended	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
gets ()	Library	Extended		
getservbyname ()	Library	Yes	TNS/R, TNS/E, TNS/X	
getservbyname_r ()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getservbyport ()	Library	Yes	TNS/R, TNS/E, TNS/X	
getservbyport_r ()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getservent ()	Library	Yes	TNS/R, TNS/E, TNS/X	
getservent_r ()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
getsid ()	System	Extended		H06.07, G06.29
getsockname ()	System	Extended	TNS/R, TNS/E, TNS/X	
getsockopt ()	System	Extended	TNS/R, TNS/E, TNS/X	
getsubopt ()	Library		TNS/R, TNS/E, TNS/X	
gettimeofday ()	System	Yes		
getuid ()	System			
getw ()	Library	Extended		
getwc ()	Library	Extended		
getwchar ()	Library	Extended		
getwd ()	Library		TNS/R, TNS/E, TNS/X	
getws ()	Library	Extended		
gfileno ()	Library	Yes		
glob ()	Library	Extended		
globfree ()	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
gmtime()	Library			
gmtime_r()	Library		TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
HBA_CloseAdapter()	Library		TNS/E, TNS/X	
HBA_FreeLibrary()	Library		TNS/E, TNS/X	
HBA_GetAdapterAttributes()	Library		TNS/E, TNS/X	
HBA_GetAdapterName()	Library		TNS/E, TNS/X	
HBA_GetAdapterPortAttributes()	Library		TNS/E, TNS/X	
HBA_GetDiscoveredPortAttributes()	Library		TNS/E, TNS/X	
HBA_GetFcpTargetMapping()	Library		TNS/E, TNS/X	
HBA_GetNumberOfAdapters()	Library		TNS/E, TNS/X	
HBA_GetPortStatistics()	Library		TNS/E, TNS/X	
HBA_GetVersion()	Library		TNS/E, TNS/X	
HBA_LoadLibrary()	Library		TNS/E, TNS/X	
HBA_OpenAdapter()	Library		TNS/E, TNS/X	
HBA_RefreshInformation()	Library		TNS/E, TNS/X	
hcreate()	Library			
hdestroy()	Library			
heap_check()	Library	Yes		
heap_check32()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
heap_check_always()	Library	Yes		
heap_check_always32()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
heap_min_block_size()	Library	Yes		
hsearch()	Library			
htonl()	Library			
htons()	Library			
hypot()	Library			
hypotf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
hypotl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
iconv()	Library			
iconv_close()	Library			
iconv_open()	Library			
if_freenameindex()	Library			
if_indextoname()	Library			
if_nameindex()	Library			
if_nametoindex()	Library			
ilogb()	Library		TNS/R, TNS/E, TNS/X	
ilogbf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ilogbl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
imaxabs()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
imaxdiv()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
index()	Library		TNS/R, TNS/E, TNS/X	
inet_addr()	Library	Yes	TNS/R, TNS/E, TNS/X	
inet_lnaof()	Library		TNS/R, TNS/E, TNS/X	

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
inet_makeaddr()	Library		TNS/R, TNS/E, TNS/X	
inet_netof()	Library		TNS/R, TNS/E, TNS/X	
inet_network()	Library		TNS/R, TNS/E, TNS/X	
inet_ntoa()	Library		TNS/R, TNS/E, TNS/X	
inet_ntop()	Library	Yes		
inet_pton()	Library	Yes		
initgroups()	Library	Yes	TNS/R, TNS/E, TNS/X	H06.18, J06.07, L15.02
initstate()	Library		TNS/R, TNS/E, TNS/X	
insque()	Library		TNS/R, TNS/E, TNS/X	
ioctl()	System	Extended		
isalnum()	Library			
isalpha()	Library			
isascii()	Library			
isatty()	Library	Extended		
isblank()	Library		TNS/E, TNS/X	H06.08, L15.02
iscntrl()	Library			
isdigit()	Library			
isfinite()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
isgraph()	Library			
isgreater()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
isgreaterequal()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
isinf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
isless()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
islessequal()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
islessgreater()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
islower()	Library			
isnan() (function)	Library			
isnan() (macro)	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
isnormal()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
isprint()	Library			
ispunct()	Library			
isspace()	Library			
isunordered()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
isupper()	Library			
iswalnum()	Library			
iswalpha()	Library			
iswblank()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
iswcntrl()	Library			
iswctype()	Library			
iswdigit()	Library			
iswgraph()	Library			
iswlower()	Library			
iswprint()	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
iswpunct ()	Library			
iswspace ()	Library			
iswupper ()	Library			
iswxdigit ()	Library			
isxdigit ()	Library			
j0 ()	Library			
j1 ()	Library			
jn ()	Library			
jrand48 ()	Library			
kill ()	System	Extended		
l64a ()	Library		TNS/R, TNS/E, TNS/X	
labs ()	Library			
lchmod ()	System	Yes		G06.28, H06.05, J06.03
lchown ()	System	Extended		G06.28, H06.05, J06.03
lcong48 ()	Library			
ldexp ()	Library			
ldexpf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ldexpl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
ldiv ()	Library			
lfind ()	Library			
lgamma ()	Library			
lgammaf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
lgammal()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
lgamma_r()	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
link()	System	Extended		
linkat()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
listen()	System	Extended	TNS/R, TNS/E, TNS/X	
llabs()	Library		TNS/E, TNS/X	H06.03, J06.03, L15.02
lldiv()	Library			
llrint()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
llrintf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
llrintl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
llround()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
llroundf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
llroundl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
localeconv()	Library			
localtime()	Library			
localtime_r()	Library		TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
log()	Library			
logb()	Library	Extended	TNS/R, TNS/E, TNS/X	
logbf()	Library	Extended	TNS/E, TNS/X	H06.21, J06.10, L15.02
logbl()	Library	Extended	TNS/E, TNS/X	H06.21, J06.10, L15.02
logf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
logl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
loglp ()	Library	Extended	TNS/R, TNS/E, TNS/X	
loglpf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
loglpl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
log2 ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
log2f ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
log2l ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
log10 ()	Library			
log10f ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
log10l ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
longjump ()	Library			
longlong_to_dec ()	Library	Yes		
lrand48 ()	Library			
lrint ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
lrintf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
lrintl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
lround ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
lroundf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
lroundl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
lsearch ()	Library			
lseek ()	System	Extended		
lseek64 ()	System	Yes		G06.29, H06.06, J06.03

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
lstat()	System	Extended		
lstat64()	System	Yes		G06.29, H06.06, J06.03
lwres_freeaddrinfo()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_freehostent()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_gai_strerror()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_getaddrinfo()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_gethostbyaddr()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_gethostbyname()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_gethostbyname2()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_getipnodebyaddr()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_getipnodebyname()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_getnameinfo()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
lwres_hstrerror()	Library		TNS/R, TNS/E, TNS/X	H06.05, G06.27, L15.02
malloc()	Library	Yes		
malloc32()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
malloc_pages()	Library	Yes	TNS/E, TNS/X	
_max()	Library			
mblen()	Library			
mbrlen()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
mbrtowc()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
mbsinit()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
mbsrtowcs()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
mbstowcs ()	Library			
mbtowc ()	Library			
memccpy ()	Library			
memchr ()	Library			
memchr64 ()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
memcmp ()	Library			
memcmp64 ()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
memcpy ()	Library			
memcpy64 ()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
memmove ()	Library			
memmove64 ()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
memset ()	Library			
memset64 ()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
_min ()	Library			
mkdir ()	System	Extended		
mkdirat ()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
mkfifo ()	Library	Extended		
mkfifoat ()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
mknod ()	System	Extended		
mknodat ()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
mkstemp ()	Library		TNS/R, TNS/E, TNS/X	
mktemp ()	Library		TNS/R, TNS/E, TNS/X	

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
mktime ()	Library			
modf ()	Library			
modff ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
modfl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
rand48 ()	Library			
msgctl ()	System	Extended		
msgget ()	System	Extended		
msgrcv ()	System	Extended		
msgsnd ()	System	Extended		
Function	Reference	HPE	Native Only	RVU Notes
nan ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nanf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nanl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nearbyint ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nearbyintf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nearbyintl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nextafter ()	Library	Extended	TNS/R, TNS/E, TNS/X	
nextafterf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nextafterl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nexttoward ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nexttowardf ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
nexttowardl ()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
nftw()	Library	Extended		
nftw64()	Library	Extended		G06.29, H06.06, J06.03
nice()	System			
nl_langinfo()	Library			
nlist()	Library			
nrnd48()	Library			
ntohl()	Library			
ntohs()	Library			
open()	System	Extended		
open64()	System	Yes		G06.29, H06.06, J06.03
openat()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
openat64()	System	Yes	TNS/E, TNS/X	J06.22, L18.02
opendir()	Library	Extended		
openlog()	Library	Extended		
pathconf()	Library	Extended		
pause()	Library	Extended		
pclose()	Library	Extended		
perror()	Library			
pipe()	System	Extended		
popen()	Library	Extended		
pow()	Library			
powf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
powl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
pread()	System	Extended	TNS/X	L18.02, J06.23
pread64_()	System	Yes	TNS/X	L18.02, J06.23
printf()	Library	Yes		
psignal()	Library		TNS/X	L20.10
pthread_atfork()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_destroy()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_getdetach state	System	Yes	TNS/R, TNS/E, TNS/X	
()				
pthread_attr_getguards ize()	System		TNS/E, TNS/X	H06.21, J06.10, L15.02
pthread_attr_getguards ize_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_getinheri tsched()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_getschedp aram()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_getschedp olicy()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_getscope()	System		TNS/E, TNS/X	H06.21, J06.10, L15.02
pthread_attr_getstacka ddr()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_getstacks ize()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_init()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_setdetach state()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_setguards ize()	System		TNS/E, TNS/X	H06.21, J06.10, L15.02
pthread_attr_setguards ize_np()	System	Yes	TNS/R, TNS/E, TNS/X	

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
pthread_attr_setinheritsched()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_setschedparam()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_setschedpolicy()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_attr_setscope()	System		TNS/E, TNS/X	H06.21, J06.10, L15.02
pthread_attr_setstacksize()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cancel()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cleanup_pop()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cleanup_push()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cond_broadcast()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cond_destroy()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cond_init()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cond_signal()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cond_signal_int_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cond_timedwait()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_cond_wait()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_condattr_destroy()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_condattr_init()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_create()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_delay_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_detach()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_equal()	System	Yes	TNS/R, TNS/E, TNS/X	

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
pthread_exit()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_get_expiration_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_getattr_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_getconcurrency()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_getschedparam()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_getspecific()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_getthreadstate_info_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_join()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_key_create()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_key_delete()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_kill()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_kill_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_lock_global_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutex_destroy()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutex_init()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutex_lock()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutex_trylock()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutex_unlock()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutexattr_destroy()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutexattr_getkind_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutexattr_gettype()	System		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
pthread_mutexattr_init()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutexattr_setkind_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_mutexattr_settype()	System		TNS/E, TNS/X	H06.21, J06.10, L15.02
pthread_once()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_self()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_setcancelstate()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_setcanceltype()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_setconcurrency()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_setschedparam()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_setspecific()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_sigmask()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_signal_to_cancel_np()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_testcancel()	System	Yes	TNS/R, TNS/E, TNS/X	
pthread_unlock_global_np()	System	Yes	TNS/R, TNS/E, TNS/X	
PUT_ABORTTRANSACTION()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_awaitio()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_BEGINTRANSACTION()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_BEGINTRANSACTION_EXTENSION()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
PUT_CANCEL()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_CONTROL()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_ENDTRANSACTION()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
put_fd_read_ready()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_fd_write_ready()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_FILE_CLOSE_()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_FILE_OPEN_()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_FILE_WRITEREAD_()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
put_generateTag()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_getTMFConcurrentTransactions()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_INITRECEIVE()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_INITRECEIVEL()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_interrupt()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_interruptTag()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_LOCKFILE()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_LOCKREC()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_READLOCKX()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_READUPDATELOCKX()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_READUPDATEX()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_READX()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_RECEIVEREAD()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_RECEIVEREADL()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_regFile()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_regFileIOHandler()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_regOSSFileIOHandler()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
put_regPathsendFile()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_regPathsendTagHandler()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_regTimerHandler()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_REPLYX()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_REPLYXL()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_RESUMETRANSACTION()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_select_single_np()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_SERVERCLASS_DIALOG_ABORT_()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_SERVERCLASS_DIALOG_BEGIN_()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_SERVERCLASS_DIALOG_BEGINL_()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
PUT_SERVERCLASS_DIALOG_END_()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_SERVERCLASS_DIALOG_SEND_()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_SERVERCLASS_SEND_()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_SERVERCLASS_SENDL_()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
PUT_SERVERCLASS_SEND_INFO_()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_SERVERCLASS_SENDL_()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_SETMODE()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_setOSSFileIOHandler()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_setTMFConcurrentTransactions()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_TMF_GetTxHandle()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_TMF_Init()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
PUT_TMF_JOIN ()	System	Yes	TNS/X	L19.03
PUT_TMF_RESUME ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_TMF_SetAndValidateTxHandle ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_TMF_SetTxHandle ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_TMF_SUSPEND ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_UNLOCKFILE ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_UNLOCKREC ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_unregFile ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_unregOSSFileIOHandler ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_unregPathsendTagHandler ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
put_wakeup ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_WRITEREADX ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_WRITEUPDATEUNLOCKX ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_WRITEUPDATEX ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
PUT_WRITEX ()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
putc ()	Library	Extended		
putc_unlocked ()	Library	Extended	TNS/E, TNS/X	H06.21, J06.10, L15.02
putchar ()	Library	Extended		
putchar_unlocked ()	Library	Extended	TNS/E, TNS/X	H06.21, J06.10, L15.02
putenv ()	Library		TNS/R, TNS/E, TNS/X	
puts ()	Library	Extended		
putw ()	Library	Extended		

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
putwc()	Library	Extended		
putwchar()	Library	Extended		
putws()	Library	Extended		
qsort()	Library			
pwrite()	System	Extended	TNS/X	L18.02, J06.23
pwrite64_()	System	Yes	TNS/X	L18.02, J06.23
raise()	Library	Extended		
rand()	Library			
rand_r()	Library		TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
random()	Library			
re_comp()	Library		TNS/R, TNS/E, TNS/X	
re_exec()	Library			
read()	System	Extended		
read64_()	System	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
readdir()	Library	Yes		
readdir_dt()	Library	Yes		J06.20, L16.05
readdir_r()	Library		TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
readdir_r_dt()	Library		TNS/E, TNS/X	J06.20, L16.05
readdir64()	Library	Yes		G06.29, H06.06, J06.03
readdir64_dt()	Library	Yes		J06.20, L16.05
readdir64_r()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
readdir64_r_dt()	Library	Yes	TNS/E, TNS/X	J06.20, L16.05
readlink()	System	Extended		
readlinkat()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
readv()	System			
realloc()	Library	Yes		
realloc32()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
realpath()	Library		TNS/R, TNS/E, TNS/X	
recv()	System	Extended		
recv64_()	System	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
recvfrom()	System	Extended		
recvfrom64_()	System	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
recvmsg()	System	Extended		
recvmsg64_()	System	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
regcmp()	Library			
regcomp()	Library			
regerror()	Library			
regex()	Library			
regexexec()	Library			
regfree()	Library			
remainder()	Library	Extended	TNS/R, TNS/E, TNS/X	
remainderf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
remainderl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>remove()</code>	Library			
<code>remove_guardian()</code>	Library	Yes		
<code>remove_oss()</code>	Library	Yes		
<code>remque()</code>	Library		TNS/R, TNS/E, TNS/X	
<code>remquo()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>remquof()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>remquol()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>rename()</code>	System			
<code>renameat()</code>	System	Extended	TNS/E, TNS/X	J06.22, L18.02
<code>rename_guardian()</code>	System	Yes		
<code>rename_oss()</code>	System	Yes		
<code>rewind()</code>	Library			
<code>rewinddir()</code>	Library	Extended		
<code>rindex()</code>	Library			
<code>rint()</code>	Library		TNS/R, TNS/E, TNS/X	
<code>rintf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>rintl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>rmdir()</code>	System	Extended		
<code>round()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>roundf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>roundl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>scalb()</code>	Library	Extended	TNS/R, TNS/E, TNS/X	

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
scalbln()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
scalblnf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
scalblnl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
scalbn()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
scalbnf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
scalbnl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
scandir()	Library	Yes		G06.29, H06.06, J06.03
scandir64()	Library	Yes		G06.29, H06.06, J06.03
scanf()	Library	Yes		
sched_get_priority_max()	System	Yes		
sched_get_priority_min()	System	Yes		
sched_yield()	System	Yes	TNS/R, TNS/E, TNS/X	
seed48()	Library			
seekdir()	Library	Extended		
select()	System	Extended		
semctl()	System	Extended		
semget()	System	Extended		
semop()	System	Extended		
send()	System	Extended		
send64_()	System	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
sendmsg()	System	Extended		
sendmsg64_()	System	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
sendto()	System	Extended		
sendto64_()	System	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
setbuf()	Library			
setegid()	System		TNS/R, TNS/E, TNS/X	H06.18, J06.07, L15.02
setenv()	Library		TNS/X	L20.10
seteuid()	System		TNS/R, TNS/E, TNS/X	H06.18, J06.07, L15.02
setfilepriv()	System	Yes		H06.22, J06.11
setgid()	System	Yes		
setgrent()	Library	Extended	TNS/R, TNS/E, TNS/X	
setgroups()	System	Yes	TNS/R, TNS/E, TNS/X	H06.18, J06.07, L15.02
sethostent()	Library	Yes	TNS/R, TNS/E, TNS/X	
setitimer()	Library	Yes	TNS/E, TNS/X	J06.20, L16.05
setjmp()	Library			
setkey()	Library	Yes	TNS/R, TNS/E, TNS/X	
setlocale()	Library			
setlocale_from_msg()	Library	Yes		
setlogmask()	Library			
setnetent()	Library	Yes	TNS/R, TNS/E, TNS/X	
setpgid()	System	Extended		
setpgrp()	System	Extended		G06.29, H06.07, J06.03
setprotoent()	Library	Yes	TNS/R, TNS/E, TNS/X	
setpwent()	Library	Extended		

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
setregid()	System	Yes		G06.27, H06.05, J06.03
setreuid()	System	Yes		G06.27, H06.05, J06.03
setservernt()	Library	Yes	TNS/R, TNS/E, TNS/X	
setsid()	System	Yes		
setsockopt()	System	Extended		
setstate()	Library			
setuid()	System			
setvbuf()	Library			
shmat()	System	Extended		
shmctl()	System	Extended		
shmdt()	System	Extended		
shmget()	System	Extended		
shutdown()	System	Extended		
sigaction()	System	Extended	TNS/R, TNS/E, TNS/X	
sigaddset()	Library	Yes		
sigaltstack()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
sigdelset()	Library	Extended		
sigemptyset()	Library	Extended		
sigfillset()	Library	Extended		
sigismember()	Library	Extended		
siglongjmp()	Library	Yes		
signal()	Library	Extended		

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
signbit()	Library		TNS/E, TNS/X	
sigpending()	System	Extended		
sigprocmask()	System	Extended		
sigsetjmp()	Library			
sigsuspend()	System	Extended		
sigwait()	System	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
sin()	Library			
sinf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
sinh()	Library			
sinhf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
sinhl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
sinl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
sleep()				H06.21, J06.10, L15.02
socketatmark()	System	Yes		
socket()	System	Extended		
socket_transport_name_get()	System	Yes		
socket_transport_name_set()	System	Yes		
socketpair()	System	Extended		
snprintf()	Library	Yes		H06.03, J06.03
sprintf()	Library	Yes		
SPT_ABORTTRANSACTION()	Library	Yes	TNS/R, TNS/E, TNS/X	
spt_accept()	System	Yes	TNS/R, TNS/E, TNS/X	

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
spt_acceptx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_alarm()	System	Yes	TNS/R, TNS/E, TNS/X	G06.29, H06.06, J06.03, L15.02
spt_awaitio()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_BEGINTRANSACTION()	Library	Yes	TNS/R, TNS/E, TNS/X	
SPT_BEGINTRANSACTION_EXTENSION()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
SPT_CANCEL()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_close()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_closex()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_closez()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
spt_connect()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_connectx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
SPT_CONTROL()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02
SPT_dup2x()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02
SPT_ENDTRANSACTION()	Library	Yes	TNS/R, TNS/E, TNS/X	
spt_fclose()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fclosex()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fcntlxl()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fcntlzl()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
spt_fd_read_ready()	System	Yes	TNS/R, TNS/E, TNS/X	

Table Continued

Function	Reference	HPE	Native Only	RVU Notes
spt_fd_write_ready()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fflush()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fflushx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fgetc()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fgetcxc()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fgets()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fgetscx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fgetwc()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fgetwcxc()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
SPT_FILE_CLOSE_()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
SPT_FILE_OPEN_()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fork()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fprintf()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fprintfx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fputc()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fputcxc()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fputs()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fputscx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
spt_fputc()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fputcx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fread()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_freadx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_fstatz()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
spt_fstat64z()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
spt_fsyncz()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
spt_ftruncatez()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
spt_ftruncate64z()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
spt_fwrite()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_fwritex()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_generateTag()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_getc()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_getchar()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_getcharx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_getcx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_gets()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_getsx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_getTMFConcurrentTransactions()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_getw()	System	Yes	TNS/R, TNS/E, TNS/X	

Table Continued

Function	Reference	HPE	Native Only	RVU Notes
spt_getwc()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_getwchar()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_getwcharx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_getwcx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_getwx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_INITRECEIVE()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_INITRECEIVEL()	System	Yes	TNS/R, TNS/E, TNS/X	H06.18, J06.07, L15.02
spt_interrupt()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_interruptTag()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_LOCKFILE()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02
SPT_LOCKREC()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02
spt_lseekz()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
spt_lseek64z()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
spt_pause()	System	Yes	TNS/R, TNS/E, TNS/X	G06.29, H06.06, J06.03, L15.02
spt_printf()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_printfx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_putc()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_putchar()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_putcharx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
spt_putcx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_puts()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_putsx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_putw()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_putwx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_putwc()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_putwchar()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_putwcharx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_putwcx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_putwx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_read()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_READLOCKX()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
SPT_READUPDATELOCKX()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
SPT_READUPDATEX()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_readv()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_readvx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_readvz()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
SPT_READX()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_readx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_readz()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
SPT_RECEIVEREAD()	System	Yes	TNS/R, TNS/E, TNS/X	G06.26, H06.03, J06.03, L15.02
SPT_RECEIVEREADL()	System	Yes	TNS/R, TNS/E, TNS/X	H06.18, J07.07, L15.02
spt_recv()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_recvfrom()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_recvfromx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_recvmsg()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_recvmsgx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_recvx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_regFile()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_regFileIOHandler()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_regOSSFileIOHandler()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_regPathsendFile()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_regPathsendTagHandler()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_regTimerHandler()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_REPLYX()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_REPLYXL()	System	Yes	TNS/R, TNS/E, TNS/X	H06.18, J06.07, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
SPT_RESUMETRANSACTION ()	Library	Yes	TNS/R, TNS/E, TNS/X	
spt_select ()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_select_single_np ()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_send ()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_sendmsg ()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_sendmsgx ()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_sendto ()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_sendtox ()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_sendx ()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
SPT_SERVERCLASS_DIALOG_ABORT ()	Library	Yes	TNS/R, TNS/E, TNS/X	
SPT_SERVERCLASS_DIALOG_BEGIN ()	Library	Yes	TNS/R, TNS/E, TNS/X	
SPT_SERVERCLASS_DIALOG_BEGINL ()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
SPT_SERVERCLASS_DIALOG_END ()	Library	Yes	TNS/R, TNS/E, TNS/X	
SPT_SERVERCLASS_DIALOG_SEND ()	Library	Yes	TNS/R, TNS/E, TNS/X	
SPT_SERVERCLASS_DIALOG_SENDL ()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
SPT_SERVERCLASS_SEND ()	Library	Yes	TNS/R, TNS/E, TNS/X	
SPT_SERVERCLASS_SEND_INFO ()	Library	Yes	TNS/R, TNS/E, TNS/X	
SPT_SERVERCLASS_SENDL ()	Library	Yes	TNS/R, TNS/E, TNS/X	H06.18, J06.07, L15.02
SPT_SETMODE ()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
spt_setOSSFileIOHandle r()	System	Yes	TNS/R, TNS/E, TNS/X	G06.26, H06.03, J06.03, L15.02
spt_setTMFConcurrentTr ansactions()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_sigaction()	System	Yes	TNS/R, TNS/E, TNS/X	G06.29, H06.06, J06.03, L15.02
spt_signal()	System	Yes	TNS/R, TNS/E, TNS/X	G06.29, H06.06, J06.03, L15.02
spt_sigpending()	System	Yes	TNS/R, TNS/E, TNS/X	G06.29, H06.06, J06.03, L15.02
spt_sigsuspend()	System	Yes	TNS/R, TNS/E, TNS/X	G06.29, H06.06, J06.03, L15.02
spt_sigwait()	System	Yes	TNS/R, TNS/E, TNS/X	G06.29, H06.06, J06.03, L15.02
spt_sleep()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_system()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_TMF_GetTxHandle()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_TMF_Init()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_TMF_RESUME()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_TMF_SetAndValidate TxHandle()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_TMF_SetTxHandle()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_TMF_SUSPEND()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_UNLOCKFILE()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02
SPT_UNLOCKREC()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02
spt_unregFile()	System	Yes	TNS/R, TNS/E, TNS/X	

Table Continued

Function	Reference	HPE	Native Only	RVU Notes
spt_unregOSSFileIOHandler()	System	Yes	TNS/R, TNS/E, TNS/X	G06.26, H06.03, J06.03, L15.02
spt_unregPathsendTagHandler()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_usleep()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_vfprintf()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_vfprintfx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_vprintf()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_vprintfx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_waitpid()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_wakeup()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_write()	System	Yes	TNS/R, TNS/E, TNS/X	
SPT_WRITEREADX()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02
SPT_WRITEUPDATEUNLOCKX()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02
SPT_WRITEUPDATEX()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.04, J06.03, L15.02
spt_writev()	System	Yes	TNS/R, TNS/E, TNS/X	
spt_writevx()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_writevz()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
SPT_WRITEX()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
spt_writex()	System	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
spt_writez()	System	Yes	TNS/R, TNS/E, TNS/X	H06.15, J06.04, L15.02
sqrt()	Library			
sqrtf()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
sqrtl()	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
srand()	Library			
srand48()	Library			
srandom()	Library		TNS/R, TNS/E, TNS/X	
sscanf()	Library	Yes		
stat()	System	Yes		
stat64()	System	Yes		G06.29, H06.06, J06.03
statvfs()	System	Yes		
statvfs64()	System	Yes		G06.29, H06.06, J06.03
step()	Library		TNS/R, TNS/E, TNS/X	
strcasecmp()	Library		TNS/R, TNS/E, TNS/X	
strcat()	Library			
strcat64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strchr()	Library			
strchr64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strcmp()	Library			
strcmp64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strcoll()	Library			
strcpy()	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
strcpy64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strcspn()	Library			
strdup()	Library			
strerror()	Library			
strerror_r()	Library	Yes	TNS/E, TNS/X	H06.21, J06.10, L15.02
strfmon()	Library			
strftime()	Library			
strlen()	Library			
strlen64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strncasecmp()	Library		TNS/R, TNS/E, TNS/X	
strncat()	Library			
strncat64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strncmp()	Library			
strncmp64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strncpy()	Library			
strncpy64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strpbrk()	Library			
strptime()	Library			
strrchr()	Library			
strrchr64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strsignal()	Library		TNS/X	L20.10
strspn()	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
strstr()	Library			
strstr64()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
strtod()	Library			
strtof()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
strtoimax()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
strtok()	Library			
strtok_r()	Library		TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
strtol()	Library			
strtoll()	Library			
strtoul()	Library			
strtoull()	Library			
strtoumax()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
strxfrm()	Library			
swab()	Library			
swprintf()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
swscanf()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
symlink()	System	Extended		
symlinkat()	System	Extended	TNS/E, TNS/X	J06.22, L18.02
sysconf()	Library	Yes		
syslog()	Library	Yes		
system()	Library	Yes		
tan()	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>tanf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>tanh()</code>	Library			
<code>tanhf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>tanh1()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>tanl()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>tcdrain()</code>	Library	Extended		
<code>tcflow()</code>	Library	Extended		
<code>tcflush()</code>	Library	Extended		
<code>tcgetattr()</code>	Library	Extended		
<code>tcgetpgrp()</code>	Library	Extended		
<code>tcsendbreak()</code>	Library	Extended		
<code>tcsetattr()</code>	Library	Extended		
<code>tcsetpgrp()</code>	Library	Extended		
<code>tdelete()</code>	Library			
<code>tdm_execve()</code>	System	Yes		
<code>tdm_execvep()</code>	System	Yes		
<code>tdm_fork()</code>	System	Yes		
<code>tdm_spawn()</code>	System	Extended		
<code>tdm_spawnp()</code>	System	Extended		
<code>telldir()</code>	Library			
<code>tempnam()</code>	Library			
<code>terminate_program()</code>	Library	Yes		

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>tfind()</code>	Library			
<code>tgamma()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>tgammaf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>tgamma1()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>time()</code>	Library			
<code>times()</code>	Library	Extended		
<code>tmpfile()</code>	Library			
<code>tmpfile_guardian()</code>	Library	Yes		
<code>tmpfile_oss()</code>	Library	Yes		
<code>tmpfile64()</code>	Library	Yes		G06.29, H06.06, J06.03
<code>tmpfile64_guardian()</code>	Library	Yes		G06.29, H06.06, J06.03
<code>tmpfile64_oss()</code>	Library	Yes		G06.29, H06.06, J06.03
<code>tmpnam()</code>	Library			
<code>tmpnam_guardian()</code>	Library			
<code>tmpnam_oss()</code>	Library			
<code>tmpnam_r()</code>	Library	Yes	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
<code>toascii()</code>	Library			
<code>_tolower()</code>	Library			
<code>tolower()</code>	Library			
<code>_toupper()</code>	Library			
<code>toupper()</code>	Library			
<code>towctrans()</code>	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
<code>towlower()</code>	Library			
<code>towupper()</code>	Library			
<code>trunc()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>truncf()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>trunc1()</code>	Library		TNS/E, TNS/X	H06.21, J06.10, L15.02
<code>tsearch()</code>	Library			
<code>ttyname()</code>	Library	Extended		
<code>ttyname_r()</code>	Library	Extended	TNS/R, TNS/E, TNS/X	G06.28, H06.05, J06.03, L15.02
<code>twalk()</code>	Library			
<code>tzset()</code>	Library			
<code>ulimit()</code>	System	Extended		
<code>umask()</code>	System			
<code>uname()</code>	System	Extended		
<code>ungetc()</code>	Library			
<code>ungetwc()</code>	Library			
<code>unlink()</code>	System	Extended		
<code>unlinkat()</code>	System	Extended	TNS/E, TNS/X	J06.22, L18.02
<code>unordered()</code>	Library	Yes	TNS/R, TNS/E, TNS/X	
<code>unsetenv()</code>	Library		TNS/X	L20.10
<code>usleep()</code>	Library			
<code>utime()</code>	System	Extended		
<code>va_arg()</code>	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
va_end()	Library			
va_start()	Library			
valloc()	Library			
vfprintf()	Library	Extended		
vfprintf()	Library			G06.27, H06.03
vfwprintf()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
vfwscanf()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
vprintf()	Library	Extended		
vscanf()	Library			G06.27, H06.03
vsnprintf()	Library	Yes		H06.03, J06.03
vsprintf()	Library			
vsscanf()	Library			G06.27, H06.03
vswprintf()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
vswscanf()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
vwprintf()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
vwscanf()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
wait()	System	Yes		
waitpid()	System	Yes		
wcrtomb()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
wcscat()	Library			
wcschr()	Library			
wcscmp()	Library			

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
wscoll()	Library			
wscopy()	Library			
wscspn()	Library			
wcsftime()	Library			
wcslen()	Library			
wcsncat()	Library			
wcsncmp()	Library			
wcsncpy()	Library			
wcspbrk()	Library			
wcsrchr()	Library			
wcsrtombs()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
wcsspn()	Library			
wcstod()	Library			
wcstof()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
wcstoimax()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
wcstok()	Library			
wcstol()	Library			
wcstoll()	Library			
wcstombs()	Library			
wcstoul()	Library			
wcstoull()	Library			
wcstoumax()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02

Table Continued



Function	Reference	HPE	Native Only	RVU Notes
wcswcs ()	Library			
wcswidth ()	Library			
wcsxfrm ()	Library			
wctob ()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
wctomb ()	Library			
wctrans ()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
wctype ()	Library			
wcwidth ()	Library			
wcmemchr ()	Library		TNS/R, TNS/E, TNS/X	
wcmemcmp ()	Library		TNS/R, TNS/E, TNS/X	
wcmemcpy ()	Library		TNS/R, TNS/E, TNS/X	
wcmemmove ()	Library		TNS/R, TNS/E, TNS/X	
wcmemset ()	Library		TNS/R, TNS/E, TNS/X	
wordexp ()	Library			
wordfree ()	Library			
wprintf ()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
write ()	System	Extended		
write64_ ()	Library	Yes	TNS/E, TNS/X	H06.24, J06.13, L15.02
writew ()	System	Extended		
wscanf ()	Library		TNS/E, TNS/X	H06.08, J06.03, L15.02
y0 ()	Library			
y1 ()	Library			
yn ()	Library			



API Interoperability in TNS Processes

This appendix contains **Table 78: API Interoperability in TNS Processes**, which describes the interoperability between the Guardian API and the Open System Services (OSS) API in TNS processes. This appendix applies only to G-series systems: the J-series and H-series OSS environments do not support TNS processes.

Table 78: API Interoperability in TNS Processes lists the functions and macros in the Guardian C Run-Time Library and the OSS API. Functions in the Guardian sockets library and Guardian system procedures are not included.

The table indicates the following characteristics for each function or macro:

- Modules it can be called from
- Processes it can be called from
- Parameters it requires
- Objects it can operate on

Calling Functions and Macros

The two factors that determine when you can call a function or macro are:

- The environment for which the module containing the call was compiled.
- The environment of the process executing the module.

An OSS module is a module compiled to have OSS semantics and execute in the OSS environment. Likewise, a Guardian module is a module compiled to have Guardian semantics and execute in the Guardian environment. Normally, the environments of a module and the process containing the module are the same.

You can mix modules of different environments in the same process. An OSS process can execute both OSS and Guardian modules, and a Guardian process can execute both Guardian and OSS modules. See **Overview of the OSS Programming Environment** for a discussion of when to write mixed-module applications.

Environment of the Module

The environment of a module is determined by the value of the SYSTYPE compiler pragma, which directs the compiler to generate code appropriate for the specified target execution environment.

In **Table 78: API Interoperability in TNS Processes**, most functions and macros have the same name in both APIs, have identical parameter lists, and the module will compile without errors regardless the environment you specify. A few functions and macros, such as the `open()` function, have different parameter lists in the two APIs. If you attempt to call the Guardian C `open()` function in a module compiled for the OSS environment, you will get a compilation error because the parameter list is incorrect.

Environment of the Process

The environment of a process is determined by the environment of the module defining the `main()` function.

Many functions and macros in the Guardian and OSS APIs can be called from a Guardian or OSS module, which in turn can be part of a Guardian or OSS process. Some functions and macros, such as the `signal()` function, can be called only from a Guardian module in a Guardian process or from an OSS module in an OSS process.

For example, if you attempt to call the `signal()` function in a module compiled for the Guardian environment, and then link this module to a `main()` function compiled for the OSS environment (an OSS process), you will get a run-time error.



This is because the system resources and services required by the Guardian function cannot be provided by the OSS process environment.

Function and Macro Behavior

A function or macro that can be called in both the Guardian and OSS environments might or might not behave in the same way in each environment. Behavior can be classified into three categories:

- Identical behavior

Many of the functions and macros behave the same way in each environment because they take environment-independent parameters. For example, string functions such as `strcat()` are identical in the Guardian and OSS environments. The `malloc()` function can allocate a block of memory in a Guardian module and the `free()` function can free the same block of memory in an OSS module.

- Nearly identical behavior

Some functions and macros behave nearly the same in each environment. Functions and macros with nearly identical behavior also take environment-independent parameters. For example, mathematical functions such as `sinh()` generate an exception on arithmetic overflow in the Guardian environment but not in the OSS environment.

- Different behavior

Some functions and macros behave differently in each environment because they take or return environment-specific parameters. For example, the Guardian C and OSS `tmpnam()` functions are both declared as:

```
char *tmpnam(char *s)
```

The Guardian C `tmpnam()` function returns a valid Guardian filename, and the OSS `tmpnam()` function returns a valid OSS pathname.

Although the name of a function or macro can be the same in the Guardian and OSS environments, each environment can have a function with different syntax and semantics. For example, the Guardian C `open()` function is declared as:

```
int open(char *file_name, int flags [, int security]
        [, int depth] [, int pext] [, int sext]);
```

The OSS `open()` function is declared as:

```
int open(char *path, int oflag [, mode_t mode]);
```

Not only does the Guardian C `open()` function require a name in Guardian filename syntax and the OSS `open()` function require a name in OSS pathname syntax, the functions have different calling sequences. Also, you cannot pass an OSS file descriptor to the Guardian C `open()` function (or to any Guardian C function), and vice versa.

An OSS function or macro that can operate on Guardian objects is limited in what it can do when compared to the corresponding Guardian C function or macro. For example, the OSS `open()` function can open some types of Guardian files (by specifying the file as an OSS pathname in the `/G` file system), but the Guardian C `open()` function can open additional types of Guardian files.

Using the API Interoperability Table

Some functions and macros in the Guardian and OSS APIs can interoperate completely between the Guardian and OSS environments: that is, they can be called from either a Guardian or OSS module, can execute as part of a Guardian or OSS process, and can operate on Guardian or OSS objects. Some functions and macros are more limited in their interoperability between environments.

For each function or macro listed in the table **Table 78: API Interoperability in TNS Processes**, the level of interoperability it provides is indicated by the following information:



- Specification or standard where the function or macro is defined.
- Whether the function or macro can be called from a Guardian module, an OSS module, a Guardian process, or an OSS process.
- Types of parameters (OSS or Guardian) the function or macro requires.
- Types of objects (OSS or Guardian) the function or macro operates on.

G and O Notation

In the columns headed “Module Compiled As,” and “Process Called From,” the letter “G” indicates “Guardian” and “O” indicates “OSS.”

Specified By

The “Specified By” column indicates where a function or macro is defined:

ISO

The function or macro is specified in the ISO/ANSI C standard.

POSIX

The function or macro is specified in the POSIX.1 or POSIX.2 standard. (Functions identified as “ISO” are also specified in the POSIX.1 standard.)

HPE

The function or macro is an extension specified by Hewlett Packard Enterprise.

X/OPEN

The function or macro is specified in the XPG4 version 2 specification, which is the basis for the X/OPEN UNIX brand.

XPG4

The function or macro is specified in the XPG4 specification. (Functions and macros identified as “ISO” or “POSIX” are also specified in the XPG4 specification.)

Functions and macros specified in the ISO standard are declared for both Guardian and OSS modules.

By default, functions and macros specified in POSIX, XPG4, and X/OPEN UNIX are declared only for OSS modules. To use such a function or macro in a Guardian module (if that is allowed), specify the `_XOPEN_SOURCE` feature test macro when you compile the module.

By default, functions and macros specified by Hewlett Packard Enterprise are declared only for Guardian modules. To use such a function or macro in an OSS module (if that is allowed), specify the `_TANDEM_SOURCE` feature test macro when you compile the module.

Regardless of whether the function or macro is being called from a Guardian or OSS module, some functions and macros specified by X/OPEN UNIX require that the `_XOPEN_SOURCE_EXTENDED` feature test macro be specified when you compile the module, and some require that either the `_XOPEN_SOURCE_EXTENDED` or the `_TANDEM_SOURCE` feature test macro be specified when you compile the module.

For the most complete information about any function, see the reference page for that function either online or in the *Open System Services System Calls Reference Manual*.

Defining Parameters and Objects

OSS parameters include OSS pathnames, stream pointers, file descriptors, signals, and process identifiers. Guardian parameters include Guardian filenames, stream pointers, file descriptors, signals, and process identifiers. OSS objects include OSS files, streams, and processes. Guardian objects include Guardian files, streams, and processes.



In some cases, OSS parameters can be used to access Guardian objects. For example, an OSS pathname parameter can identify a Guardian file using the `/G/volume/subvolume/fileid` syntax.

Usage Notes

The “Usage Notes” column provides information about the behavior of the function or macro. Unless indicated otherwise in the “Usage Notes” column, a function or macro has the same behavior regardless of the module or process environment from which it is called.

The following table describes the “Usage Notes” values that appear frequently.

Usage Note	Meaning
Checking variants	If the CHECK pragma is specified, functions and macros in both Guardian and OSS modules perform identical parameter validation. If the NOCHECK pragma is specified, functions and macros in both Guardian and OSS modules do not perform extra parameter validation. The Guardian environment compiler defaults to CHECK. The OSS environment compiler defaults to NOCHECK. Refer to the CHECK pragma in the <i>C/C++ Programmer's Guide</i> for details.
Exception variants	The function or macro in a Guardian module generates an exception on an arithmetic fault. The function or macro in an OSS module does not generate an exception on an arithmetic fault.
Guardian function operates only on Guardian objects	The Guardian function or macro operates only on Guardian objects.
Guardian objects only	The function or macro operates only on Guardian objects.
Guardian parameters only	The function or macro requires Guardian parameters (Guardian filename, stream pointer, file descriptor, or process identifier).
Limited OSS objects only	The function or macro operates on some types of OSS objects. The extent of support for OSS objects differs for each function or macro.
Limited when called from Guardian process	The function or macro's operation is limited when it is called from a Guardian process.
Only Guardian modules or only OSS modules	The function or macro can be called only from a Guardian module within a Guardian process or from an OSS module within an OSS process. The function or macro cannot be called from both Guardian and OSS modules within the same process. The module and process environments must be the same. Behavior might be different in each environment.
OSS function operates on OSS objects and limited Guardian objects	The OSS function or macro operates on OSS objects and some types of Guardian objects. The extent of support for Guardian objects differs for each function or macro.
OSS function operates only on OSS objects	The OSS function or macro operates only on OSS objects.

Table Continued



Usage Note	Meaning
OSS objects and limited Guardian objects	The function or macro operates on OSS objects and some types of Guardian objects. The extent of support for Guardian objects differs for each function or macro.
OSS objects only	The function or macro operates only on OSS objects.
OSS parameters only	The function or macro requires OSS parameters (OSS pathname, stream pointer, file descriptor, or process identifier).
Parameters must match module	When called from a Guardian module, the function or macro requires Guardian parameters. When called from an OSS module, the function or macro requires OSS parameters. If the function or macro operates on an open file, the file must have been opened in a module of the same environment.
<code>_XOPEN_SOURCE_EXTENDED</code>	The function or macro requires that the feature test macro <code>_XOPEN_SOURCE_EXTENDED</code> be specified when you compile the module.
<code>_XOPEN_SOURCE_EXTENDED</code> or <code>_TANDEM_SOURCE</code>	The function or macro requires that the feature test macro <code>_XOPEN_SOURCE_EXTENDED</code> or <code>_TANDEM_SOURCE</code> be specified when you compile the module.

Examples

The following examples illustrate how to use **Table 78: API Interoperability in TNS Processes** to determine how functions and macros can be used in Guardian and OSS programs.

1. The following information can be derived about the `acos()` function:

- Specified in the ISO/ANSI C standard.
- Can be called from Guardian and OSS modules.
- Can be called from Guardian and OSS processes.
- When called from a Guardian module, it causes an exception on arithmetic overflow; when called from an OSS module, it does not.

2. The following information can be derived about the `edlseek()` function:

- Is an extension specified by Hewlett Packard Enterprise.
- Can be called only from Guardian modules.
- Can be called from Guardian and OSS processes.
- Requires Guardian parameters.
- Operates only on Guardian objects.
- Because it is specified by Hewlett Packard Enterprise, it requires the `_TANDEM_SOURCE` feature test macro.

3. The following information can be derived about the `edpwent()` function:



- Specified in the X/OPEN specification.
- Can be called only from OSS modules.
- Can be called only from OSS processes.
- Requires the the `_XOPEN_SOURCE_EXTENDED` or `_TANDEM_SOURCE` feature test macro.

4. The following information can be derived about the `free()` function:

- Specified in the ISO/ANSI C standard.
- Can be called from Guardian and OSS modules.
- Can be called from Guardian and OSS processes.
- If the `CHECK` pragma is specified, the function performs identical parameter validation in both Guardian and OSS modules. If the `NOCHECK` pragma is specified, the function does not perform extra parameter validation in Guardian and OSS modules.

5. The following information can be derived about the `rename()` function:

- Specified in the ISO/ANSI C standard.
- Can be called from Guardian and OSS modules.
- Can be called from Guardian and OSS processes.
- When called from a Guardian module, it requires Guardian parameters; when called from an OSS module, it requires OSS parameters.
- Guardian function operates only on Guardian objects; OSS function operates on OSS objects and limited Guardian objects.

6. The following information can be derived about the `signal()` function:

- Specified in the ISO/ANSI C standard.
- Can be called only from a Guardian module in a Guardian process or from an OSS module in an OSS process. The module and process environments must be the same. The function's behavior is different in each environment.
- Guardian function operates only on Guardian objects; OSS function operates only on OSS objects.

Table 78: API Interoperability in TNS Processes

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>__ns_backup_fopen()</code>	HPE	G	G	
<code>__ns_fget_file_open_state()</code>	HPE	G	G	
<code>__ns_fget_file_state()</code>	HPE	G	G	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>__ns_fopen_special()</code>	HPE	G	G	
<code>__ns_fset_file_state()</code>	HPE	G	G	
<code>__ns_start_backup()</code>	HPE	G	G	
<code>_abs()</code>	HPE	G, O	G, O	
<code>_exit()</code>	X/OPEN	G, O	G, O	Nowait considerations when called from the Guardian API
<code>_is_system_trap()</code>	HPE	G	G	
<code>_max()</code>	HPE	G, O	G, O	
<code>_min()</code>	HPE	G, O	G, O	
<code>_tolower()</code>	XPG4	G, O	G, O	
<code>_toupper()</code>	XPG4	G, O	G, O	
<code>abort()</code>	ISO	G, O	G, O	
<code>abs()</code>	ISO	G, O	G, O	
<code>access()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>acos()</code>	ISO	G, O	G, O	Exception variants
<code>alarm()</code>	POSIX	O	O	OSS objects only
<code>asctime()</code>	ISO	G, O	G, O	
<code>asin()</code>	ISO	G, O	G, O	Exception variants
<code>assert()</code>	ISO	G, O	G, O	
<code>atan()</code>	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
atan2()	ISO	G, O	G, O	Exception variants
atexit()	ISO	G, O	G, O	
atof()	ISO	G, O	G, O	
atoi()	ISO	G, O	G, O	
atol()	ISO	G, O	G, O	
bsearch()	ISO	G, O	G, O	
calloc()	ISO	G, O	G, O	Checking variants
catclose()	XPG4	O	O	OSS parameters only
catgets()	XPG4	O	O	OSS parameters only
catopen()	XPG4	O	O	OSS parameters only
ceil()	ISO	G, O	G, O	
cfgetispeed()	POSIX	O	O	
cfgetospeed()	POSIX	O	O	
cfsetispeed()	POSIX	O	O	
cfsetospeed()	POSIX	O	O	
chdir()	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
chmod()	POSIX	G, O	G, O	OSS objects only
chown()	POSIX	G, O	G, O	OSS objects and limited Guardian objects
chroot()	XPG4	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>chvol()</code>	HPE	G	G	
<code>clearerr()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>clock()</code>	ISO	G, O	G, O	
<code>close()</code>	XPG4	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>closedir()</code>	POSIX	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>closelog()</code>	X/OPEN	G, O	G, O	
<code>confstr()</code>	XPG4	G, O	G, O	
<code>cos()</code>	ISO	G, O	G, O	
<code>cosh()</code>	ISO	G, O	G, O	Exception variants
<code>creat()</code>	HPE	G	G	Guardian parameters only Guardian objects only
<code>creat()</code>	XPG4	O	O	OSS parameters only OSS objects and limited Guardian objects
<code>ctermid()</code>	POSIX	O	O	OSS parameters only
<code>ctime()</code>	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>dec_to_longlong()</code>	HPE	G, O	G, O	
<code>difftime()</code>	ISO	G, O	G, O	
<code>div()</code>	ISO	G, O	G, O	
<code>drand48()</code>	XPG4	G, O	G, O	
<code>dup()</code>	POSIX	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>dup2()</code>	POSIX	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>ecvt()</code>	HPE	G, O	G, O	
<code>edfseek()</code>	HPE	G	G, O	Guardian parameters only Guardian objects only
<code>edftell()</code>	HPE	G	G, O	Guardian parameters only Guardian objects only
<code>edlseek()</code>	HPE	G	G, O	Guardian parameters only Guardian objects only
<code>endpwent()</code>	X/OPEN	O	O	_XOPEN_SOURCE_EXTENDED or _TANDEM_SOURCE
<code>erand48()</code>	XPG4	G, O	G, O	
<code>erf()</code>	XPG4	G, O	G, O	
<code>erfc()</code>	XPG4	G, O	G, O	
<code>execl()</code>	POSIX	O	O	OSS objects only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>execle()</code>	POSIX	O	O	OSS objects only
<code>execlp()</code>	POSIX	O	O	OSS objects only
<code>execv()</code>	POSIX	O	O	OSS objects only
<code>execve()</code>	POSIX	O	O	OSS objects only
<code>forkexecve()</code>	POSIX	O	O	OSS objects only
<code>execvp()</code>	POSIX	O	O	OSS objects only
<code>exit()</code>	ISO	G, O	G, O	
<code>exp()</code>	ISO	G, O	G, O	Exception variants
<code>fabs()</code>	ISO	G, O	G, O	
<code>fclose()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fcloseall()</code>	HPE	G	G	
<code>fcntl()</code>	HPE	G	G	Guardian parameters only Guardian objects only
<code>fcntl()</code>	XPG4	O	O	OSS parameters only OSS objects only
<code>fdopen()</code>	XPG4	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>fdtogfn()</code>	HPE	G	G, O	Guardian parameters only Guardian objects only
<code>feof()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>ferror()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fflush()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fgetc()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fgetpos()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>fgets()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fgetwc()</code>	XPG4	O	G, O	
<code>fgetws()</code>	XPG4	O	G, O	
<code>fileno()</code>	XPG4	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>floor()</code>	ISO	G, O	G, O	
<code>fmod()</code>	ISO	G, O	G, O	
<code>fnmatch()</code>	POSIX	G, O	G, O	
<code>fopen()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fopen_guardian()</code>	HPE	G, O	G, O	Guardian parameters only Guardian objects only
<code>fopen_oss()</code>	HPE	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>fopen_std_file()</code>	HPE	G	G	
<code>fork()</code>	POSIX	O	O	OSS objects only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>fpathconf()</code>	POSIX	G, O	G, O	OSS parameters only
<code>fprintf()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fputc()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fputs()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fputwc()</code>	XPG4	O	G, O	
<code>fputws()</code>	XPG4	O	G, O	
<code>fread()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>free()</code>	ISO	G, O	G, O	Checking variants
<code>freopen()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>freopen_guardian()</code>	HPE	G, O	G, O	Guardian parameters only Guardian objects only
<code>freopen_oss()</code>	HPE	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>frexp()</code>	ISO	G, O	G, O	
<code>fscanf()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fseek()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fsetpos()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fstat()</code>	POSIX	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>fsync()</code>	XPG4	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>ftell()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>ftok()</code>	X/OPEN	O	G, O	OSS parameters only OSS objects only _XOPEN_SOURCE_EXTENDED or _TANDEM_SOURCE
<code>ftw()</code>	XPG4	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>fwrite()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>gamma()</code>	XPG4	G, O	G, O	
<code>get_assign_msg()</code>	HPE	G	G	
<code>get_assign_msg_by_name()</code>	HPE	G	G	
<code>get_max_assign_msg_ordinal()</code>	HPE	G	G	
<code>get_param_by_name()</code>	HPE	G	G	
<code>get_param_msg()</code>	HPE	G	G	
<code>get_startup_msg()</code>	HPE	G	G	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
getc()	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
getchar()	ISO	G, O	G, O	Only Guardian modules or only OSS modules Guardian function operates only on Guardian objects OSS function operates only on OSS objects
getcwd()	POSIX	G, O	G, O	
getegid()	POSIX	G, O	G, O	
getenv()	ISO	G, O	G, O	
geteuid()	POSIX	G, O	G, O	
getgid()	POSIX	G, O	G, O	
getgrgid()	POSIX	O	O	Data can be overwritten by subsequent call or call to getgrnamO
getgrnam()	POSIX	O	O	Data can be overwritten by subsequent call or call to getgrgidO
getgroups()	POSIX	O	O	
getlogin()	POSIX	O	O	
getopt()	XPG4	O	O	
getpgrp()	POSIX	O	O	OSS objects only
getpid()	POSIX	O	O	OSS objects only
getppid()	POSIX	O	O	OSS objects only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
getpwent()	X/OPEN	O	O	_XOPEN_SOURCE_EXTENDED or _TANDEM_SOURCE
getpwnam()	POSIX	O	O	
getpwuid()	POSIX	O	O	
gets()	ISO	G, O	G, O	Only Guardian modules or only OSS modules Guardian function operates only on Guardian objects OSS function operates only on OSS objects
getsid()	XPG4	O	O	
getuid()	POSIX	G, O	G, O	
getw()	XPG4	O	G, O	OSS function operates on OSS objects and limited Guardian objects
getwc()	XPG4	O	G, O	
getwchar()	XPG4	O	O	
getws()	XPG4	O	O	
glob()	POSIX	G, O	G, O	
globfree()	POSIX	G, O	G, O	
gmtime()	ISO	G, O	G, O	
hcreate()	XPG4	G, O	G, O	
hdestroy()	XPG4	G, O	G, O	
heap_check()	HPE	G, O	G, O	
heap_check_always()	HPE	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
heap_min_block_size()	HPE	G, O	G, O	
hsearch()	XPG4	G, O	G, O	
hypot()	XPG4	G, O	G, O	
iconv()	XPG4	O	O	
iconv_close()	XPG4	O	O	
iconv_open()	XPG4	O	O	
ioctl()	X/OPEN	G, O	G, O	
isalnum()	ISO	G, O	G, O	
isalpha()	ISO	G, O	G, O	
isascii()	XPG4	G, O	G, O	
isatty()	POSIX	O	G, O	OSS parameters only
iscntrl()	ISO	G, O	G, O	
iscsym()	HPE	G, O	G, O	
iscsymf()	HPE	G, O	G, O	
isdigit()	ISO	G, O	G, O	
isgraph()	ISO	G, O	G, O	
islower()	ISO	G, O	G, O	
isnan()	ISO	G, O	G, O	
isprint()	ISO	G, O	G, O	
ispunct()	ISO	G, O	G, O	
isspace()	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>isupper()</code>	ISO	G, O	G, O	
<code>iswalnum()</code>	XPG4	G, O	G, O	
<code>iswalpha()</code>	XPG4	G, O	G, O	
<code>iswcntrl()</code>	XPG4	G, O	G, O	
<code>iswctype()</code>	XPG4	G, O	G, O	
<code>iswdigit()</code>	XPG4	G, O	G, O	
<code>iswgraph()</code>	XPG4	G, O	G, O	
<code>iswlower()</code>	XPG4	G, O	G, O	
<code>iswprint()</code>	XPG4	G, O	G, O	
<code>iswpunct()</code>	XPG4	G, O	G, O	
<code>iswspace()</code>	XPG4	G, O	G, O	
<code>iswupper()</code>	XPG4	G, O	G, O	
<code>iswxdigit()</code>	XPG4	G, O	G, O	
<code>isxdigit()</code>	XPG4	G, O	G, O	
<code>j0()</code>	XPG4	G, O	G, O	
<code>j1()</code>	XPG4	G, O	G, O	
<code>jn()</code>	XPG4	G, O	G, O	
<code>jrand48()</code>	XPG4	G, O	G, O	
<code>kill()</code>	POSIX	O	O	OSS objects only
<code>labs()</code>	ISO	G, O	G, O	
<code>lcong48()</code>	XPG4	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
ldexp()	ISO	G, O	G, O	Exception variants
ldiv()	ISO	G, O	G, O	
lfind()	XPG4	G, O	G, O	
lgamma()	XPG4	G, O	G, O	
link()	POSIX	O	G, O	OSS parameters only OSS objects only
localeconv()	ISO	G, O	G, O	
localtime()	ISO	G, O	G, O	
log()	ISO	G, O	G, O	Exception variants
log10()	ISO	G, O	G, O	Exception variants
longjmp()	ISO	G, O	G, O	Only Guardian modules or only OSS modules
longlong_to_dec()	HPE	G, O	G, O	
lrand48()	XPG4	G, O	G, O	
lsearch()	XPG4	G, O	G, O	
lseek()	XPG4	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
lstat()	X/OPEN	G, O	G, O	_XOPEN_SOURCE_EXTENDED
malloc()	ISO	G, O	G, O	Checking variants
mblen()	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
mbstowcs()	ISO	G, O	G, O	
mbtowc()	ISO	G, O	G, O	
memcpy()	ISO	O	G, O	
memchr()	ISO	G, O	G, O	Checking variants
memcmp()	ISO	G, O	G, O	Checking variants
memcpy()	ISO	G, O	G, O	Checking variants
memmove()	ISO	G, O	G, O	Checking variants
memset()	ISO	G, O	G, O	Checking variants
memswap()	HPE	G, O	G, O	Checking variants
mkdir()	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
mkfifo()	POSIX	G, O	G, O	OSS parameters only
mknod()	X/OPEN	G, O	G, O	_XOPEN_SOURCE_EXTENDED
mktime()	ISO	G, O	G, O	
modf()	ISO	G, O	G, O	
movmem()	HPE	G, O	G, O	Checking variants
mrnd48()	XPG4	G, O	G, O	
msgctl()	XPG4	O	O	OSS objects only
msgget()	XPG4	O	O	OSS objects only
msgrcv()	XPG4	O	O	OSS objects only
msgsnd()	XPG4	O	O	OSS objects only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>nl_langinfo()</code>	XPG4	G, O	G, O	
<code>nlist()</code>	X/OPEN	G, O	G, O	
<code>nrand48()</code>	XPG4	G, O	G, O	
<code>offsetof()</code>	ISO	G, O	G, O	
<code>open()</code>	HPE	G	G	Guardian parameters only Guardian objects only
<code>open()</code>	XPG4	O	O	OSS parameters only OSS objects and limited Guardian objects
<code>opendir()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>openlog()</code>	X/OPEN	G, O	G, O	Limited when called from Guardian process
<code>pathconf()</code>	POSIX	G, O	G, O	OSS parameters only
<code>pause()</code>	POSIX	O	O	OSS objects only
<code>pclose()</code>	POSIX	G, O	G, O	OSS parameters only
<code>perror()</code>	ISO	G, O	G, O	
<code>pipe()</code>	POSIX	O	G, O	OSS parameters only
<code>popen()</code>	POSIX	G, O	G, O	OSS parameters only
<code>pow()</code>	ISO	G, O	G, O	Exception variants
<code>printf()</code>	ISO	G, O	G, O	Only Guardian modules or only OSS modules
<code>putc()</code>	ISO	G, O	G, O	Parameters must match module

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>putchar()</code>	ISO	G, O	G, O	Only Guardian modules or only OSS modules Guardian function operates only on Guardian objects OSS function operates only on OSS objects
<code>putenv()</code>	XPG4	O	G, O	
<code>puts()</code>	ISO	G, O	G, O	Only Guardian modules or only OSS modules
<code>putw()</code>	XPG4	O	G, O	
<code>putwc()</code>	XPG4	O	G, O	
<code>putwchar()</code>	XPG4	O	O	
<code>putws()</code>	XPG4	O	O	
<code>qsort()</code>	ISO	G, O	G, O	
<code>raise()</code>	ISO	G, O	G, O	Only Guardian modules or only OSS modules Guardian function operates only on Guardian objects OSS function operates only on OSS objects
<code>rand()</code>	ISO	G, O	G, O	
<code>read()</code>	XPG4	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>readdir()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>readlink()</code>	X/OPEN	G, O	G, O	OSS objects only _XOPEN_SOURCE_EXTENDED
<code>readupdate()</code>	HPE	G	G, O	Guardian parameters only
<code>realloc()</code>	ISO	G, O	G, O	Checking variants
<code>regcomp()</code>	XPG4	O	O	
<code>regerror()</code>	XPG4	O	O	
<code>regexexec()</code>	XPG4	O	O	
<code>regfree()</code>	XPG4	O	O	
<code>remove()</code>	ISO	G, O	G, O	Parameters must match module
<code>remove_guardian()</code>	HPE	G, O	G, O	Guardian parameters only
<code>remove_oss()</code>	HPE	G, O	G, O	OSS parameters only
<code>rename()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>rename_guardian()</code>	HPE	G, O	G, O	Guardian parameters only Guardian objects only
<code>rename_oss()</code>	HPE	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>reply()</code>	HPE	G	G, O	
<code>repmem()</code>	HPE	G, O	G, O	Checking variants
<code>rewind()</code>	ISO	G, O	G, O	Parameters must match module
<code>rewinddir()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>rmdir()</code>	POSIX	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>scanf()</code>	ISO	G, O	G, O	Only Guardian modules or only OSS modules
<code>seed48()</code>	XPG4	G, O	G, O	
<code>seekdir()</code>	XPG4	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>semctl()</code>	XPG4	O	O	
<code>semget()</code>	XPG4	O	O	
<code>semop()</code>	XPG4	O	O	
<code>setbuf()</code>	ISO	G, O	G, O	Parameters must match module
<code>setgid()</code>	POSIX	G, O	G, O	
<code>setjmp()</code>	ISO	G, O	G, O	Only Guardian modules or only OSS modules
<code>setlocale()</code>	ISO	G, O	G, O	
<code>setlocale_from_msg()</code>	HPE	O	O	OSS parameters only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
setlogmask()	X/OPEN	G, O	G, O	
setmem()	HPE	G, O	G, O	Checking variants
setnbuf()	HPE	G, O	G, O	Parameters must match module
setpgid()	POSIX	G, O	O	OSS objects only
setpgrp()	POSIX	O	O	OSS objects only
setpwent()	X/OPEN	O	O	_XOPEN_SOURCE_EXTENDED or _TANDEM_SOURCE
setregid()	XPG4	O	O	OSS objects only
setreuid()	XPG4	O	O	OSS objects only
setsid()	POSIX	O	O	OSS objects only
setuid()	POSIX	G, O	G, O	
setvbuf()	ISO	G, O	G, O	Parameters must match module
shmat()	XPG4	O	O	
shmctl()	XPG4	O	O	
shmdt()	XPG4	O	O	
shmget()	XPG4	O	O	
sigaction()	POSIX	O	O	OSS objects only
sigaddset()	POSIX	O	O	OSS objects only
sigdelset()	POSIX	O	O	OSS objects only
sigemptyset()	POSIX	O	O	OSS objects only
sigfillset()	POSIX	O	O	OSS objects only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>sigismember()</code>	POSIX	O	O	OSS objects only
<code>siglongjmp()</code>	POSIX	O	O	OSS objects only
<code>signal()</code>	ISO	G, O	G, O	Only Guardian modules or only OSS modules Guardian function operates only on Guardian objects OSS function operates only on OSS objects
<code>sigpending()</code>	POSIX	O	O	OSS objects only
<code>sigprocmask()</code>	POSIX	O	O	OSS objects only
<code>sigsetjmp()</code>	POSIX	O	O	OSS objects only
<code>sigsuspend()</code>	POSIX	O	O	OSS objects only
<code>sin()</code>	ISO	G, O	G, O	
<code>sinh()</code>	ISO	G, O	G, O	Exception variants
<code>snprintf()</code> (H-series only)	ISO	G, O	G, O	
<code>sprintf()</code>	ISO	G, O	G, O	
<code>sqrt()</code>	ISO	G, O	G, O	Exception variants
<code>rand()</code>	ISO	G, O	G, O	
<code>rand48()</code>	XPG4	G, O	G, O	
<code>sscanf()</code>	ISO	G, O	G, O	
<code>stat()</code>	POSIX	O	G, O	OSS parameters only OSS objects and limited Guardian objects

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
stcarg()	HPE	G, O	G, O	Checking variants
stccpy()	HPE	G, O	G, O	Checking variants
stcd_i()	HPE	G, O	G, O	Checking variants
stcd_l()	HPE	G, O	G, O	Checking variants
stch_i()	HPE	G, O	G, O	Checking variants
stci_d()	HPE	G, O	G, O	Checking variants
stcis()	HPE	G, O	G, O	Checking variants
stciscn()	HPE	G, O	G, O	Checking variants
stclen()	HPE	G, O	G, O	Checking variants
stcpm()	HPE	G, O	G, O	Checking variants
stcpma()	HPE	G, O	G, O	Checking variants
stcu_d()	HPE	G, O	G, O	Checking variants
stpblk()	HPE	G, O	G, O	Checking variants
stpbrk()	HPE	G, O	G, O	Checking variants
stpchr()	HPE	G, O	G, O	Checking variants
stpsym()	HPE	G, O	G, O	Checking variants
stptok()	HPE	G, O	G, O	Checking variants
strcat()	ISO	G, O	G, O	Checking variants
strchr()	ISO	G, O	G, O	Checking variants
strcmp()	ISO	G, O	G, O	Checking variants
strcoll()	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
strcpy()	ISO	G, O	G, O	Checking variants
strcspn()	ISO	G, O	G, O	Checking variants
strdup()	X/OPEN	G, O	G, O	_XOPEN_SOURCE_EXTENDED
strerror()	ISO	G, O	G, O	
strfmon()	XPG4	G, O	G, O	
strftime()	ISO	G, O	G, O	
strlen()	ISO	G, O	G, O	Checking variants
strncat()	ISO	G, O	G, O	Checking variants
strncmp()	ISO	G, O	G, O	Checking variants
strncpy()	ISO	G, O	G, O	Checking variants
strpbrk()	ISO	G, O	G, O	Checking variants
strptime()	XPG4	G, O	G, O	
strrchr()	ISO	G, O	G, O	Checking variants
strspn()	ISO	G, O	G, O	Checking variants
strstr()	ISO	G, O	G, O	Checking variants
strtod()	ISO	G, O	G, O	Checking variants
strtok()	ISO	G, O	G, O	Checking variants
strtol()	ISO	G, O	G, O	Checking variants
strtoll()	ISO	G, O	G, O	Checking variants
strtoul()	ISO	G, O	G, O	Checking variants
strtoull()	ISO	G, O	G, O	Checking variants

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
strxfrm()	ISO	G, O	G, O	
stscmp()	HPE	G, O	G, O	
swab()	XPG4	G, O	G, O	
symlink()	X/OPEN	G, O	G, O	OSS objects only _XOPEN_SOURCE_EXTENDED
sysconf()	POSIX	G, O	G, O	
syslog()	X/OPEN	G, O	G, O	
system()	ISO	G, O	G, O	Only Guardian modules or only OSS modules
tan()	ISO	G, O	G, O	
tanh()	ISO	G, O	G, O	
tcdrain()	POSIX	O	O	
tcflow()	POSIX	O	O	
tcflush()	POSIX	O	O	
tcgetattr()	POSIX	O	O	
tcgetpgrp()	POSIX	O	O	
tcsendbreak()	POSIX	O	O	
tcsetattr()	POSIX	O	O	
tcsetpgrp()	POSIX	O	O	
tdelete()	XPG4	G, O	G, O	
tdm_execve()	HPE	O	O	OSS parameters only
tdm_execvep()	HPE	O	O	OSS parameters only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>tdm_fork()</code>	HPE	O	O	OSS parameters only
<code>tdm_spawn()</code>	HPE	O	O	OSS parameters only
<code>tdm_spawnp()</code>	HPE	O	O	OSS objects only
<code>tellldir()</code>	XPG4	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>tempnam()</code>	XPG4	G, O	O	
<code>terminate_program()</code>	HPE	G, O	G, O	
<code>tfind()</code>	XPG4	G, O	G, O	
<code>time()</code>	ISO	G, O	G, O	
<code>times()</code>	POSIX	G, O	G, O	
<code>tmpfile()</code>	ISO	G, O	G, O	Parameters must match module
<code>tmpfile_guardian()</code>	HPE	G, O	G, O	Guardian parameters only
<code>tmpfile_oss()</code>	HPE	G, O	G, O	OSS parameters only
<code>tmpnam()</code>	ISO	G, O	G, O	Parameters must match module
<code>tmpnam_guardian()</code>	HPE	G, O	G, O	Guardian parameters only
<code>tmpnam_oss()</code>	HPE	G, O	G, O	OSS parameters only
<code>toascii()</code>	XPG4	G, O	G, O	
<code>tolower()</code>	ISO	G, O	G, O	
<code>toupper()</code>	ISO	G, O	G, O	
<code>towlower()</code>	XPG4	G, O	G, O	
<code>towupper()</code>	XPG4	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>trap_overflows()</code>	HPE	G	G	
<code>tsearch()</code>	XPG4	G, O	G, O	
<code>ttyname()</code>	XPG4	O	G, O	
<code>twalk()</code>	XPG4	G, O	G, O	
<code>tzset()</code>	XPG4	G, O	G, O	
<code>ulimit()</code>	XPG4	G, O	G, O	
<code>umask()</code>	POSIX	G, O	G, O	
<code>uname()</code>	POSIX	G, O	G, O	
<code>ungetc()</code>	ISO	G, O	G, O	Parameters must match module
<code>ungetwc()</code>	XPG4	O	G, O	
<code>unlink()</code>	ISO	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>utime()</code>	POSIX	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>va_arg()</code>	ISO	G, O	G, O	
<code>va_end()</code>	ISO	G, O	G, O	
<code>va_start()</code>	ISO	G, O	G, O	
<code>vfprintf()</code>	ISO	G, O	G, O	Parameters must match module
<code>vfscanf()</code>	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>vprintf()</code>	ISO	G, O	G, O	Only Guardian modules or only OSS modules
<code>vscanf()</code>	ISO	G, O	G, O	
<code>vsprintf()</code>	ISO	G, O	G, O	
<code>vsscanf()</code>	ISO	G, O	G, O	
<code>wait()</code>	POSIX	O	O	OSS objects only
<code>waitpid()</code>	POSIX	O	O	OSS objects only
<code>wscat()</code>	XP4	G, O	G, O	
<code>wcschr()</code>	XP4	G, O	G, O	
<code>wcscmp()</code>	XP4	G, O	G, O	
<code>wscoll()</code>	XP4	G, O	G, O	
<code>wscpy()</code>	XP4	G, O	G, O	
<code>wscspn()</code>	XP4	G, O	G, O	
<code>wcsftime()</code>	XP4	G, O	G, O	
<code>wcslon()</code>	XP4	G, O	G, O	
<code>wcsncat()</code>	XP4	G, O	G, O	
<code>wcsncmp()</code>	XP4	G, O	G, O	
<code>wcsncpy()</code>	XP4	G, O	G, O	
<code>wcspbrk()</code>	XP4	G, O	G, O	
<code>wcsrchr()</code>	XP4	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
wcsspn()	XPG4	G, O	G, O	
wcstod()	XPG4	G, O	G, O	
wcstok()	XPG4	G, O	G, O	
wcstol()	XPG4	G, O	G, O	
wcstoll()	XPG4	G, O	G, O	
wcstombs()	XPG4	G, O	G, O	
wcstoul()	XPG4	G, O	G, O	
wcstoull()	XPG4	G, O	G, O	
wcswcs()	XPG4	G, O	G, O	
wcswidth()	XPG4	G, O	G, O	
wcsxfrm()	XPG4	G, O	G, O	
wctomb()	XPG4	G, O	G, O	
wctype()	XPG4	G, O	G, O	
wcwidth()	XPG4	G, O	G, O	
wmemchr()	ISO	G, O	G, O	Checking variants
wmemcmp()	ISO	G, O	G, O	Checking variants
wmemcpy()	ISO	G, O	G, O	Checking variants
wmemmove()	ISO	G, O	G, O	Checking variants
wmemset()	ISO	G, O	G, O	Checking variants
wordexp()	POSIX	G, O	G, O	
wordfree()	POSIX	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>write()</code>	XPG4	G, O	G, O	Parameters must match module Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>writeread()</code>	HPE	G	G, O	Guardian parameters only
<code>y0()</code>	XPG4	G, O	G, O	
<code>y1()</code>	XPG4	G, O	G, O	
<code>yn()</code>	XPG4	G, O	G, O	



API Interoperability in Native Processes

This appendix contains the API Interoperability tables, which describe the interoperability between the Guardian API and the OSS API in TNS/R, TNS/E, and TNS/X native processes.

These tables list the functions and macros in the native C Run-Time Library and the OSS API. Functions in the Guardian sockets library and Guardian system procedures are not included.

The tables indicate the following characteristics for each function or macro:

- Modules it can be called from
- Processes it can be called from
- Parameters it requires
- Objects it can operate on

Comparing API Interoperability in TNS and Native Processes

The native C run-time library supports both the OSS and Guardian environments. Having a single library for functions that are part of both APIs means that most differences in behavior that exist in TNS processes do not exist in native processes.

For example, in TNS processes the `signal()` function can be called only from a Guardian module within a Guardian process or from an OSS module within an OSS process. The Guardian function operates only on Guardian objects, and the OSS function operates only on OSS objects. None of these differences in behavior of the `signal()` function exist in native processes.

There are still some functions, however, such as `fopen()` and `rename()`, that have environment-specific behavior and require environment-specific parameters.

In addition, for the native C run-time library:

- Eight functions that require environment-specific parameters have environment-specific variants. These functions are `fopen()`, `fopen64()`, `freopen()`, `freopen64()`, `remove()`, `rename()`, `tmpnam()`, and `tmpfile()`. The environment-specific variants of these functions have suffixes, such as `fopen_guardian()` and `fopen_oss()`. For example, you can use `fopen_oss()` to open an OSS file from within a Guardian process.
- There are no separate Guardian variants of functions that operate on file descriptors instead of streams. For example, the `open()` and `read()` functions can be accessed from both the Guardian and OSS environments, but always operate on OSS file descriptors.
- Some functions that are specified by Hewlett Packard Enterprise have been eliminated, such as `readupdate()` and `edlseek()`.
- Some functions, such as `erf()` and `strdup()`, have been added to the native library for compliance to the XPG4 specification.

For a detailed description of the differences between the TNS and TNS/R, TNS/E, or TNS/X native C run-time libraries, see the *TNS/R Native Application Migration Guide*, *TNS/E Native Application Conversion Guide*, or the *TNS/X Native Application Conversion Guide*.

Calling Functions and Macros

The two factors that determine when you can call a function or macro are:



- The environment for which the module containing the call was compiled.
- The environment of the process executing the module.

An OSS module is a module compiled to have OSS semantics and execute in the OSS environment. Likewise, a Guardian module is a module compiled to have Guardian semantics and execute in the Guardian environment. Normally, the environments of a module and the process executing the module are the same.

You can mix modules of different environments in the same process. An OSS process can execute both OSS and Guardian modules, and a Guardian process can execute both Guardian and OSS modules. See **Overview of the OSS Programming Environment** for a discussion of when to write mixed-module applications.

Environment of the Module

The environment of a module is determined by the value of the `SYSTYPE` compiler pragma, which directs the compiler to generate code appropriate for the specified target execution environment.

Environment of the Process

The environment of a process is determined by the environment of the module defining the `main()` function.

Most functions and macros in the Guardian and OSS APIs can be called from a Guardian or OSS module, which in turn can be part of a Guardian or OSS process. For example, the `system()` function can be called only from a Guardian module in a Guardian process or from an OSS module in an OSS process.

If you attempt to call the `system()` function in a module compiled for the Guardian environment, and then link this module to a `main()` module compiled for the OSS environment (an OSS process), you will get a run-time error. This is because the system resources and services required by the Guardian variant function cannot be provided by the OSS process environment.

Some functions, such as `fopen()`, require environment-specific parameters. For example, if you call the `fopen()` function with an OSS pathname in a Guardian process, the type of the parameter does not match the type of the process, and a run-time error is generated.

Function and Macro Behavior

A function or macro that can be called in both the Guardian and OSS environments might behave the same or differently in each environment. In general, behavior is identical when function parameters are environment independent, and it is different when function parameters are environment specific.

Functions With Environment-Specific Parameters

The `fopen()`, `fopen64()`, `freopen()`, `freopen64()`, `remove()`, `rename()`, `tmpnam()`, and `tmpfile()` functions require that the type of the parameters match the type of the process. For example, in an OSS process, the file to be operated on must be indicated by an OSS pathname. (Remember that files in `/G` have OSS pathnames.) In a Guardian process, the file to be operated on must be indicated by a Guardian filename.

Note that the parameter type need not match the process type in the environment-specific variants of these functions. For example, the `fopen_guardian()` function opens a Guardian file from within an OSS or Guardian process.

Functions With Environment-Independent Parameters

Most of the functions and macros available in both the Guardian and OSS environments behave the same, regardless of the environment, because they take environment-independent parameters. For example, string functions such as `strcat()` are identical in the Guardian and OSS environments. The `malloc()` function can allocate a block of memory in a Guardian module and the `free()` function can free the same block of memory in an OSS module.



Internationalization Functions

In Guardian processes, regardless of whether modules are compiled to run in the Guardian or OSS environment, only the C/POSIX locale is available. In OSS processes, full internationalization support is available if the required locales are purchased and installed on the system.

Using the API Interoperability Tables

Some functions and macros in the Guardian and OSS APIs can interoperate completely between the Guardian and OSS environments: that is, they can be called from either a Guardian or OSS module, can execute as part of a Guardian or OSS process, and can operate on Guardian or OSS objects. Some functions and macros are more limited in their interoperability between environments.

For each function or macro listed in the API Interoperability tables, the level of interoperability it provides is indicated by the following information:

- Specification or standard where the function or macro is defined.
- Whether the function or macro can be called from a Guardian module, an OSS module, a Guardian process, or an OSS process.
- Types of parameters (OSS or Guardian) the function or macro requires.
- Types of objects (OSS or Guardian) the function or macro operates on.

G and O Notation

In the columns headed “Module Compiled As,” and “Process Called From,” the letter “G” indicates “Guardian” and “O” indicates “OSS.”

Specified By

The “Specified By” column indicates where a function or macro is defined:

FC-MI

The function is specified in the Fibre Channel - Methodologies for Interconnects (FC-MI) Revision 1.6 Annexure A by the Storage Network Industry Association (SNIA).

ISO

The function or macro is specified in the ISO/ANSI C standard.

POSIX

The function or macro is specified in the POSIX.1 or POSIX.2 standard. (Functions identified as “ISO” are also specified in the POSIX.1 standard.)

HPE

The function or macro is an extension specified by Hewlett Packard Enterprise.

Threads

The function is part of the Standard POSIX Threads and is specified in the International Standard ISO/IEC 9945-1; 1996 (E) IEEE Std 1003.1, 1996 (Incorporating ANSI/IEEE Std 1003.1-1990, 1003.1b-1993, 1003.1c-1995, and 1003.1i-1995), or

The function is part of the POSIX User Thread Model and is specified in the IEEE Std 1003.1, 2004.

X/OPEN

The function or macro is specified in the XPG4 version 2 specification, which is the basis for the X/OPEN UNIX brand.



XLFS

The function or macro is a Hewlett Packard Enterprise extension to the XPG4 specification based on the *Adding Support for Arbitrary File Sizes to the Single Unix Specification*, X/Open Large File Summit (LFS) submission, version 1.5 (March 20, 1996). This function or macro is supported on systems running G06.29 or later G-series RVUs or H06.06 or later H-series RVUs.

XPG4

The function or macro is specified in the XPG4 specification. (Functions and macros identified as “ISO” or “POSIX” are also specified in the XPG4 specification.)

Functions and macros specified in the ISO standard are declared for both Guardian and OSS modules.

By default, functions and macros specified in POSIX, XPG4, and X/OPEN UNIX are declared only for OSS modules. To use such a function or macro in a Guardian module (if that is allowed), specify the `_XOPEN_SOURCE` feature test macro when you compile the module.

By default, functions and macros specified by Hewlett Packard Enterprise are declared only for Guardian modules. To use such a function or macro in an OSS module (if that is allowed), specify the `_TANDEM_SOURCE` feature test macro when you compile the module.

Regardless of whether the function or macro is being called from a Guardian or OSS module, some functions and macros specified by X/OPEN UNIX require that the `_XOPEN_SOURCE_EXTENDED` feature test macro be specified when you compile the module, and some require that either the `_XOPEN_SOURCE_EXTENDED` or the `_TANDEM_SOURCE` feature test macro be specified when you compile the module. `_XOPEN_SOURCE_EXTENDED` must be defined as the constant 1.

Defining Parameters and Objects

OSS parameters include OSS pathnames, stream pointers, file descriptors, and process identifiers. Guardian parameters include Guardian filenames, stream pointers, and process identifiers. OSS objects include OSS files, streams, and processes. Guardian objects include Guardian files, streams, and processes.

In some cases, OSS parameters can be used to access Guardian objects. For example, an OSS pathname parameter can identify a Guardian file using the `/G/volume/subvolume/fileid` syntax.

Usage Notes

The “Usage Notes” column provides information about the behavior of the function or macro. Unless indicated otherwise in the “Usage Notes” column, a function or macro has the same behavior regardless of the module or process environment from which it is called.

The following table describes the “Usage Notes” values that appear frequently.

Usage Note	Meaning
Guardian function operates only on Guardian objects	The Guardian function or macro operates only on Guardian objects.
Guardian objects only	The function or macro operates only on Guardian objects.
Guardian parameters only	The function or macro requires Guardian parameters (Guardian filename, stream pointer, or process identifier).

Table Continued



Usage Note	Meaning
<code>IEEE_float</code> only	The function or macro is only implemented for the <code>IEEE_float</code> floating point option. Application using this function or macro cannot be compiled if the specified floating-point option is <code>Tandem_float</code> .
<code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>	An application can explicitly call this function when you use the <code>#define _LARGEFILE64_SOURCE 1</code> feature test macro or an equivalent compiler command option to compile the application. An application call to <code>interface ()</code> is automatically mapped to this <code>interface64 ()</code> function when you use the <code>#define _FILE_OFFSET_BITS 64</code> feature test macro or an equivalent compiler command option to compile the application.
Limited OSS objects only	The function or macro operates on some types of OSS objects. The extent of support for OSS objects differs for each function or macro.
Limited when called from Guardian process	The function or macro's operation is limited when it is called from a Guardian process.
Only Guardian modules or only OSS modules	The function or macro can be called only from a Guardian module within a Guardian process or from an OSS module within an OSS process. The function or macro cannot be called from both Guardian and OSS modules within the same process. The module and process environments must be the same. Behavior might be different in each environment.
OSS function operates on OSS objects and limited Guardian objects	The OSS function or macro operates on OSS objects and some types of Guardian objects. The extent of support for Guardian objects differs for each function or macro.
OSS objects and limited Guardian objects	The function or macro operates on OSS objects and some types of Guardian objects. The extent of support for Guardian objects differs for each function or macro.
OSS objects only	The function or macro operates only on OSS objects.
OSS parameters only	The function or macro requires OSS parameters (OSS pathname, stream pointer, file descriptor, or process identifier).

Table Continued



Usage Note	Meaning
Parameters must match module	When called from a Guardian module, the function or macro requires Guardian parameters. When called from an OSS module, the function or macro requires OSS parameters. If the function or macro operates on an open file, the file must have been opened in a module of the same environment.
Parameters must match process	When called from a Guardian process, the function or macro requires Guardian parameters. When called from an OSS process, the function or macro requires OSS parameters. If the function or macro operates on an open file, the file must have been opened in a module of the same environment.
<code>_PUT_MODEL_</code>	To use this function with the PUT library on systems running J06.10 or later RVUs or H06.21 or later RVUs, the feature test macro <code>_PUT_MODEL_</code> must be specified when you compile the application.
<code>_PUT_MODEL_</code> or <code>_SPT_MODEL_</code>	To use this function with the PUT library on systems running J06.10 or later RVUs or H06.21 or later RVUs, the feature test macro <code>_PUT_MODEL_</code> must be specified when you compile the application, or to use this function with the SPT library on systems running J06.10 or later RVUs or H06.21 or later RVUs, the feature test macro <code>_SPT_MODEL_</code> must be specified when you compile the application.
<code>_SPT_MODEL_</code>	To use this function with the SPT library on systems running J06.10 or later RVUs or H06.21 or later RVUs, the feature test macro <code>_SPT_MODEL_</code> must be specified when you compile the application.
<code>SPT_THREAD_AWARE</code> or <code>SPT_THREAD_AWARE_PRAGMA</code>	An application call to <code>interface()</code> is automatically mapped to this <code>spt_interface()</code> function when you use the <code>#define SPT_THREAD_AWARE</code> feature test macro (for C applications) or the <code>#define SPT_THREAD_AWARE_PRAGMA</code> (for C++ applications) or an equivalent compiler command option to compile the application.

Table Continued

Usage Note	Meaning
SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK	An application call to <i>interface()</i> is automatically mapped to this <i>spt_interfacex()</i> function when you use the #define SPT_THREAD_AWARE_NONBLOCK feature test macro (for C applications) or the #define SPT_THREAD_AWARE_PRAGMA_NONBLOCK (for C++ applications) or an equivalent compiler command option to compile the application.
SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK	An application call to <i>interface()</i> is automatically mapped to this <i>spt_interfacez()</i> function when you use the #define SPT_THREAD_AWARE_XNONBLOCK feature test macro (for C applications) or the #define SPT_THREAD_AWARE_PRAGMA_XNONBLOCK (for C++ applications) or an equivalent compiler command option to compile the application. If a function has a corresponding <i>spt_*x()</i> function but not a corresponding <i>spt_*z()</i> function, the function is mapped to the corresponding <i>spt_*x()</i> function.
SPT_THREAD_AWARE_SIGNAL	To enable thread-aware behavior for this function, the SPT_THREAD_AWARE_SIGNAL environmental variable must be exported to 1.
SPT_THREAD_SIGNAL or SPT_THREAD_SIGNAL_PRAGMA	An application call to <i>interface()</i> is automatically mapped to this <i>spt_interface()</i> function when you use the #define SPT_THREAD_SIGNAL feature test macro (for C applications) or the #define SPT_THREAD_SIGNAL_PRAGMA (for C++ applications) or an equivalent compiler command option to compile the application.
Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers	Beginning with the H06.24 and J06.13 RVUs, 32-bit processes using 64-bit pointers can call these functions on TNS/E systems and on TNS/X (L-series). These functions can also be called from 64-bit OSS processes on TNS/E and TNS/X systems.
Support for TNS/E and TNS/X 64-bit OSS callers	Beginning with the H06.24 and J06.13 RVUs, 64-bit OSS processes can call these functions on TNS/E systems (and TNS/X systems on L-series).

Table Continued

Usage Note	Meaning
<code>_XOPEN_SOURCE_EXTENDED</code>	The function or macro requires that the feature test macro <code>_XOPEN_SOURCE_EXTENDED</code> be specified when you compile the module.
<code>_XOPEN_SOURCE_EXTENDED</code> or <code>_TANDEM_SOURCE</code>	The function or macro requires that the feature test macro <code>_XOPEN_SOURCE_EXTENDED</code> or <code>_TANDEM_SOURCE</code> be specified when you compile the module.

Examples

The following examples illustrate how to use **API Interoperability in Native Processes** to determine how functions and macros can be used in Guardian and OSS programs.

Procedure

1. The following information can be derived about the `edfseek()` function:
 - a. Is an extension specified by Hewlett Packard Enterprise.
 - b. Can be called only from a Guardian module.
 - c. Can be called from Guardian and OSS processes.
 - d. Requires Guardian parameters.
 - e. Operates only on Guardian objects.
 - f. Because it is specified by Hewlett Packard Enterprise, it requires the `_TANDEM_SOURCE` feature test macro.
2. The following information can be derived about the `free()` function:
 - a. Specified in the ISO/ANSI C standard.
 - b. Can be called from Guardian and OSS modules.
 - c. Can be called from Guardian and OSS processes.
3. The following information can be derived about the `rename()` function:
 - a. Specified in the ISO/ANSI C standard.
 - b. Can be called from Guardian and OSS modules.
 - c. Can be called from Guardian and OSS processes.
 - d. Guardian function operates only on Guardian objects; OSS function operates on OSS objects and limited Guardian objects.
4. The following information can be derived about the `rename_oss()` function:
 - a. Is an extension specified by Hewlett Packard Enterprise.
 - b. Can be called from Guardian or OSS modules.
 - c. Can be called from Guardian or OSS processes.



- d. Must be called with OSS parameters.
- e. Operates on OSS objects and limited Guardian objects.
- f. Because it is specified by Hewlett Packard Enterprise, it requires the `_TANDEM_SOURCE` feature test macro.

Table 79: API Interoperability in Native Processes (Functions a through m)

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>a64l()</code>	XPG4	G, O	G, O	
<code>abort()</code>	ISO	G, O	G, O	
<code>_abs()</code>	HPE	G, O	G, O	
<code>abs()</code>	ISO	G, O	G, O	
<code>accept()</code>	XPG4	O	O	<code>_PUT_MODEL_</code> <code>_XOPEN_SOURCE_EXTENDED</code>
<code>access()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>acl()</code>	HPE	G, O	G, O	Available on Version 3 OSS filesets only
<code>aclsort()</code>	HPE	G, O	G, O	Available on Version 3 OSS filesets only
<code>acos()</code>	ISO	G, O	G, O	
<code>acosf()</code>	ISO	G, O	G, O	
<code>acosh()</code>	ISO	G, O	G, O	
<code>acoshf()</code>	ISO	G, O	G, O	
<code>acoshl()</code>	ISO	G, O	G, O	
<code>acosl()</code>	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>advance()</code>				The functionality provided by this macro is defined as obsolete in POSIX. Functions such as <code>regcomp()</code> and <code>regexexec()</code> should be used instead.
<code>alarm()</code>	POSIX	G, O	G, O	<code>_PUT_MODEL_</code>
<code>_alloca()</code>	HPE	G, O	G, O	
<code>asctime()</code>	ISO	G, O	G, O	
<code>asctime_r()</code>	Threads	G, O	G, O	<code>_SPT_MODEL_</code>
<code>asin()</code>	ISO	G, O	G, O	
<code>asinf()</code>	ISO	G, O	G, O	
<code>asinh()</code>	ISO	G, O	G, O	
<code>asinhf()</code>	ISO	G, O	G, O	
<code>asinh1()</code>	ISO	G, O	G, O	
<code>asinl()</code>	ISO	G, O	G, O	
<code>assert()</code>	ISO	G, O	G, O	
<code>atan()</code>	ISO	G, O	G, O	
<code>atanf()</code>	ISO	G, O	G, O	
<code>atanh()</code>	ISO	G, O	G, O	
<code>atanhf()</code>	ISO	G, O	G, O	
<code>atanh1()</code>	ISO	G, O	G, O	
<code>atanl()</code>	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
atan2()	ISO	G, O	G, O	
atan2f()	ISO	G, O	G, O	
atan2l()	ISO	G, O	G, O	
atexit()	ISO	G, O	G, O	
atof()	ISO	G, O	G, O	
atoi()	ISO	G, O	G, O	
atol()	ISO	G, O	G, O	
atoll()	ISO	G, O	G, O	
basename()	POSIX	O	O	
bcmp()	XPG4	G, O	G, O	
bcopy()	XPG4	G, O	G, O	
bind()	XPG4	O	O	_XOPEN_SOURCE_EXTENDED
bsearch()	ISO	G, O	G, O	
btowc()	ISO	G, O	G, O	
bzero()	XPG4	G, O	G, O	
cabs()	ISO	G, O	G, O	IEEE_float only
cabsf()	ISO	G, O	G, O	IEEE_float only
cabsl()	ISO	G, O	G, O	IEEE_float only
cacos()	ISO	G, O	G, O	IEEE_float only
cacosf()	ISO	G, O	G, O	IEEE_float only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>cacosh()</code>	ISO	G, O	G, O	IEEE_float only
<code>cacoshf()</code>	ISO	G, O	G, O	IEEE_float only
<code>cacoshl()</code>	ISO	G, O	G, O	IEEE_float only
<code>cacosl()</code>	ISO	G, O	G, O	IEEE_float only
<code>calloc()</code>	ISO	G, O	G, O	
<code>calloc32()</code>	HPE	G, O	G, O	Support for 64-bit OSS callers
<code>carg()</code>	ISO	G, O	G, O	IEEE_float only
<code>cargf()</code>	ISO	G, O	G, O	IEEE_float only
<code>cargl()</code>	ISO	G, O	G, O	IEEE_float only
<code>casin()</code>	ISO	G, O	G, O	IEEE_float only
<code>casinf()</code>	ISO	G, O	G, O	IEEE_float only
<code>casinh()</code>	ISO	G, O	G, O	IEEE_float only
<code>casinhf()</code>	ISO	G, O	G, O	IEEE_float only
<code>casinhl()</code>	ISO	G, O	G, O	IEEE_float only
<code>casinl()</code>	ISO	G, O	G, O	IEEE_float only
<code>catan()</code>	ISO	G, O	G, O	IEEE_float only
<code>catanf()</code>	ISO	G, O	G, O	IEEE_float only
<code>catanh()</code>	ISO	G, O	G, O	IEEE_float only
<code>catanhf()</code>	ISO	G, O	G, O	IEEE_float only
<code>catanhl()</code>	ISO	G, O	G, O	IEEE_float only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
catanl()	ISO	G, O	G, O	IEEE_float only
catclose()	XPG4	O	O	OSS parameters only
catgets()	XPG4	O	O	OSS parameters only
catopen()	XPG4	O	O	OSS parameters only
cbrt()	XPG4	G, O	G, O	
cbrtf()	ISO	G, O	G, O	
cbrt1()	ISO	G, O	G, O	
ccos()	ISO	G, O	G, O	IEEE_float only
ccosf()	ISO	G, O	G, O	IEEE_float only
ccosh()	ISO	G, O	G, O	IEEE_float only
ccoshf()	ISO	G, O	G, O	IEEE_float only
ccosh1()	ISO	G, O	G, O	IEEE_float only
ccosl()	ISO	G, O	G, O	IEEE_float only
ceil()	ISO	G, O	G, O	
ceilf()	ISO	G, O	G, O	
ceil1()	ISO	G, O	G, O	
cexp()	ISO	G, O	G, O	IEEE_float only
cexpf()	ISO	G, O	G, O	IEEE_float only
cexpl()	ISO	G, O	G, O	IEEE_float only
cfgetispeed()	POSIX	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>cfgetospeed()</code>	POSIX	G, O	G, O	
<code>cfsetispeed()</code>	POSIX	G, O	G, O	
<code>cfsetospeed()</code>	POSIX	G, O	G, O	
<code>chdir()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>chmod()</code>	POSIX	G, O	G, O	OSS objects only
<code>chown()</code>	POSIX	G, O	G, O	OSS objects and limited Guardian objects
<code>chroot()</code>	XPG4	G, O	G, O	
<code>chvol()</code>	HPE	G	G, O	
<code>cimag()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>cimagf()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>cimagl()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>clearerr()</code>	ISO	G, O	G, O	
<code>clock()</code>	ISO	G, O	G, O	
<code>clog()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>clogf()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>clogl()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>close()</code>	XPG4	O	O	OSS parameters only OSS objects and limited Guardian objects <code>_PUT_MODEL_</code>

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>closedir()</code>	POSIX	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>closelog()</code>	X/OPEN	G, O	G, O	
<code>compile()</code>				The functionality provided by this macro is defined as obsolete in POSIX. Functions such as <code>regcomp()</code> and <code>regexexec()</code> should be used instead.
<code>confstr()</code>	XPG4	G, O	G, O	
<code>conj()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>conjf()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>conjl()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>connect()</code>	XPG4	O	O	<code>_PUT_MODEL_</code> <code>_XOPEN_SOURCE_EXTENDED</code>
<code>copysign()</code>	ISO	G, O	G, O	
<code>copysignf()</code>	ISO	G, O	G, O	
<code>copysignl()</code>	ISO	G, O	G, O	
<code>cos()</code>	ISO	G, O	G, O	
<code>cosf()</code>	ISO	G, O	G, O	
<code>cosh()</code>	ISO	G, O	G, O	
<code>coshf()</code>	ISO	G, O	G, O	
<code>coshl()</code>	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>cosl()</code>	ISO	G, O	G, O	
<code>cpow()</code>	ISO	G, O	G, O	IEEE_float only
<code>cpowf()</code>	ISO	G, O	G, O	IEEE_float only
<code>cpowl()</code>	ISO	G, O	G, O	IEEE_float only
<code>cproj()</code>	ISO	G, O	G, O	IEEE_float only
<code>cprojf()</code>	ISO	G, O	G, O	IEEE_float only
<code>cprojl()</code>	ISO	G, O	G, O	IEEE_float only
<code>creal()</code>	ISO	G, O	G, O	IEEE_float only
<code>crealf()</code>	ISO	G, O	G, O	IEEE_float only
<code>creall()</code>	ISO	G, O	G, O	IEEE_float only
<code>creat()</code>	XPG4	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>creat64()</code>	XLFS	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>crypt()</code>	XPG4	G, O	G, O	
<code>csin()</code>	ISO	G, O	G, O	IEEE_float only
<code>csinf()</code>	ISO	G, O	G, O	IEEE_float only
<code>csinh()</code>	ISO	G, O	G, O	IEEE_float only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>csinhf()</code>	ISO	G, O	G, O	IEEE_float only
<code>csinhl()</code>	ISO	G, O	G, O	IEEE_float only
<code>csinl()</code>	ISO	G, O	G, O	IEEE_float only
<code>csqrt()</code>	ISO	G, O	G, O	
<code>csqrtf()</code>	ISO	G, O	G, O	
<code>csqrtl()</code>	ISO	G, O	G, O	
<code>ctan()</code>	ISO	G, O	G, O	IEEE_float only
<code>ctanf()</code>	ISO	G, O	G, O	IEEE_float only
<code>ctanh()</code>	ISO	G, O	G, O	IEEE_float only
<code>ctanhf()</code>	ISO	G, O	G, O	IEEE_float only
<code>ctanhl()</code>	ISO	G, O	G, O	IEEE_float only
<code>ctanl()</code>	ISO	G, O	G, O	IEEE_float only
<code>ctermid()</code>	POSIX	O	O	OSS parameters only
<code>ctermid_r()</code>	Threads	O	O	OSS parameters only _SPT_MODEL_
<code>ctime()</code>	ISO	G, O	G, O	
<code>ctime_r()</code>	Threads	G, O	G, O	_SPT_MODEL_
<code>cuserid()</code>	XPG4	G, O	G, O	
<code>dbm_clearerr()</code>	XPG4	O	O	
<code>dbm_close()</code>	XPG4	O	O	
<code>dbm_delete()</code>	XPG4	O	O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
dbm_error()	XPG4	O	O	
dbm_fetch()	XPG4	O	O	
dbm_firstkey()	XPG4	O	O	
dbm_nextkey()	XPG4	O	O	
dbm_open()	XPG4	O	O	
dbm_store	XPG4	O	O	
dec_to_longlong()	HPE	G, O	G, O	
difftime()	ISO	G, O	G, O	
dirname()	POSIX	O	O	
dirfd()	POSIX	G, O	G, O	
div()	ISO	G, O	G, O	
dlclose()	HPE	G, O	G, O	
dLError()	HPE	G, O	G, O	
dlopen()	HPE	G, O	G, O	
dlresultcode()	HPE	G, O	G, O	
dlsym()	HPE	G, O	G, O	
drand48()	XPG4	G, O	G, O	
dup()	POSIX	O	O	OSS parameters only OSS objects and limited Guardian objects

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
dup2 ()	POSIX	O	O	OSS parameters only OSS objects and limited Guardian objects _PUT_MODEL_
ecvt ()	HPE	G, O	G, O	Data can be overwritten by a subsequent call
edfseek ()	HPE	G	G, O	Guardian parameters only Guardian objects only
edftell ()	HPE	G	G, O	Guardian parameters only Guardian objects only
encrypt ()	XPG4	G, O	G, O	
endgrent ()	POSIX	G, O	G, O	
endhostent ()	XPG4	O	O	
endnetent ()	XPG4	O	O	
endprotoent ()	XPG4	O	O	
endpwent ()	X/OPEN	G, O	G, O	_XOPEN_SOURCE_EXTENDED or _TANDEM_SOURCE
endservent ()	XPG4	O	O	
erand48 ()	XPG4	G, O	G, O	
erf ()	XPG4	G, O	G, O	
erfc ()	XPG4	G, O	G, O	
erfcf ()	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>erfc1()</code>	ISO	G, O	G, O	
<code>erff()</code>	ISO	G, O	G, O	
<code>erfl()</code>	ISO	G, O	G, O	
<code>exec()</code>	POSIX	O	O	OSS objects only
<code>execl()</code>	POSIX	O	O	OSS objects only
<code>execle()</code>	POSIX	O	O	OSS objects only
<code>execvp()</code>	POSIX	O	O	OSS objects only
<code>execv()</code>	POSIX	O	O	OSS objects only
<code>execve()</code>	POSIX	O	O	OSS objects only
<code>fexecve()</code>	POSIX	O	O	OSS objects only
<code>execvp()</code>	POSIX	O	O	OSS objects only
<code>_exit()</code> <code>_Exit()</code>	POSIX	G, O	G, O	Nowait considerations when called from the Guardian API
<code>exit()</code>	ISO	G, O	G, O	
<code>exp()</code>	ISO	G, O	G, O	
<code>expf()</code>	ISO	G, O	G, O	
<code>expl()</code>	ISO	G, O	G, O	
<code>expml()</code>	XPG4	G, O	G, O	
<code>expm1f()</code>	ISO	G, O	G, O	
<code>expm1l()</code>	ISO	G, O	G, O	
<code>exp2()</code>	ISO	G, O	G, O	IEEE_float only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>exp2f()</code>	ISO	G, O	G, O	IEEE_float only
<code>exp2l()</code>	ISO	G, O	G, O	IEEE_float only
<code>fabs()</code>	ISO	G, O	G, O	
<code>fabsf()</code>	ISO	G, O	G, O	
<code>fabsl()</code>	ISO	G, O	G, O	
<code>faccessat()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>fchdir()</code>	POSIX	G, O	G, O	OSS objects and limited Guardian objects
<code>fchmod()</code>	POSIX	G, O	G, O	
<code>fchmodat()</code>	POSIX	G, O	G, O	OSS objects only
<code>fchown()</code>	POSIX	G, O	G, O	OSS objects and limited Guardian objects
<code>fchownat()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>fclose()</code>	ISO	G, O	G, O	
<code>fcntl()</code>	XPG4	O	O	OSS parameters only OSS objects only _PUT_MODEL_
<code>fcvt()</code>	XPG4	G, O	G, O	
<code>fdim()</code>	ISO	G, O	G, O	IEEE_float only
<code>fdimf()</code>	ISO	G, O	G, O	IEEE_float only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>fdiml()</code>	ISO	G, O	G, O	IEEE_float only
<code>fdopen()</code>	XPG4	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>feclearexcept()</code>	ISO	G, O	G, O	IEEE_float only
<code>fegetenv()</code>	ISO	G, O	G, O	IEEE_float only
<code>fegetexceptflag()</code>	ISO	G, O	G, O	IEEE_float only
<code>fegetround()</code>	ISO	G, O	G, O	IEEE_float only
<code>fehldexcept()</code>	ISO	G, O	G, O	IEEE_float only
<code>feof()</code>	ISO	G, O	G, O	
<code>feraiseexcept()</code>	ISO	G, O	G, O	IEEE_float only
<code>ferror()</code>	ISO	G, O	G, O	
<code>fesetenv()</code>	ISO	G, O	G, O	IEEE_float only
<code>fesetexceptflag()</code>	ISO	G, O	G, O	IEEE_float only
<code>fesetround()</code>	ISO	G, O	G, O	IEEE_float only
<code>fetestexcept()</code>	ISO	G, O	G, O	IEEE_float only
<code>feupdateenv()</code>	ISO	G, O	G, O	IEEE_float only
<code>fflush()</code>	ISO	G, O	G, O	
<code>ffs()</code>	XPG4	G, O	G, O	
<code>fgetc()</code>	ISO	G, O	G, O	
<code>fgetpos()</code>	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>fgetpos64()</code>	XLFS	G, O	G, O	<code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>
<code>fgets()</code>	ISO	G, O	G, O	
<code>fgetwc()</code>	XPG4	G, O	G, O	
<code>fgetws()</code>	XPG4	G, O	G, O	
<code>fileno()</code>	XPG4	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>finite()</code>	XPG4	G, O	G, O	
<code>flockfile()</code>	Threads	G, O	G, O	
<code>floor()</code>	ISO	G, O	G, O	
<code>floorf()</code>	ISO	G, O	G, O	
<code>floorl()</code>	ISO	G, O	G, O	
<code>fma()</code>	ISO	G, O	G, O	IEEE_float only
<code>fmaf()</code>	ISO	G, O	G, O	IEEE_float only
<code>fmal()</code>	ISO	G, O	G, O	IEEE_float only
<code>fmax()</code>	ISO	G, O	G, O	IEEE_float only
<code>fmaxf()</code>	ISO	G, O	G, O	IEEE_float only
<code>fmaxl()</code>	ISO	G, O	G, O	IEEE_float only
<code>fmin()</code>	ISO	G, O	G, O	IEEE_float only
<code>fminf()</code>	ISO	G, O	G, O	IEEE_float only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>fminl()</code>	ISO	G, O	G, O	IEEE_float only
<code>fmod()</code>	ISO	G, O	G, O	
<code>fmodf()</code>	ISO	G, O	G, O	
<code>fmodl()</code>	ISO	G, O	G, O	
<code>fmtmsg()</code>	XPG4	O	O	
<code>fnmatch()</code>	POSIX	O	O	
<code>fopen()</code>	ISO	G, O	G, O	Parameters must match process Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>fopendir()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>fopen_guardian()</code>	HPE	G, O	G, O	Guardian parameters only Guardian objects only
<code>fopen_oss()</code>	HPE	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>fopen_std_file()</code>	HPE	G	G	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>fopen64()</code>	XLFS	G, O	G, O	Parameters must match process Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects <code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>
<code>fopen64_guardian()</code>	XLFS	G, O	G, O	Guardian parameters only Guardian objects only <code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>
<code>fopen64_oss()</code>	XLFS	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects <code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>
<code>fopen64_std_file()</code>	XLFS	G	G	<code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>
<code>fork()</code>	POSIX	O	O	OSS objects only <code>_PUT_MODEL_</code>
<code>fp_class()</code>	HPE	G, O	G, O	Hewlett Packard Enterprise extension to XPG4
<code>fpathconf()</code>	POSIX	G, O	G, O	OSS parameters only
<code>fpclassify()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>fprintf()</code>	ISO	G, O	G, O	
<code>fputc()</code>	ISO	G, O	G, O	
<code>fputs()</code>	ISO	G, O	G, O	
<code>fputwc()</code>	XPG4	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>fputws()</code>	XPG4	G, O	G, O	
<code>fread()</code>	ISO	G, O	G, O	
<code>free()</code>	ISO	G, O	G, O	
<code>free32()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X 64-bit OSS callers
<code>freeaddrinfo()</code>	XPG4	O	O	
<code>freehostent()</code>	XPG4	O	O	
<code>freopen()</code>	ISO	G, O	G, O	Parameters must match process Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>freopen_guardian()</code>	HPE	G, O	G, O	Guardian parameters only Guardian objects only
<code>freopen_oss()</code>	HPE	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>freopen64()</code>	XLFS	G, O	G, O	Parameters must match process Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects objects_LARGEFILE64_SOURCE or _FILE_OFFSET_BITS

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>freopen64_guardian()</code>	XLFS	G, O	G, O	Guardian parameters only Guardian objects only _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>freopen64_oss()</code>	XLFS	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>frexp()</code>	ISO	G, O	G, O	
<code>frexpf()</code>	ISO	G, O	G, O	
<code>frexpl()</code>	ISO	G, O	G, O	
<code>fscanf()</code>	ISO	G, O	G, O	
<code>fseek()</code>	ISO	G, O	G, O	
<code>fseeko()</code>	XLFS	G, O	G, O	_LARGEFILE64_SOURCE
<code>fseeko64()</code>	XLFS	G, O	G, O	_LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>fsetpos()</code>	ISO	G, O	G, O	
<code>fsetpos64()</code>	XLFS	G, O	G, O	_LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>fstat()</code>	POSIX	O	O	OSS parameters only OSS objects and limited Guardian objects _PUT_MODEL_
<code>fstatat()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>fstatat64()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>fstat64()</code>	XLFS	O	O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS _PUT_MODEL_
<code>fstatvfs()</code>	XPG4	O	O	This function provides compatibility with the System V Interface Definition, Revision 3
<code>fstatvfs64()</code>	XLFS	O	O	This function provides compatibility with the System V Interface Definition, Revision 3 _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>fsync()</code>	XPG4	O	O	_PUT_MODEL_
<code>ftell()</code>	ISO	G, O	G, O	
<code>ftello()</code>	XLFS	G, O	G, O	_LARGEFILE64_SOURCE
<code>ftello64()</code>	XLFS	G, O	G, O	_LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>ftime()</code>	XPG4	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>ftok()</code>	X/OPEN	O	G, O	OSS parameters only OSS objects only _XOPEN_SOURCE_EXTEN DEDor _TANDEM_SOURCE
<code>ftruncate()</code>	XPG4	O	O	_PUT_MODEL_
<code>ftruncate64</code>	XLFS	O	O	_LARGEFILE64_SOURCE or _FILE_OFFSET_BITS _PUT_MODEL_
<code>ftrylockfile()</code>	Threads	G, O	G, O	
<code>ftw()</code>	XPG4	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>ftw64()</code>	XLFS	O	G, O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>funlockfile()</code>	Threads	G, O	G, O	
<code>fwide()</code>	ISO	G, O	G, O	
<code>fwprintf()</code>	ISO	G, O	G, O	
<code>fwrite()</code>	ISO	G, O	G, O	
<code>fwscanf()</code>	ISO	G, O	G, O	
<code>gai_strerror()</code>	XPG4	O	O	
<code>gamma()</code>	XPG4	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>gamma_r()</code>	Threads	G, O	G, O	<code>_SPT_MODEL_</code>
<code>gcvt()</code>	XPG4	G, O	G, O	
<code>get_assign_msg()</code>	HPE	G	G	
<code>get_assign_msg_by_name()</code>	HPE	G	G	
<code>get_max_assign_msg_ordinal()</code>	HPE	G	G	
<code>get_param_by_name()</code>	HPE	G	G	
<code>get_param_msg()</code>	HPE	G	G	
<code>get_startup_msg()</code>	HPE	G	G	
<code>getaddrinfo()</code>	XPG4	O	O	
<code>getc()</code>	ISO	G, O	G, O	
<code>getc_unlocked()</code>	Threads	G, O	G, O	
<code>getchar()</code>	ISO	G, O	G, O	
<code>getchar_unlocked()</code>	Threads	G, O	G, O	
<code>getcwd()</code>	POSIX	G, O	G, O	
<code>getdate()</code>	XPG4	G, O	G, O	
<code>getdtablesize()</code>	XPG4	O	O	
<code>getegid()</code>	POSIX	O	O	
<code>getenv()</code>	ISO	G, O	G, O	
<code>getenv_r()</code>	HPE	G, O	G, O	
<code>geteuid()</code>	POSIX	O	O	
<code>getgid()</code>	POSIX	O	O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>getgrent()</code>	POSIX	G, O	G, O	The functions <code>getgrnam()</code> and <code>getgrgid()</code> should be used instead of this function to ensure portability to future systems Data is overwritten by subsequent call or call to <code>getgrnam()</code>
<code>getgrent_r()</code>	Threads	G, O	G, O	The functions <code>getgrnam()</code> and <code>getgrgid()</code> should be used instead of this function to ensure portability to future systems Data is overwritten by subsequent call or call to <code>getgrnam()</code> <code>_SPT_MODEL_</code>
<code>getgrgid()</code>	POSIX	G, O	G, O	Data is overwritten by subsequent call or call to <code>getgrnam()</code>
<code>getgrgid_r()</code>	Threads	G, O	G, O	<code>_SPT_MODEL_</code>
<code>getgrnam()</code>	POSIX	G, O	G, O	Data is overwritten by subsequent call or call to <code>getgrgid()</code>
<code>getgrnam_r()</code>	Threads	G, O	G, O	<code>_SPT_MODEL_</code>
<code>getgroups()</code>	POSIX	O	O	
<code>gethostbyaddr()</code>	XPG4	O	O	
<code>gethostbyaddr_r()</code>	Threads	O	O	<code>_SPT_MODEL_</code>
<code>gethostbyname()</code>	XPG4	O	O	
<code>gethostbyname_r()</code>	Threads	O	O	<code>_SPT_MODEL_</code>

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
gethostbyname2 ()	HPE	O	O	
gethostent ()	XPG4	O	O	
gethostent_r ()	Threads	O	O	_SPT_MODEL_
gethostid ()	XPG4	O	O	
gethostname ()	XPG4	O	O	
getipnodebyaddr ()	HPE	O	O	
getipnodebyname ()	HPE	O	O	
getitimer ()	ISO	G, O	G, O	Only real-time timer is supported.
getlogin ()	POSIX	O	O	
getlogin_r ()	Threads	O	O	_SPT_MODEL_
getnameinfo ()	XPG4	O	O	
getnetbyaddr ()	XPG4	O	O	
getnetbyaddr_r ()	Threads	O	O	_SPT_MODEL_
getnetbyname ()	XPG4	O	O	
getnetbyname_r ()	HPE	O	O	_SPT_MODEL_
getnetent ()	XPG4	O	O	
getnetent_r ()	HPE	O	O	_SPT_MODEL_
getopt ()	XPG4	G, O	G, O	
getpagesize ()	ISO	O	O	
getpass ()	XPG4	O	O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
getpeername()	XPG4	O	O	_XOPEN_SOURCE_EXTENDED
getpgrp()	POSIX	O	O	OSS objects only
getpgid()	XPG4	O	O	OSS objects only
getpid()	POSIX	O	O	OSS objects only
getppid()	POSIX	O	O	OSS objects only
getpriority()	XPG4	O	O	OSS objects only
getprotobyname()	XPG4	O	O	
getprotobyname_r()	HPE	O	O	_SPT_MODEL_
getprotobynumber()	XPG4	O	O	
getprotobynumber_r()	HPE	O	O	_SPT_MODEL_
getprotoent()	XPG4	O	O	
getprotoent_r()	HPE	O	O	_SPT_MODEL_
getpwent()	X/OPEN	G, O	G, O	_TANDEM_SOURCE _XOPEN_SOURCE_EXTENDED
getpwent_r()	HPE	G, O	G, O	_SPT_MODEL_
getpwnam()	POSIX	G, O	G, O	
getpwnam_r()	Threads	G, O	G, O	_SPT_MODEL_
getpwuid()	POSIX	G, O	G, O	
getpwuid_r()	Threads	G, O	G, O	_SPT_MODEL_
gets()	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
getservbyname()	XPG4	O	O	
getservbyname_r()	HPE	O	O	_SPT_MODEL_
getservbyport()	XPG4	O	O	
getservbyport_r()	HPE	O	O	_SPT_MODEL_
getservent()	XPG4	O	O	
getservent_r()	HPE	O	O	_SPT_MODEL_
getsid()	XPG4	O	O	
getsockname()	XPG4	O	O	_XOPEN_SOURCE_EXTENDED
getsockopt()	XPG4	O	O	_XOPEN_SOURCE_EXTENDED
getsubopt()	XPG4	O	O	
gettimeofday()	XPG4	O	O	
getuid()	POSIX	O	O	
getw()	XPG4	O	G, O	OSS function operates on OSS objects and limited Guardian objects
getwc()	XPG4	G, O	G, O	
getwchar()	XPG4	G, O	G, O	
getwd()	XPG4	O	O	For portability to implementations conforming to earlier versions of the XPG4 specification, <code>getcwd()</code> is preferred over this function
getws()	XPG4	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>gfileno()</code>	HPE	G, O	G, O	Guardian parameters only
<code>glob()</code>	POSIX	G, O	G, O	
<code>globfree()</code>	POSIX	G, O	G, O	
<code>gmtime()</code>	ISO	G, O	G, O	
<code>gmtime_r()</code>	Threads	G, O	G, O	<code>_SPT_MODEL_</code>
<code>HBA_CloseAdapter()</code>	FC-MI	O	O	
<code>HBA_FreeLibrary()</code>	FC-MI	O	O	
<code>HBA_GetAdapterAttributes()</code>	FC-MI	O	O	
<code>HBA_GetAdapterName()</code>	FC-MI	O	O	
<code>HBA_GetAdapterPortAttributes()</code>	FC-MI	O	O	
<code>HBA_GetDiscoveredPortAttributes()</code>	FC-MI	O	O	
<code>HBA_GetFcpTargetMapping()</code>	FC-MI	O	O	
<code>HBA_GetNumberOfAdapters()</code>	FC-MI	O	O	
<code>HBA_GetPortStatistics()</code>	FC-MI	O	O	
<code>HBA_GetVersion()</code>	FC-MI	O	O	
<code>HBA_LoadLibrary()</code>	FC-MI	O	O	
<code>HBA_OpenAdapter()</code>	FC-MI	O	O	
<code>HBA_RefreshInformation()</code>	FC-MI	O	O	
<code>hcreate()</code>	XPG4	G, O	G, O	
<code>hdestroy()</code>	XPG4	G, O	G, O	
<code>heap_check()</code>	HPE	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
heap_check32()	HPE	G, O	G, O	Support for TNS/E and TNS/X 64-bit OSS callers
heap_check_always()	HPE	G, O	G, O	
heap_check_always32()	HPE	G, O	G, O	Support for TNS/E and TNS/X 64-bit OSS callers
heap_min_block_size()	HPE	G, O	G, O	
hsearch()	XPG4	G, O	G, O	
htonl()	XPG4	O	O	
htons()	XPG4	O	O	
hypot()	XPG4	G, O	G, O	
hypotf()	ISO	G, O	G, O	
hypotl()	ISO	G, O	G, O	
iconv()	XPG4	O	O	
iconv_close()	XPG4	O	O	
iconv_open()	XPG4	O	O	
if_freenameindex()	XPG4	O	O	
if_indextoname()	XPG4	O	O	
if_nameindex()	XPG4	O	O	
if_nametoindex()	XPG4	O	O	
ilogb()	XPG4	G, O	G, O	
ilogbf()	ISO	G, O	G, O	
ilogbl()	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>imaxabs()</code>	ISO	G, O	G, O	
<code>imaxdiv()</code>	ISO	G, O	G, O	
<code>index()</code>	XPG4	G, O	G, O	
<code>inet_addr()</code>	XPG4	O	O	
<code>inet_lnaof()</code>	XPG4	O	O	
<code>inet_makeaddr()</code>	XPG4	O	O	
<code>inet_netof()</code>	XPG4	O	O	
<code>inet_network()</code>	XPG4	O	O	
<code>inet_ntoa()</code>	XPG4	O	O	
<code>inet_ntop()</code>	XPG4	O	O	
<code>inet_pton()</code>	XPG4	O	O	
<code>initgroups()</code>	XPG4	O	O	
<code>initstate()</code>	XPG4	G, O	G, O	
<code>insque()</code>	XPG4	O	O	
<code>ioctl()</code>	X/OPEN	G, O	G, O	
<code>isalnum()</code>	ISO	G, O	G, O	
<code>isalpha()</code>	ISO	G, O	G, O	
<code>isascii()</code>	XPG4	G, O	G, O	
<code>isatty()</code>	POSIX	G, O	G, O	OSS parameters only
<code>isblank()</code>	ISO	G, O	G, O	
<code>iscntrl()</code>	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
isdigit()	ISO	G, O	G, O	
isfinite()	ISO	G, O	G, O	IEEE_float only
isgraph()	ISO	G, O	G, O	
isgreater()	ISO	G, O	G, O	IEEE_float only
isgreaterequal()	ISO	G, O	G, O	IEEE_float only
isinf()	ISO	G, O	G, O	IEEE_float only
isless()	ISO	G, O	G, O	IEEE_float only
islessequal()	ISO	G, O	G, O	IEEE_float only
islessgreater()	ISO	G, O	G, O	IEEE_float only
islower()	ISO	G, O	G, O	
isnan() function	ISO	G, O	G, O	Returns a value of 0 (zero) for Tandem floating-point data.
isnan() macro	ISO	G, O	G, O	IEEE_float only
isnormal()	ISO	G, O	G, O	IEEE_float only
isprint()	ISO	G, O	G, O	
ispunct()	ISO	G, O	G, O	
isspace()	ISO	G, O	G, O	
isunordered()	ISO	G, O	G, O	IEEE_float only
isupper()	ISO	G, O	G, O	
iswalnum()	XPG4	G, O	G, O	
iswalpha()	XPG4	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>iswblank()</code>	ISO	G, O	G, O	
<code>iswcntrl()</code>	XPG4	G, O	G, O	
<code>iswctype()</code>	XPG4	G, O	G, O	
<code>iswdigit()</code>	XPG4	G, O	G, O	
<code>iswgraph()</code>	XPG4	G, O	G, O	
<code>iswlower()</code>	XPG4	G, O	G, O	
<code>iswprint()</code>	XPG4	G, O	G, O	
<code>iswpunct()</code>	XPG4	G, O	G, O	
<code>iswspace()</code>	XPG4	G, O	G, O	
<code>iswupper()</code>	XPG4	G, O	G, O	
<code>iswxdigit()</code>	XPG4	G, O	G, O	
<code>isxdigit()</code>	XPG4	G, O	G, O	
<code>j0()</code>	XPG4	G, O	G, O	
<code>j1()</code>	XPG4	G, O	G, O	
<code>jn()</code>	XPG4	G, O	G, O	
<code>jrand48()</code>	XPG4	G, O	G, O	
<code>kill()</code>	POSIX	O	O	OSS objects only
<code>l64a()</code>	XPG4	G, O	G, O	
<code>labs()</code>	ISO	G, O	G, O	
<code>lchmod()</code>	HPE	G, O	G, O	OSS objects only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
lchown()	POSIX	G, O	G, O	OSS objects and limited Guardian objects
lcong48()	XPG4	G, O	G, O	
ldexp()	ISO	G, O	G, O	
ldexpf()	ISO	G, O	G, O	
ldexpl()	ISO	G, O	G, O	
ldiv()	ISO	G, O	G, O	
lfind()	XPG4	G, O	G, O	
lgamma()	XPG4	G, O	G, O	
lgammaf()	ISO	G, O	G, O	
lgammal()	ISO	G, O	G, O	
lgamma_r()	Threads	G, O	G, O	__SPT_MODEL__
link()	POSIX	O	G, O	OSS parameters only OSS objects only
linkat()	POSIX	G, O	G, O	OSS parameters only OSS objects only
listen()	XPG4	O	O	__XOPEN_SOURCE_EXTENDED
llabs()	ISO	G, O	G, O	
lldiv()	XPG4	G, O	G, O	
llrint()	ISO	G, O	G, O	IEEE_float only
llrintf()	ISO	G, O	G, O	IEEE_float only
llrintl()	ISO	G, O	G, O	IEEE_float only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
llround()	ISO	G, O	G, O	IEEE_float only
llroundf()	ISO	G, O	G, O	IEEE_float only
llroundl()	ISO	G, O	G, O	IEEE_float only
localeconv()	ISO	G, O	G, O	
localtime()	ISO	G, O	G, O	
localtime_r()	Threads	G, O	G, O	_SPT_MODEL_
log()	ISO	G, O	G, O	
logb()	XPG4	G, O	G, O	
logbf()	ISO	G, O	G, O	
logbl()	ISO	G, O	G, O	
logf()	ISO	G, O	G, O	
logl()	ISO	G, O	G, O	
loglp()	XPG4	G, O	G, O	
loglpf()	ISO	G, O	G, O	
loglpl()	ISO	G, O	G, O	
log2()	ISO	G, O	G, O	IEEE_float only
log2f()	ISO	G, O	G, O	IEEE_float only
log2l()	ISO	G, O	G, O	IEEE_float only
log10()	ISO	G, O	G, O	
log10f()	ISO	G, O	G, O	
log10l()	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>longjmp()</code>	ISO	G, O	G, O	
<code>longlong_to_dec()</code>	HPE	G, O	G, O	
<code>lrand48()</code>	XPG4	G, O	G, O	
<code>lrint()</code>	ISO	G, O	G, O	IEEE_float only
<code>lrintf()</code>	ISO	G, O	G, O	IEEE_float only
<code>lrintl()</code>	ISO	G, O	G, O	IEEE_float only
<code>lround()</code>	ISO	G, O	G, O	IEEE_float only
<code>lroundf()</code>	ISO	G, O	G, O	IEEE_float only
<code>lroundl()</code>	ISO	G, O	G, O	IEEE_float only
<code>lsearch()</code>	XPG4	G, O	G, O	
<code>lseek()</code>	XPG4	O	O	OSS parameters only OSS objects and Guardian objects _PUT_MODEL_
<code>lseek64()</code>	XLFS	O	O	OSS parameters only OSS objects and Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS _PUT_MODEL_
<code>lstat()</code>	X/OPEN	G, O	G, O	_XOPEN_SOURCE_EXTENDED

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
lstat64()	XLFS	G, O	G, O	<code>_XOPEN_SOURCE_EXTENDED</code> <code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>
lwres_freeaddrinfo()	HPE	O	O	
lwres_freehostent()	HPE	O	O	
lwres_gai_strerror()	HPE	O	O	
lwres_getaddrinfo()	HPE	O	O	
lwres_gethostbyaddr()	HPE	O	O	
lwres_gethostbyname()	HPE	O	O	
lwres_gethostbyname2()	HPE	O	O	
lwres_getipnodebyaddr()	HPE	O	O	
lwres_getipnodebyname()	HPE	O	O	
lwres_getnameinfo()	HPE	O	O	
lwres_hstrerror()	HPE	O	O	
malloc()	ISO	G, O	G, O	
malloc32()	HPE	G, O	G, O	Support for TNS/E and TNS/X 64-bit OSS callers
malloc_pages()	HPE	G, O	G, O	This function can be called by TNS/E and TNS/X native processes
_max()	HPE	G, O	G, O	
mblen()	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>mbrlen()</code>	ISO	G, O	G, O	
<code>mbrtowc()</code>	ISO	G, O	G, O	
<code>mbsinit()</code>	ISO	G, O	G, O	
<code>mbsrtowcs()</code>	ISO	G, O	G, O	
<code>mbstowcs()</code>	ISO	G, O	G, O	
<code>mbtowc()</code>	ISO	G, O	G, O	
<code>memccpy()</code>	ISO	G, O	G, O	
<code>memchr()</code>	ISO	G, O	G, O	
<code>memchr64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>memcmp()</code>	ISO	G, O	G, O	
<code>memcmp64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>memcpy()</code>	ISO	G, O	G, O	
<code>memcpy64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>memmove()</code>	ISO	G, O	G, O	
<code>memmove64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>memset()</code>	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>memset64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>_min()</code>	HPE	G, O	G, O	
<code>mkdir()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>mkdirat()</code>	POSIX	G, O	G, O	OSS objects and limited Guardian objects OSS objects only
<code>mkfifo()</code>	POSIX	G, O	G, O	OSS parameters only
<code>mkfifoat()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects only
<code>mknod()</code>	X/OPEN	G, O	G, O	<code>_XOPEN_SOURCE_EXTENDED</code>
<code>mknodat()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>mkstemp()</code>	XPG4	O	O	
<code>mktemp()</code>	XPG4	O	O	
<code>mktime()</code>	ISO	G, O	G, O	
<code>modf()</code>	ISO	G, O	G, O	
<code>modff()</code>	ISO	G, O	G, O	
<code>modfl()</code>	ISO	G, O	G, O	
<code>mrnd48()</code>	XPG4	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>msgctl()</code>	XPG4	O	O	OSS objects only
<code>msgget()</code>	XPG4	O	O	OSS objects only
<code>msgrcv()</code>	XPG4	O	O	OSS objects only
<code>msgsnd()</code>	XPG4	O	O	OSS objects only

Table 80: API Interoperability in Native Processes (Functions n through z)

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>nan()</code>	ISO	G, O	G, O	IEEE_float only
<code>nanf()</code>	ISO	G, O	G, O	IEEE_float only
<code>nanl()</code>	ISO	G, O	G, O	IEEE_float only
<code>nearbyint()</code>	ISO	G, O	G, O	IEEE_float only
<code>nearbyintf()</code>	ISO	G, O	G, O	IEEE_float only
<code>nearbyintl()</code>	ISO	G, O	G, O	IEEE_float only
<code>nextafter()</code>	XPG4	G, O	G, O	
<code>nextafterf()</code>	ISO	G, O	G, O	
<code>nextafterl()</code>	ISO	G, O	G, O	
<code>nexttoward()</code>	ISO	G, O	G, O	IEEE_float only
<code>nexttowardf()</code>	ISO	G, O	G, O	IEEE_float only
<code>nexttowardl()</code>	ISO	G, O	G, O	IEEE_float only

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
nftw()	XPG4	O	G, O	OSS parameters only OSS objects and limited Guardian objects
nftw64()	XLFS	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
nice()	XPG4	G, O	G, O	OSS objects and limited Guardian objects
nl_langinfo()	XPG4	G, O	G, O	
nlist()	HPE	G, O	G, O	
nrnd48()	XPG4	G, O	G, O	
ntohl()	XPG4	O	O	
ntohs()	XPG4	O	O	
open()	XPG4	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
open64()	XLFS	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
openat()	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>openat64()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>opendir()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>openlog()</code>	X/OPEN	G, O	G, O	Limited when called from Guardian process
<code>pathconf()</code>	POSIX	G, O	G, O	OSS parameters only
<code>pause()</code>	POSIX	G, O	G, O	<code>__PUT_MODEL__</code>
<code>pclose()</code>	POSIX	O	O	OSS parameters only
<code>perror()</code>	ISO	G, O	G, O	
<code>pipe()</code>	POSIX	O	G, O	OSS parameters only
<code>popen()</code>	POSIX	O	O	OSS parameters only
<code>pow()</code>	ISO	G, O	G, O	
<code>powf()</code>	ISO	G, O	G, O	
<code>powl()</code>	ISO	G, O	G, O	
<code>pread()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>pread64_()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>printf()</code>	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>pthread_atfork()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_destroy()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_getdetachstate()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_getguardsize()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>pthread_attr_getguardsize_np()</code>	Threads	O	O	OSS objects only
<code>pthread_attr_getinheritsched()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_getschedparam()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_getschedpolicy()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_getscope()</code>	Threads	O	O	OSS objects only _PUT_MODEL_

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>pthread_attr_getstackaddr()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_getstacksize()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_init()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_setdetachstate()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_setguardsize()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>pthread_attr_setguardsize_np()</code>	Threads	O	O	OSS objects only
<code>pthread_attr_setinheritsched()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_setschedparam()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_attr_setschedpolicy()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>pthread_attr_setscope()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>pthread_attr_setstacksize()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_cancel()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_cleanup_pop()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_cleanup_push()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_cond_broadcast()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_cond_destroy()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_cond_init()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>pthread_cond_signal()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_cond_signal_int_np()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_cond_timedwait()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_cond_wait()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_condattr_destroy()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_condattr_init()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_create()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_delay_np()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>pthread_detach()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_equal()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_exit()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_get_expiration_np()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_getattr_np()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_getconcurrency()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_getschedparam()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_getspecific()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>pthread_getthreadstateinfo_np()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_join()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_key_create()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_key_delete()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_kill()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_kill_np()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_lock_global_np()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_mutex_destroy()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>pthread_mutex_init()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_mutex_lock()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_mutex_trylock()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_mutex_unlock()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_mutexattr_destroy()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_mutexattr_getkind_np()</code>	Threads	O	O	OSS objects only
<code>pthread_mutexattr_gettype()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or
<code>pthread_mutexattr_init()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_mutexattr_setkind_np()</code>	Threads	O	O	OSS objects only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>pthread_mutexattr_settype()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_once()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_self()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_setcancelstate()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_setcanceltype()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_setconcurrency()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_setschedparam()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_setspecific()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>pthread_sigmask()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_signal_to_cancel_np()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_testcancel()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>pthread_unlock_global_np()</code>	Threads	O	O	OSS objects only _PUT_MODEL_ or _SPT_MODEL_
<code>PUT_ABORTTRANSACTION()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>put_waitio()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_waitiou()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_waitiou64()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_BEGINTRANSACTION()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_BEGINTRANSACTION_EXT_()</code>	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
PUT_CANCEL()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_CONTROL()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_ENDTRANSACTION()	HPE	O	O	Guardian objects only _PUT_MODEL_
put_fd_read_ready()	Threads	O	O	OSS objects only _PUT_MODEL_
put_fd_write_ready()	Threads	O	O	OSS objects only _PUT_MODEL_
PUT_FILE_CLOSE_()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_FILE_OPEN_()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_FILE_WRITE_READ_()	HPE	O	O	Guardian objects only _PUT_MODEL_ Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
put_generateTag()	Threads	O	O	OSS objects only _PUT_MODEL_
put_getTMFConcurrentTransactions()	Threads	O	O	OSS objects only _PUT_MODEL_
put_INITRECEIVE()	HPE	O	O	Guardian objects only _PUT_MODEL_

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>put_INITRECEIVE()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>put_interrupt()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_interruptTag()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_LOCKFILE()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_LOCKREC()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_READLOCKX()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_READUPDATELOCKX()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_READUPDATEX()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_READX()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>put_RECEIVEREAD()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>put_RECEIVEREADL()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>put_regFile()</code>	Threads	O	O	OSS objects only _PUT_MODEL_

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>put_regFileIOHandler()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_regOSSFileIOHandler()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_regPathsendFile()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_regPathsendTagHandler()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_regTimerHandler()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_REPLYX()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>put_REPLYXL()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_RESUMETRANSACTION()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>put_regTimerHandler()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_regTimerHandler()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_select_single_np()</code>	Threads	O	O	Requires both _PUT_MODEL_ and _PUT_SELECT_SINGLE_ feature test macros be specified when you compile the application.

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
PUT_SERVERCLASS_DIALOG_ABORT_()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_SERVERCLASS_DIALOG_BEGIN_()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_SERVERCLASS_DIALOG_BEGINL_()	HPE	O	O	Guardian objects only _PUT_MODEL_ Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
PUT_SERVERCLASS_DIALOG_END_()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_SERVERCLASS_DIALOG_SEND_()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_SERVERCLASS_DIALOG_SENDL_()	HPE	O	O	Guardian objects only _PUT_MODEL_ Support for TNS/E native Guardian and 32-bit OSS callers
PUT_SERVERCLASS_SEND_()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_SERVERCLASS_SEND_INFO_()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_SERVERCLASS_SENDL_()	HPE	O	O	Guardian objects only _PUT_MODEL_
PUT_SETMODE_()	HPE	O	O	Guardian objects only _PUT_MODEL_

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>put_setOSSFileIOHandler()</code>	HPE	O	O	OSS objects only _PUT_MODEL_
<code>put_setTMFConcurrentTransactions()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_TMF_GetTxHandle()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_TMF_Init()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_TMF_JOIN()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_TMF_RESUME()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_TMF_SetAndValidateTxHandle()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_TMF_SetTxHandle()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_TMF_SUSPEND()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_UNLOCKFILE()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_UNLOCKREC()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>put_unregFile()</code>	Threads	O	O	OSS objects only _PUT_MODEL_

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>put_unregOSSFileIOHandler()</code>	HPE	O	O	OSS objects only _PUT_MODEL_
<code>put_unregPathsendTagHandler()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_USEREVENT_GET()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_USEREVENT_SET()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>put_wakeup()</code>	Threads	O	O	OSS objects only _PUT_MODEL_
<code>PUT_WRITEREADX()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_WRITEUPDATEUNLOCKX()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_WRITEUPDATEX()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>PUT_WRITEX()</code>	HPE	O	O	Guardian objects only _PUT_MODEL_
<code>putc()</code>	ISO	G, O	G, O	
<code>putc_unlocked()</code>	Threads	G, O	G, O	
<code>putchar()</code>	ISO	G, O	G, O	
<code>putchar_unlocked()</code>	Threads	G, O	G, O	
<code>putenv()</code>	XPG4	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>puts()</code>	ISO	G, O	G, O	
<code>putw()</code>	XPG4	G, O	G, O	
<code>putwc()</code>	XPG4	G, O	G, O	
<code>putwchar()</code>	XPG4	G, O	G, O	
<code>putws()</code>	XPG4	G, O	G, O	
<code>pwrite()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>pwrite64_()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>qsort()</code>	ISO	G, O	G, O	
<code>raise()</code>	ISO	G, O	G, O	
<code>rand()</code>	ISO	G, O	G, O	
<code>rand_r()</code>	Threads	G, O	G, O	<code>__SPT_MODEL__</code>
<code>random()</code>	XPG4	G, O	G, O	
<code>re_comp()</code>	XPG4	G, O	G, O	The behavior of this function in locales other than POSIX is undefined
<code>re_exec()</code>	XPG4	G, O	G, O	The behavior of this function in locales other than POSIX is undefined
<code>read()</code>	XPG4	O	O	OSS parameters only OSS objects and limited Guardian objects <code>__PUT_MODEL__</code>

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>read64_()</code>	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>readdir()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>readdir_dt()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>readdir_r()</code>	Threads	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects <code>_SPT_MODEL_</code>
<code>readdir_r_dt()</code>	Threads	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects <code>_SPT_MODEL_</code>
<code>readdir64()</code>	XLFS	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects <code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>
<code>readdir64_dt()</code>	XLFS	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects <code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>readdir64_r()</code>	Threads	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS _SPT_MODEL_
<code>readdir64_r_dt()</code>	Threads	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS _SPT_MODEL_
<code>readlink()</code>	X/OPEN	G, O	G, O	OSS objects only _XOPEN_SOURCE_EXTENDED
<code>readlinkat()</code>	POSIX	G, O	G, O	OSS objects only
<code>readv()</code>	XPG4	O	O	OSS parameters only OSS objects and Guardian objects _PUT_MODEL_
<code>realloc()</code>	ISO	G, O	G, O	
<code>realloc32()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X 64-bit OSS callers
<code>realpath()</code>		O	O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>recv()</code>	XPG4	O	O	<code>_PUT_MODEL_</code> <code>_XOPEN_SOURCE_EXTENDED</code>
<code>recv64_()</code>	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>recvfrom()</code>	XPG4	O	O	<code>_PUT_MODEL_</code> <code>_XOPEN_SOURCE_EXTENDED</code>
<code>recvfrom64_()</code>	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>recvmsg()</code>	XPG4	O	O	<code>_PUT_MODEL_</code> <code>_XOPEN_SOURCE_EXTENDED</code>
<code>recvmsg64_()</code>	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>regcmp()</code>	XPG4	G, O	G, O	The behavior of this function in locales other than POSIX is undefined
<code>regcomp()</code>	XPG4	G, O	G, O	
<code>regerror()</code>	XPG4	G, O	G, O	
<code>regex()</code>	XPG4	G, O	G, O	The behavior of this function in locales other than POSIX is undefined
<code>regexec()</code>	XPG4	G, O	G, O	
<code>regfree()</code>	XPG4	G, O	G, O	
<code>remainder()</code>	XPG4	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>remainderf()</code>	ISO	G, O	G, O	
<code>remainderl()</code>	ISO	G, O	G, O	
<code>remove()</code>	ISO	G, O	G, O	Parameters must match process
<code>remove_guardian()</code>	HPE	G, O	G, O	Guardian parameters only
<code>remove_oss()</code>	HPE	G, O	G, O	OSS parameters only
<code>remque()</code>	XPG4	G, O	G, O	
<code>remquo()</code>	ISO	G, O	G, O	IEEE_float only
<code>remquof()</code>	ISO	G, O	G, O	IEEE_float only
<code>remquol()</code>	ISO	G, O	G, O	IEEE_float only
<code>rename()</code>	ISO	G, O	G, O	Parameters must match process Guardian function operates only on Guardian objects OSS function operates on OSS objects and limited Guardian objects
<code>renameat()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>rename_guardian()</code>	HPE	G, O	G, O	Guardian parameters only Guardian objects only
<code>rename_oss()</code>	HPE	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>rewind()</code>	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>rewinddir()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>rindex()</code>	XPG4	G, O	G, O	
<code>rint()</code>	XPG4	G, O	G, O	
<code>rintf()</code>	ISO	G, O	G, O	
<code>rintl()</code>	ISO	G, O	G, O	
<code>rmdir()</code>	POSIX	G, O	G, O	OSS parameters only Limited OSS objects only
<code>round()</code>	ISO	G, O	G, O	IEEE_float only
<code>roundf()</code>	ISO	G, O	G, O	IEEE_float only
<code>roundl()</code>	ISO	G, O	G, O	IEEE_float only
<code>scalb()</code>	XPG4	G, O	G, O	
<code>scalbln()</code>	ISO	G, O	G, O	IEEE_float only
<code>scalblnf()</code>	ISO	G, O	G, O	IEEE_float only
<code>scalblnl()</code>	ISO	G, O	G, O	IEEE_float only
<code>scalbn()</code>	ISO	G, O	G, O	IEEE_float only
<code>scalbnf()</code>	ISO	G, O	G, O	IEEE_float only
<code>scalbnl()</code>	ISO	G, O	G, O	IEEE_float only
<code>scandir()</code>	HPE	O	O	Hewlett Packard Enterprise extension to XPG4 specification

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>scandir64()</code>	XLFS	O	O	<code>_LARGEFILE64_SOURCE</code> or <code>_FILE_OFFSET_BITS</code>
<code>scanf()</code>	ISO	G, O	G, O	
<code>sched_get_priority_max()</code>	Threads	O	O	OSS parameters only <code>_PUT_MODEL_</code> or <code>_SPT_MODEL_</code>
<code>sched_get_priority_min()</code>	Threads	O	O	OSS parameters only <code>_PUT_MODEL_</code> or <code>_SPT_MODEL_</code>
<code>sched_yield()</code>	Threads	O	O	OSS parameters only <code>_PUT_MODEL_</code> or <code>_SPT_MODEL_</code>
<code>seed48()</code>	XPG4	G, O	G, O	
<code>seekdir()</code>	XPG4	O	O	OSS parameters only OSS objects and limited Guardian objects
<code>select()</code>	XPG4	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects <code>_PUT_MODEL_</code>
<code>semctl()</code>	XPG4	O	O	
<code>semget()</code>	XPG4	O	O	
<code>semop()</code>	XPG4	O	O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
send()	XPG4	O	O	_PUT_MODEL_ _XOPEN_SOURCE_EXTENDED
send64_()	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
sendmsg()	XPG4	O	O	_PUT_MODEL_ _XOPEN_SOURCE_EXTENDED _PUT_MODEL_ _XOPEN_SOURCE_EXTENDED
sendmsg64_()	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
sendto()	XPG4	O	O	_PUT_MODEL_ _XOPEN_SOURCE_EXTENDED
sendto64_()	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
setbuf()	ISO	G, O	G, O	
setegid()	XPG4	O	O	
seteuid()	XPG4	O	O	
setfilepriv()	HPE	G, O	G, O	
setgid()	POSIX	O	O	
setgrent()	POSIX	G, O	G, O	
setgroups()	HPE	O	O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
sethostent()	XPG4	O	O	
setitimer()	ISO	G, O	G, O	Only real-time timer is supported.
setjmp()	ISO	G, O	G, O	
setkey()	XPISOG4	G, O	G, O	Programs compiled with this version of the function cannot be exported from the United States of America.
setlocale()	ISO	G, O	G, O	
setlocale_from_msg()	HPE	O	O	OSS parameters only
setlogmask()	X/OPEN	O	O	
setnetent()	XPG4	O	O	
setpgid()	POSIX	O	O	OSS objects only
setpgrp()	POSIX	O	O	OSS objects only
setprotoent()	XPG4	O	O	
setpwent()	X/OPEN	G, O	G, O	<code>_XOPEN_SOURCE_EXTENDED</code> <code>_TANDEM_SOURCE</code>
setregid()	XPG4	O	O	OSS objects only
setreuid()	XPG4	O	O	OSS objects only
setservent()	XPG4	O	O	
setsid()	POSIX	O	O	OSS objects only
setsockopt()	XPG4	O	O	<code>_XOPEN_SOURCE_EXTENDED</code>
setstate()	XPG4	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
setuid()	POSIX	O	O	
setvbuf()	ISO	G, O	G, O	
shmat()	XPG4	G*, O	G*, O	* Guardian support for this function is for L-series and J06.12 and later J-series RVUs and H06.23 and later H-series RVUs only.
shmctl()	XPG4	G*, O	G*, O	* Guardian support for this function is for L-series and J06.12 and later J-series RVUs and H06.23 and later H-series RVUs only.
shmdt()	XPG4	G*, O	G*, O	* Guardian support for this function is for L-series and J06.12 and later J-series RVUs and H06.23 and later H-series RVUs only.
shmget()	XPG4	G*, O	G*, O	* Guardian support for this function is for L-series and J06.12 and later J-series RVUs and H06.23 and later H-series RVUs only.
shutdown()	XPG4	O	O	
sigaction()	POSIX	G, O	G, O	_PUT_MODEL_
sigaddset()	POSIX	O	O	
sigaltstack()	POSIX	G, O	G, O	_PUT_MODEL_
sigdelset()	POSIX	O	O	
sigemptyset()	POSIX	O	O	
sigfillset()	POSIX	O	O	
sigismember()	POSIX	O	O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>siglongjmp()</code>	POSIX	G, O	G, O	
<code>signal()</code>	ISO	G, O	G, O	<code>_PUT_MODEL_</code>
<code>signbit()</code>	ISO	G, O	G, O	IEEE_float only
<code>sigpending()</code>	POSIX	G, O	G, O	<code>_PUT_MODEL_</code>
<code>sigprocmask()</code>	POSIX	G, O	G, O	
<code>sigsetjmp()</code>	POSIX	G, O	G, O	
<code>sigsuspend()</code>	POSIX	G, O	G, O	<code>_PUT_MODEL_</code>
<code>sigwait()</code>	POSIX	O	O	<code>_PUT_MODEL_</code>
<code>sin()</code>	ISO	G, O	G, O	
<code>sinf()</code>	ISO	G, O	G, O	
<code>sinh()</code>	ISO	G, O	G, O	
<code>sinhf()</code>	ISO	G, O	G, O	
<code>sinhl()</code>	ISO	G, O	G, O	
<code>sinl()</code>	ISO	G, O	G, O	
<code>sleep()</code>	POSIX	O	O	<code>_PUT_MODEL_</code>
<code>socketatmark()</code>	HPE	O	O	
<code>socket()</code>	XPG4	O	O	<code>_XOPEN_SOURCE_EXTENDED</code>
<code>socket_transport_name_get()</code>	XPG4	O	O	<code>_XOPEN_SOURCE_EXTENDED</code>
<code>socket_transport_name_set()</code>	XPG4	O	O	<code>_XOPEN_SOURCE_EXTENDED</code>
<code>socketpair()</code>	XPG4	O	O	<code>_XOPEN_SOURCE_EXTENDED</code>

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>snprintf()</code>	ISO	G, O	G, O	
<code>sprintf()</code>	ISO	G, O	G, O	
<code>SPT_ ABORTTRANSACTION()</code>	HPE	O	O	Guardian objects only
<code>spt_accept()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA _XOPEN_SOURCE_EXTEN DED
<code>spt_acceptx()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK _XOPEN_SOURCE_EXTEN DED
<code>spt_alarm()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_ SIGNAL and SPT_THREAD_SIGNAL or SPT_THREAD_SIGNAL_ PRAGMA
<code>spt_waitio()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>SPT_ BEGINTRANSACTION ()</code>	HPE	O	O	Guardian objects only
<code>SPT_ BEGINTRANSACTION_EXT_ ()</code>	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>SPT_ CANCEL ()</code>	HPE	O	O	Guardian objects only
<code>spt_close ()</code>	Threads	O	O	OSS objects only <code>SPT_ THREAD_ AWARE</code> or <code>SPT_ THREAD_ AWARE_ PRAGMA</code>
<code>spt_closex ()</code>	Threads	O	O	OSS objects only <code>SPT_ THREAD_ AWARE_ NONBLOCK</code> or <code>SPT_ THREAD_ AWARE_ PRAGMA_ NONBLOCK</code>
<code>spt_closez ()</code>	Threads	O	O	OSS objects only <code>_ SPT_ MODEL_</code> <code>SPT_ THREAD_ AWARE_ XNONBLOCK</code> or <code>SPT_ THREAD_ AWARE_ PRAGMA_ XNONBLOCK</code>
<code>spt_connect ()</code>	Threads	O	O	OSS objects only <code>SPT_ THREAD_ AWARE</code> or <code>SPT_ THREAD_ AWARE_ PRAGMA</code> <code>_ XOPEN_ SOURCE_ EXTENDED</code>

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_connectx()</code>	Threads	O	O	OSS objects only <code>_SPT_MODEL_</code> <code>SPT_THREAD_AWARE_</code> <code>NONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_NONBLOCK;</code> or <code>SPT_THREAD_AWARE_</code> <code>XNONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_XNONBLOCK</code> <code>_XOPEN_SOURCE_EXTENDED</code>
<code>SPT_CONTROL()</code>	HPE	O	O	Guardian objects only
<code>spt_dup2x()</code>	Threads	O	O	OSS objects only <code>_SPT_MODEL_</code> <code>SPT_THREAD_AWARE_</code> <code>NONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_NONBLOCK;</code> or <code>SPT_THREAD_AWARE_</code> <code>XNONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_XNONBLOCK</code>
<code>SPT_ENDTRANSACTION()</code>	HPE	O	O	Guardian objects only
<code>spt_fclose()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA</code>

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_fclosex()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fcntlx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK
<code>spt_fcntlz()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fd_read_ready()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_fd_write_ready()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_fflush()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_fflushx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fgetc()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_fgetcxc()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fgets()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_fgetsx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_NONBLOCK or SPT_THREAD_AWARE_PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_XNONBLOCK or SPT_THREAD_AWARE_PRAGMA_XNONBLOCK
<code>spt_fgetwc()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
<code>spt_fgetwcx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_NONBLOCK or SPT_THREAD_AWARE_PRAGMA_NONBLOCK; SPT_THREAD_AWARE_XNONBLOCK or SPT_THREAD_AWARE_PRAGMA_XNONBLOCK
<code>SPT_FILE_CLOSE_()</code>	HPE	O	O	Guardian objects only
<code>SPT_FILE_OPEN_()</code>	HPE	O	O	Guardian objects only
<code>spt_fork()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_fprintf()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_printfx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fputc()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_fputcx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fputs()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_fputsx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fputwc()</code>	Threads	O	O	OSS objects only
<code>spt_fputwcx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fread()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_freadx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fstatz()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fstat64z()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fsyncz()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_ftruncatez()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_ftruncate64z()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_fwrite()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_fwritex()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_generateTag()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_getc()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_getchar()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_getcharx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_getcx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_gets()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_getsx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_NONBLOCK or SPT_THREAD_AWARE_PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_XNONBLOCK or SPT_THREAD_AWARE_PRAGMA_XNONBLOCK
<code>spt_getTMFConcurrentTransactions()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
<code>spt_getw()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
<code>spt_getwc()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
<code>spt_getwchar()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_getwcharx()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE_NONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_NONBLOCK</code>; or <code>SPT_THREAD_AWARE_XNONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_XNONBLOCK</code>
<code>spt_getwcx()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE_NONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_NONBLOCK</code>; or <code>SPT_THREAD_AWARE_XNONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_XNONBLOCK</code>
<code>spt_getwx()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE_NONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_NONBLOCK</code>; or <code>SPT_THREAD_AWARE_XNONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_XNONBLOCK</code>
<code>spt_INITRECEIVE()</code>	HPE	O	O	Guardian objects only
<code>spt_INITRECEIVEL()</code>	HPE	O	O	Guardian objects only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_interrupt()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
<code>spt_interruptTag()</code>	Threads	O	O	OSS objects only
<code>SPT_LOCKFILE()</code>	HPE	O	O	Guardian objects only
<code>SPT_LOCKREC()</code>	HPE	O	O	Guardian objects only
<code>spt_lseekz()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_XNONBLOCK or SPT_THREAD_AWARE_PRAGMA_XNONBLOCK
<code>spt_lseek64z()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_XNONBLOCK or SPT_THREAD_AWARE_PRAGMA_XNONBLOCK
<code>spt_pause()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_SIGNAL and SPT_THREAD_SIGNAL or SPT_THREAD_SIGNAL_PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_printf()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_printfx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_putc()</code>	Threads	O	O	OSS objects only
<code>spt_putchar()</code>	Threads	O	O	OSS objects only
<code>spt_putcharx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_putcx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_puts()</code>	Threads	O	O	OSS objects only
<code>spt_putsx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_putw()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_putwc()</code>	Threads	O	O	OSS objects only
<code>spt_putwchar()</code>	Threads	O	O	OSS objects only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_putwchar()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE_NONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_NONBLOCK</code>; or <code>SPT_THREAD_AWARE_XNONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_XNONBLOCK</code>
<code>spt_putwcx()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE_NONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_NONBLOCK</code>; or <code>SPT_THREAD_AWARE_XNONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_XNONBLOCK</code>
<code>spt_putwx()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE_NONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_NONBLOCK</code>; or <code>SPT_THREAD_AWARE_XNONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_XNONBLOCK</code>
<code>spt_read()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE</code> or <code>SPT_THREAD_AWARE_PRAGMA</code>

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>SPT_READLOCKX()</code>	HPE	O	O	Guardian objects only
<code>SPT_READUPDATELOCKX()</code>	HPE	O	O	Guardian objects only
<code>SPT_READUPDATEX()</code>	HPE	O	O	Guardian objects only
<code>spt_readv()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_readvx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK
<code>spt_readvz()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>SPT_READX()</code>	HPE	O	O	Guardian objects only
<code>spt_readx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_readz()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK
<code>spt_RECEIVEREAD()</code>	HPE	O	O	Guardian objects only
<code>spt_RECEIVEREADL()</code>	HPE	O	O	Guardian objects only
<code>spt_recv()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA _XOPEN_SOURCE_EXTEN DED
<code>spt_recvfrom()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA _XOPEN_SOURCE_EXTEN DED

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_recvfromx()</code>	Threads	O	O	OSS objects only <code>_SPT_MODEL_</code> <code>SPT_THREAD_AWARE_</code> <code>NONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_NONBLOCK;</code> or <code>SPT_THREAD_AWARE_</code> <code>XNONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_XNONBLOCK</code> <code>_XOPEN_SOURCE_EXTENDED</code>
<code>spt_recvmsg()</code>	Threads	O	O	OSS objects only <code>_XOPEN_SOURCE_EXTENDED</code>
<code>spt_recvmsgx()</code>	Threads	O	O	OSS objects only <code>_SPT_MODEL_</code> <code>SPT_THREAD_AWARE_</code> <code>NONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_NONBLOCK;</code> or <code>SPT_THREAD_AWARE_</code> <code>XNONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_XNONBLOCK</code> <code>_XOPEN_SOURCE_EXTENDED</code>

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_recvx()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PRAGMA_XNONBLOCK _XOPEN_SOURCE_EXTEN DED
<code>spt_regFile()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_regFileIOHandler()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_regOSSFileIOHandler()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_regPathsendFile()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_regPathsendTagHandler()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
<code>spt_regTimerHandler()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
<code>spt_REPLYX()</code>	HPE	O	O	Guardian objects only
<code>spt_REPLYXL()</code>	HPE	O	O	Guardian objects only
<code>SPT_RESUMETRANSACTION()</code>	HPE	O	O	Guardian objects only
<code>spt_select()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
<code>spt_select_single_np()</code>	Threads	O	O	Mapped to <code>from select()</code> if <code>SPT_SELECT_SINGLE</code> is defined and one of the thread aware defines are used. _SPT_MODEL_
<code>spt_send()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA _XOPEN_SOURCE_EXTENDED

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_sendmsg()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA _XOPEN_SOURCE_EXTENDED
<code>spt_sendmsgx()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_NONBLOCK or SPT_THREAD_AWARE_PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_XNONBLOCK or SPT_THREAD_AWARE_PRAGMA_XNONBLOCK _XOPEN_SOURCE_EXTENDED
<code>spt_sendto()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA _XOPEN_SOURCE_EXTENDED

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_sendtox()</code>	Threads	O	O	OSS objects only <code>_SPT_MODEL_</code> <code>SPT_THREAD_AWARE_</code> <code>NONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_NONBLOCK;</code> or <code>SPT_THREAD_AWARE_</code> <code>XNONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_XNONBLOCK</code> <code>_XOPEN_SOURCE_EXTENDED</code>
<code>spt_sendx()</code>	Threads	O	O	OSS objects only <code>_SPT_MODEL_</code> <code>SPT_THREAD_AWARE_</code> <code>NONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_NONBLOCK;</code> or <code>SPT_THREAD_AWARE_</code> <code>XNONBLOCK</code> or <code>SPT_THREAD_AWARE_</code> <code>PRAGMA_XNONBLOCK</code> <code>_XOPEN_SOURCE_EXTENDED</code>
<code>SPT_SERVERCLASS_DIALOG_ABORT_()</code>	HPE	O	O	Guardian objects only
<code>SPT_SERVERCLASS_DIALOG_BEGIN_()</code>	HPE	O	O	Guardian objects only
<code>SPT_SERVERCLASS_DIALOG_BEGINL_()</code>	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>SPT_SERVERCLASS_DIALOG_END_()</code>	HPE	O	O	Guardian objects only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>SPT_SERVERCLASS_DIALOG_SEND_()</code>	HPE	O	O	Guardian objects only
<code>SPT_SERVERCLASS_DIALOG_SENDL_()</code>	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>SPT_SERVERCLASS_SEND_()</code>	HPE	O	O	Guardian objects only
<code>SPT_SERVERCLASS_SEND_INFO_()</code>	HPE	O	O	Guardian objects only
<code>SPT_SERVERCLASS_SENDL_()</code>	HPE	O	O	Guardian objects only
<code>SPT_SETMODE()</code>	HPE	O	O	Guardian objects only
<code>spt_setOSSFileIOHandler()</code>	HPE	O	O	OSS objects only
<code>spt_setTMFConcurrentTransactions()</code>	Threads	O	O	OSS objects only
<code>spt_sigaction()</code>	Threads	O	O	OSS objects only Set <code>SPT_THREAD_AWARE_SIGNAL</code> to catch externally generated signals at the thread level <code>_SPT_MODEL_</code>
<code>spt_signal()</code>	Threads	O	O	OSS objects only <code>_SPT_MODEL_</code> <code>SPT_THREAD_AWARE_SIGNAL</code> and <code>SPT_THREAD_SIGNAL</code> or <code>SPT_THREAD_SIGNAL_PRAGMA</code>
<code>spt_sigpending()</code>	Threads	O	O	OSS objects only <code>_SPT_MODEL_</code> <code>SPT_THREAD_AWARE</code> or <code>SPT_THREAD_AWARE_PRAGMA</code>

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_sigsuspend()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ Set SPT_THREAD_AWARE_ SIGNAL to catch externally generated signals at the thread level
<code>spt_sigwait()</code>	Threads	O	O	OSS objects only Set SPT_THREAD_AWARE_ SIGNAL to catch externally generated signals at the thread level _SPT_MODEL_
<code>spt_sleep()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_system()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>SPT_TMF_GetTxHandle()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>SPT_TMF_Init()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
SPT_TMF_RESUME ()	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
SPT_TMF_SetAndValidateTxHandle ()	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
SPT_TMF_SetTxHandle ()	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
SPT_TMF_SUSPEND ()	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
SPT_UNLOCKFILE ()	HPE	O	O	Guardian objects only
SPT_UNLOCKREC ()	HPE	O	O	Guardian objects only
spt_unregFile ()	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
spt_unregOSSFileIOHandler ()	HPE	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_unregPathsendTagHandler()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_usleep()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_vfprintf()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA
<code>spt_vfprintfx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_ NONBLOCK or SPT_THREAD_AWARE_ PRAGMA_NONBLOCK; or SPT_THREAD_AWARE_ XNONBLOCK or SPT_THREAD_AWARE_ PROSS objects only AGMA_XNONBLOCK
<code>spt_vprintf()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_ PRAGMA

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_vprintfx()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE_NONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_NONBLOCK</code>; or <code>SPT_THREAD_AWARE_XNONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_XNONBLOCK</code>
<code>spt_waitpid()</code>	Threads	O	O	OSS objects only <code>_SPT_MODEL_</code> To use nonblocking feature set <code>SPT_THREAD_AWARE_NONBLOCK</code> or <code>SPT_THREAD_AWARE_PRAGMA_NONBLOCK</code>
<code>spt_wakeup()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE</code> or <code>SPT_THREAD_AWARE_PRAGMA</code>
<code>spt_write()</code>	Threads	O	O	OSS objects only <code>SPT_THREAD_AWARE</code> or <code>SPT_THREAD_AWARE_PRAGMA</code>
<code>SPT_WRITEREADX()</code>	HPE	O	O	Guardian objects only
<code>SPT_WRITEUPDATEUNLOCKX()</code>	HPE	O	O	Guardian objects only
<code>SPT_WRITEUPDATEX()</code>	HPE	O	O	Guardian objects only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>spt_writev()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE or SPT_THREAD_AWARE_PRAGMA
<code>spt_writevx()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_NONBLOCK or SPT_THREAD_AWARE_PRAGMA_NONBLOCK
<code>spt_writevz()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_XNONBLOCK or SPT_THREAD_AWARE_PRAGMA_XNONBLOCK
<code>SPT_WRITEX()</code>	HPE	O	O	Guardian objects only
<code>spt_writex()</code>	Threads	O	O	OSS objects only SPT_THREAD_AWARE_NONBLOCK or SPT_THREAD_AWARE_PRAGMA_NONBLOCK
<code>spt_writez()</code>	Threads	O	O	OSS objects only _SPT_MODEL_ SPT_THREAD_AWARE_XNONBLOCK or SPT_THREAD_AWARE_PRAGMA_XNONBLOCK
<code>sqrt()</code>	XPG4	G, O	G, O	
<code>sqrtf()</code>	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>sqrtrl()</code>	ISO	G, O	G, O	
<code>srand()</code>	ISO	G, O	G, O	
<code>srand48()</code>	XPG4	G, O	G, O	
<code>srandom()</code>	XPG4	G, O	G, O	
<code>sscanf()</code>	ISO	G, O	G, O	
<code>stat()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>stat64()</code>	XLFS	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>statvfs()</code>	XPG4	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>statvfs64()</code>	XLFS	O	G, O	OSS parameters only OSS objects and limited Guardian objects _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>step()</code>	XPG4	G, O	G, O	
<code>strcasecmp()</code>	XPG4	G, O	G, O	
<code>strcat()</code>	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>strcat64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>strchr()</code>	ISO	G, O	G, O	
<code>strchr64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>strcmp()</code>	ISO	G, O	G, O	
<code>strcmp64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>strcoll()</code>	ISO	G, O	G, O	
<code>strcpy()</code>	ISO	G, O	G, O	
<code>strcpy64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>strcspn()</code>	ISO	G, O	G, O	
<code>strdup()</code>	X/OPEN	G, O	G, O	<code>_XOPEN_SOURCE_EXTENDED</code>
<code>strerror()</code>	ISO	G, O	G, O	
<code>strerror_r()</code>	Threads	G, O	G, O	
<code>strfmon()</code>	XPG4	G, O	G, O	
<code>strftime()</code>	ISO	G, O	G, O	
<code>strlen()</code>	ISO	G, O	G, O	
<code>strlen64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>strncasecmp()</code>	XPG4	G, O	G, O	
<code>strncat()</code>	ISO	G, O	G, O	
<code>strncat64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>strncmp()</code>	ISO	G, O	G, O	
<code>strncmp64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>strncpy()</code>	ISO	G, O	G, O	
<code>strncpy64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>strpbrk()</code>	ISO	G, O	G, O	
<code>strptime()</code>	XPG4	G, O	G, O	
<code>strrchr()</code>	ISO	G, O	G, O	
<code>strrchr64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>strspn()</code>	ISO	G, O	G, O	
<code>strstr()</code>	ISO	G, O	G, O	
<code>strstr64()</code>	HPE	G, O	G, O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
<code>strtod()</code>	ISO	G, O	G, O	
<code>strtof()</code>	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
strtoimax()	ISO	G, O	G, O	
strtok()	ISO	G, O	G, O	
strtok_r()	Threads	G, O	G, O	_SPT_MODEL_
strtol()	ISO	G, O	G, O	
strtoll()	ISO	G, O	G, O	
strtoul()	ISO	G, O	G, O	
strtoull()	ISO	G, O	G, O	
strtoumax()	ISO	G, O	G, O	
strxfrm()	ISO	G, O	G, O	
swab()	XPG4	G, O	G, O	
swprintf()	ISO	G, O	G, O	
swscanf()	ISO	G, O	G, O	
symlink()	X/OPEN	G, O	G, O	OSS objects only _XOPEN_SOURCE_EXTENDED
symlinkat()	POSIX	G, O	G, O	OSS objects only
sysconf()	POSIX	G, O	G, O	
syslog()	X/OPEN	G, O	G, O	
system()	ISO	G, O	G, O	Only Guardian modules or only OSS modules
tan()	ISO	G, O	G, O	
tanf()	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>tanh()</code>	ISO	G, O	G, O	
<code>tanhf()</code>	ISO	G, O	G, O	
<code>tanhll()</code>	ISO	G, O	G, O	
<code>tanl()</code>	ISO	G, O	G, O	
<code>tcdrain()</code>	POSIX	O	O	
<code>tcflow()</code>	POSIX	O	O	
<code>tcflush()</code>	POSIX	O	O	
<code>tcgetattr()</code>	POSIX	O	O	
<code>tcgetpgrp()</code>	POSIX	O	O	
<code>tcsendbreak()</code>	POSIX	O	O	
<code>tcsetattr()</code>	POSIX	O	O	
<code>tcsetpgrp()</code>	POSIX	O	O	
<code>tdelete()</code>	XPG4	G, O	G, O	
<code>tdm_execve()</code>	HPE	O	O	OSS parameters only
<code>tdm_execvep()</code>	HPE	O	O	OSS parameters only
<code>tdm_fork()</code>	HPE	O	O	OSS parameters only
<code>tdm_spawn()</code>	HPE	O	O	OSS parameters only
<code>tdm_spawnp()</code>	HPE	O	O	OSS objects only
<code>tellldir()</code>	XPG4	O	O	OSS parameters only OSS objects and limited Guardian objects
<code>tempnam()</code>	XPG4	O	O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>terminate_program()</code>	HPE	G, O	G, O	
<code>tfind()</code>	XPG4	G, O	G, O	
<code>tgamma()</code>	ISO	G, O	G, O	IEEE_float only
<code>tgammaf()</code>	ISO	G, O	G, O	IEEE_float only
<code>tgamma1()</code>	ISO	G, O	G, O	IEEE_float only
<code>time()</code>	ISO	G, O	G, O	
<code>times()</code>	POSIX	G, O	G, O	
<code>tmpfile()</code>	ISO	G, O	G, O	Result matches process temp file created in
<code>tmpfile_guardian()</code>	HPE	G, O	G, O	Guardian parameters only
<code>tmpfile_oss()</code>	HPE	G, O	G, O	OSS parameters only
<code>tmpfile64()</code>	XLFS	G, O	G, O	_LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>tmpfile64_guardian()</code>	XLFS	G, O	G, O	Guardian parameters only _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>tmpfile64_oss()</code>	XLFS	G, O	G, O	OSS parameters only _LARGEFILE64_SOURCE or _FILE_OFFSET_BITS
<code>tmpnam()</code>	ISO	G, O	G, O	Parameters must match process
<code>tmpnam_guardian()</code>	HPE	G, O	G, O	Guardian parameters only

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>tmpnam_oss()</code>	HPE	G, O	G, O	OSS parameters only
<code>tmpnam_r()</code>	Threads	G, O	G, O	<code>__SPT_MODEL__</code>
<code>toascii()</code>	XPG4	G, O	G, O	
<code>_tolower()</code>	XPG4	G, O	G, O	
<code>tolower()</code>	ISO	G, O	G, O	
<code>_toupper()</code>	XPG4	G, O	G, O	
<code>toupper()</code>	ISO	G, O	G, O	
<code>towctrans()</code>	ISO	G, O	G, O	
<code>towlower()</code>	XPG4	G, O	G, O	
<code>towupper()</code>	XPG4	G, O	G, O	
<code>trunc()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>truncf()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>truncl()</code>	ISO	G, O	G, O	<code>IEEE_float</code> only
<code>tsearch()</code>	XPG4	G, O	G, O	
<code>ttynam()</code>	XPG4	O	G, O	
<code>ttynam_r()</code>	Threads	O	G, O	<code>__SPT_MODEL__</code>
<code>twalk()</code>	XPG4	G, O	G, O	
<code>tzset()</code>	XPG4	G, O	G, O	
<code>ulimit()</code>	XPG4	O	O	
<code>umask()</code>	POSIX	O	O	
<code>uname()</code>	POSIX	O	O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>ungetc()</code>	ISO	G, O	G, O	
<code>ungetwc()</code>	XPG4	G, O	G, O	
<code>unlink()</code>	ISO	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>unlinkat()</code>	POSIX	G, O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>unordered()</code>	XPG4	G, O	G, O	
<code>usleep()</code>	POSIX	O	O	<code>_PUT_MODEL_</code>
<code>utime()</code>	POSIX	O	G, O	OSS parameters only OSS objects and limited Guardian objects
<code>va_arg()</code>	ISO	G, O	G, O	
<code>va_end()</code>	ISO	G, O	G, O	
<code>va_start()</code>	ISO	G, O	G, O	
<code>valloc()</code>	XPG4	G, O	G, O	
<code>vfprintf()</code>	ISO	G, O	G, O	
<code>vfscanf()</code>	ISO	G, O	G, O	
<code>vfwprintf()</code>	ISO	G, O	G, O	
<code>vfwscanf()</code>	ISO	G, O	G, O	
<code>vprintf()</code>	ISO	G, O	G, O	
<code>vscanf()</code>	ISO	G, O	G, O	

Table Continued

Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
<code>vsnprintf()</code>	ISO	G, O	G, O	
<code>vsprintf()</code>	ISO	G, O	G, O	
<code>vsscanf()</code>	ISO	G, O	G, O	
<code>vswprintf()</code>	ISO	G, O	G, O	
<code>vswscanf()</code>	ISO	G, O	G, O	
<code>vwprintf()</code>	ISO	G, O	G, O	
<code>vwscanf()</code>	ISO	G, O	G, O	
<code>wait()</code>	POSIX	O	O	OSS objects only
<code>waitpid()</code>	POSIX	O	O	OSS objects only _PUT_MODEL_
<code>wcrtomb()</code>	ISO	G, O	G, O	
<code>wcscat()</code>	XPG4	G, O	G, O	
<code>wcschr()</code>	XPG4	G, O	G, O	
<code>wcscmp()</code>	XPG4	G, O	G, O	
<code>wcscoll()</code>	XPG4	G, O	G, O	
<code>wcscpy()</code>	XPG4	G, O	G, O	
<code>wcscspn()</code>	XPG4	G, O	G, O	
<code>wcsftime()</code>	XPG4	G, O	G, O	
<code>wcslen()</code>	XPG4	G, O	G, O	
<code>wcsncat()</code>	XPG4	G, O	G, O	
<code>wcsncmp()</code>	XPG4	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
wcsncpy()	XPG4	G, O	G, O	
wcspbrk()	XPG4	G, O	G, O	
wcsrchr()	XPG4	G, O	G, O	
wcsrtombs()	ISO	G, O	G, O	
wcsspn()	XPG4	G, O	G, O	
wctod()	XPG4	G, O	G, O	
wctof()	ISO	G, O	G, O	
wctoimax()	ISO	G, O	G, O	
wctok()	XPG4	G, O	G, O	
wctol()	XPG4	G, O	G, O	
wctoll()	XPG4	G, O	G, O	
wctombs()	XPG4	G, O	G, O	
wctoul()	XPG4	G, O	G, O	
wctoull()	XPG4	G, O	G, O	
wctoumax()	ISO	G, O	G, O	
wcswcs()	XPG4	G, O	G, O	
wcswidth()	XPG4	G, O	G, O	
wcsxfrm()	XPG4	G, O	G, O	
wctob()	ISO	G, O	G, O	
wctomb()	XPG4	G, O	G, O	
wctrans()	ISO	G, O	G, O	

Table Continued



Function or Macro	Specified By	Module Compiled As	Process Called From	Usage Notes
wctype()	XPG4	G, O	G, O	
wcwidth()	XPG4	G, O	G, O	
wmemchr()	ISO	G, O	G, O	
wmemcmp()	ISO	G, O	G, O	
wmemcpy()	ISO	G, O	G, O	
wmemmove()	ISO	G, O	G, O	
wmemset()	ISO	G, O	G, O	
wordexp()	POSIX	O	O	
wordfree()	POSIX	O	O	
wprintf()	ISO	G, O	G, O	
write()	XPG4	O	O	OSS parameters only OSS objects and limited Guardian objects _PUT_MODEL_
write64_()	HPE	O	O	Support for TNS/E and TNS/X native Guardian and 32-bit OSS callers
writew()	XPG4	O	O	OSS parameters only OSS objects only _PUT_MODEL_
wscanf()	ISO	G, O	G, O	
y0()	XPG4	G, O	G, O	
y1()	XPG4	G, O	G, O	
yn()	XPG4	G, O	G, O	

Glossary

\$RECEIVE

A special Guardian file through which a process receives and optionally replies to messages from other processes.

32-bit address space

The portion of the address space available to users of 32-bit addresses. On TNS/E and TNS/X systems, 32-bit addresses become 64-bit addresses by sign extension. Addresses 0 through 0x7FFFFFFF designate the first 2 gigabytes (GB) of per-process address space, starting at address 0. Addresses 0x80000000 through 0xFFFFFFFF designate the last 2 GB of global address space, starting at address 0xFFFFFFFF 80000000.

See also **address space**

32-bit addressable

In 32-bit address space; accessible via a 32-bit address. If something is 32-bit addressable, it is also 64-bit addressable on a TNS/E or TNS/X system.

32-bit object file

An object file resulting from compilation units that specified or defaulted to the ILP32 data model.

64-bit address space

See **address space**

64-bit addressable

On TNS/R systems, all memory is 32-bit addressable. On TNS/E and TNS/X systems, all memory is 64-bit addressable; a subset is 32-bit addressable. Ranges outside that subset are addressable only by 64-bit addresses.

See also **32-bit addressable**

64-bit object file

An object file resulting from compilation units that specified the LP64 data model.

absolute pathname

A pathname that begins with a slash (/) character and is resolved beginning with the root directory. Contrast with **relative pathname**.

accelerated object code

The MIPS RISC instructions (in the MIPS region) that result from processing a TNS object file with the Accelerator or the Intel® Itanium® instructions (in the Itanium instruction region) that result from processing a TNS object file with the Object Code Accelerator (OCA).

accelerated object file

A TNS object file that, in addition to its TNS instructions, has been augmented with equivalent but faster sequences of instructions in the language of the target processor. An object can be accelerated for TNS/R systems using Accel, for TNS/E systems using OCA, or for TNS/X systems using OCAX.

See also **accelerated object code**.

Accelerator

A program optimization tool that processes a TNS object file and produces an accelerated object file that also contains equivalent MIPS RISC instructions (called the MIPS region). TNS object code that is accelerated runs faster on TNS/R processors than TNS object code that is not accelerated. Contrast with **Object Code Accelerator (OCA)**.

access control list (ACL)

A structure attached to a software object that defines access permissions for multiple users and groups. It extends the permissions defined by the file system permission bits by allowing you specify the access rights of many individuals and groups instead of just one of each.

ACL

See **access control list (ACL)**.

address space

The addressable memory ranges. Memory addresses are 32 bits wide on TNS/R processors and 64 bits wide on TNS/E and TNS/X processors. The NonStop Kernel supports distinct mappings for each process, beginning at address 0. On TNS/R processors, this range extends 2 GB. On TNS/E and TNS/X processors, it extends to 512 GB, but 2 GB beginning at address 0x00000000 80000000 are reserved (never mapped), to avoid ambiguity with zero-extended 32-bit global addresses. Negative addresses are global—shared by all processes. On TNS/R processors, global addresses occupy 2 GB. On TNS/E and TNS/X processors, global mappings span the last 512 GB of the address space, beginning at 0xFFFFF80 00000000. (Additional global addresses below that range are used internally by low-level system code.) Memory references can be made to any currently mapped area of the address space, subject to access permissions.

ANSI

The American National Standards Institute.

API

See **application program interface (API)**.

application program interface (API)

A set of services (such as programming language functions or procedures) that are called by an application program to communicate with other software components. For example, an application program in the form of a client might use an API to communicate with a server program.

appropriate privileges

An implementation-defined means of associating privileges with a process for function calls or function call options that need special privileges.

authorization attributes

Security attributes of a process that can change through use of functions such as `setuid()` (or of Guardian procedures such as `PROCESS_CREATE_()`) without reauthentication. The authorization attributes include the effective user ID, saved-set user ID, saved-set group ID, user audit flags, and the effective user name.

background process

A process that belongs to a background process group.

background process group

A process group that is both of the following:

- Not a **foreground process group**
- A member of a session that has a connection with a controlling terminal

base computing platform

The minimum software implementation that is the foundation for the X/Open common applications environment (CAE).

See also **common applications environment (CAE)**.

BSD

Berkeley Software Distribution.

buffer

A piece of contiguous address space that is being used for some specific purpose, including (but not limited to) input/output or data transmission.

A buffer is identified by its starting address and its length and is typically a portion of a segment.

CAE

See **common applications environment (CAE)**.

canonical input mode

A terminal input mode in which data is not made available to a process until an entire logical line (delimited by a newline, EOF, or EOL character) is entered. This mode is sometimes called line mode or nontransparent mode.

Contrast with **noncanonical input mode**.

character

A sequence of one or more bytes representing a single character; used for the organization, representation, or control of data. A single-byte character consists of eight bits that represent a character. A multibyte character uses one or more bytes to represent a character. A wide character is a fixed-width character wide enough to hold any coded character supported by an implementation.

The ISO C standard defines the term multibyte character; a single-byte character is a special case of multibyte character.

character set

A finite set of characters (letters, digits, symbols, ideographs, or control functions) used for the organization, representation, or control of data.

See also **code set**.

child process

A process created by another process. The creating process becomes the parent process of the new process.

See also **parent process**.

client application

An application that requests a service from a **server application**. Execution of remote procedure calls is an example of such a service.

code set

Codes that map a unique numeric value to each character in a character set, using a designated number of bits to represent each character. Single-byte code sets use 7 or 8 bits to represent each character. The ASCII and ISO 646 code sets use 7 bits to represent each character in Roman-based alphabets; these code sets are very limited and are not appropriate for international use. The single-byte code sets in the ISO 8859 code sets use 8 bits to represent each character and can therefore support Roman-based alphabets as well as many others including Greek, Arabic, Hebrew, and Turkish. Multibyte code sets represent characters that require more than one byte, such as East Asian ideographic characters.

common applications environment (CAE)

A computer environment in which applications can be ported across all X/Open branded products because of the use of international and industry standards. A CAE is an open system application development environment, an open system execution environment, or a combination of the two.

compliance

The testing and verification process that precedes X/Open licensing.

conformance

Meeting the requirement of a specific standard.

conforming POSIX.1 application

An application that is either an ISO/IEC-conforming POSIX.1 application or a national-standards-body conforming POSIX.1 application.

constraint

An object that helps protect the integrity of data in a table by specifying a condition or conditions that all the values in a particular column of the table must satisfy. Unlike other SQL objects, a constraint has only an SQL name, not an operating system name, and a constraint does not have a file label.

controlling terminal

A terminal that might be associated with a session. A session can have only one controlling terminal, and a controlling terminal can control only one session at a time. When a session has a controlling terminal, all the following are true:

- Certain character sequences entered from that terminal cause signals to be sent to all processes in the process groups of that session.
- Certain characters entered from that terminal might receive special treatment.

Members of background process groups of the session are restricted from certain kinds of access to the controlling terminal.

See also **session**.

core dump file

See process snapshot file or **saveabend file**.

core file

See see process snapshot file or **saveabend file**.

daemon

See **demon**.

data-model neutral

Refers to a data item whose size is the same in both the ILP32 and the LP64 data models. Also refers to a function whose arguments and return value are data-model neutral.

See also **data-model sensitive**.

data-model sensitive

Refers to a data item whose size is different in the ILP32 and the LP64 data models.

See also **data-model neutral**.

demon

On a UNIX system, a process that runs continuously to provide a specific service for other processes. A demon does not have a controlling terminal and is not explicitly invoked. On a NonStop system, a demon runs in the OSS environment and has an OSS process ID.

See also **static server**.

device

A computer peripheral or an object that appears to the application as such. See also **terminal**.

directory

A type of OSS special file that contains directory entries which associate names with files. No two directory entries in the same directory have the same name.

DLL.

See **dynamic-link library (DLL)**.

dynamic window

A window, or virtual terminal, that Telserv creates when it receives a connection request.

dynamic-link library (DLL)

A collection of procedures whose code and data can be loaded and executed at any virtual memory address, with run-time resolution of links to and from the main program and other independent libraries. The same DLL can be used by more than one process. Each process gets its own copy of DLL static data. Contrast with **shared run-time library (SRL)**.

See also **position-independent code (PIC)**.

EDIT file

An unstructured file with the file code 101 in the Guardian file system. An EDIT file can be processed by either the EDIT or PS Text Edit (TEDIT) editor. An EDIT file typically contains source program or script code, documentation, or program output. OSS functions can open an EDIT file only for reading. Guardian functions can create, read and write EDIT files.

See also **unstructured file**.

ELF

See **executable and linkable format (ELF)**.

empty directory

A directory that contains only an entry for itself and an entry for the directory directly above it in the hierarchy.

EMS

See **Event Management Service (EMS)**.

enoft utility

A utility that reads and displays information from TNS/E native object files..

See also **noft utility**.

errno

An external variable that contains the most recent error condition set by a C function.

event

A significant change in some condition in the system or network. An event can be an operational error, notification of limits exceeded, request for action, notification of lines and other objects becoming available, and so on. Note that an event is not always an error.

Event Management Service (EMS)

A part of the Distributed Systems Management (DSM) product used to provide event collection, event logging, and event distribution facilities. It provides for different event descriptions for interactive and programmatic interfaces, lets an operator or application select specific event-message data, and allows for flexible distribution of event messages within a system or network.

executable and linkable format (ELF)

The common standard file format for an object file used for native code on NonStop systems.

feature test macro

A symbol defined by the compiler to indicate the presence of some feature in the compilation.

FIFO

A type of special file that is always read and written in a first-in, first-out manner.

FIFO special file

See **FIFO**.

file

An object to which data can be written or from which data can be read. A file has attributes such as access permissions and a file type. In the Open System Services (OSS) environment, file types include regular file, character special file, block special file, FIFO, and directory. In the Guardian environment, file types include disk files, processes, and subdevices.

file descriptor

The nonnegative integer that uniquely identifies a single open of a file to a running process. Each file descriptor is associated with an open file description that contains data about the file.

file identifier

In the Guardian environment, the portion of a filename following the subvolume name. In the OSS environment, a portion of the internal information used to identify a file in the OSS file system. The two identifiers are not necessarily equivalent. However, the same character sequence can be used in both contexts if it satisfies the following conditions: it contains up to eight characters, all of which are lower-case letters or decimal digits, and the first character is a letter.

file name

A string of characters that uniquely identifies a file.

In the PC environment, file names for disk files normally have at least two parts (the disk name and the file name); for example, B:MYFILE.

In the Guardian environment, disk file names include an Expand node name, volume name, subvolume name, and file identifier; for example, \NODE.\$DISK.SUBVOL.MYFILE.

In the OSS environment, a file is identified by a **pathname**; for example, /usr/john/workfile.

See also **filename**.

file permission bits

Information about a file that is used, along with other information, to determine whether a process or user has read, write, or execute/search permission to that file. The bits are divided into three parts: owner, group, and other. Each part is used with the corresponding file class of processes.

file privileges

Additional privileges that can allow programs special access to files in restricted-access filesets

See also **restricted-access fileset**.

file system

In the Open System Services (OSS) environment, a collection of files and file attributes. A file system provides the namespace for the file serial numbers that uniquely identify its files. Open System Services provides a file system (see also ISO/IEC IS 9945-1:1990 [ANSI/IEEE Std. 1003.1-1990], Clause 2.2.2.38); the Guardian application program interface (API) provides a file system; and OSS Network File System (NFS) provides a file system. (OSS NFS filenames and pathnames are governed by slightly different rules than OSS filenames and pathnames.) Within the OSS and OSS NFS file systems, filesets exist as manageable objects.

On a NonStop system, the Guardian file system for an Expand node is a subset of the OSS virtual file system. Traditionally, the API for file access in the Guardian environment is referred to as the Guardian file system.

In some UNIX and NFS implementations, the term file system means the same thing as fileset. That is, a file system is a logical grouping of files that, except for the root of the file system, can be contained only by directories within the file system.

See also **fileset**.

file transfer protocol (FTP)

1. The Internet-standard, high-level protocol for transferring files from one machine to another.
2. The application used to send complete files over Transmission Control Protocol/Internet Protocol (TCP/IP) services

filename

In the OSS environment, a component of a **pathname** containing any valid characters other than slash (/) or null.

See also **file name**:

fileset

In the OSS environment, a set of files with a common mount point within the file hierarchy. A fileset can be part or all of a single virtual file system.

On a NonStop system, the Guardian file system for an Expand node has a mount point and is a subset of the OSS virtual file system. The entire Guardian file system therefore could be viewed as a single fileset. However, each volume and each process of subtype 30 within the Guardian file system is actually a separate fileset.

The term file system is often used interchangeably with fileset in UNIX publications.

FIPS

A Federal Information Processing Standard of the United States government.

FIPS 151-1

The Federal Information Processing Standard that specifies the requirements for conformance to an older draft of POSIX.1 than the version adopted as ISO/IEC IS 9945-1:1990 (FIPS 151-1 describes conformance to IEEE Std. 1003.1-1988) and imposes some additional requirements.

FIPS 151-2

The Federal Information Processing Standard that specifies the requirements for conformance to POSIX.1 as ISO/IEC IS 9945-1:1990 and imposes some additional requirements.

flat segment

A segment with its own distinct address range that never overlaps the range of any other allocated segment within the process address space. Thus all allocated flat segments for a process are always available for use concurrently. All data and code segments associated with native loadfiles are flat, as are segments created or shared by `shmget()`. `SEGMENT_ALLOCATE_` optionally creates flat segments; `SEGMENT_ALLOCATE64_` does so by default. Contrast with **selectable segment**.

See also **segment**.

foreground process group

A process group whose members have privileges for access to their controlling terminal that are denied to processes in background process groups of that terminal. Each session with a controlling terminal has only one foreground process group for that terminal. Contrast with **background process group**.

FTP

See **file transfer protocol (FTP)**.

function

1. A programming abstraction collecting one or more statements into a named entity that can be invoked, typically accepting parameters and returning a value (thus analogous to mathematical functions).
2. In C and C++ (among other programming languages), the term applied to such entities.

See also **procedure**.

group ID

The nonnegative integer used to identify a group of users of a NonStop network node. Each user of that node is a member of at least one group. When the identity of a group is associated with a process, a group ID value is referred to as one of the following identifiers:

- Real group ID
- Effective group ID
- Supplementary group ID
- Saved-set group ID

Guardian

An environment available for interactive or programmatic use with the NonStop operating system. Processes that run in the Guardian environment use the Guardian system procedure calls as their application program interface; interactive users of the Guardian environment use the Tandem Advanced Command Language (TACL) or another Hewlett Packard Enterprise product's command interpreter.

Contrast with **Open System Services (OSS)**.

Guardian environment

The Guardian application program interface (API), tools, and utilities.

Guardian module

A module compiled to execute in the Guardian environment.

Guardian services

An application program interface (API) to the NonStop operating system, plus the tools and utilities associated with that API. This term is synonymous with Guardian environment.

See also **Guardian**.

hard link

The relationship between two directory entries for the same file. A hard link acts as an additional pointer to a file. A hard link cannot be used to point to a file in another fileset. Contrast with **symbolic link**.

header

1. Generically, a collection of information, typically at or before the beginning of a structure, aggregate, message or other object, containing information about that object.
2. A header file.

header file

A source file included by reference in other source files, typically to define an interface or set of facilities.

HPE NonStop Open System Services (OSS)

The product name for the OSS environment.

See also **Open System Services (OSS)**.

HPE NonStop operating system

The operating system for NonStop systems.

IEC

International Electrotechnical Committee. IEC is a professional organization that creates or adopts standards for computer hardware, environments, and physical interconnections.

IEEE

Institute of Electrical and Electronics Engineers. IEEE is a professional organization whose committees develop and propose computer standards that define the physical and data link protocols of entities such as communication networks.

ILP32

The C/C++ data model where `int`, `long`, and pointer data types are all 32 bits wide; also used as the C/C++ 32-bit compiler pragma.

See also **LP64**.

Intel® Itanium® instructions

Register-oriented Itanium instructions that are native to and directly executed by a TNS/E system. Itanium instructions do not execute on TNS and TNS/R systems. Contrast with **TNS instructions** and **RISC instructions**.

Intel® x86 instructions

Register-oriented x86 instructions that are native to and directly executed by a TNS/X system. x86 instructions do not execute on TNS, TNS/R, and TNS/E systems. Contrast with **TNS instructions**, **Intel® Itanium® instructions**, and **RISC instructions**.

internationalization.

The process of designing and coding software so that it can be adapted to meet the needs of different languages, cultures, and character sets, with the ability to handle various linguistic and cultural conventions. Internationalization methods enable the processing of character-based data independently of the underlying character encoding, allowing choice among character sets. Sometimes referred to as I18N, derived from the 18 letters between the initial I and the final N of the word internationalization. See also **character set**.

interoperability

The ability of programs and functions in one environment to operate on data, files, and other objects from another environment. For example, OSS processes can perform many operations on Guardian files, and Guardian processes can perform some operations on OSS files.

ISO

International Organization for Standardization. ISO is an international body that drafts, discusses, proposes, and specifies standards for network protocols. ISO is best known for its seven-layer reference model that describes the conceptual organization of protocols.

ISO is sometimes called the “International Standards Organization”; although ISO is the official abbreviation, it does not correspond to the organization’s name in any language.

ISO/IEC-conforming POSIX.1 application

An application that both:

- Uses only the facilities described in ISO/IEC IS 9945-1:1990 and approved conforming language bindings for any ISO or IEC standard.
- Is documented as using only the facilities described in ISO/IEC IS 9945-1:1990 and approved conforming language bindings for any ISO or IEC standard.

Itanium instructions

See **Intel® Itanium® instructions**.

job control

The OSS features that allow processes to be stopped, continued, and moved from or to the background.

labeled tape

A record at the front of a tape identifying the tape and containing information about its contents.

large file aware

An application or function that can process large OSS files in addition to small OSS files. For example, it must be able to access large files as input and generate large files as output. Contrast with **large file safe**.

large file safe

An application or function that causes no loss of data or corruption of data when it encounters a large OSS file. A large file safe application or function is not required to process large OSS files with the same ability as when it processes small files, but it must handle errors or warnings detected during file manipulation operations and fail gracefully. Contrast with **large file aware**.

large OSS file

A regular file that has a size greater than or equal to approximately 2 gigabytes. Contrast with **small OSS file**.

link count

The number of directory entries that refer to a particular file.

link name

The filename associated with a specific file within a directory. The length of a filename, and therefore the length of a link name, depends on the file system.

See also **filename**.

linkfile

The object file that is created by a TNS/R, TNS/E, or TNS/X compiler or a TNS/R, TNS/E, or TNS/X native linker when the linker's `-r` option is used; a linkfile is not executable.

See also **loadfile**.

loadfile

The object file output of a TNS/R native linker (`nld` or `ld`), the TNS/E native linker (`eld`), or the TNS/X native linker (`xld`); a loadfile is executable but not re-linkable. Loadfiles include programs and DLLs.

See also **linkfile**.

locale

In localization, the definition of the subset of a user's environment that depends on language and cultural conventions.

localization

The process of adapting computer interfaces, data, and documentation to the culturally accepted way of presenting information in the culture. Sometimes referred to as “L10N,” derived from the 10 letters between the initial “L” and the final “N” of the word “localization.”

login

The activity by which a user establishes a locally authenticated identity on a NonStop network node. Each login has one login name.

login name

A user name associated with a session.

LP64

The C/C++ data model where the `int` data type is 32 bits wide, but `long` and pointer data types are 64 bits wide; also used as the C/C++ 64-bit compiler pragma.

See also **ILP32**.

man page

The online or hard-copy version of a file that provides reference information for a software facility. Some UNIX documentation uses the term “manpage” or “manual page” instead. The online delivery mechanism used to display the file is usually the shell `man` command.

See also **reference page**.

mixed data-model programming

Programming in an environment in which both 32-bit and 64-bit pointers are used within the same compilation unit.

mount point

A directory that contains a mounted fileset. The mounted fileset can be in a different file system.

mount.

To make a fileset accessible to the users of a node.

native mode

See **TNS/R native mode**, **TNS/E native mode**, or **TNS/X native mode**.

native object code

See **TNS/R native object code**, **TNS/E native object code**, or **TNS/X native object code**.

native object file

See **TNS/R native object file**, **TNS/E native object file**, or **TNS/X native object file**.

native object file tool

See **noft utility**, **enoft utility**, or **xnoft utility**.

native process

See **TNS/R native process**, **TNS/E native process**, or **TNS/X native process**.

native signal

See **TNS/R native signal**, **TNS/E native signal**, or **TNS/X native signal**.

native-compiled Itanium instructions.

See **Intel® Itanium® instructions**.

native-compiled RISC instructions

See **RISC instructions**.

nesting

The inclusion of items of one type within items of that type. For example, in the OSS environment, directories can contain directories.

noft utility

A utility that reads and displays information from TNS/R native object files.

non-regular file

An OSS file that is not a regular (disk) file.

noncanonical input mode

A terminal input mode in which data is made available to a process when a timer expires or when a certain number of characters have been entered. Noncanonical data is not grouped into logical lines of input. This mode is sometimes called block mode or transparent mode. Contrast with **canonical input mode**.

NonStop operating system.

See **HPE NonStop operating system**.

nowait I/O

The ability of an application process to continue executing in parallel with read or write operations that it initiates. Contrast with **waited I/O**.

Object Code Accelerator (OCA)

A program optimization tool that processes a TNS object file and produces an accelerated file for a TNS/E system. OCA augments a TNS object file with equivalent sequences of Intel Itanium instructions. TNS object code that is accelerated runs faster on TNS/E systems than TNS object code that is not accelerated. Contrast with **Accelerator**.

Object Code Accelerator for TNS/X (OCAX)

A program optimization tool that processes a TNS object file and produces an accelerated file for a TNS/X system. OCA augments a TNS object file with equivalent sequences of Intel x86 instructions. TNS object code that is accelerated runs faster on TNS/X systems than TNS object code that is not accelerated.

object file

A file containing compiled machine instructions for one or more routines. This file can be an executable loadfile for a program or library or a not-yet-executable linkfile for some program module.

OCA

See **Object Code Accelerator (OCA)**.

odd-unstructured files

Files that have no built-in record structures and permit reading and writing of both even and odd byte counts and positioning to both even and odd byte addresses.

open file

A file with a file descriptor.

open file description

A data structure within a NonStop Expand network node that contains information about the access of a process or of a group of processes to a file. An open file description records such attributes as the file offset, file status, and file access modes. An open file description is associated with only one open file but can be associated with one or more file descriptors.

open system

A system with interfaces that conform to international computing standards and therefore appear the same regardless of the system's manufacturer. For example, the OSS environment on a NonStop systems conforms to international standards such as ISO/IEC IS 9945-1:1990 (ANSI/IEEE Std. 1003.1-1990, also known as POSIX.1),

national standards such as FIPS 151-2, and portions of industry specifications such as the X/Open Portability Guide Version 4 (XPG4).

Open System Services (OSS)

An open system environment available for interactive or programmatic use with the NonStop operating system. Processes that run in the OSS environment use the OSS application program interface; interactive users of the OSS environment use the OSS shell for their command interpreter. Synonymous with Open System Services (OSS) environment. Contrast with **Guardian**.

Open System Services (OSS) environment

The NonStop Open System Services (OSS) application program interface (API), tools, and utilities.

Open System Services (OSS) Monitor

A Guardian utility that accepts requests through an interactive interface named the Subsystem Control Facility (SCF).

Open System Services (OSS) signal

A signal model defined in the POSIX.1 specification and available to OSS processes. OSS signals can be sent between processes.

See also: **signal**.

operator message

A message, intended for an operator, that describes a significant event on a NonStop server.

OSS

See **Open System Services (OSS)**.

OSS environment

See **Open System Services (OSS) environment**.

OSS module

A module compiled to execute in the OSS environment.

OSS Monitor

See **Open System Services (OSS) Monitor**.

OSS personality

See **Open System Services (OSS) environment**.

OSS process ID (PID)

The unique identifier that represents a process during the lifetime of the process and during the lifetime of the process group of that process.

See also **PID**.

OSS SEEP-protected

Additional file-access authorization provided by an OSS Security Event-Exit Process (SEEP). For details of OSS SEEP, see [Accessing OSS SEEP-Protected Files](#).

OSS signal

See **Open System Services (OSS) signal**.

parent process

The process that created a given process, or (if that process has stopped) a process that has inherited a given process.

See also **child process**.

path prefix

In the OSS environment, a pathname, with an optional final slash (/) character, that refers to a directory.

pathname

In the OSS file system and Network File System (NFS), the string of characters that uniquely identifies a file within its file system. A pathname can be either relative or absolute. See also ISO/IEC IS 9945-1:1990 (ANSI/IEEE Std. 1003.1-1990 or POSIX.1), Clause 2.2.2.57.

pathname component

See **filename**.

pathname resolution

In the OSS environment, the process of associating a single file with a specified pathname.

pathname-variable limits

Limits that can vary within the OSS file hierarchy. That is, the limits on a pathname variable can vary according to the directory in which pathname resolution begins.

persistent process

A process that must always exist. Persistent processes are usually controlled by a monitor process that checks on the status of persistent processes and restarts them, if necessary.

PIC

See **position-independent code (PIC)**.

PID

In the Open System Services (OSS) environment, a synonym for **process ID**. OSS process ID is the preferred term in NonStop system publications.

In the Guardian environment, PID is sometimes used to mean either:

- A Guardian process identifier such as the **process ID**
- The *cpu*, *pin* value that is unique to a process within an Expand node.

See also **OSS process ID (PID)**.

pipe

An unnamed FIFO, created programmatically by invoking the `pipe()` function or interactively with the shell pipe syntax character (`|`). A shell pipe redirects the standard output of one process to become the standard input of another process. A programmatic pipe is an interprocess communication mechanism.

PLINSTL

See **Public Library Installation Tool**.

portable application

An application that can execute on a wide range of hardware systems from multiple manufacturers. A portable application is a program that can be moved with little or no change in its source code from another manufacturer's system to a NonStop system.

position-independent code (PIC)

Executable program or library code that is designed to be loaded and executed at any virtual memory address, without any modification. Addresses that can be modified by the loader do not appear in PIC code, only in data that can be modified by the loader.

See also **dynamic-link library (DLL)**.

POSIX

The Portable Operating System Interface, as defined by the Institute of Electrical and Electronics Engineers (IEEE) and the American National Standards Institute (ANSI). Each POSIX interface is separately defined in a numbered ANSI/IEEE standard or draft standard. The application program interface (API), known as POSIX.1, has become ISO/IEC IS 9945-1: 1990.

procedure

1. A programming abstraction that collects one or more statements into a named entity to be invoked, also often called “routine”, “subroutine”, or “function”.
2. In the TAL and pTAL programming languages, a PROC or SUBPROC.
3. Because of the TAL legacy, a term used for Guardian functions in the system library, which are listed in the *Guardian Procedure Calls Reference Manual*.

See also **function**.

process

An instance of the execution of a program. A process has an address space and other resources necessary to execute the program.

process group

A set of OSS processes that can signal associated processes. Each OSS process in an Expand node is a member of a process group; the process group has a process group ID. A new process becomes a member of the process group of its creator.

process group ID

The unique identifier representing a process group during its lifetime.

process group leader

The process that has the process group ID of its process group as its OSS process ID.

process group lifetime

The period that begins when a process group is created and ends when the lifetime of the last remaining process of the group ends.

process handle

An object that uniquely identifies an OSS or Guardian process within a network of NonStop systems.

process ID

See **OSS process ID (PID)**.

process image file

On a UNIX system, an executable object file. In some Guardian documentation, an executable object file is referred to as a “program file.” See **object file**.

process lifetime

The period that begins when a process is created and ends when its OSS process ID is returned to the system for reuse.

process snapshot

The contents of a saveabend file or process snapshot file.

process snapshot file.

1. A file containing dump information needed by a debugging tool. In UNIX systems, such files are usually called core files or core dump files.
2. A file containing the state of a running process at a specific moment, regardless of whether the process terminated abnormally

A snapshot file can be created by the Native Inspect SAVE command and the Visual Inspect CREATE SNAPSHOT command.

See also **save file** and **saveabend file**.

program

A **loadfile** that contains a main entry point.

Public Library Installation Tool

A utility that prepares a set of public DLLs for use and that creates the public DLL registry file (ZREG).

reentrant function

A function that can safely be called while an instance of the same function is running in the same process environment. Such functions must either avoid static or global data, or appropriately synchronize their use.

reference page

The online or hard-copy version of a file that provides reference information for a software facility. Some UNIX documentation uses the term man page instead, referring either to the online delivery mechanism used to display the file (usually the shell `man` command) or to the nature of the file as part of a manual.

regular file

In the OSS file system, a file that is a randomly accessible sequence of bytes. A regular file contains binary or text data and has no structure imposed by the system. Contrast with **special file**.

relative pathname

A pathname that does not begin with a slash (/) character. A relative pathname is resolved beginning with the current working directory. Contrast with **absolute pathname**.

restricted-access fileset

A fileset that has a RESTRICTEDACCESS attribute value of ENABLED or LOCAL. A restricted-access fileset requires the super ID to adhere to the same file permissions and owner privileges as any other user ID.

See also **fileset**.

RISC instructions

Register-oriented 32-bit machine instructions that are directly executed on TNS/R processors. Reduced instruction-set computing (RISC) instructions execute only on TNS/R systems, not on TNS, TNS/E, or TNS/X systems.

Accelerator-generated RISC instructions are produced by accelerating TNS object code using the Accelerator program (AXCEL). Native-compiled RISC instructions are produced by compiling source code with a TNS/R compiler.

root

See **root fileset** and **root directory**. See also **super ID**.

root directory

A directory associated with a process that the system uses for pathname resolution when a pathname begins with a slash (/) character.

root fileset

The fileset with the device number of 0, normally containing the root directory for the OSS file system in a NonStop system. Hewlett Packard Enterprise recommends that this fileset be named “root”.

save file

A file created by the Inspect subsystem in response to a command from a debugger. A save file contains enough information about a running process at a given time to restart the process at the same point in its execution. A save file contains an image of the process, data for the process, and the status of the process at the time the save file was created.

A save file can be created through an Inspect SAVE command at any time. A save file called a saveabend file can be created when a process's SAVEABEND attribute is set and the process terminates abnormally. Other debuggers can create a save file but refer to the result as a process snapshot file.

See also **process snapshot file**.

saveabend file

A file containing dump information needed by the system debugging tool. In UNIX systems, such files are usually called core files or core dump files. A saveabend file is a special case of a save file. See also **save file** and **process snapshot file**.

segment

An aggregate of consecutive virtual addresses that are identified and managed as an entity, with consistent access and other properties. A data segment contains data, as opposed to instructions. A code segment contains instructions and some constant data.

See also **selectable segment**, **flat segment**, **unitary segment (unitSeg)**.

selectable segment

A data segment that starts at address 0x80000 and can be up to 127.5 MB in size; thus all selectable segments occupy the same portion of 32-bit address space. A process can have an arbitrary number of selectable segments, but select at most one at a time to be visible in that reserved address range. Selectable segments are a legacy TNS mechanism, still supported on NonStop systems. SEGMENT_ALLOCATE_ creates selectable segments by default; SEGMENT_ALLOCATE64_ does so optionally. The selected segment can be dynamically switched by calls to the SEGMENT_USE_ procedure. The effect is similar to a rapid overlaying of one large data area. Contrast with **flat segment**.

See also **segment**.

semaphore

A mechanism used to serialize the operations of multiple processes or threads. For example, a semaphore can coordinate access to a shared data object.

server application

An application that provides a service to a client application. Execution of remote procedure calls is an example of such a service.

session

A set of process groups associated for job control purposes. A session can have a controlling terminal.

session leader

The process that created a session.

session lifetime

The period that begins when a session is created and ends when the lifetime of the last remaining process group of the session ends.

shared memory

An interprocess communication mechanism that allows two or more processes to share a given segment or other region of memory. Selectable segments cannot be shared with flat segments, or vice versa. Segments created with `SEGMENT_ALLOCATE_` or `SEGMENT_ALLOCATE64_` cannot be shared with segments created by `shmget()`, or vice versa.

shared run-time library (SRL)

A collection of procedures whose code and data can be loaded and executed only at a specific assigned virtual memory address (the same address in all processes). SRLs use direct addressing and do not have run-time resolution of links to and from the main program and other independent libraries. Contrast with **dynamic-link library (DLL)**.

shell

1. A program that interprets sequences of text input as commands. A shell can operate on an input stream or it can interactively prompt and read commands from a terminal.
2. An envelope of code that provides a convenient way to call a procedure in another execution environment.

signal

The method by which an environment notifies a process of an event. Signals are used to notify a process when an error or some other occurrence of interest has occurred. On NonStop systems, signals are characterized as deferrable or nondeferrable. Nondeferrable signals are those generated by the system when a process performs some action that prevents continued execution of the current instruction stream, such as an attempt to access an unusable address. OSS processes can send deferrable signals to other OSS processes; guardian processes cannot send signals to other processes. A process can block some deferrable signals but not others. A signal is nondeferrable by the nature of its generation, not its identity; thus `kill(pid, SIGSEGV)` sends a deferrable signal to the target `pid`. Signals are used in all native and all OSS processes, but not in Guardian TNS processes (which use traps instead).

signal delivery

The action taken in or upon the target process because of the signal. Delivery may entail invoking a designated signal handler function, entering DEBUG, stopping (suspending) or terminating the process. Deferrable signals are not delivered while the process is executing in kernel (privileged) code.

signal generation

The process of initiating a signal. Signals can be generated by a process invoking the `raise()` or `kill()` function, or by the system, as in the case of a process attempting some impossible action.

signal handler

A function that is executed when a specific signal is delivered to a specific process.

signal mask

The set of signals that are currently blocked from delivery to a specific process.

small OSS file

A regular file that is smaller than approximately 2 gigabytes. Contrast with **large OSS file**.

snapshot file

See **process snapshot file**.

socket

An end-point for stream-oriented communication. A socket has a file descriptor.

special character

A character entered from a terminal that has an effect other than being part of the input stream from that terminal.

special file

A file associated with an I/O device. Often called a device file. Special files are read and written the same as ordinary files, but requests to read or write result in activation of the associated device. Due to convention and consistency, these files should always reside in the /dev directory. See also **file**.

static server

In the Guardian environment, a process that runs continuously and provides a specific service to other processes. A static server differs from a traditional UNIX demon in that a demon actively looks for tasks to perform, while a static server performs only tasks brought to its attention by a client (requester) process.

See also **demon**.

static window

A window, or virtual terminal, that is opened with the Subsystem Control Facility (SCF). You do not need to establish a Telserv connection before opening a static window.

storage pool

A set of physical disk volumes administered as a set of logical disk volumes. A logical disk volume can span multiple physical disk volumes. When a logical disk volume becomes full, more physical disk volumes can be added.

strictly conforming POSIX.1 application

An application that requires only the facilities described in ISO/IEC IS 9945-1: 1990 and the applicable computer language standards. Such an application must accept any behavior or value described in ISO/IEC IS 9945-1: 1990 as unspecified or implementation-defined and, for symbolic constants, accept any value permitted by ISO/IEC IS 9945-1: 1990.

super ID

On NonStop systems, a privileged user who can read, write, execute, and purge all files on the system. The super ID is usually a member of a system-supervisor group.

The super ID has the set of special permissions called appropriate privileges. In the Guardian environment, the structured view of the super ID, which is (255, 255), is most commonly used; in the OSS environment, the scalar view of the super ID, which is 65535, is most commonly used.

SVID

The *System V Interface Definition* for UNIX, published by ATT.

SVR4

System V Release 4, a specific implementation of UNIX.

See also **System V**.

symbolic link

A type of special file that acts as a name pointer to another file. A symbolic link contains a pathname and can be used to point to a file in another fileset. Symbolic links are not included in ISO/IEC IS 9945-1: 1990. Compare to hard link.

system

1. A single copy of the NonStop operating system, also called a node.
2. The entire set of computational resources involved in hosting and running a process.

system library

The set of functions available to all processes in the system. There are separate libraries of TNS functions and native functions. On TNS/E and TNS/X systems, the native library is a set of implicit DLLs.

system process

A process running as part of or on behalf of the NonStop operating system. A system process does not have an OSS process ID.

System V

A version of UNIX developed and marketed originally by AT&T.

terminal

A type of character special file that conforms to the interface description in Clause 7 of ISO/IEC IS 9945-1: 1990.

TNS

Pertaining to the original family of fault-tolerant NonStop computers: the architecture and instruction set, code compiled into that instruction set, and processes executing that code, as well as to the obsolete processors and systems that implemented that architecture. TNS processors execute the TNS instruction set using microcoded Complex Instruction-Set Computing (CISC) technology. TNS/R and TNS/E systems run TNS programs by emulating the TNS instruction set. Contrast with **TNS/R** and **TNS/E**.

TNS C compiler

The C compiler that generates TNS object files. TNS object files can be processed by the Accelerator or Object Code Accelerator to produce accelerated object files. Compare with **TNS/R C compiler** and **TNS/E C compiler**.

TNS instructions

Stack-oriented, 16-bit machine instructions that are directly executed on TNS systems by hardware and microcode. TNS instructions can be emulated on TNS/E and TNS/R systems by using millicode, an interpreter, and either translation or acceleration. Contrast with **RISC instructions**, **Intel® Itanium® instructions**, and **Intel® x86 instructions**.

TNS object code

The TNS instructions that result from processing program source code with a TNS language compiler. TNS object code executes directly on TNS processors, and by emulation on TNS/R, TNS/E, and TNS/X systems.

TNS object file

An object file created by a TNS compiler or the Binder. A TNS object file contains TNS instructions. TNS object files can be processed by the Accelerator or by the TNS Object Code Accelerator (OCA) to produce to produce accelerated object files. A TNS object file can be run on TNS, TNS/R, TNS/E, and TNS/X systems.

TNS process

A process whose main program object file is a TNS object file, compiled using a TNS compiler. A TNS process executes in interpreted or accelerated mode while within itself, when calling a user library, or when calling into TNS system libraries. A TNS process temporarily executes in native mode when calling into native-compiled parts of the system library. Object files within a TNS process might be accelerated or not, with automatic switching between accelerated and interpreted modes on calls and returns between those parts. Contrast with **TNS/R native process** and **TNS/E native process**.

TNS/E

Refers to fault-tolerant Hewlett Packard Enterprise computers that support the NonStop operating system and are based on the Intel Itanium processor. TNS/E systems run the Itanium instruction set and can run TNS object files by interpretation or after acceleration. TNS/E systems include all NonStop systems that use NSE-x processors. Contrast with **TNS** and **TNS/R**.

TNS/E C compiler

The C compiler that generates TNS/E object files. Contrast with **TNS C compiler** and **TNS/R C compiler**.

TNS/E native mode

The primary execution environment on a TNS/E system, in which native-compiled Intel® Itanium® object code executes, following TNS/E native-mode compiler conventions for data locations, addressing, stack frames, registers, and call linkage.

TNS/E native object code

The Intel® Itanium® instructions that result from processing program source code with a TNS/E compiler. TNS/E native object code executes only on TNS/E systems, not on TNS or TNS/R systems.

TNS/E native object file

An object file created by a TNS/E compiler that contains Itanium instructions and other information needed to construct the code spaces and the initial data for a TNS/E native process.

TNS/E native process

A process initiated by executing a TNS/E native object file. Contrast with **TNS process** and **TNS/R native process**.

TNS/E native signal

A signal model available to TNS/E native processes in the Guardian and OSS environments. TNS/E native signals are used for error exception handling.

TNS/E native user library

A user library available to TNS/E native processes in the Guardian and OSS environments. A TNS/E native user library is implemented as a TNS/E native dynamic-link library (DLL).

TNS/R

Refers to fault-tolerant Hewlett Packard Enterprise computers that support the NonStop operating system and are based on 32-bit reduced instruction-set computing (RISC) technology. TNS/R systems run the MIPS-1 RISC instruction set and can run TNS object files by interpretation or after acceleration. TNS/R systems include all Hewlett Packard Enterprise systems that use NSR-x processors. Contrast with **TNS** and **TNS/E**.

TNS/R C compiler

The C compiler that generates TNS/R object files. Compare with **TNS C compiler** and **TNS/E C compiler**.

TNS/R native mode

The primary execution environment on a TNS/R system, in which native-compiled MIPS object code executes, following TNS/R native-mode compiler conventions for data locations, addressing, stack frames, registers, and call linkage.

See also **TNS/E native mode**.

TNS/R native object code

The RISC instructions that result from processing program source code with a TNS/R compiler. TNS/R native object code executes only on TNS/R systems, not on TNS, TNS/E, or TNS/X systems.

TNS/R native object file

An object file created by a TNS/R compiler that contains RISC instructions and other information needed to construct the code spaces and the initial data for a TNS/R native process.

TNS/R native process

A process initiated by executing a TNS/R native object file. Contrast with **TNS process** and **TNS/E native process**.

TNS/R native shared run-time library (TNS/R native SRL)

A shared run-time library (SRL) available to TNS/R native processes in the Guardian and OSS environments on TNS/R systems. TNS/R native SRLs can be either public or private. A TNS/R native process can use multiple SRLs.

TNS/R native signal

A signal model available to TNS/R native processes in the Guardian and OSS environments. TNS/R native signals are used for error exception handling.

TNS/R native SRL

See **TNS/R native shared run-time library (TNS/R native SRL)**.

TNS/R native user library

A user library available to TNS/R native processes in the Guardian and OSS environments. A TNS/R native user library is implemented as a special private TNS/R native shared run-time library (SRL) for non-PIC programs or as a dynamic-link library (DLL) for PIC programs.

TNS/X

Refers to fault-tolerant Hewlett Packard Enterprise computers that support the NonStop operating system and are based on the Intel x86 processor. TNS/X systems run the x86 instruction set and can run TNS object files by interpretation or after acceleration. TNS/X systems include all NonStop systems that use NSE-x processors. Contrast with **TNS**, **TNS/R**, and **TNS/E**.

TNS/X C compiler

The C compiler that generates TNS/X object files. Contrast with **TNS C compiler**, **TNS/R C compiler**, and **TNS/E C compiler**.

TNS/X native mode

The primary execution environment on a TNS/X system, in which native-compiled Intel® x86 object code executes, following TNS/X native-mode compiler conventions for data locations, addressing, stack frames, registers, and call linkage.

TNS/X native object code

The Intel® x86 instructions that result from processing program source code with a TNS/X compiler. TNS/X native object code executes only on TNS/X systems, not on TNS, TNS/R, or TNS/E systems.

TNS/X native object file

An object file created by a TNS/X compiler that contains x86 instructions and other information needed to construct the code spaces and the initial data for a TNS/X native process.

TNS/X native process

A process initiated by executing a TNS/X native object file. Contrast with **TNS process**, **TNS/R native process**, and **TNS/E native process**.

TNS/X native signal

A signal model available to TNS/X native processes in the Guardian and OSS environments. TNS/X native signals are used for error exception handling.

TNS/X native user library

A user library available to TNS/X native processes in the Guardian and OSS environments. A TNS/X native user library is implemented as a TNS/X native dynamic-link library (DLL).

UID

A nonnegative integer that uniquely identifies a user within a node.

The UID is a scalar number consisting of the group number multiplied by 256 and added to the member number; for example, the UID of the super ID is $(255 \times 256) + 255$, or 65535. The UID is used in the OSS environment for functions normally associated with a UNIX user ID.

See also **user ID**.

unitary segment (unitSeg)

The term “segment” is overloaded in the TNS architecture to describe a unit of virtual address space spanning 128 KB and aligned on a 128-KB boundary. This unit still occurs in NonStop memory management, and is called a unitary segment or unitSeg. The TNS user data segment and the TNS code segments are unitSegs.

unmount

To make a fileset inaccessible to the users of a node.

unqualified pointer

A pointer whose declaration includes neither the `_ptr32` nor the `_ptr64` modifier.

unstructured file

A disk file that has no structure imposed by the Guardian file system. Contrast “structured” files, such as entry-sequenced or key-sequenced files, which are divided into blocks and records and contains other information, such as keys and index blocks to identify or find those records. An unstructured file is essentially a large byte array, often used as a code file or an EDIT file. (Both object and EDIT files have definite and complex structure, but not imposed by the file system.) An unstructured file in the Guardian file system appears as a regular file to OSS.

user database

A database within an NonStop network node that contains the user name, user ID, group ID, initial working directory, and initial user program for each user of the node.

user ID

The unique identification of a user within a node.

In the Guardian environment, the term “user ID” usually means the group number and member number pair; for example, the user ID of the super ID is usually described as (255, 255). In the OSS environment, the term “user ID” usually refers to the single number that is the scalar view of the user ID—a number called the UID. See also **UID**.

user name

A string that uniquely identifies a user within the user database for a node.

volume

A logical disk. In NonStop systems, volumes have names that begin with a dollar sign (\$), such as \$DATA. A volume name can refer to a physical magnetic disk drive or mirrored pair of drives, or to a virtual volume managed by the Storage Management Foundation (SMF). The volume name is also the name of the process that manages the volume.

waited I/O

I/O operations for which the initiating process does not resume execution until the I/O operations are completed. Contrast with **nowait I/O**.

working directory

In the Open System Services (OSS) environment, a directory, associated with a process, that is used in pathname resolution for relative pathnames.

x86 instructions

See **Intel® x86 instructions**.

xnoft utility

A utility that reads and displays information from TNS/X native object files.

See also **enoft utility**.